BİLGİSAYAR MÜHENDİSLİĞİ · 1. SINIF

```cpp
#include <iostream>

int main() {
    std::cout << "Merhaba, C++!\n";
    return 0;
}
```

C++ ile Programlamaya Giriş

Konu anlatımı, sınav tuzakları ve çözümlü soru bankası

17	208	2	2
bölüm	soru	mini proje	deneme sınavı

HAZIRLAYAN

Ahmet Şerif Kart · BARÜ – Bartın Üniversitesi

C++17 · VS Code · g++ · 2026

İçindekiler

Bu Kitap Nasıl Kullanılır? . . . 8

1 Programlamaya ve C++'a Giriş . . . 13

1.1 Program ve programlama dili . . . 13
1.2 C++'ın kısa tarihi . . . 14
1.3 Kurulum: g++ ve VS Code . . . 15
1.4 İlk program: Merhaba Dünya . . . 17
1.5 Derleme süreci . . . 18
1.6 Terminalden derleme ve çalıştırma . . . 19
1.7 cout ve kaçış dizileri . . . 21
1.8 Yorum satırları . . . 23
1.9 Hata türleri . . . 25
1.10 main fonksiyonu ve return 0 . . . 29
1.11 Büyük/küçük harf duyarlılığı . . . 30

2 Değişkenler, Veri Tipleri ve Operatörler . . . 38

2.1 Değişken nedir? . . . 38
2.2 Tanımlama, ilk değer verme ve atama . . . 38
2.3 Değişken adları ve anahtar sözcükler . . . 40
2.4 Temel veri tipleri ve sizeof . . . 41
2.5 İlk değer verilmemiş değişken ve çöp değer . . . 42
2.6 Süslü parantez {} ile daralma (narrowing) hatasını önleme . . . 42
2.7 Sabitler ve auto . . . 43
2.8 Aritmetik operatörler . . . 45
2.9 Tip dönüşümü . . . 46
2.10 Bileşik atama, ++ ve -- . . . 49
2.11 Operatör önceliği . . . 50
2.12 Taşma ve ondalıklı sayıların hassasiyeti . . . 51
2.13 <cmath> kütüphanesi . . . 51

3 Klavyeden Girdi ve Biçimli Çıktı . . . 59

3.1 Akış kavramı ve cout . . . 59
3.2 cin ile klavyeden okuma . . . 61
3.3 getline ile satır okumak . . . 64
3.4 Biçimli çıktı: iomanip . . . 66
3.5 Hatalı girdi ve akışın kilitlenmesi . . . 71
3.6 Hata mesajları için cerr . . . 74
3.7 Örnek uygulama: vücut kitle indeksi . . . 75
3.8 Türkçe karakterler ve Windows konsolu . . . 75

4 Koşullu İfadeler . . . 84

4.1 Karar vermek ve ilişkisel operatörler . . . 84
4.2 bool türü: true, false ve boolalpha . . . 86
4.3 C++'ta doğru ve yanlış . . . 87
4.4 if, if-else ve else if . . . 87
4.5 Süslü parantez tuzakları . . . 88
4.6 İç içe if ve sarkan else . . . 89
4.7 Mantıksal operatörler . . . 90
4.8 = ile == karışıklığı . . . 92
4.9 0 < x < 10 tuzağı . . . 93
4.10 İşaretli ve işaretsiz sayıları karşılaştırmak . . . 94
4.11 Metinleri ve karakterleri karşılaştırmak . . . 95
4.12 Operatör önceliği . . . 96
4.13 Üçlü operatör ?: . . . 97
4.14 switch deyimi . . . 98
4.15 double değerlerini karşılaştırmak . . . 102
4.16 Artık yıl kuralı . . . 102

5 Döngüler . . . 109

5.1 Neden döngü? while . . . 109
5.2 do-while . . . 110
5.3 for döngüsü . . . 110
5.4 Sayaç, toplam ve çarpım . . . 112
5.5 break ve continue . . . 113
5.6 Noktalı virgül tuzağı . . . 114
5.7 İç içe döngüler ve desenler . . . 115
5.8 İzleme tablosu ve off-by-one . . . 116
5.9 Klasik algoritmalar . . . 117
5.10 Kullanıcı girdisiyle döngüler . . . 121

6 Fonksiyonlar . . . 130

6.1 Neden fonksiyon? . . . 130
6.2 Fonksiyonun anatomisi . . . 130
6.3 void fonksiyonlar, dönüş değeri ve erken çıkış . . . 131
6.4 Bildirim (prototip) . . . 132
6.5 Parametre ve argüman . . . 134
6.6 Değer ile geçirme . . . 134
6.7 Referans ile geçirme . . . 135
6.8 const & ile geçirme . . . 137
6.9 Varsayılan parametreler . . . 138
6.10 Fonksiyon aşırı yükleme . . . 139
6.11 Çağrı yığını . . . 140
6.12 Kapsam . . . 142
6.13 static yerel değişken . . . 143
6.14 Fonksiyonu return olmadan bitirmek . . . 144
6.15 Hazır fonksiyonlar . . . 144
6.16 Algoritmaları fonksiyona dönüştürme . . . 146
6.17 main de bir fonksiyondur . . . 146

7 Diziler ve vector . . . 155
7.1 Neden dizi? Tanımlama ve bellek . . . 155
7.2 İlk değer verme . . . 156
7.3 İndeks ve sınır dışı erişim . . . 157
7.4 Eleman sayısı: sizeof . . . 157
7.5 Aralık tabanlı for . . . 158
7.6 Diziler ve fonksiyonlar . . . 159
7.7 İki boyutlu diziler . . . 160
7.8 vector: büyüyen dizi . . . 161
7.9 [] ile at() arasındaki fark . . . 162
7.10 vector'ü fonksiyona geçirmek . . . 163
7.11 Kopyalama ve karşılaştırma . . . 164
7.12 İç içe vector: vector<vector<int>> . . . 165
7.13 Temel algoritmalar . . . 166
7.14 Sıralama: kabarcık ve seçmeli . . . 168
7.15 std::array (İleri) . . . 169

8 string ve Karakter İşlemleri . . . 177
8.1 Karakter, sayıdır . . . 177
8.2 cctype: tek karakter üzerinde işlem . . . 178
8.3 std::string: metin için gerçek tip . . . 180
8.4 Arama: find ve string::npos . . . 185
8.5 Parçalama ve değiştirme . . . 186
8.6 string ile döngü . . . 187
8.7 Satır okumak: getline . . . 188
8.8 String ile sayı arasında dönüşüm . . . 189
8.9 Türkçe karakterler ve bayt uzunluğu . . . 190
8.10 C tarzı karakter dizileri . . . 191
8.11 Ters çevirme ve palindrom . . . 193

P1 Proje: Öğrenci Not Sistemi . . . 201

S Deneme Vize Sınavı . . . 217

9 İşaretçiler ve Referanslar . . . 234
9.1 Bellek ve adres . . . 234
9.2 İşaretçi tanımlama ve dolaylama . . . 235
9.3 İşaretçinin tipi . . . 237
9.4 Boş işaretçi: nullptr . . . 237
9.5 Referanslar . . . 238
9.6 İşaretçi ve referansla fonksiyona geçirme . . . 239
9.7 İşaretçi aritmetiği . . . 243
9.8 Diziler ve işaretçiler . . . 245
9.9 const ve işaretçiler . . . 247
9.10 Sarkan işaretçi: yok olmuş değişkeni göstermek . . . 249

10 Dinamik Bellek ve Akıllı İşaretçiler . . . 259
10.1 Neden dinamik bellek? . . . 259
10.2 new ve delete . . . 260
10.3 new[] ve delete[]: dinamik dizi . . . 262
10.4 Bellek hataları . . . 263
10.5 Elle büyüyen dizi: vector içeride ne yapar? . . . 265
10.6 RAII: belleği bir nesneye emanet et . . . 267
10.7 unique_ptr: tek sahipli akıllı işaretçi . . . 268
10.8 shared_ptr: paylaşımlı sahiplik (kısa) . . . 271
10.9 Ne zaman hangisi? . . . 272

11 Sınıflar ve Nesneler . . . 282
11.1 Neden sınıf? struct ile veri gruplama . . . 282
11.2 Sınıf, nesne ve üye fonksiyon . . . 283
11.3 Erişim belirteçleri ve kapsülleme . . . 284
11.4 Yapıcı (constructor) . . . 286
11.5 Yıkıcı (destructor) . . . 288
11.6 this: nesnenin kendisi . . . 290
11.7 const üye fonksiyon . . . 290
11.8 static üyeler . . . 291
11.9 Nesne dizisi ve vector<Sinif> . . . 293
11.10 Nesneyi fonksiyona geçirmek: const & ya da & . . . 294
11.11 Sınıfı .h ve .cpp dosyalarına bölmek . . . 294
11.12 enum class: adlandırılmış seçenekler . . . 296

12 Kalıtım ve Polimorfizm . . . 308
12.1 Kalıtım nedir? . . . 308
12.2 Erişim: public, protected, private . . . 309
12.3 Yapıcı ve yıkıcı sırası . . . 311
12.4 Fonksiyonu yeniden tanımlama . . . 313
12.5 virtual, override ve dinamik bağlama . . . 314
12.6 Çok biçimli koleksiyon: vector<unique_ptr<Sekil>> . . . 316
12.7 Saf sanal fonksiyon ve soyut sınıf . . . 317
12.8 Sanal yıkıcı neden gerekli? . . . 319
12.9 Nesne dilimleme tuzağı . . . 320
12.10 final (kısa) . . . 321
12.11 Çoklu kalıtım (kısa) . . . 322

13 Operatör Aşırı Yükleme ve Şablonlar . . . 331
13.1 Neden operatör aşırı yükleme? . . . 331
13.2 Üye fonksiyon olarak operatörler . . . 332
13.3 << ile cout'a yazdırma . . . 334
13.4 Kopyalama: kopya yapıcı ve atama operatörü . . . 336
13.5 Fonksiyon şablonu . . . 340
13.6 Sınıf şablonu . . . 342
13.7 Şablon hata mesajları . . . 345

P2 Proje: Banka Hesabı Simülasyonu . . . 356

14 İstisna Yönetimi . . . 374
14.1 Hata kodu yaklaşımı ve sorunları . . . 374
14.2 throw, try ve catch . . . 375
14.3 std::exception hiyerarşisi . . . 377
14.4 stoi ve at() istisnaları . . . 378
14.5 Birden çok catch ve sıra tuzağı . . . 379
14.6 catch (...) ve yeniden fırlatma . . . 380
14.7 Kendi istisna sınıfını yazmak . . . 381
14.8 Banka projesine throw eklemek . . . 382
14.9 Yığın çözülmesi ve RAII . . . 384
14.10 noexcept . . . 386
14.11 İstisna mı, hata kodu mu? . . . 387

15 STL: Kapsayıcılar ve Algoritmalar . . . 397
15.1 STL'nin üç parçası . . . 397
15.2 vector: hatırlatma ve yeni üyeler . . . 397
15.3 İteratörler . . . 399
15.4 Algoritmalar: sort, find, count ve diğerleri . . . 401
15.5 Lambda ifadeleri . . . 403
15.6 pair: iki değeri tek pakette tutmak . . . 405
15.7 map: anahtar–değer çiftleri . . . 406
15.8 set: tekrarsız küme . . . 409
15.9 stack ve queue . . . 410
15.10 unordered_map (kısa) . . . 411
15.11 Hangi kapsayıcı ne zaman? . . . 412

16 Dosya İşlemleri . . . 423
16.1 Dosya neden gerekir? . . . 423
16.2 fstream: ofstream, ifstream, fstream . . . 423
16.3 Dosyadan okuma . . . 426
16.4 Döngü koşulunda eof tuzağı . . . 428
16.5 Akış durumu: fail ve clear (İleri) . . . 429
16.6 Dosyaya ekleme: ios::app . . . 431
16.7 CSV dosyaları . . . 431
16.8 cerr ve yönlendirme . . . 433
16.9 Dosya yolu ve çalışma klasörü . . . 433
16.10 RAII: dosya kendiliğinden kapanır . . . 434
16.11 Rastgele erişim: seekg, tellg, seekp, tellp (İleri) . . . 435
16.12 İkili dosyalar: write ve read (İleri) . . . 436
16.13 Metin ve ikili mod, Türkçe karakterli dosyalar . . . 438

17 Özyineleme (Recursion) . . . 447
17.1 Özyineleme nedir . . . 447
17.2 Temel durum ve özyinelemeli durum . . . 448
17.3 Çağrı yığını ve faktoriyel . . . 448
17.4 Sayılarla özyineleme . . . 450
17.5 Fibonacci . . . 452
17.6 vector ve string ile özyineleme . . . 455
17.7 Hanoi kuleleri . . . 457
17.8 Özyinelemeli ikili arama . . . 458
17.9 Yazdırmayı çağrıdan önce mi sonra mı yapmalı . . . 459
17.10 Temel durumu unutmak: yığın taşması . . . 460
17.11 Özyineleme mi, döngü mü? . . . 462
S Deneme Final Sınavı . . . 470
A C++ Cheat Sheet . . . 496
B Terimler Sözlüğü . . . 504
C Sık Karşılaşılan Hata Mesajları . . . 520
D ASCII Tablosu . . . 550

Bu Kitap Nasıl Kullanılır?

Bu kitap, bilgisayar mühendisliğinin ilk yılında C++ dersini alan ve daha önce hiç kod yazmamış öğrenci için yazıldı. Bilgisayarın ne olduğunu ve derleyicinin kabaca ne işe yaradığını biliyorsun; ama değişken, döngü, işaretçi, sınıf gibi kavramlarla henüz tanışmadın. Hedef iki tane: **C++'ı gerçekten öğrenmen** ve **vize/finalde soru kaçırmaman**.

C kitabını okuduysan koşul, döngü ve fonksiyon gibi konular tanıdık gelecek. Okumadıysan da sorun yok: her kavram sıfırdan anlatılır.

Anlatım bilerek sade tutuldu: kısa cümleler, bol örnek, "sınavda ne sorulur?" uyarıları. Kitabı bir roman gibi okumana gerek yok; bilgisayarın başında, kodu yazarak ilerle.

Kitabın yapısı

Kitap, C++ dersinin tipik sırasını takip eder. Önce temel konular gelir, sonra C++'ın öne çıkardığı kısımlar: sınıflar, kalıtım, şablonlar ve standart kütüphane.

Kısım	Bölümler	Ne öğreneceksin
Temeller	1–8	Derleme süreci, değişken ve operatörler, `cout` ve `cin`, koşul, döngü, fonksiyon, `vector` ve `string`
Bellek ve işaretçiler	9–10	İşaretçi (pointer), referans (reference), dinamik bellek (`new` ve `delete`)
Sınıflar	11–13	Sınıf (class), kalıtım, çok biçimlilik (polymorphism), operatör aşırı yükleme, şablon (template)
İstisnalar, kütüphane, dosyalar	14–16	İstisna (exception), STL kapsayıcıları, dosya okuma ve yazma
Özyineleme	17	Özyineleme (recursion) ve temel durum
Projeler ve sınav provaları	8 ve 13'ten sonra; final 17'den sonra	Bölümleri birleştiren uygulamalar, vize ve final denemeleri
Ekler	A–D	A: Cheat sheet, B: Terimler sözlüğü, C: Hata mesajları, D: ASCII tablosu

Her bölüm aynı düzende ilerler: önce **"Bu bölümde öğreneceklerin"** kutusu, sonra konu anlatımı ve örnekler, ardından **Bölüm Özeti**, en sonda **Sorular** ve yeni bir sayfada **Çözümler**. Derleyici ve editör kurulumunu Bölüm 1'de adım adım anlatıyoruz. Önce onu yap, sonra örnekleri yazarak ilerle.

Hangi C++ ve hangi araçlar?

Kitap standart **C++17**'yi anlatır. C++20 ve sonrasındaki yenilikler yalnızca `cpp20` kutularında ve çalıştırılmayan kodlarda görünür. Derleyici olarak **g++**, editör olarak **VS Code** kullanılır. Windows'ta g++, MSYS2 üzerinden kurulan MinGW-w64 ile gelir. macOS'ta Homebrew ile kurulan g++ kullanılabilir.

Yazdığın metin, **kaynak kod** (source code) olarak bir `.cpp` dosyasında durur. Bu metni makine diline çeviren programa **derleyici** (compiler) denir; `g++` budur. Derleyici önce **ön işlemci** (preprocessor) ile `#include` satırlarını işler, sonra kodu çevirir. En son **bağlayıcı** (linker) parçaları birleştirip **çalıştırılabilir dosya** (executable) üretir. C++ programı, Python'daki gibi bir **yorumlayıcı** (interpreter) tarafından doğrudan çalıştırılmaz: önce tamamı derlenmek zorundadır. Bu yüzden birçok hata program daha çalışmadan yakalanır.

Bir programı şu iki komutla derleyip çalıştırırsın (Windows'ta ikinci satır `.\ornek.exe` olur):

```
g++ -std=c++17 -Wall -Wextra -Wpedantic -O0 ornek.cpp -o ornek
./ornek
```

Bu kitaptaki çalıştırılan kodların hepsi bu bayraklarla derlenir. `-std=c++17` standardı seçer; `-Wall -Wextra -Wpedantic` ise derleyicinin uyarı vermesini sağlar. Bayrakların ne işe yaradığını Bölüm 1'de göreceksin.

Kutular

Metinde sekiz tür kutu göreceksin. Her birinin ayrı bir görevi var; aşağıda hepsinden küçük bir örnek var. Kutu türlerinin adı kitabın her yerinde aynıdır.

Bu Bölümde Öğreneceklerin

- C++ programının nasıl derlenip çalıştırıldığını
- Kaynak kod ile çalıştırılabilir dosya arasındaki farkı
- Ekrana yazı yazdırmanın en basit yolunu

İpucu: Önceki komutu geri çağır

Derleme komutunu bir kez yazdıktan sonra terminalde yukarı ok tuşuna basarak geri çağırabilirsin. Derleme hata verdiyse önce hatayı düzelt; hatalı bir kodu çalıştırmak sana yanlış sonuç gösterebilir.

Sık Yapılan Hata: C alışkanlığıyla `printf` yazmak

C'de ekrana yazmak için `printf` kullanırdın. C++'ta bu da derlenir, ama bu kitap ekran ve klavye için `cout` ve `cin` kullanır. Kitabın yöntemine uy ve bir programda tek yöntemde kal.

Sık Yapılan Hata: Örnekleri kopyalayıp geçmek

Kodu kopyala-yapıştır yaparsan program çalışır ama bir şey öğrenmezsin. Sınavda boş bir kâğıda kod yazman istenir. Her örneği elle yaz; derleyici hata verirse mesajı oku.

Sınavda Çıkar: Derleme hatası ile çalışma zamanı hatası

Hocalar bu iki hata türünü sık sorar. **Derleme hatası** (compile-time error) olduğunda program hiç çalışmaz; derleyici `error:` mesajı verir. **Çalışma zamanı hatası** (runtime error) olduğunda ise program başlar, sonra çöker ya da yanlış sonuç üretir. "Bu kod derlenir mi?" sorusunu önce cevapla, sonra "ne yazdırır?" sorusuna geç.

Sınavda Çıkar: Hata satırını bulmak

Derleyici mesajı hatanın yerini söyler: `ornek.cpp:5:12: error: ...` Önce satır numarasına, sonra `error:` ifadesinden sonra gelen açıklamaya bak. "Hatayı bul" tipi sorular bu mesajı okumaya dayanır.

C++20/23 Notu: C++20 ile gelen bir kolaylık

C++20 ile `std::format` geldi. Metni `printf` benzeri biçimde yazar, ama tür hatalarına karşı daha güvenlidir. Bu kitap C++17 kullanır; bu kutu yalnızca bilgi içindir ve çalıştırılmaz.

```
#include <format>
#include <iostream>

int main() {
    int yas{19};
    std::cout << std::format("Yaşın: {}\n", yas);
    return 0;
}
```

Not: Kalın terimler

Kalın yazılan bir terim, o kavramın ilk kez tanıtıldığını gösterir. Terimin İngilizce karşılığı parantez içinde verilir ve kitabın sonundaki terimler sözlüğünde (Ek B) da bulunur.

Bölüm Özeti – Bilmen Gerekenler

- Bir C++ programı önce derlenir, sonra çalıştırılır.
- Derleme hatası program çalışmadan yakalanır; çalışma zamanı hatası program çalışırken ortaya çıkar.
- Uyarılar yok sayılmaz; çoğu zaman gerçek bir hatanın habercisidir.
- Çıktı tahmini sorularında kodu kâğıt üzerinde satır satır izlersin.

Kendin Geliştir

Bir `ornek.cpp` dosyası aç. İçine kendi adını ve yaşını `cout` ile ekrana yazan bir program yaz. Derleyip çalıştır. Sonra yaşını bir yıl ileri sayan bir satır ekle ve yeniden derle.

Kod ve çıktılar

Kitaptaki her program derlendi ve çalıştırıldı. Çıktı kutuları elle yazılmadı; programın gerçek çıktısıdır. Yani bir örneği kendi bilgisayarında çalıştırdığında aynı sonucu görmelisin. Kod kutusunun üstündeki etiket (`ornek.cpp`) dosyanın adıdır.

ornek.cpp

```
#include <iostream>

using namespace std;

int main() {
    cout << "Kod kutusu böyle görünür.\n";
    return 0;
}
```

Çıktı

```
Kod kutusu böyle görünür.
```

Bu programda üç şey var. `#include` satırı ekrana yazmak için gereken kütüphaneyi getirir. `int main()` programın başladığı yerdir. `cout` ise ekrana yazı gönderen bir **akış** (stream) nesnesidir. Şimdilik `#include` ve `using namespace std;` satırlarını kalıp olarak al; Bölüm 1'de ayrıntılarını göreceksin.

Programın klavyeden bir şey okuduğu örneklerde, senin yazdığın kısım çıktı kutusunda **mavi** görünür; terminalde gördüğün gibi. Aşağıdaki `cin` satırını Bölüm 3'te öğreneceksin. Şimdilik sadece çıktının nasıl göründüğüne bak:

selam.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int yas{0};
    cout << "Yaşın: ";
    cin >> yas;
    cout << "Gelecek yıl " << yas + 1 << " yaşında olacaksın.\n";
    return 0;
}
```

Çıktı

```
Yaşın: 19
Gelecek yıl 20 yaşında olacaksın.
```

Çıktı kutularının renkleri de bir şey söyler. Koyu zeminli normal kutu programın çıktısıdır. **Sarı** kutu bir **uyarıdır** (warning): program derlenir ama derleyici "buraya dikkat et" der. Kırmızımsı kutu bir **derleme hatasıdır**: program derlenmez ve kutuda derleyicinin gerçek hata mesajı görünür.

kullanilmayan.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int sayi{5};
    cout << "Merhaba\n";
    return 0;
}
```

Derleyici uyarısı — g++ -Wall -Wextra

```
kullanilmayan.cpp: In function 'int main()':
kullanilmayan.cpp:6:9: warning: unused variable 'sayi' [-Wunused-variable]
    6 |     int sayi{5};
      |         ^~~~
```

Bu program çalışır, ama derleyici `sayi` değişkeninin hiç kullanılmadığını söyler. Uyarıları yok sayma: çoğu zaman gerçek bir hatanın habercisidir.

tanimsizAd.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    cout << yas << '\n';
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
tanimsizAd.cpp: In function 'int main()':
tanimsizAd.cpp:6:13: error: 'yas' was not declared in this scope
    6 |     cout << yas << '\n';
      |             ^~~
```

Bu program derlenmez, çünkü `yas` hiç tanımlanmadı. Hata mesajında önce dosya adı, satır ve sütun numarası gelir; ardından `error:` ve hatanın açıklaması yazılır. Bu mesajları okumayı Bölüm 1'de öğreneceksin.

> **Not: Çıktılar her bilgisayarda aynı mı?**
>
> Çıktılar ondalık **nokta** ile yazılmıştır (`3.75`). Windows konsolunda Türkçe karakterler (`ş`, `ğ`, `ü`) bozuk görünebilir; çözümünü Bölüm 3'te göreceksin. Rastgele sayılar ve bellek adresleri gibi bilgisayardan bilgisayara değişen çıktılarda bunu metinde ayrıca belirtiyoruz.
>
> Kitaptaki çıktılar Apple silikonlu bir Mac üzerinde GCC 16 ile alındı. macOS'ta `g++` komutu çoğu zaman Apple clang'dir ve bazı hata mesajları farklı görünür. Windows'ta MinGW-w64 g++ ile de aynı kod derlenmeli ve aynı çıktıyı vermeli; hata metni birebir aynı olmayabilir. Hata metnindeki satır numarasına ve açıklamaya bakmak yeterlidir.

Soru türleri ve nasıl çözülmeli?

Her bölümün sonunda üç tür soru var:

- **Test soruları (5 soru):** Kavramları ölçer. Her soruda A–E arasında beş şık vardır. Önce cevabı kendin bul, sonra şıklara bak.
- **Çıktı tahmini soruları (3 soru):** "Bu kod ne yazdırır?" Sınavların vazgeçilmezi. Kodu kâğıt üzerinde satır satır izle, değişkenlerin değerlerini bir tabloya yaz. Cevabını bilgisayarda çalıştırmadan önce kâğıda yaz.
- **Kod yazma soruları (2 soru):** Küçük programlar. Önce kendin yaz, derleyip çalıştır, sonra çözümle karşılaştır. Çözümdeki kod tek doğru cevap değildir; senin kodun doğru çıktı veriyorsa doğrudur.

Kitap, sonucu belirsiz kodları (**tanımsız davranış**, undefined behavior) çıktı sorusu olarak sormaz. Bu tür bir kodla karşılaşırsan metin bunu açıkça söyler.

Cevap anahtarı, çözümlerin başında tek bir tabloda toplanır. Sınavda bir sorunun cevabını bulmakla yetinme; **neden öbür şıkların yanlış olduğunu** da çözümden oku.

> **İpucu: Çalışma önerisi**
>
> Bölümü oku → örnekleri **elle yazarak** derleyip çalıştır → soruları çöz → yanlışlarını çözümlerden kontrol et → bir hafta sonra yalnızca kutuları ve özeti tekrar oku. Programlama izleyerek değil, yazarak öğrenilir. Derleyici hata verdiğinde mesajı atlama, oku.

BÖLÜM 1

Programlamaya ve C++'a Giriş

Bu Bölümde Öğreneceklerin

- Programın, makine dilinin ve derleyici ile yorumlayıcı arasındaki farkın ne olduğunu
- C++'ın kısa tarihini, C ile ilişkisini ve standartlarını (C++98'den C++23'e)
- Bir C++ programının çalıştırılabilir dosyaya dönüşme aşamalarını (ön işlemci, derleme, bağlama)
- g++ ve VS Code kurulumunu
- İlk programını yazıp terminalden derleyip çalıştırmayı
- Ekrana metin yazdırmayı ve kaçış dizilerini (`\n` gibi)
- Yorum satırlarını ve iç içe yorum tuzağını
- Derleme, bağlama, çalışma zamanı ve mantık hatası arasındaki farkı; hata mesajını okumayı
- main fonksiyonunun ve `return 0` ifadesinin anlamını

1.1 Program ve programlama dili

Program, bilgisayara ne yapacağını adım adım anlatan komutlar dizisidir. İşlemci (CPU) yalnızca kendi **makine dilini** (machine language), yani 0 ve 1'lerden oluşan komutları anlar. Makine diliyle program yazmak çok zor ve hataya açık olduğu için **yüksek seviyeli programlama dilleri** (high-level language) geliştirilmiştir: C++, Java, Python bunlardan birkaçıdır.

Yüksek seviyeli dilde yazdığımız metne **kaynak kod** (source code) denir. İşlemci kaynak kodu doğrudan anlamaz; önce makine koduna çevrilmesi gerekir. Bunun iki klasik yolu vardır:

	Derleyici (compiler)	Yorumlayıcı (interpreter)
Nasıl çalışır?	Bütün programı **önceden** makine koduna çevirir, sonra çıkan dosya çalıştırılır	Kodu satır satır okur ve **o anda** çalıştırır
Sonuç	Ayrı bir **çalıştırılabilir dosya** (executable; Windows'ta `.exe`)	Ayrı bir dosya üretmez
Hatalar	Sözdizimi hatalarını program çalışmadan önce yakalar	Hataya o satıra gelindiğinde rastlanır
Hız	Genellikle çok hızlıdır	Genellikle daha yavaştır
Örnek diller	C, C++	Python (en basit anlatımla)

C++ bir **derlenen dildir** (compiled language). Derleme bir kez yapılır; çıkan dosya, derleyicisi olmayan bir bilgisayarda bile çalışır. Ama bu dosya, derlendiği işlemci ve işletim sistemine özgüdür: Windows'ta derlediğin `.exe` dosyası Linux'ta çalışmaz. Kaynak kodu ise başka bir platformda yeniden derleyerek çalıştırabilirsin.

Sınavda Çıkar

"Derleyici ile yorumlayıcı arasındaki fark nedir?" klasik bir sorudur. Anahtar cümle: Derleyici programın **tamamını çalıştırmadan önce** çevirir; yorumlayıcı **satır satır, çalışırken** çevirir. "C++ hangisini kullanır?" sorusunun cevabı: derleyici.

1.2 C++'ın kısa tarihi

C++'ı **Bjarne Stroustrup**, 1979'da Bell Laboratuvarları'nda (Bell Labs) geliştirmeye başladı. O sırada C dili, aynı laboratuvarda **Dennis Ritchie** tarafından 1970'lerin başında yazılmış ve UNIX işletim sistemini yazmak için kullanılmıştı. Stroustrup, C'nin hızını ve donanıma yakınlığını korurken büyük programları düzenli yazmayı kolaylaştıracak özellikler eklemek istedi. İlk adı "C with Classes" (sınıflı C) olan dil, 1983'te **C++** adını aldı. Ad bir şakadır: `++` operatörü C'de "bir artır" demektir; yani C++, "C'nin bir adım ilerisi"dir.

C++ yaklaşık kırk yıldır yaşıyor ve hâlâ en çok kullanılan dillerden biri. Çünkü hız ve bellek üzerinde sıkı denetim gereken yerlerde vazgeçilmezdir: oyun motorları, tarayıcılar, işletim sistemlerinin önemli bölümleri, finans ve borsa yazılımları, grafik ve ses programları, gömülü sistemler, robotik. Üstelik C++ öğrenmek, bilgisayarın gerçekte nasıl çalıştığını (bellek, adres, nesne ömrü) anlamanın en iyi yollarından biridir; sonrasında Java, C# ya da Python gibi dilleri öğrenmek de kolaylaşır.

C ile ilişkisi

C++, C'nin üzerine kurulmuştur ve C programlarının büyük bölümünü olduğu gibi derleyebilir; yine de **ayrı bir dildir**. C++ şunları ekler: sınıflar ve nesneler, `string` ve `vector` gibi hazır türler, referanslar, istisnalar, şablonlar ve zengin bir standart kütüphane. Bunların çoğunu bu kitapta ileride ayrıntılı göreceksin. Bu kitap, C'yi önceden bilmeni **gerektirmez**; hiç kod yazmamış biri için baştan yazılmıştır.

Sık Yapılan Hata: C ile C++ aynı şey değildir

Çok yapılan bir yanlış: "C++ öğrenmek için önce C öğrenmeliyim" ya da "C kodu C++ kodudur". İkisi de doğru değil. Modern C++ ile iyi bir C programı çok farklı yazılır. C'den gelen `scanf`, `printf`, `malloc` gibi araçları C++ kitabında kullanmayacağız; yerlerini `cin`, `cout`, `string`, `vector` alır (`cin` Bölüm 3'te, `vector` Bölüm 7'de, `string` Bölüm 8'de anlatılır). Kardeş C kitabında ele alınan konular burada yalnız gerekince anılır.

C++ standartları

C++ de **standartlaştırılmıştır**: dilin kurallarını bir uluslararası komite (ISO) belirler, derleyiciler bu kurallara uyar. Standartlar çıktıkları yıla göre adlandırılır:

Standart	Yıl	Öne çıkanlar
C++98	1998	İlk ISO standardı; sınıflar, şablonlar, istisnalar, standart şablon kütüphanesi (STL)
C++11	2011	Büyük yenilik: `auto`, aralık tabanlı `for`, lambda ifadeleri, akıllı işaretçiler, `nullptr`
C++14	2014	C++11'in rötuşu
C++17	2017	`if` içinde ilk değer, `optional`, `string_view`, dosya sistemi kütüphanesi. **Bu kitabın standardı**
C++20	2020	Kavramlar (concepts), aralıklar (ranges), `<=>` operatörü, `std::format`
C++23	2023	`std::print`, `std::expected` ve başka eklemeler

Standartlar büyük ölçüde geriye dönük uyumludur. Okulların çoğu C++17 ya da daha eskisini bekler; bu yüzden kitap boyunca kodu `-std=c++17` ile derleyeceğiz. Daha yeni standartlara ait özellikleri yalnızca ayrı bir kutuda ve çalıştırmadan göstereceğiz:

> **C++20/23 Notu: `std::print`**
>
> C++23 ile gelen `std::print`, yazdırmayı çok kolaylaştırır. Yeni derleyiciler bunu `-std=c++23` ile destekler; ama okul laboratuvarındaki derleyici eski olabilir, o yüzden kitapta kullanmıyoruz.

```
#include <print>

int main() {
    std::print("Merhaba Dünya!\n");   // C++23
}
```

1.3 Kurulum: g++ ve VS Code

Programı yazmak için bir **editör**, derlemek için bir **derleyici** gerekir. Kitap boyunca derleyici olarak **g++**'ı (GNU Compiler Collection'ın C++ derleyicisi), editör olarak **VS Code**'u kullanıyoruz. Önce g++'ı kur: işletim sistemine uygun olanı yap (1 ya da 2), sonra 3'e geç. Adımlar zamanla küçük değişiklikler gösterebilir; takılırsan kurulum sayfalarındaki güncel yönergelere bak.

1. Windows: MSYS2 ile g++

Windows'ta g++'ın **MinGW-w64** adlı sürümü kullanılır. Kurmanın en kolay yolu **MSYS2**'dir:

1. msys2.org adresinden kurulum dosyasını (`msys2-x86_64-....exe`) indir ve çalıştır. Her şeyi varsayılan bırakıp kur; klasör `C:\msys64` olur.
2. Kurulum bitince Başlat menüsünden **MSYS2 UCRT64** terminalini aç ve aşağıdaki komutu yaz. Sorular çıkarsa Enter ile varsayılanı kabul et, onay istenince `Y` yaz.

```
pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
```

3. g++'ı her terminalden çağırabilmek için klasörünü **PATH**'e ekle. PATH, işletim sisteminin bir komut yazdığında programı hangi klasörlerde arayacağını söyleyen listedir. Başlat menüsünde "ortam değişkenleri" (environment variables) ara ve açılan pencerede **Ortam Değişkenleri / Environment Variables** düğmesine bas. Kullanıcı değişkenlerinden `Path`'i seç, **Düzenle / Edit** → **Yeni / New** ile şu satırı ekle ve pencereleri **Tamam / OK** ile kapat:

```
C:\msys64\ucrt64\bin
```

4. Açık bütün terminalleri kapat; **yeni** bir PowerShell ya da Komut İstemi aç ve şunu yaz:

```
g++ --version
```

İlk satırda `g++` sözcüğünü ve bir sürüm numarasını görmelisin (sürüm numaran farklı olabilir). Görüyorsan kurulum tamamdır.

Sık Yapılan Hata: "g++ tanınmıyor" hatası

`g++ --version` yazınca "'g++' is not recognized..." (Türkçe Windows'ta "...tanınmıyor") benzeri bir mesaj alıyorsan iki ihtimal var: PATH'e eklediğin klasör yanlıştır (`C:\msys64\ucrt64\bin` klasörünün içinde `g++.exe` olduğunu Dosya Gezgini'nde kontrol et) ya da terminali PATH'i ekledikten **sonra** yeniden açmadın. VS Code açıksa onu da kapatıp yeniden aç.

2. macOS ve Linux

macOS: Terminal uygulamasını aç ve şunu yaz; açılan pencerede **Install**'a bas. Bu komut Apple'ın komut satırı araçlarını (derleyici dahil) kurar.

```
xcode-select --install
```

Linux (Debian/Ubuntu): Terminalde şunu yaz. Fedora'da `sudo dnf install gcc-c++` kullanılır.

```
sudo apt install g++
```

Kurulumdan sonra her iki sistemde de `g++ --version` ile kontrol et.

Not: macOS'ta "g++" aslında clang'dir

macOS'ta `g++` yazınca çalışan program GNU g++ değil, Apple'ın **clang** derleyicisidir; `g++ --version` çıktısında "Apple clang" yazar. Kitaptaki programlar ikisinde de çalışır, ama hata ve uyarı **mesajlarının sözcükleri** biraz farklıdır; mesajın düzeni (dosya:satır:sütun) aynıdır. Kitaptaki çıktılar gerçek g++ ile üretildi. Gerçek g++ istersen Homebrew ile `brew install gcc` kurabilirsin; komutun adı sürüm numaralı olur (ör. `g++-16`).

3. VS Code ve C/C++ eklentisi

1. code.visualstudio.com adresinden VS Code'u kur.
2. Sol menüden **Extensions**'a gir (Ctrl+Shift+X; macOS'ta Cmd+Shift+X), **"C/C++"** eklentisini (yayıncı: Microsoft) bul ve **Install**'a bas. Eklenti kod renklendirme, otomatik tamamlama ve hata altı çizme getirir. Derleyici **değildir**; derleyiciyi 1 ya da 2'de kurdun.
3. Bilgisayarında bir klasör oluştur (ör. `CppDersleri`) ve VS Code'da **File → Open Folder** ile aç.
4. Klasörde yeni bir dosya oluştur ve adını `merhaba.cpp` koy. Uzantının `.cpp` olması şarttır.
5. **View → Terminal** menüsüyle alttaki terminal panelini aç (kısayolu Ctrl+` ama Türkçe klavyede bu tuş çoğu zaman çalışmaz; menüyü kullan). Terminal, açık klasörde başlar.

İpucu: Derlemeyi terminalden yap

VS Code'un sağ üstündeki ▷ (Run C/C++ File) düğmesi de çalışır, ama arka planda `-Wall` olmadan derler. Biz derlemeyi terminalde elle yapacağız: komutu kendin yazarsan laboratuvarda ya da sınavda hangi bilgisayar olursa olsun aynı komutu bilirsin. ▷ düğmesini kullanacaksan, açılan `.vscode/tasks.json` dosyasındaki `"args"` listesine `"-std=c++17"`, `"-Wall"` ve `"-Wextra"` ekle.

4. Alternatif: online derleyici

Hiçbir şey kurmadan hızlı bir deneme için onlinegdb.com gibi bir online derleyiciye gir, dil olarak **C++** seç, kodu yaz ve **Run**'a bas. Küçük denemeler için iyidir; ama derleyici ayarları sitenin elindedir. Asıl çalışmanı kendi bilgisayarındaki g++ ile yap. Okulundaki laboratuvarda Dev-C++ ya da Visual Studio kullanılıyorsa, orada da aynı kitap kodları çalışır; yalnızca uyarı ve hata pencereleri farklı görünür.

1.4 İlk program: Merhaba Dünya

Programlama geleneğidir: Her yeni dilde ilk program ekrana bir selam yazar. `merhaba.cpp` dosyasına şunu yaz:

merhaba.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    cout << "Merhaba Dünya!\n";
    return 0;
}
```

Çıktı

```
Merhaba Dünya!
```

Altı satırlık bu program C++'ın neredeyse bütün temel kurallarını içinde barındırır. Satır satır inceleyelim:

Kod	Anlamı
`#include <iostream>`	**Ön işlemci komutu.** `iostream` (input/output stream) **başlık dosyasını** (header file) programa ekler. Ekrana yazı yazan `cout` bu dosyada tanıtılmıştır. `#` ile başlar, `;` ile **bitmez**.
`using namespace std;`	`cout` gibi standart kütüphane adlarını kısaca yazabilmeni sağlar (aşağıdaki nota bak). Bu satır bir **deyimdir**, `;` ile biter.
`int main()`	**main fonksiyonu**: Programın başladığı yer. Program çalışınca işletim sistemi ilk olarak burayı çağırır. `int` sözcüğünün anlamını bu bölümün sonunda göreceksin.
`{ ... }`	Süslü parantezler bir **blok** oluşturur: burada `main`'in gövdesi. Her açılan `{` mutlaka kapanmalıdır.
`cout << "Merhaba Dünya!\n";`	Çift tırnak içindeki metni ekrana yazar. Çift tırnak içindeki metne **string sabiti** (string literal) denir. Sondaki `\n` alt satıra geçirir.
`;`	Her **deyim** (statement) noktalı virgülle biter.
`return 0;`	`main` fonksiyonunu bitirir ve işletim sistemine "program başarıyla tamamlandı" der.

`cout` (okunuşu "si-aut"), ekrana yazmak için kullanılan **standart çıkış akışıdır** (stream). **Akış**, verinin bir kaynaktan hedefe doğru bir boru gibi aktığını düşünmeni sağlayan bir kavramdır; Bölüm 3'te klavyeden okumayla birlikte ayrıntısını göreceksin. `<<` işareti **ekleme operatörüdür** (insertion operator): sağındaki veriyi soldaki akışa gönderir. Oku: "`"Merhaba Dünya!\n"` metnini `cout`'a gönder". İşaretin ağzı verinin aktığı yönü gösterir.

Boş satırlar ve girintiler derleyiciyi ilgilendirmez, insanlar için vardır: Bloğun içindeki satırları **4 boşluk** içeri yaz, kodun okunması kolaylaşır. `main` satırındaki sözcüklerin ayrıntısını Bölüm 6'da fonksiyonlarla birlikte göreceksin; şimdilik bu kalıbı her programda **aynen** yaz.

Not: using namespace std; ne yapar?

C++ standart kütüphanesindeki her şey (`cout`, `string`, `vector` ...) `std` adlı bir **ad alanının** (namespace) içindedir. Ad alanı, aynı adı taşıyan şeylerin birbirine karışmaması için kullanılan bir "soyadıdır". `cout`'un gerçek adı `std::cout`'tur. Şunu da yazabilirdin:

merhabaStd.cpp

```cpp
#include <iostream>

int main() {
    std::cout << "Merhaba Dünya!\n";
    return 0;
}
```

Çıktı

```
Merhaba Dünya!
```

Çıktı aynıdır. `using namespace std;` satırı, "bundan sonra `std::` yazmayı atlayabilirsin" demektir. Bu kitapta kısa tutmak için her programda kullanıyoruz; öğrenirken sorun çıkarmaz. Ama gerçek projelerde **başlık dosyalarında hiç kullanılmaz**, büyük programlarda da genellikle kaçınılır: çünkü kendi yazdığın bir ad ile standart kütüphanedeki ad çakışabilir ve hangisinin kastedildiği karışır. Bu satırı Bölüm 1'de bir kez açıkladık; sonraki bölümlerde tekrar etmeyeceğiz.

Sık Yapılan Hata: Dosya uzantısı

C++ kaynak dosyalarının uzantısı `.cpp`'dir. Windows varsayılan olarak dosya uzantılarını gizler. Dosyayı Not Defteri ile kaydedersen adı `merhaba.cpp.txt` olabilir; g++ böyle bir dosyayı C++ kodu olarak tanımaz. Dosyayı VS Code'da `merhaba.cpp` adıyla oluştur. Uzantıları görmek için Dosya Gezgini'nin **Görünüm** menüsünden (Windows 11'de **Görüntüle** → **Göster**) "Dosya adı uzantıları" kutusunu işaretle.

Sık Yapılan Hata: `#include` satırının sonuna `;` konmaz

`#include` gibi `#` ile başlayan satırlar C++ deyimi değil, **ön işlemci komutlarıdır**; noktalı virgülle bitmezler. `#include <iostream>;` yazarsan `g++ extra tokens at end of '#include' directive` uyarısı verir. Uyarı programı durdurmaz ama hatalı alışkanlıktır. Tersine, `using namespace std;` satırı bir deyimdir ve `;` ile **biter**: unutursan derleme hatası alırsın.

1.5 Derleme süreci

Elinde yalnızca bir metin dosyası (`merhaba.cpp`) varken çalışan bir programa nasıl ulaşırsın? Bu iş tek adımda olmaz; üç ana aşamadan geçer. `g++` komutu bu aşamaları sırayla kendisi çalıştırır:

Aşama	Ne yapar?	Girdi → Çıktı
1. **Ön işlemci** (preprocessor)	`#` ile başlayan satırları işler: `#include <iostream>` satırını `iostream` dosyasının içeriğiyle değiştirir, yorumları siler	`merhaba.cpp` → genişletilmiş kaynak kod
2. **Derleyici** (compiler)	Yazım kurallarını denetler ve kodu makine koduna çevirir; bu sırada önce **assembly** diline (işlemci komutlarının metin hali), sonra **nesne dosyasına** (object file) çevirir	genişletilmiş kaynak → `merhaba.o`
3. **Bağlayıcı** (linker)	Nesne dosyasını, kullandığın hazır kodları taşıyan kütüphanelerle (library) birleştirir	`merhaba.o` + kütüphane → `merhaba.exe` (Linux ve macOS'ta `merhaba`)

Şu noktalara dikkat:

- `iostream` bir **başlık dosyasıdır**. İçinde `cout` gibi şeylerin "tanıtımı" bulunur; derleyici bir adı görünce ne olduğunu bilmek zorundadır.
- `cout`'un **kendi kodu** ise başlık dosyasında değil, C++ **standart kütüphanesinde** (standard library) hazır olarak bulunur. Onu bağlayıcı programına ekler. Yani sen yazmadığın bir kodu kullanabilirsin, ama bağlayıcı o kodu **bulabilmelidir**.
- Günlük dilde "derlemek" çoğu zaman üç aşamayı birden anlatır. Dar anlamda "derleyici" ise yalnızca ikinci aşamadır. Bazı kaynaklar assembly'den makine koduna çeviren adımı ayrı bir aşama (assembler) sayıp dört aşama der; sınavda hocanın derste anlattığı modeli kullan.

Aşamaları istersen tek tek de çalıştırabilirsin. Aşağıdaki komutları kurulumdan sonra deneyebilirsin:

```
g++ -E merhaba.cpp          ← yalnız ön işlemci: genişletilmiş kodu ekrana yazar
g++ -S merhaba.cpp          ← ön işlemci + derleme: merhaba.s (assembly) oluşur
g++ -c merhaba.cpp          ← ön işlemci + derleme: merhaba.o oluşur
g++ merhaba.o -o merhaba    ← bağlayıcı: çalıştırılabilir dosya oluşur
```

`g++ -E` çıktısına bak: Tek satırlık `#include <iostream>`, binlerce satıra dönüşür. Ön işlemcinin yaptığı iş budur.

> **Sınavda Çıkar**
>
> "C++ programının derleme aşamalarını sırayla yaz", "`#include` hangi aşamada işlenir?" ve "Bağlayıcı ne işe yarar?" sık sorulur. Sıra: **ön işlemci → derleyici → bağlayıcı**. `#include` ön işlemcinin, `cout` gibi hazır kodları programa eklemek bağlayıcının işidir.

1.6 Terminalden derleme ve çalıştırma

> **Sık Yapılan Hata: Kaydetmeden derlemek**
>
> VS Code'da otomatik kaydetme (Auto Save) varsayılan olarak **kapalıdır**. Dosyanın sekmesinde adın yanında beyaz bir nokta görüyorsan değişiklikler henüz diske yazılmamıştır ve g++ dosyanın **eski** hâlini derler. Yeni oluşturduğun boş bir dosyayı kaydetmeden derlersen bağlayıcı `main`'i bulamaz: Linux'ta `undefined reference to 'main'`, MinGW'de `undefined reference to 'WinMain'` hatası alırsın. Her derlemeden önce **Ctrl+S** (macOS'ta **Cmd+S**) ile kaydet ya da **File → Auto Save**'i aç.

Terminali `merhaba.cpp` dosyasının bulunduğu klasörde aç (VS Code terminali zaten açık klasörde başlar; başka bir terminalde `cd klasorAdi` ile git, `dir` ya da `ls` ile dosyanı gör). En basit derleme komutu:

```
g++ merhaba.cpp -o merhaba
```

`g++` derleyiciyi çağırır, `merhaba.cpp` kaynak dosyadır, `-o merhaba` (output) çıktı dosyasının adını belirler. Windows'ta dosya `merhaba.exe` olarak oluşur. `-o` yazmazsan Linux ve macOS'ta `a.out`, Windows'ta `a.exe` adı kullanılır.

Çalıştırma komutu işletim sistemine göre değişir:

Terminal	Komut
Linux, macOS	`./merhaba`
Windows PowerShell	`.\merhaba`
Windows Komut İstemi (cmd)	`merhaba`

`./` neden gerekli? Linux, macOS ve PowerShell güvenlik gereği içinde bulunduğun klasörü otomatik aramaz; "şu klasördeki `merhaba`" demek için `./` yazarsın. Bütün oturum şöyle görünür:

```
g++ merhaba.cpp -o merhaba     ← derle: çalıştırılabilir dosya oluşur
./merhaba                      ← çalıştır (PowerShell'de .\merhaba)
Merhaba Dünya!
```

Uyarıları açma alışkanlığı

g++'ı yalnızca `g++ merhaba.cpp -o merhaba` ile çağırırsan çok az uyarı verir. Kitap boyunca derlemeyi şu komutla yapacağız:

```
g++ -std=c++17 -Wall -Wextra merhaba.cpp -o merhaba
```

Seçenek	Anlamı
`-std=c++17`	C++17 kurallarına göre derle. Varsayılan standart derleyici sürümüne göre değişir; açıkça yazarsan herkes aynı kuralları kullanır
`-Wall`	Warnings: all. Şüpheli görünen yaygın durumlar için uyarıları aç
`-Wextra`	`-Wall`'ın kaçırdığı ek uyarıları da aç
`-Wpedantic`	Standartta olmayan derleyici uzantılarını (ör. Bölüm 7'deki değişken uzunluklu dizi) da uyar
`-O0`	Kodu iyileştirme (optimizasyon yapma); varsayılan budur. Program yazdığın sırayla çalışır, hata ayıklaması kolaydır
`-o ad`	Çıktı dosyasının adı

Kitaptaki programlar `-Wpedantic -O0` eklenerek derlendi (Bu Kitap Nasıl Kullanılır? bölümündeki komut). Kendi çalışmanda ilk dört bayrak yeter; `-Wpedantic` eklemek de iyi bir alışkanlıktır.

Uyarı (warning) bir hata değildir; program yine de derlenir. Ama uyarılar çoğu zaman gerçek bir hatanın erken işaretidir. Aşağıdaki program geçerlidir, fakat içinde hiç kullanılmayan bir değişken vardır (`int kullanilmayan = 5;` satırını Bölüm 2'de öğreneceksin; şimdilik "5 değerini tutan bir kutu aç" olarak oku):

kullanilmayan.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int kullanilmayan = 5;
    cout << "Merhaba\n";
    return 0;
}
```

Derleyici uyarısı – g++ -Wall -Wextra

```
kullanilmayan.cpp: In function 'int main()':
kullanilmayan.cpp:6:9: warning: unused variable 'kullanilmayan' [-Wunused-variable]
    6 |     int kullanilmayan = 5;
      |         ^~~~~~~~~~~~
```

Program yine de derlendi ve çalıştı:

Çıktı
```
Merhaba
```

`-Wall` olmadan derleseydin g++ bu programı sessizce derlerdi ve hatayı fark etmezdin. Kural basit: **Uyarısız derle.** Ödevde ve sınavda uyarı veren kod, çoğu zaman puan kaybettirir.

1.7 cout ve kaçış dizileri

`cout`, `<<` ile verilen her şeyi ekrana yazar. Çift tırnak içindeki metni olduğu gibi basar. Bir ayrıntıya dikkat: `cout` kendiliğinden **alt satıra geçmez**.

satirSonuYok.cpp
```cpp
#include <iostream>

using namespace std;

int main() {
    cout << "Merhaba";
    cout << "Dünya";
    cout << "!";
    return 0;
}
```

Çıktı
```
MerhabaDünya!
```

Üç `cout` satırı arka arkaya, boşluksuz yazdı. Bazı terminallerde komut satırı (prompt) da çıktının hemen yanına yapışır. Alt satıra geçmek için metne **yeni satır karakteri** `\n` eklenir:

satirSonuVar.cpp
```cpp
#include <iostream>

using namespace std;

int main() {
    cout << "Merhaba\n";
    cout << "Dünya\n";
    cout << "\n";
    cout << "Bu satırdan önce boş satır var.\n";
    return 0;
}
```

Çıktı
```
Merhaba
Dünya

Bu satırdan önce boş satır var.
```

`\n` iki karakter gibi görünse de C++ için **tek bir karakterdir**. Ters bölü ile başlayan bu özel yazımlara **kaçış dizisi** (escape sequence) denir; yazılamayan ya da özel anlamı olan karakterleri metne sokmak için kullanılır:

Yazım	Anlamı
`\n`	Yeni satır (alt satıra geç)
`\t`	Sekme (tab), genelde birkaç boşluk kadar ileri atlar
`\"`	Çift tırnak karakteri
`\'`	Tek tırnak karakteri
`\\`	Ters bölü karakterinin kendisi
`\0`	Boş karakter (null character); Bölüm 8'de `string`'leri anlatırken geri döneceğiz

Çift tırnağı metnin içinde yazmak istersen, tırnak metni bitirmiş sayılmasın diye başına ters bölü koyarsın. Aynı mantıkla ters bölünün kendisi için de iki tane yazarsın:

kacis.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    cout << "Ad\tSoyad\tNot\n";
    cout << "Ali\tYılmaz\t85\n";
    cout << "Hoca \"çok kolay\" dedi.\n";
    cout << "Dosya: C:\\cpp\\merhaba.cpp\n";
    cout << "Başarı oranı: %95\n";
    cout << "Satır 1\nSatır 2\n";
    return 0;
}
```

Çıktı

```
Ad      Soyad   Not
Ali     Yılmaz  85
Hoca "çok kolay" dedi.
Dosya: C:\cpp\merhaba.cpp
Başarı oranı: %95
Satır 1
Satır 2
```

Bir not: `%` işareti `cout` için **özel değildir**, olduğu gibi yazılır (C'nin `printf`'inde `%%` yazılırdı; `cout`'ta buna gerek yok).

Zincirleme yazdırma

`<<` operatörlerini arka arkaya dizebilirsin. Her `<<`, sağındakini akışa ekler ve akışın kendisini geri verdiği için bir sonraki `<<` devam edebilir. Metin, tek karakter (tek tırnak) ve sayı bir arada yazılabilir:

zincir.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    cout << "Ali" << " " << "Veli" << '\n';
    cout << "Yaş: " << 19 << '\n';
    cout << "Not: " << 85.5 << " (" << 'A' << ")" << '\n';
    return 0;
}
```

Çıktı

```
Ali Veli
Yaş: 19
Not: 85.5 (A)
```

Dikkat edilecek üç şey: (1) **Çift tırnak** metin için (`"Ali"`), **tek tırnak** tek bir karakter için (`'A'`, `'\n'`) kullanılır. (2) Sayılar tırnaksız yazılır; `19` sayıdır, `"19"` metindir. (3) Satır sonu için `"\n"` ya da `'\n'` ikisi de olur; bu kitapta tek karakter olduğu için çoğunlukla `'\n'` yazacağız. Değişkenlerle yazdırmayı Bölüm 2'de, `endl` ve `cout`'un biçimlendirme özelliklerini Bölüm 3'te göreceksin.

Sık Yapılan Hata: Ters bölü ve eğik çizgi

Kaçış dizileri **ters bölü** `\` ile yazılır. `cout << "Merhaba/n";` yazarsan ekrana olduğu gibi `Merhaba/n` basılır ve alt satıra geçilmez. Tanımsız bir kaçış dizisi de yazma: `"C:\ders"` içindeki `\d`, C++'ta tanımlı bir kaçış dizisi değildir; g++ `unknown escape sequence` uyarısı verir ve sonuç derleyiciye bağlıdır. Ters bölünün kendisini yazmak için her zaman `\\` kullan.

Sınavda Çıkar

Çıktı sorularında `\n`, `\t` ve `\\` çok sık kullanılır; en sık tuzaklardan biri de `cout << "Ali\\n"` ile `cout << "Ali\n"` farkıdır. Kural: Ekrandaki "imleç" yalnızca **gerçek** `\n` ile alt satıra iner. Soruyu çözerken imlecin hangi satırda kaldığını adım adım takip et. Son `cout`'ta `\n` yoksa imleç o satırda kalır.

1.8 Yorum satırları

Yorumlar (comments), derleyicinin tamamen görmezden geldiği, kodu okuyan insana yönelik notlardır:

yorumlar.cpp

```cpp
/* Bu program yorum türlerini gösterir.
   Çok satırlı yorum birden fazla satıra yayılabilir. */
#include <iostream>

using namespace std;

int main() {
    // Tek satırlık yorum: satırın sonuna kadar geçerlidir
    cout << "Merhaba\n"; // kodun yanına da yazılabilir

    /* Bu iki satır arasındaki her şey yok sayılır.
    cout << "Bu satır çalışmaz\n"; */
    cout << "Bitti\n";
    return 0;
}
```

Çıktı
```
Merhaba
Bitti
```

Yorum türü	Sözdizimi	Kullanım
Tek satır	`// ...`	Kısa açıklamalar. C++'ta baştan beri vardır
Çok satır	`/* ... */`	Uzun açıklamalar, bir kod parçasını geçici olarak devre dışı bırakma

Tersi de geçerlidir: **string'in içindeki** `//` ya da `/*` yorum başlatmaz. `cout << "a // b\n";` ekrana `a // b` yazar; yorum işareti ancak tırnak dışında yorum işaretidir.

İyi yorum, kodun **ne yaptığını** tekrar etmez, **neden öyle yaptığını** söyler. `// ekrana yazar` gibi yorumlar gürültüdür; `// Not 50'nin altındaysa öğrenci kalır` işe yarar.

İç içe `/* */` tuzağı

C++'ta `/* */` yorumları **iç içe yazılamaz**. Yorum, ilk gördüğü `*/` işaretinde kapanır:

icIceYorum.cpp
```cpp
#include <iostream>

using namespace std;

int main() {
    /* Başlangıç yorumu
       cout << "Bir\n"; /* iç yorum */
       cout << "İki\n";
    */
    cout << "Üç\n";
    return 0;
}
```

Derleyici çıktısı – program derlenmez
```
icIceYorum.cpp:7:25: warning: '/*' within comment [-Wcomment]
    7 |        cout << "Bir\n"; /* iç yorum */
icIceYorum.cpp: In function 'int main()':
icIceYorum.cpp:9:6: error: expected primary-expression before '/' token
    9 |     */
      |      ^
```

Ne oldu? Dıştaki yorum, **içteki** `*/` işaretinde kapandı. Bundan sonra `cout << "İki\n";` normal kod gibi okundu, ardından gelen yalnız `*/` ise derleyiciye anlamsız geldi: `expected primary-expression before '/' token`. g++ önce bir de uyarı verdi: `'/*' within comment`, yani "yorumun içinde yorum başlatmaya mı çalıştın?". Bir kod parçasını geçici olarak yorum yapacaksan ve içinde `/* */` varsa, satırları tek tek `//` ile yorumla.

> **İpucu: Yorum yapma kısayolu**
>
> VS Code'da bir veya birkaç satırı seçip **Ctrl+/** (macOS'ta **Cmd+/**) tuşlarına basarsan satırların başına `//` eklenir; tekrar basarsan kaldırılır. İç içe yorum sorunu yaşamamanın en kolay yolu da budur.

1.9 Hata türleri

Program yazarken dört farklı türde hata ile karşılaşırsın. Hangisi olduğunu anlamak, hatayı çözmenin yarısıdır. Hataları ortaya çıktıkları sırayla ele alıyoruz: derleme, bağlama, çalışma zamanı, mantık.

1. Derleme hatası (compile-time error)

Kod C++'ın **yazım kurallarına** (sözdizimi, syntax) uymuyorsa g++ derlemez ve çalıştırılabilir dosya oluşmaz. En klasik örnek unutulan noktalı virgüldür:

noktaliVirgul.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    cout << "Merhaba\n"
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
noktaliVirgul.cpp: In function 'int main()':
noktaliVirgul.cpp:6:24: error: expected ';' before 'return'
    6 |     cout << "Merhaba\n"
      |                        ^
      |                        ;
    7 |     return 0;
      |     ~~~~~~
```

g++ mesajı **dosya:satır:sütun: tür: açıklama** düzeninde yazar. İlk mesaj satırını parçalara ayıralım:

Parça	Anlamı
Dosya	Hata hangi dosyada (`noktaliVirgul.cpp`)
Satır	Hata kaçıncı satırda
Sütun	O satırın kaçıncı karakterinde
Tür	`error` hata demektir ve derleme durur; `warning` uyarıdır ve derleme sürer
Açıklama	Ne olduğunu anlatan cümle (İngilizce): `return`'den önce `;` bekleniyordu

Altındaki satırlarda g++ sorunlu satırı gösterir, `^` işaretiyle noktayı işaretler ve bazen çözümü önerir (burada `;` eklemeyi). `In function 'int main()':` satırı da hatanın hangi fonksiyonun içinde olduğunu söyler. Bir ayrıntı: Eksik `;` gibi hatalarda derleyici şikâyeti, asıl hatalı satırdan **sonraki** satırda bildirebilir. Mesajdaki satırda bir şey göremezsen bir üst satıra da bak.

Kitaptaki mesajlar g++ 16'dan alınmıştır. Senin g++ sürümün ya da macOS'taki clang sözcükleri biraz farklı yazabilir, ama dosya:satır:sütun düzeni hepsinde aynıdır.

İkinci klasik hata, adın yanlış yazılmasıdır:

cout_yazim.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    cuot << "Merhaba\n";
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
cout_yazim.cpp: In function 'int main()':
cout_yazim.cpp:6:5: error: 'cuot' was not declared in this scope
    6 |     cuot << "Merhaba\n";
      |     ^~~~
```

`was not declared in this scope`, "bu bölgede tanıtılmamış bir ad kullandın" demektir. Bazen g++ benzer bir ad bulup tahmin de yürütür (`did you mean ...?`); burada bulamadı. Yazım hatası, büyük/küçük harf hatası ve eksik `#include` hep aynı mesajı verir; mesajı okuyup hangisi olduğuna kendin karar verirsin.

Üçüncü klasik hata, `#include` satırını unutmaktır:

includeYok.cpp

```cpp
using namespace std;

int main() {
    cout << "Merhaba\n";
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
includeYok.cpp: In function 'int main()':
includeYok.cpp:4:5: error: 'cout' was not declared in this scope
    4 |     cout << "Merhaba\n";
      |     ^~~~
includeYok.cpp:1:1: note: 'std::cout' is defined in header '<iostream>'; this is probably fixable by
adding '#include <iostream>'
  +++ |+#include <iostream>
    1 | using namespace std;
```

Mesajı baştan sona oku: g++ `cout`'u tanımadığını söylüyor, ardından da çözümü veriyor: `std::cout` `<iostream>` başlık dosyasında tanımlıdır, `#include <iostream>` eklenmelidir. Hatta eklenecek satırı `+++ |+#include <iostream>` ile gösteriyor. Hata mesajları çoğu zaman çözümün kendisini taşır; okumak zorundasın.

Dördüncü klasik hata, `using namespace std;` satırını unutup `std::` da yazmamaktır:

stdYok.cpp

```cpp
#include <iostream>

int main() {
    cout << "Merhaba\n";
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
stdYok.cpp: In function 'int main()':
stdYok.cpp:4:5: error: 'cout' was not declared in this scope; did you mean 'std::cout'?
    4 |     cout << "Merhaba\n";
      |     ^~~~
      |     std::cout
In file included from stdYok.cpp:1:
/opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/iostream:65:18: note: 'std::cout' declared here
   65 |   extern ostream cout;          ///< Linked to standard output
      |                  ^~~~
```

g++ burada `std::cout` yazmak gerektiğini açıkça söyler. Bu iki hata birbirine çok benzer; ayırt etmenin yolu, `#include <iostream>` satırının var olup olmadığına bakmaktır.

> **İpucu: İlk hatadan başla**
>
> Bir hata çoğu zaman arkasından başka hata mesajlarını da getirir (örneğin kapanmayan bir `{` yüzünden dosyanın sonuna kadar şikâyet edilir). Mesaj listesi uzun olsa bile **en üstteki ilk hatayı** düzelt ve yeniden derle. Çoğu zaman diğerleri kendiliğinden kaybolur.

2. Bağlama hatası (linker error)

Kodun yazımı doğrudur ve derleyici aşamasını geçer; ama **bağlayıcı** programın ihtiyaç duyduğu bir parçayı bulamaz. En tipik örnek, `main` yerine `Main` yazmaktır:

```cpp
#include <iostream>

using namespace std;

int Main() {              // main değil, Main!
    cout << "Merhaba\n";
    return 0;
}
```

C++ için `Main` sıradan bir fonksiyon adıdır, bu yüzden dosya sorunsuz **derlenir**. Ama bağlayıcı, programın başlayacağı `main` fonksiyonunu bulamaz. Windows'ta (MinGW g++) şuna benzer bir mesaj çıkar (uzun yol bilgileri `...` ile kısaltıldı):

```
.../libmingw32.a(lib64_libmingw32_a-crt0_c.o): in function `main':
.../crt0_c.c:18:(.text.startup+0x2e): undefined reference to `WinMain'
collect2.exe: error: ld returned 1 exit status
```

Linux'ta g++ şunu yazar:

```
/usr/bin/ld: .../Scrt1.o: in function `_start':
(.text+0x1b): undefined reference to `main'
collect2: error: ld returned 1 exit status
```

Windows'taki `WinMain`, Windows'un grafik programlar için kullandığı başlangıç fonksiyonudur; anlam yine aynıdır: "programın başlangıç noktası bulunamadı". macOS'ta ise `Undefined symbols for architecture arm64: "_main"` ile başlayan bir mesaj görürsün. Hepsinin ortak noktası **`undefined reference`** (macOS'ta *undefined symbols*) ifadesidir; Windows ve Linux mesajları `ld returned 1 exit status` satırıyla biter (`ld`, bağlayıcının adıdır). Ayrıca senin kaynak dosyandaki bir satırı gösteren `dosya.cpp:satır:sütun:` konumu yoktur, çünkü bağlayıcı kaynak satırlarına değil nesne dosyalarına bakar.

Bağlama hatasının ikinci tipik sebebi, **tanıtılmış ama kodu hiç yazılmamış** bir fonksiyonu çağırmaktır. Fonksiyonları Bölüm 6'da göreceksin; şimdilik şu kalıbı olduğu gibi al: ilk satır "`topla` adında, iki `int` alan bir fonksiyon var" der (gövdesi yoktur), ama gövdeyi yazan kimse yoktur.

```cpp
#include <iostream>

using namespace std;

int topla(int a, int b);      // sadece tanıtım, gövde hiçbir yerde yok

int main() {
    cout << topla(2, 3) << '\n';
    return 0;
}
```

Derleyici bu kodu geçer, çünkü `topla`'nın var olduğuna güvenir. Bağlayıcı ise gövdeyi bulamaz. g++ (Linux ve MinGW) bunu `undefined reference to 'topla(int, int)'` diye bildirir; macOS'ta ise `Undefined symbols` ile başlayan bir mesaj çıkar. C++'ta fonksiyonun adı derleyici tarafından parametre türleriyle birlikte kodlandığı için mesajda `topla(int, int)` biçiminde görünür.

3. Çalışma zamanı hatası (runtime error)

Kod kurallara uygundur, derlenir ve bağlanır; ama **çalışırken** yapılamayacak bir işlemle karşılaşılır. Tipik örnekler: bir tamsayıyı sıfıra bölmek, geçersiz bir bellek adresine erişmek, olmayan bir dosyayı açmaya çalışmak, yakalanmayan bir istisna (exception, Bölüm 14).

```cpp
int bolen = 0;
cout << 10 / bolen;       // derlenir, ama çalışırken sorun çıkar
```

Bu kod derlenir (`int bolen = 0;` ve `10 / bolen` ifadelerini Bölüm 2'de öğreneceksin). Çalıştırdığında ise sonuç sisteme göre değişir, çünkü C++ tamsayıyı sıfıra bölmeyi **tanımsız davranış** (undefined behavior) sayar: standart sonucu vaat etmez. Program çökebilir, yanlış değer üretebilir ya da sanki çalışıyormuş gibi görünebilir. Windows ve x86 işlemcili Linux'ta program çoğunlukla çöker (Linux'ta `Floating point exception` yazar); Apple M serisi işlemcilerde ise çökmeyebilir. Bu yüzden böyle bir kodu kitapta çalıştırıp çıktısını göstermiyoruz. Çalışma zamanı hatalarının ayrıntılarını konularıyla birlikte ileriki bölümlerde göreceksin.

4. Mantık hatası (logic error)

En sinsi hata budur: Program derlenir, çalışır, çökmez; ama **yanlış sonuç** üretir. Bilgisayar sana bir şey söylemez, hatayı sen bulmak zorundasın.

geriSayim.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    // Amaç: 3, 2, 1, Başla! şeklinde geri sayım yapmak
    cout << "1\n";
    cout << "2\n";
    cout << "3\n";
    cout << "Başla!\n";
    return 0;
}
```

```
Çıktı
1
2
3
Başla!
```

Hiçbir hata mesajı yok, ama geri sayım ters çıktı. Gerçek programlarda mantık hataları çoğunlukla hesaplarda ortaya çıkar; örneğin parantezi unutup işlemleri yanlış sırayla yapmak. Bunları Bölüm 2'de operatör öncelikleriyle göreceksin.

Hata türü	Ne zaman ortaya çıkar?	Program çalışır mı?	Örnek
Derleme hatası	Derleme aşamasında	Hayır, çalıştırılabilir dosya oluşmaz	Eksik `;`, yanlış yazılmış `cout`, unutulan `#include`
Bağlama hatası	Bağlayıcı aşamasında	Hayır, çalıştırılabilir dosya oluşmaz	`main` yerine `Main` yazmak
Çalışma zamanı hatası	Program çalışırken	Başlar; çökebilir ya da yanlış davranabilir	Tamsayıyı sıfıra bölmek
Mantık hatası	Sonuçlara bakınca	Evet, ama yanlış sonuç verir	Yanlış sırayla yazdırmak, unutulan parantez

> **Sınavda Çıkar**
>
> "Aşağıdaki hata hangi türdendir?" sorusu çok sık çıkar. İpucu: Eksik `;`, kapanmayan `}`, yanlış yazılmış `cout`, eksik `#include` → **derleme**. `undefined reference` → **bağlama**. Sıfıra bölme, geçersiz bellek erişimi → **çalışma zamanı**. Yanlış formül, yanlış sıra → **mantık**. Derleme ve bağlama hatalarında program hiç çalışmaz; ikisini ayırt etmenin yolu mesajdır: Mesajda `undefined reference` geçiyorsa bağlama hatasıdır, `dosya.cpp:satır:sütun: error` biçimindeki hatalar derleme hatasıdır.

1.10 main fonksiyonu ve return 0

Her C++ programı `main` fonksiyonundan başlar. Birden fazla `.cpp` dosyası olsa bile programda tam bir tane `main` bulunur. İlk satırı parçalayalım:

Parça	Anlamı
`int`	`main` işini bitirince işletim sistemine bir **tamsayı** (integer) döndürür
`main`	Fonksiyonun adı. Bu ad sabittir; bağlayıcı tam olarak `main`'i arar
`()`	`main` hiçbir **parametre** (dışarıdan değer) almıyor demektir

`return 0;` bu tamsayıyı döndürür. Buna **çıkış kodu** (exit status) denir. Gelenek şudur: `0` "her şey yolunda, program başarıyla bitti", sıfırdan farklı bir sayı "bir sorun oldu" demektir. Aşağıdaki program bilerek `3` döndürüyor:

donus3.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    cout << "Bir sorun oluştu!\n";
    return 3;
}
```

Çıktı – program 3 çıkış koduyla biter

```
Bir sorun oluştu!
```

Ekranda bir fark görmezsin, program normal bitmiş gibidir; ama çıkış kodu `3`'tür. Terminalde son programın çıkış kodunu şu komutlarla görebilirsin:

Terminal	Komut
Linux, macOS (bash, zsh)	`echo $?`
Windows PowerShell	`$LASTEXITCODE`
Windows Komut İstemi (cmd)	`echo %ERRORLEVEL%`

`./donus3` çalıştırıp hemen ardından `echo $?` yazarsan ekrana `3` gelir. Bu bilgi, bir programı başka bir programın ya da betiğin çalıştırdığı gerçek hayatta çok işe yarar (derleme sistemleri çıkış koduna bakarak devam eder ya da durur). Derste ise kuralı bil: Normal biten programlar `return 0;` ile biter.

`main`, C++'ta özel bir fonksiyondur: sonuna `return 0;` yazmazsan derleyici sonuna kendisi `return 0;` ekler (başka hiçbir fonksiyon için böyle bir kolaylık yoktur). Yani yazmasan da program çalışır. Ama sınavda ve bu kitapta **yaz**: Okuyan kişi, programın nasıl bittiğini açıkça görsün.

`int main(void)` biçimini de görebilirsin; C'den kalan bu yazım C++'ta da geçerlidir ve `int main()` ile aynı anlama gelir. Kitapta `int main()` kullanıyoruz. `main`'in bir de komut satırı argümanlarını alan biçimi vardır; ihtiyaç olunca değineceğiz.

> **Sık Yapılan Hata: void main yazma**
>
> İnternetteki eski örneklerde şu satırı görürsün:
>
> ```cpp
> void main() {          // YANLIŞ: standart dışı
> ```
>
> `main` **her zaman** `int` döndürmelidir. `void main` standart değildir; g++ bunu `'::main' must return 'int'` hatasıyla reddeder, bazı derleyiciler ise kabul eder. Hocalar bunu yanlış sayar. Aynı eski kaynaklardan gelen şunları da kullanma: `conio.h` ve `getch()` yalnızca eski Windows derleyicilerinde vardır; `system("pause")` işletim sistemine bağımlıdır ve gereksiz yere başka bir program çalıştırır; `#include <bits/stdc++.h>` yalnızca g++'a özgüdür ve standart değildir.

1.11 Büyük/küçük harf duyarlılığı

C++, **büyük/küçük harf duyarlı** (case-sensitive) bir dildir: `cout` ile `Cout`, `main` ile `Main` bambaşka şeylerdir. Dilin sözcükleri (`int`, `return`) ve standart kütüphane adları (`cout`, `string`) küçük harfle yazılır.

buyukCout.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    Cout << "Merhaba\n";
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
buyukCout.cpp: In function 'int main()':
buyukCout.cpp:6:5: error: 'Cout' was not declared in this scope
    6 |     Cout << "Merhaba\n";
      |     ^~~~
```

g++ `Cout` adında bir şey bilmiyor. Mesaj, `cuot` örneğiyle aynıdır: yazım hatasını da büyük/küçük harf hatasını da aynı şekilde bildirir. Aynı hatayı `Main` yazarsan bağlama aşamasında alırsın (az önce gördün).

`int`, `return` gibi anahtar sözcükler de tamamen küçük harftir: `Int` ya da `RETURN` yazarsan derleme hatası alırsın.

Windows dosya adlarında büyük/küçük harfi umursamaz; `Merhaba.cpp` ile `merhaba.cpp` aynı dosya sayılır. Linux'ta ise farklı dosyalardır. Dosya adlarını küçük harf ve ASCII karakterlerle (İngiliz alfabesindeki harfler, rakamlar ve `_`; `ş`, `ğ` gibi Türkçe harfler değil) yazmayı alışkanlık haline getir.

Bölüm Özeti – Bilmen Gerekenler

- Program, bilgisayara ne yapacağını söyleyen komutlar dizisidir. **Derleyici** kodu önceden bütün olarak makine koduna çevirir; **yorumlayıcı** satır satır çalıştırır. **C++ derlenen bir dildir.**
- C++'ı Bjarne Stroustrup 1979'da Bell Labs'ta C'nin üzerine kurmaya başladı; 1983'te adı C++ oldu. Standartlar: C++98, C++11, C++14, **C++17 (kitabın standardı)**, C++20, C++23.
- Derleme hattı: **ön işlemci** (`#include`) → **derleyici** (kaynak → nesne dosyası) → **bağlayıcı** (nesne dosyası + kütüphane → çalıştırılabilir dosya).
- Derleme komutu: `g++ -std=c++17 -Wall -Wextra merhaba.cpp -o merhaba`. Çalıştırma: Linux ve macOS'ta `./merhaba`, PowerShell'de `.\merhaba`, cmd'de `merhaba`. Uyarısız derle.
- Program `int main()` fonksiyonundan başlar; sonunda `return 0;` programın başarıyla bittiğini söyler. `#include` satırı `;` ile **bitmez**; her deyim `;` ile biter, her `{` bir `}` ile kapanır.
- `cout`, `<iostream>` başlık dosyasındadır; standart kütüphane adları `std` ad alanındadır. `using namespace std;` bu adları `std::` yazmadan kullanmayı sağlar; büyük projelerde ve başlık dosyalarında kullanılmaz.
- `cout` alt satıra kendiliğinden geçmez. `\n` yeni satır, `\t` sekme, `\"` tırnak, `\\` ters bölü, `\'` tek tırnak için `\'` yazılır; `%` özel değildir.
- Yorumlar: `//` tek satır, `/* */` çok satır. `/* */` yorumları **iç içe girmez**; string içindeki `//` yorum değildir.
- Hata türleri: **derleme** (sözdizimi, `dosya.cpp:satır:sütun: error`), **bağlama** (`undefined reference`), **çalışma zamanı** (çökme ya da tanımsız davranış), **mantık** (yanlış sonuç).
- C++ **büyük/küçük harf duyarlıdır**: `Cout` ve `Main` hatadır. `void main` standart dışıdır.

Sorular

A) Çoktan Seçmeli

1 Bir C++ programı çalıştırılabilir dosyaya dönüşürken geçtiği ana aşamalar hangi sıradadır?

(A) Bağlayıcı, derleyici, ön işlemci
(B) Ön işlemci, derleyici, bağlayıcı
(C) Derleyici, ön işlemci, bağlayıcı
(D) Ön işlemci, bağlayıcı, derleyici
(E) Yorumlayıcı, derleyici, bağlayıcı

2 Bir program hatasız derleniyor, ama oluşturulurken g++ `undefined reference to 'main'` mesajı veriyor ve `ld returned 1 exit status` ile bitiyor. Bu hangi tür hatadır?

(A) Mantık hatası
(B) Çalışma zamanı hatası
(C) Derleme (sözdizimi) hatası
(D) Bağlama hatası
(E) Uyarı; program yine de çalışır

3 Aşağıdaki durumlardan hangisi bir **mantık hatasıdır**?

(A) `cout << "Merhaba\n"` satırının sonundaki `;` unutulmuştur.
(B) `cout` yerine `Cout` yazılmıştır.
(C) `main` yerine `Main` yazılmıştır.
(D) Program çalışırken bir tamsayı sıfıra bölünmektedir.
(E) Program hatasız derlenir ve çalışır, ama 3, 2, 1 diye sayması gereken satırlar 1, 2, 3 diye yazdırılır.

4 Bir programın başında `using namespace std;` satırı varsa aşağıdakilerden hangisi doğrudur?

(A) `cout` gibi standart kütüphane adları `std::` öneki yazılmadan kullanılabilir.
(B) `#include <iostream>` satırı yazmaya gerek kalmaz.
(C) Program, `std` adlı bir dosyayı derlemeye ekler.
(D) Program otomatik olarak C++17 kurallarıyla derlenir.
(E) `main` fonksiyonu yazılmak zorunda olmaz.

5 Aşağıdaki komutlardan hangisi ekrana tam olarak `Yol: C:\ders` yazdırır? (Satır sonu önemsizdir.)

(A) `cout << "Yol: C:\ders";`
(B) `cout << "Yol: C:/ders";`
(C) `cout << "Yol: C:\\ders";`
(D) `cout << "Yol: C:\\\\ders";`
(E) `cout << 'Yol: C:\ders';`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>

using namespace std;

int main() {
    cout << "Not: %85\n";
    cout << "C:\\ders\\cpp\n";
    cout << "\"Merhaba\"\n";
    cout << "A" << '\t' << "B" << '\t' << "C" << '\n';
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>

using namespace std;

int main() {
    cout << "1\n";
    // cout << "2\n";
    cout << "3\n"; // cout << "4\n";
    /* cout << "5\n";
       cout << "6\n"; */
    cout << "7 // 8\n";
    cout << "9 /* 10 */\n";
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır? Boş satır olup olmadığına dikkat et.

```cpp
#include <iostream>

using namespace std;

int main() {
    cout << "Ali";
    cout << " Veli\n";
    cout << "Ayse\\n";
    cout << "Fatma\n\n";
    cout << "Son\n";
    return 0;
}
```

C) Kod Yazma

9 Adını ve okulunu aşağıdaki gibi bir **kutu içinde** yazdıran `kutu` programını yaz. Her satır için ayrı bir `cout` kullan. Kenarların hizalı olması için boşlukları dikkatle say.

```
+----------------------+
| Ad    : Elif Demir   |
| Okul  : Ankara Univ. |
+----------------------+
```

10 Aşağıdaki **desenli isim kartını** yazdıran `isimKarti` programını yaz. Desende ters bölü ve çift tırnak geçtiğine dikkat et.

```
/\/\/\/\/\/\/\/\/\
|                 |
|    Elif Demir    |
|  "Kod Yazarim"   |
|   Hedef: %100    |
|                 |
\/\/\/\/\/\/\/\/\/
```

Çözümler

Cevap anahtarı

1	2	3	4	5
B	D	E	A	C

1 Cevap: B

Sıra **ön işlemci → derleyici → bağlayıcı**'dır. Ön işlemci `#include` gibi `#` satırlarını işler, derleyici kodu makine koduna (nesne dosyasına) çevirir, bağlayıcı da nesne dosyasını kütüphanelerle birleştirip çalıştırılabilir dosyayı üretir. A, C ve D sıralaması yanlıştır; E'de yorumlayıcı yoktur, C++ derlenen bir dildir.

2 Cevap: D

`undefined reference` ve `ld returned 1 exit status` bağlayıcının mesajıdır: Kodun yazımı doğrudur (derleme geçmiştir) ama bağlayıcı `main` fonksiyonunu bulamıyor. A yanlış, çünkü program hiç çalışmıyor. B yanlış, çünkü hata program çalışırken değil, çalıştırılabilir dosya üretilirken çıkıyor. C yanlış, çünkü sözdizimi hatalarında mesaj `dosya.cpp:satır:sütun: error` düzenindedir. E yanlış, çünkü çalıştırılabilir dosya oluşmaz.

3 Cevap: E

Mantık hatasında program derlenir ve çalışır, ama yanlış sonuç verir; E tam olarak bunu anlatıyor. A ve B **derleme hatasıdır** (eksik `;`, tanınmayan `Cout`), C **bağlama hatasıdır** (`main` bulunamaz), D **çalışma zamanı hatasıdır**.

4 Cevap: A

`using namespace std;` standart ad alanının adlarını kısa yazmayı sağlar: `std::cout` yerine `cout`. B yanlış, `#include <iostream>` yine gerekir; ad alanı bildirimi dosya eklemez. C yanlış, `std` bir dosya değil bir ad alanıdır. D yanlış, standardı derleme komutundaki `-std=c++17` seçer. E yanlış, her program `main` ile başlar.

5 Cevap: C

`\\` ters bölü karakterinin kendisini yazdırır; `Yol: C:\ders` için `C:\\ders` gerekir. A'da `\d` C++ standardında tanımlı bir kaçış dizisi değildir; g++ `unknown escape sequence` uyarısı verir ve sonuç derleyiciye bağlıdır. B eğik çizgiyi `/` yazdırır. D iki ters bölü yazdırır (`C:\\ders`). E'de çift tırnak yerine tek tırnak kullanılmıştır; tek tırnak tek bir karakter içindir, metin için değil.

6

Satır satır izle:

1. `%` `cout` için özel değildir, olduğu gibi yazılır: `Not: %85`.
2. `\\` tek ters bölü yazdırır, son `\n` alt satıra geçirir: `C:\ders\cpp`.
3. `\"` çift tırnak karakteridir: `"Merhaba"`.
4. `'\t'` sekme boşluğudur: `A`, `B`, `C` arasında birer tab vardır.

Çıktı

```
Not: %85
C:\ders\cpp
"Merhaba"
A       B       C
```

7

Yorumların içindekiler çalışmaz, string'in içindeki işaretler yorum değildir.

1. `cout << "1\n";` normal çalışır: `1`.
2. `// cout << "2\n";` tamamen yorumdur: **yok**.
3. `cout << "3\n";` çalışır: `3`. Satır sonundaki `// cout << "4\n";` yoktur.
4. `/* ... */` iki satırı birden yok sayar: 5 ve 6 **yok**.
5. `"7 // 8\n"`: `//` tırnağın içinde olduğu için yorum değildir, aynen yazılır.
6. `"9 /* 10 */\n"`: aynı şekilde aynen yazılır.

Çıktı

```
1
3
7 // 8
9 /* 10 */
```

8

İmlecin yerini izleyelim:

1. `cout << "Ali";` → `Ali`, imleç aynı satırda.
2. `cout << " Veli\n";` → `" Veli"` aynı satıra eklenir, `\n` alt satıra geçirir: `Ali Veli`.
3. `cout << "Ayse\\n";` → `\\` tek ters bölüdür, ardından gelen `n` sıradan bir harftir. Ekrana `Ayse\n` yazılır ve **alt satıra geçilmez**.
4. `cout << "Fatma\n\n";` → `Fatma` aynı satırda devam eder, ilk `\n` satırı bitirir, ikinci `\n` boş bir satır bırakır.
5. `cout << "Son\n";` → `Son`.

Çıktı

```
Ali Veli
Ayse\nFatma

Son
```

9

Her satırı ayrı bir `cout` ile yazdırmak yeterli. Kutunun sağ kenarı hizalı olsun diye "Elif Demir" satırının sonunda üç boşluk bırakıldığına dikkat et.

kutu.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    cout << "+----------------------+\n";
    cout << "| Ad    : Elif Demir   |\n";
    cout << "| Okul  : Ankara Univ. |\n";
    cout << "+----------------------+\n";
    return 0;
}
```

Çıktı

```
+----------------------+
| Ad    : Elif Demir   |
| Okul  : Ankara Univ. |
+----------------------+
```

10

Ters bölü için `\\`, çift tırnak için `\"` kullanılır; yüzde işareti ise olduğu gibi yazılır. Üst ve alt kenardaki her `\` karakteri çift yazılır; satır sonu için ayrıca `\n` eklenir. Üst kenarın sonundaki `\\\n`, "bir ters bölü, ardından yeni satır" demektir.

isimKarti.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    cout << "/\\/\\/\\
```

BÖLÜM 2

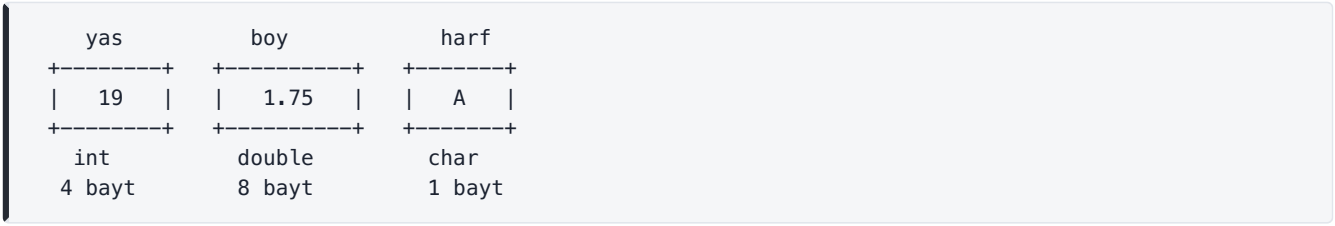
Değişkenler, Veri Tipleri ve Operatörler

Bu Bölümde Öğreneceklerin

- Değişkenin ne olduğunu; tanımlamayı, ilk değer vermeyi ve atamayı
- Geçerli değişken adı kurallarını ve camelCase adlandırma alışkanlığını
- Temel veri tiplerini, bellek boyutlarını ve `sizeof` operatörünü
- Süslü parantezle ilk değer vermeyi ve daralma hatasını önlemeyi
- İlk değer verilmemiş değişkenin neden tehlikeli olduğunu ve çöp değerin ne olduğunu
- Sabitleri ve `auto` ile otomatik tip çıkarımını
- Aritmetik operatörleri, tamsayı bölmesini ve kalan operatörünü
- Örtük ve açık tip dönüşümünü, C++ tarzı dönüştürmeyi
- Bileşik atamayı ve artırma/azaltma operatörlerinin ön ek/son ek farkını
- Operatör önceliğini, tamsayı taşmasını ve ondalıklı sayıların hassasiyetini
- Standart matematik kütüphanesindeki hazır fonksiyonları

2.1 Değişken nedir?

Bir program veriyle çalışır: öğrencinin notu, ürünün fiyatı, kullanıcının yaşı. Bu verileri program çalıştığı sürece bir yerde tutman gerekir. Bu yer bilgisayarın **belleğidir** (RAM). **Değişken** (variable), bellekte **adı olan** ve içinde bir değer tutan kutudur. Her değişkenin üç özelliği vardır: **ad** (kutunun etiketi), **tip** (kutunun büyüklüğü ve içine ne konabileceği) ve **değer** (kutunun o anki içeriği).



Değişkene "değişken" dedirten şey, değerinin program boyunca **değişebilmesidir**: kutudaki sayıyı çıkarıp yenisini koyabilirsin. Tip ise değişmez; `int` açtığın kutu sonuna kadar `int` kalır. Bellekteki her kutunun bir de **adresi** vardır; adresleri Bölüm 9'da (işaretçiler) göreceksin.

2.2 Tanımlama, ilk değer verme ve atama

Bir değişkeni kullanmadan önce **tanımlaman** (definition) gerekir: derleyiciye "şu tipte, şu adda bir kutu aç" dersin. C++'ta üç farklı kalıpla ilk değer verebilirsin:

```
tip ad;                // tanımlama
tip ad = ilkDeger;     // tanımlama + atama operatörüyle ilk değer verme
tip ad{ilkDeger};      // tanımlama + süslü parantezle ilk değer verme (C++11)
ad = yeniDeger;        // atama (assignment): içeriği sonradan değiştirme
```

İlk değer verme (initialization), kutuyu açarken içine ilk değeri koymaktır; atama ise kutu zaten varken içeriği değiştirmektir. Sınavda "ikisi arasındaki fark nedir?" diye sorulabilir. Modern C++'ta değişkenlere ilk değer verirken eşittir (`=`) yerine süslü parantezle ilk değer verme (uniform initialization) tavsiye edilir.

ilkDegisken.cpp

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    int yas{19};
    double boy{1.75};
    char harf{'A'};
    string isim{"Ali"};

    cout << "Ad: " << isim << '\n';
    cout << "Yaş: " << yas << '\n';
    cout << "Boy: " << boy << '\n';
    cout << "Harf: " << harf << '\n';
    return 0;
}
```

Çıktı

```
Ad: Ali
Yaş: 19
Boy: 1.75
Harf: A
```

Ekrana yazdırmak için `<iostream>` kütüphanesindeki `cout` nesnesi ve ekleme operatörü (`<<`) kullanılır. `'\n'` satır sonunu (yeni satıra geçmeyi) temsil eder (endl farkı Bölüm 3'te). Karakterler tek tırnakla (`'A'`), metinler çift tırnakla (`"Ali"`) yazılır. Metinleri tutmak için C++'a özgü `string` tipi kullanılır ve `<string>` başlık dosyası gerekir. `string`'in ayrıntılarını Bölüm 8'de göreceksin.

`=` işareti matematikteki "eşittir" değil, **atama operatörüdür**: Sağ taraf hesaplanır, sonuç soldaki kutuya konur. Bu yüzden `sayi = sayi + 1;` geçerlidir.

atamaOrnek.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int sayi{10};
    sayi = 25;                 // eski değer silinir
    sayi = sayi + 1;           // önce sağ taraf hesaplanır: 25 + 1
    cout << "sayi = " << sayi << '\n';

    int a{3}, b{8};            // birden çok tanım tek satırda
    int gecici{a};             // takas: a'nın değerini sakla
    a = b;
    b = gecici;
    cout << "a = " << a << ", b = " << b << '\n';
```

```
    int x, y, z;
    x = y = z = 0;              // sağdan sola: önce z, sonra y, sonra x
    cout << x << " " << y << " " << z << '\n';
    return 0;
}
```

Çıktı

```
sayi = 26
a = 8, b = 3
0 0 0
```

Üç nokta: (1) Aynı tipte birden çok değişkeni virgülle tek satırda tanımlayabilirsin. (2) İki değişkenin değerini değiştirmek (**takas**, swap) için üçüncü bir geçici kutu gerekir. (3) Değişkenleri kullanmadan hemen önce tanımlamak en iyi alışkanlıktır.

2.3 Değişken adları ve anahtar sözcükler

Değişkene istediğin adı veremezsin. Kurallar:

1. Ad **harf, rakam ve alt çizgiden** (`_`) oluşur ve **rakamla başlamaz**.
2. Boşluk, `-`, `+`, `.`, `$` gibi özel karakterler kullanılamaz.
3. **Anahtar sözcük** (keyword) olamaz (`int`, `for`, `class`, `return` …).
4. Büyük/küçük harf farklıdır: `sayi`, `Sayi` ve `SAYI` üç ayrı değişkendir.
5. Türkçe karakterleri (`ş`, `ğ`, `ı`, `ö`, `ü`, `ç`) kullanma. Kodlama sorunları çıkarabilir.

Ad	Geçerli mi?	Neden
`ogrenciSayisi`, `sayi2`, `_toplam`	Evet	Rakam sonda olabilir, `_` ile başlanabilir
`2sayi`	**Hayır**	Rakamla başlıyor
`ogrenci sayisi`	**Hayır**	Boşluk var
`ortalama-not`	**Hayır**	`-` çıkarma işaretidir
`class`	**Hayır**	Anahtar sözcük

Hatalı ad derleyicide şöyle görünür:

adRakam.cpp

```
int main() {
    int 2sayi = 5;
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
adRakam.cpp: In function 'int main()':
adRakam.cpp:2:9: error: expected unqualified-id before numeric constant
    2 |     int 2sayi = 5;
      |         ^~~~
```

Adlandırma alışkanlığı. Kitapta **camelCase** kullanıyoruz: İlk sözcük küçük, sonrakilerin baş harfi büyük (`ogrenciSayisi`, `ortalamaNot`). Ad, değişkenin ne tuttuğunu söylemeli: `x` yerine `yas`. Sabitlerin adı BÜYÜK_HARFLE yazılır (`MAX_NOT`). Sınıf adları PascalCase (`BankaHesabi`) yazılır. `i`, `j` gibi tek harfli adlar yalnızca döngü sayaçlarında kabul görür.

2.4 Temel veri tipleri ve sizeof

Bellek **bayt** (byte) denen küçük parçalara bölünmüştür; **1 bayt = 8 bit** ve n bitle 2^n farklı değer saklanabilir. Tip, değişkene **kaç bayt ayrılacağını** belirler:

Tip	Ne tutar?	Boyut	Aralık (işaretli)
`char`	Karakter ya da küçük tamsayı	1 bayt	-128 ... 127
`short`	Küçük tamsayı	2 bayt	-32768 ... 32767
`int`	Tamsayı	4 bayt	yaklaşık ±2,1 milyar
`long long`	Çok büyük tamsayı	8 bayt	yaklaşık $\pm 9{,}2 \times 10^{18}$
`float`	Ondalıklı sayı, tek duyarlık	4 bayt	yaklaşık 7 anlamlı basamak
`double`	Ondalıklı sayı, çift duyarlık	8 bayt	yaklaşık 15 anlamlı basamak
`bool`	Mantıksal değer	1 bayt	`true` (1) ya da `false` (0)

- Tamsayı tipleri varsayılan olarak **işaretlidir** (signed). Başına `unsigned` (işaretsiz) yazarsan negatif değer tutmaz, pozitif tarafı iki katına çıkar: `unsigned int` 0 ... 4294967295.
- Ondalıklı sayı için kitapta **`double`** kullanacağız.
- `bool` tipi C++'ın yerleşik tipidir; `true` bellekte genellikle 1, `false` 0 olarak tutulur.
- `string` aslında bir sınıf (class) tipidir (sınıfları Bölüm 11'de göreceksin); boyutu derleyiciye bağlıdır (genellikle 24 veya 32 bayt).

Boyutları `sizeof` operatörüyle öğrenirsin:

boyutlar.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    cout << "char      : " << sizeof(char) << '\n';
    cout << "int       : " << sizeof(int) << '\n';
    cout << "long long : " << sizeof(long long) << '\n';
    cout << "float     : " << sizeof(float) << '\n';
    cout << "double    : " << sizeof(double) << '\n';
    cout << "bool      : " << sizeof(bool) << '\n';
    return 0;
}
```

Çıktı

```
char      : 1
int       : 4
long long : 8
float     : 4
double    : 8
bool      : 1
```

`sizeof` bir fonksiyon değil **operatördür**. Sonucu bayt sayısıdır. Değişkene de uygulanır: `sizeof(yas)`.

> **Not: C++'ta long kaç bayt?**
>
> C++ standardı `int`'in en az 2, `long`'un en az 4, `long long`'un en az 8 bayt olmasını garanti eder. Günümüzde `int` genellikle 4 bayttır. `long` ise Windows'ta 4, macOS/Linux'ta 8 bayttır. Aynı program senin bilgisayarında `long` satırında farklı bir sayı yazabilir. Taşınabilir kod için büyük sayılarda `long long` kullan.

2.5 İlk değer verilmemiş değişken ve çöp değer

Değişkeni tanımlayıp içine hiçbir şey koymazsan kutuda **o bellek bölgesinde daha önce kalmış rastgele bir değer** bulunur. Buna **çöp değer** (garbage value) denir:

```
int sayi;                  // ilk değer verilmedi
cout << sayi << '\n';      // çöp değer yazar
```

Bu değeri okumak **tanımsız davranıştır** (undefined behavior): standart, programın ne yapacağı konusunda hiçbir söz vermez; çöp değer yazabilir, tesadüfen `0` yazabilir, çökebilir. Her çalıştırmada farklı bir sayı çıkabilir. Kötüsü, tesadüfen `0` yazıp programın çalışıyormuş gibi görünmesidir; hata sınavda patlar. `-Wall` bunu yakalar.

copDeger.cpp

```
#include <iostream>

using namespace std;

int main() {
    int sayi;
    cout << sayi << '\n';
    return 0;
}
```

Derleyici uyarısı – g++ -Wall -Wextra

```
copDeger.cpp: In function 'int main()':
copDeger.cpp:7:21: warning: 'sayi' is used uninitialized [-Wuninitialized]
    7 |     cout << sayi << '\n';
      |                     ^~~~
copDeger.cpp:6:9: note: 'sayi' was declared here
    6 |     int sayi;
      |         ^~~~
```

Kural: Değişkene ilk değerini tanımlarken ver. Sonradan klavyeden okuyacak olsan bile `int sayi{0};` yaz.

> **Sık Yapılan Hata: int a, b, c{0}; kalıbı**
>
> `int a, b, c{0};` satırı **üç değişkeni de** sıfırlamaz; yalnız `c` sıfır olur. `a` ile `b` çöp değerle kalır. Doğrusu: `int a{0}, b{0}, c{0};`.

2.6 Süslü parantez `{}` ile daralma (narrowing) hatasını önleme

C tarzı `=` ile ilk değer verme, ondalıklı sayıyı tamsayıya atarken kesir kısmını sessizce atar. Bu olaya **daralma** (narrowing) denir. C++11 ile gelen `{}` ile ilk değer verme ise veri kaybına izin vermez ve derleme hatası üretir.

daralma.cpp

```cpp
int main() {
    int a = 5.9;   // Derlenir, a = 5 olur (veri kaybı)
    int b{5.9};    // HATA: narrowing conversion
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
daralma.cpp: In function 'int main()':
daralma.cpp:3:11: error: narrowing conversion of '5.9000000000000004e+0' from 'double' to 'int' [-
Wnarrowing]
    3 |     int b{5.9};    // HATA: narrowing conversion
      |           ^~~
daralma.cpp:2:9: warning: unused variable 'a' [-Wunused-variable]
    2 |     int a = 5.9;   // Derlenir, a = 5 olur (veri kaybı)
      |         ^
daralma.cpp:3:9: warning: unused variable 'b' [-Wunused-variable]
    3 |     int b{5.9};    // HATA: narrowing conversion
      |         ^
```

Bu özellik, yanlışlıkla büyük tipten küçük tipe atama yapmanı engeller. Modern C++'ta `{}` kullanımı bu yüzden önerilir.

2.7 Sabitler ve auto

const ile sabit

Bazı değerler program boyunca değişmemelidir: π sayısı, KDV oranı. Tanımlamanın başına `const` yazarsan değişken **salt okunur** (read-only) olur. İlk değeri tanımlarken vermek zorundasın. Koda doğrudan yazılan sayılara **sihirli sayı** (magic number) denir, kodun okunmasını zorlaştırır; `const` kullanmak bunu önler.

constAlan.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    const double PI{3.14159};
    const int HAFTA_GUN{7};
    double yaricap{2.0};

    cout << "Çevre: " << 2 * PI * yaricap << '\n';
    cout << "Haftada " << HAFTA_GUN << " gün var.\n";
    return 0;
}
```

Çıktı

```
Çevre: 12.5664
Haftada 7 gün var.
```

Sabite sonradan değer atamaya çalışırsan derleyici hata verir ve program derlenmez:

constHata.cpp

```cpp
int main() {
    const int HAFTA_GUN{7};
    HAFTA_GUN = 8;
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
constHata.cpp: In function 'int main()':
constHata.cpp:3:15: error: assignment of read-only variable 'HAFTA_GUN'
    3 |     HAFTA_GUN = 8;
      |     ~~~~~~~~~~^~~
```

auto ile otomatik tip çıkarımı

C++11 ile `auto` anahtar sözcüğü, derleyicinin değişkenin tipini ilk değerinden otomatik anlamasını sağlar. Sağ tarafın tipine bakıp kutunun tipini o an belirler.

autoOrnek.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    auto a{5};          // a bir int olur
    auto b{5.5};        // b bir double olur
    auto c = 'X';       // c bir char olur
    auto gecerli{true}; // gecerli bir bool olur

    cout << "a'nın boyutu: " << sizeof(a) << '\n';
    cout << "b'nin boyutu: " << sizeof(b) << '\n';
    cout << "c'nin boyutu: " << sizeof(c) << '\n';
    cout << "gecerli'nin boyutu: " << sizeof(gecerli) << '\n';
    return 0;
}
```

Çıktı

```
a'nın boyutu: 4
b'nin boyutu: 8
c'nin boyutu: 1
gecerli'nin boyutu: 1
```

`auto` kullanmak için değişkene mutlaka ilk değer vermelisin, yoksa derleyici tipi bilemez (`auto x;` hatalıdır). Uzun tiplerde (örneğin Bölüm 15'teki iteratörlerde) kod yazmayı çok kolaylaştırır.

Sabit değer türleri (literal)

Koda doğrudan yazdığın değerlere **sabit değer** (literal) denir. Yazılışı tipini belirler:

- `5`: `int`
- `5LL`: `long long`
- `5.0`: `double` (noktalı sayı)
- `5.0f`: `float`
- `'A'`: `char` (tek tırnak)
- `"Ali"`: C tarzı metin dizisi (Bölüm 8'de `string`'e çevrilir)

- `true` / `false`: `bool`

2.8 Aritmetik operatörler

Operatör bir işlem yapan simgedir; işlemin uygulandığı değerlere **işlenen** (operand) denir. Dört işlemi bildiğin gibi kullanırsın (`+ - * /`); beşincisi `%`, bölümden **kalanı** verir.

aritmetik.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int a{17}, b{5};
    cout << a << " + " << b << " = " << a + b << '\n';
    cout << a << " - " << b << " = " << a - b << '\n';
    cout << a << " * " << b << " = " << a * b << '\n';
    cout << a << " / " << b << " = " << a / b << '\n';
    cout << a << " % " << b << " = " << a % b << '\n';
    cout << "1 / 2    = " << 1 / 2 << '\n';
    cout << "-7 / 2   = " << -7 / 2 << '\n';
    cout << "7 / 2.0  = " << 7 / 2.0 << '\n';
    return 0;
}
```

Çıktı

```
17 + 5 = 22
17 - 5 = 12
17 * 5 = 85
17 / 5 = 3
17 % 5 = 2
1 / 2    = 0
-7 / 2   = -3
7 / 2.0  = 3.5
```

`17 / 5` neden `3.4` değil de `3`? Çünkü **tamsayı bölmesi** vardır: İki işlenen de tamsayıysa sonuç tamsayıdır, kesirli kısım **atılır** (yuvarlanmaz). `1 / 2` bu yüzden `0`'dır. İşlenenlerden en az biri ondalıklıysa sonuç ondalıklı olur. Bölme sıfıra doğru keser (`-7 / 2` sonucu `-3`). Tamsayıyı sıfıra bölmek ise tanımsız davranıştır: Windows'ta (x86) program çöker, Apple M4'te çökmeden çalışabilir. Bu yüzden `x / 0` yazma, çıktısını da burada göstermiyoruz.

> **Sık Yapılan Hata: Tamsayı bölmesinde ondalık beklemek**
>
> `double sonuc = 7 / 2;` yazdığında sonucun `3.5` olacağını sanabilirsin. Ancak C++ önce sağ tarafı iki `int`'in bölmesi olarak hesaplar (`3`); ardından `double`'a atarken `3.0` yapar. Kesirli sonuç için işlenenlerden en az birini ondalıklı yazmalısın (`7.0 / 2` veya `static_cast<double>(7) / 2`).

Kalan operatörü: %

`a % b`, `a`'yı `b`'ye böldüğünde artan kalandır. Kalanın işareti daima bölünenin işaretiyle aynıdır:

modNegatif.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    cout << "7 % 2    = " << 7 % 2 << '\n';
    cout << "-7 % 2   = " << -7 % 2 << '\n';
    cout << "7 % -2   = " << 7 % -2 << '\n';
    cout << "-7 % -2  = " << -7 % -2 << '\n';
    cout << "2 % 5    = " << 2 % 5 << '\n';
    return 0;
}
```

Çıktı

```
7 % 2    = 1
-7 % 2   = -1
7 % -2   = 1
-7 % -2  = -1
2 % 5    = 2
```

`%` yalnızca **tamsayılarla** çalışır; işlenenlerden biri `double` ise derlenmez (küsüratlı kalan için `<cmath>` kütüphanesindeki `fmod` fonksiyonu kullanılır). `1453 % 10` son basamağı (`3`), `1453 / 10` son basamağı atılmış sayıyı (`145`) verir. Küçük sayıyı büyüğe bölünce kalan sayının kendisidir (`2 % 5 == 2`).

> **Sınavda Çıkar**
>
> " `-17 % 5` kaçtır?" ve "`7 / 2 * 2.0` kaçtır?" tarzı sorular sınavlarda sıkça çıkar. Unutma: İki tamsayının bölmesi tamsayıdır. `%`'de kalan bölünenin işaretini taşır (`-17 % 5 = -2`). Aynı öncelikteki işlemler soldan sağa yapılır: `7 / 2 = 3`, sonra `3 * 2.0 = 6.0`.

2.9 Tip dönüşümü

Farklı tipler karışırsa derleyici dönüşüm yapar (örtük) veya sen zorlarsın (açık).

Örtük dönüşüm

İşlemin iki işleneni farklıysa C++ küçük tipi büyük tipe yükseltir: `int` ile `double` toplanırsa sonuç `double` olur. Atamada sağdaki değer soldakinin tipine çevrilir. `double`'dan `int`'e atarken ondalık **atılır** (yuvarlama yapılmaz, sıfıra doğru kesilir).

ortukDonusum.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int tamsayi{5};
    double toplam = tamsayi + 2.5;   // int, double'a yükselir
    int a = 3.99;                    // ondalık kısım atılır, 3
    double c = 7 / 2;                // önce tamsayı bölmesi (3), sonra 3.0
    double d = 7 / 2.0;              // ondalıklı bölme: 3.5

    cout << "toplam = " << toplam << '\n';
    cout << "a = " << a << '\n';
    cout << "c = " << c << ", d = " << d << '\n';
    cout << "5 / 2 + 0.5 = " << 5 / 2 + 0.5 << '\n';
    return 0;
}
```

Çıktı

```
toplam = 7.5
a = 3
c = 3, d = 3.5
5 / 2 + 0.5 = 2.5
```

Dönüşüm **işlem işlem** yapılır. `double c = 7 / 2;` klasik tuzaktır: Değişkenin `double` olması bölmenin ondalıklı yapılmasını sağlamaz, çünkü bölme atamadan önce yapılır.

macOS'taki `g++` (Apple clang) `int a = 3.99;` satırı için `implicit conversion from 'double' to 'int' changes value from 3.99 to 3` uyarısı verir; gerçek g++ `-Wall -Wextra` ile bu satırda susar. Uyarı haklıdır: veri kaybını bilerek yapıyorsan `static_cast<int>(3.99)` yaz (aşağıda).

Açık dönüşüm (cast)

Bir tipin değerini başka tipe zorlamak için **cast** kullanılır. C++'ta bunun için modern `static_cast<Tip>` kalıbı vardır. C tarzı dönüştürme `(tip)ifade` de çalışır ancak C++'ta bu eski yöntem önerilmez.

castFark.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int a{7}, b{2};
    cout << "a / b                    : " << a / b << '\n';
    cout << "static_cast<double>(a) / b : " << static_cast<double>(a) / b << '\n';
    cout << "static_cast<double>(a / b) : " << static_cast<double>(a / b) << '\n';
    cout << "(double)a / b            : " << (double)a / b << '\n';
    return 0;
}
```

Çıktı

```
a / b                    : 3
static_cast<double>(a) / b : 3.5
static_cast<double>(a / b) : 3
(double)a / b            : 3.5
```

`static_cast<double>(a) / b` ifadesinde cast, bölmeden **önce** `a`'ya uygulanır: `7.0 / 2` olur ve sonuç `3.5` çıkar. Parantez içindeki ifadeye uygulanırsa `(a / b)`, önce `3` hesaplanır, sonra `3.0`'a çevrilir. Son satırdaki `(double)a / b` **C tarzı dönüştürmedir**; burada aynı sonucu verir. C++'ta `static_cast` tercih edilir: aranması kolaydır ve `const` gibi nitelikleri kaldıran tehlikeli dönüşümleri yapamaz. C tarzı cast ise bunların hepsini sessizce yapabilir. Eski kodlarda karşılaşırsan okuyabilirsin, kendi kodunda yazma. Cast değişkenin kendisini değiştirmez, yeni tipte bir kopya üretir.

> **Sınavda Çıkar**
>
> Cast'in **yeri** sonucu tamamen değiştirir: `static_cast<double>(toplam) / adet` doğru ortalamadır, `static_cast<double>(toplam / adet)` ise tamsayı bölmesi yapıldıktan sonra dönüştürdüğü için yanlıştır. Cast bölmeden önce işlenenlerden birine uygulanmalıdır.

> **İpucu: Ortalamayı tamsayı bölmesiyle hesaplamak**
>
> `toplam` ve `adet` `int` iken `double ortalama = toplam / adet;` yazarsan `ortalama` kesilmiş çıkar. Çözüm işlenenlerden birini `double` yapmaktır: `double ortalama = static_cast<double>(toplam) / adet;`.

İşaretli ile işaretsiz karşılaştırma tuzağı

`int` ile `unsigned int` aynı işlemde karşılaşırsa C++ **işaretliyi işaretsize çevirir**, tersini değil. `-1` işaretsize çevrilince en büyük değere (4294967295) sarar. Aşağıdaki programda `if`/`else` kullanıyoruz; ayrıntısını Bölüm 4'te göreceksin, şimdilik "koşul doğruysa ilk süslü parantezi, değilse `else`'ten sonrakini çalıştır" diye oku:

isaretKarsilastirma.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int a{-1};
    unsigned int b{1};

    cout << "a'nın işaretsiz hali: " << static_cast<unsigned int>(a) << '\n';
    if (a < b) {
        cout << "-1, 1'den küçük\n";
    } else {
        cout << "-1, 1'den küçük DEĞİL (!)\n";
    }
    return 0;
}
```

Derleyici uyarısı – g++ -Wall -Wextra

```
isaretKarsilastirma.cpp: In function 'int main()':
isaretKarsilastirma.cpp:10:11: warning: comparison of integer expressions of different signedness:
'int' and 'unsigned int' [-Wsign-compare]
   10 |     if (a < b) {
      |         ~~^~~
```

Çıktı

```
a'nın işaretsiz hali: 4294967295
-1, 1'den küçük DEĞİL (!)
```

Bu uyarı `-Wall -Wextra` sayesindedir.

char: küçük bir tamsayı

`char` bellekte karakteri değil onun ASCII **kodunu** saklar (Ek D). `'A'` için 65, `'a'` için 97, `'0'` için 48. Bu yüzden `char` aslında bir tamsayı tipidir.

charTamsayi.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    char harf{'A'};
    int kod = harf;             // 65
    cout << harf << " kodu: " << kod << '\n';

    char sonraki = harf + 1;
    cout << "A'nın bir fazlası: " << sonraki << '\n';

    cout << "'d' - 'a' = " << 'd' - 'a' << '\n';
    return 0;
}
```

Çıktı

```
A kodu: 65
A'nın bir fazlası: B
'd' - 'a' = 3
```

Büyük ve küçük harf arasında hep 32 fark vardır. `'7'` karakteri ile `7` sayısı aynı şey değildir; rakam karakterinin sayı değerini bulmak için `'0'` karakterini çıkarırsın: `'7' - '0' == 7`. Aritmetikte `char` otomatik olarak `int`'e yükseltilir.

2.10 Bileşik atama, ++ ve --

Bileşik atama

`x = x + 5` işleminin kısaltması **bileşik atama**dır (compound assignment): `x += 5`. `+=`, `-=`, `*=`, `/=`, `%=` operatörleri mevcuttur.

bilesik.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int x{10};
    x += 5;                 // 15
    x %= 3;                 // 15 % 3 = 0
    cout << "x = " << x << '\n';

    int y{4};
    y *= 2 + 3;             // y = y * (2 + 3)
    cout << "y = " << y << '\n';
    return 0;
}
```

Çıktı
```
x = 0
y = 20
```

`y *= 2 + 3` ifadesi sağ tarafı parantez içindeymiş gibi işler: `y = y * (2 + 3)` olur, sonuç 20'dir.

Artırma ve azaltma: ++ ve --

Değişkeni 1 artırmak için `x++` veya `++x` kullanılır. `x++` (son ek, postfix) ifade olarak **eski** değeri verir, sonra değişken artar. `++x` (ön ek, prefix) önce artırır, **yeni** değeri verir.

onSonEk.cpp
```cpp
#include <iostream>

using namespace std;

int main() {
    int n{5};
    cout << n++ << '\n';  // eski değer (5) yazdırılır, sonra 6 olur
    cout << n << '\n';    // 6
    cout << ++n << '\n';  // 7 olur, sonra 7 yazdırılır
    return 0;
}
```

Çıktı
```
5
6
7
```

Tek başına kullanıldıklarında `x++;` ile `++x;` aynı işi yapar. `x` bir ifadenin içindeyse (örneğin atamada) fark yaratır. Aynı ifadede bir değişkeni birden fazla kez `++` ile değiştirmek tanımsız davranıştır (`int a = b++ + ++b;` yazma).

2.11 Operatör önceliği

Aynı ifadedeki birden çok operatörün sırasını **öncelik** (precedence) belirler: `* / %` operatörleri `+ -` işlemlerinden önce yapılır. Aynı öncelikliler **soldan sağa** işlenir. Atama ise sağdan sola işler.

oncelik.cpp
```cpp
#include <iostream>

using namespace std;

int main() {
    cout << "2 + 3 * 4     = " << 2 + 3 * 4 << '\n';         // 14
    cout << "(2 + 3) * 4   = " << (2 + 3) * 4 << '\n';       // 20
    cout << "100 / 10 * 5  = " << 100 / 10 * 5 << '\n';      // 50
    cout << "7 + 8 % 3 * 2 = " << 7 + 8 % 3 * 2 << '\n';     // 11
    return 0;
}
```

Çıktı
```
2 + 3 * 4     = 14
(2 + 3) * 4   = 20
100 / 10 * 5  = 50
7 + 8 % 3 * 2 = 11
```

`100 / 10 * 5` işleminde `100 / 50 = 2` değildir. `/` ve `*` aynı seviyededir, soldan sağa işlenir: önce `100 / 10 = 10`, sonra `10 * 5 = 50`. Sırayı parantezlerle kontrol edebilirsin. Karmaşık ifadeleri yazarken kodu okuyana niyetini göstermek için parantez kullan.

2.12 Taşma ve ondalıklı sayıların hassasiyeti

Tamsayı taşması

Bir hesabın sonucu tipin tutabileceği aralığın dışına çıkarsa **taşma** (overflow) olur. İşaretsiz (`unsigned`) taşma tanımlıdır ve başa sarar (0'a döner). Ancak standart işaretli (`int`) taşma için C++'ta davranış **tanımsızdır** (UB).

```
int a = 2000000000;
int b = a * 2; // int aralığını aşar: TANIMSIZ DAVRANIŞ
```

Büyük sayılarla çalışıyorsan baştan `long long` tipini kullan. Çarpmada işlenenlerden birini **çarpmadan önce** dönüştür; sonucu da `long long` bir değişkene koy: `long long b = static_cast<long long>(a) * 2;`. (`static_cast<long long>(a * 2)` yazmak işe yaramaz: taşma çarpma sırasında zaten olmuştur.)

Ondalıklı sayıların hassasiyeti

Bilgisayar sayıları ikilik tabanda saklar. `0.1` gibi sayılar ikilik sistemde sonsuz devirlidir ve tam olarak saklanamaz. Yakın bir değere yuvarlanır.

hassasiyet.cpp

```
#include <iostream>

using namespace std;

int main() {
    double toplam = 0.1 + 0.2;
    cout << (toplam == 0.3) << '\n'; // Eşit mi? (1: Evet, 0: Hayır)
    return 0;
}
```

Çıktı

```
0
```

Ondalıklı sayılar genellikle `==` ile karşılaştırılmamalıdır (0 yazdı, yani false). `float` yaklaşık 7, `double` yaklaşık 15 anlamlı basamak hassasiyetindedir. Bu yüzden iki ondalıklı sayının eşitliğini `==` ile değil, aralarındaki farkın küçük bir sayıdan az olmasıyla kontrol edersin (Bölüm 4'te `fabs` ile göreceksin).

2.13 `<cmath>` kütüphanesi

Matematiksel fonksiyonlar için `<cmath>` başlık dosyası eklenir. Fonksiyonlar çoğunlukla `double` alır ve `double` döndürür (`abs` tamsayıyı da kabul eder): `sqrt` (karekök), `pow(x, y)` (x üssü y), `abs` (mutlak değer), `ceil` (yukarı yuvarla), `floor` (aşağı yuvarla), `round` (en yakına yuvarla).

matematik.cpp

```cpp
#include <iostream>
#include <cmath>

using namespace std;

int main() {
    cout << "sqrt(144)   = " << sqrt(144.0) << '\n';
    cout << "pow(2, 10)  = " << pow(2, 10) << '\n';
    cout << "abs(-9)     = " << abs(-9) << '\n';
    cout << "ceil(3.2)   = " << ceil(3.2) << '\n';
    cout << "floor(-3.2) = " << floor(-3.2) << '\n';
    cout << "round(3.5)  = " << round(3.5) << '\n';
    cout << "round(-3.5) = " << round(-3.5) << '\n';
    return 0;
}
```

Çıktı

```
sqrt(144)   = 12
pow(2, 10)  = 1024
abs(-9)     = 9
ceil(3.2)   = 4
floor(-3.2) = -4
round(3.5)  = 4
round(-3.5) = -4
```

C++'ta üs alma için `^` operatörü yoktur (bu bit düzeyinde XOR işlemidir, `2 ^ 3 == 1`). Üs almak için `pow` kullanılır. `floor(-3.2)` sonucu `-3.0` değil `-4.0`'dır ("aşağı" demek daha küçük sayı demektir).

Bölüm Özeti – Bilmen Gerekenler

- **Değişken**, bellekte adı olan kutudur. İlk değeri süslü parantez ile `{}` vermek daralma (narrowing) hatalarını derleme zamanında engeller: `int yas{19};`.
- **Tipler:** `char` (1 bayt), `int` (4), `long long` (8), `float` (4), `double` (8), `bool` (1), `string` (metin). `sizeof` bayt boyutunu verir.
- **Çöp değer:** İlk değer verilmemiş değişkeni okumak tanımsız davranıştır.
- **Sabit ve auto:** Değişmeyecek değerlere `const` eklenir. `auto`, tipi derleyicinin bulmasını sağlar.
- **Tamsayı bölmesi:** İki tamsayının bölmesi tamsayıdır: `7 / 2 == 3`. `%` kalanı verir, işaret bölünenin işaretidir: `-7 % 2 == -1`.
- **Dönüşüm:** Açık dönüştürme `static_cast<double>(x)` ile yapılır. Atamada `double`'dan `int`'e atama, ondalığı sıfıra doğru keser.
- **Bileşik atama:** `x += 5`, `x = x + 5` ile aynıdır. `x++` eski değeri kullanıp artırır, `++x` artırıp yeni değeri kullanır.
- **Taşma:** `int` taşıdığında tanımsız davranış olur. Ondalıklı sayılar tam tutulamaz, `==` ile kıyaslanmaz.
- **`<cmath>`:** Matematiksel işlemler (`sqrt`, `pow`, `abs`) için kullanılır. `^` üs alma değildir.

Sorular

A) Çoktan Seçmeli

1 Aşağıdaki kod parçası çalıştırılırsa ekrana ne yazılır?

```cpp
#include <iostream>
using namespace std;
int main() {
    int a{17}, b{5};
    double c = a / b;
    cout << c << '\n';
    return 0;
}
```

(A) 3.4
(B) 3
(C) 3.5
(D) 4
(E) Program derlenmez

2 `cout << -17 % 5 << '\n';` satırı ekrana ne yazdırır?

(A) 3 (B) 2 (C) -3 (D) 0 (E) -2

3 Aşağıdakilerden hangisi geçerli bir C++ değişken adıdır?

(A) `_toplam2` (B) `2toplam` (C) `toplam-not` (D) `class` (E) `ogrenci sayisi`

4 Aşağıdaki tanımlamalardan hangisi modern C++ standartlarında daralma (narrowing) hatası vererek derlemeyi durdurur?

(A) `int a = 5.9;` (B) `double b = 5;` (C) `int c{5.9};` (D) `auto d = 5.9;` (E) `char e = 'A';`

5 Aşağıdaki kod parçasından sonra ekrana ne yazılır?

```cpp
#include <iostream>
using namespace std;
int main() {
    int n{5};
    cout << n++ << " ";
    cout << ++n << '\n';
    return 0;
}
```

(A) 5 6
(B) 6 7
(C) 6 6
(D) 5 7
(E) Davranış tanımsızdır, çıktı tahmin edilemez.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
using namespace std;

int main() {
    int x{17}, y{5};
    double d1 = x / y;
    double d2 = static_cast<double>(x) / y;
    double d3 = static_cast<double>(x / y);
    int m = -x % y;
    cout << d1 << " " << d2 << " " << d3 << '\n';
    cout << m << " " << x / -y << '\n';
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
using namespace std;

int main() {
    char c{'a'};
    int fark = 'd' - c;
    c = c + 2;
    cout << c << " " << fark << '\n';
    char buyuk = c - 'a' + 'A';
    cout << buyuk << '\n';
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
using namespace std;

int main() {
    int a{4}, b{3};
    int c = a++ + b;
    int d = ++a * 2;
    b += a--;
    cout << a << " " << b << " " << c << " " << d << '\n';
    return 0;
}
```

C) Kod Yazma

9 Bir dersin **vize** notu `70`, **final** notu `85` olsun (ikisi de `int`). Ders ortalaması, vizenin `%40`'ı ile finalin `%60`'ı toplanarak hesaplanır. Ağırlıkları `const double` olarak tanımla, ortalamayı `double` bir değişkende tut ve yazdır. `agirlikliOrtalama` programını yaz.

```
Vize: 70
Final: 85
Ortalama: 79
```

10 Verilen toplam saniyeyi **saat, dakika, saniye** olarak gösteren `sureCevir` programını yaz. Toplam saniye `10000` olsun; yalnız `/` ve `%` operatörlerini kullan (1 saat = 3600 saniye, 1 dakika = 60 saniye).

```
10000 saniye = 2 saat 46 dakika 40 saniye
```

Çözümler

Cevap anahtarı

1 B	2 E	3 A	4 C	5 D

1 Cevap: B

`a` ve `b` iki tamsayıdır, bu yüzden `17 / 5` **tamsayı bölmesidir** ve sonuç `3` olur. Bu `3`, atamada `double`'a çevrilip `3.0` (ekranda `3` görünür) olur. `double c` olması bölmenin ondalıklı yapılmasını sağlamaz.

2 Cevap: E

Bölme sıfıra doğru kesilir: `-17 / 5 = -3`. Kalanın işareti bölünenin (`-17`) işaretidir: `-17 % 5 = -2`.

3 Cevap: A

Ad harf, rakam ve alt çizgiden oluşur, rakamla başlamaz; `_toplam2` buna uyar. B rakamla başlıyor. C'deki `-` çıkarma operatörüdür. D `class`, bir anahtar sözcük. E'de boşluk var.

4 Cevap: C

C++11 ile gelen süslü parantez `{}` ile ilk değer atamada daralma (narrowing) koruması vardır. `int c{5.9}` ifadesi veri kaybı olacağı için derlenmez. `int a = 5.9;` ise C tarzı atamadır, derlenir ve ondalık atılır.

5 Cevap: D

`n++` ilk `cout` içinde eski değeri `5` verir, sonra `n` 6 olur. İkinci `cout` içinde `++n` önce artırır (7 olur) ve yeni değer olan `7`'yi verir. Ekrana `5 7` yazılır; doğru cevap D'dir.

6

`x = 17`, `y = 5`.

1. `d1 = x / y`: iki `int` bölünür (`3`), atamada `3.0`'a çevrilir, ekranda `3` yazar.
2. `d2 = static_cast<double>(x) / y`: cast önce `x`'e uygulanır, bölme ondalıklı yapılır: `3.4`.
3. `d3 = static_cast<double>(x / y)`: bölme önce yapılır (`3`), sonra `double`'a dönüştürülür: `3`.
4. `m = -x % y`: tekli eksi `%`'den önce uygulanır, `(-17) % 5 = -2`.
5. `x / -y = 17 / -5`: sıfıra doğru kesilir: `-3`.

Çıktı

```
3 3.4 3
-2 -3
```

7

`'a'` kodu 97, `'d'` kodu 100.

1. `fark = 'd' - c = 100 - 97 = 3`. `c = c + 2` → 99, yani `'c'`. İlk yazdırılan: `c 3`.
2. `buyuk = c - 'a' + 'A' = 99 - 97 + 65 = 67` → `'C'`. İkinci yazdırılan: `C`.

Çıktı
```
c 3
C
```

8

Değişkenleri adım adım izle:

- başta: `a = 4`, `b = 3`
- `c = a++ + b`: `4 + 3 = 7` atanır, sonra `a` artar (a = 5)
- `d = ++a * 2`: önce `a` artar (a = 6), sonra `6 * 2 = 12`
- `b += a--`: `a--` eski değer `6`'yı verir: `b = 3 + 6 = 9`; sonra `a` azalır (a = 5) Sonuç: `a = 5`, `b = 9`, `c = 7`, `d = 12`.

Çıktı
```
5 9 7 12
```

9

Ağırlıklar `const double`, notlar `int`. `0.40 * vize` işleminde `int`, `double`'a yükseltilir.

agirlikliOrtalama.cpp
```cpp
#include <iostream>

using namespace std;

int main() {
    const double VIZE_AGIRLIK{0.40};
    const double FINAL_AGIRLIK{0.60};
    int vize{70};
    int finalNotu{85};

    double ortalama = VIZE_AGIRLIK * vize + FINAL_AGIRLIK * finalNotu;

    cout << "Vize: " << vize << '\n';
    cout << "Final: " << finalNotu << '\n';
    cout << "Ortalama: " << ortalama << '\n';
    return 0;
}
```

Çıktı
```
Vize: 70
Final: 85
Ortalama: 79
```

10

Saat için `10000 / 3600 = 2` (tamsayı bölmesi tam saati verir). Geriye `10000 % 3600 = 2800` saniye kalır. Bu kalandan dakika `2800 / 60 = 46`, saniye `2800 % 60 = 40` bulunur.

sureCevir.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int toplamSaniye{10000};

    int saat = toplamSaniye / 3600;
    int kalan = toplamSaniye % 3600;
    int dakika = kalan / 60;
    int saniye = kalan % 60;

    cout << toplamSaniye << " saniye = " << saat << " saat "
         << dakika << " dakika " << saniye << " saniye\n";
    return 0;
}
```

Çıktı

```
10000 saniye = 2 saat 46 dakika 40 saniye
```

BÖLÜM 3

Klavyeden Girdi ve Biçimli Çıktı

Bu Bölümde Öğreneceklerin

- Ekrana birden çok değeri tek satırda yazdırmayı ve satır sonu yazmanın iki yolunu (`cout`)
- Klavyeden bir ya da birden çok değer okumayı; boşlukla ayrılmış girdinin nasıl bölündüğünü (`cin`)
- Boşluk içeren bir satırı (ad soyad gibi) okumayı ve sayıdan sonra satır okurken düşülen tuzağı
- Sayıları sütunlara hizalamayı, ondalık hane sayısını ve dolgu karakterini ayarlamayı
- Programın sayıları varsayılan olarak kaç basamakla yazdığını
- Sayı beklenirken harf girilince ne olduğunu ve bozulan girdi akışının nasıl onarıldığını
- Hata mesajlarını normal çıktıdan ayırmayı
- Türkçe karakterlerin Windows konsolunda neden bozulduğunu ve çözümünü

Şimdiye kadar yazdığın programlar tek yönlüydü: program ekrana bir şey yazdı, sen izledin. Bu bölümde program **sana soru soracak ve cevabını bekleyecek**. Bunun için `cin` ile klavyeden okumayı öğreneceksin. Bölüm 2'de `cout`'u hep en basit biçimiyle kullandın; şimdi onu zincirlemeyi, `'\n'` ile `endl` arasındaki farkı ve çıktıyı sütunlara hizalamayı göreceksin. Kitabın başında sözünü verdiğimiz gibi, Windows'ta Türkçe karakterlerin neden bozulduğunu da bu bölümün sonunda anlatacağız.

Kitapta kullanıcının klavyeden yazdığı kısım çıktı kutusunda **mavi** görünür.

3.1 Akış kavramı ve cout

C++'ta ekrana yazmak, klavyeden okumak, ileride göreceğin gibi dosyaya yazmak ve dosyadan okumak aynı fikre dayanır: **akış** (stream). Akış, baytların bir yönde aktığı bir borudur. Programın bir ucunda durursun, boruya bir şey koyarsan öbür uçtan çıkar.

```
       cout  (karakter çıkış akışı)
program ──────────────────────────► ekran

       cin   (karakter giriş akışı)
program ◄────────────────────────── klavye
```

İkisi de `<iostream>` başlık dosyasındadır. Bölüm 2'den tanıdığın `cout` **ekrana** akan borudur, `cin` **klavyeden** gelen borudur. Oklar `<<` ve `>>` operatörlerinin yönünü de gösterir: `cout << değer` değeri boruya iter, `cin >> değişken` borudan alıp değişkene koyar.

Zincirleme yazdırma

Bölüm 1'de birkaç `<<`'yu art arda yazmış ve `<<`'nun adının ekleme operatörü olduğunu görmüştün. Şimdi bunun neden çalıştığına bakalım. `cout << x` ifadesinin bir özelliği çok işine yarayacak: ifadenin değeri yine `cout`'un kendisidir. Bu yüzden art arda eklemeler soldan sağa zincirlenir; `cout << a << b` ifadesi, önce `a`'yı yazar, sonra elde kalan `cout` ile `b`'yi yazar:

zincir.cpp

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    string ad{"Elif"};
    int yas{19};
    double boy{1.68};

    cout << "Merhaba " << ad << "!\n";
    cout << "Yaş: " << yas << ", boy: " << boy << " m\n";
    cout << "Gelecek yıl " << yas + 1 << " yaşında olacaksın.\n";
    cout << 'A' << ' ' << 'A' + 1 << ' ' << static_cast<char>('A' + 1) << '\n';
    return 0;
}
```

Çıktı

```
Merhaba Elif!
Yaş: 19, boy: 1.68 m
Gelecek yıl 20 yaşında olacaksın.
A 66 B
```

Dikkat edilecekler: `cout` türe bakarak doğru biçimi kendisi seçer, `printf`'teki gibi `%d` belirteçlerine gerek yoktur. `yas + 1` gibi ifadeler doğrudan yazdırılır. Son satırdaki tuzağa bak: `'A' + 1` bir `int` ifadesidir (Bölüm 2'deki tip yükseltmesi) ve `66` yazdırır; karakter olarak görmek istersen `static_cast<char>(...)` gerekir.

Sık Yapılan Hata: `cout` içinde karşılaştırma

`<<` operatörü `+` ve `*`'dan sonra, ama `>`, `==`, `&&`, `?:` gibi operatörlerden **önce** işlenir. Bu karşılaştırma (`>`, `==`) ve mantık (`&&`, `||`, `?:`) operatörlerini Bölüm 4'te göreceksin; şimdilik `a > b`'nin "a, b'den büyük mü?" sorusunu sorduğunu bilmen yeter. Toplama çarpma parantezsiz çalışır, karşılaştırma çalışmaz:

```cpp
cout << "Toplam: " << a + b << '\n';        // doğru
cout << "Büyük mü: " << a > b << '\n';      // YANLIŞ: derlenmez
cout << "Büyük mü: " << (a > b) << '\n';    // doğru: parantez şart
```

İkinci satır `(cout << "Büyük mü: " << a) > (b << '\n')` gibi anlamsız bir ifade olur. Karşılaştırma, `&&`, `||` ve `?:` kullanacaksan **parantez** koy.

Kaçış dizileri

Kaçış dizilerini (escape sequence) Bölüm 1'de tanıdın: çift tırnaklı metinde ters bölü (`\`) ile başlayan özel gösterimler. Burada hepsini tek tabloda topluyoruz; ekranda ters bölü karakterini ya da çift tırnağı yazdırmanın başka yolu yoktur:

Dizi	Anlamı
`\n`	Yeni satır
`\t`	Sekme (tab); sonraki sütun durağına atlar
`\\`	Ters bölü karakterinin kendisi
`\"`	Çift tırnak karakterinin kendisi
`\'`	Tek tırnak karakterinin kendisi

Örneğin `cout << "Ona \"merhaba\" dedi.\n";` ekrana `Ona "merhaba" dedi.` yazar. Windows yollarını yazarken her `\` için iki tane yazmayı unutma; tek `\k` yazarsan g++ bunu geçerli bir kaçış dizisi olarak tanımaz ve `unknown escape sequence` uyarısı verir.

'\n' ile endl farkı

Satır sonu yazmanın iki yolu vardır. Bölüm 2'de hep `'\n'` kullandık; bunun yerine `endl` de yazılabilir (*end line*). `cout << "Merhaba" << endl;` ile `cout << "Merhaba\n";` ekranda aynı görünür, ama arkada fark vardır.

`cout`'a yazdıkların hemen ekrana gitmez; önce bir **tampona** (buffer) konur, tampon uygun bir anda toplu olarak boşaltılır. `'\n'` yalnızca satır sonu karakterini yazar. `endl` ise satır sonunu yazar **ve ayrıca tamponu zorla boşaltır** (*flush*). Yani `endl`, `'\n'` artı bir boşaltmadır.

	`'\n'`	`endl`
Yeni satıra geçer	evet	evet
Tamponu boşaltır	hayır	**evet**
Hız	hızlı	her satırda boşalttığı için yavaş
Bu kitapta	varsayılan	gerektiğinde

Çok satır yazan bir programda her satırda boşaltma yapmak gereksiz yere yavaştır. Kural: **satır sonu için `'\n'` kullan**; tamponun hemen boşalmasını özellikle istediğin nadir durumlarda `endl` yaz.

> **İpucu: Prompt neden `cin`'den önce görünür?**
>
> `cout << "Yaşını gir: ";` satırında satır sonu yok, yani tampon boşalmak için bir neden yok gibi görünür. Ama C++ burada senin için çalışır: `cin` ile `cout` birbirine **bağlıdır**; program `cin`'den okumaya başlamadan hemen önce `cout`'un tamponu otomatik boşaltılır. Bu yüzden soru her zaman sen yazmadan önce ekranda belirir ve `endl` ya da başka bir şey eklemene gerek kalmaz. (C'de bunun için `fflush(stdout)` gerekebiliyordu.)

3.2 cin ile klavyeden okuma

Programın kullanıcıdan bilgi alabilmesi, programı herkes için yeniden yazmadan çalıştırabilmen demektir. Klavyeden okumak için `cin` ve **çıkarma operatörü** (extraction operator) `>>` kullanılır. Genel kalıp:

```
cin >> değişken;
```

İlk örnek, kullanıcıdan yaşını alıp bir yıl sonraki yaşını hesaplıyor:

yasOku.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int yas{0};
    cout << "Yaşını gir: ";
    cin >> yas;
    cout << "Gelecek yıl " << yas + 1 << " yaşında olacaksın.\n";
    return 0;
}
```

```
Çıktı
Yaşını gir: 19
Gelecek yıl 20 yaşında olacaksın.
```

`cin >> yas` satırında program **durur** ve sen Enter'a basana kadar bekler. Bu yüzden okumadan önce `cout` ile ne istediğini yazarsın (kullanıcıya soru, ya da *prompt*). Prompt'un sonunda `'\n'` yoktur ki cevap aynı satırda yazılsın.

Dikkat edilecek üç nokta:

- **Biçim belirteci yok.** Okuma türünü değişkenin türü belirler: `int` değişkene `>>` tamsayı, `double` değişkene ondalıklı sayı, `char` değişkene tek karakter, `string` değişkene bir sözcük okur. `scanf("%lf")` gibi tür-belirteç uyuşmazlığı derdin yoktur.
- **`&` işareti yok.** C'deki `scanf("%d", &yas)` çağrısında `&` koyardın; `cin >> yas` yazarken koymazsın. `cin >> yas` ifadesi `yas`'ı doğrudan değiştirebilir; bunun nasıl olduğunu referansları işlediğimiz Bölüm 6 ve 9'da göreceksin. Şimdilik kural: `cin >> değişken`.
- Değişkene **ilk değer ver** (`int yas{0};`). Okuma başarısız olursa değişkenin içinde ne kalacağını birazdan göreceğiz.

Birden çok değer okumak

Çıkarma operatörü de `<<` gibi zincirlenir. Aşağıdaki program dört değişkeni tek satırda, farklı türlerle okuyor. `string` tipini Bölüm 8'de ayrıntılı göreceksin; şimdilik Bölüm 2'de kullandığın gibi al:

uclu.cpp

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    string ad{""};
    int yas{0};
    double boy{0.0};
    char grup{' '};
    cout << "Ad, yaş, boy (m) ve kan grubu harfi: ";
    cin >> ad >> yas >> boy >> grup;
    cout << ad << " " << yas << " " << boy << " m " << grup << '\n';
    return 0;
}
```

```
Çıktı
Ad, yaş, boy (m) ve kan grubu harfi: Elif
19
1.78
A
Elif 19 1.78 m A
```

Kullanıcı dört değeri **dört ayrı satıra** yazdı; aynı sonuç `Elif 19 1.78 m A` biçiminde tek satırda da çıkardı. Bunun nedeni `>>` operatörünün boşluk kurallarıdır:

1. Okumaya başlamadan önce baştaki **boşluk karakterlerini** (boşluk, sekme, Enter) atlar. Bu karakterlere genel olarak *whitespace* denir.
2. Sonra değişkenin türüne uyan karakterleri okur.
3. Türe **uymayan ilk karakterde** durur. O karakteri tampondan almaz.

Bu yüzden birden çok değeri ister boşlukla, ister Enter'la ayırabilirsin: `cin >> a >> b >> c` için `12 30 7` yazmakla her sayıyı ayrı satıra yazmak aynıdır.

`char` okurken boşluk atlanır. C'deki `scanf("%c")` tuzağını hatırlıyorsan iyi haber: `cin >> harf` de baştaki boşluk ve Enter'ları atlar. Yani `cin >> harf` ile hiçbir zaman "yanlışlıkla Enter'ı okuma" sorunu yaşamazsın. Aynı kural bir sorun da yaratır: Boşluk karakterini `>>` ile okuyamazsın. Boşluk dahil her karakteri okumak istersen `cin.get()` kullanılır, bu bölümde ihtiyacımız olmayacak.

Kalıplı girdi

`15/03/2005` ya da `09:30` gibi girdilerde sayıların arasında ayraç karakterleri vardır. C'de bunu biçim metnine yazardın; `cin`'de ayracı bir `char` değişkenine okursun. `>>` boşluk atladığı için `15 / 03 / 2005` yazılsa da çalışır:

tarih.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int gun{0}, ay{0}, yil{0};
    char ayrac{' '};
    cout << "Doğum tarihin (gg/aa/yyyy): ";
    cin >> gun >> ayrac >> ay >> ayrac >> yil;
    cout << "Gün: " << gun << ", ay: " << ay << ", yıl: " << yil << '\n';
    return 0;
}
```

Çıktı

```
Doğum tarihin (gg/aa/yyyy): 15/03/2005
Gün: 15, ay: 3, yıl: 2005
```

`gun` için `15` okundu, `/` karakteri `ayrac`'a girdi, `ay` için `03` okunup `3` oldu (sayı olarak baştaki sıfırın anlamı yoktur), ikinci `/` yine `ayrac`'a, sonra `yil` okundu. Programın hangi ayracı okuduğunu **denetlemedik**; `15-03-2005` yazılırsa da çalışır. Ayracı denetlemek `if` ister (Bölüm 4). Kalıbın başındaki `0`'ı yazdırmak için `setfill` ve `setw` kullanman gerekir; birazdan göreceksin.

Girdi tamponu

Klavyede yazdığın karakterler programa **tek tek** gitmez. İşletim sistemi yazdıklarını bir **tamponda** biriktirir ve sen **Enter**'a basınca satırı programa verir. Enter'a bastığında tampona `'\n'` (yeni satır) karakteri de eklenir. `cin >> değişken` bu tampondan **ihtiyacı kadarını** alır; geri kalanı tamponda bekler.

```
Klavyede yazdığın:      2  0  [Enter]
Tamponun içi:          '2' '0' '\n'

cin >> yas okur:        '2' '0'  → yas = 20
Tamponda kalan:                  '\n'
```

`>>` sayıyı okuyunca durur; Enter'ın bıraktığı `'\n'` tamponda **kalır**. Bir sonraki `>>` için sorun değildir, çünkü `>>` baştaki boşluk ve Enter'ları atlar. Ama bir sonraki okuma **satır okuyan** `getline` ise sorun çıkar. Önce bunun neden ayrı bir iş olduğunu görelim.

Bir sözcük mü, bir satır mı?

`cin >> ad` ile `string` okuyunca `>>` ilk boşlukta durur. Kullanıcı `Ayşe Nur` yazarsa `ad` yalnız `Ayşe` olur; `Nur` tamponda bekler ve programın devamındaki bir `cin >> soyad` kullanıcıya sormadan onu alır. Boşluklu bir metni bütün hâlinde almak için `>>` yerine **`getline`** gerekir.

3.3 getline ile satır okumak

`getline(cin, değişken)`, tampondan **Enter'a kadar olan her şeyi** (boşluklar dahil) `string` değişkene koyar ve Enter karakterini tampondan alıp atar:

tamAd.cpp

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    string tamAd{""};
    cout << "Ad soyad: ";
    getline(cin, tamAd);
    cout << "Merhaba, " << tamAd << "!\n";
    return 0;
}
```

Çıktı

```
Ad soyad: Ayşe Nur Kaya
Merhaba, Ayşe Nur Kaya!
```

`getline` bir fonksiyondur, `string` ile birlikte `<string>` başlığından gelir. Sonucu yazacağı değişkeni ikinci argüman olarak alır. Ad-soyad, adres, cümle gibi boşluk içerebilen her girdi için kullanılır.

cin >> sonrası getline tuzağı

Şimdi iki okumayı birleştirelim: önce sayı, sonra satır. Bu, öğrencilerin en sık takıldığı noktalardan biridir:

getlineTuzak.cpp

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    int yas{0};
    string ad{"(okunmadı)"};
    cout << "Yaşın: ";
    cin >> yas;
    cout << "Ad soyad: ";
    getline(cin, ad);
    cout << "yas = " << yas << ", ad = [" << ad << "]\n";
    return 0;
}
```

Çıktı

```
Yaşın: 20
Ad soyad: yas = 20, ad = []
```

Kullanıcı `20` yazıp Enter'a bastı; program `Ad soyad:` sorusunu yazdı ama adın girilmesini **beklemeden** bitti ve `ad` **boş** çıktı. İzleyelim: Enter'dan sonra tamponda `20` ve `'\n'` var. `cin >> yas`, `20`'yi okuyup durur; `'\n'` tamponda kalır. `getline` tampondan bir satır ister, ilk karakter zaten `'\n'`'dir; **boş bir satır** okur ve hemen döner. Kullanıcı `Ayşe Nur` yazmaya fırsat bile bulamaz. Programı kendi bilgisayarında çalıştırırsan `Ad soyad:` sorusunda durmadığını görürsün.

Çözüm: `getline`'dan önce tamponda kalan Enter'ı **yok say**. Bunu `cin.ignore()` yapar:

getlineCozum.cpp

```cpp
#include <iostream>
#include <limits>
#include <string>

using namespace std;

int main() {
    int yas{0};
    string ad{""};
    cout << "Yaşın: ";
    cin >> yas;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    cout << "Ad soyad: ";
    getline(cin, ad);
    cout << "yas = " << yas << ", ad = [" << ad << "]\n";
    return 0;
}
```

Çıktı

```
Yaşın: 20
Ad soyad: Ayşe Nur
yas = 20, ad = [Ayşe Nur]
```

`cin.ignore(n, ayrac)`, tampondan en fazla `n` karakteri atar; ama `ayrac` karakteriyle karşılaşırsa onu da atıp durur. Yani satırın kalanını, Enter dahil, siler. Buradaki `numeric_limits<streamsize>::max()` "mümkün olan en büyük sayı" demektir (`<limits>` başlığı gerekir); pratikte "ne kadar olursa olsun satırın sonuna kadar" anlamına gelir. Bu satırı bir kalıp olarak ezberle.

`cin.ignore()` parametresiz yazılırsa **yalnız 1 karakter** atar. Kullanıcı sayıdan hemen sonra Enter'a basarsa yetişir, ama sayının ardına yanlışlıkla boşluk (`"20 "`) bırakırsa atılan boşluk olur, `'\n'` tamponda kalır ve `getline` yine boş satır okur. Uzun biçim her durumda çalışır, onu tercih et.

Sıra	Tuzak var mı?
`cin >>` sonra `cin >>`	Hayır, `>>` baştaki boşluk ve Enter'ı atlar
`getline` sonra `getline`	Hayır, `getline` Enter'ı kendisi tüketir
`getline` sonra `cin >>`	Hayır, `>>` baştaki Enter'ı atlar
`cin >>` sonra `getline`	**Evet:** araya `cin.ignore(...)` koy

> **Sınavda Çıkar**
>
> "Aşağıdaki programda `getline` neden beklemeden geçer?" sorusu klasiktir. Cevap: `cin >> yas` Enter'ın bıraktığı `'\n'` karakterini tamponda bırakır; `getline` bunu boş bir satır olarak okur. Çözüm `cin >> yas;` ile `getline(...)` arasına `cin.ignore(numeric_limits<streamsize>::max(), '\n');` koymaktır. Cevabında "tamponda kalan Enter" ifadesi geçsin.

3.4 Biçimli çıktı: iomanip

`cout` sayıları kendi bildiği gibi yazar: ondalık sayıyı kaç basamakla yazacağına, sütun genişliğine, hizalamaya o karar verir. Fiş, tablo, not çizelgesi gibi çıktılarda bunları sen ayarlamak istersin. Bunun için araya **manipülatör** (manipulator) denen özel değerler koyarsın. Manipülatör, `cout`'a yazılan ama ekrana bir şey yazmayan, akışın ayarını değiştiren bir değerdir; `endl` de bir manipülatördür. Parametre alanlar (`setw`, `setprecision`, `setfill`) `<iomanip>` başlığındadır; parametresizler (`fixed`, `left`, `right`) `<iostream>` ile gelir.

Varsayılan hâl: 6 anlamlı basamak

Hiçbir şey ayarlamazsan `cout` bir ondalıklı sayıyı **en çok 6 anlamlı basamakla** yazar. *Anlamlı basamak*, sayının soldan ilk sıfırdan farklı rakamından başlayarak sayılan rakamlardır; noktadan sonraki hane sayısı **değildir**. Gereksiz sıfırlar yazılmaz, çok büyük ya da çok küçük sayılar bilimsel gösterime geçer:

varsayilanBasamak.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    cout << 3.14159265 << '\n';
    cout << 100.0 << '\n';
    cout << 2.50 << '\n';
    cout << 1.0 / 3 << '\n';
    cout << 123456.789 << '\n';
    cout << 1234567.0 << '\n';
    cout << 0.00001 << '\n';
    cout << 0.1 + 0.2 << '\n';
    return 0;
}
```

Çıktı

```
3.14159
100
2.5
0.333333
123457
1.23457e+06
1e-05
0.3
```

Çıktıdan çıkan dersler:

- `3.14159265` yedi haneli bir sayıydı, 6 anlamlı basamakla `3.14159` yazıldı. Sayı **kesilmedi**, yuvarlanarak yazıldı; bellekteki değer değişmedi.
- `100.0` ve `2.50` için `100` ve `2.5` yazıldı. Sondaki sıfırlar atılır; `100.0` yazdırınca `100` görmek şaşırtıcı değildir.
- `123456.789`, 6 basamağa sığması için `123457` olarak yuvarlandı; ondalık kısım tamamen kayboldu. `1234567.0` ise 6 basamağa sığmadığı için `1.23457e+06` yani 1.23457×10^6 biçimine geçti. `0.00001` de `1e-05` oldu.
- `0.1 + 0.2` ikilik tabanda tam `0.3` etmez, ama 6 basamakla yazılınca `0.3` görünür. Gerçek değeri görmek için daha çok basamak istemen gerekir; bunu birazdan yapacağız.

> **Sınavda Çıkar**
>
> "`cout << 100.0;` ne yazar?" sorusunun cevabı `100`'dür, `100.0` değil. "Varsayılan hâlde kaç hane yazılır?" sorusunun cevabı "6 **anlamlı basamak**"tır, "noktadan sonra 6 hane" değil. Bu ikisini karıştırmak sınavda en sık düşülen tuzaktır. (C'deki `%f` ise noktadan sonra 6 hane yazardı; `cout` farklıdır.)

fixed ve setprecision

Hane sayısını `setprecision(n)` ayarlar. Ne anlama geldiği `fixed` manipülatörünün açık olup olmamasına bağlıdır:

Ayar	`setprecision(n)` ne demek?	`3.14159265` için `n = 3`
`fixed` yok (varsayılan)	Toplam **anlamlı basamak** sayısı	`3.14`
`fixed` var	Noktadan sonraki **hane** sayısı	`3.142`

Para, ortalama, VKİ gibi "iki ondalık" isteyen her yerde ikisini birlikte kullanırsın:

fixedOrnek.cpp

```cpp
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    double pi{3.14159265};
    cout << setprecision(3) << pi << '\n';
    cout << fixed << setprecision(3) << pi << '\n';
    cout << pi * 1000 << '\n';
    cout << 100.0 << '\n';
    cout << defaultfloat << setprecision(6) << pi << '\n';
    return 0;
}
```

Çıktı

```
3.14
3.142
3141.593
100.000
3.14159
```

Satır satır:

1. `fixed` olmadan `setprecision(3)`: 3 anlamlı basamak, `3.14`.
2. `fixed` ile `setprecision(3)`: noktadan sonra 3 hane, `3.142` (yuvarlandı).
3. `pi * 1000` = `3141.59265`; hâlâ `fixed` açık, noktadan sonra 3 hane: `3141.593`.
4. `100.0` bile üç ondalıkla yazıldı: `100.000`. `fixed` sayıyı kısaltmaz, gereken sıfırları ekler.
5. `defaultfloat` varsayılan biçime döner, `setprecision(6)` varsayılan basamak sayısını geri getirir: `3.14159`.

Bu manipülatörlerin ortak özelliği **kalıcı** olmalarıdır: Bir kez yazdın mı, sen değiştirene kadar bütün sonraki ondalık çıktılar için geçerlidir. Bu yüzden `cout << fixed << setprecision(2);` satırını programın başında bir kez yazman yeter. Kalıcılığın tersi olan tuzağı birazdan `setw`'de göreceksin.

Yuvarlama

`setprecision` ile `fixed` sayıyı gösterirken **yuvarlar**. Tamsayıya dönüştürmek (`static_cast<int>`) ise ondalık kısmı **keser** (Bölüm 2). İkisinin farkına bak:

yuvarlama.cpp

```cpp
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    cout << fixed << setprecision(2);
    cout << 3.999 << '\n';
    cout << 2.675 << '\n';
    cout << 1.005 << '\n';
    cout << setprecision(0) << 2.7 << '\n';
    cout << static_cast<int>(2.7) << '\n';
    return 0;
}
```

Çıktı

```
4.00
2.67
1.00
3
2
```

`3.999` iki haneye yuvarlanınca `4.00` oldu. `setprecision(0)` ile `2.7` yuvarlanıp `3` yazıldı, `static_cast<int>` ise keserek `2` verdi. Son iki dönüşüm farklı işlerdir.

İlginç olan `2.675` ve `1.005`: elle yuvarlasan `2.68` ve `1.01` bulurdun, ekranda ise `2.67` ve `1.00` yazıyor. Sebep ondalık sayıların bellekte ikilik tabanda saklanmasıdır: `2.675` tam saklanamaz, bellekteki gerçek değeri `2.67499999...` biçimindedir ve `cout` bu gerçek değeri yuvarlar. Bu bir hata değil, ondalık kesirlerin doğal sonucudur (Bölüm 2'deki `0.1 + 0.2` olayının akrabası).

setw: alan genişliği

`setw(n)` (*set width*), bir sonraki değerin **en az** `n` karakterlik bir alana yazılmasını sağlar. Değer kısaysa kalan yer dolgu karakteriyle (varsayılan: boşluk) doldurulur. Dolgunun hangi yana konacağını `left` ve `right` belirler: varsayılan **sağa yaslıdır**. Köşeli parantezler boşlukları görmeni sağlar:

setwOrnek.cpp

```cpp
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    cout << "[" << setw(6) << 42 << "]\n";
    cout << "[" << left << setw(6) << 42 << "]\n";
    cout << "[" << right << setw(6) << "Ali" << "]\n";
    cout << "[" << setw(2) << 12345 << "]\n";
    cout << "[" << setw(6) << 7 << 8 << "]\n";
    return 0;
}
```

Çıktı
```
[    42]
[42    ]
[   Ali]
[12345]
[     78]
```

1. `setw(6)` ile `42`: iki karakter, önüne **dört boşluk** konur (sağa yaslı).
2. `left`: sayıyı sola yaslar, boşluklar sağa gider.
3. `right`: yine sağa yaslı. Metinler de aynı kuralla çalışır.
4. `setw(2)` ile `12345`: genişlik **en az** demektir, değer sığmıyorsa **kesilmez**.
5. `setw(6)` yalnızca `7` için geçerli oldu; `8` genişliksiz yazıldı. Aşağıdaki kutuya bak.

> **Sık Yapılan Hata: setw'nin kalıcı olduğunu sanmak**
>
> `left`, `right`, `fixed`, `setprecision` ve `setfill` **kalıcıdır**; `setw` ise **yalnızca bir sonraki değer** için geçerlidir. Şu satırın sütunları hizalamayacağını beklemelisin:
>
> ```
> cout << setw(8) << ad << puan << notu << '\n';
> ```
>
> Burada yalnız `ad` 8 karakterlik alana yazılır; `puan` ile `notu` bitişik gelir. Her sütundan önce `setw` yazmak zorundasın: `cout << setw(8) << ad << setw(6) << puan << setw(6) << notu`.

setfill: dolgu karakteri

`setfill(c)` boşluk yerine hangi karakterle dolduracağını belirler. En sık kullanımı, sayıları başına sıfır koyarak sabit genişliğe tamamlamaktır (`09:05`). Menülerdeki noktalı satırlar gibi kullanımları da vardır:

dolgu.cpp
```
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    cout << setfill('0') << setw(5) << 42 << '\n';
    cout << setfill('*') << setw(8) << "Ali" << '\n';
    cout << setfill('.') << left << setw(12) << "Cay" << right
         << setw(6) << 15 << '\n';
    cout << setw(4) << 7 << '\n';
    cout << setfill(' ') << setw(4) << 7 << '\n';
    return 0;
}
```

Çıktı
```
00042
*****Ali
Cay.........15
...7
   7
```

Üçüncü satır, menülerde görülen "ürün adı ..... fiyat" görünümünü verir. Dördüncü satır `setfill` kalıcılığının sonucudur: Nokta dolgu hâlâ açık olduğundan `7` sayısı `...7` yazıldı. Son satırda `setfill(' ')` ile varsayılan boşluğa döndük. **Kural: `setfill`'i kullandıktan sonra eski haline getir.**

Hepsi bir arada: not tablosu

Genişlik, hizalama ve hassasiyeti birleştirerek düzgün bir tablo yazdıralım. İsimler sola, sayılar sağa yaslı, ortalama iki ondalıklı:

notTablosu.cpp

```cpp
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    cout << fixed << setprecision(2);
    cout << left << setw(10) << "Ad" << right << setw(6) << "Vize"
         << setw(7) << "Final" << setw(8) << "Ort." << '\n';
    cout << "--------------------------------\n";

    cout << left << setw(10) << "Ali" << right << setw(6) << 70
         << setw(7) << 85 << setw(8) << 0.4 * 70 + 0.6 * 85 << '\n';
    cout << left << setw(10) << "Zeynep" << right << setw(6) << 95
         << setw(7) << 90 << setw(8) << 0.4 * 95 + 0.6 * 90 << '\n';
    cout << left << setw(10) << "Mehmet" << right << setw(6) << 55
         << setw(7) << 62 << setw(8) << 0.4 * 55 + 0.6 * 62 << '\n';
    return 0;
}
```

Çıktı

```
Ad          Vize  Final    Ort.
--------------------------------
Ali           70     85   79.00
Zeynep        95     90   92.00
Mehmet        55     62   59.20
```

Tablonun hizalı çıkmasının sırrı, başlık ve veri satırlarında **aynı genişlikleri** (10, 6, 7, 8) kullanmaktır. Her satırın başında `left` yazıp isimden sonra `right`'a dönmek zorundayız, çünkü `left` kalıcıdır. `0.4 * 70 + 0.6 * 85` gibi bir ifadeyi `cout`'a doğrudan verebilirsin: Önce hesaplanır, sonra `setw(8)` ve `fixed` ayarlarıyla yazılır. Tamsayılar (70, 85) `fixed`'den etkilenmez.

Tekrarlayan satırlar sana "bunu bir döngüyle yazmalıyım" dedirtiyorsa doğru yoldasın; döngüleri Bölüm 5'te öğreneceksin.

Diğer sık kullanılan manipülatörler

Aşağıdakiler de kalıcıdır. Çoğunu ilerleyen bölümlerde ihtiyaç oldukça kullanacaksın:

Manipülatör	Etkisi
`boolalpha` / `noboolalpha`	`bool` değerleri `1`/`0` yerine `true`/`false` yazar
`showpos` / `noshowpos`	Pozitif sayılara da `+` koyar
`hex`, `oct`, `dec`	Tamsayıları onaltılık, sekizlik, onluk tabanda yazar
`showbase` / `noshowbase`	`hex` ve `oct` için `0x`, `0` ön ekini ekler
`uppercase` / `nouppercase`	Onaltılık harfleri ve bilimsel gösterimdeki `e`'yi büyük yazar
`scientific`	Her zaman bilimsel gösterim (`3.141593e+00`)

`5 > 3` gibi bir `bool` ifadeyi `boolalpha` olmadan yazdırırsan `1`, açıkken `true` görürsün. `255` onaltılık tabanda `ff`'dir ($15 \times 16 + 15$); `showbase` ile `0xff`, `uppercase` ile `0XFF` olur. Bu ayarlar kalıcı olduğu için işin bitince geri almak (`dec`, `noshowbase`, `noboolalpha` ...) iyi alışkanlıktır.

`setw`, `setprecision` ve `setfill` yalnız `<iomanip>` başlığındadır; unutursan derleyici `'setw' was not declared in this scope` gibi bir hata verir (macOS'taki clang `use of undeclared identifier` der). Kural: parantezle argüman alan manipülatör görüyorsan `<iomanip>` ekle.

3.5 Hatalı girdi ve akışın kilitlenmesi

Kullanıcı her zaman beklediğin şeyi yazmaz. Yaş sorduğunda `yirmi` yazabilir, bir sayı yerine yanlışlıkla harf basabilir. Bu bölümün en önemli konusu, bu durumda **ne olduğunu** bilmektir. Önce değişkenin ilk değerini `-1` yapıp `abc` yazan bir kullanıcıyı izleyelim:

harfGirdim.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int sayi{-1};
    cout << "Bir sayı gir: ";
    cin >> sayi;
    cout << "sayi = " << sayi << ", cin.fail() = " << cin.fail() << '\n';
    return 0;
}
```

Çıktı

```
Bir sayı gir: abc
sayi = 0, cin.fail() = 1
```

`cin >> sayi` bir rakam bekliyordu, `a` rakam değil. Okuma **başarısız oldu**. Program çökmedi, uyarı vermedi, hata mesajı basmadı; `cin` kendi içinde bir hata bayrağını açtı ve `cin.fail()` fonksiyonu bu bayrağı okuyup `1` (yani `true`) döndürdü. `bool` değer `cout`'a `1/0` olarak gider.

Dikkat edilecek iki nokta:

- **`sayi` değişti: `0` oldu.** C++11'den beri standart, geçersiz girdide değişkene `0` yazılacağını söyler. (Eski kitaplarda "değişken olduğu gibi kalır" yazar; modern derleyicilerde öyle değildir.) Ama `0` zaten geçerli bir girdi olabilir, bu yüzden `sayi == 0` kontrolüyle hatayı **anlayamazsın**. Doğru kontrol `cin.fail()`'dir.
- **`abc` tamponda kaldı.** `cin` hiçbir şeyi tüketmedi. Bu, birazdan göreceğimiz sorunun kaynağıdır.

Akış durumu

Her akış, başına neler geldiğini birkaç işaretle (bayrak) tutar. Bunlara **akış durumu** (stream state) denir ve fonksiyonlarla sorulur:

Fonksiyon	`true` ise anlamı
`cin.good()`	Her şey yolunda, okuma yapılabilir
`cin.fail()`	Okuma türe uymadığı için başarısız oldu (örneğin sayı yerine harf)
`cin.eof()`	Girdi bitti (Linux ve macOS'ta Ctrl+D, Windows'ta Ctrl+Z ve Enter)
`cin.bad()`	Akışın kendisi bozuldu (donanım sorunu gibi, nadir)

İşin özü: bir okuma başarısız olunca `failbit` bayrağı açılır. Girdi bittiğinde (`eof`) okunacak değer bulunamazsa okuma yine başarısız sayılır ve `fail()` de `true` olur.

Çıkarma ifadesi akışın kendisini döndürdüğü için başarıyı doğrudan şöyle de sorabilirsin: `if (cin >> yas) { ... }` "okuma başarılıysa" demektir. `if`'i Bölüm 4'te öğreneceksin; şimdilik "koşul doğruysa süslü parantez içini çalıştır" diye oku.

Kilitlenme

Akışın hata durumuna düşmesinin asıl sonucu, **sonraki bütün okumaların sessizce atlanmasıdır**. `failbit` açık olduğu sürece `cin >> değişken` hiçbir şey yapmaz, değişkene dokunmaz:

kilitlenme.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int a{-1};
    int b{-1};
    cin >> a;
    cin >> b;
    cout << a << " " << b << " " << cin.fail() << '\n';
    return 0;
}
```

Çıktı

```
abc 5
0 -1 1
```

Kullanıcı `abc`'nin ardından geçerli bir `5` de yazmıştı, ama `b` hâlâ `-1`. İlk okuma `abc` yüzünden başarısız oldu ve `a` `0` oldu; akış hata durumuna girdiği için ikinci `cin >> b` **hiç çalışmadı**: değişken `-1` olarak kaldı. Bu tür programlarda gördüğün "program sorular peş peşe atlanıyor" şikâyetinin nedeni budur.

Onarım: clear ve ignore

Akışı eski haline getirmek için iki şey yapman gerekir ve **sırası önemlidir**:

1. `cin.clear()`: hata bayraklarını temizler (`failbit` kapanır). Akış yine okumaya hazırdır.
2. `cin.ignore(numeric_limits<streamsize>::max(), '\n')`: tamponda kalan **bozuk girdiyi** (`abc`) satırın sonuna kadar siler. Aynı kalıbı `getline` bölümünde görmüştün.

onarim.cpp

```cpp
#include <iostream>
#include <limits>

using namespace std;

int main() {
    int yas{0};
    cout << "Yaş: ";
    cin >> yas;
    if (cin.fail()) {
        cout << "Geçersiz girdi, satır atlandı.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "Yaş: ";
        cin >> yas;
    }
    cout << "Okunan yaş: " << yas << '\n';
    return 0;
}
```

Çıktı

```
Yaş: abc
Geçersiz girdi, satır atlandı.
Yaş: 25
Okunan yaş: 25
```

İlk girdi `abc` idi: `cin.fail()` `true` oldu, mesaj yazıldı, akış onarıldı ve ikinci denemede `25` okundu. Sırayı bozarsan olacaklar:

- Yalnız `clear()` yazarsan akış okumaya hazır olur ama `abc` tamponda durur; bir sonraki `>>` aynı yerde yine takılır ve akış yeniden kilitlenir.
- `ignore()`'u `clear()`'dan **önce** yazarsan `ignore` bir şey yapmaz, çünkü hata durumundaki akışta hiçbir işlem çalışmaz.

Geçerli bir sayı girilene kadar soruyu tekrar tekrar sormak için bu kalıbı bir **döngü** içine koymak gerekir. Onu Bölüm 5'te yapacağız; şimdilik tek seferlik onarımı bil.

Yarım okunan girdi

Hata her zaman `fail()` ile görünmez. `cin >> sayi` türe uyan kısmı okur ve uymayan ilk karakterde durur; kalan **tamponda** beklemeye devam eder. Kullanıcı `int` beklerken `3.9` yazarsa:

ondalikGirdi.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int tamsayi{0};
    char sonraki{' '};
    cin >> tamsayi >> sonraki;
    cout << "tamsayi = " << tamsayi << ", sonraki = " << sonraki << '\n';
    cout << "cin.fail() = " << cin.fail() << '\n';
    return 0;
}
```

```
Çıktı
3.9
tamsayi = 3, sonraki = .
cin.fail() = 0
```

`int` değişkenine `3` okundu ve okuma **hatasız** bitti: `3` bir tamsayıdır, `.` ile durdu. `.9` tamponda kaldı ve sıradaki okuma (burada `char`) `.` karakterini aldı. Program çökmedi, `fail()` de `false`; sadece veri kullanıcının istediği gibi değil. Aynı şekilde `12abc` yazılırsa `12` okunur, `abc` tamponda kalır.

Çok büyük sayılarda da benzer bir durum vardır. `int` değişkene `99999999999` (tipin sığabileceğinden büyük) yazılırsa `int`'in en büyük değeri olan `2147483647` atanır ve `failbit` açılır.

> **Sık Yapılan Hata: Ondalık sayıda virgül**
>
> Türkçe klavyede sayı tuş takımındaki ondalık tuş virgül (`,`) yazabilir, ama C++ ondalık ayırıcı olarak **noktayı** bekler. Kullanıcı `1,78` yazarsa `double`'a `1` okunur, `,78` tamponda kalır; tıpkı `3.9` örneğindeki gibi hata vermeden yanlış veri alırsın. Programlarına ondalık sayıyı `1.78` biçiminde gir. Çıktıda da `cout` her zaman nokta yazar (`1.78`).

3.6 Hata mesajları için cerr

Programın iki çeşit şey yazması gerekir: işin sonucu ve bir sorun olduğunu bildiren mesajlar. İkisi aynı yere, ekrana gidebilir ama ayrı akışlar olarak ayırmak işine yarar. C++ buna hazır bir akış verir: `cerr` (*character error*), standart **hata akışı**dır.

- `cout` normal çıktı içindir. Tamponludur.
- `cerr` hata mesajları içindir. Her yazımdan sonra tamponu hemen boşaltır: yazdığın mesaj gecikmeden görünür. Bu yüzden program çökse bile `cerr`'ın mesajı kaybolmaz.

Ekranda ikisi birbirinden ayırt edilemez, fark programı çalıştırırken **yönlendirince** ortaya çıkar:

hataAkisi.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    cout << "Sonuç: 42\n";
    cerr << "Uyarı: girdi dosyası bulunamadı\n";
    return 0;
}
```

```
Çıktı
Sonuç: 42
Uyarı: girdi dosyası bulunamadı
```

Komut satırında `>` normal çıktıyı, `2>` hata akışını bir dosyaya yönlendirir. Windows (cmd, PowerShell) ve Linux/macOS aynı yazımı kullanır; yalnız program adı Bölüm 1'deki gibi değişir (Linux/macOS'ta `./hataAkisi`, PowerShell'de `.\hataAkisi.exe`):

```
hataAkisi.exe > sonuc.txt
hataAkisi.exe 2> hatalar.txt
```

İlk komutta ekranda yalnız uyarı görünür, `sonuc.txt` dosyasında `Sonuç: 42` yazar. İkincisinde ise tersi olur. Programın kullanıcıya gösterdiği sonuçları dosyaya alırken hata mesajlarının dosyaya karışmaması bu yüzden değerlidir. Bu bölümdeki programlarda hata mesajlarını `cerr`'a, geri kalanı `cout`'a yazmak iyi bir alışkanlıktır.

3.7 Örnek uygulama: vücut kitle indeksi

Vücut kitle indeksi (VKİ), kiloyu boyun karesine bölerek bulunur: `kilo / (boy * boy)`; kilo kilogram, boy metre cinsindendir. Kilo ve boy kesirli olabileceği için `double` kullanıyoruz; sonucu iki ondalıkla yazdırıyoruz:

vki.cpp

```cpp
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    double kilo{0.0};
    double boy{0.0};
    cout << "Kilon (kg) ve boyun (m): ";
    cin >> kilo >> boy;
    double vki{kilo / (boy * boy)};
    cout << fixed << setprecision(2);
    cout << "Vücut kitle indeksi: " << vki << '\n';
    return 0;
}
```

Çıktı

```
Kilon (kg) ve boyun (m): 72.5 1.78
Vücut kitle indeksi: 22.88
```

`72.5 / (1.78 × 1.78)` = `72.5 / 3.1684` ≈ `22.88`. Sonucu "zayıf, normal, fazla kilolu" gibi sınıflandırmak için `if` gerekir; bunu Bölüm 4'te göreceksin.

3.8 Türkçe karakterler ve Windows konsolu

Kitabın başında sözünü verdiğimiz soruyu cevaplayalım: `cout << "Merhaba Dünya!\n";` neden Windows'ta `Merhaba D╝nya!` gibi çıkabilir? Program doğru çalışıyor; sorun **karakter kodlamasında** (character encoding). Üç ayrı parça vardır ve üçü aynı kodlamada buluşmalı:

1. **Kaynak dosya.** Editörün `.cpp` dosyasını diske hangi kodlamayla yazdığı. Biz **UTF-8** kullanıyoruz. UTF-8'de `a`, `b`, `1` gibi karakterler 1 bayt, `ç`, `ğ`, `ı`, `ö`, `ş`, `ü` gibi Türkçe harflerin her biri **2 bayttır**.
2. **g++.** String'in içindeki baytları olduğu gibi çalıştırılabilir dosyaya koyar. Dosya UTF-8 ise programın ürettiği çıktı da UTF-8 baytlarıdır.
3. **Konsol.** Gelen baytları ekranda hangi harfe çevireceğini kendi **kod sayfası**na (code page) göre seçer. Türkçe Windows'ta varsayılan kod sayfası UTF-8 değildir. Türkçe bir harfin iki UTF-8 baytı konsolda iki ayrı garip karakter olarak görünür; `ü` yerine `Ã¼` ya da `╝` gibi şeyler görürsün.

Yani bozulma Türkçe karakterleri yazarken değil, **gösterirken** olur; program bellekte doğru baytları üretiyordur. Linux ve macOS terminalleri varsayılan olarak UTF-8 kullandığı için orada sorun çıkmaz.

Çözüm 1: Konsolu UTF-8 yap. Programı çalıştırmadan önce aynı terminalde şunu yaz:

```
chcp 65001
```

`chcp` (*change code page*), konsolun kod sayfasını değiştirir; `65001` UTF-8'dir. Komut yalnızca **o terminal penceresi** için geçerlidir, terminali kapatıp açınca eski haline döner; her yeni terminalde yeniden yazman gerekir. Ekranda yazılar hâlâ bozuksa terminalin yazı tipini (font) değiştirmeyi dene; bazı eski yazı tipleri Türkçe harfleri çizemez. Aynı kod sayfası klavyeden Türkçe karakter girdisi için de geçerlidir.

Çözüm 2: Kaynak dosya UTF-8 olsun. VS Code dosyaları varsayılan olarak UTF-8 kaydeder ve pencerenin **sağ alt köşesindeki alt bilgi çubuğunda** kodlamayı gösterir (`UTF-8`). Orada `Windows 1254` gibi başka bir şey yazıyorsa dosya başka kodlamayla kaydedilmiştir: O yazıya tıkla, **Save with Encoding** seç, **UTF-8**'i seç. Dev-C++ gibi bazı eski editörler dosyayı Windows-1254 ile kaydedebilir; o durumda `chcp 65001` yetmez, çünkü baytlar UTF-8 değildir.

Her platformda geçerli bir ayrıntı daha var: Türkçe karakterli string'leri **hizalarken** dikkat etmelisin. `setw` genişliği **bayt** sayar, harf saymaz:

turkceGenislik.cpp

```cpp
#include <iostream>
#include <iomanip>
#include <string>

using namespace std;

int main() {
    string ali{"Ali"};
    string ayse{"Ayşe"};
    cout << "[" << left << setw(8) << ali << "]\n";
    cout << "[" << left << setw(8) << ayse << "]\n";
    cout << ali.size() << " " << ayse.size() << '\n';
    return 0;
}
```

Çıktı

```
[Ali     ]
[Ayşe    ]
3 5
```

`Ali` 3 bayt olduğu için 5 boşlukla tamamlandı ve parantez 10. sütunda kapandı. `Ayşe` ise `ş` yüzünden 5 bayttır (4 harf), 3 boşluk eklendi; ekranda bir karakter **eksik** görünüyor. Son satırdaki `size()` (Bölüm 8'de göreceksin) string'in **bayt** sayısını verir: `Ali` için 3, `Ayşe` için 5. Bu yüzden Türkçe karakterli metinleri hizalarken genişliği bayt sayısına göre değil, ekranda kaç harf göründüğüne göre hesaplaman gerekir.

Bir de tek tırnak tuzağı: `'ş'` gibi tek bir `char` sabiti yazamazsın, çünkü `ş` 2 bayttır ve `char` 1 baytlıktır; g++ `char c = 'ş';` yazımını uyarıyla (`character not encodable in a single execution character code unit`) kabul eder ve değeri bozar; kitabın kullandığı `char c{'ş'};` yazımında ise daralma hatası (`narrowing conversion`) verip derlemez. Apple clang ikisini de hatayla reddeder (`character too large for enclosing character literal type`). Türkçe karakter yalnız **string** içinde (çift tırnakta) kullanılır. Ayrıntısını Bölüm 8'de göreceksin.

Bölüm Özeti – Bilmen Gerekenler

- `cout` zincirlenir: `cout << a << b`. Satır sonu için `'\n'` kullan; `endl` satır sonunu yazar ve ayrıca tamponu boşaltır (yavaştır). `\\`, `\"`, `\t` kaçış dizileridir.
- `<<`, `+` ve `*`'dan sonra işlenir; ama `>`, `==`, `&&` gibi karşılaştırma ve mantıksal operatörlerden **önce** işlenir. Bunları `cout` içinde parantezle yaz.

- `cin >> x`: biçim belirteci ve `&` yoktur, tür değişkenden anlaşılır. `>>` baştaki boşluk ve Enter'ları atlar, türe uymayan ilk karakterde durur; `char` okurken de boşluk atlanır. Tampondaki artık karakterler bir sonraki okumaya kalır.
- `cin >> ad` yalnız ilk sözcüğü alır. Boşluklu satır için `getline(cin, ad)`; `cin >>`'dan sonra `getline` gelirse araya `cin.ignore(numeric_limits<streamsize>::max(), '\n')` koy (`<limits>` gerekir).
- Varsayılan ondalık çıktı **6 anlamlı basamaktır**: `100.0` → `100`, `1234567.0` → `1.23457e+06`. `fixed` olmadan `setprecision(n)` toplam basamağı, `fixed` ile noktadan sonraki haneyi belirler. İkisi kalıcıdır; `defaultfloat` geri döner.
- `setw(n)` yalnız **bir sonraki** değeri `n` genişliğe getirir ve sığmazsa kesmez; varsayılan sağa yaslıdır. `left`/`right` ve `setfill(c)` kalıcıdır. `setw`, `setprecision`, `setfill` için `<iomanip>` gerekir.
- `fixed << setprecision(2)` yuvarlar (`2.675` → `2.67`, ikilik taban yüzünden); `static_cast<int>` keser.
- Sayı beklerken harf girilirse `cin.fail()` `true` olur, değişken `0` olur, `abc` tamponda kalır ve sonraki bütün okumalar atlanır. Onarım sırası: önce `cin.clear()`, sonra `cin.ignore(...)`. `3.9` gibi girdiler `int`'e `3` olarak, hatasız okunur.
- `cerr` her yazımdan sonra boşaltılan hata akışıdır; `2>` ile ayrı dosyaya yönlendirilebilir.
- Türkçe karakterler UTF-8'de 2 bayttır, `setw` bayt sayar; hizalamada bunu hesaba kat. Windows konsolunda bozuk görünürse `chcp 65001`; VS Code alt bilgisinde kodlamanın UTF-8 olduğunu kontrol et.

Sorular

A) Çoktan Seçmeli

1 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    cout << setprecision(2) << 3.14159 << '\n';
    return 0;
}
```

(A) `3.14` (B) `3` (C) `3.1` (D) `3.14159` (E) `3.10`

2 Bir program önce `int yas` değişkenini `cin >> yas;` ile, hemen ardından `string ad` değişkenini `getline(cin, ad);` ile okuyor. Kullanıcı ilk satıra `20`, ikinci satıra `Ayşe Nur` yazıp her birinin ardından Enter'a basıyor. `ad` değişkeni hakkında hangisi doğrudur?

(A) Program ad için beklemeden hata verip çöker.

(B) `ad` değişkeni `Ayşe Nur` olur.

(C) Program derlenmez, çünkü `cin` ile `getline` aynı programda kullanılamaz.

(D) `ad` değişkeni `20` olur.

(E) `ad` boş kalır, çünkü `getline` sayıdan sonra tamponda kalan Enter'ı satır olarak okuyup hemen döner.

3 Aşağıdaki program ekrana ne yazdırır? (Köşeli parantezler çıktının parçasıdır.)

```
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    cout << "[" << setw(5) << 1 << 2 << 3 << "]\n";
    return 0;
}
```

(A) `[    123]`

(B) `[    1    2    3]`

(C) `[1    2    3]`

(D) `[123]`

(E) `[12345]`

4 Bir program `int` bekleyen `cin >> sayi;` satırında `abc` girdisini aldı ve akış hata durumuna düştü. Akışı yeniden kullanılabilir hale getirmek için aşağıdakilerden hangisi doğrudur?

(A) Önce `cin.ignore(numeric_limits<streamsize>::max(), '\n');`, sonra `cin.clear();` yazılır.

(B) Önce `cin.clear();`, sonra `cin.ignore(numeric_limits<streamsize>::max(), '\n');` yazılır.

(C) Yalnızca `cin.clear();` yazmak yeterlidir, bozuk girdi kendiliğinden silinir.

(D) Yalnızca `cin.fail();` çağırmak hata durumunu temizler.

(E) Yalnızca `endl` yazmak tamponu temizler.

5 `'\n'` ile `endl` arasındaki fark hangisidir?

(A) İkisi birebir aynıdır, hiçbir fark yoktur.

(B) `endl` yalnız Windows'ta, `'\n'` yalnız Linux'ta çalışır.

(C) `'\n'` ekrana iki karakter yazar, `endl` bir karakter yazar.

(D) İkisi de yeni satıra geçer; `endl` ayrıca çıkış tamponunu boşaltır.

(E) `'\n'` tamponu boşaltır, `endl` boşaltmaz.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır? Köşeli parantezlerin ve çizgilerin içindeki boşlukları dikkatle say.

```cpp
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    cout << fixed << setprecision(1);
    cout << "[" << setw(6) << 3.14159 << "|" << 2.0 << "]\n";
    cout << left << "[" << setw(4) << 7 << "|" << setw(4) << 7 << "]\n";
    cout << setfill('0') << right << setw(4) << 12 << setw(4) << 3 << '\n';
    cout << setw(3) << 5 << 6 << '\n';
    return 0;
}
```

7 Aşağıdaki program çalışırken klavyeden `12 abc 7` yazılıp Enter'a basılıyor. Program ekrana ne yazdırır?

```cpp
#include <iostream>

using namespace std;

int main() {
    int x{-1};
    int y{-1};
    int z{-1};
    cin >> x >> y >> z;
    cout << x << " " << y << " " << z << " " << cin.fail() << '\n';
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    double d{1234.5678};
    cout << d << '\n';
    cout << d * 100 << '\n';
    cout << d * 1000 << '\n';
    cout << fixed << setprecision(1) << d << '\n';
    cout << 5 / 2 << " " << 5 / 3.0 << '\n';
    return 0;
}
```

C) Kod Yazma

9 Kullanıcıdan vize, ödev ve final notunu (üç tamsayı, aralarında boşlukla) okuyan **notKarti** programını yaz. Ortalama `vize * 0.3 + odev * 0.2 + final * 0.5` formülüyle bulunur. Çıktı iki satırlık bir tablo olmalı: ilk üç sütun 6, son sütun 8 karakter genişliğinde ve **sağa yaslı**; ortalama iki ondalıklı. Örnek girdi `60 80 70` için, girdi isteyen yazıdan sonra şu tablo çıkmalı:

```
  Vize  Odev Final    Ort.
    60    80    70   69.00
```

10 Önce bir tarihi `gg/aa/yyyy` biçiminde, sonra bir saati `ss:dd` biçiminde okuyan **tarihSaat** programını yaz. Tarih ve saati `yyyy-aa-gg ss:dd` biçiminde, gün, ay, saat ve dakika **iki haneli** (gerekirse başına sıfır konarak), yıl dört haneli yazdır. Örnek: İlk satıra `7/3/2026`, ikinci satıra `9:5` yazılırsa son satır `2026-03-07 09:05` olmalı.

Çözümler

Cevap anahtarı

1 C	2 E	3 A	4 B	5 D

1 Cevap: C

`fixed` olmadan `setprecision(2)` **toplam anlamlı basamak** sayısını belirler: `3.14159` iki basamakla `3.1` olur. A (`3.14`) `fixed` ile gelen sonuçtur; yani noktadan sonra iki hane isteseydin `cout << fixed << setprecision(2)` yazman gerekirdi. B yanlış, bu yalnızca tek anlamlı basamaktır. D varsayılan hâlin (6 anlamlı basamak) sonucudur. E yanlış: `fixed` olmadan sondaki sıfır yazılmaz, sonuç `3.1`'dir.

Çıktı
```
3.1
```

2 Cevap: E

`cin >> yas` `20`'yi okuyup Enter'ın bıraktığı `'\n'` karakterini tamponda bırakır. `getline` ise satır sonuna kadar okur; ilk karakter zaten `'\n'` olduğu için **boş bir satır** alır ve hemen döner. `Ayşe Nur` tamponda okunmadan kalır. A yanlış, program çökmez. B yanlış, `ad` boş çıkar (`Ayşe Nur` okunmaz). C yanlış, ikisi aynı programda kullanılabilir ama araya `cin.ignore(...)` koymak gerekir. D yanlış, `20` zaten `yas`'a okundu.

3 Cevap: A

`setw(5)` yalnızca kendisinden sonra gelen **ilk değer** olan `1` için geçerlidir. `1` beş karakterlik alana sağa yaslı yazılır (önüne dört boşluk eklenir: `[    1`), ardından `2` ve `3` genişliksiz yazılır. B, `setw`'nin kalıcı olduğunu varsayar. D, `setw`'nin hiç etkisi olmadığını varsayar. C ve E ise hiçbir kuralla üretilemez.

Çıktı
```
[    123]
```

4 Cevap: B

Önce `cin.clear()` hata bayraklarını temizler, sonra `cin.ignore(...)` tamponda kalan bozuk girdiyi (`abc`) siler. Sıra önemlidir: A'daki gibi `ignore` önce yazılırsa, hata durumundaki akışta çalışmaz; ardından `clear()` akışı açar ama bozuk girdi hâlâ yerinde durur. C yanlış, `clear()` tek başına `abc`'yi silmez; sonraki `>>` aynı yerde yine takılır. D yanlış, `fail()` yalnızca durumu **sorar**, temizlemez. E yanlış, `endl` çıkış tamponuyla ilgilidir.

5 Cevap: D

`'\n'` yalnızca satır sonu karakterini yazar. `endl` aynı karakteri yazar ve ek olarak çıkış tamponunu boşaltır (flush); bu yüzden döngüde çok satır yazarken daha yavaştır. A yanlış, fark gerçekten vardır. B uydurmadır, ikisi de her platformda çalışır. C yanlış, `'\n'` tek karakterdir. E tersini söylüyor: Boşaltan `endl`'dir.

6

Önce `fixed` ve `setprecision(1)` kalıcı olarak açılır.

1. `setw(6)` ile `3.14159`: bir ondalıkla `3.1`, üç karakter, önüne üç boşluk. Ardından `|` ve `2.0` (genişliksiz, `setw` bitti), sonra `]`.

2. `left` kalıcı olarak açılır. `setw(4)` ile `7`: `7` ve üç boşluk. `|` yazılır. İkinci `setw(4)` ile yine `7` ve üç boşluk.
3. `setfill('0')` ve `right` açılır. `setw(4)` ile `12`: `0012`. İkinci `setw(4)` ile `3`: `0003`. Yan yana `00120003`.
4. `setw(3)` yalnız `5` için geçerli: dolgu hâlâ `0` olduğu için `005`. `6` genişliksiz yazılır: `0056`.

Çıktı
```
[   3.1|2.0]
[7   |7   ]
00120003
0056
```

7

Girdi `12 abc 7`:

1. `cin >> x` → `12` okunur, `x = 12`.
2. `cin >> y` → `abc` sayı değildir; okuma başarısız olur, `y` `0` olur ve akış hata durumuna girer.
3. `cin >> z` → akış hata durumunda olduğu için hiçbir şey yapılmaz. `7` okunmaz, `z` ilk değeri olan `-1`'de kalır.
4. `cin.fail()` `true` olduğundan `1` yazılır.

Çıktı
```
12 0 -1 1
```

8

1. `d` 6 anlamlı basamakla yazılır: `1234.5678` → `1234.57`.
2. `d * 100` = `123456.78` → 6 basamakla `123457` (ondalık kısım yuvarlanıp kayboldu).
3. `d * 1000` = `1234567.8` → 6 basamağa sığmaz, bilimsel gösterim: `1.23457e+06`.
4. `fixed` ve `setprecision(1)` açılır: `1234.5678` → `1234.6`.
5. `5 / 2` iki `int`, tamsayı bölmesi: `2` (`fixed` tamsayıyı etkilemez). `5 / 3.0` ondalıklıdır: `1.666...` → bir ondalıkla `1.7`.

Çıktı
```
1234.57
123457
1.23457e+06
1234.6
2 1.7
```

9

Üç tamsayıyı `cin >> vizeNotu >> odevNotu >> finalNotu` ile okuruz. Ortalamada `0.3`, `0.2`, `0.5` kesirli sabitler olduğu için sonuç `double` olur. Başlık ve veri satırı için aynı genişlikleri (`6`, `6`, `6`, `8`) kullanırız; `right` varsayılan olduğundan yazmak zorunda değiliz, ama okunaklılık için ekledik.

notKarti.cpp

```cpp
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    int vizeNotu{0};
    int odevNotu{0};
    int finalNotu{0};
    cout << "Vize, ödev ve final notları: ";
    cin >> vizeNotu >> odevNotu >> finalNotu;
    double ort{vizeNotu * 0.3 + odevNotu * 0.2 + finalNotu * 0.5};

    cout << right << setw(6) << "Vize" << setw(6) << "Odev"
         << setw(6) << "Final" << setw(8) << "Ort." << '\n';
    cout << fixed << setprecision(2);
    cout << setw(6) << vizeNotu << setw(6) << odevNotu
         << setw(6) << finalNotu << setw(8) << ort << '\n';
    return 0;
}
```

Çıktı

```
Vize, ödev ve final notları: 60 80 70
  Vize  Odev Final    Ort.
    60    80    70   69.00
```

10

Ayracı bir `char` değişkenine okuruz: `cin >> gun >> ayrac >> ay >> ayrac >> yil` tarihi, `cin >> saat >> ayrac >> dakika` saati okur. Tarih satırının sonundaki Enter tamponda kalır; `>>` onu baştaki boşluk gibi atlar. Yazdırırken `setfill('0')` bir kez, `setw` her sayıdan önce yazılır (`setw` kalıcı değildir). Yıl için `setw(4)`, diğerleri için `setw(2)`.

tarihSaat.cpp

```cpp
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    int gun{0}, ay{0}, yil{0}, saat{0}, dakika{0};
    char ayrac{' '};
    cout << "Tarih (gg/aa/yyyy): ";
    cin >> gun >> ayrac >> ay >> ayrac >> yil;
    cout << "Saat (ss:dd): ";
    cin >> saat >> ayrac >> dakika;
    cout << setfill('0') << setw(4) << yil << "-" << setw(2) << ay << "-"
         << setw(2) << gun << " " << setw(2) << saat << ":" << setw(2)
         << dakika << '\n';
    return 0;
}
```

Çıktı

```
Tarih (gg/aa/yyyy): 7/3/2026
Saat (ss:dd): 9:5
2026-03-07 09:05
```

BÖLÜM 4

Koşullu İfadeler

- **Bu Bölümde Öğreneceklerin**
  - İki değeri karşılaştırıp sonucun neden "doğru" ya da "yanlış" çıktığını, bunu ekrana nasıl yazdıracağını
  - Programın bir koşula göre farklı yollara gitmesini sağlamayı
  - Birden fazla seçenek arasından değere göre birini seçmeyi
  - Birden fazla koşulu birleştirmeyi ve bunların hangi sırayla denendiğini
  - Sık yapılan hataları: eşittir işaretini yanlış yazmayı, gereksiz noktalı virgülü, unutulan parantez ve süslü parantezi, unutulan `break`'i
  - Ondalıklı sayılarla eşitlik denemesinin neden güvenilmez olduğunu

Şimdiye kadarki programlar yukarıdan aşağıya, her satırı sırayla çalıştırıyordu. Gerçek programlar ise **karar verir**: "Yaş 18 ya da daha büyükse şunu yap, değilse bunu yap." Bu bölümde programın akışını koşullara göre dallandırmayı öğreneceksin. Sınavlarda en çok tuzak sorusu çıkan bölümlerden biridir; çünkü küçük bir `=` ya da `;` programın anlamını tamamen değiştirir. Döngüleri (bir işi tekrar tekrar yapmayı) Bölüm 5'e bırakıyoruz.

4.1 Karar vermek ve ilişkisel operatörler

Karar vermek için önce iki değeri karşılaştırmak gerekir. Bunu **ilişkisel operatörler** (relational operators) yapar:

Operatör	Anlamı	Örnek	Sonuç
`<`	küçüktür	`3 < 5`	doğru
`>`	büyüktür	`3 > 5`	yanlış
`<=`	küçük eşittir	`5 <= 5`	doğru
`>=`	büyük eşittir	`4 >= 5`	yanlış
`==`	eşittir	`5 == 5`	doğru
`!=`	eşit değildir	`5 != 5`	yanlış

Eşitlik denetimi **iki** eşittir işaretiyle yapılır; tek `=` atama operatörüdür (Bölüm 2). İki karakterli operatörlerin sırası sabittir: `=<` ya da `=>` diye bir operatör yoktur, `<>` de "eşit değil" anlamına gelmez.

C++'ta bu operatörlerin sonucu `bool` türündedir: karşılaştırma doğruysa `true`, yanlışsa `false`. `cout` bir `bool` değerini varsayılan olarak `1` ya da `0` yazar. Karşılaştırmayı **parantez içine** alarak doğrudan yazdırabiliriz:

iliskisel.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int a{7};
    int b{4};

    cout << "a > b       : " << (a > b) << '\n';
    cout << "a == b      : " << (a == b) << '\n';
    cout << "'a' < 'b'   : " << ('a' < 'b') << '\n';
    cout << "a + b == 11 : " << (a + b == 11) << '\n';
    return 0;
}
```

Çıktı

```
a > b       : 1
a == b      : 0
'a' < 'b'   : 1
a + b == 11 : 1
```

Karakterler de sayı olduğu için karşılaştırılabilir: `'a' < 'b'` doğrudur çünkü ASCII kodları 97 ve 98'dir (Ek D). Son satırda önce toplama, sonra karşılaştırma yapıldı; aritmetik operatörler karşılaştırmalardan önce işlenir. Bunu "Operatör önceliği" başlığında tabloyla toplayacağız.

Parantez neden şart?

`cout << a > b` yazmak çok doğal görünür ama çalışmaz. Bölüm 1'de gördüğün `<<` aslında bir operatördür ve `>` **operatöründen önce** işlenir. Yani ifade `(cout << a) > b` olarak okunur; `cout << a` bir akış (stream) verir ve akışı bir sayıyla karşılaştıracak bir `>` yoktur:

coutTuzagi.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int a{7};
    int b{4};

    cout << a > b;
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
coutTuzagi.cpp: In function 'int main()':
coutTuzagi.cpp:8:15: error: no match for 'operator>' (operand types are 'std::basic_ostream<char>'
and 'int')
    8 |     cout << a > b;
      |     ~~~~~~~~~ ^ ~
      |           |     |
      |           |     int
      |           std::basic_ostream<char>
  * there are 13 candidates
    coutTuzagi.cpp:8:15:
        8 |     cout << a > b;
          |     ~~~~~~~~~~^~~
    * candidate 1: 'operator>(int, int)' (built-in)
      * no known conversion for argument 1 from 'std::basic_ostream<char>' to 'int'
... (g++ 122 satır daha yazar; burada kısaltıldı)
```

Hata mesajı "`operator>` için eşleşen yok" der. g++ bunun ardından `>`'ın denenebilecek 13 sürümünü ve kütüphanedeki şablonları sayfalarca listeler; burada yalnız başını gösterdik. Asıl bilgi her zaman ilk `error:` satırındadır; uzun mesajları okumayı Ek C'de göreceksin. Çözüm basit: karşılaştırmayı `cout << (a > b)` diye parantezle. `cout` ile karşılaştırma yazdırırken bu parantezleri hiç unutma.

4.2 bool türü: true, false ve boolalpha

`bool`, C++'a yerleşik bir türdür; hiçbir başlık dosyası eklemen gerekmez. Yalnızca iki değer taşır: `true` ve `false`. Bölüm 2'de gördüğün gibi bellekte 1 bayt yer kaplar. Değişken adı olarak niyeti gösterir (`yetiskin`, `ogrenciMi`, `bulundu`): `if (yetiskin)` satırı cümle gibi okunur.

Bir `bool` yazdırınca ekranda `1` ya da `0` görmek her zaman istediğin şey değildir. `cout`'a `boolalpha` dersen bundan sonraki `bool` değerleri `true` ve `false` sözcükleri olarak yazılır. `boolalpha`, Bölüm 3'te gördüğün `fixed` ve `setprecision` gibi bir **akış manipülatörü**dür (manipulator). Eski haline `noboolalpha` ile dönersin:

boolKullanimi.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    bool yetiskin{20 >= 18};
    bool ogrenci{false};
    bool cokluDeger = 42;

    cout << yetiskin << ' ' << ogrenci << ' ' << cokluDeger << '\n';

    cout << boolalpha;
    cout << yetiskin << ' ' << ogrenci << ' ' << cokluDeger << '\n';
    cout << (5 > 3) << ' ' << (5 > 3 && 2 > 9) << '\n';

    cout << noboolalpha << (5 > 3) << '\n';
    return 0;
}
```

Çıktı

```
1 0 1
true false true
true false
1
```

Üç gözlem:

1. `bool cokluDeger = 42;` satırında `42`, `true` olarak saklandı: `bool` değişkenine **sıfır dışı** her sayı `true`, `0` ise `false` olarak girer. Bu yüzden üçüncü değer `1` yazıldı, `42` değil. Bu satırda kitabın alışkanlığı olan `{}` yerine bilerek `=` kullandık: `bool cokluDeger{42};` yazsaydın `42`'yi `bool`'a sığdırmak bir daralma (Bölüm 2) sayılır ve program derlenmezdi.
2. `boolalpha` bir kere yazılınca **kalıcıdır**: Sonraki bütün `bool` yazdırmaları sözcük olur, ta ki `noboolalpha` diyene kadar.
3. `boolalpha` yalnız `bool` türünü etkiler. `int` türündeki bir `1` yine `1` yazılır.

4.3 C++'ta doğru ve yanlış

Karşılaştırma `true` ya da `false` üretiyorsa, karar verirken **bir sayı** nasıl yorumlanır? C++ (C'den miras kalan) kuralı çok basittir:

- `0` **yanlıştır** (false).
- **Sıfırdan farklı her değer doğrudur** (true): `1`, `5`, `-1`, hatta `0.5` bile.

Özellikle `-1` **doğrudur**: "negatif sayı yanlıştır" diye düşünmek klasik bir hatadır. Yani `if (5)` her zaman, `if (0)` hiçbir zaman çalışır. `if (sayi != 0)` yerine kısaca `if (sayi)` yazıldığını da göreceksin; ikisi aynı anlama gelir. Bu kuralı bir sonraki başlıktaki `if` ile hemen deneyeceksin.

> **Sınavda Çıkar**
>
> "`if (-1)` doğru mu yanlış mı?" ve "`cout << (3 > 5);` ne yazdırır?" tipi sorular çok çıkar. Cevaplar: `-1` **doğrudur** (sıfır dışı); `3 > 5` yanlıştır, yani `boolalpha` yoksa `0` yazdırılır. Sıfır dışı = doğru, sıfır = yanlış. Başka kural yok.

4.4 if, if-else ve else if

`if` koşul doğruysa bir bloğu çalıştırır; `else` koşul yanlışken çalışacak yolu verir. Koşul yanlışsa ve `else` yoksa blok atlanır, program bir sonraki satırdan devam eder:

```
if (koşul) {
    // koşul doğruysa çalışır
} else {
    // koşul yanlışsa çalışır (else kısmı isteğe bağlıdır)
}
```

Koşulun **parantez içinde** olması zorunludur. `if-else`'te iki bloktan **tam biri** çalışır. Örneğin sayının tek mi çift mi olduğunu `if (sayi % 2 == 0) { ... } else { ... }` ile bulursun. Negatif sayılarda dikkat: `%` sonucu bölünenin işaretini taşır, yani `-7 % 2` sonucu `-1`'dir. Bu yüzden tek sayıyı `sayi % 2 == 1` ile aramak negatiflerde yanılır; `sayi % 2 != 0` güvenlidir.

else if merdiveni

İkiden fazla yol olduğunda `else if` ile **merdiven** kurarsın. Koşullar **yukarıdan aşağıya** denenir; ilk doğru çıkan bloğun içi çalışır, merdivenin geri kalanı atlanır. Hiçbiri doğru değilse en sondaki `else` çalışır:

harfNotu.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int puan{0};

    cout << "Puanın (0–100): ";
    cin >> puan;

    if (puan >= 90) {
        cout << "Harf notu: A\n";
    } else if (puan >= 80) {
        cout << "Harf notu: B\n";
    } else if (puan >= 70) {
        cout << "Harf notu: C\n";
    } else {
        cout << "Harf notu: F\n";
    }
    return 0;
}
```

Çıktı

```
Puanın (0–100): 74
Harf notu: C
```

`74` için sırayla bakıldı: `>= 90` yanlış, `>= 80` yanlış, `>= 70` doğru. `C` yazıldı ve geri kalanına **hiç bakılmadı**. Bu yüzden `puan >= 70 && puan < 80` yazmana gerek yok: `puan >= 80` zaten yukarıda elenmiştir. Merdivenin **sırası** önemlidir; `puan >= 70` koşulunu en üste koysaydın 95 alan öğrenci de `C` alırdı.

> **Sık Yapılan Hata: Ayrı `if`'ler ile `else if` aynı şey değildir**
>
> Her koşulu ayrı bir `if` ile yazarsan her biri bağımsız denenir ve birden fazlası çalışabilir. Yukarıdaki merdiveni `else` olmadan yazıp `puan = 95` verirsen hem `A` hem `B` hem `C` yazılır; çünkü `95` her koşulu sağlar. Tek sonuç istiyorsan `else if` kullan.

4.5 Süslü parantez tuzakları

Gövdesinde tek deyim olan `if`, süslü parantezsiz de yazılabilir. Ama bu durumda koşula **yalnızca ilk deyim** bağlıdır. Girinti derleyiciyi ilgilendirmez; yalnız insanı yanıltır:

suslusuz.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int puan{80};

    if (puan < 50)
        cout << "Kaldın.\n";
        cout << "Tekrar dene.\n";
    return 0;
}
```

g++ girintinin gerçek akışla uyuşmadığını fark edip uyarır:

Derleyici uyarısı – g++ -Wall -Wextra

```
suslusuz.cpp: In function 'int main()':
suslusuz.cpp:7:5: warning: this 'if' clause does not guard... [-Wmisleading-indentation]
    7 |     if (puan < 50)
      |     ^~
suslusuz.cpp:9:9: note: ...this statement, but the latter is misleadingly indented as if it were
guarded by the 'if'
    9 |         cout << "Tekrar dene.\n";
      |         ^~~~
```

Puan `80` olduğu için koşul yanlış ve `Kaldın.` yazılmadı; ama ikinci `cout` `if`'e bağlı **değildi**, yine de çalıştı:

Çıktı

```
Tekrar dene.
```

> **İpucu: Süslü parantezleri her zaman yaz**
>
> Gövde tek satır bile olsa `{ }` kullan; sonradan araya satır eklediğinde bu hataya düşmezsin. Sınavda süslü parantezsiz kodla karşılaşırsan girintiye değil, koşula bağlı **tek deyime** bak.

if (koşul); tuzağı

İkinci tuzak, koşulun hemen arkasına yanlışlıkla `;` koymaktır. C++'ta tek başına `;` bir **boş deyimdir** (empty statement): hiçbir şey yapmaz. `if (sayi > 5);` yazarsan koşul yalnızca bu boş deyime bağlanır; alttaki süslü blok ise sıradan bir blok olarak **her zaman** çalışır. Örneğin `sayi` 3 iken bile "Sayı 5'ten büyük" yazılır. g++ bu alışkanlığı `-Wempty-body` ile (ve yanıltıcı girinti uyarısıyla) yakalar. Sınavda `)` sonrasındaki `;` işaretini ara.

4.6 İç içe if ve sarkan else

Bir `if`'in gövdesine başka bir `if` yazabilirsin (iç içe if, nested if). **Sarkan else** (dangling else) sorunu süslü parantez yazılmadığında çıkar. Kural: **`else`, kendisinden önceki, henüz `else` almamış en yakın `if`'e bağlanır.** Girinti buna bakmaz:

sarkanElse.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int a{0};
    int b{0};

    if (a == 1)
        if (b == 1)
            cout << "A\n";
    else
        cout << "B\n";
    cout << "Bitti\n";
    return 0;
}
```

`else`'i dış `if`'in hizasına yazdık; "a 1 değilse B yaz" demek istediğimizi sanırsın. g++ şüphelenir (`-Wdangling-else`, `-Wall` içindedir):

```
Derleyici uyarısı – g++ -Wall -Wextra
sarkanElse.cpp: In function 'int main()':
sarkanElse.cpp:8:8: warning: suggest explicit braces to avoid ambiguous 'else' [-Wdangling-else]
    8 |     if (a == 1)
      |        ^
```

Gerçekte `else` iç `if`'e (`b == 1`) bağlanır. `a` sıfır olduğu için iç `if`'e hiç girilmez ve `B` de yazılmaz:

```
Çıktı
Bitti
```

Niyet `else`'in dış `if`'e ait olması idiyse iç `if`'i süslü parantezle kapatırsın: `if (a == 1) { if (b == 1) { ... } } else { ... }`. Bu kez `else` dış `if`'e bağlanır ve aynı `a` ile `b` değerleri için program `B` yazar.

> **Sınavda Çıkar**
>
> Sarkan else sorularında kod bilerek yanıltıcı girintilenir. Önce girintiyi **yok say**, `else`'i en yakın `if`'e bağla, sonra çalıştır. Süslü parantez varsa bağlama süslü parantezlere göredir. "Girinti öyle gösteriyor" cevabı sınavda yanlıştır.

4.7 Mantıksal operatörler

Birden fazla koşulu birleştirmek için üç **mantıksal operatör** (logical operators) vardır: `&&` (VE), `||` (VEYA), `!` (DEĞİL). Doğruluk tablosu (`0` yanlış, `1` doğru):

a	b	`a && b`	`a || b`	`!a`
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

`&&` yalnız **ikisi de** doğruysa, `||` **en az biri** doğruysa doğrudur; `!` doğruyu yanlış, yanlışı doğru yapar. Sonuç yine bir `bool`'dur. İşlenen olarak sıfır dışı her değer doğru sayılır; yani `5 && 3` sonucu `3` değil `true`'dur. En çok kullanıldığı yer **aralık denetimidir**: Puanın geçerli olması için `puan >= 0 && puan <= 100` ("içinde mi?"), geçersiz olması için `puan < 0 || puan > 100` ("dışında mı?") yazarsın.

Bu iki koşul birbirinin tersidir; bu eşdeğerliğe **De Morgan kuralları** denir: `!(a && b)` ile `!a || !b` aynıdır, `!(a || b)` ile `!a && !b` aynıdır. Parantezin önündeki `!`'yi içeri dağıtırken `&&` ile `||` yer değiştirir, her karşılaştırma da tersine döner (`>=` yerine `<`, `<=` yerine `>`).

> **Not: and, or ve not da var**
>
> C++'ta `&&`, `||` ve `!` yerine `and`, `or` ve `not` sözcüklerini de yazabilirsin; `x >= 0 and x <= 100` geçerli C++'tır (C'de bunlar yalnızca `<iso646.h>` başlığıyla gelir). Kitapta ve sınavlarda simge hâllerini kullanıyoruz; ikisini de tanımalısın.

● Sık Yapılan Hata: x == 1 || 2 yazmak

"x, 1 ya da 2'ye eşit mi?" sorusunu `if (x == 1 || 2)` diye yazmak çok yaygın bir hatadır. İfade `(x == 1) || (2)` diye okunur ve `2` sıfır dışı olduğu için **her zaman doğrudur**: `x` 7 bile olsa blok çalışır. Üstelik g++ buna uyarı bile vermez. Doğrusu her koşulu ayrı yazmaktır: `x == 1 || x == 2`. (Tek karakterli `&` ve `|` ise bit düzeyinde işlem yapan başka operatörlerdir; mantıksal VE/VEYA için her zaman çift karakter yaz.)

Kısa devre değerlendirme

`&&` ve `||` işlenenlerini **soldan sağa** değerlendirir ve sonuç belli olur olmaz durur. Buna **kısa devre değerlendirme** (short-circuit evaluation) denir:

İfade	Sağ taraf ne zaman hiç hesaplanmaz?
`sol && sag`	`sol` yanlışsa (sonuç zaten yanlış)
`sol || sag`	`sol` doğruysa (sonuç zaten doğru)

Bu yalnız hız için değil, **güvenlik** için de işe yarar. Sıfıra bölmeyi engellemek için önce `x != 0` denetimini yaz:

guvenliBolme.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int x{0};

    cout << "x: ";
    cin >> x;

    if (x != 0 && 10 / x > 1) {
        cout << "10 / x, 1'den büyük.\n";
    } else {
        cout << "Ya x sıfır ya da 10 / x, 1'den büyük değil.\n";
    }
    return 0;
}
```

Çıktı

```
x: 0
Ya x sıfır ya da 10 / x, 1'den büyük değil.
```

`x` sıfırken `x != 0` yanlış olduğundan `10 / x` **hiç hesaplanmadı**. Sırayı ters yazarsan (`10 / x > 1 && x != 0`) önce bölme yapılır. Tamsayıyı sıfıra bölmek tanımsız davranıştır (undefined behavior): Windows'ta program çökebilir, başka bir bilgisayarda çökmeden çalışıp anlamsız bir sonuç verebilir. Yani koruyucu koşul **solda** durur.

Kısa devrenin ikinci sonucu, sağ taraftaki **yan etkilerin** (side effect) de atlanmasıdır. `++b` sağ taraftaysa her zaman çalışacağını sanma:

yanEtki.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int a{0};
    int b{0};

    bool s1 = a > 0 && ++b > 0;
    cout << "s1 = " << s1 << ", b = " << b << '\n';
    bool s2 = a == 0 || ++b > 0;
    cout << "s2 = " << s2 << ", b = " << b << '\n';
    bool s3 = a == 0 && ++b > 0;
    cout << "s3 = " << s3 << ", b = " << b << '\n';
    return 0;
}
```

Çıktı

```
s1 = 0, b = 0
s2 = 1, b = 0
s3 = 1, b = 1
```

1. `a > 0` yanlış → `&&`'in sağı atlanır, `++b` çalışmaz. `b` hâlâ `0`.
2. `a == 0` doğru → `||`'nin sağı atlanır, `++b` yine çalışmaz. `b` hâlâ `0`.
3. `a == 0` doğru → `&&`'nin sonucu henüz belli değil, sağ taraf hesaplanır. `++b` çalışır, `b` olur `1`.

> **Sınavda Çıkar**
>
> "`&&` ya da `||` içindeki `++` çalıştı mı?" sorusunun kuralı hep aynıdır: `&&`'de sol yanlışsa, `||`'de sol doğruysa sağ taraf **atlanır**. Çıktı sorularında değişkenleri adım adım yaz ve sağ tarafın çalışıp çalışmadığını işaretle. En sık hata, `++b`'nin "her halükarda" çalıştığını varsaymaktır.

4.8 = ile == karışıklığı

`=` **atama**, `==` **karşılaştırma** operatörüdür. Koşula yanlışlıkla `=` yazarsan derleyici hata vermez; çünkü atama da bir ifadedir ve **değeri atanan değerdir**: `x = 5` ifadesinin değeri `5`, yani doğrudur:

atamaTuzagi.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int x{3};

    if (x = 5) {
        cout << "x 5'e eşit\n";
    }
    cout << "x = " << x << '\n';
    return 0;
}
```

g++ koşulun etrafına parantez önerir:

Derleyici uyarısı – g++ -Wall -Wextra

```
atamaTuzagi.cpp: In function 'int main()':
atamaTuzagi.cpp:7:11: warning: suggest parentheses around assignment used as truth value [-
Wparentheses]
    7 |     if (x = 5) {
      |         ~~^~~
```

Program yine de derlenir ve çalışır:

Çıktı

```
x 5'e eşit
x = 5
```

Kötü haber iki tane: `x` 3 iken "5'e eşit" yazıldı **ve** `x` kalıcı olarak 5 oldu. `if (x = 0)` yazsaydın değer `0` (yanlış) olacağından blok hiç çalışmaz, `x` de sıfırlanırdı. Bilerek atama yapıp sonucuna bakacaksan uyarıyı çift parantezle sustursun: `if ((x = 5))`.

> **İpucu: Yoda yazımı: sabiti sola yaz**
>
> Karşılaştırmada sabiti sola yazarsan (`if (5 == x)`) bu hatadan korunursun: Yanlışlıkla `5 = x` yazarsan 5'e değer atanamayacağı için g++ `lvalue required as left operand of assignment` hatası verir ve hemen fark edersin (*lvalue*, yani **sol değer**: atamanın sol tarafında durabilen, genellikle bir değişken). Adı "Yoda koşulu"dur (Yoda gibi ters konuşur). Bu kitapta okunaklı olduğu için `x == 5` yazıyoruz, ama ikisini de tanımalısın.

4.9 `0 < x < 10` tuzağı

Matematikte "x, 0 ile 10 arasındadır" diye `0 < x < 10` yazılır. C++'ta bu derlenir ama **matematikteki anlamı yoktur**. İlişkisel operatörler soldan sağa işlenir: önce `0 < x`, sonra bu sonucun (`false` ya da `true`, yani `0` ya da `1`) `10`'dan küçük olup olmadığına bakılır. `0` ya da `1` her zaman 10'dan küçüktür, yani ifade **her zaman doğrudur**:

zincir.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int x{15};

    if (0 < x < 10) {
        cout << "x 0 ile 10 arasında\n";
    } else {
        cout << "x aralıkta değil\n";
    }
    return 0;
}
```

Derleyici uyarısı – g++ -Wall -Wextra

```
zincir.cpp: In function 'int main()':
zincir.cpp:7:15: warning: comparison of constant '10' with boolean expression is always true [-
Wbool-compare]
    7 |     if (0 < x < 10) {
      |         ~~~~~~^~~~
zincir.cpp:7:11: warning: comparisons like 'X<=Y<=Z' do not have their mathematical meaning [-
Wparentheses]
    7 |     if (0 < x < 10) {
      |         ~~^~~
```

Çıktı

```
x 0 ile 10 arasında
```

g++ iki uyarı verdi: Karşılaştırmanın sonucu hep doğru (`-Wbool-compare`) ve bu yazımın matematikteki anlamı yok (`-Wparentheses`). `x` 15 olduğu hâlde "arasında" yazıldı: `0 < 15` doğru → `1`; `1 < 10` doğru. Doğrusu iki koşulu `&&` ile bağlamaktır: `x > 0 && x < 10`. Aynı tuzak eşitlikte de var: `a == b == c`, "üçü de eşit" değil `(a == b) == c` demektir; doğrusu `a == b && b == c`.

> **Not: Aynı satır clang'de hata verebilir**
>
> macOS'ta `g++` yazınca çalışan şey aslında Apple clang'dir; kitaptaki mesajlar ise gerçek g++ çıktısıdır, clang'in mesajları farklı olabilir. Bu satır g++'ta yalnız uyarı üretir, program derlenir. Apple clang'in yeni sürümleri ise aynı `0 < x < 10` satırını **derleme hatası** sayar (`chained comparison ... does not behave the same as a mathematical expression`). Hangi derleyiciyle çalışırsan çalış, bu yazımı kullanma.

! operatörünün önceliği

`!` bir **tekli operatör** (unary operator) olup karşılaştırmalardan **önce** işlenir. Bu yüzden "x, 5 değil mi?" diye `if (!x == 5)` yazmak klasik bir hatadır: ifade `(!x) == 5` diye okunur. `!x` yalnızca `false` ya da `true` (`0` ya da `1`) verebildiği için bu koşul **hiçbir `x` için doğru olamaz**; g++ da `-Wlogical-not-parentheses` ile uyarır. Doğru yazım `x != 5` ya da `!(x == 5)`'tir. Aynı sebeple `!x + 1` ifadesi `!(x + 1)` değil `(!x) + 1` demektir.

4.10 İşaretli ve işaretsiz sayıları karşılaştırmak

Bölüm 2'de `unsigned` tipleri ve bu tuzağın ilk örneğini gördün; artık `if`'i bildiğine göre üzerinden bir kez daha geçelim. Bir `int` ile bir `unsigned` sayıyı karşılaştırırsan C++ önce `int`'i `unsigned`'a çevirir. Negatif bir sayı bu çevirmede **çok büyük bir pozitif sayıya** dönüşür:

isaretsiz.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    int eksi{-1};
    unsigned int bir{1};
    string kelime{"merhaba"};

    cout << boolalpha;
    cout << "-1 < 1u            : " << (eksi < bir) << '\n';
    cout << "-1 < kelime.size() : " << (eksi < kelime.size()) << '\n';
    return 0;
}
```

g++ `-Wall` ile tam bu tuzağı yakalar:

Derleyici uyarısı – g++ -Wall -Wextra

```
isaretsiz.cpp: In function 'int main()':
isaretsiz.cpp:11:46: warning: comparison of integer expressions of different signedness: 'int' and
'unsigned int' [-Wsign-compare]
   11 |     cout << "-1 < 1u           : " << (eksi < bir) << '\n';
      |                                        ~~~~~^~~~
isaretsiz.cpp:12:46: warning: comparison of integer expressions of different signedness: 'int' and
'std::__cxx11::basic_string<char>::size_type' {aka 'long unsigned int'} [-Wsign-compare]
   12 |     cout << "-1 < kelime.size() : " << (eksi < kelime.size()) << '\n';
      |                                        ~~~~~^~~~~~~~~~~~~~~
```

Çıktı

```
-1 < 1u           : false
-1 < kelime.size() : false
```

`-1 < 1` matematikte doğrudur ama ilk satırda `false` çıktı: `-1` işaretsize çevrilince 4294967295 oldu. İkinci satır daha sinsidir: `kelime.size()` bir `string`'in uzunluğunu verir (Bölüm 8) ve türü **işaretsizdir**. Bu yüzden `string` ve `vector` (Bölüm 7) uzunluklarını bir `int` ile karşılaştırırken g++ aynı uyarıyı verir. Çözüm, işaretsiz değeri `static_cast<int>` ile `int`'e çevirmek ya da değişkeni baştan uygun türde tanımlamaktır. Uyarıyı susturmak için değil, **sonucu doğru** yapmak için çevir.

4.11 Metinleri ve karakterleri karşılaştırmak

C++'ın en güzel yanlarından biri, `string` değerlerini doğrudan `==`, `!=`, `<`, `>` ile karşılaştırabilmendir (C'de bunun için `strcmp` gerekiyordu). `==` iki metnin **harfi harfine aynı** olup olmadığına bakar; `<` ve `>` ise sözlük sırasına (karakter kodlarına) göre karşılaştırır. `string`'i Bölüm 8'de ayrıntılı göreceksin; şimdilik karşılaştırma yeteneği yeter:

metinKarsilastir.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string sifre{"kedi123"};
    string girilen{};

    cout << "Şifre: ";
    cin >> girilen;

    if (girilen == sifre) {
        cout << "Giriş başarılı.\n";
    } else {
        cout << "Hatalı şifre.\n";
    }
    return 0;
}
```

Çıktı

```
Şifre: Kedi123
Hatalı şifre.
```

Girilen `Kedi123`, `kedi123`'ten yalnız baş harfiyle ayrılıyor ama **büyük/küçük harf farklıdır**; bu yüzden şifre kabul edilmedi. `<` ve `>` de karakter kodlarına bakar: büyük harfler (`'A'` = 65) küçük harflerden (`'a'` = 97) önce gelir; yani `string a{"Armut"}, b{"elma"};` için `a < b` doğru, `b < a` yanlıştır. Bu, günlük sözlük sırası değildir.

> **Sık Yapılan Hata: İki çift tırnaklı sabiti karşılaştırmak**
>
> Karşılaştırmanın en az bir tarafı `string` değişkeni olmalı. `"Armut" < "elma"` gibi iki çift tırnaklı sabiti doğrudan karşılaştırırsan harflere **bakılmaz**: bu sabitler `string` değil, karakter dizisidir ve karşılaştırma bellekteki adreslerine göre yapılır (Bölüm 9). Sonuç anlamsızdır; g++ bunun için uyarı da verir. `==` için de aynısı geçerlidir.

4.12 Operatör önceliği

Bir ifadede birden fazla operatör varsa hangisinin önce işleneceğini **öncelik** belirler. Bölüm 2'de aritmetik operatörlerin sırasını gördün; şimdi hepsini tek tabloda topluyoruz (yüksekten düşüğe):

Sıra	Operatörler	Birleşme
1	`( )` gruplama	içten dışa
2	son ek `x++`, `x--`; fonksiyon çağrısı `f(x)`, dizi indeksi `a[i]`, `.` nokta (Bölüm 6–7, 11)	soldan sağa
3	`!`, `~`, ön ek `++x`, `--x`, tekli `-` ve `+`, `static_cast<>` ve `(tip)` dönüşümü, `sizeof` (Bölüm 9'daki `*p` ve `&x` de burada)	sağdan sola
4	`*`, `/`, `%`	soldan sağa
5	`+`, `-`	soldan sağa
6	`<<`, `>>` (hem bit kaydırma hem `cout` / `cin` operatörleri)	soldan sağa
7	`<`, `<=`, `>`, `>=`	soldan sağa
8	`==`, `!=`	soldan sağa
9	`&`, `^`, `|` (bit VE, XOR, VEYA)	soldan sağa
10	`&&`	soldan sağa
11	`||`	soldan sağa
12	`?:` (üçlü operatör), `=`, `+=`, `-=`, `*=`, `/=`, `%=` ve bit atamaları	sağdan sola

Son ek `x++` ile ön ek `++x` **farklı satırlardadır**: son ek daha güçlüdür ve soldan sağa birleşir. Bu yüzden `!x++` ifadesi `!(x++)` diye okunur.

Hatırlama yolu: **önce aritmetik, sonra karşılaştırma, sonra mantık, en son atama.** Bit operatörleri araya girer: kaydırma (`<<`, `>>`) aritmetikten hemen sonra, `&`, `^`, `|` ise karşılaştırmadan sonra gelir. Bu yüzden `x & 1 == 0` ifadesi `x & (1 == 0)` diye işlenir; bit işleminde parantez kullan. `&&` her zaman `||`'den güçlüdür (matematikte çarpmanın toplamadan önce gelmesi gibi).

`<<` satırı sana tanıdık gelmeli: `cout << a > b` hatasının sebebi `<<`'in `>`'den önce gelmesiydi. `cout << a + b` ise çalışır, çünkü `+` daha yüksek öncelikli olduğu için `<<`'ten önce hesaplanır.

oncelik.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int a{5};
    int b{3};
    int c{2};

    cout << boolalpha;
    cout << (a + b > 7 && c * 2 == 4) << '\n';
    cout << (a > 3 || b > 5 && c > 5) << '\n';
    cout << ((a > 3 || b > 5) && c > 5) << '\n';
    return 0;
}
```

Çıktı

```
true
true
false
```

1. `a + b > 7 && c * 2 == 4`: önce `a + b` (8) ve `c * 2` (4); sonra `8 > 7` doğru, `4 == 4` doğru; sonuç `true`.
2. `a > 3 || b > 5 && c > 5`: önce `&&`: `b > 5` yanlış olduğundan `b > 5 && c > 5` yanlış. `a > 3` doğru olduğu için `||` sonucu `true`.
3. Parantez önceliği değiştirir: `(a > 3 || b > 5)` doğru, ama `c > 5` yanlış; sonuç `false`.

İkinci satırda `&&` ile `||` parantezsiz karıştığı için `-Wall` uyarır:

Derleyici uyarısı – g++ -Wall -Wextra

```
oncelik.cpp: In function 'int main()':
oncelik.cpp:11:29: warning: suggest parentheses around '&&' within '||' [-Wparentheses]
   11 |     cout << (a > 3 || b > 5 && c > 5) << '\n';
      |                       ~~~~~~^~~~~~~
```

> **İpucu: Karışık &&/|| için parantez koy**
>
> `&&`'nin `||`'den önce işlendiğini bilsen bile parantez koy: `a > 3 || (b > 5 && c > 5)`. Hem g++ susar hem de kodu okuyan (hoca dahil) hesap yapmak zorunda kalmaz. Sınavda ise parantezsiz yazılmış ifadeyi **sen** tabloyla çözersin.

4.13 Üçlü operatör ?:

C++'ta üç işlenenli tek operatör `?:`'dir. Bir koşula göre iki değerden birini **seçer**:

```
koşul ? değerDoğruysa : değerYanlışsa
```

`if-else` bir **deyimdir** (statement) ve değer üretmez; `?:` ise bir **ifadedir** (expression) ve değer üretir. Bu yüzden sonucu doğrudan bir değişkene atayabilir ya da `cout`'un içine koyabilirsin:

ucluOperator.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int a{12};
    int b{20};
    int sayi{7};
    int puan{85};

    int enBuyuk = a > b ? a : b;
    cout << "En büyük: " << enBuyuk << '\n';
    cout << sayi << " sayısı " << (sayi % 2 == 0 ? "çift" : "tek") << '\n';
    cout << "Not: " << (puan >= 90 ? "A" : puan >= 80 ? "B" : "C") << '\n';
    return 0;
}
```

Çıktı

```
En büyük: 20
7 sayısı tek
Not: B
```

`cout` içinde `?:` kullanırken ifadeyi **parantez içine al**. `?:`, `<<`'ten düşük öncelikli olduğu için `cout << "Durum: " << yetiskin ? "yetişkin" : "çocuk";` satırı `(cout << "Durum: " << yetiskin) ? ...` diye çalışır: ekrana `Durum: 1` yazılır, seçilen metin ise hiç yazılmaz (g++ iki işlenen için "has no effect" uyarısı verir). Doğrusu `cout << "Durum: " << (yetiskin ? "yetişkin" : "çocuk")`.

`?:` sağdan sola birleştiği için `a ? b : c ? d : e` yazımı `a ? b : (c ? d : e)` demektir. Çalışır ama okunması zorlaşır; birden fazla işlem gerektiğinde `else if` merdiveni kullan.

4.14 switch deyimi

Bir değişkeni **belirli sabit değerlerle** karşılaştırıyorsan uzun bir `else if` merdiveni yerine `switch` kullanabilirsin:

```cpp
switch (ifade) {
    case sabit1:
        // ifade == sabit1 ise buradan başlar
        break;
    case sabit2:
        break;
    default:
        // hiçbir case uymazsa (isteğe bağlı)
}
```

`ifade` hesaplanır, değeri hangi `case` etiketine eşitse program **o etiketin altından** çalışmaya başlar. `break` görülünce `switch`'ten çıkılır; hiçbir etiket uymazsa `default`'tan devam edilir:

sesliHarf.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    char harf{' '};

    cout << "Bir harf yaz (a-z): ";
    cin >> harf;

    switch (harf) {
        case 'a': case 'e': case 'i': case 'o': case 'u':
            cout << "'" << harf << "' bir sesli harftir.\n";
            break;
        default:
            cout << "'" << harf << "' sesli harf değil.\n";
    }
    return 0;
}
```

Çıktı

```
Bir harf yaz (a-z): e
'e' bir sesli harftir.
```

`cin >> harf` baştaki boşlukları kendiliğinden atlar. İlk satırdaki beş etiketin arasında deyim olmadığı için hangisi eşleşirse aynı `cout` satırına düşülür. Bu **bilerek** yapılan bir düşmedir; aşağıda göreceğin, `break` unutulduğu için olan düşmeden farklıdır.

`switch`'in kuralları:

- `switch (ifade)` içindeki ifade **tamsayı türünde** olmalıdır: `int`, `char`, `short`, `long long` (ve `bool`). `double` ya da `float` olmaz; `string` de olmaz.
- `case` etiketleri derleme zamanında bilinen **sabit** olmalıdır: `3`, `'a'`, `2 + 3` ya da `const` olarak tanımlanmış bir değer. Sıradan değişken olmaz.
- `default` isteğe bağlıdır ve herhangi bir yerde olabilir; sona koymak iyi alışkanlıktır. Aynı değer iki `case`'de yazılamaz.
- `switch`, yalnız **eşitlik** denetler. "Puan 70'ten büyükse" gibi aralıklar için `else if` merdiveni kullan.

break unutulursa: fall-through

`break` yazmazsan program etiketi geçince durmaz; **bir alttaki case'in kodunu da çalıştırmaya devam eder**. Buna düşme (fall-through) denir ve `switch`'in en bilinen tuzağıdır:

breakYok.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int secim{1};

    switch (secim) {
        case 1: cout << "Bir\n";
        case 2: cout << "İki\n";
        default: cout << "Başka\n";
    }
    return 0;
}
```

g++, `-Wextra` içindeki `-Wimplicit-fallthrough` ile yakalar (macOS'taki `g++`, yani Apple clang, `-Wall -Wextra` ile bu uyarıyı **vermez**; görmek için komuta `-Wimplicit-fallthrough` ekle):

Derleyici uyarısı – g++ -Wall -Wextra

```
breakYok.cpp: In function 'int main()':
breakYok.cpp:8:25: warning: this statement may fall through [-Wimplicit-fallthrough=]
    8 |         case 1: cout << "Bir\n";
      |                         ^~~~~~~
breakYok.cpp:9:9: note: here
    9 |         case 2: cout << "İki\n";
      |         ^~~~
breakYok.cpp:9:25: warning: this statement may fall through [-Wimplicit-fallthrough=]
    9 |         case 2: cout << "İki\n";
      |                         ^~~~~~~
breakYok.cpp:10:9: note: here
   10 |         default: cout << "Başka\n";
      |         ^~~~~~~
```

Çıktı

```
Bir
İki
Başka
```

`secim` 1 olduğu için `case 1`'in altından başlandı ve `break` olmadığı için `case 2` ile `default`'taki satırlar da çalıştı. Etiket yalnızca **nereden başlanacağını** söyler; nerede duracağını `break` söyler. Son bloğa `break` gerekmez. Sınavda yöntem şudur: eşleşen etiketi bul, oradan aşağı doğru ilk `break`'e kadar (ya da `switch`'in sonuna kadar) her satırı çalıştır; aradaki etiketler yalnız giriş noktasıdır, kendileri bir şey yapmaz.

Bilerek düşmek: [[fallthrough]]

Araya hiç deyim girmeyen boş etiketler (`case 'a': case 'e':`) g++'yı uyarmaz. Bazen ise bir case'in işini yapıp **bilerek** bir alttakine düşmek istersin. C++17'de bunu `[[fallthrough]];` ile açıkça belirtirsin; hem uyarı kesilir hem de kodu okuyan "unutmuş" demez:

```
case 2:
    cout << "Film\n";
    [[fallthrough]];   // bilerek düşüyoruz
case 1:
    cout << "Haber\n";
    break;
```

`case 2` çalışırsa önce `Film`, sonra düşülüp `Haber` yazılır.

case etiketinde neler yazılamaz

`case` etiketi derleme zamanında bilinen bir **sabit ifade** (constant expression) olmalıdır. Bu yüzden iki yazım derlenmez:

- `case sinir:`, `sinir` sıradan bir değişkense yazılamaz; g++ "the value of 'sinir' is not usable in a constant expression" hatası verir.
- `case 2.5:` ondalıklı sayıdır; etiket tamsayı sabit olmalıdır.

C'den farkı: C++'ta `const` ile tanımlanıp sabitle ilk değer almış bir tamsayı `case` etiketi olabilir. `const int SINIR{5};` yazarsan `case SINIR:` geçerlidir (C'de değildi).

Adlı sabitler hem okunabilirlik hem de "sihirli sayı" (Bölüm 2) yüzünden `case 1:` yazmaktan iyidir.

case içinde değişken tanımlamak

`case` etiketleri yalnız **giriş noktasıdır**; `switch` bloğu tek bir bloktur. Bir `case`'in altında ilk değerli bir değişken tanımlarsan sonraki etiketler o tanımı **atlayarak** içeri atlayabilir ve C++ buna izin vermez:

caseTanim.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int secim{2};

    switch (secim) {
        case 1:
            int kare{secim * secim};
            cout << kare << '\n';
            break;
        case 2:
            cout << "İki\n";
            break;
    }
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
caseTanim.cpp: In function 'int main()':
caseTanim.cpp:12:14: error: jump to case label
   12 |         case 2:
      |              ^
caseTanim.cpp:9:17: note:   crosses initialization of 'int kare'
    9 |             int kare{secim * secim};
      |                 ^~~~
```

Çözüm, o `case`'in gövdesini kendi süslü parantezine almaktır; değişkenin kapsamı (scope) bu bloğun içiyle sınırlı kalır:

```cpp
case 1: {
    int kare{secim * secim};
    cout << kare << '\n';
    break;
}
```

string ile switch olmaz

`switch` yalnız tamsayı türleriyle çalışır; `string` (ya da `double`) ile denersen g++ `switch quantity not an integer` hatası verir. Metinlerde çözüm `if` / `else if` merdivenidir; `string` için `==` zaten metni karşılaştırır (`komut == "ekle"`).

> **Sınavda Çıkar**
>
> "`switch`'te hangisi kullanılamaz?" sorusunun kalıbı: `double` ifade, `string` ifade ya da değişken `case` etiketi. `char` ve `int` kullanılabilir; `const` sabitler C++'ta etiket olabilir. Bir `case`'in altında süslü parantezsiz değişken tanımı da derleme hatasıdır.

4.15 double değerlerini karşılaştırmak

`double` değerler bellekte ikilik tabanda tutulur ve `0.1` gibi sayılar tam yazılamaz (Bölüm 2'deki hassasiyet konusu). Bu yüzden iki hesabın sonucu "matematikte eşit" olsa da bitleri aynı olmayabilir. `==` ise **tam** eşitlik arar:

ondalik.cpp

```cpp
#include <cmath>
#include <iomanip>
#include <iostream>
using namespace std;

int main() {
    double a{0.1 + 0.2};
    double b{0.3};
    const double tolerans{1e-9};

    cout << fixed << setprecision(17);
    cout << "a = " << a << '\n';
    cout << "b = " << b << '\n';
    cout << "fark = " << fabs(a - b) << '\n';

    cout << boolalpha;
    cout << "a == b : " << (a == b) << '\n';
    cout << "fabs(a - b) < tolerans : " << (fabs(a - b) < tolerans) << '\n';
    return 0;
}
```

Çıktı

```
a = 0.30000000000000004
b = 0.29999999999999999
fark = 0.00000000000000006
a == b : false
fabs(a - b) < tolerans : true
```

`0.1 + 0.2`, `0.3`'e çok yakın ama tam eşit değil; `==` yanlış çıktı. Doğru yöntem, farkın mutlak değerinin çok küçük bir **tolerans**tan (burada `1e-9`: "1 çarpı 10 üzeri eksi 9", yani `0.000000001`) küçük olup olmadığına bakmaktır. `fabs` (mutlak değer) `<cmath>` başlığındadır. Toleransı sorunun ölçeğine göre seçersin; burada `1e-9` yeterlidir.

Asıl tehlike `==` ve `!=`'dir; `<` ve `>` de matematikte eşit olması gereken iki değerde yanlış sonuç verebilir. `0.25 + 0.25 == 0.5` gibi ikinin kuvvetlerinden oluşan sayılar tam gösterilebildiği için doğru çıkabilir, bu seni yanıltmasın.

4.16 Artık yıl kuralı

Bir yıl **4'e bölünüyorsa artık yıldır; ama 100'e de bölünüyorsa artık değildir; 400'e de bölünüyorsa yine artıktır.** Örneğin 2024 artıktır; 1900 artık değildir (100'e bölünür, 400'e bölünmez); 2000 artıktır (400'e bölünür). Bu kural tek bir koşula çevrilir: `(yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0`. Bu koşulu aşağıdaki kod sorusunda Şubat'ın kaç gün çektiğini bulurken kullanacaksın.

Bölüm Özeti – Bilmen Gerekenler

- İlişkisel operatörler (`<`, `>`, `<=`, `>=`, `==`, `!=`) `bool` üretir. `cout` ile yazdırırken **parantez** koy: `cout << (a > b)`. `boolalpha` ile `true`/`false` sözcükleri yazılır.
- **Sıfır yanlıştır, sıfır dışı her değer doğrudur** (`-1` ve `0.5` dahil). `bool` değişkenine sıfır dışı sayı atarsan `true` olur.
- `if`, `if-else`, `else if`: koşullar yukarıdan aşağıya denenir, ilk doğru olan çalışır. Gövde tek satır bile olsa `{ }` yaz. `if (x > 5);` boş gövdedir; `else` en yakın `if`'e bağlanır (sarkan else).
- `&&` (VE), `||` (VEYA), `!` (DEĞİL). **Kısa devre:** `&&`'de sol yanlışsa, `||`'de sol doğruysa sağ taraf hiç hesaplanmaz (`++` dahil).
- `=` atama, `==` karşılaştırmadır; `if (x = 5)` hata vermez ama her zaman doğrudur. `0 < x < 10` her zaman doğrudur; doğrusu `x > 0 && x < 10`. `x == 1 || 2` de her zaman doğrudur.
- `int` ile `unsigned` karşılaştırınca negatif sayı çok büyük pozitife dönüşür; `size()` işaretsizdir.
- `string` değerleri `==`, `<` ile doğrudan karşılaştırılır; büyük/küçük harf farklıdır.
- Öncelik: `( )` → `!` → `* / %` → `+ -` → `<< >>` → `< <= > >=` → `== !=` → `&&` → `||` → `?:` ve atama.
- `koşul ? a : b` bir değer üretir; `cout` içinde parantezle yaz, iç içe yazma.
- `switch` tamsayı/`char` ifadesini sabit `case`'lerle karşılaştırır. `break` yoksa sonraki case'ler de çalışır (fall-through, bilerek düşmek için `[[fallthrough]];`). `case`'de değişken, `double` ve `string` olmaz; `const` sabit olur.
- Ondalıklı sayıları `==` ile değil `fabs(a - b) < tolerans` ile karşılaştır.

Sorular

A) Çoktan Seçmeli

1 `x` değişkeninin değerinin 0 ile 100 arasında (uçlar dahil) olduğunu doğru biçimde denetleyen ifade hangisidir?

(A) `x >= 0 && x <= 100`
(B) `0 <= x <= 100`
(C) `x >= 0 || x <= 100`
(D) `x > 0 && x < 100`
(E) `x >= 0 && <= 100`

2 Aşağıdaki ifadelerden hangisinde `x` değeri `0` iken `10 / x` işlemi **hiçbir zaman çalıştırılmaz**?

(A) `10 / x > 1 && x != 0`
(B) `x != 0 || 10 / x > 1`
(C) `x != 0 && 10 / x > 1`
(D) `x == 0 && 10 / x > 1`
(E) `10 / x > 1 || x != 0`

3 Aşağıdakilerden hangisi `switch` içinde `case` etiketi olarak **kullanılamaz**? (Programda `const int SINIR{10};` ve `int sinir{10};` tanımlıdır.)

(A) `case 3:` (B) `case 'a':` (C) `case SINIR:` (D) `case 2 + 3:` (E) `case sinir:`

4 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
using namespace std;
int main() {
    int x{5};
    cout << boolalpha;
    cout << (x > 3 && x < 5 || x == 5) << ' ' << !x << '\n';
    return 0;
}
```

(A) `1 0`
(B) `true false`
(C) `true true`
(D) `false false`
(E) Derleme hatası verir

5 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
using namespace std;
int main() {
    int x{4};
    if (x = 0) { cout << "A\n"; } else { cout << "B\n"; }
    cout << x << '\n';
    return 0;
}
```

(A) `A` ve alt satırda `4`
(B) `B` ve alt satırda `4`
(C) `A` ve alt satırda `0`
(D) `B` ve alt satırda `0`
(E) Derleme hatası verir

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
using namespace std;
int main() {
    int a{2};
    int b{9};
    if (a > 5)
        if (b > 5)
            cout << "X\n";
    else
        cout << "Y\n";
    cout << "Z\n";
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
using namespace std;
int main() {
    int a{3};
    int b{1};
    int c{0};
    if (a > 5 && ++b > 0) { c = 10; }
    if (a < 5 || ++b > 0) { c += 1; }
    if (a > 1 && ++b > 1) { c += 20; }
    if (!b || b == 2) { c += 4; }
    cout << a << ' ' << b << ' ' << c << '\n';
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
using namespace std;
int main() {
    char harf{'D'};
    int puan{0};
    switch (harf) {
        case 'A': puan += 4;
        case 'B': puan += 3;
        case 'C': puan += 2; break;
        case 'D': puan += 1;
        default: puan += 10;
    }
    cout << puan << '\n';
    return 0;
}
```

C) Kod Yazma

9 Yaşı (`int`) ve öğrenci olup olmadığını (`e` ya da `h` karakteri) okuyan `biletFiyati` programını yaz. Yaş 0–120 dışındaysa `Geçersiz yaş!` yazıp bitsin. 6 yaşından küçükler için `Bilet: ücretsiz`; 17 yaş ve altı, 65 yaş ve üstü ya da öğrenci olanlar için `Bilet: 50 TL`; diğerleri için `Bilet: 100 TL` yazdır. Karar vermeden önce yaş denetimini yap.

```
Yaş ve öğrenci mi (e/h): 25 e
Bilet: 50 TL
```

10 Ay numarasını (1–12) ve yılı okuyan `gunSayisi` programını yaz. `switch` kullanarak o ayın kaç gün çektiğini `ay/yıl: N gün` biçiminde yazdır. Şubat için bu bölümdeki artık yıl kuralını kullan. Ay geçersizse `Geçersiz ay!` yazdır. Aynı sonucu veren ayları `break` yazmadan gruplayarak kod tekrarını azalt.

```
Ay ve yıl: 2 2024
2/2024: 29 gün
```

Çözümler

Cevap anahtarı

1	2	3	4	5
A	C	E	B	D

1 Cevap: A

Aralığın içinde olmak, iki koşulun **birlikte** sağlanmasıdır: `x >= 0 && x <= 100`. B'de `0 <= x` sonucu `false`/`true` (0 ya da 1) olur ve bu sayı `100`'den hep küçüktür; ifade her zaman doğrudur. C her `x` için doğrudur: Hiçbir sayı hem 0'dan küçük hem 100'den büyük olamaz. D uçları (0 ve 100) dışarıda bırakır. E'de `&&`'nin sağında işlenen eksiktir, derlenmez.

2 Cevap: C

Kısa devre: `x != 0 && ...` ifadesinde `x` sıfırsa sol taraf yanlıştır ve sağ taraf **hiç hesaplanmaz**. A ve E'de bölme en başta, solda yapılır. B'de sol taraf yanlış olunca `||` sağ tarafa geçer; D'de `x == 0` doğru olunca `&&` sağ tarafı hesaplar. Yani A, B, D ve E'de sıfıra bölme gerçekleşir.

3 Cevap: E

`case` etiketi derleme zamanında bilinen bir sabit olmalıdır. `3` ve `'a'` sabittir (`'a'` 97 değerinde bir tamsayıdır), `2 + 3` sabit bir ifadedir. `SINIR` ise sabitle ilk değer almış bir `const` tamsayıdır ve C++'ta (C'nin aksine) `case` etiketi olabilir; bu yüzden C yanlıştır. `sinir` sıradan bir **değişkendir**; g++ bunu "the value of 'sinir' is not usable in a constant expression" hatasıyla reddeder.

4 Cevap: B

`&&`, `||`'den önce işlenir: `(x > 3 && x < 5) || x == 5` → `(true && false) || true` → `true`. `!x` ise `!5` = `false`. `boolalpha` açık olduğu için ikisi de sözcük olarak yazılır (A yanlış). Program derlenir (E yanlış); g++ yalnız karışık `&&`/`||` için parantez uyarısı verir.

Çıktı

```
true false
```

C, `!x`'in `true` çıktığını; D ise ilk ifadenin `false` çıktığını varsayar.

5 Cevap: D

`if (x = 0)` karşılaştırma değil **atamadır**: `x`'e `0` atanır ve ifadenin değeri `0`, yani yanlış olur. `else` çalışır (`B`) ve `x` artık `0`'dır. g++ uyarır ama program derlenir (E yanlış). A ve C'de `if` bloğu çalışır sanılmıştır; B'de `x`'in değişmediği sanılmıştır.

Çıktı

```
B
0
```

6

Sarkan else: `else`, girintisine rağmen **en yakın `if`'e**, yani `b > 5`'e bağlanır.

1. `a > 5` → `2 > 5` yanlış. İç `if` ve ona bağlı `else` hiç değerlendirilmez; ne `X` ne `Y` yazılır.
2. `cout << "Z\n";` `if`'lerin dışında olduğu için her zaman çalışır.

Sık hata, `Y` de yazıldığını sanmaktır; `Y` yalnızca `a > 5` doğruyken ve `b > 5` yanlışken yazılırdı.

Çıktı
```
Z
```

7

Kısa devre ve değişken izleme (`a` hiç değişmez):

Adım	Açıklama	b	c
Başlangıç		1	0
1. `if`	`a > 5` yanlış → `++b` **atlanır**, blok çalışmaz	1	0
2. `if`	`a < 5` doğru → `||` kısa devre, `++b` **atlanır**, `c += 1`	1	1
3. `if`	`a > 1` doğru → `++b` çalışır, `b` = 2; `2 > 1` doğru, `c += 20`	2	21
4. `if`	`!b` yanlış, `b == 2` doğru, `c += 4`	2	25

Çıktı
```
3 2 25
```

8

`harf` `'D'` olduğundan program `case 'D'` etiketinin altından başlar:

1. `case 'D'`: `puan` 0 → 1. `break` yok, aşağı düşer.
2. `default`: `puan` 1 → 11. `switch` biter.
3. `case 'A'`, `'B'` ve `'C'` hiç çalışmaz.

`default` en sonda olduğu için ondan sonra düşülecek bir etiket yok; `case 'D'` ise `default`'a düşer.

Çıktı
```
11
```

9

Önce yaşın geçerliliği denetlenir. Sonra sıra önemlidir: `6`'dan küçükler ücretsiz olduğu için bu koşul en üste yazılır; kalanlarda `||` ile "indirimli mi?" sorulur. Girdi `25 e`: yaş 6 ile 17 arasında değil, 65'ten küçük, ama öğrenci.

biletFiyati.cpp
```cpp
#include <iostream>
using namespace std;

int main() {
    int yas{0};
    char ogrenci{'h'};
```

```
    cout << "Yaş ve öğrenci mi (e/h): ";
    cin >> yas >> ogrenci;

    if (yas < 0 || yas > 120) {
        cout << "Geçersiz yaş!\n";
    } else if (yas < 6) {
        cout << "Bilet: ücretsiz\n";
    } else if (yas <= 17 || yas >= 65 || ogrenci == 'e') {
        cout << "Bilet: 50 TL\n";
    } else {
        cout << "Bilet: 100 TL\n";
    }
    return 0;
}
```

Çıktı

```
Yaş ve öğrenci mi (e/h): 25 e
Bilet: 50 TL
```

10

31 ve 30 günlük aylar kendi içinde gruplanır; araya deyim girmeyen etiketler aynı kola düşer. Şubat kolunda artık yıl koşulu `?:` ile seçim yapar, geçersiz ayda `default` mesaj yazıp çıkar.

gunSayisi.cpp

```
#include <iostream>
using namespace std;

int main() {
    int ay{0};
    int yil{0};
    int gun{0};

    cout << "Ay ve yıl: ";
    cin >> ay >> yil;

    switch (ay) {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            gun = 31;
            break;
        case 4: case 6: case 9: case 11:
            gun = 30;
            break;
        case 2:
            gun = ((yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0) ? 29 : 28;
            break;
        default:
            cout << "Geçersiz ay!\n";
            return 0;
    }
    cout << ay << '/' << yil << ": " << gun << " gün\n";
    return 0;
}
```

Çıktı

```
Ay ve yıl: 2 2024
2/2024: 29 gün
```

BÖLÜM 5

Döngüler

Bu Bölümde Öğreneceklerin

- Tekrarlayan işleri yapan üç döngüyü ve hangisini ne zaman seçeceğini
- Döngü değişkeninin nerede yaşadığını ve ilk değer vermenin neden şart olduğunu
- Döngüyü erken bitirmeyi, bir turu atlamayı ve sonsuz döngüleri
- İç içe döngülerle yıldız ve sayı desenlerini
- İzleme tablosunu (trace table), off-by-one hatasını ve "kaç kez döner" sorusunu
- Klasik algoritmaları: faktöriyel, basamaklar, asal sayı, EBOB-EKOK, Fibonacci
- Bitiş işaretiyle okumayı, klavyeden yanlış girişi yakalayıp tekrar sormayı ve girdi bitene kadar okumayı

5.1 Neden döngü? while

1'den 1000'e kadar sayıları yazdırmak için bin tane `cout` satırı yazmazsın. **Döngü** (loop), bir kod bloğunu koşul doğru olduğu sürece tekrar tekrar çalıştırır. Tekrarlanan bloğa **döngü gövdesi** (loop body) denir. En yalın döngü `while`'dır:

```
while (koşul) {
    // koşul doğru olduğu sürece tekrarlanan gövde
}
```

Sıra şöyledir: koşul denetlenir; doğruysa gövde bir kez çalışır; başa dönülüp koşul yeniden denetlenir; yanlış olunca döngü biter. Bölüm 4'te gördüğün gibi `bool` koşul ve sıfırdan farklı her sayı "doğru" sayılır.

whileTemel.cpp

```
#include <iostream>

using namespace std;

int main() {
    int sayi{1};
    while (sayi <= 5) {
        cout << sayi << ' ';
        sayi++;
    }
    cout << "\nDöngü bitti, sayi = " << sayi << '\n';
    return 0;
}
```

Çıktı

```
1 2 3 4 5
Döngü bitti, sayi = 6
```

Her `while`'da üç parça aranır: **ilk değer** (`sayi{1}`), **koşul** (`sayi <= 5`) ve koşulu sonunda bozacak bir **değişiklik** (`sayi++`). Döngü bitince `sayi` 5 değil 6'dır: döngü, koşulu bozan ilk değerde durur.

`sayi++;` satırını unutursan koşul hep doğru kalır ve program sonsuza kadar `1 1 1 ...` yazar (terminalde **Ctrl+C** ile durdurulur). Her döngüde kendine sor: "Bu döngü hangi satırda biter?"

5.2 do-while

`while`'da koşul gövdeden **önce** denetlenir; baştan yanlışsa gövde hiç çalışmaz. `do-while`'da koşul gövdeden **sonra** denetlenir, yani gövde **en az bir kez** çalışır:

```
do {
    // en az bir kez çalışan gövde
} while (koşul);       // ← bu noktalı virgül ZORUNLU
```

Koşul baştan yanlışsa `while` gövdeyi 0 kez, `do-while` 1 kez çalıştırır. Klasik kullanım yeri **menü**dür: menü en az bir kez gösterilmeli, seçime göre tekrarlanmalıdır.

menu.cpp

```
#include <iostream>

using namespace std;

int main() {
    int secim{0};
    do {
        cout << "1) Merhaba  0) Çıkış: ";
        cin >> secim;
        if (secim == 1) {
            cout << "Merhaba!\n";
        }
    } while (secim != 0);
    cout << "Program bitti.\n";
    return 0;
}
```

Çıktı

```
1) Merhaba  0) Çıkış: 1
Merhaba!
1) Merhaba  0) Çıkış: 1
Merhaba!
1) Merhaba  0) Çıkış: 0
Program bitti.
```

Mavi satırlar senin klavyeden yazdıklarındır. Seçim gövdenin içinde okunduğu için `do-while` burada doğal seçimdir. Seçim kuralı: kaç kez döneceği belliyse `for`; koşula bağlıysa ve hiç dönmeyebilirse `while`; en az bir kez çalışmalıysa (menü, girdi doğrulama) `do-while`.

> **Sınavda Çıkar**
>
> "Bu kod kaç kez çalışır?" sorularında önce döngü türüne bak. Koşul baştan yanlışsa `while` ve `for` **0 kez**, `do-while` **1 kez** çalışır. `do-while`'ın sonundaki `;` unutulursa derleme hatası olur; "kodda hata var mı?" sorusunun cevabı çoğu kez budur.

5.3 for döngüsü

Sayaçlı işlerde ilk değer, koşul ve değişikliği tek satıra toplayan `for` kullanılır:

```
for (başlangıç; koşul; adım) {
    // gövde
}
```

Sıra: (1) **başlangıç** yalnız bir kez, döngüye girerken çalışır; (2) **koşul** denetlenir, yanlışsa döngü biter; (3) **gövde** çalışır; (4) **adım** çalışır ve 2'ye dönülür.

forTemel.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    for (int i{1}; i <= 5; i++) {
        cout << i << ' ';
    }
    cout << '\n';
    for (int i{10}; i >= 0; i -= 5) {
        cout << i << ' ';
    }
    cout << '\n';
    return 0;
}
```

Çıktı

```
1 2 3 4 5
10 5 0
```

Adım `i++` ile sınırlı değildir: `i--` geriye sayar, `i += 5` beşer atlar, `i *= 2` ikiye katlar (1, 2, 4, 8, ...). Virgülle birden fazla değişken de yönetilebilir: `for (int a{0}, b{10}; a < b; a++, b--)`.

for ile while dönüşümü. Her `for` bir `while`'a çevrilebilir: başlangıç döngünün **önüne**, adım gövdenin **sonuna** gider. `for (int i{1}; i <= 3; i++) { gövde }` ile `int i{1}; while (i <= 3) { gövde i++; }` aynı işi yapar. Aradaki iki fark kapsam ve `continue`'dur: kapsamı sıradaki başlık anlatıyor, `continue` farkını "break ve continue" başlığında göreceksin.

Döngü değişkeninin kapsamı

`for (int i{0}; ...)` ile tanımlanan `i` yalnızca bu `for`'un içinde yaşar. Bir değişkenin kullanılabildiği bölgeye **kapsam** (scope) denir; `i`'nin kapsamı döngüyle sınırlıdır ve döngü bitince `i` yok olur:

kapsamHata.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    for (int i{0}; i < 3; i++) {
        cout << i << ' ';
    }
    cout << "\nSon değer: " << i << '\n';
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
kapsamHata.cpp: In function 'int main()':
kapsamHata.cpp:9:32: error: 'i' was not declared in this scope
    9 |     cout << "\nSon değer: " << i << '\n';
      |                                ^
```

Değer döngüden sonra da lazımsa değişkeni **dışarıda** tanımla: `int i{0}; for (; i < 3; i++) { ... }`. Başlangıç kısmı boş bırakılabilir, ama `;` kalır. Ardışık iki `for` aynı `i` adını kullanabilir; her biri kendi `i`'sini yaratır (`forTemel`'de gördün). Gövdenin içinde tanımlanan değişken de her turda yeniden yaratılır ve tur bitince yok olur; bu yüzden bir sonraki tura bilgi taşımak istiyorsan değişkeni döngünün **dışında** tanımlamalısın.

> **Not: Apple clang ve hata mesajı**
>
> Kitaptaki derleme hataları gerçek g++ ile üretilir (`'i' was not declared in this scope`). macOS'ta `g++` yazınca çalışan Apple clang aynı hatayı `use of undeclared identifier 'i'` diye bildirir. Dil kuralı aynıdır, yalnız mesaj metni farklıdır.

5.4 Sayaç, toplam ve çarpım

Döngülerde üç tür değişken sürekli karşına çıkar. Sayaç bir şeyin kaç kez olduğunu sayar (`0` ile başlar, `sayac++;`), toplam değerleri biriktirir (`0` ile başlar, `toplam += x;`), çarpım değerleri çarpar (`1` ile başlar, `carpim *= x;`). Bu üç rol sınavlarda sürekli karşına çıkacak.

İlk değerler rastgele değildir: `0` toplamanın, `1` çarpmanın etkisiz elemanıdır. Çarpımı `0` ile başlatırsan sonuç hep `0` çıkar. Program, `n` için 1'den `n`'e **toplamı**, `n!` çarpımını ve çift sayıların sayısını bulur:

roller.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int n{0};
    cout << "n = ";
    cin >> n;

    int toplam{0};
    int carpim{1};
    int sayac{0};
    for (int i{1}; i <= n; i++) {
        toplam += i;
        carpim *= i;
        if (i % 2 == 0) {
            sayac++;
        }
    }
    cout << "1'den " << n << "'e toplam: " << toplam << '\n';
    cout << "Çarpım (" << n << "!): " << carpim << '\n';
    cout << "Çift sayı adedi: " << sayac << '\n';
    return 0;
}
```

```
Çıktı
n = 5
1'den 5'e toplam: 15
Çarpım (5!): 120
Çift sayı adedi: 2
```

Toplam için kapalı formül de vardır: `n * (n + 1) / 2` (n = 5 için 15). Çarpım `n` büyüyünce `int`'e sığmaz; bunu aşağıdaki faktöriyel başlığında göreceksin.

> **Sık Yapılan Hata: Toplam değişkenine ilk değer vermemek**
>
> `int toplam;` yazıp ilk değer vermeden döngüde `toplam += i;` dersen toplam çöp değere eklenir. İlk değer verilmemiş yerel değişkenin içinde bellekten kalma rastgele bir sayı (Bölüm 2'deki **çöp değer**) vardır; sonuç `15 + çöp` olur. Daha kötüsü, çöp bazen tesadüfen `0` çıkar: kod senin bilgisayarında doğru çalışır, sınavda yanlış sonuç verir. Bu tanımsız davranış olduğu için çıktısını göstermiyoruz. Kural: sayaç, toplam ve çarpım **tanımlandıkları satırda** ilk değerlerini alır: `int toplam{0};`.

5.5 break ve continue

`break`, içinde bulunduğu döngüyü **tamamen** bitirir. `continue` gövdenin **kalanını atlar** ve bir sonraki tura geçer. (`break`'i Bölüm 4'te `switch` içinde görmüştün; döngüde de aynı işi yapar: bulunduğu yapıdan çıkar.)

breakContinue.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    for (int i{50}; i <= 100; i++) {
        if (i % 7 == 0) {
            cout << "İlk 7'ye bölünen: " << i << '\n';
            break;
        }
    }
    for (int i{1}; i <= 10; i++) {
        if (i % 3 == 0) {
            continue;
        }
        cout << i << ' ';
    }
    cout << '\n';
    return 0;
}
```

```
Çıktı
İlk 7'ye bölünen: 56
1 2 4 5 7 8 10
```

İlk döngü 56'da bitti (57..100 hiç denenmedi); ikinci döngü 3, 6 ve 9'da `cout`'u atladı. İki kural: (1) `break` yalnızca **en içteki** döngüyü bitirir; (2) `for`'da `continue` yine de **adım kısmını** çalıştırır, `while`'da ise adım gövdede olduğu için atlanabilir:

```
int i{1};
while (i <= 5) {
    if (i == 3) {
        continue;       // i++ atlanır, i hep 3 kalır: SONSUZ DÖNGÜ
    }
    i++;
}
```

Sonsuz döngü ve break ile çıkış

Koşulu hep doğru olan döngüye **sonsuz döngü** (infinite loop) denir: `while (true)` ya da boş koşullu `for (;;)`. Hata değildir; çıkışı `break` ile yapılır ve kaç tur döneceği önceden bilinmeyen işlerde kullanılır:

```
int kuvvet{1};
while (true) {
    kuvvet *= 2;
    if (kuvvet > 1000) {
        break;          // 1024'te çıkılır
    }
}
```

`break` olmasaydı bu döngü sonsuza kadar dönerdi (hem de `kuvvet` taşardı). Sonsuz döngü yazdığında `break` yolunun gerçekten çalışacağından emin ol.

5.6 Noktalı virgül tuzağı

`for (...)` ya da `while (...)` satırının sonuna yanlışlıkla `;` koyarsan C++ bunu **boş bir gövde** sayar. Altındaki satır döngüden bağımsız, sıradan bir satır olur:

noktaliVirgul.cpp

```
#include <iostream>

using namespace std;

int main() {
    int i{0};
    for (i = 1; i <= 3; i++);
        cout << "Merhaba i=" << i << '\n';
    return 0;
}
```

Derleyici uyarısı – g++ -Wall -Wextra

```
noktaliVirgul.cpp: In function 'int main()':
noktaliVirgul.cpp:7:5: warning: this 'for' clause does not guard... [-Wmisleading-indentation]
    7 |     for (i = 1; i <= 3; i++);
      |     ^~~
noktaliVirgul.cpp:8:9: note: ...this statement, but the latter is misleadingly indented as if it
were guarded by the 'for'
    8 |         cout << "Merhaba i=" << i << '\n';
      |         ^~~~
```

Girinti insanı kandırır, derleyiciyi kandırmaz. Döngü boş gövdeyle 3 tur döndü (`i` 4 oldu), `cout` ise **bir kez** çalıştı:

Çıktı

```
Merhaba i=4
```

`while (i <= 3);` yazarsan `i` hiç değişmez ve program sonsuz döngüye girer (çalıştırmıyoruz).

> **Sık Yapılan Hata: Noktalı virgül nerede olur, nerede olmaz?**
>
> `for (...)` ve `while (...)` başlığından sonra `;` **konmaz**; `do-while`'ın sonunda ise `} while (...);` diye **konur**. Karıştırırsan bir döngü "bir kez çalışıyor" ya da "hiç ilerlemiyor" diye şaşırtır. Derleyici bu satırı her zaman uyarmayabilir; uyarıya güvenme, başlık satırının sonuna bak.

5.7 İç içe döngüler ve desenler

Bir döngünün gövdesine başka bir döngü yazabilirsin; buna **iç içe döngü** (nested loop) denir. Kural: dış döngünün **her turunda** iç döngü baştan sona çalışır. İç döngü bir **satırı**, dış döngü satırların **sayısını** üretir; satır sonundaki `'\n'` iç döngünün dışında, dış döngünün içindedir. `i` satır numarasıysa dik üçgende `i` yıldız vardır.

Sayı üçgeninde `*` yerine iç sayaç `j` yazdırılır. **Piramit**te satırda boşluk da vardır; desen sorularında önce her satır için "kaç boşluk, kaç yıldız" tablosunu kâğıda çiz. `n = 4` için: satır 1 → 3 boşluk, 1 yıldız; satır 2 → 2 boşluk, 3 yıldız; satır 3 → 1 boşluk, 5 yıldız; satır 4 → 0 boşluk, 7 yıldız. Yani satır `i`'de `n - i` boşluk ve `2i - 1` yıldız vardır. Üç deseni tek programda görelim:

desenler.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int n{4};
    cout << "Dik üçgen:\n";
    for (int i{1}; i <= n; i++) {
        for (int j{1}; j <= i; j++) {
            cout << '*';
        }
        cout << '\n';
    }
    cout << "Sayı üçgeni:\n";
    for (int i{1}; i <= n; i++) {
        for (int j{1}; j <= i; j++) {
            cout << j;
        }
        cout << '\n';
    }
    cout << "Piramit:\n";
    for (int i{1}; i <= n; i++) {
        for (int bosluk{1}; bosluk <= n - i; bosluk++) {
            cout << ' ';
        }
        for (int yildiz{1}; yildiz <= 2 * i - 1; yildiz++) {
            cout << '*';
        }
        cout << '\n';
    }
    return 0;
}
```

```
Çıktı
Dik üçgen:
*
**
***
****
Sayı üçgeni:
1
12
123
1234
Piramit:
   *
  ***
 *****
*******
```

Piramitte her satırda **iki ayrı iç döngü** art arda çalışır: önce boşluklar, sonra yıldızlar. Sınırları kâğıtta çıkardıysan döngüler kendiliğinden yazılır.

> **Sınavda Çıkar**
>
> "Bu kod kaç `*` yazdırır?" sorusunda iç gövdenin kaç kez çalıştığını say. İç sınır dış sayaca bağlıysa (`j <= i`) çarpma, tur başına değişen sayıları **topla**: 1 + 2 + 3 + 4 = 10. İç sınır sabitse (`j <= 4`) çarp: 3 dış tur × 4 = 12. İç döngüdeki `break` yalnızca iç döngüyü bitirir, dış döngü sürer.

5.8 İzleme tablosu ve off-by-one

Çıktı sorusunun en güvenilir çözümü **izleme tablosu**dur (trace table): her değişken için bir sütun açılır, her turdan sonra yeni değerler yazılır.

izleme.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int a{1};
    int b{10};
    int tur{0};
    while (a < b) {
        a += 2;
        b -= 1;
        tur++;
    }
    cout << "a=" << a << " b=" << b << " tur=" << tur << '\n';
    return 0;
}
```

Tur	Koşul `a < b`	`a`	`b`	`tur`
başlangıç		1	10	0
1	1 < 10 doğru	3	9	1
2	3 < 9 doğru	5	8	2
3	5 < 8 doğru	7	7	3
bitiş	7 < 7 yanlış	7	7	3

Çıktı
```
a=7 b=7 tur=3
```

> **İpucu: İzleme tablosunun kuralları**
>
> (1) İlk değerleri yaz. (2) Her tur başında **koşulu** yaz, doğru mu yanlış mı. (3) Gövdeyi sırayla uygulayıp değişkenleri güncelle. (4) Son satır koşulun yanlış çıktığı satırdır; döngü sonrası `cout`'u o satırdaki değerlerle çöz.

Off-by-one ("bir eksik/fazla") hatası, döngünün bir tur eksik ya da fazla dönmesidir; `<` ile `<=` karışıklığından doğar. Tur sayısı:

Döngü başlığı	Tur sayısı
`for (i = a; i < b; i++)`	`b - a` (`a <= b` iken; örnek: `i = 0; i < 5` → 5 tur)
`for (i = a; i <= b; i++)`	`b - a + 1` (`a <= b` iken; örnek: `i = 0; i <= 5` → 6 tur)
`for (i = 0; i < 10; i += 3)`	`i` = 0, 3, 6, 9 → 4 tur (adım k ise say)

1'den `n`'e toplarken `i <= n` yerine `i < n` yazarsan `n` toplama girmez: `n = 5` için 15 yerine 10 bulursun. Döngü sonrası değer de buradan çıkar: `for (i = 0; i < 5; i++)` biterken `i` **5**'tir (bu `i`, döngüden önce `int i;` ile tanımlanmıştır); `break` ile bitmiş döngüde `i`, `break`'in çalıştığı değerde kalır.

5.9 Klasik algoritmalar

Bunların hepsi sınavda kod yazma sorusu olarak çıkar. Bölüm 6'da fonksiyonlara dönüştüreceksin; şimdilik hepsi `main` içindedir. (1'den `n`'e toplam için `roller` programına bak.)

Faktöriyel ve int taşması

`n! = 1 · 2 · ... · n` ve `0! = 1` olduğundan çarpım `1` ile başlar. Faktöriyel hızla büyür: `12! = 479001600` bir `int`'e (en fazla 2147483647) sığar, `13!` sığmaz. İşaretli tamsayı taşması tanımsız davranıştır; bu yüzden `long long` kullanılır. `long long`'a `20!` sığar, `21!` sığmaz.

faktoriyel.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    long long f{1};
    for (int n{1}; n <= 20; n++) {
        f *= n;
        if (n % 5 == 0 || n == 12 || n == 13) {
            cout << n << "! = " << f << '\n';
        }
    }
    return 0;
}
```

Çıktı

```
5! = 120
10! = 3628800
12! = 479001600
13! = 6227020800
15! = 1307674368000
20! = 2432902008176640000
```

`13!`'in `long long`'da doğru göründüğüne dikkat et; `int` ile yapsaydın sonuç yanlış (ve tanımsız) olurdu.

Basamak sayısı, toplamı ve ters çevirme

Anahtar fikir: `n % 10` **son basamağı** verir, `n / 10` son basamağı **atar** (Bölüm 2'deki tamsayı bölmesi). Sayı 0 olana kadar bu iki işlem tekrarlanır:

basamaklar.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int n{0};
    cout << "Pozitif bir sayı: ";
    cin >> n;

    int kalan{n};
    int basamakSayisi{0};
    int basamakToplami{0};
    int ters{0};
    while (kalan > 0) {
        int son{kalan % 10};
        basamakSayisi++;
        basamakToplami += son;
        ters = ters * 10 + son;
        kalan /= 10;
    }
    cout << "Basamak sayısı: " << basamakSayisi << '\n';
    cout << "Basamak toplamı: " << basamakToplami << '\n';
    cout << "Ters: " << ters << '\n';
    return 0;
}
```

```
Çıktı
Pozitif bir sayı: 4712
Basamak sayısı: 4
Basamak toplamı: 14
Ters: 2174
```

`ters` değişkeni tur tur şöyle büyür: 2, 21, 217, 2174. Üç nokta: (1) `n` bozulmasın diye kopyası `kalan` üzerinde çalışıldı. (2) `n` sıfırsa `while` hiç dönmez ve basamak sayısı `0` çıkar, oysa `0` tek basamaklıdır; bunu `do-while` çözer. (3) Sondaki sıfırlar tersi kısaltır (`1200` → `21`); on basamaklı sayıların tersi `int`'e sığmayabilir.

Asal sayı testi

Asal sayı (prime number), 1 ve kendisinden başka böleni olmayan 1'den büyük sayıdır. Bölen aramak için `n`'e kadar gitmek gerekmez: `n = a · b` ise küçük çarpan `√n`'i geçemez, yani `i * i <= n` yeter. Bir **bayrak** (flag) değişkeni "bölen bulundu mu?" bilgisini taşır. Program 2'den `n`'e kadar asalları listeler:

asalListe.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int n{0};
    cout << "Üst sınır: ";
    cin >> n;

    for (int sayi{2}; sayi <= n; sayi++) {
        bool asal{true};
        for (int i{2}; i * i <= sayi; i++) {
            if (sayi % i == 0) {
                asal = false;
                break;
            }
        }
        if (asal) {
            cout << sayi << ' ';
        }
    }
    cout << '\n';
    return 0;
}
```

```
Çıktı
Üst sınır: 50
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

`break` iç döngüden çıkar: bölen bulununca o sayı için aramanın anlamı yok. `bool asal{true};` satırı **dış döngünün içinde** olmalıdır; dışarıya alırsan bir kez `false` olan bayrak bir daha `true` olmaz. (`i * i`, `int` sınırına yakın sayılarda taşabilir; kaygın varsa koşulu `i <= sayi / i` yaz.)

EBOB ve EKOK

Öklid algoritması (Euclidean algorithm): `a` ile `b`'nin **EBOB**'u (en büyük ortak bölen), `b` ile `a % b`'nin EBOB'u ile aynıdır; `b` sıfır olunca cevap `a`'dır. **EKOK** (en küçük ortak kat), EBOB'dan çıkar: `EKOK = a / EBOB * b` (bölmeyi önce yapmak `a * b` çarpımının taşma riskini azaltır).

ebobEkok.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int a{0}, b{0};
    cout << "İki sayı: ";
    cin >> a >> b;

    int x{a};
    int y{b};
    while (y != 0) {
        int kalan{x % y};
        x = y;
        y = kalan;
    }
    cout << "EBOB = " << x << '\n';
    cout << "EKOK = " << a / x * b << '\n';
    return 0;
}
```

Çıktı

```
İki sayı: 48 18
EBOB = 6
EKOK = 144
```

`(x, y)` çifti şöyle değişir: (48, 18) → (18, 12) → (12, 6) → (6, 0). `y` sıfır olunca döngü biter, EBOB `x`'te kalır.

> **Not: C++17: hazır gcd ve lcm**
>
> C++17'den beri `<numeric>` başlığında hazır `gcd(a, b)` ve `lcm(a, b)` fonksiyonları var (`gcd(48, 18)` 6, `lcm(48, 18)` 144 verir). Sınavda döngüyle yazman istenir; gerçek projede hazırını kullanırsın.

Fibonacci

Fibonacci dizisi `0, 1, 1, 2, 3, 5, ...` her terimi önceki iki terimin toplamı olan dizidir. `onceki`, `simdiki` ve geçici `sonraki` değişkenleri "kayar"; atama sırası bozulursa dizi bozulur:

fibonacci.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    long long onceki{0};
    long long simdiki{1};
    for (int i{1}; i <= 10; i++) {
        cout << onceki << ' ';
        long long sonraki{onceki + simdiki};
        onceki = simdiki;
        simdiki = sonraki;
    }
    cout << '\n';
    return 0;
}
```

Çıktı
```
0 1 1 2 3 5 8 13 21 34
```

Sıra önemlidir: önce `sonraki`, sonra `onceki`, en son `simdiki` güncellenir.

5.10 Kullanıcı girdisiyle döngüler

Sentinel: -1 girilene kadar

Kaç değer gireceği baştan belli değilse, girdinin bittiğini bildiren özel bir değer kararlaştırılır: **sentinel** (nöbetçi değer). Notlar 0-100 arasındaysa `-1` hiçbir not olamaz, bitiş işareti olabilir. Kalıp: değeri döngüden **önce** oku (**öncelikli okuma**, priming read), döngü sonunda **yeniden** oku:

ortalama.cpp
```cpp
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    int notu{0};
    int toplam{0};
    int adet{0};
    cout << "Notları gir (bitirmek için -1): ";
    cin >> notu;
    while (notu != -1) {
        toplam += notu;
        adet++;
        cin >> notu;
    }
    if (adet == 0) {
        cout << "Hiç not girilmedi.\n";
    } else {
        double ortalama{toplam / static_cast<double>(adet)};
        cout << adet << " not, ortalama: " << fixed << setprecision(2)
             << ortalama << '\n';
    }
    return 0;
}
```

Çıktı
```
Notları gir (bitirmek için -1): 70 85 90 -1
3 not, ortalama: 81.67
```

`static_cast<double>(adet)`, `adet`'i geçici olarak `double`'a çevirir (Bölüm 2); böylece bölme tamsayı bölmesi olmaz (245 / 3 = 81 değil, 81.67). `-1` ortalamaya katılmaz, çünkü veri değil işarettir. Hiç not girilmezse `adet` sıfır kalır; bölmeyi `if` ile engelliyoruz. Döngü içindeki okumayı atlarsan aynı değer sonsuz kez toplanır; baştaki okumayı atlarsan `notu`'nun başlangıç değeri olan `0` hem toplama hem adete girer. "İki `cin`" kalıbı bu yüzden vardır.

Girdi doğrulama: cin akış durumu

Kullanıcı yanlış girerse programın tekrar sorması gerekir; “en az bir kez sor” olduğu için `do-while` uygundur. Bölüm 3’te gördüğün gibi `cin >> sayi` harf görünce **hata durumuna** geçer: `failbit` açılır, `sayi`’ya `0` yazılır, harfler tamponda kalır ve akış onarılana kadar sonraki **hiçbir okuma çalışmaz**. Bu yüzden döngüyle tekrar sorarken akışı onarmazsan sonsuz döngüye girersin. Onarım yine Bölüm 3’teki iki adımdır ve sıra önemlidir:

1. `cin.clear();` hata durumunu temizler (önce bu gelir; hata durumundaki akışta `ignore` da çalışmaz).
2. `cin.ignore(numeric_limits<streamsize>::max(), '\n');` bozuk satırı atar (`<limits>` gerekir).

Yeni olan, bu ikiliyi bir döngünün içine koymak:

dogrulama.cpp

```cpp
#include <iostream>
#include <limits>

using namespace std;

int main() {
    int notu{0};
    bool gecerli{false};
    do {
        cout << "Notunu gir (0-100): ";
        if (!(cin >> notu)) {
            if (cin.eof()) {
                cout << "Girdi bitti.\n";
                return 1;
            }
            cout << "Sayı girmedin!\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
        } else if (notu < 0 || notu > 100) {
            cout << "Not 0-100 arasında olmalı!\n";
        } else {
            gecerli = true;
        }
    } while (!gecerli);
    cout << "Kabul edildi: " << notu << '\n';
    return 0;
}
```

Çıktı

```
Notunu gir (0-100): abc
Sayı girmedin!
Notunu gir (0-100): -5
Not 0-100 arasında olmalı!
Notunu gir (0-100): 150
Not 0-100 arasında olmalı!
Notunu gir (0-100): 85
Kabul edildi: 85
```

`abc` için okuma başarısız oldu, akış tamir edilip satır atıldı; `-5` ve `150` sayıdır ama aralık dışıdır (akış sağlam olduğu için `clear` gerekmedi); `85` kabul edilince `gecerli` doğru oldu ve `do-while` bitti. Girdi tümüyle biterse (dosyadan yönlendirmede ya da Ctrl+D ile) `cin.eof()` doğru olur; tamir etmenin anlamı yoktur, programı bitiririz.

Bölüm 3’teki yarım okuma tuzağını unutma: `12abc` girersen `cin` önce `12`’yi **başarıyla** okur, `abc` girdide kalır; hata bir sonraki `cin >> ...` satırında patlar.

Girdi bitene kadar oku: while (cin >> x)

`cin >> x` ifadesi bir koşulda kullanılırsa okuma başarılıysa doğru, başarısızsa yanlış sayılır. Böylece "okuyabildiğin sürece oku" döngüsü tek satır olur:

eofOku.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int x{0};
    int adet{0};
    int toplam{0};
    while (cin >> x) {
        toplam += x;
        adet++;
    }
    cout << adet << " sayı okundu, toplam " << toplam << '\n';
    return 0;
}
```

Çıktı

```
4 8 15
16 23 42
6 sayı okundu, toplam 108
```

Döngü, `cin` bir sayı okuyamayınca biter: girdi tükenince (**EOF**, *end of file*) ya da sayı yerine harf gelince. Klavyeden çalıştırırken girdiyi bitirmek için Windows'ta **Ctrl+Z** ve Enter, Linux ve macOS'ta satır başında **Ctrl+D** basılır.

Sık Yapılan Hata: while (!cin.eof()) kalıbı

"Girdi bitmediği sürece oku" mantıklı görünür ama yanlıştır. Son sayıyı okuduktan sonra akış henüz EOF'u görmemiştir (arkada satır sonu karakteri durur); koşul doğru kalır, gövde bir tur daha çalışır. Bu turda `cin >> x` başarısız olur; girdi bittiği için `x` değişmez ve son sayı (42) bir kez daha toplanır. Sayaç da artar: altı sayılık girdi için `adet` 7, `toplam` 150 çıkar (doğrusu 108). Doğru kalıp okumayı **koşulun içine** koymaktır: `while (cin >> x)`.

Sık Yapılan Hata: double sayaçla döngü kurmak

Ondalıklı sayılar bellekte ikilik tabanda **yaklaşık** tutulur; `0.1` tam yazılamaz. `double toplam{0.0};` değişkenine `0.1`'i on kez eklesen `1.0` değil `0.99999999999999989` çıkar; bu yüzden `for (double x{0.0}; x < 1.0; x += 0.1)` döngüsü beklenen 10 yerine **11** kez döner, `x != 1.0` koşullu olanı ise hiç bitmez. Kural: döngü sayacı **hep tamsayı** olsun; ondalık değer lazımsa `double x{i / 10.0};` gibi tamsayı sayaçtan hesapla. Geriye sayarken de sayacı `unsigned` yapma: `0`'ın bir eksiği en büyük sayıdır, `i >= 0` hep doğru kalır.

Bölüm Özeti – Bilmen Gerekenler

- `while` koşulu önce denetler (**0 kez** de dönebilir); `do { } while (koşul);` sonra denetler (**en az 1 kez**) ve sonunda `;` ister; `for (başlangıç; koşul; adım)` sayaçlı işlerde kullanılır.
- `for`'da sıra başlangıç (bir kez) → koşul → gövde → adım şeklindedir. `for (int i{0}; ...)` ile tanımlanan `i` yalnız döngüde yaşar; dışarıda kullanırsan derleme hatası olur. `for`, başlangıcı önüne, adımı gövde sonuna alınarak `while`'a çevrilir.

- Sayaç ve toplam `0` ile, çarpım `1` ile başlar; ilk değersiz değişkenin içinde **çöp** vardır (tanımsız davranış).
- `break` en içteki döngüyü bitirir, `continue` tur sonuna atlar (`while`'da adımı kendin ilerlet). `while (true)` ve `for (;;)` sonsuz döngüdür, `break` ile çıkılır.
- `for (...);` ve `while (...);` boş gövde demektir: `for` tek yazdırır gibi görünür, `while` sonsuz döngü olur. İç içe döngüde dış her turda iç döngü baştan çalışır; iç sınır dış sayaca bağlıysa tur sayıları **toplanır**, sabitse çarpılır.
- İzleme tablosu çıktı sorusunun anahtarıdır. `i < b` → `b - a` tur, `i <= b` → `b - a + 1` tur. Döngü bitince sayaç, koşulu bozan ilk değerdedir.
- Algoritmalar: basamak için `% 10` ve `/ 10`; asal için `i * i <= n` + bayrak; EBOB için Öklid; `EKOK = a / ebob * b`. Büyük faktöriyel ve Fibonacci için `long long` (`13!` `int`'e sığmaz, `20!` `long long`'a sığar).
- Sentinel döngüsünde değeri döngüden önce ve tur sonunda oku (öncelikli okuma). Yanlış girişte `cin` hata durumuna geçer: `cin.clear();` ve ardından `cin.ignore(numeric_limits<streamsize>::max(), '\n');` yaz. Girdi bitene kadar `while (cin >> x)` (EOF: Windows'ta Ctrl+Z, Linux/macOS'ta Ctrl+D); `!cin.eof()` kullanma.
- `double` sayaçla döngü kurma: `0.1` adım birikerek şaşar. Sayacı tamsayı yap; geriye sayan sayacı `unsigned` yapma.

Sorular

A) Çoktan Seçmeli

1 `for (int i{2}; i < 20; i += 4)` döngüsünün gövdesi kaç kez çalışır?

(A) 4 (B) 5 (C) 6 (D) 9 (E) 18

2 Döngülerle ilgili aşağıdaki ifadelerden hangisi **doğrudur**?

(A) `while` döngüsünde koşul, gövde bir kez çalıştıktan sonra denetlenir.

(B) `break`, yalnızca döngünün o turunun kalanını atlar ve bir sonraki tura geçer.

(C) `for (int i{0}; i < 3; i++)` ile tanımlanan `i`, döngü bittikten sonra da kullanılabilir.

(D) `for` döngüsünde `continue` çalışınca adım kısmı (`i++` gibi) yine de çalıştırılır.

(E) `do-while` döngüsünün gövdesi, koşul baştan yanlışsa hiç çalışmaz.

3 Aşağıdaki iç içe döngüde `say++` satırı toplam kaç kez çalışır?

```
for (int i{1}; i <= 4; i++) {
    for (int j{i}; j <= 4; j++) say++;
}
```

(A) 10 (B) 16 (C) 12 (D) 6 (E) 4

4 Program `int` beklerken kullanıcı `abc` yazdı ve `cin >> sayi` başarısız oldu. Döngüyle yeniden soracaksan, sonsuz döngüye girmemek için hangisi doğru çözümdür?

(A) Yalnızca `cin.ignore(numeric_limits<streamsize>::max(), '\n');` yazmak
(B) Yalnızca `cin.clear();` yazmak
(C) Önce `cin.ignore(numeric_limits<streamsize>::max(), '\n');`, sonra `cin.clear();` yazmak
(D) Hiçbir şey yapmadan yeniden `cin >> sayi` çağırmak
(E) Önce `cin.clear();`, sonra `cin.ignore(numeric_limits<streamsize>::max(), '\n');` yazmak

5 `n` pozitif bir tamsayıdır. Aşağıdaki döngülerden hangisinin gövdesi **tam `n` kez** çalışır?

(A) `for (int i{0}; i <= n; i++)`
(B) `for (int i{1}; i < n; i++)`
(C) `for (int i{1}; i <= n; i++)`
(D) `for (int i{0}; i < n - 1; i++)`
(E) `for (int i{1}; i <= n + 1; i++)`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
int toplam{0};
for (int i{10}; i > 0; i -= 3) {
    if (i == 4) {
        continue;
    }
    toplam += i;
    cout << toplam << ' ';
}
cout << "| " << toplam << '\n';
```

7 Aşağıdaki program ekrana ne yazdırır?

```
for (int i{1}; i <= 3; i++) {
    for (int j{1}; j <= 3; j++) {
        if (j == 2) {
            continue;
        }
        if (i == 3) {
            break;
        }
        cout << i * 10 + j << ' ';
    }
}
cout << '\n';
```

8 Aşağıdaki program ekrana ne yazdırır? Satır sayısına dikkat et.

```
int a{5};
int sayac1{0};
while (a-- > 2) {
    sayac1++;
}
cout << a << ' ' << sayac1 << '\n';

int b{5};
int sayac2{0};
while (--b > 2) {
    sayac2++;
}
cout << b << ' ' << sayac2 << '\n';
```

C) Kod Yazma

9 **Negatif bir sayı girilene kadar** tamsayı okuyan ve girilen sayıların toplamını ve içlerindeki çift sayıların adedini yazdıran `toplamCift` programını yaz. Negatif sayı veri değildir, yalnız bitiş işaretidir. Örnek girdi `8 3 10 7 -1` için beklenen çıktı:

```
Toplam: 28
Çift sayı adedi: 2
```

10 Kullanıcıdan pozitif bir `n` okuyup **sayısal kökünü** bulan `sayisalKok` programını yaz: sayının basamakları toplanır; sonuç hâlâ iki ya da daha fazla basamaklıysa işlem tek basamak kalana kadar tekrarlanır. Her adımı ekrana yaz. Örnek girdi `9875` için beklenen çıktı:

```
9875 -> 29 -> 11 -> 2
```

Çözümler

Cevap anahtarı

1 B	2 D	3 A	4 E	5 C

1 Cevap: B

`i` değerleri 2, 6, 10, 14, 18 olur (her tur 4 artar); sonraki değer 22, koşulu bozar. Beş değer olduğundan gövde **5** kez çalışır. A, 18'i saymayı unutmaktır; C altı değer sanmaktır; D adımı 2 sanıp 2, 4, ..., 18 saymaktır; E ise adımı hiç hesaba katmayıp `20 - 2` işleminin sonucunu yazmaktır.

2 Cevap: D

`for` döngüsünde `continue`, gövdenin kalanını atlar ama **adım kısmını** çalıştırır; ancak bundan sonra koşul denetlenir. A, `do-while`'ın özelliğini `while`'a yakıştırır; `while` koşulu önce denetler. B, `continue`'nun tanımıdır, `break` döngüyü bitirir. C yanlış: `for` içinde tanımlanan değişken döngünün kapsamındadır, dışarıda kullanılırsa derleme hatası olur. E yanlış: `do-while` gövdesi koşul baştan yanlış olsa bile bir kez çalışır.

3 Cevap: A

`i = 1` için iç döngü `j = 1..4` yani 4 kez, `i = 2` için 3 kez, `i = 3` için 2 kez, `i = 4` için 1 kez döner. İç başlangıç dış sayaca bağlı olduğundan sayılar **toplanır**: 4 + 3 + 2 + 1 = **10**. B (4 × 4) iç döngüyü sabit sanmaktır; C ve E rastgele çarpımlardır (3 × 4, 4); D ise `j = i + 1` başlangıcının sonucudur (3 + 2 + 1).

4 Cevap: E

Hata durumundaki akış hiçbir okumayı yapmaz, bu yüzden önce `cin.clear()` ile hata durumu temizlenir; sonra `cin.ignore(...)` bozuk satırı atar. A tek başına yetmez, çünkü `ignore` hata durumundaki akışta çalışmaz. B'de harfler girdide kalır, bir sonraki okuma yine onlara takılır. C'de sıra yanlıştır: `ignore` hata durumundayken bir şey yapmaz. D, akış bozuk olduğu için okuma yapmadan sonsuza kadar başarısız olur.

5 Cevap: C

Tur sayısı: `i < b` için `b - a`, `i <= b` için `b - a + 1`. C'de `1`'den `n`'e `<=` ile `n` tur olur. A'da 0'dan `n`'e `<=` ile `n + 1` tur, B'de 1'den `n`'e `<` ile `n - 1` tur, D'de `n - 1` tur, E'de `n + 1` tur vardır; hepsi off-by-one hatasıdır.

6

İzleme: `i = 10`: `toplam = 10`, `10` yazılır. `i = 7`: `toplam = 17`, `17`. `i = 4`: `continue`, ne toplanır ne yazılır. `i = 1`: `toplam = 18`, `18`. Sonraki `i = -2`, koşul yanlış. Döngü sonrası `| 18` yazılır.

Çıktı

```
10 17 18 | 18
```

7

`j == 2` iken `continue` yazdırmayı atlar. `i = 1`: `j = 1` için `11`, `j = 2` atlanır, `j = 3` için `13`. `i = 2`: `21` ve `23`. `i = 3`: `j = 1` iken `j == 2` değil, ama `i == 3` olduğundan `break` iç döngüyü bitirir; hiçbir şey yazılmaz. Satır sonu en son `cout << '\n'` ile gelir.

Çıktı
```
11 13 21 23
```

8

1. parça (son ek): `a-- > 2` önce `a`'nın eski değerini karşılaştırır, sonra azaltır. Eski değerler 5, 4, 3 için koşul doğru (`sayac1` 3 olur); eski değer 2 iken `2 > 2` yanlış, ama azaltma yine de yapıldığı için `a` 1 olur: `1 3`.

2. parça (ön ek): `--b > 2` önce azaltır, sonra karşılaştırır. `b` 4 ve 3 iken koşul doğru (`sayac2` 2 olur); `b` 2 olunca `2 > 2` yanlış ve döngü biter: `2 2`.

Çıktı
```
1 3
2 2
```

9

Kalıp sentinel okumadır: ilk sayı döngüden önce, sonrakiler tur sonunda okunur. Negatif sayı gelince koşul bozulur ve toplama girmez.

toplamCift.cpp
```cpp
#include <iostream>

using namespace std;

int main() {
    int sayi{0};
    int toplam{0};
    int ciftAdedi{0};
    cin >> sayi;
    while (sayi >= 0) {
        toplam += sayi;
        if (sayi % 2 == 0) {
            ciftAdedi++;
        }
        cin >> sayi;
    }
    cout << "Toplam: " << toplam << '\n';
    cout << "Çift sayı adedi: " << ciftAdedi << '\n';
    return 0;
}
```

Çıktı
```
8 3 10 7 -1
Toplam: 28
Çift sayı adedi: 2
```

10

Dış `while` sayı tek basamağa inene kadar döner (`n >= 10`). İçteki `while` basamakları `% 10` ve `/ 10` ile toplar. Her turda yeni değer `->` ile yazılır. Girdi baştan tek basamaklıysa dış döngü hiç dönmez ve yalnız sayının kendisi yazılır.

sayisalKok.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int n{0};
    cin >> n;
    cout << n;
    while (n >= 10) {
        int toplam{0};
        while (n > 0) {
            toplam += n % 10;
            n /= 10;
        }
        n = toplam;
        cout << " -> " << n;
    }
    cout << '\n';
    return 0;
}
```

Çıktı

```
9875
9875 -> 29 -> 11 -> 2
```

BÖLÜM 6

Fonksiyonlar

- **Bu Bölümde Öğreneceklerin**
  - Bir fonksiyonun parçalarını (adı, aldığı değerler, gövdesi, verdiği sonuç) ve hiçbir sonuç vermeyen fonksiyonları
  - Fonksiyonu kullanmadan önce derleyiciye haber vermeyi ve bunu unutursan alacağın hatayı
  - Çağrıda verdiğin değer ile fonksiyonun içindeki karşılığı arasındaki farkı; değer ile geçirmenin neden takas yaptırmadığını
  - Referans ile geçirmeyi: fonksiyonun çağıranın değişkenini doğrudan nasıl değiştirdiğini
  - Büyük verileri gereksiz kopyalamadan, ama değiştirilmeden geçirmeyi
  - Değer verilmezse kendiliğinden kullanılan değerleri ve aynı adlı birden çok fonksiyon yazmayı
  - Bir değişkenin programın hangi satırlarında göründüğünü ve çağrılar arasında değer saklamayı
  - Her fonksiyon çağrısının bellekte geçici bir alan açtığını
  - Hazır matematik fonksiyonlarını ve rastgele sayı üretmeyi
  - Klasik algoritmaları fonksiyona dönüştürmeyi

6.1 Neden fonksiyon?

Şimdiye kadar bütün kodu `main` içine yazdın. Program büyüdükçe bu işe yaramaz: aynı hesabı üç yerde yapman gerekir, `main` yüz satıra uzar, hata ararken nereye bakacağını bilemezsin. **Fonksiyon** (function), bir işi yapan ve adı olan kod parçasıdır. Bir kez yazarsın, istediğin kadar çağırırsın. Üç nedeni vardır:

- **Tekrarı önler.** Kopyala-yapıştır yerine fonksiyonu çağırırsın; hata varsa tek yerde düzeltirsin.
- **Okunabilirlik sağlar.** `if (asalMi(n))` satırı, yirmi satırlık bölen arama döngüsünden çok daha okunaklıdır.
- **Böl ve yönet.** Büyük problemi küçük parçalara ayırır, her parçayı ayrı yazar, ayrı denersin.

Aslında fonksiyon kullanıyorsun: `main` bir fonksiyondur, Bölüm 2'de kullandığın `sqrt` ve `pow` da öyle. **Kütüphane fonksiyonunu** (`sqrt`, `rand`) başkaları yazmıştır; bildirimi `#include` ettiğin başlık dosyasındadır, kodunu bağlayıcı programa ekler. **Kendi fonksiyonunu** (`asalMi`, `ebob`) sen yazarsın.

Sınıfların içindeki fonksiyonlara Bölüm 11'de **üye fonksiyon** diyeceğiz; bu bölümdekiler düz, bağımsız fonksiyonlardır.

6.2 Fonksiyonun anatomisi

Bir fonksiyon tanımı şu kalıba uyar: `dönüşTipi ad(parametreListesi) { gövde }`. Gövdenin içinde değeri geri veren `return` bulunur.

Parçalar `int kareAl(int x) { return x * x; }` için şunlardır: **dönüş tipi** (return type) `int`, çağırana geri verilen değerin tipidir; **ad** `kareAl`; **parametre listesi** `int x`, fonksiyonun dışarıdan aldığı değerlerdir (tip + ad); **gövde** (body) `{ ... }` içindeki deyimlerdir; `return` değeri çağırana verir ve fonksiyonu bitirir.

kareAl.cpp

```cpp
#include <iostream>

using namespace std;

int kareAl(int x) {
    return x * x;
}

int main() {
    int a = kareAl(5);
    int b = kareAl(a);
    cout << a << " " << b << '\n';
    cout << kareAl(2) + kareAl(3) << '\n';
    return 0;
}
```

Çıktı

```
25 625
13
```

Çağrıda şunlar olur: `main` durur, kontrol `kareAl`'a geçer, `x` parametresi `5` değerini alır, `return` değeri geri verir ve `main` kaldığı yerden devam eder. **Çağrı ifadesi (`kareAl(5)`), döndürdüğü değerin yerine geçer**; onu bir değişkene atayabilir, başka bir fonksiyona argüman yapabilir ya da `kareAl(2) + kareAl(3)` gibi işlemlerde kullanabilirsin.

- Fonksiyon adı değişken adı kurallarına uyar. Kitapta Türkçe ASCII camelCase kullanıyoruz ve adı **ne yaptığını söyleyen fiil ya da soru** seçiyoruz: `ortalamaHesapla`, `asalMi`, `menuyuGoster`.
- Fonksiyonun içinde başka bir fonksiyon **tanımlayamazsın**; tanımlar yan yana yazılır. (Fonksiyon içinde fonksiyon gibi davranan lambda ifadesini kitabın ilerisinde göreceksin.)
- Dönüş tipi `int`, `double`, `char`, `bool`, `string` gibi herhangi bir tip ya da `void` olabilir.

İyi fonksiyon **tek iş** yapar ve adı o işi anlatır: `asalMi` sayının asal olup olmadığına karar verir; ekrana yazmak ya da klavyeden okumak `main`'in işidir. "Hesaplayan fonksiyon yazdırmaz" kuralı fonksiyonu başka programlarda da kullanılır kılar.

6.3 void fonksiyonlar, dönüş değeri ve erken çıkış

Bazı fonksiyonlar değer üretmez, yalnızca bir iş yapar (ekrana yazar, menü gösterir). Bunların dönüş tipi **`void`**'dir (boş). `void` fonksiyonda `return deger;` yazılmaz; istersen yalnızca `return;` yazarak fonksiyonu erken bitirebilirsin. Sona gelince fonksiyon zaten kendiliğinden biter. Değer üretmediği için `void` fonksiyonun çağrısını bir değişkene **atayamazsın**: `int s = selamVer();` yazarsan g++ `void value not ignored as it ought to be` hatası verir.

`return` çalıştığı anda fonksiyon biter; altındaki satırlar çalışmaz. Bu yüzden özel durumları fonksiyonun başında ele alıp hemen çıkabilirsin:

erkenCikis.cpp

```cpp
#include <iostream>

using namespace std;

void bol(int pay, int payda) {
    if (payda == 0) {
        cout << "Sıfıra bölünemez!\n";
        return;
    }
    cout << pay << " / " << payda << " = " << pay / payda << '\n';
}

int main() {
    bol(17, 5);
    bol(8, 0);
    return 0;
}
```

Çıktı

```
17 / 5 = 3
Sıfıra bölünemez!
```

`bol(8, 0)` çağrısında `return;` bölmeyi atlattı; Bölüm 2'de tamsayıyı sıfıra bölmenin tehlikeli olduğunu görmüştün.

Değer döndüren bir fonksiyonun sonucunu kullanmak zorunda değilsin: `kareAl(7);` tek başına yazılırsa fonksiyon çalışır, değer sessizce atılır. Bir fonksiyon **en fazla bir değer** döndürür; birden fazla sonuç gerekirse bölümün ilerisinde göreceğin referans parametreleri kullanırsın.

> **Sık Yapılan Hata: Sonucu kullanmayı unutmak**
>
> `kareAl(x);` tek başına yazarsan `x` kare olmaz; hesaplanan değer atılır. Sonucu istiyorsan bir yere koy: `x = kareAl(x);`. Bu hata, fonksiyonun çağıranın değişkenini değiştirdiğini sanmaktan gelir; nedenini "Değer ile geçirme" başlığında göreceksin.

6.4 Bildirim (prototip)

g++ dosyayı yukarıdan aşağıya okur. Bir fonksiyon çağrısına geldiğinde o fonksiyonu **daha önce görmüş** olmalıdır. Fonksiyonu `main`'den sonra yazarsan sorun çıkar:

prototipsiz.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int sonuc = kareAl(5);
    cout << sonuc << '\n';
    return 0;
}

int kareAl(int x) {
    return x * x;
}
```

Derleyici çıktısı – program derlenmez

```
prototipsiz.cpp: In function 'int main()':
prototipsiz.cpp:6:17: error: 'kareAl' was not declared in this scope
    6 |     int sonuc = kareAl(5);
      |                 ^~~~~~
```

Fonksiyon kodda aşağıda **var**, ama g++ çağrı satırına geldiğinde henüz okumadı. İki çözüm vardır:

1. Fonksiyonu `main`'den **önce** tanımla. Küçük programlarda yeterlidir.
2. Fonksiyonu `main`'den önce **tanıt**: **prototip** (function prototype) yaz. Prototip, tanımın gövdesiz hâlidir ve `;` ile biter: "ad şu, dönüş tipi şu, parametreler şunlar; tanımı aşağıda". Buna **bildirim** (declaration) de denir; gövdeli hâli **tanım** (definition) olur.

prototipli.cpp

```cpp
#include <iostream>

using namespace std;

int kareAl(int x);
double ortalama(int a, int b);

int main() {
    cout << kareAl(6) << '\n';
    cout << ortalama(3, 4) << '\n';
    return 0;
}

int kareAl(int x) {
    return x * x;
}

double ortalama(int a, int b) {
    return (a + b) / 2.0;
}
```

Çıktı

```
36
3.5
```

Önce prototipler, sonra `main`, en altta tanımlar: sınavlarda en çok görülen düzen budur, çünkü dosyayı açan kişi ilk olarak programın akışını okur. Prototip kuralları:

- Dönüş tipi, ad ve parametre **tipleri** tanımla birebir aynı olmalıdır. Parametre **adları** serbesttir ve yazılmayabilir: `int kareAl(int);` da geçerlidir.
- `#include <iostream>` satırı, `cout` ve benzerlerinin bildirimlerini programına ekler. Başlık dosyası, bildirimler topluluğudur.
- C'den farklı olarak C++'ta `int menu();` ile `int menu(void);` aynı anlama gelir: parametresiz fonksiyon. Kitapta boş parantez kullanıyoruz.

Prototip ile tanım uyuşmazsa g++ şikâyet eder. Parametre listesi aynı, yalnızca dönüş tipi farklıysa (prototipte `int alan(int, int);`, tanımda `double alan(int en, int boy)`) mesaj `ambiguating new declaration of 'double alan(int, int)'` olur. Parametre sayısı ya da tipi farklıysa bu iki bildirim kendi başına hata vermez; çünkü o zaman "aşırı yükleme" başlığında göreceğin gibi iki ayrı fonksiyon sayılırlar. Hata çağrıda ortaya çıkar. Çağrı prototiple eşleşiyorsa program derlenir, ama tanımı olmayan prototip için bağlama hatası alırsın. Çağrı yalnız

aşağıdaki tanımla eşleşiyorsa (prototip `double ortalama(int a);`, çağrı `ortalama(3, 4)`) derleyici o satırda henüz yalnız prototipi gördüğü için bağlamaya hiç geçmeden `too many arguments to function` derleme hatası verir.

> **Not: Mesajlar derleyiciye göre değişir**
>
> macOS'ta `g++` komutu aslında Apple clang'dir; kitaptaki çıktılar gerçek g++ ile üretildi. Hata aynıdır ama metin farklıdır. Prototipsiz çağrıda clang `use of undeclared identifier 'kareAl'`, dönüş tipi çakışmasında `functions that differ only in their return type cannot be overloaded` der. Visual Studio ise `C3861: identifier not found` yazar. Hatanın nedenini ayırt etmeyi öğren, birebir metni ezberleme.

6.5 Parametre ve argüman

Terim	Nerede?	Örnek
Parametre (parameter)	Fonksiyonun tanımında, parantez içinde	`int topla(int a, int b)`: `a` ve `b`
Argüman (argument)	Fonksiyonun çağrısında	`topla(5, y + 1)`: `5` ve `y + 1`

Her parametrenin tipi **ayrı ayrı** yazılır: `int topla(int a, int b)` doğrudur, `int topla(int a, b)` değil (g++ `'b' has not been declared` der). Tip yalnızca tanımda ve prototipte yazılır, çağrıda yazılmaz. Argümanlar parametrelere **sırayla** eşlenir. Parametreler fonksiyonun içinde birer **yerel değişken** gibi davranır ve çağrıda argümanların değeriyle başlatılır. Argüman sayısı da parametre sayısına eşit olmalıdır (varsayılan parametreler hariç, onları aşağıda göreceksin): `topla(int a, int b)` fonksiyonunu `topla(5)` diye çağırırsan g++ `too few arguments to function 'int topla(int, int)'` hatası verir ve fonksiyonun tanımlandığı yeri de gösterir (`note: declared here`). Fazla argümanda mesaj `too many arguments to function` olur.

Tip dönüşümü

Prototip ya da tanım görünürken argüman, parametrenin tipine **kendiliğinden** çevrilir; `return` ile döndürülen değer de dönüş tipine çevrilir. Bölüm 2'deki kurallar geçerlidir: `int` → `double` kayıpsızdır (`double` parametreli fonksiyona `5` verirsen `5.0` olur), `double` → `int` ondalık kısmı **atar** (sıfıra doğru).

İki örnek: `int tamKisim(double x) { return x; }` fonksiyonu `7.9` için `7`, `-7.9` için `-7` döndürür (yuvarlama değil, kesme); süslü parantezli ilk değer vermedeki daralma koruması burada **yoktur**, sessizce veri kaybedersin. İkincisi bir **tuzaktır**: `double ortalama(int a, int b) { return (a + b) / 2; }` fonksiyonu `ortalama(3, 4)` için `3` verir, çünkü `(a + b) / 2` iki `int`'in bölmesidir (`7 / 2 = 3`) ve ancak sonra `double`'a çevrilir. Dönüş tipinin `double` olması bölmeyi `double` yapmaz; `/ 2.0` yazmalısın (`prototipli` programındaki `ortalama` böyle yazıldı ve `3.5` verdi).

6.6 Değer ile geçirme

Bu bölümün en önemli kuralı: varsayılan olarak argümanlar fonksiyona **değer ile** (pass by value) geçirilir. Fonksiyon argümanın **kopyasını** alır; parametreyi fonksiyon içinde ne kadar değiştirirsen değiştir, çağıran fonksiyondaki değişken aynı kalır. Günlük benzetme: bir belgenin fotokopisine not almak asıl belgeyi değiştirmez. Bu kuralın en meşhur sonucu, iki değişkenin değerini değiştirmeyi (takas, swap) deneyen fonksiyondur:

takasCalismaz.cpp

```cpp
#include <iostream>

using namespace std;

void takas(int a, int b) {
    int gecici = a;
    a = b;
    b = gecici;
    cout << "takas içinde: a = " << a << ", b = " << b << '\n';
}

int main() {
    int x{3};
    int y{8};
    takas(x, y);
    cout << "main içinde : x = " << x << ", y = " << y << '\n';
    return 0;
}
```

Çıktı

```
takas içinde: a = 8, b = 3
main içinde : x = 3, y = 8
```

Takas `a` ve `b` kopyaları üzerinde başarıyla yapıldı, ama `x` ve `y` yerinde durdu: fonksiyon bitince kopyalar yok oldu ve işin hiçbir izi kalmadı. Parametre adlarını `x` ve `y` yapsan da değişmez; aynı adlı iki değişken yine **iki ayrı değişkendir**.

Değer ile geçirmenin kötü olduğunu sanma: `ebob` gibi fonksiyonlarda parametreyi rahatça değiştirip eritebilirsin, çağıranın değişkeni bozulmaz. Ama çağıranın değişkenini **değiştirmek** istiyorsan başka bir yol gerekir.

6.7 Referans ile geçirme

Referans (reference), var olan bir değişkene verilen ikinci bir addır; takma ad gibi düşün. Tanımlarken tipin yanına `&` yazılır ve mutlaka bir değişkene bağlanarak başlatılır.

Örneğin `int x{10}; int &r = x;` yazdıktan sonra `r = 20;` dersen `x` de `20` olur, sonra `x++;` dersen `r` de `21` görünür: ikisi aynı kutunun iki adıdır.

Referans bir kez bağlanır, sonra başka değişkene bağlanamaz (`r = y;` yazmak `r`'yi `y`'ye bağlamaz, `y`'nin değerini `x`'e kopyalar). Referansları işaretçilerle karşılaştırarak Bölüm 9'da ayrıntılı işleyeceğiz; bu bölümde yalnızca en yaygın kullanımına ihtiyacın var: **parametrede**.

Parametreyi `int &a` diye yazarsan fonksiyon argümanın kopyasını değil **kendisini** alır; `a` çağıran fonksiyondaki değişkenin takma adı olur. Buna **referans ile geçirme** (pass by reference) denir. Takas artık çalışır:

takasReferans.cpp

```cpp
#include <iostream>

using namespace std;

void takas(int &a, int &b) {
    int gecici = a;
    a = b;
    b = gecici;
}

int main() {
    int x{3};
    int y{8};
    takas(x, y);
    cout << "x = " << x << ", y = " << y << '\n';
    return 0;
}
```

Çıktı

```
x = 8, y = 3
```

Fonksiyonun gövdesi değişmedi; yalnızca parametrelere `&` eklendi. Çağrı satırı `takas(x, y)` da aynı: dikkat, çağrıya bakarak bir parametrenin referans olup olmadığını **anlayamazsın**; prototipe bakman gerekir. C++ standart kitaplığında bu fonksiyon hazırdır (`swap(x, y)`, `<utility>` başlığı); kitap boyunca elle yazdığımızda öğrenme amacıyla yazıyoruz.

Referans parametreye **değişken** vermelisin; bir sabit ya da hesap sonucu veremezsin, çünkü değiştirilecek bir kutu yoktur. `takas(x, 5)` yazarsan g++ `cannot bind non-const lvalue reference of type 'int&' to an rvalue of type 'int'` hatası verir.

Birden fazla sonuç döndürmek

`return` yalnızca bir değer taşır. Fonksiyondan iki sonuç gerekiyorsa (bölme işleminde hem bölüm hem kalan gibi), sonucu koyacağı kutuları referans parametre olarak ver. Bunlara **çıkış parametresi** (output parameter) denir:

cikisParametresi.cpp

```cpp
#include <iostream>

using namespace std;

void bolmeYap(int bolunen, int bolen, int &bolum, int &kalan) {
    bolum = bolunen / bolen;
    kalan = bolunen % bolen;
}

int main() {
    int b{0};
    int k{0};
    bolmeYap(47, 5, b, k);
    cout << "47 / 5 = " << b << ", kalan " << k << '\n';
    return 0;
}
```

Çıktı

```
47 / 5 = 9, kalan 2
```

Girdi parametreleri (`bolunen`, `bolen`) değer ile, çıktı parametreleri (`bolum`, `kalan`) referansla geçti. Bu düzen sınavlarda sık görülür. İleride (Bölüm 11'de sınıflar, Bölüm 15'te `pair`) birden çok değeri tek pakette döndürmenin başka yollarını da öğreneceksin.

Sık Yapılan Hata: Yerel değişkene referans döndürmek

Fonksiyon bitince yerel değişkenlerin çerçevesi silinir (aşağıda "Çağrı yığını" başlığında göreceksin). Bu yüzden yerel değişkenin referansını geri vermek, ortadan kalkmış bir kutuyu göstermektir:

```
int &hatali() {
    int x{5};
    return x;       // x fonksiyonla birlikte yok olur: tanımsız davranış
}
```

g++ bunu `-Wall` ile yakalar (`reference to local variable 'x' returned`). Program çökebilir, çöp değer yazabilir ya da tesadüfen çalışıyormuş gibi görünebilir. Çıktısını göstermiyoruz. Kural: referansla yalnızca fonksiyon bittikten sonra da yaşayan bir şeyi geri ver; başlangıç seviyesinde `return` ile **değer** döndürmek en güvenlisidir.

6.8 const & ile geçirme

Değer ile geçirme küçük tiplerde (`int`, `double`, `char`, `bool`) hızlı ve zararsızdır. `string` gibi büyük nesnelerde ise her çağrıda koca bir kopya çıkar. Referans ile geçirirsen kopya çıkmaz, ama bu kez fonksiyon asıl nesneyi yanlışlıkla değiştirebilir. Çözüm, referansı `const` yapmaktır: **sabit referans** (const reference). Kopya yok, değiştirme de yok.

`string`'i Bölüm 8'de ayrıntılı göreceksin; şimdilik "metin tutan tip" olarak al.

sabitReferans.cpp

```
#include <iostream>
#include <string>

using namespace std;

void selamla(const string &ad) {
    cout << "Merhaba " << ad << "!\n";
}

int main() {
    string isim{"Ayşe"};
    selamla(isim);
    selamla("Mehmet");          // sabit değer de verilebilir
    return 0;
}
```

Çıktı

```
Merhaba Ayşe!
Merhaba Mehmet!
```

`const &` parametreye sabit değer de verilebilir (`&` tek başınayken `takas(x, 5)` derlenmiyordu). Fonksiyon değiştiremeyeceği için güvenlidir; `selamla` içinde `ad = "Ali";` yazmaya kalkarsan derleyici seni durdurur. `const int &n` gibi temel tipte bir parametrede mesaj kısadır (`n = n + 1;` için `assignment of read-only reference 'n'`); `string`'de ise `no match for 'operator='` ile başlayan uzun bir mesaj gelir. İkisinde de anlam aynıdır: salt okunur olanı değiştiremezsin.

> **İpucu: Dört soruyla parametre tipi**
>
> Parametre için kendine sor: (1) Fonksiyon bu değeri **değiştirip çağırana yansıtacak mı?** Evet ise `&`. (2) Hayır, ama tip büyük mü (`string`, `vector`; ikisini Bölüm 7 ve 8'de göreceksin)? Evet ise `const &`. (3) Küçük bir sayı mı? Düz değer. (4) Fonksiyon kendi içinde bir kopyayı değiştirip kullanacak mı? Düz değer. Sınavda "neden `const &`?" sorusuna cevap: kopyalamamak için ve değiştirmeyi engellemek için.

6.9 Varsayılan parametreler

Bazı parametrelerin çoğu çağrıda aynı değeri alır. Bu parametrelere her seferinde değer yazmak yerine **varsayılan parametre** (default argument) tanımlayabilirsin; çağrıda o argüman atlanırsa varsayılan kullanılır:

varsayilan.cpp

```cpp
#include <iostream>
#include <string>

using namespace std;

double kdvliFiyat(double fiyat, double oran = 0.20) {
    return fiyat * (1 + oran);
}

void cizgiCiz(int uzunluk = 10, char karakter = '-') {
    cout << string(uzunluk, karakter) << '\n';   // string: Bölüm 8
}

int main() {
    cout << kdvliFiyat(100) << '\n';
    cout << kdvliFiyat(100, 0.10) << '\n';
    cizgiCiz();
    cizgiCiz(5);
    cizgiCiz(8, '*');
    return 0;
}
```

Çıktı

```
120
110
----------
-----
********
```

Kurallar:

- Varsayılan değerler **sağdan sola** verilir: bir parametrenin varsayılanı varsa onun sağındaki hepsinin de olmalıdır.
- Argümanlar soldan sağa eşlenir; ortadakini atlayıp sondakini veremezsin. `cizgiCiz('*')` yazarsan `'*'` karakteri `uzunluk` parametresine (`int`) gider, `42` tire çizer; yalnızca `karakter`'i vermenin yolu yoktur.
- Varsayılan değer fonksiyonun derleyicinin **ilk gördüğü** bildiriminde yazılır. Yukarıdaki programda prototip yok, fonksiyonun ilk görünen hâli tanımın kendisi; bu yüzden varsayılanlar tanımda. Prototip varsa varsayılan prototipe yazılır ve tanımda tekrarlanmaz.

`int topla(int a = 1, int b)` yazarsan g++ `default argument missing for parameter 2` hatası verir.

Prototip kullanıyorsan tanımda aynı varsayılanı yeniden yazarsan da hata alırsın (`default argument given for parameter 2`). Doğru düzen: `double kdvliFiyat(double fiyat, double oran = 0.20);` prototipte, tanımda yalnızca `double kdvliFiyat(double fiyat, double oran) { ... }`.

6.10 Fonksiyon aşırı yükleme

İki sayının büyüğünü bulan fonksiyon yazalım. `int` için `enBuyukInt`, `double` için `enBuyukDouble` demek zorunda mısın? C++'ta hayır: **aynı ada sahip, parametre listeleri farklı** birden çok fonksiyon tanımlayabilirsin. Buna **aşırı yükleme** (overloading) denir. Derleyici çağrıdaki argümanlara bakarak hangisini çağıracağını seçer:

asiriYukleme.cpp

```cpp
#include <iostream>

using namespace std;

int enBuyuk(int a, int b) {
    return a > b ? a : b;
}

double enBuyuk(double a, double b) {
    return a > b ? a : b;
}

int enBuyuk(int a, int b, int c) {
    return enBuyuk(enBuyuk(a, b), c);
}

int main() {
    cout << enBuyuk(3, 8) << '\n';
    cout << enBuyuk(2.5, 1.5) << '\n';
    cout << enBuyuk(4, 9, 6) << '\n';
    return 0;
}
```

Çıktı

```
8
2.5
9
```

Fonksiyonları birbirinden **parametre listesi** (sayı, tip, sıra) ayırır; bunun toplamına fonksiyonun **imzası** (signature) denir. Dönüş tipi imzaya girmez.

Yalnızca dönüş tipi farklı iki fonksiyon tanımlanamaz: `int yarisi(int x)` ile `double yarisi(int x)` birlikte yazılırsa g++ `ambiguating new declaration of 'double yarisi(int)'` der (çünkü `yarisi(7)` çağrısında hangisinin istendiği belli olmaz).

Çağrıda tam uyan bir fonksiyon yoksa derleyici şu sırayla en iyi adayı arar: (1) **tam eşleşme**, (2) küçük tipten büyüğe **terfi** (`char`, `short`, `bool` → `int`; `float` → `double`), (3) diğer **dönüşümler** (`int` → `double` gibi). Aynı düzeyde iki aday varsa çağrı **belirsizdir** ve derlenmez. Önce `int` ve `double` ile yüklenmiş `yaz` fonksiyonlarına terfinin etkisine bak:

yuklemeSecimi.cpp

```cpp
#include <iostream>

using namespace std;

void yaz(int x) {
    cout << "int: " << x << '\n';
}

void yaz(double x) {
    cout << "double: " << x << '\n';
}

int main() {
    yaz(5);
    yaz(5.5);
    yaz('A');           // char -> int terfisi, double'a dönüşümden iyidir
    yaz(2.5f);          // float -> double terfisi
    yaz(true);          // bool -> int terfisi
    return 0;
}
```

Çıktı

```
int: 5
double: 5.5
int: 65
double: 2.5
int: 1
```

`yaz('A')` karakteri değil `65` sayısını yazdırır: elindeki seçenekler `int` ve `double`, ve `char` → `int` terfi sayıldığından o kazanır. Şimdi iki adayın da dönüşüm gerektirdiği bir çağrı:

Aynı iki fonksiyonla `yaz(5L)` (bir `long`) çağırırsan `long`, ne `int`'e ne `double`'a tam uyar; ikisine de dönüşüm gerekir ve ikisi eşit iyidir. g++ `call of overloaded 'yaz(long int)' is ambiguous` hatası verir. Çözüm, çağrıda açık dönüşüm yapmaktır: `yaz(static_cast<int>(5L))`. Varsayılan parametre de belirsizlik yaratabilir: `f(int a)` ile `f(int a, int b = 0)` birlikte tanımlıysa `f(1)` çağrısı belirsizdir.

> **Sınavda Çıkar**
>
> Aşırı yükleme sorularında üç kural istenir: (1) imzayı **parametre listesi** belirler, dönüş tipi değil; (2) seçim sırası tam eşleşme, terfi, dönüşümdür; (3) eşit iyi iki aday varsa **belirsiz çağrı** hatasıdır. `char` argümanın `int` ve `double` seçenekleri arasında `int`'e gittiğini bilmen beklenir.

6.11 Çağrı yığını

Değer ile geçirmenin arkasında bir mekanizma vardır. Program çalışırken her fonksiyon çağrısı için bellekte bir **yığın çerçevesi** (stack frame) açılır. Çerçevede fonksiyonun parametreleri, yerel değişkenleri ve fonksiyon bitince nereye dönüleceği bilgisi bulunur. Çerçeveler **çağrı yığını** (call stack) denen bölgede üst üste dizilir; fonksiyon `return` edince çerçevesi **silinir**. Yerel değişkenler ve parametreler bu yüzden fonksiyon bitince yok olur ve aynı adlı değişkenler birbirini etkilemez: her çağrının kendi çerçevesi, yani kendi kopyaları vardır.

Aşağıdaki programda `main`, `karelerToplami`'nı, o da `kare`'yi çağırır:

cagriYigini.cpp

```cpp
#include <iostream>

using namespace std;

int kare(int x) {
    return x * x;
}

int karelerToplami(int a, int b) {
    int ka = kare(a);
    int kb = kare(b);
    return ka + kb;
}

int main() {
    int sonuc = karelerToplami(3, 4);
    cout << "Sonuç: " << sonuc << '\n';
    return 0;
}
```

Çıktı

```
Sonuç: 25
```

Programın baştan sona yığın hareketi (`?`: değişken tanımlanmış ama henüz değer atanmamış; her satırda ilk yazılan çerçeve en üsttedir, yani o an çalışandır):

Adım	Olay	Yığın (üstten alta)
1	`main` başlar	`main` (sonuc=?)
2	`karelerToplami(3, 4)` çağrılır	`karelerToplami` (a=3, b=4, ka=?, kb=?) / `main`
3	`kare(3)` çağrılır	`kare` (x=3) / `karelerToplami` / `main`
4	`kare` 9 döndürür, çerçevesi silinir; `ka = 9`	`karelerToplami` (a=3, b=4, ka=9, kb=?) / `main`
5	`kare(4)` çağrılır, 16 döner; `kb = 16`	`kare` (x=4) / `karelerToplami` / `main`; sonra yine `karelerToplami` / `main`
6	`karelerToplami` 25 döndürür, çerçevesi silinir; `sonuc = 25`	`main` (sonuc=25)

Üç şeye dikkat et. (1) Aynı `kare` fonksiyonu 3. ve 5. adımlarda iki ayrı çerçeve açtı; `x` her seferinde sıfırdan oluştu. (2) Çerçevedeki `a` ve `b`, `main`'deki değerlerin kopyasıdır. Referans parametre olsaydı çerçevede kopya değil, çağıranın kutusuna giden bir bağlantı bulunurdu. (3) Çağıran fonksiyon, çağırdığı bitene kadar **bekler**.

`birEkle(ikiKati(5))` gibi iç içe çağrılarda (`ikiKati` sayının iki katını, `birEkle` bir fazlasını versin) dıştaki fonksiyonun argümanı içteki çağrının sonucudur, bu yüzden **içteki önce çalışır**: `ikiKati(5) = 10`, sonra `birEkle(10) = 11`. Sıra sonucu değiştirir: `ikiKati(birEkle(5))` `12` verir. Sınavda `f(g(x))` gördüğünde içten dışa doğru tek tek yaz. Ancak `f(g(x), h(y))` çağrısında `g` ile `h`'den hangisinin önce çalışacağını C++ belirlemez; aynı çağrıda yan etkisi olan (ekrana yazan, global değişkeni değiştiren) iki çağrıyı argüman yapma.

Fonksiyon kendi kendini de çağırabilir; buna özyineleme denir ve her çağrı kendi çerçevesini açtığı için çalışır. Durma koşulu olmazsa çerçeveler sonsuza kadar açılır ve yığın (sınırlı bir bölgedir) dolar. Özyinelemeyi Bölüm 17'de ayrıntılı işleyeceğiz.

6.12 Kapsam

Bir değişkenin **kapsamı** (scope), programın hangi bölgelerinden o değişkene erişilebildiğini belirler. Fonksiyonun içinde tanımlanan değişken **yereldir** (local): yalnızca tanımlandığı satırdan sonra, o fonksiyonda görünür ve fonksiyon bitince yok olur; parametreler de yereldir. `hesapla` içinde tanımlı `sonucDegeri`'ni `main`'de kullanırsan g++ `'sonucDegeri' was not declared in this scope` der. Değeri çağırana `return` ile (ya da referans parametreyle) göndermelisin. Yerel değişkene ilk değer vermezsen içinde önceden kalmış **çöp değer** bulunur ve okumak tanımsız davranıştır (Bölüm 2); her yerel değişkene değer ver.

Blok kapsamı ve gölgeleme

`{ }` ile oluşturulan her **blok** kendi kapsamını açar. `for` döngüsünün sayacı da yalnızca döngünün içinde yaşar. İç blokta dış bloktaki bir değişkenle **aynı adı** kullanırsan iç olan dış olanı **gölgeler** (shadowing): blok boyunca o adın altında iç değişken görünür, blok bitince dış olan yeniden görünür.

Örneğin `main` içinde `int x{10};` yazılmışsa ve bir iç blokta `int x{20};` tanımlanıp yazdırılıyorsa iç blokta `20`, blok bittikten sonra yine `10` görürsün (bölümün sonundaki 8. soruda bu tuzağı çözeceksin).

İç bloktaki `x` dıştaki `x`'i **değiştirmez**; ayrı, yeni bir değişkendir. Blok bitince içteki değişkenler yok olur. Gölgeleme geçerlidir ama okunması zordur; kendi kodunda kaçın, sınavda ise bu soruyu hazır bekle. Tek sınır: fonksiyonun **en dış bloğunda** parametreyle aynı adı yeniden tanımlayamazsın (`int ikiKati(int n) { int n = 2; ... }`); g++ `declaration of 'int n' shadows a parameter` der.

Global değişkenler

Hiçbir fonksiyonun içinde olmayan, dosyanın üstünde tanımlanan değişkene **global değişken** denir. Tanımından sonraki bütün fonksiyonlar onu görür ve değiştirebilir; program boyunca yaşar. İlk değer verilmezse otomatik olarak **0** olur (yerel değişkenlerden farkı budur). Yerel bir değişken globali gölgeliyorsa, globale `::` (kapsam çözümleme operatörü) ile ulaşırsın:

globalYerel.cpp

```cpp
#include <iostream>

using namespace std;

int sayi{100};

void yerelGolgeler() {
    int sayi{7};
    cout << "yerelGolgeler: sayi = " << sayi << ", global = " << ::sayi << '\n';
}

int main() {
    cout << "main         : sayi = " << sayi << '\n';
    yerelGolgeler();
    sayi = sayi + 1;
    cout << "main         : sayi = " << sayi << '\n';
    return 0;
}
```

Çıktı

```
main         : sayi = 100
yerelGolgeler: sayi = 7, global = 100
main         : sayi = 101
```

`yerelGolgeler` içinde yeni bir yerel `sayi` tanımlandığı için o fonksiyonda `sayi` adı yereli gösterir; global olana yalnızca `::sayi` ile ulaşılır. `main` ise yerel tanımı olmadığından global olanı değiştirdi (100 → 101).

Global değişken kolaydır: parametre geçirmeden her yerden erişirsin. Ama bu yüzden **az kullanılır**: Değeri hangi fonksiyonun değiştirdiğini bulmak zorlaşır; fonksiyon global değişkene bağımlı olunca başka programa taşınamaz ve tek başına denenemez; girdi ve çıktısı artık parametre ve `return` değil, görünmeyen bir ortak yerdir. Kural: Veriyi **parametre** ile ver, sonucu `return` ile al. Global tanımı yalnızca programın her yerinde gereken ve değişmeyen değerler (`const` sabitler) için kullan.

> **Sınavda Çıkar**
>
> Yerel/global soruları "bu program ne yazdırır?" biçiminde gelir. Tek kural: **bir ad, kullanıldığı yerden başlayıp içten dışa doğru aranır**; önce kendi bloğu (yalnızca o satıra kadar tanımlanmış olanlar), sonra dıştaki bloklar, en son global. Bulduğun ilk tanım geçerlidir. Bir yerel değişkenin tanımından **önceki** satırlarda aynı ad hâlâ dıştakini gösterir. Gölgelenmiş global değişken, blok bitince eski hâliyle görünür.

6.13 static yerel değişken

Yerel değişkenler her çağrıda sıfırdan oluşur ve fonksiyon bitince silinir (çağrı yığını!). Bazen fonksiyonun çağrılar arasında bir değeri **hatırlamasını** istersin: kaç kez çağrıldığını saymak gibi. Global değişken çözümdür ama onu herkes görür. Daha iyisi `static` **yerel değişken**dir: değişkenin başına `static` yazarsın.

staticSayac.cpp

```cpp
#include <iostream>

using namespace std;

int normalSayac() {
    int sayac{0};
    sayac++;
    return sayac;
}

int kaliciSayac() {
    static int sayac{0};
    sayac++;
    return sayac;
}

int main() {
    for (int i = 1; i <= 4; i++) {
        int n = normalSayac();
        int k = kaliciSayac();
        cout << "Çağrı " << i << ": normal = " << n << ", static = " << k << '\n';
    }
    return 0;
}
```

Çıktı

```
Çağrı 1: normal = 1, static = 1
Çağrı 2: normal = 1, static = 2
Çağrı 3: normal = 1, static = 3
Çağrı 4: normal = 1, static = 4
```

`normalSayac` her çağrıda `sayac`'ı 0'dan başlatır, 1 yapar ve döndürür; hep `1` verir. `kaliciSayac`'taki `static int sayac{0};` satırı ise **yalnızca bir kez**, fonksiyona ilk girişte işlenir. Değişken fonksiyon bitince silinmez, sonraki çağrıda kaldığı değerle devam eder. `static` değişken çağrı yığınında değil, program boyunca yaşayan ayrı bir bölgede durur ve başlatılmazsa 0 olur. Adı yine yalnızca tanımlandığı fonksiyonda görünür: ömrü uzar, görünürlüğü artmaz.

Özet olarak: normal yerel değişken her çağrıda yeniden (ilk değer vermezsen çöp değerle) başlar ve fonksiyon bitince yok olur; `static` yerel ve global değişken program boyunca yaşar, ilk değer verilmezse 0 olur; `static` yerelin ilk değeri yalnızca bir kez verilir. `static` yerel yalnızca kendi fonksiyonunda görünür, global ise tanımından sonraki her fonksiyonda.

> **Sınavda Çıkar**
>
> `static` yerel değişkenli bir fonksiyonu art arda çağırıp çıktıyı sormak çok sevilen sorudur. Anahtar cümle: `static` değişkenin ilk değeri **yalnızca bir kez** verilir; sonraki çağrılar önceki değerle devam eder. Normal yerel değişken her çağrıda ilk değerinden başlar. Her çağrıdan sonraki değeri bir tabloya yaz.

6.14 Fonksiyonu return olmadan bitirmek

Dönüş tipi `void` olmayan bir fonksiyon her yolda bir değer **döndürmelidir**. Örneğin `if (n > 0) { return 1; } else if (n < 0) { return -1; }` gövdesine sahip `isaret(int n)` fonksiyonu, `n` sıfırsa hiçbir `return`'e uğramadan sona erer. `g++ -Wall` ile `control reaches end of non-void function` uyarısı verir. Program derlenir, ama `isaret(0)` çağrılırsa fonksiyon değer döndürmeden biter ve bu **tanımsız davranıştır** (çıktısını göstermiyoruz, çünkü sonuca güvenilemez). Uyarıyı hata gibi ciddiye al: en sık hata, `return`'leri yalnızca `if` / `else if` kollarına koyup "başka ihtimal olamaz" sanmaktır; derleyici ihtimalleri senin kadar bilmez. Çözüm, fonksiyonun son satırını bir `return` yapmaktır: `if (n < 0) { return -1; } return 0;` gibi. `void` fonksiyonlarda sorun yoktur; `main`'in sonundaki `return 0;`'ın zorunlu olmaması da özel bir kuraldır, başka hiçbir fonksiyon için geçerli değildir.

6.15 Hazır fonksiyonlar

Her hazır fonksiyon bir başlık dosyasında bildirilmiştir ve kullanmak için o dosyayı `#include` etmen gerekir. Bölüm 2'de `<cmath>` ile `sqrt`, `pow`, `ceil`, `floor`, `round` gibi fonksiyonları görmüştün. Burada iki şey ekliyoruz: hazır fonksiyonları kendi fonksiyonlarının içinde kullanmak ve `<cstdlib>` (`abs`, `rand`, `srand`) ile `<ctime>` (`time`).

hazirFonksiyonlar.cpp

```cpp
#include <cmath>
#include <cstdlib>
#include <iostream>

using namespace std;

double mesafe(double x1, double y1, double x2, double y2) {
    return sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));
}

int main() {
    cout << "Mesafe: " << mesafe(0, 0, 3, 4) << '\n';
    cout << abs(-7) << " " << abs(-3.7) << '\n';
    cout << ceil(2.1) << " " << floor(2.9) << " " << round(2.5) << '\n';
    return 0;
}
```

Çıktı

```
Mesafe: 5
7 3.7
3 2 3
```

`mesafe` iki hazır fonksiyonu birlikte kullanan sıradan bir fonksiyondur. `abs`, C++'ta `int` ve `double` için ayrı aşırı yüklenmiş sürümlere sahiptir: `abs(-7)` `7`, `abs(-3.7)` `3.7` verir. (Bu, C dilindeki `abs`'tan farklıdır; orada ondalık için `fabs` gerekirdi.) Sen yine de ondalık sayılarda `<cmath>` eklemeyi unutma.

rand ve srand

`rand()` her çağrıda `0` ile `RAND_MAX` arasında bir **sözde rastgele** (pseudo-random) tamsayı döndürür. `RAND_MAX` en az 32767'dir; Windows'ta (MinGW) tam 32767, Linux ve macOS'ta 2147483647'dir. Sayılar formülle üretildiği için aynı başlangıç değeri (**tohum**, seed) hep aynı diziyi verir. Tohumu `srand(tohum)` ile belirlersin.

Aynı tohumla `srand` çağırırsan `rand`'ın verdiği dizi her seferinde aynıdır; değerler ise Windows, Linux ve macOS'ta farklıdır. Programın her çalışmasında farklı sayılar istersen tohumu değişen bir değerden alırsın; en yaygın kalıp **şimdiki zaman**dır: `srand(time(nullptr));`. `time` fonksiyonu `<ctime>` içindedir. (`nullptr` boş işaretçidir; Bölüm 9'da göreceksin, şimdilik kalıp olarak al.) İstediğin aralığa sığdırmak için kalan (`%`) işlemi kullanılır:

İstenen	İfade	Neden?
0 ile 99 arası	`rand() % 100`	Kalan 0..99
Zar (1 ile 6)	`rand() % 6 + 1`	Kalan 0..5, `+ 1` ile 1..6
`a` ile `b` arası	`a + rand() % (b - a + 1)`	10..20 için `10 + rand() % 11`
0 ile 1 arası `double`	`static_cast<double>(rand()) / RAND_MAX`	Bölme `double` yapılır

zarAtma.cpp

```cpp
#include <cstdlib>
#include <ctime>
#include <iostream>

using namespace std;

int zarAt() {
    return rand() % 6 + 1;
}

int main() {
    srand(static_cast<unsigned>(time(nullptr)));
    for (int i = 1; i <= 5; i++) {
        cout << "Zar " << i << ": " << zarAt() << '\n';
    }
    return 0;
}
```

Çıktı

```
Zar 1: 1
Zar 2: 5
Zar 3: 3
Zar 4: 2
Zar 5: 1
```

Bu çıktı sana özgü bir örnektir. Programı kendi bilgisayarında çalıştırınca farklı sayılar görürsün, her çalıştırışta da farklı çıkar. Sınavlarda `rand` çıktısı sorulmaz; "sonuç hangi aralıkta olabilir?" sorulur. Her sayı yine de 1 ile 6 arasındadır. (C++11'den beri `<random>` kitaplığı daha iyi rastgele sayılar sunar; giriş düzeyinde `rand` yeterlidir.)

Sık Yapılan Hata: srand'ı döngü içinde çağırmak

`srand(time(nullptr));` satırı programın başında **bir kez** çalışmalıdır. Döngünün içine koyarsan, `time` saniye çözünürlüklü olduğundan döngü bir saniyeden kısa sürdüğünde her tur aynı tohumla başlar ve `rand()` hep aynı sayıyı verir. Bir de `rand() % 6` yazıp `+ 1`'i unutursan zar 0 ile 5 arasında çıkar.

inline fonksiyonlar

Çok küçük bir fonksiyonun başına `inline` yazmak, derleyiciye "çağrı yapmak yerine gövdeyi çağrı yerine yapıştırabilirsin" demektir; böylece çerçeve açma maliyeti ortadan kalkar (`inline int kareAl(int x) { return x * x; }`). Derleyici bu isteğe uymak zorunda değildir ve optimizasyon açıkken `inline` yazmasan da kısa fonksiyonlarda bu kararı kendisi verir; yani başlangıç düzeyinde `inline` yazman gerekmez. Anahtar sözcüğün bugünkü asıl işi, çok dosyalı programlarda aynı tanımın başlık dosyasında bulunmasına izin vermektir.

6.16 Algoritmaları fonksiyona dönüştürme

Bölüm 5'te klasik algoritmaları (asal sayı, EBOB, faktöriyel) `main` içinde yazdın. Fonksiyona çevirmenin tarifi basittir: algoritmanın **girdileri** parametre olur, **sonucu** `return` ile döner, `cin` ve `cout` fonksiyonun dışında, `main`'de kalır. `asalMi` bölen bulunca doğrudan `return false;` yazar, Bölüm 5'teki bayrağa ve `break`'e gerek kalmaz; kullanımı `if (asalMi(n))` olur. `ebob` parametrelerini rahatça eritebilir, çünkü `a` ve `b` zaten **kopyadır**; değer ile geçirmenin işe yaradığı yer burasıdır. Fonksiyonlar birbirini de çağırabilir: `ekok` tek satırdır, `return a / ebob(a, b) * b;`. Kod yazma sorularında bu tarifi uygularsın.

İpucu: Soru çeşidine göre dönüş tipi

"... mı?" diye sorulan fonksiyonlar `bool` döndürür (`asalMi`, `ciftMi`, `mukemmelMi`). "Kaç tane / ne kadar?" soruları `int` ya da `long long`, ortalama gibi ondalıklı sonuçlar `double` döndürür. Yazdırma ve menü gibi iş yapanlar `void` olur. Dönüş tipini seçerken fonksiyonun adını soru gibi oku.

6.17 main de bir fonksiyondur

`main` ayrıcalıklı ama sıradan bir fonksiyondur: dönüş tipi `int`'tir ve onu **işletim sistemi** çağırır. `return` ile verdiği değer **çıkış kodu** olur (Bölüm 1); hata durumunda `return 1;` ile erken ve hatalı çıkış yapabilirsin. Büyük programlarda fonksiyonlar birden çok dosyaya bölünür ve prototipler başlık dosyalarında durur; bu bölümdeki her şey tek dosyaydı.

Bölüm Özeti – Bilmen Gerekenler

- Fonksiyon: `dönüşTipi ad(parametreler) { gövde }`. Tekrarı önler, okunabilirliği artırır, problemi parçalara böler.
- `void` fonksiyon değer döndürmez (`return;` ile erken çıkabilir); çağrısı bir değişkene atanamaz. Diğerleri `return` ile **en fazla bir** değer verir; `return` çalıştığı anda fonksiyon biter.
- Fonksiyon çağrılmadan önce **tanıtılmış** olmalıdır: tanımı `main`'den önce yaz ya da **prototip** (`int f(int);`) ekle. Prototipsiz çağrı `was not declared in this scope` hatasıdır; prototip ile tanım dönüş tipi dahil birebir uyuşmalıdır.
- **Parametre** tanımdaki, **argüman** çağrıdaki değerdir. Sayı ve sıra uyuşmalıdır; tip kendiliğinden çevrilir (`double` → `int` ondalığı atar). `(a + b) / 2` tamsayı bölmesidir.
- Varsayılan olarak argümanlar **değer ile** geçirilir: fonksiyon kopya alır, `takas(x, y)` çağıranın değişkenlerini değiştirmez.

- **Referans ile geçirme** (`int &a`): parametre çağıranın değişkeninin takma adıdır; `takas` çalışır, `&` parametresine sabit verilemez. Birden çok sonuç çıkış parametreleriyle döndürülür. Yerel değişkenin referansı döndürülmez.
- `const &` kopyayı ve değiştirmeyi engeller; büyük nesneleri (`string`, `vector`; Bölüm 7 ve 8) yalnızca okuyacaksan böyle geçir.
- **Varsayılan parametreler** sağdan sola verilir ve bildirimde bir kez yazılır; **aşırı yükleme**: aynı ad, farklı parametre listesi; dönüş tipi ayırt etmez. Seçim sırası: tam eşleşme, terfi, dönüşüm; eşitlikte belirsiz çağrı hatası.
- Her çağrı **çağrı yığınında** kendi **yığın çerçevesini** açar, fonksiyon bitince çerçeve silinir. İç içe çağrıda içteki önce çalışır.
- **Kapsam:** ad kullanıldığı yerden içe-dışa aranır; iç blokta aynı ad dıştakini **gölgeler**, global değişkene `::ad` ile ulaşılır. Global değişkenin ilk değeri 0'dır ve az kullanılmalıdır.
- `static` yerel değişken çağrılar arasında değerini korur, ilk değeri bir kez verilir, başlatılmazsa 0'dır.
- Değer döndüren fonksiyonun her yolu `return` ile bitmelidir; yoksa uyarı ve tanımsız dönüş değeri.
- `abs` hem `int` hem `double` için çalışır. `rand() % n` 0..n-1 verir (zar: `rand() % 6 + 1`); `srand(time(nullptr))` programın başında **bir kez** çağrılır; `rand` çıktısı sistemlere göre değişir.
- Algoritma → fonksiyon: girdiler parametre, sonuç `return`, `cin`/`cout` `main`'de. `main` de bir fonksiyondur: `return 0;` başarı, sıfırdan farklı değer hata.

Sorular

A) Çoktan Seçmeli

1 `double ortalama(int a, int b)` fonksiyonu `main`'den **sonra** tanımlanmıştır. `main` içinde sorunsuz çağrılabilmesi için `main`'den önce hangi satır yazılmalıdır?

(A) `double ortalama;`
(B) `ortalama(int a, int b);`
(C) `double ortalama(int, int);`
(D) `void ortalama(int a, int b);`
(E) `double ortalama(int a, int b) { }`

2 `void artir(int &n) { n++; }` fonksiyonu tanımlıdır. `main` içinde `int x{5}; artir(x);` çalıştırılırsa aşağıdakilerden hangisi **doğrudur**?

(A) `x` 6 olur, çünkü `n`, `x`'in takma adıdır.
(B) `x` 5 kalır, çünkü argüman kopyalanır.
(C) Derleme hatası olur; çağrıda `artir(&x)` yazılmalıdır.
(D) `artir(5)` çağrısı da geçerlidir ve `5`'i `6` yapar.
(E) `n` bağımsız bir kopyadır, ancak fonksiyon bitince değeri `x`'e geri yazılır.

3 Aşağıdaki `topla` fonksiyonu `main` içinde art arda üç kez çağrılıp döndürdüğü değerler sırayla yazdırılırsa ekranda ne görülür?

```cpp
#include <iostream>
using namespace std;
int topla(int k) {
    static int t{100};
    t += k;
    return t;
}
int main() {
    int a = topla(1);
    int b = topla(2);
    int c = topla(3);
    cout << a << " " << b << " " << c << '\n';
    return 0;
}
```

(A) `101 101 101` (B) `101 102 103` (C) `100 101 103` (D) `101 103 105` (E) `101 103 106`

4 Aşağıdaki fonksiyon bildirimlerinden hangisi **derleme hatasına** yol açar?

(A) `int f(int a, int b = 2);`

(B) `int f(int a = 1, int b = 2);`

(C) `int f(int, int = 2);`

(D) `int f(int a = 1, int b);`

(E) `int f(int a, int b = 2, int c = 3);`

5 `void goster(int)` ve `void goster(double)` aşırı yüklenmiş iki fonksiyondur; ilki ekrana `I`, ikincisi `D` yazar. Aşağıdaki çağrılar sırayla yapılırsa ekranda ne görülür?

```cpp
#include <iostream>
using namespace std;
void goster(int) {
    cout << "I ";
}

void goster(double) {
    cout << "D ";
}
int main() {
    goster(7);
    goster(7.0);
    goster('A');
    goster(2.5f);
    cout << '\n';
    return 0;
}
```

(A) `I D D D` (B) `I D I D`

(C) `I D I I` (D) `I I I D`

(E) Derleme hatası (belirsiz çağrı)

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
int say(int &toplam) {
    static int cagri{0};
    cagri++;
    toplam += cagri;
    return cagri * 10;
}

int main() {
    int t{0};
    int a = say(t);
    int b = say(t);
    int c = say(t);
    cout << a << " " << b << " " << c << " " << t << '\n';
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```
int g(int x, int y = 2) {
    return x * y;
}

double g(double x) {
    return x / 2;
}

void h(int &r) {
    r += 5;
}

void k(int v) {
    v += 5;
    cout << v << " ";
}

int main() {
    int a{4};
    cout << g(a) << " " << g(a, 3) << " " << g(3.0) << " " << g('A') << '\n';
    h(a);
    k(a);
    cout << a << '\n';
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır? Tüm sayılar aynı satırda, aralarında birer boşlukla yazılır.

```
int x{5};

void yaz() {
    cout << x << " ";
}

void degistir() {
    x = x + 1;
    int x{50};
    x++;
    cout << x << " ";
    yaz();
}

int main() {
    int x{1};
    yaz();
    degistir();
    {
        int x{100};
        x++;
        cout << x << " ";
        yaz();
    }
    cout << x << " " << ::x << '\n';
    return 0;
}
```

C) Kod Yazma

9 Toplam saniyeyi saat, dakika ve saniyeye ayıran `void sureAyir(int toplamSaniye, int &saat, int &dakika, int &saniye)` fonksiyonunu yaz. `main`, toplam saniyeyi klavyeden okusun, fonksiyonu çağırsın ve sonucu yazdırsın. Fonksiyon yazdırma ya da okuma yapmasın. Örnek (`3725` girildiğinde):

```
Saniye: 3725
1 saat 2 dakika 5 saniye
```

10 `bool asalMi(int n)` fonksiyonunu yaz. `main`, bir üst sınır okuyup 2'den bu sınıra kadar olan asal sayıları yazdırsın ve kaç tane olduklarını belirtsin. Sayma işini `main` yapsın; `asalMi` yazdırma ya da okuma yapmasın. Örnek (`30` girildiğinde):

```
Üst sınır: 30
Asallar: 2 3 5 7 11 13 17 19 23 29
Toplam: 10 tane
```

Çözümler

Cevap anahtarı

1 C	2 A	3 E	4 D	5 B

1 Cevap: C

Prototip, tanımın gövdesiz hâlidir: dönüş tipi, ad, parametre **tipleri**, sonunda `;`; parametre adları yazılmayabilir. A bir değişken bildirimidir. B'de dönüş tipi yok. D'de dönüş tipi farklı olduğundan çakışma (`ambiguating new declaration`) hatası verir. E bir tanımdır, prototip değildir; gerçek tanımla birlikte fonksiyon iki kez tanımlanmış olur.

2 Cevap: A

`int &n` parametresi, çağıran fonksiyondaki `x`'in takma adıdır; `n++` aslında `x`'i artırır. B değer ile geçirmeyi anlatıyor, oysa burada `&` var. C, adres operatörünü (`&x`) referansla karıştırıyor; referans parametreye düz değişken verilir, işaretçi gerekmez. D yanlış: `5` bir sabittir, değiştirilecek kutusu yoktur, `cannot bind non-const lvalue reference` hatası verir. E yanlış: kopya yoktur, geri yazma da yoktur; `n` ile `x` aynı kutudur.

3 Cevap: E

`static int t{100};` satırı yalnızca ilk çağrıda işlenir. `t` sırayla 100 → 101 (döner 101), 101 → 103 (döner 103), 103 → 106 (döner 106) olur. A, `t`'nin her çağrıda 100'e döndüğünü ve hep 1 eklendiğini sanmaktır. B her çağrıda 1 eklendiğini varsayar, oysa eklenen `k` değeridir. C ilk değeri döndürüp sonra artırmaktır. D son değeri yanlış toplamdır (`103 + 3 = 106`).

Çıktı

```
101 103 106
```

4 Cevap: D

Varsayılan değerler sağdan sola verilir: bir parametrenin varsayılanı varsa sağındaki hepsinin de olmalıdır. D'de `a`'nın varsayılanı var ama sağındaki `b`'nin yok; g++ `default argument missing for parameter 2` hatası verir. A, B, C ve E'de varsayılanlar sağdan sola sıralıdır; C'de parametre adlarının yazılmaması da geçerlidir.

5 Cevap: B

Seçim sırası: tam eşleşme, terfi, dönüşüm. `goster(7)`: `int` tam eşleşir, `I`. `goster(7.0)`: `double` tam eşleşir, `D`. `goster('A')`: tam eşleşme yok; `char` → `int` bir terfidir, `char` → `double` ise dönüşümdür; terfi kazanır, `I`. `goster(2.5f)`: `float` → `double` terfi, `int`'e dönüşümden iyidir, `D`. E yanlış: her çağrıda bir aday diğerinden daha iyidir, belirsizlik yoktur.

Çıktı

```
I D I D
```

6

`cagri` `static`: bir kez başlar, çağrılar arasında kalır. `toplam` referans: `main`'deki `t`'nin kendisidir.

Çağrı	`cagri` sonra	`toplam` (yani `t`) sonra	Döner
1	1	0 + 1 = 1	10
2	2	1 + 2 = 3	20
3	3	3 + 3 = 6	30

`a = 10`, `b = 20`, `c = 30` ve `t = 6`.

Çıktı
```
10 20 30 6
```

7

`a = 4`.

- `g(a)`: `int` argüman, `g(int, int = 2)` tam eşleşir; `y = 2`, sonuç `8`.
- `g(a, 3)`: `4 * 3 = 12`.
- `g(3.0)`: `double` argüman, `g(double)` tam eşleşir: `3.0 / 2 = 1.5`.
- `g('A')`: `'A'` kodu 65. `g(int, int = 2)` için `char` → `int` terfidir, `g(double)` için dönüşümdür; terfi kazanır: `65 * 2 = 130`.
- `h(a)` referansla geçer, `a` 9 olur. `k(a)` kopya alır: kopya 14 olur ve `14` yazılır (satır sonu yok, boşluk var), `main`'deki `a` ise yine 9; son `cout` `9` yazar.

Çıktı
```
8 12 1.5 130
14 9
```

8

Ad, kullanıldığı yerden içe-dışa aranır.

- `yaz()` her zaman global `x`'i yazar, ilk çağrıda `5`.
- `degistir()` içinde `x = x + 1;` satırında yerel `x` henüz tanımlanmadığı için global `x` kullanılır: global `x` 6 olur. Sonra `int x{50};` yeni bir yerel tanımlar, `x++` onu 51 yapar ve `51` yazılır. `yaz()` global `x`'i yazar: `6`.
- `main`'in iç bloğunda `x` 100, `x++` ile 101 olur ve `101` yazılır; `yaz()` yine global: `6`.
- Blok bitince `main`'in `x`'i görünür: `1`. `::x` global olanı gösterir: `6`.

Çıktı
```
5 51 6 101 6 1 6
```

9

Saat, dakika ve saniye çıkış parametreleridir (`&`); toplam saniye girdidir ve değer ile geçer. `3725 / 3600 = 1` saat, kalan `125`; `125 / 60 = 2` dakika, `125 % 60 = 5` saniye.

sureAyir.cpp
```cpp
#include <iostream>

using namespace std;

void sureAyir(int toplamSaniye, int &saat, int &dakika, int &saniye) {
    saat = toplamSaniye / 3600;
```

```
    int kalan = toplamSaniye % 3600;
    dakika = kalan / 60;
    saniye = kalan % 60;
}

int main() {
    int toplam{0};
    cout << "Saniye: ";
    cin >> toplam;

    int s{0};
    int d{0};
    int sn{0};
    sureAyir(toplam, s, d, sn);
    cout << s << " saat " << d << " dakika " << sn << " saniye\n";
    return 0;
}
```

Çıktı

```
Saniye: 3725
1 saat 2 dakika 5 saniye
```

10

`asalMi` yalnızca karar verir; `i * i <= n` koşuluyla bölenleri karekökünde keser. Yazdırma ve sayma `main`'dedir.

asalListesi.cpp

```
#include <iostream>

using namespace std;

bool asalMi(int n) {
    if (n < 2) {
        return false;
    }
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            return false;
        }
    }
    return true;
}

int main() {
    int sinir{0};
    cout << "Üst sınır: ";
    cin >> sinir;

    int adet{0};
    cout << "Asallar:";
    for (int n = 2; n <= sinir; n++) {
        if (asalMi(n)) {
            cout << " " << n;
            adet++;
        }
    }
    cout << '\n';
    cout << "Toplam: " << adet << " tane\n";
    return 0;
}
```

Çıktı

```
Üst sınır: 30
Asallar: 2 3 5 7 11 13 17 19 23 29
Toplam: 10 tane
```

BÖLÜM 7

Diziler ve vector

Bu Bölümde Öğreneceklerin

- Dizinin ne olduğunu, nasıl tanımlandığını ve elemanlara nasıl ulaşıldığını
- Sınırın dışına çıkmanın neden tehlikeli olduğunu
- Dizinin eleman sayısını bulmayı, her elemanı dolaşmayı ve diziyi fonksiyona geçirmeyi
- Satır ve sütunlu tabloları (matris)
- Boyutu kendiliğinden büyüyen `vector`'ü ve güvenli erişim yolunu
- Toplam, en büyük/en küçük, arama, ters çevirme ve sıralamayı (kabarcık, seçmeli) elle izlemeyi

7.1 Neden dizi? Tanımlama ve bellek

50 öğrencinin notunu tutan bir program yazacaksın. Şimdiye kadarki bilgilerinle `int not1, not2, ..., not50;` diye elli değişken tanımlarsın, ortalama için elli terimli bir toplama yazarsın. Döngü (Bölüm 5) yardım edemez, çünkü `not1`, `not2` adlarını döngüye sokamazsın: "`i`'inci değişken" demenin yolu yok.

Çözüm **dizi**dir (array): aynı tipten birçok değeri **tek bir adla** tutar, her değere **sıra numarasıyla** (**indeks**, index) ulaşmanı sağlar. Dizinin her değerine **eleman** (element) denir. Önce C'den miras kalan yalın diziyi öğreneceksin, sonra C++'ın asıl aracı olan `vector`'e geçeceksin.

```
tip diziAdi[boyut];
```

Boyut, derleme sırasında bilinen sıfırdan büyük bir **sabit** olmalıdır; Bölüm 2'deki `const int N{4};` bu iş için yeterlidir. Dizi tanımlandıktan sonra **büyüyemez ve küçülemez**.

ilkDizi.cpp

```
#include <iostream>

using namespace std;

int main() {
    const int N{4};
    int notlar[N];
    notlar[0] = 70;
    notlar[1] = 85;
    notlar[2] = 90;
    notlar[3] = 55;
    int i{2};
    cout << "İlk not: " << notlar[0] << ", son not: " << notlar[N - 1] << '\n';
    cout << "notlar[i - 1] + notlar[i + 1] = "
         << notlar[i - 1] + notlar[i + 1] << '\n';
    cout << "Dizi: " << sizeof(notlar) << " bayt\n";
    return 0;
}
```

```
Çıktı
İlk not: 70, son not: 55
notlar[i - 1] + notlar[i + 1] = 140
Dizi: 16 bayt
```

`notlar[2]` dizinin 2 numaralı elemanıdır ve **sıradan bir `int` değişkeni gibi** kullanılır: `notlar[2] = 90;`, `notlar[2]++`, `cin >> notlar[2]` geçerlidir. `sizeof(notlar)` bütün dizinin bayt sayısıdır: 4 eleman × 4 bayt = 16. Elemanlar bellekte **boşluksuz, art arda** durur:

İndeks	`notlar[0]`	`notlar[1]`	`notlar[2]`	`notlar[3]`
Değer	70	85	90	55
Başlangıçtan uzaklık	0 bayt	4 bayt	8 bayt	12 bayt

> **Sık Yapılan Hata: Dizi boyutunu çalışma anında vermek**
>
> `int n; cin >> n; int a[n];` standart C++'ta geçersizdir: g++ bunu eklenti olarak sessizce derler, `-Wpedantic` uyarır, Visual Studio reddeder. Boyutu çalışma anında belirlenen veri için doğru araç `vector`'dür.

7.2 İlk değer verme

Tanım satırında küme parantezi içinde **ilk değer listesi** (initializer list) verebilirsin. Biçimlerin hepsi tek programda:

ilkDegerler.cpp

```cpp
#include <iomanip>
#include <iostream>

using namespace std;

int main() {
    int a[5] = {10, 20, 30, 40, 50};
    int b[5] = {1, 2, 3};
    int c[5] = {0};
    int d[5] = {7};
    int e[] = {4, 8, 15, 16, 23, 42};
    int f[5]{};

    cout << "i   a   b   c   d   f\n";
    for (int i = 0; i < 5; i++) {
        cout << i << setw(4) << a[i] << setw(4) << b[i] << setw(4) << c[i]
             << setw(4) << d[i] << setw(4) << f[i] << '\n';
    }
    cout << "e[5] = " << e[5] << '\n';
    return 0;
}
```

```
Çıktı
i   a   b   c   d   f
0  10   1   0   7   0
1  20   2   0   0   0
2  30   3   0   0   0
3  40   0   0   0   0
4  50   0   0   0   0
e[5] = 42
```

- **Liste boyuttan kısaysa** kalan elemanlar `0` olur (`b`). Çöp değil, `0`!

- `{0}` **bütün diziyi sıfırlar** (`c`); boş liste de aynı işi yapar (`f`).
- `{7}` **hepsini 7 yapmaz** (`d`): yalnız ilk eleman 7'dir, kalanlar `0`.
- **Boyut yazılmazsa** derleyici liste uzunluğunu boyut yapar (`e`: 6 eleman, indeksler 0–5).
- Liste **yalnızca tanım satırında** yazılır; `a = {1, 2, 3};` dizilerde geçersizdir. Listede boyuttan **fazla** eleman olması C++'ta derleme hatasıdır (`too many initializers`), ondalıklı değeri `int` dizisine listeyle koymak da (daralma, Bölüm 2).

İlk değer listesi olmayan yerel dizi (`int x[5];`) çöp değer taşır; elemanını okumak tanımsız davranıştır. Dizi, **okunmadan önce** değer almalıdır. Kolay yol: `int x[5]{};`.

7.3 İndeks ve sınır dışı erişim

İndeks **0'dan başlar**: `N` elemanlı dizide ilk eleman `a[0]`, son eleman `a[N-1]`'dir. İndeks, "dizinin başından kaç eleman ileride" demektir; ilk eleman başlangıçtadır, yani 0 eleman ileridedir. İndeks `a[i + 1]` ya da `a[b[j]]` gibi her tamsayı ifadesi olabilir; iç içe indekslerde (`a[a[0]]`) **içten dışa** çöz.

`int a[5]` için `a[5]`, `a[-1]`, `a[100]` **geçerli değildir**; dizinin dışındaki belleğe uzanırlar. Kritik nokta: **C++ bunu denetlemez.** Şu satırlar derlenir:

```
int a[5] = {10, 20, 30, 40, 50};
a[5] = 99;                  // geçersiz: son indeks 4
a[-1] = 7;                  // geçersiz: indeks negatif
cout << a[100] << '\n';     // geçersiz: okumak bile
```

Bu bir **sınır dışı erişimdir** (out-of-bounds access) ve **tanımsız davranıştır**. Okumada çöp ya da komşu değişkenin değeri gelir; yazmada **komşu bir değişken sessizce bozulur**; program çökebilir (Linux ve macOS'ta `Segmentation fault`, Windows'ta çoğu zaman sessiz kapanma) ya da hiçbir şey olmamış gibi **doğru çalışıyormuş** görünür. Java ve Python'da sınır dışı erişim programı hata ile durdurur; C++'ta denetim programcıya aittir. Birçok gerçek güvenlik açığı dizi sınırını aşan yazmalardan doğmuştur.

> **Sık Yapılan Hata: Döngüde `<=` yazmak**
>
> Sınır dışı erişimin en yaygın nedeni, Bölüm 5'te gördüğün off-by-one hatasıdır: `for (int i = 0; i <= N; i++)` yazarsan son turda `i == N` olur ve `a[N]` dizinin dışına çıkar. Kalıbı ezberle: `for (int i = 0; i < N; i++)`. "C++ sınır dışı erişimde hata verir" cümlesi sınavda hep yanlış şıktır.

7.4 Eleman sayısı: sizeof

Eleman sayısını, dizinin bayt sayısını bir elemanın bayt sayısına bölerek bulursun: `sizeof(a) / sizeof(a[0])`. Bölen `sizeof(int)` yerine `sizeof(a[0])` yazılır, çünkü dizinin tipi değişse formül doğru kalır. Formül **yalnız dizinin tanımlandığı fonksiyonda** çalışır. Aşağıdaki programda diziyi başka bir fonksiyona parametre olarak veriyoruz; bu yazımın ayrıntısını birazdan "Diziler ve fonksiyonlar" başlığında göreceksin:

sizeofTuzak.cpp

```cpp
#include <iostream>

using namespace std;

void yanlis(int a[]) {
    int n = sizeof(a) / sizeof(a[0]);
    cout << "Fonksiyon içinde n = " << n << '\n';
}

int main() {
    int dizi[6] = {1, 2, 3, 4, 5, 6};
    int n = sizeof(dizi) / sizeof(dizi[0]);
    cout << "Dizi " << sizeof(dizi) << " bayt, eleman " << sizeof(dizi[0])
         << " bayt\n";
    cout << "main içinde n = " << n << '\n';
    yanlis(dizi);
    return 0;
}
```

Derleyici uyarısı – g++ -Wall -Wextra

```
sizeofTuzak.cpp: In function 'void yanlis(int*)':
sizeofTuzak.cpp:6:20: warning: 'sizeof' on array function parameter 'a' will return size of 'int*'
[-Wsizeof-array-argument]
    6 |     int n = sizeof(a) / sizeof(a[0]);
      |                    ~^~
sizeofTuzak.cpp:5:17: note: declared here
    5 | void yanlis(int a[]) {
      |             ~~~~^~~
```

Çıktı

```
Dizi 24 bayt, eleman 4 bayt
main içinde n = 6
Fonksiyon içinde n = 2
```

Fonksiyon dizinin kendisini değil **başlangıç adresini** alır (nedenini Bölüm 9'da göreceksin); `int a[]` yazılsa da `a` bir adres tutar ve `sizeof(a)` adresin boyutunu verir. 64 bit bilgisayarlarda adres 8 bayttır, 8 / 4 = 2 çıkar (senin bilgisayarında farklı olabilir). Sonuç ne olursa olsun yanlıştır. **Çözüm:** eleman sayısını dizinin tanımlandığı yerde hesapla ve fonksiyona ayrı bir `int n` parametresiyle geçir (ya da `vector` kullan).

7.5 Aralık tabanlı for

Çoğu zaman indeksin kendisine ihtiyacın yoktur, yalnız **her elemanı** görmek istersin. C++11 bunun için **aralık tabanlı for** (range-based for) getirdi: `for (tip degisken : dizi)`. Dikkat: `degisken` varsayılan olarak elemanın **kopyasıdır**. Diziyi değiştirmek için `&` ile **referans** yaz (Bölüm 6'daki referans parametresi gibi: kopya değil, elemanın kendisi). Yalnız okuyorsan `const` ekle:

araliklifor.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int a[4] = {1, 2, 3, 4};

    for (int x : a) {
        x = x * 10;            // kopya değişir, dizi değişmez
```

```
    }
    cout << "Kopyayla: " << a[0] << ' ' << a[3] << '\n';

    for (int& x : a) {
        x = x * 10;             // elemanın kendisi değişir
    }
    cout << "Referansla: " << a[0] << ' ' << a[3] << '\n';

    int toplam{0};
    for (const int& x : a) {    // yalnız okuyoruz
        toplam += x;
    }
    cout << "Toplam: " << toplam << '\n';
    return 0;
}
```

Çıktı

```
Kopyayla: 1 4
Referansla: 10 40
Toplam: 100
```

İndeks değişkeni olmadığı için sınır dışına çıkamazsın. Klasik `for`'u indeksi **kullanacaksan** seç: kaçıncı eleman olduğunu yazdırmak, iki komşuyu karşılaştırmak, sondan başa gitmek. Yukarıda `a` `main`'de tanımlandığı için aralık tabanlı `for` çalıştı. Diziyi başka bir fonksiyona parametre olarak geçirirsen (`void f(int a[])`) fonksiyonun içindeki `a` bir adres olur ve aralık tabanlı `for` ona **uygulanamaz** (derleyici dizinin nerede bittiğini bilmez); `vector` ile bu sorun da kalkar.

7.6 Diziler ve fonksiyonlar

Dizi parametresi köşeli parantezli ama **boyutsuz** yazılır; eleman sayısı ayrı bir `int n` parametresidir. Çağrıda dizi **yalnız adıyla** verilir: `yazdir(a, N)`. Diziyi yalnız okuyan fonksiyon parametreyi `const int a[]` yazar; fonksiyon bir elemanı değiştirmeye kalkarsa derleme hatası alır.

diziFonksiyon.cpp

```
#include <iostream>

using namespace std;

void yazdir(const int a[], int n) {
    for (int i = 0; i < n; i++) {
        cout << a[i] << ' ';
    }
    cout << '\n';
}

void ikiyeKatla(int a[], int n) {
    for (int i = 0; i < n; i++) {
        a[i] *= 2;
    }
}

void artir(int x) {
    x++;
    cout << "artir içinde x = " << x << '\n';
}

int main() {
    const int N{5};
    int a[N] = {1, 2, 3, 4, 5};
```

```
    int sayi{10};
    ikiyeKatla(a, N);
    yazdir(a, N);
    artir(sayi);
    cout << "main içinde sayi = " << sayi << '\n';
    return 0;
}
```

Çıktı

```
2 4 6 8 10
artir içinde x = 11
main içinde sayi = 10
```

`ikiyeKatla` diziyi değiştirdi ve **değişiklik `main`'deki dizide göründü**; `artir` ise kendi `x`'ini artırdı, `sayi` 10 kaldı. Bölüm 6'daki değer ile geçirme kuralıyla çelişmiyor:

	`int x` parametresi	`int a[]` parametresi
Fonksiyona giden	Değerin **kopyası**	Dizinin **adresi**
Fonksiyonda yapılan değişiklik	Çağırana **yansımaz**	Çağırana **yansır**

Büyük diziyi her çağrıda kopyalamak yavaş olurdu; dizi adres olarak geçer ve fonksiyon **gerçek dizi üzerinde** çalışır (ayrıntısı Bölüm 9'da). Dizi **döndürülemez**; sonuç, çağıranın verdiği diziye yazılır.

7.7 İki boyutlu diziler

Tablo ya da **matris** (matrix) gibi **satır ve sütunlu** veriyi iki boyutlu dizi tutar: `tip ad[SATIR][SUTUN];`. Eleman `m[i][j]`'dir; `i` **satır** (0 ile `SATIR-1`), `j` **sütun** numarasıdır. `int m[3][4];` 3 satır, 4 sütun, 12 eleman demektir. Bellekte elemanlar **satır satır** art arda durur. `m[i, j]` yazma: bu `m[j]` demektir. Köşeli parantez içindeki virgül, iki ifadeyi sırayla hesaplayıp **sağdakinin** değerini veren virgül operatörüdür; `i` hesaplanır ve atılır (g++ uyarır). İlk değer satır başına bir küme parantezi ile verilir; eksikler `0` olur. Yalnız **ilk** boyut atlanabilir: `int y[][3] = { {1, 2, 3}, {4, 5, 6} };`.

İç içe iki `for` kullanılır: dış döngü satırları, iç döngü sütunları dolaşır. Aşağıdaki kare matriste satır toplamı ve **köşegenler** (diagonal) hesaplanıyor. **Ana köşegen** sol üstten sağ alta, **ikincil köşegen** sağ üstten sol alta iner.

matris.cpp

```
#include <iomanip>
#include <iostream>

using namespace std;

int main() {
    const int N{3};
    int m[N][N] = {
        {5, 1, 2},
        {0, 3, 4},
        {9, 8, 7}
    };
    int ana{0};
    int ikincil{0};
    for (int i = 0; i < N; i++) {
        int satirToplam{0};
        for (int j = 0; j < N; j++) {
            cout << setw(3) << m[i][j];
            satirToplam += m[i][j];
            if (i == j) {
```

```
                ana += m[i][j];
            }
            if (i + j == N - 1) {
                ikincil += m[i][j];
            }
        }
        cout << "  | satır toplamı: " << satirToplam << '\n';
    }
    cout << "Ana köşegen: " << ana << ", ikincil köşegen: " << ikincil << '\n';
    return 0;
}
```

Çıktı

```
  5  1  2  | satır toplamı: 8
  0  3  4  | satır toplamı: 7
  9  8  7  | satır toplamı: 24
Ana köşegen: 15, ikincil köşegen: 14
```

Satır toplamında toplam her satırın başında `0`'dan başlar; **sütun toplamında** döngüler yer değiştirir (dış döngü `j`, iç döngü `i`). Köşegenler `i == j` (ana) ve `i + j == N - 1` (ikincil) koşullarıyla seçilir. **Transpoz** (transpose; satırları sütun yapmak) için `t[j][i] = m[i][j]` yazılır. `sizeof(m) / sizeof(m[0])` satır sayısını verir (36 / 12 = 3).

Fonksiyona geçirirken **sütun sayısı yazılmak zorundadır**, satır sayısı ayrı parametre olur: `void yazdir(const int m[][SUTUN], int satir)`. Çünkü elemanlar art arda durur ve derleyici `m[i][j]`'nin yerini `başlangıç + (i × SUTUN + j) × 4` bayt diye hesaplar: formülde `SATIR` yoktur ama `SUTUN` vardır. `int m[][]` yazarsan derleme hatası alırsın.

7.8 vector: büyüyen dizi

C tarzı dizinin üç sıkıntısı var: boyutu derlemede sabitlenir, fonksiyona geçerken boyutunu unutur, bütün olarak kopyalanamaz (`b = a` derlenmez). Gerçek programlarda veri miktarı çoğu zaman önceden bilinmez. C++'ın standart kütüphanesindeki **`vector`** bu sıkıntıları çözer: elemanları art arda tutar, ama **eleman sayısı çalışma anında büyüyüp küçülebilir**, kendi boyutunu bilir ve kopyalanabilir. Bu tür hazır veri yapılarına **kapsayıcı** (container) denir; `vector` en çok kullanılanıdır (diğerleri Bölüm 15'te).

`<vector>` başlığı gerekir. Eleman tipi **açısal parantezle** yazılır: `vector<int>` "int tutan vector" demektir (açısal parantezin ne olduğunu Bölüm 13'te şablonlarda göreceksin; şimdilik kalıp olarak al). `push_back` elemanı **sona** ekler ve boyut kendiliğinden artar; `size()` eleman sayısını, `front()` / `back()` ilk/son elemanı verir; `pop_back()` sonuncuyu siler, `empty()` boş mu diye bakar, `clear()` hepsini siler. Elemanlara dizideki gibi `v[i]` ile ulaşılır. `v.size()` nokta ile, nesnenin üzerinden çağrılan bir fonksiyondur; bunun ayrıntısı Bölüm 11'de.

vectorTanim.cpp

```
#include <iostream>
#include <vector>

using namespace std;

// Bu fonksiyonu birazdan "vector'ü fonksiyona geçirmek" bölümünde açıklıyoruz;
// şimdilik elemanları yazdıran hazır bir kalıp olarak al.
void yazdir(const vector<int>& v) {
    cout << "boyut " << v.size() << ":";
    for (int x : v) {
        cout << ' ' << x;
    }
    cout << '\n';
}
```

```cpp
int main() {
    vector<int> a;                  // boş vector
    vector<int> b{10, 20, 30};      // ilk değer listesi
    vector<int> c(4, 7);            // 4 eleman, hepsi 7
    vector<int> d{4, 7};            // DİKKAT: 2 eleman, 4 ve 7
    yazdir(a);
    yazdir(b);
    yazdir(c);
    yazdir(d);

    for (int i = 1; i <= 3; i++) {
        a.push_back(i * i);
    }
    a[0] = 40;
    yazdir(a);
    cout << "İlk: " << a.front() << ", son: " << a.back() << '\n';
    a.pop_back();
    cout << "pop_back sonrası boyut " << a.size() << '\n';
    a.clear();
    cout << "clear sonrası boş mu? " << a.empty() << '\n';
    return 0;
}
```

Çıktı

```
boyut 0:
boyut 3: 10 20 30
boyut 4: 7 7 7 7
boyut 2: 4 7
boyut 3: 40 4 9
İlk: 40, son: 9
pop_back sonrası boyut 2
clear sonrası boş mu? 1
```

Parantez ve küme parantezi **farklı iş yapar**: `vector<int> c(4, 7)` "4 tane 7" demektir, `vector<int> d{4, 7}` ise "elemanları 4 ve 7 olan vector". Sınavda bu fark sorulur. `vector<int> v(4)` yazarsan 4 eleman çöp değil `0` olur: `vector` elemanlarına her zaman ilk değer verir. `empty()` bir `bool` döndürür; `cout` onu `true` için `1`, `false` için `0` yazar. Bellek yönetimini `vector` senin yerine yapar (nasıl yaptığını Bölüm 10'da anlayacaksın).

> **Sık Yapılan Hata: size() işaretsizdir**
>
> `size()` bir `int` değil, **işaretsiz** bir tamsayı (`size_t`) döndürür. `for (int i = 0; i < v.size(); i++)` yazarsan `g++ -Wall` ile "comparison of integer expressions of different signedness" uyarısı verir; sayacı `size_t` yap ya da indeks gerekmiyorsa aralık tabanlı `for` kullan. Asıl tuzak boş vector'de çıkar: `0 - 1` eksi bir değil **çok büyük bir pozitif sayıdır**.
>
> Örneğin boş `bos` için `for (size_t i = 0; i <= bos.size() - 1; i++)` koşulu her zaman doğrudur (`bos.size() - 1` en büyük sayıdır): döngü hiç bitmez ve gövdede `bos[i]` varsa ilk turda tanımsız davranış olur. `for (size_t i = v.size() - 1; i >= 0; i--)` ise boş olsun dolu olsun **her** `v` için hiç bitmez, çünkü işaretsiz `i` hiçbir zaman negatif olmaz: `0`'dan bir eksiltilince en büyük sayıya sarar ve `v[i]` sınır dışına çıkar. Sondan başa giderken `for (size_t i = v.size(); i > 0; i--)` yaz ve elemana `v[i - 1]` ile ulaş.

7.9 [] ile at() arasındaki fark

`v[i]` dizilerdeki gibi **denetimsizdir**: `i` sınır dışındaysa tanımsız davranış olur. **Boş bir vector'ün** `v[0]`'ına yazmak da sınır dışıdır, çünkü henüz hiç eleman yoktur; eleman eklemenin yolu `push_back`'tir.

```
vector<int> v;
v[0] = 5;              // tanımsız davranış: v'nin hiç elemanı yok
vector<int> w(3);
cout << w[3];          // tanımsız davranış: son indeks 2
```

Güvenli alternatif **`at(i)`**'dir: `v[i]` ile aynı elemanı verir, ama önce indeksi denetler. Sınır dışıysa tanımsız davranış yerine **`out_of_range` istisnası** fırlatır. İstisna, programın normal akışını kesen bir hata sinyalidir (ayrıntısı Bölüm 14'te). Yakalanmazsa program durur:

atHata.cpp

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    vector<int> v{10, 20, 30};
    cout << "v.at(1) = " << v.at(1) << '\n';
    cout << "v.at(5) = " << v.at(5) << '\n';
    cout << "Bu satıra gelinmez\n";
    return 0;
}
```

Çıktı – program çöker (Aborted)

```
v.at(1) = 20
terminate called after throwing an instance of 'std::out_of_range'
  what():  vector::_M_range_check: __n (which is 5) >= this->size() (which is 3)
v.at(5) =
```

Çıktıda `v.at(5) =` yazısı görünür ama sayı yazılmaz: `<<` soldan sağa ilerler, yazı basılır, sonra `at(5)` istisna fırlatır. Çıktının son kısmı g++'ya özgüdür (Visual Studio'da programı durduran farklı bir pencere çıkar), ama davranış aynıdır: program düzgün bir hata vererek durur, sessizce yanlış sonuç üretmez. İstisnayı `try` / `catch` ile yakalarsan program devam edebilir (şimdilik kalıp olarak al):

```
try { cout << v.at(5) << '\n'; }
catch (const out_of_range& e) { cout << "5. indeks yok\n"; }   // <stdexcept>
```

`[]`'i indeksin geçerli olduğundan eminsen (döngü sınırı `i < v.size()`), `at()`'i indeks kullanıcıdan ya da hesaptan geliyorsa kullan.

> **Sınavda Çıkar**
>
> "`v[10]` sınır dışıyken hata mesajı verir" yanlıştır: tanımsız davranıştır. "`v.at(10)` sınır dışıyken `0` döndürür" da yanlıştır: istisna fırlatır. Bu ikili her yıl çıkan bir çoktan seçmelidir.

7.10 vector'ü fonksiyona geçirmek

Dizi fonksiyona adres olarak geçiyordu. `vector` ise diğer tipler gibi davranır: **değer ile geçirilirse tamamı kopyalanır**. Bölüm 6'daki iki geçirme yolu burada da geçerlidir:

vectorGecir.cpp

```cpp
#include <iostream>
#include <vector>

using namespace std;

void kopyaylaDegistir(vector<int> v) {
    v.push_back(99);
    v[0] = 100;
    cout << "kopyada: " << v.size() << ' ' << v[0] << '\n';
}

void referanslaDegistir(vector<int>& v) {
    v.push_back(99);
    v[0] = 100;
}

int main() {
    vector<int> sayilar{1, 2, 3};
    kopyaylaDegistir(sayilar);
    cout << "dönüşte: " << sayilar.size() << ' ' << sayilar[0] << '\n';
    referanslaDegistir(sayilar);
    cout << "sonra: " << sayilar.size() << ' ' << sayilar[0] << '\n';
    return 0;
}
```

Çıktı

```
kopyada: 4 100
dönüşte: 3 1
sonra: 4 100
```

`kopyaylaDegistir` kendi kopyasını değiştirdi, `main`'deki `sayilar` etkilenmedi; `referanslaDegistir` gerçek vector'e eleman ekledi (her satırda önce boyut, sonra ilk eleman yazılır). Üç yazımın özeti:

Parametre	Kopyalanır mı?	Fonksiyon değiştirebilir mi?	Ne zaman?
`vector<int> v`	Evet (tamamı)	Yalnız kendi kopyasını	Kopyayla çalışmak istiyorsan (nadir)
`vector<int>& v`	Hayır	Evet, çağırana yansır	Fonksiyon vector'ü **değiştirecekse**
`const vector<int>& v`	Hayır	Hayır	Fonksiyon vector'ü **yalnız okuyacaksa**

Kural, dizilerdeki `const int a[]` kuralının aynısıdır; büyük bir vector'ü değer ile geçirmek her çağrıda tamamını kopyalar, bu yüzden okuyan fonksiyonda `const vector<int>&` yazmayı alışkanlık yap. Dizinin aksine **eleman sayısı için ayrı parametre gerekmez**: `v.size()` her zaman doğrudur. Fonksiyon `vector` **döndürebilir** de (`vector<int> karelerListesi(int n)`); C tarzı dizide bu mümkün değildi.

7.11 Kopyalama ve karşılaştırma

C tarzı dizide `b = a` derlenmez ve `a == b` adresleri karşılaştırırdı. `vector`'de ikisi de **beklediğin gibi** çalışır: `=` bütün elemanları kopyalar, `==` boyutların ve elemanların tek tek eşit olup olmadığına bakar.

vectorKopya.cpp

```cpp
#include <iostream>
#include <vector>

using namespace std;

int main() {
    vector<int> a{1, 2, 3};
    vector<int> b = a;              // kopya
    cout << (a == b) << '\n';
    b[0] = 99;                      // yalnız b değişir
    cout << a[0] << ' ' << b[0] << ' ' << (a != b) << '\n';
    b = {7, 8};                     // yeni liste atanabilir
    cout << b.size() << '\n';
    return 0;
}
```

Çıktı

```
1
1 99 1
2
```

7.12 İç içe vector: `vector<vector<int>>`

Her elemanı kendisi bir `vector<int>` olan `vector`, iki boyutlu tablo gibi davranır (tipin sonundaki iki `>` bitişik yazılabilir). `m[i][j]`: önce `m[i]` satır vector'üdür, sonra onun `j` numaralı elemanı alınır. `vector<vector<int>> m(2, vector<int>(3, 0));` "2 satır, her satır 3 tane `0`" demektir. C tarzı iki boyutlu dizinin aksine **satır uzunlukları farklı olabilir**, boyutlar çalışma anında belirlenir ve fonksiyona geçerken sütun sayısını yazmak gerekmez:

vectorMatris.cpp

```cpp
#include <iostream>
#include <vector>

using namespace std;

int main() {
    vector<vector<int>> m(2, vector<int>(3, 0));
    m[1][2] = 5;
    m.push_back({7, 8});                // uzunluğu farklı yeni satır
    for (const vector<int>& satir : m) {
        for (int x : satir) {
            cout << x << ' ';
        }
        cout << '\n';
    }
    cout << "Satır sayısı " << m.size() << ", ilk satırın uzunluğu "
         << m[0].size() << '\n';
    return 0;
}
```

Çıktı

```
0 0 0
0 0 5
7 8
Satır sayısı 3, ilk satırın uzunluğu 3
```

`m.size()` satır sayısı, `m[0].size()` ilk satırın sütun sayısıdır. Fonksiyona `const vector<vector<int>>&` ya da `vector<vector<int>>&` olarak geçer.

7.13 Temel algoritmalar

Sınavdaki dizi sorularının çoğu beş kalıba iner: toplam, ortalama, en büyük/en küçük (ve indeksi), sayma, koşula uyanları bulma. Hepsi bir `vector<int>` üzerinde, tek programda:

istatistik.cpp

```cpp
#include <iomanip>
#include <iostream>
#include <vector>

using namespace std;

int main() {
    vector<int> notlar{72, 45, 88, 60, 95, 38, 81, 67};
    int toplam{0};
    int enBuyuk{notlar[0]};
    size_t enBuyukIndeks{0};
    int enKucuk{notlar[0]};
    for (size_t i = 0; i < notlar.size(); i++) {
        toplam += notlar[i];
        if (notlar[i] > enBuyuk) {
            enBuyuk = notlar[i];
            enBuyukIndeks = i;
        }
        if (notlar[i] < enKucuk) {
            enKucuk = notlar[i];
        }
    }
    double ortalama = static_cast<double>(toplam) / notlar.size();
    cout << fixed << setprecision(2);
    cout << "Toplam: " << toplam << ", ortalama: " << ortalama << '\n';
    cout << "En büyük: " << enBuyuk << " (indeks " << enBuyukIndeks << ")\n";
    cout << "En küçük: " << enKucuk << '\n';
    cout << "Ortalamanın üstündekiler:";
    for (int x : notlar) {
        if (x > ortalama) {
            cout << ' ' << x;
        }
    }
    cout << '\n';
    return 0;
}
```

Çıktı

```
Toplam: 546, ortalama: 68.25
En büyük: 95 (indeks 4)
En küçük: 38
Ortalamanın üstündekiler: 72 88 95 81
```

- **Toplam** `0` ile başlar (Bölüm 5). **Ortalama** için bölenlerden biri `double`'a çevrilmelidir; yoksa tamsayı bölmesi olur ve ondalık kısım kaybolur.
- **En büyük/en küçük** için aday **ilk elemanla** başlatılır (`0` ile değil: bütün sayılar negatifse `0` hep kazanır). `>` kullandığın için tekrar eden en büyük değerlerde **ilk** bulunanın indeksi kalır.
- **Ortalamanın üstündekiler iki geçiş** ister: ortalama bilinmeden kimin üstte olduğu bilinemez.

Doğrusal arama

Doğrusal arama (linear search), vector'ü baştan sona gezer; aranan elemanı bulursa **indeksini** döndürür, bulamazsa geçerli bir indeks olamayacak değeri, gelenek olarak `-1` döndürür. `-1` işaretsiz bir `size_t` için anlamlı bir indeks olmadığı için indeksi `int` tutarız ve `size()`'ı bir kez `int`'e çevirririz:

dogrusalArama.cpp

```cpp
#include <iostream>
#include <vector>

using namespace std;

int ara(const vector<int>& v, int aranan) {
    int n{static_cast<int>(v.size())};
    for (int i = 0; i < n; i++) {
        if (v[i] == aranan) {
            return i;
        }
    }
    return -1;
}

int main() {
    vector<int> v{3, 7, 12, 7, 25, 31};
    cout << "ara(7)  -> " << ara(v, 7) << '\n';
    cout << "ara(31) -> " << ara(v, 31) << '\n';
    cout << "ara(10) -> " << ara(v, 10) << '\n';
    return 0;
}
```

Çıktı

```
ara(7)  -> 1
ara(31) -> 5
ara(10) -> -1
```

`return` ilk bulunanda fonksiyonu bitirir (7 iki kez var, ilk indeks döner); `-1` döngüden **sonra** gelir. En kötü durumda `n` karşılaştırma yapılır; vector'ün sıralı olması gerekmez.

Ters çevirme

Birçok algoritma iki elemanın **yer değiştirmesine** dayanır. Geçici değişken şarttır (Bölüm 2'deki takas): `v[i] = v[j]; v[j] = v[i];` yazarsan ikisi de `v[j]`'yi alır. Standart kütüphanenin hazır `swap(x, y)` fonksiyonu (`<utility>`) bu işi yapar. **Ters çevirme:** biri baştan, biri sondan ilerleyen iki indeks elemanları değiştirir ve birbirine yaklaşır; birbirini geçince biter.

tersCevir.cpp

```cpp
#include <iostream>
#include <utility>
#include <vector>

using namespace std;

void tersCevir(vector<int>& v) {
    int bas{0};
    int son{static_cast<int>(v.size()) - 1};
    while (bas < son) {
        swap(v[bas], v[son]);
        bas++;
        son--;
```

```
    }
}

int main() {
    vector<int> a{1, 2, 3, 4, 5, 6};
    vector<int> b{1, 2, 3, 4, 5};
    tersCevir(a);
    tersCevir(b);
    for (int x : a) { cout << x << ' '; }
    cout << '\n';
    for (int x : b) { cout << x << ' '; }
    cout << '\n';
    return 0;
}
```

Çıktı

```
6 5 4 3 2 1
5 4 3 2 1
```

`a` için (`bas`, `son`) = (0, 5), (1, 4), (2, 3) çiftleri sırayla yer değiştirir; sonra `bas = 3`, `son = 2` olur ve döngü biter. Eleman sayısı **tek** olunca ortadaki yerinde kalır (`b`'de 3). `son`'u `int` tutmak sayesinde boş vector'de `son = -1` olur ve döngü hiç girmez; `size_t` kullansaydın `0 - 1` çok büyük bir sayı olurdu.

7.14 Sıralama: kabarcık ve seçmeli

Diziyi küçükten büyüğe dizmek **sıralama** (sorting) problemidir. (C++'ın hazır bir sıralama fonksiyonu var; onu Bölüm 15'te göreceksin. Bu bölümde amaç, mantığı elle kurabilmen.) **Kabarcık sıralaması** (bubble sort): komşular karşılaştırılır, soldaki büyükse yer değiştirilir; bir **geçiş** (pass) sonunda en büyük eleman kabarcık gibi yüzüp sona ulaşır. **Seçmeli sıralama** (selection sort): her turda sıralanmamış kısmın **en küçüğü** bulunur ve o kısmın başındaki elemanla değiştirilir.

siralama.cpp

```
#include <iostream>
#include <utility>
#include <vector>

using namespace std;

void yazdir(const vector<int>& v) {
    for (int x : v) {
        cout << x << ' ';
    }
    cout << '\n';
}

void kabarcikSirala(vector<int>& v) {
    int n{static_cast<int>(v.size())};
    for (int gecis = 1; gecis < n; gecis++) {
        for (int i = 0; i < n - gecis; i++) {
            if (v[i] > v[i + 1]) {
                swap(v[i], v[i + 1]);
            }
        }
        cout << gecis << ". geçiş: ";
        yazdir(v);
    }
}

void secmeliSirala(vector<int>& v) {
```

```
    int n{static_cast<int>(v.size())};
    for (int i = 0; i < n - 1; i++) {
        int enKucuk{i};
        for (int j = i + 1; j < n; j++) {
            if (v[j] < v[enKucuk]) {
                enKucuk = j;
            }
        }
        swap(v[i], v[enKucuk]);
        cout << "i=" << i << " (en küçük indeks " << enKucuk << "): ";
        yazdir(v);
    }
}

int main() {
    vector<int> a{5, 1, 4, 2, 8};
    vector<int> b{64, 25, 12, 22, 11};
    cout << "Kabarcık, başlangıç: ";
    yazdir(a);
    kabarcikSirala(a);
    cout << "Seçmeli, başlangıç: ";
    yazdir(b);
    secmeliSirala(b);
    return 0;
}
```

Çıktı

```
Kabarcık, başlangıç: 5 1 4 2 8
1. geçiş: 1 4 2 5 8
2. geçiş: 1 2 4 5 8
3. geçiş: 1 2 4 5 8
4. geçiş: 1 2 4 5 8
Seçmeli, başlangıç: 64 25 12 22 11
i=0 (en küçük indeks 4): 11 25 12 22 64
i=1 (en küçük indeks 2): 11 12 25 22 64
i=2 (en küçük indeks 3): 11 12 22 25 64
i=3 (en küçük indeks 3): 11 12 22 25 64
```

Kabarcıkta ilk geçişte `5 1 4 2 8` için sırayla (5,1), (5,4), (5,2) değişir, (5,8) değişmez: `1 4 2 5 8`. Her geçişte iç döngü `n - gecis` karşılaştırma yapar, çünkü sondaki `gecis - 1` eleman zaten yerindedir. Bu örnekte vector 2. geçişte sıralandı, 3. ve 4. geçişte hiç değiş-tokuş olmadı. **Hiç değiş-tokuş olmayan geçiş** vector'ün sıralı olduğunu kanıtlar; bir `bool` bayrağı tutup `break` ile çıkarsan gereksiz geçişler atlanır. Seçmelide `i` yalnız `n - 2`'ye kadar gider (son eleman kendiliğinden yerine oturur); `i = 3` satırında 25 kendisiyle yer değiştirdi (zararsız).

Karşılaştırma sayısı ikisinde de `n(n-1)/2`'dir. Seçmelide değiş-tokuş en çok `n - 1`, kabarcıkta çok daha fazla olabilir. `n` iki katına çıkınca iş yaklaşık **dört katına** çıkar ("O(n^2)"). **Büyükten küçüğe** sıralamak için karşılaştırma tersine çevrilir.

> **Sınavda Çıkar**
>
> "Kabarcık sıralamasının `k` geçişi sonundaki vector nedir?" sorusunda sondaki `k` eleman **kesinlikle yerindedir** (en büyük `k` eleman); baştakiler hâlâ karışık olabilir. Seçmelide `k` tur sonra ise baştaki `k` eleman, yani en küçükler yerindedir. Tabloyu geçiş geçiş çıkar, kestirmeye kalkma.

7.15 std::array (İleri)

C tarzı dizinin boyutunu bilen, kopyalanabilen kardeşi `std::array`'dir (`<array>`). Boyutu `vector` gibi çalışma anında değil **derleme sırasında** sabittir ve tipin parçasıdır: `array<int, 5>` "5 tane `int`" demektir.

arrayOrnek.cpp

```cpp
#include <array>
#include <iostream>

using namespace std;

int main() {
    array<int, 5> a{10, 20, 30, 40, 50};
    array<int, 5> b = a;                // kopyalanabilir
    b[0] = 1;
    cout << a.size() << ' ' << a.at(2) << ' ' << a[0] << ' ' << b[0] << '\n';
    return 0;
}
```

Çıktı

```
5 30 10 1
```

`array` fonksiyona `const array<int, 5>&` olarak geçer. `[]` denetimsiz, `at()` denetimlidir (`vector` gibi). `array<int, 5> x;` yazıp ilk değer vermezsen elemanlar çöp değerli kalır; sıfırlamak için `{}` ekle.

İpucu: Hangisini seçeceksin?

Varsayılan olarak `vector` kullan. Boyut derleme sırasında sabit ve küçükse `array`, eski kodla uğraşıyorsan C tarzı dizi. Bu kitabın geri kalanında elemanları tutmak için `vector` göreceksin.

Bölüm Özeti – Bilmen Gerekenler

- **Dizi** aynı tipten elemanları tek adla tutar: `tip ad[N];`. Boyut derlemede bilinen sabittir (`const int N{4};`); elemanlar bellekte art arda durur. Çalışma anında belirlenen boyut için `int a[n];` yazma, `vector` kullan.
- İlk değer listesi: eksikler `0` olur (`{1, 2, 3}`), `{0}` ve `{}` hepsini sıfırlar, `{7}` hepsini 7 yapmaz, `int a[] = {...}` boyutu listeden çıkarır. Fazla eleman C++'ta derleme hatasıdır. Listesiz yerel dizi çöp taşır.
- **İndeks 0'dan başlar**, son eleman `a[N-1]`'dir. C++ sınırı **denetlemez**: sınır dışı erişim tanımsız davranıştır. Döngüde `i < N` yaz. İndeks gerekmiyorsa **aralık tabanlı for** kullan (`int x`: kopya, `int& x`: değiştirir, `const int& x`: okur).
- Eleman sayısı `sizeof(a) / sizeof(a[0])` ile bulunur ama yalnız dizinin tanımlandığı yerde; fonksiyonda **yanlış** sonuç verir. Dizi fonksiyona **adres olarak** geçer: değişiklik çağırana yansır; okuyan fonksiyon `const int a[]` alır, `n` ayrı parametredir.
- İki boyutlu dizi `m[i][j]` (i satır, j sütun); köşegenler `i == j`, `i + j == N - 1`; transpoz `t[j][i] = m[i][j]`. Fonksiyona geçirirken **sütun sayısı zorunludur**: `int m[][SUTUN]`.
- `vector` (`<vector>`): `vector<int> v(5, 7)` (5 tane 7), `vector<int> v{5, 7}` (iki eleman). `push_back`, `size()` (işaretsiz: `size_t`), `back()`, `pop_back()`, `empty()`, `clear()`. `v = w` kopyalar, `v == w` elemanları karşılaştırır.
- `v[i]` denetimsizdir (sınır dışıysa tanımsız davranış; boş vector'e `v[0] = x` yazma); `v.at(i)` sınır dışıysa `out_of_range` istisnası fırlatır. Boş vector'de `size() - 1` çok büyük sayıdır.
- Okuyan fonksiyon `const vector<int>&`, değiştiren `vector<int>&` alır; değer ile geçirmek kopyalar. `vector<vector<int>>` iki boyutlu tablodur. `std::array<int, N>` boyutu derlemede sabit, kopyalanabilen dizidir; varsayılan tercih `vector`.
- Kalıplar: toplam (`0`'dan başla), ortalama (`static_cast<double>(toplam) / n`), en büyük/en küçük (ilk elemandan başla), doğrusal arama (bulunursa indeks, bulunamazsa `-1`, döngüden **sonra**), ters çevirme (iki indeks ve `swap`).

- Kabarcık: `k` geçiş sonunda en büyük `k` eleman yerindedir (iç döngü `i < n - gecis`). Seçmeli: her turda en küçüğü bul, `i` konumuna koy. İkisi de `n(n-1)/2` karşılaştırma yapar.

Sorular

A) Çoktan Seçmeli

1 `vector<int> v{4, 7};` ve `vector<int> w(4, 7);` tanımlarından sonra `v.size()` ile `w.size()` sırasıyla kaçtır?

(A) 4 ve 4
(B) 2 ve 2
(C) 2 ve 4
(D) 4 ve 2
(E) İkinci tanım derlenmez.

2 `vector<int> v{10, 20, 30};` için aşağıdakilerden hangisi doğrudur?

(A) `v.at(3)` bir `out_of_range` istisnası fırlatır; `v[3]` ise denetlenmez ve tanımsız davranıştır.
(B) `v.at(3)` ve `v[3]` ikisi de `0` döndürür.
(C) `v[3]` bir `out_of_range` istisnası fırlatır; `v.at(3)` tanımsız davranıştır.
(D) `v[3]` derleme hatası verir.
(E) `v.at(3)` vector'e yeni bir eleman ekler.

3 `f(vector<int> v)` ve `g(vector<int>& v)` fonksiyonları, aldıkları vector'e `push_back` ile birer eleman ekliyor. `vector<int> v{0};` ile başlayıp sırayla `g(v); f(v); g(v);` çağrılırsa `v.size()` kaç olur?

(A) 0 (B) 1 (C) 2 (D) 3 (E) 4

4 `int m[3][4];` dizisini `m[i][j]` biçiminde kullanan ve satır sayısını ayrı parametre olarak alan fonksiyon için hangi başlık **geçerlidir**?

(A) `void yaz(int m[][], int satir)`
(B) `void yaz(int m[][4], int satir)`
(C) `void yaz(int m[3][], int satir)`
(D) `void yaz(int m[], int satir)`
(E) `void yaz(int m[4][3], int satir)`

5 `vector<int> bos;` tanımından sonra `bos.size() - 1` ifadesi hakkında hangisi doğrudur?

(A) Değeri `-1`'dir.
(B) Değeri `0`'dır.
(C) Boş vector'den çıkarma yapılamayacağı için derleme hatası verir.
(D) Çalışma anında `out_of_range` istisnası fırlatır.
(E) `size()` işaretsiz olduğundan sonuç çok büyük bir pozitif sayıdır.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v{3, 0, 4, 1, 2};
    cout << v[v[0]] << '\n';
    cout << v[v[v[0]]] << '\n';
    v[v[1]] = 9;
    int toplam{0};
    for (size_t i = 1; i < v.size(); i++) {
        toplam += v[i];
    }
    cout << toplam << '\n';
    cout << v[0] << ' ' << v[v[4]] << '\n';
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```
#include <iostream>
#include <vector>
using namespace std;

void a(int x) { x = x * 2; }
void b(int& x) { x = x * 2; }
void c(vector<int> v) { v[0] = 100; v.push_back(1); }
void d(vector<int>& v) { v[0] = v[0] + 1; v.push_back(5); }

int main() {
    int s{5};
    vector<int> v{1, 2, 3};
    a(s);
    b(s);
    c(v);
    d(v);
    cout << s << '\n';
    for (int x : v) { cout << x << ' '; }
    cout << '\n' << v.size() << '\n';
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
#include <stdexcept>
#include <vector>
using namespace std;

int main() {
    vector<vector<int>> t(2, vector<int>(3, 1));
    t[1][2] = 5;
    t[0].push_back(8);
    int toplam{0};
    for (const vector<int>& satir : t) {
        for (int x : satir) { toplam += x; }
    }
    cout << t.size() << ' ' << t[0].size() << ' ' << t[1].size() << '\n';
    cout << toplam << '\n';
    cout << t.at(1).at(2) << '\n';
    cout << t[0].at(3) << '\n';
    try {
        cout << t.at(2).at(0) << '\n';
    } catch (const out_of_range& e) {
        cout << "sınır dışı\n";
    }
}
```

C) Kod Yazma

9 Klavyeden önce not sayısını, sonra o kadar tamsayı notu okuyup bir `vector<int>` içinde saklayan `notAnalizi` programını yaz. En büyük notu ve **indeksini**, ortalamayı (iki ondalık basamak) ve ortalamanın **üstündeki** not sayısını yazdırsın (not sayısı en az 1).

```
Girdi : 6
        55 80 92 47 80 66
Çıktı : En büyük: 92 (indeks 2)
        Ortalama: 70.00
        Ortalamanın üstünde: 3 not
```

10 Klavyeden `vector<vector<int>>` içine **3 × 3 bir matris** okuyan `matrisAnaliz` programını yaz. Her **sütunun** toplamını, sonra ana ve ikincil köşegen toplamlarını yazdırsın.

```
Girdi : 2 7 1
        4 5 9
        3 8 6
Çıktı : 1. sütun: 9
        2. sütun: 20
        3. sütun: 16
        Ana köşegen: 13
        İkincil köşegen: 9
```

Çözümler

Cevap anahtarı

1 C	2 A	3 D	4 B	5 E

1 Cevap: C

Parantez ve küme parantezi farklıdır: `v{4, 7}` elemanları 4 ve 7 olan **iki elemanlı** vector'dür, `w(4, 7)` **dört tane 7** tutar. A ve B bu farkı yok sayar, D sırayı ters alır, E yanlış: iki tanım da geçerlidir.

2 Cevap: A

`at(i)` indeksi denetler ve sınır dışında `out_of_range` istisnası fırlatır; `[]` denetlemez, `v[3]` tanımsız davranıştır (geçerli indeksler 0, 1, 2). B, C ve D bu ikiliyi karıştırır ya da uydurur; E yanlış, `at()` hiçbir zaman eleman eklemez.

3 Cevap: D

Başlangıçta boyut 1. `g(v)` **referansla** alır ve gerçek vector'e ekler: 2. `f(v)` **kopyayı** değiştirir, `v` etkilenmez: 2. İkinci `g(v)`: **3**. E, `f`'nin eklemesini de sayar; C, ikinci `g`'yi unutur.

4 Cevap: B

Çok boyutlu dizi parametresinde yalnız **ilk** boyut atlanabilir; derleyici `m[i][j]`'nin yerini `i × SUTUN + j` ile bulur, bu yüzden sütun sayısı (4) şarttır. A ve C'de sütun sayısı yok. D'de `m[i]` bir `int`'tir, `m[i][j]` yazılamaz. E derlenir, ama sütun sayısı 3'tür; `int m[3][4]` bu başlığa geçirilemez (tür uyuşmazlığı).

5 Cevap: E

`size()` işaretsiz (`size_t`) döndürür; `0 - 1` işaretsiz aritmetikte eksi bir değil, en büyük değerdir (64 bitte 18446744073709551615). Davranış tanımlıdır ama istediğin sonuç değildir. A ve B yanlış, C yanlış: ifade geçerlidir. D yanlış, `size()` istisna fırlatmaz.

6

`v = {3, 0, 4, 1, 2}`. İçten dışa: `v[v[0]]` = `v[3]` = **1**; `v[v[v[0]]]` = `v[v[3]]` = `v[1]` = **0**. `v[v[1]] = 9;` satırında `v[1]` = 0 olduğundan `v[0] = 9` olur: `{9, 0, 4, 1, 2}`. Toplam `i = 1`'den başlar, `v[0]` **toplamda yoktur**: 0 + 4 + 1 + 2 = **7**. Son satır: `v[0]` = 9, `v[v[4]]` = `v[2]` = 4.

Çıktı

```
1
0
7
9 4
```

7

`a(s)` kopyayı değiştirir, `s` 5 kalır; `b(s)` referansla alır, `s` **10** olur. `c(v)` vector'ün **kopyasına** `100` yazar ve `1` ekler, `v` etkilenmez. `d(v)` gerçek vector'ü değiştirir: `v[0]` 2 olur, sona `5` eklenir: `{2, 2, 3, 5}`, boyut 4.

Çıktı
```
10
2 2 3 5
4
```

8

`t` önce 2 satır, her satır `{1, 1, 1}`. `t[1][2] = 5` ikinci satırı `{1, 1, 5}` yapar; `t[0].push_back(8)` yalnız ilk satırı uzatır: `{1, 1, 1, 8}`. Boyutlar `2 4 3`; toplam (1+1+1+8) + (1+1+5) = **18**. `t.at(1).at(2)` = `5`, `t[0].at(3)` = `8` geçerlidir. `t.at(2)` sınır dışıdır (satırlar 0 ve 1): `out_of_range` fırlatılır ve `catch` çalışır.

Çıktı
```
2 4 3
18
5
8
sınır dışı
```

9

Yaklaşım: Boyut `n` klavyeden geldiği için `vector<int> notlar(n)` kur (elemanlar `0` ile başlar) ve okurken hem toplamı hem en büyüğün **indeksini** güncelle. Ortalamayı `static_cast<double>(toplam) / n` ile hesapla (tamsayı bölmesi tuzağı), üstündekileri **ikinci geçişte** say.

notAnalizi.cpp
```cpp
#include <iomanip>
#include <iostream>
#include <vector>

using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> notlar(n);
    int toplam{0};
    int enBuyuk{0};
    for (int i = 0; i < n; i++) {
        cin >> notlar[i];
        toplam += notlar[i];
        if (notlar[i] > notlar[enBuyuk]) {
            enBuyuk = i;
        }
    }
    double ortalama = static_cast<double>(toplam) / n;
    int ustunde{0};
    for (int x : notlar) {
        if (x > ortalama) {
            ustunde++;
        }
    }
    cout << "En büyük: " << notlar[enBuyuk] << " (indeks " << enBuyuk << ")\n";
    cout << fixed << setprecision(2) << "Ortalama: " << ortalama << '\n';
    cout << "Ortalamanın üstünde: " << ustunde << " not\n";
    return 0;
}
```

Çıktı

```
6
55 80 92 47 80 66
En büyük: 92 (indeks 2)
Ortalama: 70.00
Ortalamanın üstünde: 3 not
```

10

Yaklaşım: `m(N, vector<int>(N))` ile 3 × 3 tablo kur, iç içe iki `for` ile oku. Sütun toplamında `j` sabit, `i` değişir; köşegenler tek döngüyle bulunur: `m[i][i]` ve `m[i][N - 1 - i]`.

matrisAnaliz.cpp

```cpp
#include <iostream>
#include <vector>

using namespace std;

int main() {
    const int N{3};
    vector<vector<int>> m(N, vector<int>(N));
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            cin >> m[i][j];
        }
    }
    for (int j = 0; j < N; j++) {
        int toplam{0};
        for (int i = 0; i < N; i++) {
            toplam += m[i][j];
        }
        cout << j + 1 << ". sütun: " << toplam << '\n';
    }
    int ana{0};
    int ikincil{0};
    for (int i = 0; i < N; i++) {
        ana += m[i][i];
        ikincil += m[i][N - 1 - i];
    }
    cout << "Ana köşegen: " << ana << '\n';
    cout << "İkincil köşegen: " << ikincil << '\n';
    return 0;
}
```

Çıktı

```
2 7 1
4 5 9
3 8 6
1. sütun: 9
2. sütun: 20
3. sütun: 16
Ana köşegen: 13
İkincil köşegen: 9
```

BÖLÜM 8

string ve Karakter İşlemleri

- **Bu Bölümde Öğreneceklerin**
  - Bir karakterin bellekte sayı olarak tutulduğunu ve ASCII tablosunu
  - Tek bir karakterin harf mi, rakam mı olduğunu denetlemeyi ve büyük/küçük harfe çevirmeyi
  - `string` tipinin neden C'deki karakter dizilerinden çok daha rahat olduğunu
  - String'leri birleştirmeyi, karşılaştırmayı, içinde arama yapmayı ve parçalarını değiştirmeyi
  - Bir string'in karakterlerini döngüyle dolaşmayı
  - Klavyeden boşluklu bir satır okumayı
  - String ile sayı arasında dönüşüm yapmayı
  - Türkçe harflerin neden uzunluk hesabını şaşırttığını
  - Ters çevirme, palindrom ve harf sayma gibi klasik string sorularını

8.1 Karakter, sayıdır

Bilgisayar yalnız sayı tutar; harf diye bir şey bilmez. Bir karakteri saklamak için herkesin üzerinde anlaştığı bir tablo kullanılır: her karaktere bir **kod** (sayı) verilir. İngilizce harfleri, rakamları ve noktalama işaretlerini kapsayan bu tabloya **ASCII** (American Standard Code for Information Interchange) denir. Bölüm 2'de gördüğün `char` tipi tam olarak bu kodu tutan 1 baytlık bir tamsayıdır. `'A'` yazdığında derleyici bunu 65 sayısı olarak saklar.

Karakterler	Kodlar	Not
`'\0'`	0	Null karakter; "değer yok" anlamında
`'\n'`	10	Satır sonu
`' '`	32	Boşluk
`'0'` … `'9'`	48 … 57	Rakamlar art arda gelir
`'A'` … `'Z'`	65 … 90	Büyük harfler art arda gelir
`'a'` … `'z'`	97 … 122	Küçük harfler art arda gelir

Tablodan üç sonuç çıkar ve bölümün geri kalanında hepsini kullanacaksın:

1. Rakam karakterinden sayıya geçmek için `'0'` çıkarılır: `'7' - '0'` = 55 - 48 = 7.
2. Küçük harf ile büyük harf arasında hep 32 fark vardır: `'a' - 'A'` = 32.
3. Harfler alfabe sırasındadır: `'a'` + 2 = `'c'`.

`cout` bir `char` gördüğünde karakteri yazdırır; bir `int` gördüğünde sayıyı yazdırır. Aritmetik işlemde `char` önce `int`'e çevrilir, bu yüzden sonuç artık karakter değil sayıdır:

karakterKod.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    char harf{'A'};
    cout << harf << " " << static_cast<int>(harf) << '\n';
    cout << harf + 1 << '\n';
    cout << static_cast<char>(harf + 1) << '\n';
    harf++;
    cout << harf << '\n';
    cout << 'a' - 'A' << " " << '7' - '0' << '\n';
    char rakam{static_cast<char>(5 + '0')};
    cout << rakam << '\n';
    return 0;
}
```

Çıktı

```
A 65
66
B
B
32 7
5
```

Satır satır: `harf` yazılınca `A`, `static_cast<int>` ile sayıya çevrilince 65 çıkar. `harf + 1` işlemi `int` ürettiği için `66` yazdırır; `B` görmek istiyorsan sonucu `static_cast<char>` ile karaktere geri çevirirsin (Bölüm 2'deki C++ tarzı dönüştürme). `harf++` ise değişkenin kendisini 66 yapar ve değişken hâlâ `char` olduğundan `B` yazılır. Son satırda `5 + '0'` rakamın karakterini üretir: 5 sayısından `'5'` karakterine.

> **Sınavda Çıkar**
>
> "`char c = 'b'; cout << c + 1;` ne yazdırır?" Cevap `99`, `c` değil. `char` ile `int` toplanınca sonuç `int` olur; `'b'` = 98, 98 + 1 = 99. Karakter görmek için `static_cast<char>(c + 1)` yazmak gerekir. Bu tuzak her yıl çıkar.

8.2 cctype: tek karakter üzerinde işlem

Bir karakterin harf mi, rakam mı olduğunu kod tablosuna bakarak kendin de yazabilirsin (`c >= '0' && c <= '9'`) ama hazır fonksiyonlar daha okunaklıdır ve hata riski taşımaz. Hepsi `#include <cctype>` ister:

Fonksiyon	Doğru döndürür, eğer karakter...
`isalpha(c)`	harfse
`isdigit(c)`	rakamsa (`'0'` ... `'9'`)
`isalnum(c)`	harf ya da rakamsa
`isupper(c)` / `islower(c)`	büyük / küçük harfse
`isspace(c)`	boşluk, sekme ya da `'\n'` gibi boşluk karakteriyse
`ispunct(c)`	noktalama işaretiyse (`.`, `,`, `!`, `?` ...)
`toupper(c)` / `tolower(c)`	büyük / küçük harfe çevrilmiş halini döndürür; harf değilse aynısını

karakterSinif.cpp

```cpp
#include <iostream>
#include <cctype>

using namespace std;

void sinifla(char c) {
    cout << '\'' << c << "' -> ";
    if (isalpha(c)) {
        cout << (isupper(c) ? "buyuk harf" : "kucuk harf");
    } else if (isdigit(c)) {
        cout << "rakam";
    } else if (isspace(c)) {
        cout << "bosluk";
    } else if (ispunct(c)) {
        cout << "noktalama";
    } else {
        cout << "baska";
    }
    cout << '\n';
}

int main() {
    sinifla('k');
    sinifla('K');
    sinifla('7');
    sinifla(' ');
    sinifla('?');
    return 0;
}
```

Çıktı

```
'k' -> kucuk harf
'K' -> buyuk harf
'7' -> rakam
' ' -> bosluk
'?' -> noktalama
```

Fonksiyonların adı `is...` ile başlayanlar "evet/hayır" sorusu sorar. Kesin değeri `1` değildir: "sıfırdan farklı" demektir. Bu yüzden `if (isdigit(c))` yaz, `if (isdigit(c) == 1)` yazma. (C kitabında da aynı kural vardı.)

`toupper` ve `tolower` ise karakteri **değiştirmez**, dönüştürülmüş halini **döndürür**. Asıl tuzak: ikisi de `char` değil `int` döndürür (C'den kalma bir alışkanlık). `cout` bir `int` gördüğü için karakter yerine sayı yazdırır:

donusumTuzagi.cpp

```cpp
#include <iostream>
#include <cctype>

using namespace std;

int main() {
    char k{'q'};
    cout << toupper(k) << '\n';
    cout << static_cast<char>(toupper(k)) << '\n';
    char buyuk = toupper(k);
    cout << buyuk << '\n';
    toupper(k);
    cout << k << '\n';
    return 0;
}
```

```
Çıktı
81
Q
Q
q
```

İlk satır `'Q'`'nun kodu olan `81` yazdırdı. Karakter görmek için ya `static_cast<char>` kullanırsın ya da sonucu bir `char` değişkene atarsın (üçüncü satır). `toupper(k);` satırı ise sonucu hiçbir yere koymadığı için işe yaramaz: `k` hâlâ `q`.

> **Sık Yapılan Hata: toupper'ı cout'a doğrudan vermek**
>
> `cout << toupper(c);` ekrana harf değil **sayı** yazar, çünkü `toupper` `int` döndürür. `toupper(c);` tek başına yazarsan hiçbir şey değişmez. Doğru kalıplar: `c = toupper(c);` ve `cout << static_cast<char>(toupper(c));`.
>
> String'e eklerken de aynı tuzak var: `s += toupper(c);` çalışır, çünkü `+=` gelen `int`'i `char` olarak ekler. Ama `s = s + toupper(c);` derlenmez (`no match for 'operator+'`): `string` ile `int`'i toplayan bir `+` yoktur. Orada `s = s + static_cast<char>(toupper(c));` yaz.

`cctype` fonksiyonları yalnız **ASCII** karakterler içindir; `ç ğ ı ö ş ü` harflerini harf saymazlar (nedeni bu bölümün Türkçe başlığında). Ayrıca teknik bir ayrıntı: parametreleri `int`'tir ve yalnız 0-255 aralığındaki değerler için tanımlıdır. `char` işaretli olduğundan ASCII dışı karakterler negatif görünebilir ve doğrudan vermek tanımsız davranıştır. ASCII metinde sorun çıkmaz; Türkçe harf içerebilecek metinde güvenli yazım `isalpha(static_cast<unsigned char>(c))`'dir.

8.3 std::string: metin için gerçek tip

C'de metin, sonunda özel bir işaret bulunan `char` dizisiydi; uzunluğu kendin izler, kopyalamak için fonksiyon çağırır, yer yetmezse taşmaya karşı kendin önlem alırdın. C++'ın standart kütüphanesi bunların hepsini senin yerine yapan hazır bir tip sunar: **`string`**. `#include <string>` ile gelir. Bölüm 2'den beri kullandığın bu tip aslında bir **sınıftır** (class); sınıfları Bölüm 11'de yazmayı öğreneceksin, şimdilik hazır bir araç olarak kullanıyoruz.

`string` metni senin yerine yönetir: uzunluğunu kendisi bilir, metin uzadıkça yerini kendisi büyütür, `=` ile kopyalanır, `+` ile birleştirilir, `==` ile karşılaştırılır. Yani sayılarla yaptığın gibi rahat çalışırsın.

Tanımlama ve ilk değer

stringTanim.cpp

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    string ad{"Ali"};
    string soyad = "Veli";
    string bos;
    string yildiz(5, '*');
    string kopya{ad};

    kopya += "cim";
    cout << ad << " " << soyad << " " << kopya << '\n';
    cout << "[" << bos << "] " << yildiz << '\n';
    cout << bos.empty() << " " << ad.empty() << '\n';
    return 0;
}
```

Çıktı

```
Ali Veli Alicim
[] *****
1 0
```

- **`string ad{"Ali"};`** ve **`string soyad = "Veli";`**: İkisi de ilk değer vermedir. Metin çift tırnak içinde yazılır; bunu Bölüm 1'den **string sabiti** olarak tanıyorsun.
- **`string bos;`**: İlk değer vermesen bile `string` **boş metin** (`""`) ile başlar. Çöp değer sorunu yoktur; bu, `int`'ten farklıdır.
- **`string yildiz(5, '*');`**: "5 tane `'*'`" demektir: `*****`.
- **`string kopya{ad};`**: Kopya oluşturur. `kopya`'yı değiştirmek `ad`'ı etkilemez; çıktıda `ad` hâlâ `Ali`, `kopya` `Alicim`.
- **`empty()`** metin boşsa `true` döndürür. `cout` bir `bool`'u `1` ya da `0` olarak yazar; çıktıdaki `1 0` budur.

> **Sık Yapılan Hata: string(5, '*') yerine süslü parantez**
>
> `string yildiz{5, '*'};` yazarsan beklediğin `*****` oluşmaz: süslü parantez, değerleri tek tek karakter listesi sanır ve 5 kodlu görünmez bir karakterle `'*'` içeren **2 karakterlik** bir string üretir. "Şu karakterden şu kadar" kalıbı için **yuvarlak parantez** kullanılır: `string yildiz(5, '*');`.

Birleştirme

`+` iki string'i uç uca ekler, `+=` soldakinin sonuna ekler. Bir yanı `string` olduğu sürece diğer yanı string sabiti, `string` ya da tek `char` olabilir:

birlestirme.cpp

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    string ad{"Ali"};
    string soyad{"Yilmaz"};
    string tam = ad + " " + soyad;
    string selam = "Merhaba " + tam;
    tam += '!';
    tam += " :)";
    cout << selam << '\n';
    cout << tam << '\n';
    return 0;
}
```

Çıktı

```
Merhaba Ali Yilmaz
Ali Yilmaz! :)
```

İfade soldan sağa işlenir: `ad + " "` bir `string` üretir, sonra o string `soyad`'la birleşir. Her adımda en az bir taraf `string` olduğu için sorun çıkmaz. İki **string sabitini** doğrudan toplamaya kalkarsan C'den kalma bir kural devreye girer:

sabitToplama.cpp

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    string s = "Ali" + " Veli";
    cout << s << '\n';
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
sabitToplama.cpp: In function 'int main()':
sabitToplama.cpp:7:22: error: invalid operands of types 'const char [4]' and 'const char [6]' to
binary 'operator+'
    7 |     string s = "Ali" + " Veli";
      |                ~~~~~ ^ ~~~~~~~
      |                |       |
      |                |       const char [6]
      |                const char [4]
```

String sabiti `string` değil, C'deki gibi ham bir karakter dizisidir ve iki ham diziyi toplamak anlamsızdır. Çözüm, zincirin başına bir `string` koymak ya da sabitleri yan yana yazmaktır (`"Ali" " Veli"` derleyici tarafından birleştirilir).

Not: macOS'ta g++ yazarsan

macOS'taki `g++` komutu aslında Apple clang'dir. Yukarıdaki hata aynı sebeple clang'de de çıkar ama mesajın metni farklıdır (`invalid operands to binary expression ('const char[4]' and 'const char[6]')` gibi). Bu kitaptaki çıktılar gerçek g++ ile üretilmiştir; kendi bilgisayarında bire bir aynı cümleyi görmeyebilirsin.

Sık Yapılan Hata: İki string sabitini toplamak

`"Merhaba " + "Dunya"` derlenmez; `"Merhaba " + ad + "!"` derlenir, çünkü soldan sağa ilk toplama (`"Merhaba " + ad`) bir `string` verir. Ama `"Merhaba " + "Dunya" + ad` yine derlenmez, çünkü ilk toplama iki sabit arasındadır. Sayıyı metne eklemek için `"Not: " + 85` de yazamazsın; bunun için `to_string` var (aşağıda).

Karşılaştırma

`==`, `!=`, `<`, `>`, `<=`, `>=` string'lerde **içeriği** karşılaştırır. Bu, C'den en büyük farktır: orada `==` yalnız adres karşılaştırırdı. Sıralama **karakter kodlarına göre, soldan sağa** yapılır (sözlük sırası, **leksikografik karşılaştırma**): ilk farklı karakter sonucu belirler; biri diğerinin başlangıcıysa kısa olan küçüktür.

karsilastirma.cpp

```cpp
#include <iostream>
#include <string>

using namespace std;

void karsilastir(string a, string b) {
    cout << a << " ve " << b << ": ";
    if (a == b) {
        cout << "esit\n";
    } else if (a < b) {
        cout << "ilki kucuk\n";
    } else {
        cout << "ilki buyuk\n";
    }
}

int main() {
    karsilastir("Ali", "Ali");
    karsilastir("Ali", "Veli");
    karsilastir("Ali", "Alice");
    karsilastir("ali", "Ali");
    karsilastir("kedi", "Zebra");
    return 0;
}
```

Çıktı

```
Ali ve Ali: esit
Ali ve Veli: ilki kucuk
Ali ve Alice: ilki kucuk
ali ve Ali: ilki buyuk
kedi ve Zebra: ilki buyuk
```

`Ali` ile `Veli` ilk harfte ayrılır (`'A'` = 65, `'V'` = 86). `Ali` ile `Alice`: üç harf aynı, `Ali` orada biter, kısa olan küçüktür. `"ali"` ile `"Ali"`: küçük `'a'` (97), büyük `'A'`'dan (65) büyüktür. **Tüm büyük harfler tüm küçük harflerden küçüktür**, o yüzden `"kedi"`, `"Zebra"`'dan büyük çıkar. Büyük/küçük harfi yok sayan karşılaştırma istiyorsan iki tarafı da önce `tolower` ile küçültürsün (aşağıdaki algoritmalarda göreceksin).

Uzunluk, karakterlere erişim

`size()` ve `length()` aynı işi yapar: karakter sayısını (bayt olarak, aşağıda Türkçe başlığına bak) döndürür. `s[i]` `i`. karaktere ulaşır, indeks **0'dan** başlar; son karakter `s[s.size() - 1]`'dir. `front()` ilk, `back()` son karakterdir. `s[i]` yazılabilir de: karakteri yerinde değiştirir.

erisim.cpp

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    string s{"Merhaba"};
    cout << s.size() << " " << s.length() << '\n';
    cout << s[0] << s[3] << " " << s.at(6) << '\n';
    cout << s.front() << s.back() << '\n';
    s[0] = 'm';
    s.at(6) = 'A';
    cout << s << '\n';
    return 0;
}
```

Çıktı

```
7 7
Mh a
Ma
merhabA
```

`size()` bir `int` değil, **işaretsiz** bir tamsayı (`size_t`) döndürür; bu ayrıntının sonuçlarını döngü başlığında göreceksin.

`[]` ile `at()` aynı karaktere ulaşır, fark **sınır dışı** erişimde ortaya çıkar. `s[i]` indeksi denetlemez: `i` geçersizse **tanımsız davranış** oluşur (çöp karakter okunur ya da program çöker). `s.at(i)` ise indeksi denetler ve geçersizse `out_of_range` türünde bir **istisna** (exception) atar. İstisnaları Bölüm 14'te ayrıntılı göreceksin; şimdilik "program hata mesajıyla sonlanır" olarak oku:

sinirAt.cpp

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    string s{"abc"};
    cout << "basliyor\n";
    cout << s.at(10) << '\n';
    cout << "bitti\n";
    return 0;
}
```

Çıktı – program çöker (Aborted)

```
basliyor
terminate called after throwing an instance of 'std::out_of_range'
  what():  basic_string::at: __n (which is 10) >= this->size() (which is 3)
```

Yakalanmayan istisnanın mesajı (`terminate called after throwing ...`) g++'ya özgüdür. Visual Studio ile derlersen farklı bir hata penceresi çıkar. İki durumda da `bitti` yazılmaz.

Hangisini kullanmalı? Hız gerektirmeyen ödev ve sınav kodunda `at()`: hata sessizce geçmez, nerede olduğu belli olur. Kendi sınırını döngüyle zaten denetlediğin yerde `[]` yeterlidir.

> **Sınavda Çıkar**
>
> "`s[10]` ile `s.at(10)` farkı nedir?" Cevap: `[]` indeksi denetlemez (tanımsız davranış), `at()` denetler ve `out_of_range` istisnası atar. Sınavda "hangisi güvenlidir" diye sorulursa `at()`.

8.4 Arama: find ve string::npos

Bir metnin içinde başka bir metin ya da karakter aramak çok yapılan iştir. `find(aranan)` aranan şeyin **ilk bulunduğu indeksi** döndürür. Aranan karakter de olabilir, metin de. İkinci parametre aramanın başlayacağı indeksi verir. `rfind` sondan başa doğru arar.

Peki bulamazsa? `-1` döndürmez. Özel bir değer döndürür: **`string::npos`** (no position). Bu, `size_t` tipinin alabileceği **en büyük sayıdır** (`-1`'in işaretsiz hâli). Bulunup bulunmadığını **her zaman** `npos` ile karşılaştırarak denetlersin:

aramaFind.cpp

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    string s{"Merhaba Dunya, merhaba"};
    cout << s.find("haba") << '\n';
    cout << s.find('a') << " " << s.find('a', 5) << '\n';
    cout << s.rfind("haba") << '\n';

    size_t konum = s.find("xyz");
    if (konum == string::npos) {
        cout << "xyz bulunamadi\n";
    } else {
        cout << "xyz " << konum << ". indekste\n";
    }
    return 0;
}
```

Çıktı

```
3
4 6
18
xyz bulunamadi
```

`Merhaba` içinde `haba` 3. indekste başlar. `find('a')` ilk `a`'yı (4), `find('a', 5)` 5. indeksten sonrakini (6) bulur. `rfind("haba")` sondaki `merhaba` içindeki `haba`'yı (18) yakalar. Küçük `m` ile başlayan ikinci `merhaba` büyük `M`'den farklı bir harf olduğu için `find("Merhaba")` onu görmez: arama büyük/küçük harfe duyarlıdır.

Bir metinde bir şeyin **kaç kez** geçtiğini bulmak için aramayı, her bulunan konumdan bir sonrasından yeniden başlatırsın:

```
size_t konum = metin.find("ana");
while (konum != string::npos) {
    say++;
    konum = metin.find("ana", konum + 1);
}
```

Sık Yapılan Hata: find sonucunu if'e doğrudan vermek

`if (s.find("ab"))` yazmak çok yaygın bir hatadır. Bulamazsa `npos` döner ve bu sıfırdan farklı olduğu için koşul **doğru** çıkar; `"ab"` metnin en başındaysa indeks 0 döner ve koşul **yanlış** çıkar. Yani tam ters çalışır. Doğrusu: `if (s.find("ab") != string::npos)`. Sonucu `int` değişkene atamak da kötü alışkanlıktır; `size_t` kullan.

Sınavda Çıkar

"`find` aranan metni bulamazsa ne döndürür?" Cevap: `string::npos`. `-1`, `0` ya da `false` yanlıştır. `npos`'u ekrana yazdırırsan `-1` değil, çok büyük bir sayı çıkar.

8.5 Parçalama ve değiştirme

`substr(başlangıç, uzunluk)` metnin bir parçasını **yeni bir string** olarak döndürür; asıl metin değişmez. Uzunluğu yazmazsan başlangıçtan sona kadar alır. Başlangıç metnin uzunluğundan büyükse `out_of_range` istisnası atar.

Metnin kendisini değiştiren fonksiyonlar:

Fonksiyon	Ne yapar?
`insert(i, metin)`	`i`. indeksin önüne `metin` ekler
`erase(i, n)`	`i`. indeksten başlayarak `n` karakter siler; `erase(i)` sona kadar siler
`replace(i, n, metin)`	`i`'den başlayan `n` karakteri `metin` ile değiştirir
`push_back(c)` / `pop_back()`	Sona bir karakter ekler / son karakteri siler
`append(metin)`	Sona ekler (`+=` ile aynı)
`clear()`	Metni boşaltır

degistirme.cpp

```
#include <iostream>
#include <string>

using namespace std;

int main() {
    string s{"Merhaba Dunya"};
    cout << s.substr(8, 3) << " " << s.substr(8) << '\n';

    s.insert(7, " guzel");
    cout << s << '\n';
    s.erase(0, 8);
    cout << s << '\n';
    s.replace(0, 5, "iyi");
```

```
    cout << s << '\n';
    s.push_back('!');
    cout << s << " " << s.size() << '\n';
    s.pop_back();
    s.erase(3);
    cout << "[" << s << "]\n";
    s.clear();
    cout << s.size() << '\n';
    return 0;
}
```

Çıktı

```
Dun Dunya
Merhaba guzel Dunya
guzel Dunya
iyi Dunya
iyi Dunya! 10
[iyi]
0
```

`insert(7, " guzel")` 7. indeksin önüne ekler (`Merhaba` 7 karakterdir). `erase(0, 8)` baştaki `Merhaba` kısmını (boşluk dahil 8 karakter) siler. `replace(0, 5, "iyi")` ilk 5 karakteri (`guzel`) 3 karakterlik `iyi` ile değiştirir; uzunluk değişse de sorun çıkmaz. `erase(3)` tek parametreyle çağrılınca 3. indeksten **sona kadar** siler ve geriye `iyi` kalır. Bu son ayrıntı sık karıştırılır: `erase(3)` "3. karakteri sil" demek **değildir**.

Bu fonksiyonlar çoğu sınavda iç içe sorulur. Her adımda metnin güncel hâlini alt alta yazıp indeksleri tek tek say; zihinden yapma.

> **İpucu: += tercih et**
>
> Uzun bir metni adım adım kurarken `s += parca;` yaz; `parca` bir `string` ya da `char` ise `s = s + parca;` aynı sonucu verir ama her seferinde yeni bir geçici string oluşturur. Küçük programlarda fark etmez, ama doğru alışkanlıktır.

8.6 string ile döngü

String'in her karakterine tek tek bakmak çok yaygındır: harf say, büyüt, kontrol et. İki yolun var. **İndeksli `for`**: `s[i]` ile her karaktere ulaşır, karakteri değiştirebilirsin. **Aralık tabanlı `for`**: `for (char c : s)` (Bölüm 7'de `vector` ile gördüğün kalıp) karakterleri sırayla `c`'ye verir; yalnız okur. Karakterleri değiştirmeyeceksen en temiz yazım budur.

stringDongu.cpp

```
#include <iostream>
#include <string>
#include <cctype>

using namespace std;

int main() {
    string s{"Merhaba Dunya 2025"};

    int harf = 0;
    int rakam = 0;
    for (char c : s) {
        if (isalpha(c)) {
            harf++;
        } else if (isdigit(c)) {
            rakam++;
```

```
        }
    }
    cout << "harf: " << harf << ", rakam: " << rakam << '\n';

    for (size_t i = 0; i < s.size(); i++) {
        s[i] = toupper(s[i]);
    }
    cout << s << '\n';
    return 0;
}
```

Çıktı

```
harf: 12, rakam: 4
MERHABA DUNYA 2025
```

Birinci döngü sadece okuduğu için aralık tabanlı yazıldı. İkinci döngü karakterleri değiştirdiği için indeksli: `s[i] = toupper(s[i]);` ile her harfin büyüğünü geri yazıyor. `toupper` `int` döndürür ama `char`'a atanırken sorunsuz dönüşür. Rakam ve boşluklar `toupper`'dan değişmeden çıkar. (Aralık tabanlı döngüyle de değiştirmek mümkün; bunun için Bölüm 7'deki `for (int& x : a)` gibi `for (char& c : s)` yazman gerekir. `&` olmadan `c` yalnız bir kopyadır.)

İndeksli döngüde `i` değişkeninin tipi de önemlidir. `s.size()` işaretsiz bir tür (`size_t`) döndürdüğünden `for (int i = 0; i < s.size(); i++)` yazarsan g++ işaretli ile işaretsiz sayıyı karşılaştırdığın için uyarı verir. Bu yüzden indeks için **`size_t`** kullan; `size()`, `find()` ve `substr()` hep bu tipi kullanır.

İşaretsiz olmanın bir tuzağı daha var, **geriye doğru** döngüde:

```
for (size_t i = s.size() - 1; i >= 0; i--) { ... }   // hatalı
```

`size_t` hiçbir zaman negatif olmaz; `i` 0'dayken `i--` onu çok büyük bir sayıya taşır ve `i >= 0` her zaman doğru kalır: döngü bitmez, `s[i]` sınırın çok dışına çıkar. Güvenli yazım, `i`'yi "kaç karakter kaldı" gibi kullanıp içeride `i - 1` indeksine bakmaktır:

```
for (size_t i = s.size(); i > 0; i--) {
    cout << s[i - 1];
}
```

> **● Sık Yapılan Hata: Boş string ve size() - 1**
>
> `s.size() - 1` işaretsiz çıkarmadır. `s` boşsa sonuç `-1` değil, çok büyük bir sayıdır. Son karaktere `s[s.size() - 1]` ile ulaşmadan önce `s.empty()` denetle ya da doğrudan `s.back()` kullan (o da boş string'de geçersizdir, önce `empty()` bak). Aynı sebeple `s.size() - 3 > 0` gibi karşılaştırmalar kısa string'lerde yanlış çıkar; çıkarma yerine `s.size() > 3` yaz.

8.7 Satır okumak: getline

Bölüm 3'te `cin >> ad` ile okumanın boşlukta durduğunu, boşluklu bir satırı ise `getline(cin, degisken)` ile okuduğunu görmüştün. Burada yalnız `string` açısından hatırlatalım: `getline` Enter'a kadar olan her şeyi boşluklar dahil `string`'e koyar; sondaki `'\n'` karakterini okur ama string'e **koymaz**, yani `size()` içinde sayılmaz (C kitabındaki `fgets`'ten farkı: temizlemeye gerek yok).

Bölüm 3'teki tuzak `string` ile de aynen geçerlidir: `cin >> sayi` Enter'ın `'\n'`'ini tamponda bırakır, hemen ardından gelen `getline` bunu görüp kullanıcıya hiç sormadan boş string ile döner. Çare, ikisinin arasında bekleyen satır sonunu atmaktır:

```
cin >> yas;
cin.ignore(1000, '\n');   // Enter'ın '\n'ini at
getline(cin, ad);         // artık gerçekten kullanıcıyı bekler
```

Sık Yapılan Hata: cin >> sonrası getline

`cin >> sayi;` ardından `getline(cin, satir);` yazarsan `satir` boş gelir. Çare: ikisinin arasına `cin.ignore(1000, '\n');` koy. Sınavda "program neden kullanıcıdan isim sormadan devam ediyor?" sorusunun cevabı budur. Tersi sorun yoktur: `getline`'dan sonra `cin >>` kullanmak sorunsuzdur.

Dosya sonuna (Ctrl+D ya da Ctrl+Z) kadar satır okumak için `while (getline(cin, satir)) { ... }` yazılır; `getline` okuma başarısız olunca koşulu yanlış yapar. Üçüncü bir parametre verirsen satır sonu yerine o karakterde durur: `getline(cin, parca, ',')` virgüle kadar okur. Bunu `stringstream` başlığında kullanacaksın.

8.8 String ile sayı arasında dönüşüm

Sayıları metne, metni sayıya çevirmek çok gerekir: notu yazının içine koymak, kullanıcıdan gelen `"42"` ile hesap yapmak. `"42"` otomatik olarak 42'ye çevrilmez; bunun için hazır fonksiyonlar var:

Yön	Fonksiyon	Örnek
sayı → string	`to_string(sayi)`	`to_string(42)` → `"42"`
string → `int`	`stoi(s)`	`stoi("42")` → 42
string → `long long`	`stoll(s)`	`stoll("5000000000")`
string → `double`	`stod(s)`	`stod("2.5")` → 2.5

donusumTemel.cpp

```
#include <iostream>
#include <string>

using namespace std;

int main() {
    int not1{85};
    string mesaj = "Not: " + to_string(not1);
    cout << mesaj << '\n';
    cout << to_string(3.5) << '\n';

    cout << stoi("123") + 1 << '\n';
    cout << stoi("  42") << " " << stoi("-7") << '\n';
    cout << stod("2.5") * 2 << '\n';
    cout << stoi("12abc") << '\n';
    cout << stod("3,14") << '\n';
    return 0;
}
```

```
Çıktı
Not: 85
3.500000
124
42 -7
5
12
3
```

- `to_string(3.5)` `3.5` değil **`3.500000`** verir: ondalıklı sayılarda her zaman 6 basamak yazar. Biçimi ayarlamak istiyorsan `fixed` ve `setprecision` ile doğrudan `cout`'a yazdırırsın.
- `stoi` baştaki boşlukları atlar, işaretleri anlar.
- `stoi("12abc")` hata vermez: **baştaki sayıyı** alır (12) ve geri kalanı yok sayar.
- `stod("3,14")` Türkçe alışkanlığıyla yazılan virgülü anlamaz: `3`'te durur. Ondalık ayırıcı **nokta**dır.

Peki sayıyla hiç başlamayan bir metin verilirse? `stoi("abc")` bir değer üretemez, bu yüzden `0` döndürmez, istisna (`invalid_argument`) atar; `int`'e sığmayacak kadar büyük sayı için `out_of_range` atar. Yakalanmayan istisna programı sonlandırır. Hatalı girdiyi `try/catch` ile kontrollü karşılamayı Bölüm 14'te öğreneceksin. Metnin baştan sona rakam olduğundan emin olmak istiyorsan (`"7x"` gibi yarım sayıları reddetmek için) önce her karakteri `isdigit` ile denetlemen gerekir.

> **Sınavda Çıkar**
>
> "`stoi("42abc")` ne döndürür?" Cevap `42`: baştaki sayıyı alır, geri kalanını yok sayar. "`stoi("abc")` ne yapar?" Cevap: `invalid_argument` istisnası atar; `0` döndürmez. C'deki `atoi("abc")` ise 0 dönerdi; ikisini karıştırma.

8.9 Türkçe karakterler ve bayt uzunluğu

Bölüm 3'te Türkçe harflerin UTF-8'de 2 bayt tuttuğunu görmüştün. `string` karakterleri değil **baytları** sayar:

turkceBayt.cpp

```cpp
#include <iostream>
#include <string>
#include <cctype>

using namespace std;

int main() {
    string s{"ağaç"};
    cout << s.size() << '\n';

    int karakter = 0;
    int harf = 0;
    for (char c : s) {
        unsigned char b = static_cast<unsigned char>(c);
        if (b < 0x80 || b >= 0xC0) {
            karakter++;
        }
        if (isalpha(b)) {
            harf++;
        }
    }
    cout << "karakter: " << karakter << ", isalpha: " << harf << '\n';
    return 0;
}
```

```
Çıktı
6
karakter: 4, isalpha: 2
```

- `"ağaç"` 4 karakterdir ama `ğ` ve `ç` ikişer bayt olduğu için `size()` **6** verir.
- UTF-8'de çok baytlı bir karakterin ilk baytı `0xC0` ve üstündedir, devam baytları `0x80`-`0xBF` arasındadır. Döngü yalnız ilk baytları (ve ASCII karakterleri) sayarak gerçek karakter sayısını buldu: 4.
- `isalpha` yalnız iki `a`'yı harf saydı (2); `ğ` ile `ç`'nin baytlarını tanımadı.

Sonuç olarak bu bölümdeki araçlar **ASCII metin içindir**: `size()` Türkçe harf içeren metinde bayt sayısı verir; `s[i]` çok baytlı bir karakterin yarısına ulaşabilir; `toupper` `ç`'yi büyütemez, `i`'yi `İ` değil `I` yapar; `<` ve `>` Türkçe alfabe sırasını vermez; indeksle ters çevirirsen çok baytlı karakterlerin baytları ters sıraya girer ve çıktı bozulur. Sınavda ve ödevde string işlemlerinde Türkçe harf kullanmamaya çalış. Yalnız ekrana yazdırmak (`cout << "Ağaç"`) sorun değildir.

8.10 C tarzı karakter dizileri

C++ C'nin üzerine kurulduğu için C'nin karakter dizileri hâlâ geçerlidir ve eski kodlarda, kütüphanelerde karşına çıkar. Biçimini bil, ama kendi yazdığın kodda tercih etme. C'de bir metin, sonunda **null karakter** `'\0'` (kodu 0) bulunan bir `char` dizisiydi; `"Ali"` 3 karakter ama `'A'`, `'l'`, `'i'`, `'\0'` olarak **4 bayt** yer kaplar. (Kardeş C kitabında bu konu bir bölümün tamamıydı.)

cString.cpp

```cpp
#include <iostream>
#include <string>
#include <cstring>

using namespace std;

int main() {
    char a[] = "Ali";
    char b[] = {'A', 'l', 'i', '\0'};
    cout << a << " " << b << '\n';
    cout << sizeof a << " " << strlen(a) << '\n';

    a[0] = 'a';
    cout << a << '\n';

    string s = a;
    s += " Veli";
    cout << s << " " << s.size() << '\n';
    cout << sizeof('A') << " " << sizeof("A") << '\n';
    return 0;
}
```

```
Çıktı
Ali Ali
4 3
ali
ali Veli 8
1 2
```

- `sizeof a` 4 (3 karakter + `'\0'`), `strlen(a)` ise `'\0'` hariç 3 verir (`<cstring>`'den).
- `string s = a;` C dizisini `string`'e dönüştürür; sonra `string`'in rahatlığıyla kullanılır.

- `sizeof('A')` 1, `sizeof("A")` 2: `'A'` tek karakter, `"A"` ise `'A'` ile `'\0'` taşıyan iki baytlık dizidir. `char c = "A";` bu yüzden derlenmez.

Neden `string` tercih edilir? Çünkü C dizisinde:

	C dizisi (`char a[20]`)	`string`
Boyut	Sabit; aşarsan taşma (tanımsız davranış)	Kendiliğinden büyür
Kopyalama	`strcpy(hedef, kaynak)`	`hedef = kaynak;`
Birleştirme	`strcat` (hedef yeterince büyük olmalı)	`a + b`, `a += b`
Karşılaştırma	`strcmp(a, b) == 0` (`==` adres karşılaştırır)	`a == b`
Uzunluk	`strlen` her seferinde baştan sayar	`size()`
`'\0'` yönetimi	Senin sorumluluğun	Gizli

`strcpy` ve `strcat` taşmayı denetlemez; güvenlik açıklarının klasik kaynağıdır. Yeni kodda `string` kullan. Eski bir fonksiyon C tarzı metin istiyorsa `s.c_str()` ile `string`'ten bir C metni alabilirsin; bu çağrının döndürdüğü `const char*` tipini Bölüm 9'da işaretçilerle göreceksin.

stringstream ile kelimelere bölme (İleri)

`cin`'den okumak için `>>` ve `getline` kullandın. Aynı araçları **elindeki bir string üzerinde** de kullanabilirsen, bir cümleyi kelimelere ya da virgüllerle ayrılmış alanlara bölmek kolaylaşır. Bunu `<sstream>` içindeki `stringstream` sağlar: kaynağı klavye değil bir string olan bir **akış** (stream).

kelimeBol.cpp

```cpp
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

using namespace std;

int main() {
    string cumle{"Bugun  hava cok guzel"};
    stringstream ak{cumle};
    string kelime;
    vector<string> kelimeler;
    while (ak >> kelime) {
        kelimeler.push_back(kelime);
    }
    cout << kelimeler.size() << " kelime:";
    for (string k : kelimeler) {
        cout << " [" << k << "]";
    }
    cout << '\n';

    stringstream csv{"Ali,20,Ankara"};
    string parca;
    while (getline(csv, parca, ',')) {
        cout << parca << "|";
    }
    cout << '\n';
    return 0;
}
```

Çıktı

```
4 kelime: [Bugun] [hava] [cok] [guzel]
Ali|20|Ankara|
```

`ak >> kelime` boşlukları atlayıp bir sonraki kelimeyi alır; okuyacak bir şey kalmayınca koşul yanlış olur ve döngü biter. Çift boşluk sorun çıkarmadı: 4 kelime. `getline(csv, parca, ',')` ise virgüle kadar okur; son alan virgülle bitmediği halde yine okunur. Aynı `>>` döngüsü `int` ile de çalışır, böylece `"10 20 30"` gibi bir metinden sayıları ayıklayabilirsin.

8.11 Ters çevirme ve palindrom

Ters çevirmek için `s[i]` ile onun karşısındaki `s[n - 1 - i]` yer değiştirir; ortaya (`n / 2`) kadar gitmek yeter. **Palindrom**, tersiyle aynı okunan sözcük ya da cümledir (`kayak`). Büyük/küçük harf ve noktalama göz ardı edilecekse önce yalnız harf-rakamları küçük harfe çevirerek temiz bir kopya çıkarılır, sonra kopya tersiyle karşılaştırılır; `string`'te `==` olduğu için karşılaştırma tek satırdır.

tersPalindrom.cpp

```cpp
#include <iostream>
#include <string>
#include <cctype>

using namespace std;

string tersCevir(string s) {
    size_t n = s.size();
    for (size_t i = 0; i < n / 2; i++) {
        char gecici = s[i];
        s[i] = s[n - 1 - i];
        s[n - 1 - i] = gecici;
    }
    return s;
}

bool palindromMu(string s) {
    string temiz;
    for (char c : s) {
        if (isalnum(static_cast<unsigned char>(c))) {
            temiz += tolower(static_cast<unsigned char>(c));
        }
    }
    return temiz == tersCevir(temiz);
}

int main() {
    string s{"Merhaba"};
    cout << tersCevir(s) << " " << s << '\n';
    cout << (palindromMu("Radar") ? "evet" : "hayir") << '\n';
    cout << (palindromMu("Ali") ? "evet" : "hayir") << '\n';
    cout << (palindromMu("A man, a plan, a canal: Panama") ? "evet" : "hayir")
         << '\n';
    return 0;
}
```

Çıktı

```
abahreM Merhaba
evet
hayir
evet
```

`tersCevir` parametreyi değer ile aldığı için kendi kopyasını çevirir ve sonucu döndürür; `main`'deki `s` değişmedi (ikinci kelime hâlâ `Merhaba`). Tek uzunlukta ortadaki karakter kendi karşısındadır ve yerinde kalır; boş string'de döngü hiç dönmez. Kütüphanede hazır bir `reverse` de vardır (`<algorithm>`, iteratörlerle; Bölüm 15), ama sınavda döngüyü yazman beklenir.

İpucu: Sınavda string sorusu çözerken

Önce string'i kâğıda indeksleriyle çiz (`M=0, e=1, r=2 ...`). `find` ve `erase` gibi fonksiyonlarda hangi sayının **indeks**, hangisinin **uzunluk** olduğunu işaretle. `size()` ile son indeks arasında 1 fark olduğunu (`son = size() - 1`) unutma.

Bölüm Özeti – Bilmen Gerekenler

- `char`, ASCII koduyla tutulan 1 baytlık tamsayıdır: `'A'` = 65, `'a'` = 97, `'0'` = 48. `char + int` sonucu `int` olur; karakter için `static_cast<char>`. Rakam karakteri sayıya `c - '0'` ile çevrilir.
- `<cctype>`: `isalpha`, `isdigit`, `isspace`, `isupper`, `islower` (doğru için sıfırdan farklı sayı) ve `toupper`, `tolower` (`int` döndürür, `cout`'a verilirse sayı yazar).
- `string` (`<string>`) boş başlar, kendiliğinden büyür; `+`, `+=`, `==`, `<` çalışır. İki string sabiti toplanamaz (`"a" + "b"` derlenmez). Karşılaştırma ASCII sırasıyladır: büyük harfler küçüklerden önce gelir.
- `size()` / `length()` bayt sayısıdır ve işaretsizdir (`size_t`). Döngüde `size_t i` kullan; `s.size() - 1` boş string'de çok büyük sayı olur.
- `s[i]` sınırı denetlemez (tanımsız davranış), `s.at(i)` denetler ve `out_of_range` atar.
- `find` ilk indeksi döndürür, bulamazsa `string::npos` döndürür; `if (s.find(x))` yazma, `!= string::npos` ile karşılaştır. `substr(i, n)`, `insert(i, m)`, `erase(i, n)`, `replace(i, n, m)`; `erase(i)` `i`'den sona kadar siler.
- `getline(cin, s)` boşluklu satırı okur ve `'\n'`'i koymaz; `cin >> sayi` sonrasında önce `cin.ignore(1000, '\n')` gerekir.
- `to_string(sayi)` sayıyı string yapar (ondalıkta 6 basamak); `stoi`, `stod` string'i sayıya çevirir. `stoi("12abc")` = 12; `stoi("abc")` istisna atar (Bölüm 14'te yakalanır).
- UTF-8'de `string("ağaç").size()` = 6: `size()` bayt sayar; `cctype` fonksiyonları Türkçe harfleri tanımaz.
- C tarzı dizi `char s[] = "Ali"` 4 bayttır (`'\0'` dahil), `sizeof("A")` = 2. Yeni kodda `string` kullan.
- `stringstream` (`<sstream>`): `while (ak >> kelime)` ile kelimelere, `getline(ak, parca, ',')` ile virgüllere bölünür.
- Klasik algoritma: ters çevirme (`s[i]` ile `s[n-1-i]`) ve palindrom (temizlenmiş kopya tersiyle aynı mı).

Sorular

A) Çoktan Seçmeli

1 Aşağıdaki kod parçasının çıktısı nedir?

```
cout << toupper('b') << " " << static_cast<char>(toupper('b')) << '\n';
```

(A) `B B` (B) `b B` (C) `66 B` (D) `98 B` (E) `66 66`

2 `string s{"merhaba"};` tanımından sonra `s.find("z")` ifadesi ne döndürür?

(A) `0`
(B) `-1` (ekrana `-1` yazdırılır)
(C) `s.size()`
(D) `out_of_range` istisnası atar.
(E) `string::npos`

3 Aşağıdaki kod parçasının çıktısı nedir?

```
string s{"programlama"};
s.erase(0, 3);
s.replace(0, 4, "XY");
cout << s << " " << s.size();
```

(A) `XYlama 6` (B) `XYlama 8` (C) `XYmlama 7` (D) `XYama 5` (E) `XYlama 11`

4 UTF-8 kodlamasında çalışan bir g++ ile aşağıdaki kod parçası ne yazdırır?

```
string s{"çay"};
cout << s.size();
```

(A) `3`
(B) `2`
(C) `5`
(D) `4`
(E) Derleme hatası verir.

5 Aşağıdaki kod parçasının çıktısı nedir?

```
cout << stoi("42abc") + 1 << '\n';
```

(A) `42`
(B) `43`
(C) `42abc1`
(D) Program derlenmez.
(E) `stoi` istisna atar ve program sonlanır.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    string s{"banana"};
    cout << s.find("an") << " " << s.rfind("an") << " ";
    cout << s.find('n', 3) << '\n';
    s.erase(1, 2);
    s.replace(s.find('n'), 1, "ll");
    cout << s << " " << s.size() << '\n';
    cout << s.substr(1, 3) << '\n';
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
#include <string>
#include <cctype>

using namespace std;

int main() {
    string s{"a1b2c3"};
    int toplam = 0;
    string harfler;
    for (size_t i = 0; i < s.size(); i++) {
        if (isdigit(s[i])) {
            toplam += s[i] - '0';
        } else {
            harfler += toupper(s[i]);
        }
    }
    cout << toplam << " " << harfler << '\n';
    cout << s[0] + 1 << " " << static_cast<char>(s[0] + 1) << '\n';
    return 0;
}
```

8 Aşağıdaki program çalışırken kullanıcı önce `5`, sonra ikinci satıra `Ali` yazıyor. Program ekrana ne yazdırır? Köşeli parantezlerin içine ve sayılara dikkat et.

```
#include <iostream>
#include <string>

using namespace std;

int main() {
    int n{0};
    string ad;
    cin >> n;
    getline(cin, ad);
    cout << n << "[" << ad << "] " << ad.size() << '\n';
    getline(cin, ad);
    cout << ad << '\n';
    return 0;
}
```

C) Kod Yazma

9 Kullanıcıdan `getline` ile bir satır oku. Satırdaki harfleri, rakamları ve boşlukları sayıp aşağıdaki biçimde yazdıran `karakterSay` programını yaz. Harf ve rakam dışında kalan işaretler (`!`, `,` gibi) sayılmasın.

```
Bir satir yaz: Merhaba Dunya 2025!
Harf: 12
Rakam: 4
Bosluk: 2
```

10 Kullanıcıdan `getline` ile küçük/büyük harfleri karışık bir cümle oku. Her kelimenin **ilk harfini büyük, diğer harflerini küçük** yaparak yeni cümleyi yazdıran `baslikYap` programını yaz. Kelimeler boşlukla ayrılır; yalnız ASCII harf kullanılacak.

```
Cumle: merHABA dunyA nasilsiN
Merhaba Dunya Nasilsin
```

Çözümler

Cevap anahtarı

1	2	3	4	5
C	E	A	D	B

1 Cevap: C

`toupper` `int` döndürür. İlk ifade `cout`'a `int` olarak gittiği için `'B'`'nin kodu olan `66` yazılır. İkinci ifadede sonuç `static_cast<char>` ile karaktere çevrildiği için `B` yazılır. A her iki yerde de karakter bekler (`toupper` karakter döndürmez). B `toupper`'ı hiç çalıştırmamış gibi düşünür. D `'b'`'nin kodu olan 98'dir, oysa `'B'` = 66. E ikinci ifadedeki dönüştürmeyi görmez.

2 Cevap: E

`find` bulamayınca özel bir değer olan `string::npos` döndürür; bu, `size_t`'nin alabileceği en büyük sayıdır. A yanlış: 0 "başta bulundu" demektir. B yanlış: değer `-1`'in işaretsiz hâlidir; ekrana yazdırırsan `-1` değil çok büyük bir sayı çıkar. C yanlış: `size()` ile bir ilgisi yoktur. D yanlış: `find` istisna atmaz, bulamayınca `npos` döner (istisna atan `at()` ve `substr()`'dir).

3 Cevap: A

`"programlama"` üzerinde `erase(0, 3)` baştaki `pro`'yu siler: `gramlama` (8 karakter). `replace(0, 4, "XY")` baştaki 4 karakteri (`gram`) 2 karakterlik `XY` ile değiştirir: `XYlama`. Uzunluk 8 - 4 + 2 = 6. B uzunluğun değişmediğini sanar. C `erase`'i yanlış yerden yapar. D `replace`'te 5 karakter değiştirilmiş gibi hesaplar. E ilk uzunluğu (11) taşır.

4 Cevap: D

`ç` UTF-8'de 2 bayttır, `a` ve `y` birer bayt: toplam 4. `size()` karakter değil bayt sayar. A karakter sayısını (3) verir. B ve C hiçbir hesaba uymaz. E yanlış, kod normal derlenir.

5 Cevap: B

`stoi` baştaki sayıyı (`42`) alır ve kalan `abc` kısmını yok sayar; `42 + 1` = 43. A, 1 eklenmeyi unutmuştur. C, `stoi`'nin sayıyı metin olarak bıraktığını sanır; `stoi` `int` döndürür. D yanlış, kod derlenir. E istisna `stoi("abc")` gibi sayıyla **başlamayan** metinlerde atılır; burada metin rakamla başlıyor.

6

`banana` indeksleri: `b`=0, `a`=1, `n`=2, `a`=3, `n`=4, `a`=5.

1. `find("an")` = 1, `rfind("an")` = 3 (sondan ilk eşleşme), `find('n', 3)` = 4 (3. indeksten sonra). İlk satır `1 3 4`.
2. `erase(1, 2)` 1. indeksten 2 karakter (`an`) siler: `bana`.
3. `s.find('n')` şimdi `bana` içinde 2'dir. `replace(2, 1, "ll")` o tek karakteri `ll` ile değiştirir: `balla`, uzunluk 5.
4. `substr(1, 3)`: 1. indeksten 3 karakter: `all`.

Çıktı
```
1 3 4
balla 5
all
```

7

Döngü her rakamı değerine çevirip toplar: 1 + 2 + 3 = 6. Harfleri `toupper` ile büyütüp `harfler`'e ekler: `ABC` (`+=` bir `int`'i `char` olarak ekler). İkinci satırda `s[0] + 1` işlemi `int` üretir: `'a'` = 97, 97 + 1 = 98. `static_cast<char>` ile karaktere çevrilince `b` olur.

Çıktı
```
6 ABC
98 b
```

8

`cin >> n` yalnız `5`'i okur; Enter'ın `'\n'`'i tamponda kalır. İlk `getline` bu `'\n'`'i görüp hiçbir şey beklemeden boş string ile döner: `ad` boş, uzunluğu 0, yani `5[] 0`. `Ali` tamponda hâlâ beklemektedir; ikinci `getline` onu okur ve `Ali` yazılır. Düzeltmek için `cin >> n;` satırından sonra `cin.ignore(1000, '\n');` gerekirdi.

Çıktı
```
5[] 0
Ali
```

9

Satırı `getline` ile okuyoruz; `'\n'` string'e girmediği için ayrıca temizlemek gerekmez. Her karakter harf, rakam ya da boşluktur; hiçbirine uymayan (`!`) sayılmaz. `else if` zinciri tam bir sınıf seçer. Karakter `cctype` fonksiyonlarına verilirken Türkçe harf gelirse tanımsız davranışı önlemek için `unsigned char`'a çeviriyoruz.

karakterSay.cpp
```cpp
#include <iostream>
#include <string>
#include <cctype>

using namespace std;

int main() {
    string satir;
    int harf = 0;
    int rakam = 0;
    int bosluk = 0;
    cout << "Bir satir yaz: ";
    getline(cin, satir);
    for (char c : satir) {
        unsigned char b = static_cast<unsigned char>(c);
        if (isalpha(b)) {
            harf++;
        } else if (isdigit(b)) {
            rakam++;
        } else if (isspace(b)) {
            bosluk++;
        }
    }
    cout << "Harf: " << harf << '\n';
```

```
    cout << "Rakam: " << rakam << '\n';
    cout << "Bosluk: " << bosluk << '\n';
    return 0;
}
```

Çıktı

```
Bir satir yaz: Merhaba Dunya 2025!
Harf: 12
Rakam: 4
Bosluk: 2
```

10

Bir bayrak değişkeni (`yeniKelime`) "şu an bir kelimenin başındayım" bilgisini tutar. Başlangıçta doğrudur. Boşluğa gelince bayrak yeniden doğru olur ve boşluk olduğu gibi yazılır; harfe gelince bayrak doğruysa büyük, değilse küçük yazılır ve bayrak yanlış olur. Sonucu `string`'e `+=` ile ekleriz.

baslikYap.cpp

```cpp
#include <iostream>
#include <string>
#include <cctype>

using namespace std;

int main() {
    string cumle;
    string sonuc;
    bool yeniKelime{true};
    cout << "Cumle: ";
    getline(cin, cumle);
    for (char c : cumle) {
        if (isspace(c)) {
            yeniKelime = true;
            sonuc += c;
        } else if (yeniKelime) {
            sonuc += toupper(c);
            yeniKelime = false;
        } else {
            sonuc += tolower(c);
        }
    }
    cout << sonuc << '\n';
    return 0;
}
```

Çıktı

```
Cumle: merHABA dunyA nasilsiN
Merhaba Dunya Nasilsin
```

PROJE 1

Proje: Öğrenci Not Sistemi

İlk sekiz bölümde öğrendiklerini ilk kez **tek bir gerçek programda** birleştiriyoruz: `getline` ile satır okuma (Bölüm 3 ve 8), `isdigit` ve `stoi` ile girdi doğrulama (Bölüm 8), `switch` (Bölüm 4), döngüler (Bölüm 5), fonksiyonlar, `const &` ve referans ile geçirme (Bölüm 6), `vector` (Bölüm 7), `string` (Bölüm 8) ve `setw` ile `fixed` (Bölüm 3). Programı bir kerede vermeyeceğiz: **dört adımda** büyüteceğiz ve her adımın sonunda derlenip çalışan bir program olacak. Her adımı önce kendin yazmayı dene; takıldığın yerde açıklamalara bak.

Bu projede `struct`, `class` ve işaretçi **bilerek kullanılmadı**; onları Bölüm 9 ve 11'de öğrenip bu projeyi yeniden yazacaksın. Bunların yerini dört ayrı `vector` tutacak.

Amaç

Bir dersin hocasının işini kolaylaştıran bir konsol programı yazacağız. Hoca öğrencileri tek tek ekleyecek; program her öğrencinin ortalamasını ve harf notunu hesaplayacak, sınıfın durumunu özetleyecek ve bir menü üzerinden istenen bilgiyi gösterecek. Kullanıcı ne yazarsa yazsın program **çökmemeli**.

Gereksinimler

- Program bir **menü** gösterir ve kullanıcı `0` seçene kadar menüyü tekrarlar.
- Menüde `1` ile öğrenci eklenir: **ad soyad**, **vize** ve **final** sorulur. Notlar 0–100 arası tam sayıdır. Geçersiz girişte (harf, boş satır, aralık dışı sayı) program uyarır ve **aynı soruyu tekrar sorar**; boş bırakılan ad soyad için uyarı yazmadan yalnız tekrar sorar.
- Ortalama = vize × 0.4 + final × 0.6.
- Harf notu aşağıdaki tabloya göre verilir (ortalama, harfin alt sınırına eşit veya ondan büyükse o harf alınır):

Harf	AA	BA	BB	CB	CC	DC	DD	FD	FF
En düşük ortalama	90	85	80	75	70	65	60	50	0

- Diğer menü seçenekleri:
  - `2` → Bütün öğrencileri tablo halinde listele (no, ad, vize, final, ortalama, harf).
  - `3` → İstatistik: sınıf ortalaması, en yüksek ve en düşük ortalamalı öğrenci.
  - `4` → Harf notu dağılımı: her harf için o harfi alan öğrenci sayısı kadar `*`.
  - `5` → İsimle arama: aranan metin ismin herhangi bir yerinde geçebilir; büyük/küçük harf önemsizdir (`ibra` yazınca `Ibrahim` bulunmalı).
- Henüz öğrenci yokken 2–5 seçilirse program "önce öğrenci ekle" der.

Tasarım

Veriler: paralel diziler

Her öğrencinin dört bilgisi var: ad, vize, final, ortalama. Bunları dört ayrı `vector`'de tutacağız. Dört `vector`'ün **aynı indeksi aynı öğrenciye** aittir; bu yapıya **paralel diziler** (parallel arrays) denir:

```
indeks         0              1              2              3
adlar       Ayse Yilmaz    Mehmet Can     Zeynep Kaya    Ibrahim Demir
vizeler         70             50             95             60
finaller        85             58             88             40
ortalamalar     79.0           54.8           90.8           48.0
```

Bölüm 7'de gördüğün `vector`, `push_back` ile kendiliğinden büyür; bu yüzden diziler gibi önceden bir üst sınır (ör. 50 öğrenci) belirlemene gerek yok. Tek kural şudur: bir öğrenci eklerken dört `vector`'e **birlikte** `push_back` yapılır, böylece dördünün boyutu hep eşit kalır. Biri eksik kalırsa indeksler kayar ve bir öğrencinin notu başkasına yazılır. Bölüm 11'de bu dört `vector`'ü tek bir sınıfta toplayacaksın; o zaman bütün fonksiyonlar dört yerine tek parametre alacak.

Sabitler: harf tablosu

Harf tablosunu `if-else` zinciri yerine **iki paralel sabit dizi** ile tutuyoruz: `HARFLER` ve `ALT_SINIRLAR`. Bir ortalamanın harfini bulmak için `ALT_SINIRLAR` dizisinde baştan yürürüz; ortalamanın küçük olmadığı ilk sınırın indeksi harfin indeksidir. Bu diziler fonksiyonların dışında `const` olarak tanımlanır; Bölüm 6'da "global değişkeni yalnızca değişmeyen değerler için kullan" demiştik, bu tam öyle bir durumdur. Tablo değişirse yalnız bu iki satırı düzeltmek yeter.

Fonksiyonlar

Her işi tek bir fonksiyona veriyoruz; `main` yalnız `vector`'leri tanımlar ve menü döngüsünü çalıştırır. Veriyi yalnız okuyan fonksiyonlar `vector`'ü `const &` ile alır, veriyi değiştirenler (`ogrenciEkle`) `&` ile alır (Bölüm 6'da gördüğün referans ile geçirme):

Fonksiyon	Görevi	Dönüş tipi
`ortalamaHesapla(vize, fin)`	%40 vize + %60 final	`double`
`harfIndeksi(ort)`	Ortalamanın `HARFLER` dizisindeki indeksi	`int`
`satirOku(mesaj)`	Mesajı yazar, bir satır okur	`string`
`sayiOku(mesaj, alt, ust)`	`alt`–`ust` arası geçerli bir tam sayı girilene kadar sorar	`int`
`metinOku(mesaj)`	Boş olmayan bir satır okur	`string`
`ogrenciEkle(...)`	Bir öğrenciyi sorar, dört `vector`'e ekler	`void`
`listele(...)`	Öğrenci tablosunu yazar	`void`
`sinifOrtalamasi(ort)`	Ortalamalar `vector`'ünün ortalaması	`double`
`istatistik(adlar, ort)`	Sınıf ortalaması, en yüksek, en düşük	`void`
`histogram(ort)`	Harf notu dağılımını yıldızlarla çizer	`void`
`kucult(s)`	String'in küçük harfli kopyasını döndürür	`string`
`ara(adlar, ort)`	Aranacak metni sorar, isimde geçenleri yazar	`void`
`menuYazdir()`	Menüyü ekrana yazar	`void`

C'deki karşılığından büyük bir fark: C'de bir fonksiyon `string` döndüremezdi, C++'ta `string` döndüren fonksiyon (`satirOku`, `kucult`) sıradan bir fonksiyondur.

Adım adım geliştirme

Dosyayı `notSistemi.cpp` olarak kaydet; derle ve çalıştır. Windows'ta Türkçe harfler bozuk görünürse Bölüm 3'teki gibi önce `chcp 65001` yaz.

```
g++ -std=c++17 -Wall -Wextra notSistemi.cpp -o notSistemi
./notSistemi
```

Adım 1: Ortalama ve harf notu

Önce programın **beynini** yazıyoruz: iki sayıdan ortalama, ortalamadan harf. Girdi ve menü yok; yalnız birkaç sabit deneme değeri var. Sonucu ekranda görmeden menüye geçmenin anlamı yok: hesap yanlışsa üstüne ne eklersen yanlış olur.

notHesap.cpp

```cpp
#include <iostream>
#include <iomanip>
#include <string>

using namespace std;

const int HARF_SAYISI = 9;
const string HARFLER[HARF_SAYISI] = {
    "AA", "BA", "BB", "CB", "CC", "DC", "DD", "FD", "FF"
};
const int ALT_SINIRLAR[HARF_SAYISI] = {90, 85, 80, 75, 70, 65, 60, 50, 0};

// vize * 0.4 + final * 0.6; tamsayıyla hesaplanır, ondalık hatası olmasın
double ortalamaHesapla(int vize, int fin) {
    return (vize * 4 + fin * 6) / 10.0;
}

// Ortalamanın HARFLER dizisindeki indeksi
int harfIndeksi(double ort) {
    int i = 0;
    // i sınırı: ort negatif gelse bile dizinin dışına çıkma
    while (i < HARF_SAYISI - 1 && ort < ALT_SINIRLAR[i]) {
        i++;
    }
    return i;
}

int main() {
    int vizeler[] = {70, 6, 95, 10};
    int finaller[] = {85, 96, 88, 20};
    cout << fixed << setprecision(1);
    for (int i = 0; i < 4; i++) {
        double ort = ortalamaHesapla(vizeler[i], finaller[i]);
        cout << vizeler[i] << " ve " << finaller[i] << " -> " << ort << " "
             << HARFLER[harfIndeksi(ort)] << '\n';
    }
    return 0;
}
```

```
Çıktı
70 ve 85 -> 79.0 CB
6 ve 96 -> 60.0 DD
95 ve 88 -> 90.8 AA
10 ve 20 -> 16.0 FF
```

`HARFLER` ve `ALT_SINIRLAR` paralel çalışır: `harfIndeksi` ortalama `ALT_SINIRLAR[i]`'den küçük olduğu sürece `i`'yi artırır. 79.0 için 90, 85 ve 80'den küçüktür ama 75'ten küçük değildir; döngü `i = 3`'te durur ve `HARFLER[3]` yani `CB` olur (çıktının ilk satırı da böyle). Son sınır 0 olduğu için geçerli bir ortalamada döngü her zaman durur; yine de `i < HARF_SAYISI - 1` koşulunu ekledik. Böylece beklenmedik negatif bir ortalama gelse bile dizinin dışına çıkıp tanımsız davranışa (Bölüm 7) düşmeyiz, son harfte (`FF`) durulur.

Satır 2'deki 6 ve 96 değerlerini rastgele seçmedik; ortalama tam 60.0 olmalı ve harf **DD** çıkmalı. `ortalamaHesapla`'nın neden `0.4` ve `0.6` ile yazılmadığını şimdi göreceksin:

● Sık Yapılan Hata: Sınırda yanlış harf notu

Bölüm 2'de `0.1 + 0.2`'nin tam 0.3 olmadığını görmüştün; aynı şey burada da var. `0.4` ve `0.6` ikilik sistemde tam saklanamaz:

```cpp
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    int vize = 6;
    int fin = 96;
    double a = vize * 0.4 + fin * 0.6;
    double b = (vize * 4 + fin * 6) / 10.0;
    cout << boolalpha << fixed << setprecision(1);
    cout << "0.4 ve 0.6 ile: " << a << ", 60 ve üstü mü? " << (a >= 60) << '\n';
    cout << "tam sayıyla   : " << b << ", 60 ve üstü mü? " << (b >= 60) << '\n';
    return 0;
}
```

```
Çıktı
0.4 ve 0.6 ile: 60.0, 60 ve üstü mü? false
tam sayıyla   : 60.0, 60 ve üstü mü? true
```

Gerçek ortalama tam 60, yani **DD**. İlk formülün sonucu 60'ın çok az altında kalır; `setprecision(1)` yuvarlayarak yazdığı için ekranda yine `60.0` görünür, ama `>= 60` karşılaştırması yanlış çıkar. Öğrenci tabloda "60.0" yazmasına rağmen **FD** alırdı. İkinci formülde çarpma ve toplama tam sayılarla yapılır, en sonda tek bir bölme kalır; sonuç doğrudur. Bu, Bölüm 4'teki "`double` değerlerini `==` ile karşılaştırma" uyarısının gerçek hayattaki hâlidir.

Adım 2: Güvenli girdi okuma

Programın en çok emek isteyen kısmı budur. Bütün girdileri, sayı olsun ad olsun, `getline` ile **satır olarak** okuyoruz; `cin >> sayi` hiç kullanılmıyor. Bunun iki nedeni var. Birincisi, Bölüm 3'teki `cin >>` sonrası `getline` tuzağı (tamponda kalan `'\n'`'in sonraki `getline`'a boş satır olarak gitmesi) hiç oluşmaz. İkincisi, `cin >> sayi` ile `40` (harf O) yazılırsa `4` başarıyla okunur ve `0` tamponda kalır; hata bir sonraki soruda patlar (Bölüm 5). Satırı bütün olarak alıp **kendimiz** denetlersek bu sorun olmaz.

Sayı doğrulaması iki aşamalıdır. Önce satırın dolu, kısa ve tamamen rakamdan oluştuğundan emin oluruz (`isdigit`, Bölüm 8); ancak ondan sonra `stoi` ile sayıya çevirir iz:

Sık Yapılan Hata: stoi'ye ham girdi verme

`stoi` yarım sayıları kabul eder ve sayı olmayan metinde istisna atar (Bölüm 8):

```
#include <iostream>
#include <string>

using namespace std;

int main() {
    cout << stoi("40") << '\n';
    cout << stoi("70 ") << '\n';
    return 0;
}
```

Çıktı

```
4
70
```

`40` hata vermeden **4** oldu: kullanıcı 40 yazmak isteyip yanlışlıkla harf O'ya basmış, program sessizce 4 almış olurdu. `stoi("abc")` ise `0` değil, yakalanmazsa programı sonlandıran bir istisna (`invalid_argument`) atar; çok uzun bir rakam dizisi de `out_of_range` atar. Çare, `stoi`'yi **yalnız doğrulanmış** bir satıra uygulamaktır: dolu mu, en çok 4 karakter mi, hepsi rakam mı? 4 karakter sınırı `int`'e sığmayacak sayıları eler; 0–100 aralığı için fazlasıyla yeter. İstisnaları Bölüm 14'te `try`/`catch` ile yakalamayı öğreneceksin.

`satirOku` her çağrıda mesajı yazar ve bir satır okur. `getline` başarısız olursa girdi tümüyle bitmiştir (Ctrl+D, Windows'ta Ctrl+Z ya da dosyadan yönlendirmede dosya sonu; Bölüm 5'teki `cin.eof()` durumu). Tekrar soran bir döngü bu durumda sonsuza kadar dönerdi; bu yüzden programı `exit(1)` ile bitiririz. `exit`, `<cstdlib>` başlığındadır ve programı bulunduğu **herhangi bir fonksiyonun içinden** bitirir; şimdilik kalıp olarak al.

Aşağıda yalnız yeni eklenen başlıklar (`<cctype>`, `<cstdlib>`), yeni fonksiyonlar ve yeni `main` görünüyor; Adım 1'deki sabitler ve iki fonksiyon dosyada aynen duruyor. `main`, tek bir öğrenciyi sorup ortalamasını yazıyor:

```
#include <cctype>
#include <cstdlib>

// ...

// Mesajı yazar, bir satır okur; girdi tümüyle biterse programı kapatır
string satirOku(const string& mesaj) {
    string satir;
    cout << mesaj;
    if (!getline(cin, satir)) {
        cout << "\nGirdi bitti, program kapatılıyor.\n";
        exit(1);
    }
    return satir;
}

// alt–ust aralığında geçerli bir tam sayı girilene kadar tekrar sorar
int sayiOku(const string& mesaj, int alt, int ust) {
    while (true) {
        string satir = satirOku(mesaj);
        bool gecerli = !satir.empty() && satir.size() <= 4;
        for (char c : satir) {
            if (!isdigit(static_cast<unsigned char>(c))) {
                gecerli = false;
            }
        }
```

```
        if (gecerli) {                      // artık stoi güvenle çalışır
            int deger = stoi(satir);
            if (deger >= alt && deger <= ust) {
                return deger;
            }
        }
        cout << "  Geçersiz! " << alt << "–" << ust
             << " arası bir tam sayı gir.\n";
    }
}

// Boş bırakılamayan bir satır okur (ad soyad için)
string metinOku(const string& mesaj) {
    string metin;
    do {
        metin = satirOku(mesaj);
    } while (metin.empty());
    return metin;
}

int main() {
    string ad = metinOku("Ad soyad: ");
    int vize = sayiOku("Vize: ", 0, 100);
    int fin = sayiOku("Final: ", 0, 100);
    double ort = ortalamaHesapla(vize, fin);
    cout << fixed << setprecision(1);
    cout << ad << ": " << ort << " " << HARFLER[harfIndeksi(ort)] << '\n';
    return 0;
}
```

Deneme girdisinde ad önce boş bırakılıyor (sessizce tekrar sorulmalı). Vize için sırayla harf, aralık dışı sayı; final için harfli sayı (`40`), eksi işaretli sayı ve çok uzun bir sayı deniyor:

Çıktı

```
Ad soyad:
Ad soyad: Ayse Yilmaz
Vize: abc
  Geçersiz! 0–100 arası bir tam sayı gir.
Vize: 105
  Geçersiz! 0–100 arası bir tam sayı gir.
Vize: 70
Final: 40
  Geçersiz! 0–100 arası bir tam sayı gir.
Final: -5
  Geçersiz! 0–100 arası bir tam sayı gir.
Final: 99999
  Geçersiz! 0–100 arası bir tam sayı gir.
Final: 85
Ayse Yilmaz: 79.0 CB
```

Her red nedenini izle: `abc` rakam değil; `105` rakam ama 100'den büyük; `40` içinde harf var; `-5` içindeki `-` rakam değil (negatif not zaten geçersiz, ayrı bir kontrol gerekmedi); `99999` beş karakter olduğu için `stoi`'ye hiç ulaşmadı. `satirOku` ve `metinOku`'nun dönüş tipi `string`'tir: bir `string`'i değer olarak döndürmek yeni başlayanın aklına "kopyalanıyor, yavaş olmaz mı?" sorusunu getirir; derleyici bunu çoğu zaman kopyalamadan halleder, yani kaygılanma. Parametre olarak ise `mesaj` `const string&` ile alınır (Bölüm 6): `"Vize: "` gibi bir sabit metin kopyalanmadan okunur.

`sayiOku`'daki `while (true)` Bölüm 5'te gördüğün sonsuz döngüdür; çıkışı yalnız `return deger;` ile olur. Geçerli sayı bulunmadan döngüden düşülemediği için fonksiyonun sonunda ayrıca `return` yazmaya gerek yok.

Adım 3: Vector'lerde saklama, ekleme ve tablo

Şimdi öğrencileri saklıyoruz. `ogrenciEkle` önce **üç değeri de okur**, sonra dört `vector`'e `push_back` yapar; böylece okuma sırasında bir şey ters giderse dört `vector`'ün boyutu yine eşit kalır. Dört `vector` referansla alınır, çünkü fonksiyon `main`'deki `vector`'leri büyütmek zorundadır (Bölüm 6: kopya büyütülürse `main` bundan haberdar olmaz). `listele` ise yalnız okur, bu yüzden dört `vector`'ün hepsi `const &` alır (Bölüm 7).

Tablo, Bölüm 3'teki `setw` ile hizalanır: `left` ve `right` sütunun hangi yana yaslanacağını belirler, `setw` yalnız **bir sonraki** değerin genişliğini ayarlar, `fixed` ve `setprecision(1)` ise kalıcıdır. Sütun genişliklerinin toplamı (4 + 20 + 6 + 7 + 7 + 6) 50'dir; ayraç çizgisi de `string(50, '-')` ile 50 karakter çizilir (Bölüm 8'deki `string(n, karakter)` kurucusu).

Aşağıda yine yalnız yeni satırlar görünüyor: `vector` için eklenen `#include <vector>`, iki yeni fonksiyon ve yeni `main`. Önceki adımlardaki sabitler ve fonksiyonlar dosyada aynen duruyor.

```
#include <vector>

// ...

// Önce üç değeri okur, sonra dört vector'e birlikte ekler
void ogrenciEkle(vector<string>& adlar, vector<int>& vizeler,
                 vector<int>& finaller, vector<double>& ortalamalar) {
    cout << "--- " << adlar.size() + 1 << ". öğrenci ---\n";
    string ad = metinOku("Ad soyad: ");
    int vize = sayiOku("Vize: ", 0, 100);
    int fin = sayiOku("Final: ", 0, 100);
    adlar.push_back(ad);
    vizeler.push_back(vize);
    finaller.push_back(fin);
    ortalamalar.push_back(ortalamaHesapla(vize, fin));
}

void listele(const vector<string>& adlar, const vector<int>& vizeler,
             const vector<int>& finaller, const vector<double>& ortalamalar) {
    cout << left << setw(4) << "No" << setw(20) << "Ad Soyad" << right
         << setw(6) << "Vize" << setw(7) << "Final" << setw(7) << "Ort."
         << setw(6) << "Harf" << '\n';
    cout << string(50, '-') << '\n';
    cout << fixed << setprecision(1);
    for (size_t i = 0; i < adlar.size(); i++) {
        cout << left << setw(4) << i + 1 << setw(20) << adlar[i] << right
             << setw(6) << vizeler[i] << setw(7) << finaller[i]
             << setw(7) << ortalamalar[i]
             << setw(6) << HARFLER[harfIndeksi(ortalamalar[i])] << '\n';
    }
}

int main() {
    vector<string> adlar;
    vector<int> vizeler;
    vector<int> finaller;
    vector<double> ortalamalar;

    int n = sayiOku("Öğrenci sayısı: ", 1, 50);
    for (int i = 0; i < n; i++) {
        ogrenciEkle(adlar, vizeler, finaller, ortalamalar);
    }
    listele(adlar, vizeler, finaller, ortalamalar);
    return 0;
}
```

Bu denemede öğrenci sayısı ilk önce `0` ve `dört` olarak yanlış giriliyor:

```
Çıktı
Öğrenci sayısı: 0
  Geçersiz! 1-50 arası bir tam sayı gir.
Öğrenci sayısı: dört
  Geçersiz! 1-50 arası bir tam sayı gir.
Öğrenci sayısı: 3
--- 1. öğrenci ---
Ad soyad: Ayse Yilmaz
Vize: 70
Final: 85
--- 2. öğrenci ---
Ad soyad: Mehmet Can
Vize: 50
Final: 58
--- 3. öğrenci ---
Ad soyad: Ibrahim Demir
Vize: 60
Final: 40
No  Ad Soyad              Vize  Final   Ort.  Harf
-------------------------------------------------
1   Ayse Yilmaz             70     85   79.0    CB
2   Mehmet Can              50     58   54.8    FD
3   Ibrahim Demir           60     40   48.0    FF
```

Çıktıyı kontrol et: başlık ile veri satırlarının sütunları alt alta geliyor, çünkü her ikisinde de aynı `setw` değerleri kullanıldı. Ad sütunu sola, sayılar sağa yaslı. `adlar.size() + 1` ile "1. öğrenci" yazıyoruz: `size()` o ana kadar eklenenlerin sayısıdır, sıradaki öğrencinin numarası bir fazlasıdır. Mehmet'in ortalaması `50 * 0.4 + 58 * 0.6 = 54.8`, harfi **FD** (50'nin üstünde, 60'ın altında).

> **Sık Yapılan Hata: Bölüm 6'dan hatırla: kopyayı büyütmek**
>
> `ogrenciEkle`'nin parametrelerinden `&` işaretini silersen program yine derlenir ve çalışır; ama fonksiyon `main`'deki `vector`'lerin **kopyalarına** ekleme yapar. Fonksiyon biter, kopyalar yok olur ve `listele` boş bir tablo yazar. Derleyici hata vermez, çünkü yazdığın şey kurallara uygundur; yalnız yapmak istediğin şeyi yapmaz. Bir fonksiyon çağıranın değişkenini değiştirecekse parametresi `&` almalıdır. Tersine, yalnız okuyorsa `const &` yaz: büyük bir `vector`'ü her çağrıda kopyalamazsın ve yanlışlıkla değiştirmeni derleyici engeller.

Adım 4: Menü, istatistik, histogram ve arama

Son adımda `main` bir menü döngüsüne dönüşüyor ve geriye üç iş kalıyor: istatistik, harf dağılımı ve isimle arama. Programın tamamı aşağıda; Adım 1–3'ten gelen kısımlar aynen duruyor. Önce programa göz at, sonraki başlıklar önemli kısımları açıklıyor.

notSistemi.cpp

```cpp
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
#include <cctype>
#include <cstdlib>

using namespace std;

const int HARF_SAYISI = 9;
const string HARFLER[HARF_SAYISI] = {
    "AA", "BA", "BB", "CB", "CC", "DC", "DD", "FD", "FF"
};
const int ALT_SINIRLAR[HARF_SAYISI] = {90, 85, 80, 75, 70, 65, 60, 50, 0};
```

```cpp
// ---------- Hesaplama ----------

// vize * 0.4 + final * 0.6; tamsayıyla hesaplanır, ondalık hatası olmasın
double ortalamaHesapla(int vize, int fin) {
    return (vize * 4 + fin * 6) / 10.0;
}

// Ortalamanın HARFLER dizisindeki indeksi
int harfIndeksi(double ort) {
    int i = 0;
    while (i < HARF_SAYISI - 1 && ort < ALT_SINIRLAR[i]) {
        i++;
    }
    return i;
}

double sinifOrtalamasi(const vector<double>& ort) {
    double toplam = 0;
    for (double o : ort) {
        toplam += o;
    }
    return toplam / ort.size();
}

// s DEĞER ile geçer: fonksiyon kopyayı değiştirir, asıl string bozulmaz
string kucult(string s) {
    for (char& c : s) {
        c = static_cast<char>(tolower(static_cast<unsigned char>(c)));
    }
    return s;
}

// ---------- Girdi okuma ----------

// Mesajı yazar, bir satır okur; girdi tümüyle biterse programı kapatır
string satirOku(const string& mesaj) {
    string satir;
    cout << mesaj;
    if (!getline(cin, satir)) {
        cout << "\nGirdi bitti, program kapatılıyor.\n";
        exit(1);
    }
    return satir;
}

// alt-ust aralığında geçerli bir tam sayı girilene kadar tekrar sorar
int sayiOku(const string& mesaj, int alt, int ust) {
    while (true) {
        string satir = satirOku(mesaj);
        bool gecerli = !satir.empty() && satir.size() <= 4;
        for (char c : satir) {
            if (!isdigit(static_cast<unsigned char>(c))) {
                gecerli = false;
            }
        }
        if (gecerli) {
            int deger = stoi(satir);
            if (deger >= alt && deger <= ust) {
                return deger;
            }
        }
        cout << "  Geçersiz! " << alt << "-" << ust
             << " arası bir tam sayı gir.\n";
    }
}
```

```
// Boş bırakılamayan bir satır okur (ad soyad için)
string metinOku(const string& mesaj) {
    string metin;
    do {
        metin = satirOku(mesaj);
    } while (metin.empty());
    return metin;
}

// ---------- Menü işlemleri ----------

void menuYazdir() {
    cout << "\n1) Öğrenci ekle  2) Listele  3) İstatistik\n";
    cout << "4) Harf dağılımı  5) İsimle ara  0) Çıkış\n";
}

// Önce üç değeri oku, sonra dört vector'e birlikte ekler
void ogrenciEkle(vector<string>& adlar, vector<int>& vizeler,
                 vector<int>& finaller, vector<double>& ortalamalar) {
    cout << "--- " << adlar.size() + 1 << ". öğrenci ---\n";
    string ad = metinOku("Ad soyad: ");
    int vize = sayiOku("Vize: ", 0, 100);
    int fin = sayiOku("Final: ", 0, 100);
    adlar.push_back(ad);
    vizeler.push_back(vize);
    finaller.push_back(fin);
    ortalamalar.push_back(ortalamaHesapla(vize, fin));
}

void listele(const vector<string>& adlar, const vector<int>& vizeler,
             const vector<int>& finaller, const vector<double>& ortalamalar) {
    cout << left << setw(4) << "No" << setw(20) << "Ad Soyad" << right
         << setw(6) << "Vize" << setw(7) << "Final" << setw(7) << "Ort."
         << setw(6) << "Harf" << '\n';
    cout << string(50, '-') << '\n';
    cout << fixed << setprecision(1);
    for (size_t i = 0; i < adlar.size(); i++) {
        cout << left << setw(4) << i + 1 << setw(20) << adlar[i] << right
             << setw(6) << vizeler[i] << setw(7) << finaller[i]
             << setw(7) << ortalamalar[i]
             << setw(6) << HARFLER[harfIndeksi(ortalamalar[i])] << '\n';
    }
}

void istatistik(const vector<string>& adlar, const vector<double>& ort) {
    size_t enYuksek = 0;
    size_t enDusuk = 0;
    for (size_t i = 1; i < ort.size(); i++) {
        if (ort[i] > ort[enYuksek]) {
            enYuksek = i;
        }
        if (ort[i] < ort[enDusuk]) {
            enDusuk = i;
        }
    }
    cout << fixed << setprecision(2);
    cout << "Sınıf ortalaması: " << sinifOrtalamasi(ort) << '\n';
    cout << setprecision(1);
    cout << "En yüksek       : " << adlar[enYuksek] << " (" << ort[enYuksek]
         << ")\n";
    cout << "En düşük        : " << adlar[enDusuk] << " (" << ort[enDusuk]
         << ")\n";
}
```

```
void histogram(const vector<double>& ort) {
    int sayac[HARF_SAYISI] = {0};
    for (double o : ort) {
        sayac[harfIndeksi(o)]++;
    }
    cout << "Harf notu dağılımı:\n";
    for (int i = 0; i < HARF_SAYISI; i++) {
        cout << "  " << HARFLER[i] << " | " << string(sayac[i], '*') << '\n';
    }
}

// Büyük/küçük harf duyarsız, isim parçasıyla arama
void ara(const vector<string>& adlar, const vector<double>& ort) {
    string girilen = metinOku("Aranacak isim: ");
    string aranan = kucult(girilen);
    bool bulundu = false;
    cout << fixed << setprecision(1);
    for (size_t i = 0; i < adlar.size(); i++) {
        if (kucult(adlar[i]).find(aranan) != string::npos) {
            cout << "  " << adlar[i] << " -> ortalama " << ort[i] << " ("
                 << HARFLER[harfIndeksi(ort[i])] << ")\n";
            bulundu = true;
        }
    }
    if (!bulundu) {
        cout << "  \"" << girilen << "\" bulunamadı.\n";
    }
}

int main() {
    vector<string> adlar;
    vector<int> vizeler;
    vector<int> finaller;
    vector<double> ortalamalar;

    cout << "===== ÖĞRENCİ NOT SİSTEMİ =====\n";
    int secim = -1;
    do {
        menuYazdir();
        secim = sayiOku("Seçimin: ", 0, 5);
        if (secim >= 2 && adlar.empty()) {
            cout << "  Önce en az bir öğrenci ekle (seçenek 1).\n";
            continue;
        }
        switch (secim) {
            case 1:
                ogrenciEkle(adlar, vizeler, finaller, ortalamalar);
                break;
            case 2:
                listele(adlar, vizeler, finaller, ortalamalar);
                break;
            case 3:
                istatistik(adlar, ortalamalar);
                break;
            case 4:
                histogram(ortalamalar);
                break;
            case 5:
                ara(adlar, ortalamalar);
                break;
            case 0:
                cout << "Program kapatıldı. İyi çalışmalar!\n";
                break;
        }
```

```
    } while (secim != 0);

    return 0;
}
```

Menü döngüsü

Menü bir `do-while` döngüsüdür (Bölüm 5): menü **en az bir kere** gösterilmeli, kullanıcı `0` seçene kadar tekrarlanmalıdır. Seçim yine `sayiOku` ile 0–5 aralığında okunduğu için `switch`'e (Bölüm 4) geçersiz bir değer hiç gelmez; bu yüzden `default` dalına gerek kalmadı. Her `case`'in sonundaki `break`'i unutma; unutursan seçim 1 bütün sonraki seçenekleri de çalıştırır.

Henüz öğrenci yokken 2–5 seçilirse `istatistik` boş `vector`'ün `ort[0]` elemanına dokunurdu; Bölüm 7'den biliyorsun, bu tanımsız davranıştır. Bunu her fonksiyona ayrı ayrı kontrol yazmak yerine **tek yerde**, `switch`'ten önce yakalıyoruz: `adlar.empty()` doğruysa uyarı yazılır ve `continue` ile döngünün sonuna, yani `secim != 0` koşuluna atlanır (Bölüm 5). Bu durumda `secim` en az 2 olduğundan döngü sürer. Fonksiyonların hepsi "`vector` boş değildir" varsayımıyla yazılabilir; varsayımı sağlamak `main`'in işidir.

İstatistik ve histogram

- `istatistik`, en yüksek ve en düşük ortalamanın **değerini değil indeksini** tutar (Bölüm 7). İndeksi bilirsek o öğrencinin adına da `adlar[enYuksek]` ile ulaşırız. Arama `i = 1`'den başlar, çünkü ilk eleman başlangıçta hem en yüksek hem en düşük sayılır. İndeksler `size_t` tutulur; `int` kullansaydık `ort.size()` ile karşılaştırmada Bölüm 7'deki işaretli/işaretsiz uyarısını alırdık. Sınıf ortalaması iki, öğrenci ortalamaları bir basamakla yazılır: `setprecision` kalıcı olduğundan arada değiştiriyoruz.
- `sinifOrtalamasi` yalnız okuyan bir fonksiyon olduğu için `vector`'ü `const &` alır ve aralık tabanlı `for` ile toplar (Bölüm 7). `double` ile `size_t` bölünürken `size_t` otomatik `double`'a çevrilir (Bölüm 2).
- `histogram`, Bölüm 8'deki harf frekansı fikrinin aynısıdır: `HARFLER` ile aynı boyutta bir sayaç dizisi tanımlayıp her öğrenci için `sayac[harfIndeksi(o)]++` yapar. Yıldızları `string(sayac[i], '*')` ile çizeriz: Bölüm 8'de gördüğün "karakteri n kez tekrarla" kurucusu, sayaç sıfırsa boş string verir.

> **İpucu: Tek komutla deneme**
>
> Menüyü denemek için öğrencileri her seferinde elle yazmak sıkıcıdır. Girdileri satır satır bir metin dosyasına (`girdi.txt`) yaz ve programa dosyadan ver. `getline` zaten satır okuduğu için hiçbir şeyi değiştirmen gerekmez; dosya bitince `satirOku`'daki `exit(1)` devreye girer ve program sonsuz döngüye girmeden kapanır:
>
> ```
> ./notSistemi < girdi.txt                    (Linux, macOS, MSYS2 terminali)
> .\notSistemi.exe < girdi.txt                (Windows cmd)
> Get-Content girdi.txt | .\notSistemi.exe    (Windows PowerShell)
> ```
>
> Hata ayıklarken önce tek öğrenciyle başla; bir fonksiyon bozuksa tek öğrenciyle de bozuktur.

Büyük/küçük harf duyarsız arama

`ara`, hem aranan metnin hem her ismin **küçültülmüş kopyasını** karşılaştırır: `kucult(adlar[i]).find(aranan)` (Bölüm 8'deki `find`). `kucult`, parametresini `const string&` değil **değer ile** alır (Bölüm 6) ve bunu bilerek yapar: fonksiyonun içinde değiştirdiği şey çağıranın string'inin kopyasıdır. Böylece tabloda `Ayse Yilmaz` büyük harfle yazıldığı gibi görünmeye devam eder, yalnız karşılaştırma küçük harfle yapılır. `find` bulamazsa `string::npos` döndürür; `!= string::npos` "isimde geçiyor" demektir. `for (char& c : s)` içindeki `&`, Bölüm 7'de gördüğün gibi harfin kopyasını değil kendisini değiştirir.

> **Sık Yapılan Hata: `tolower` Türkçe harfleri çeviremez**

`tolower` **tek bir bayt** üzerinde çalışır ve yalnız ASCII harfleri (`A–Z`) küçültür. UTF-8'de `İ`, `Ş`, `Ü` gibi Türkçe harfler **iki bayttır** (Bölüm 8) ve ikisi de ASCII aralığının dışındadır; `tolower` onlara dokunmaz. Sonuç olarak `İ` küçülmez, `ibra` araması `İbrahim`'i bulamaz:

turkceKucult.cpp

```cpp
#include <iostream>
#include <string>
#include <cctype>

using namespace std;

string kucult(string s) {
    for (char& c : s) {
        c = static_cast<char>(tolower(static_cast<unsigned char>(c)));
    }
    return s;
}

int main() {
    string ad = kucult("İbrahim");
    cout << "ad = " << ad << '\n';
    cout << "ibra -> "
         << (ad.find(kucult("ibra")) != string::npos ? "bulundu" : "bulunamadı")
         << '\n';
    cout << "İBRA -> "
         << (ad.find(kucult("İBRA")) != string::npos ? "bulundu" : "bulunamadı")
         << '\n';
    cout << "uzunluk = " << string("İ").size() << '\n';
    return 0;
}
```

Çıktı

```
ad = İbrahim
ibra -> bulunamadı
İBRA -> bulundu
uzunluk = 2
```

`İ` iki bayt olduğu için `size()` 2 verir ve `kucult` o iki baytı olduğu gibi bırakır. Bu yüzden `İBRA` bulundu (iki taraftaki `İ` de aynı kaldı) ama `ibra` bulunamadı. Aynı sorun `Ş` ile `ş`, `Ü` ile `ü` için de geçerlidir; üstelik Türkçede `I`'nın küçüğü `ı`'dır, oysa `tolower('I')` `i` verir. Doğru çözüm UTF-8'i bilen bir kütüphane ister; bu kitabın kapsamı dışındadır. **Bu yüzden örnek çalışmada adlar ASCII harflerle yazıldı.** Sınavda "`tolower` ile Türkçe harf küçültülür mü?" sorusunun cevabı hayırdır.

Not: Hizalama ve Türkçe harfler

`setw` genişliği karakter olarak sayar gibi görünür, ama `string`'in **bayt** uzunluğuna bakar. `Ayşe` 4 karakterdir ama 5 bayttır; tabloda bir boşluk eksik kalır ve sonraki sütunlar bir karakter sola kayar. Örnek verideki adların ASCII olması bu yüzdendir. Çözüm yolunu "Kendin Geliştir" bölümünde bulacaksın.

Sık Yapılan Hata: `cin >>` ile `getline`'ı karıştırma

Projeyi yazarken menü seçimini `cin >> secim` ile, adı `getline` ile okumak çok cazip gelir. Yapma: `cin >> secim` sayıyı alır ama Enter'ın `'\n'`'ini tamponda bırakır; ardından gelen `getline` hemen o `'\n'`'i görür ve ad olarak **boş satır** okur. Program ad sormadan vize sormaya geçmiş gibi görünür (Bölüm 3'ün klasik tuzağı). Bu projede tek kural şudur: **her şeyi `getline` ile satır olarak oku, sayıya sonradan `isdigit` ile denetleyip `stoi` ile çevir.** Satır sonu `'\n'`'i `getline` zaten string'e koymaz, ayrıca temizlemen gerekmez.

Örnek çalışma

Aşağıdaki çalışmada önce henüz öğrenci yokken `3` seçiliyor ve program uyarıyor. Sonra 6 öğrenci ekleniyor. Mehmet'ten sonra ikinci öğrencide ad yerine boş satır bırakılıyor; Zeynep'in vizesine önce `105`, sonra `abc` yazılıyor; Ibrahim'in finaline `40` yerine yanlışlıkla `40` (harf O) yazılıyor. Burak'ın notları `6` ve `96` olarak seçildi: ortalama tam 60.0 olmalı ve **DD** çıkmalı. Ardından menünün bütün seçenekleri sırayla deneniyor: iki farklı arama (`AY` ve `IBRA`), bulunamayan bir arama (`xyz`), geçersiz bir menü seçimi (`7`) ve çıkış.

Çıktı

```
===== ÖĞRENCİ NOT SİSTEMİ =====

1) Öğrenci ekle  2) Listele  3) İstatistik
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 3
  Önce en az bir öğrenci ekle (seçenek 1).

1) Öğrenci ekle  2) Listele  3) İstatistik
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 1
--- 1. öğrenci ---
Ad soyad: Ayse Yilmaz
Vize: 70
Final: 85

1) Öğrenci ekle  2) Listele  3) İstatistik
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 1
--- 2. öğrenci ---
Ad soyad: Mehmet Can
Vize: 50
Final: 58

1) Öğrenci ekle  2) Listele  3) İstatistik
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 1
--- 3. öğrenci ---
Ad soyad:
Ad soyad: Zeynep Kaya
Vize: 105
  Geçersiz! 0-100 arası bir tam sayı gir.
Vize: abc
  Geçersiz! 0-100 arası bir tam sayı gir.
Vize: 95
Final: 88

1) Öğrenci ekle  2) Listele  3) İstatistik
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 1
--- 4. öğrenci ---
Ad soyad: Ibrahim Demir
Vize: 60
Final: 4O
  Geçersiz! 0-100 arası bir tam sayı gir.
Final: 40

1) Öğrenci ekle  2) Listele  3) İstatistik
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 1
--- 5. öğrenci ---
Ad soyad: Burak Sen
Vize: 6
Final: 96
```

```
1) Öğrenci ekle  2) Listele  3) İstatistik
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 1
--- 6. öğrenci ---
Ad soyad: Elif Aydin
Vize: 82
Final: 76

1) Öğrenci ekle  2) Listele  3) İstatistik
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 2
No  Ad Soyad              Vize  Final   Ort.  Harf
-------------------------------------------------
1   Ayse Yilmaz             70     85   79.0    CB
2   Mehmet Can              50     58   54.8    FD
3   Zeynep Kaya             95     88   90.8    AA
4   Ibrahim Demir           60     40   48.0    FF
5   Burak Sen                6     96   60.0    DD
6   Elif Aydin              82     76   78.4    CB

1) Öğrenci ekle  2) Listele  3) İstatistik
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 3
Sınıf ortalaması: 68.50
En yüksek       : Zeynep Kaya (90.8)
En düşük        : Ibrahim Demir (48.0)

1) Öğrenci ekle  2) Listele  3) İstatistik
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 4
Harf notu dağılımı:
  AA | *
  BA |
  BB |
  CB | **
  CC |
  DC |
  DD | *
  FD | *
  FF | *

1) Öğrenci ekle  2) Listele  3) İstatistik
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 5
Aranacak isim: AY
  Ayse Yilmaz -> ortalama 79.0 (CB)
  Zeynep Kaya -> ortalama 90.8 (AA)
  Elif Aydin -> ortalama 78.4 (CB)

1) Öğrenci ekle  2) Listele  3) İstatistik
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 5
Aranacak isim: IBRA
  Ibrahim Demir -> ortalama 48.0 (FF)

1) Öğrenci ekle  2) Listele  3) İstatistik
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 5
Aranacak isim: xyz
  "xyz" bulunamadı.

1) Öğrenci ekle  2) Listele  3) İstatistik
4) Harf dağılımı  5) İsimle ara  0) Çıkış
```

```
Seçimin: 7
  Geçersiz! 0–5 arası bir tam sayı gir.
Seçimin: 0
Program kapatıldı. İyi çalışmalar!
```

Sonucu kontrol et: Burak'ın ortalaması 60.0 ve harfi **DD**; formülü `0.4/0.6` ile yazsaydık tabloda "60.0 FD" görecektik. `AY` araması büyük harfle yazılmasına rağmen **Ay**se Yilmaz'ı, Zeynep K**ay**a'yı ve Elif **Ay**din'i buldu; `IBRA` da `Ibrahim Demir`'i buldu. `xyz` için "bulunamadı" mesajı çıktı ve menü kapanmadı; `7` ise `switch`'e ulaşmadan `sayiOku` tarafından reddedildi.

- **Kendin Geliştir**
  - **Final barajı:** Finalden 50'nin altında alan öğrenci, ortalaması ne olursa olsun `FF` alsın. Hangi fonksiyonu değiştirmen gerektiğini düşün: `ortalamaHesapla` yalnız vize ve final alıyor, harfi ise `harfIndeksi` buluyor.
  - **Geçti/kaldı:** İstatistik ekranına geçen (DD ve üstü) ve kalan öğrenci sayısını ekle. `ALT_SINIRLAR[6]` değerini sabit olarak adlandırmak işini kolaylaştırır.
  - **Ortalama üstü:** Sınıf ortalamasının üstündeki öğrencileri listeleyen bir menü seçeneği ekle. `sinifOrtalamasi` hazır; `sayiOku`'nun üst sınırını `6`'ya çıkarmayı unutma.
  - **Sıralı liste:** Öğrencileri ortalamaya göre büyükten küçüğe sırala (Bölüm 7'deki sıralama algoritmaları). Sıralarken dört `vector`'ün elemanlarını **birlikte** yer değiştirmelisin; biri unutulursa notlar başka öğrencilere geçer. Bölüm 11'de dört `vector`'ü tek bir sınıf dizisinde toplayınca yer değiştirme tek bir atamaya iner.
  - **Not güncelleme:** İsmi verilen öğrencinin vize/final notunu değiştirip ortalamasını yeniden hesaplayan bir seçenek ekle. Tam isim karşılaştırması için `==` yeterli (Bölüm 4).
  - **Öğrenci silme:** Numarası verilen öğrenciyi sil. Dört `vector`'ün hepsinden aynı indeksteki elemanı çıkarman gerekir; elemanları tek tek sola kaydırarak başla, sonunda `pop_back` kullan.
  - **Boşluk temizleme:** `"70 "` gibi sonunda boşluk olan sayılar şu an reddediliyor. `satirOku` içinde baştaki ve sondaki boşlukları atan bir `kirp` fonksiyonu yaz (`find` ve `substr`, Bölüm 8).
  - **Hizalı Türkçe isimler:** UTF-8'de bir karakter, ilk iki biti `10` olmayan bayttan başlar (`(bayt & 0xC0) != 0x80`). Bir ismin karakter sayısını bu kuralla say, `setw` yerine kalan boşluğu `20 - karakterSayisi` kadar elle yazdır.
  - **Bütünleme:** `FD` veya `FF` alan öğrenciler için bütünleme notu iste; bütünleme notu final yerine geçsin.
  - **Dosyaya kaydetme:** Çıkarken bütün öğrencileri bir metin dosyasına yaz, açılışta oradan oku (Bölüm 16). Böylece hoca her seferinde notları baştan girmek zorunda kalmaz.

DENEME SINAVI

Deneme Vize Sınavı

Bu deneme sınavı kitabın ilk sekiz bölümünü kapsar: derleme süreci, değişkenler ve operatörler, `cin` / `cout` ve biçimli çıktı, koşullar, döngüler, fonksiyonlar, `vector` ve `string`. Soru türleri ve zorluk, bilgisayar mühendisliği 1. sınıf vize sınavlarına göre ayarlandı; tuzaklar **bilerek** kondu. Kolay görünen sorulara özellikle dikkat et: Hocalar tamsayı bölmesini, `++` / `--` sırasını, `cin >>` sonrası `getline`'ı, kısa devre değerlendirmeyi, `switch` düşmesini ve `vector<int> v{3, 7}` ile `vector<int> v(3, 7)` farkını sever.

Bölüm	Soru sayısı	Soru başına puan	Toplam
A) Çoktan seçmeli	10	4	40
B) Çıktı tahmini	4	6	24
C) Kod yazma	3	12	36
Toplam	**17**		**100**

Süre: 75 dakika. Yalnız kâğıt ve kalem kullanılır. Bütün sorularda standart C++17 ve `g++` geçerlidir; `int` 4 bayttır. Soru 1 hariç, kod parçalarında `#include` satırları ve `using namespace std;` var sayılır (kâğıtta yazılı olmasalar bile). Tanımsız davranış (undefined behavior) soran soru yoktur.

> **İpucu: Önce kâğıtta çöz**
>
> Sınavı gerçek sınav gibi çöz: 75 dakikalık bir sayaç kur ve süre bitene kadar VS Code'u açma, çözümlere bakma. Kabaca **20 dakikayı** testlere, **20 dakikayı** çıktı sorularına, **35 dakikayı** kod yazmaya ayır. Bir soruda takılırsan işaretle ve geç; sona dönersin. Süre bitince kodlarını bilgisayarda derleyip çalıştır, çözümlerle karşılaştır ve kendini puanla.

> **Not: Kendini nasıl puanlarsın?**
>
> - **Çıktı sorularında** çıktının her satırı eşit puan değerindedir. Eksik boşluk, fazladan satır veya `5` yerine `5.0` yazmak o satırı yanlış yapar; gerçek sınavda da böyledir.
> - **Kod sorularında** çözümdeki program tek doğru cevap değildir. Senin kodun aynı işi yapıyorsa tam puan al. Çözümlerin sonunda "hangi adım kaç puan" notu var. Derleme hatası olan her satır için 1 puan, yanlış sonuç veren her mantık hatası için 2–3 puan kır.
> - Her cevabın sonunda konunun anlatıldığı bölüm yazıyor. Yanlış yaptığın soruların bölümlerindeki **Sınavda Çıkar** ve **Sık Yapılan Hata** kutularını tekrar oku.

Sorular

A) Çoktan Seçmeli

1 Aşağıdaki program `g++ program.cpp -o program` komutuyla derlenirse ne olur?

```
using namespace std;

int main() {
    cout << "Merhaba\n";
    return 0;
}
```

(A) Ön işlemci (preprocessor) aşamasında, `#include` bulunamadığı için "dosya yok" hatası verir.
(B) Program derlenir; bağlayıcı (linker) `cout`'u bulamaz ve `undefined reference` hatası verir.
(C) Derleme hatası verir: derleyici `cout` adını tanımaz; çalıştırılabilir dosya oluşmaz.
(D) Program derlenir ve `Merhaba` yazdırır; `using namespace std;` satırı `cout`'u kendisi getirir.
(E) Program derlenir ve çalışır, ama ekrana hiçbir şey yazmaz.

2 Aşağıdaki kod parçası ekrana ne yazdırır?

```
int a{-7}, b{2};
cout << a / b << ' ' << a % b << ' ' << b % a << ' '
     << static_cast<double>(a) / b << '\n';
```

(A) `-4 1 2 -3.5` (B) `-3 -1 2 -3.5` (C) `-3 -1 -2 -3.5` (D) `-3 1 2 -3.5` (E) `-3 -1 2 -3`

3 Aşağıdaki kod parçası çalıştıktan sonra `i`, `j` ve `k` değişkenleri sırasıyla hangi değerleri taşır?

```
int i{3};
int j = i++ * 2;
int k = --i + j;
```

(A) `4 6 10` (B) `3 8 11` (C) `4 8 12` (D) `3 6 8` (E) `3 6 9`

4 Aşağıdaki kod parçası ekrana ne yazdırır?

```
double pi{3.14159};
cout << setprecision(3) << pi << ' ';
cout << fixed << pi << ' ';
cout << setprecision(1) << 9.99 << '\n';
```

(A) `3.14 3.142 10.0`
(B) `3.142 3.142 10.0`
(C) `3.14 3.14 10`
(D) `3.14 3.142 9.9`
(E) `3.14 3.142 9.99`

5 Aşağıdaki program çalışırken kullanıcı önce `7`, Enter, sonra `Ali Veli`, Enter yazıyor. Program ekrana ne yazdırır?

```
int n{0};
string ad;
cin >> n;
getline(cin, ad);
cout << n << '[' << ad << ']' << ad.size() << '\n';
```

(A) `7[Ali Veli]8` (B) `7[ Ali Veli]9` (C) `7[Ali]3` (D) `7[]0` (E) `0[]0`

6 Aşağıdaki program ekrana ne yazdırır?

```
int a{8}, b{3};
if (a > 5)
    if (b > 5)
        cout << "X";
else
    cout << "Y";
cout << "Z\n";
```

(A) `Z`
(B) `YZ`
(C) `XZ`
(D) `XYZ`
(E) Derleme hatası verir.

7 Aşağıdaki kod parçası çalıştıktan sonra ekrana ne yazdırılır?

```
int x{1}, y{1}, z{1};
bool r = ((x-- > 1) && (y++ > 0)) || (z++ > 0);
cout << r << ' ' << x << ' ' << y << ' ' << z << '\n';
```

(A) `0 0 1 2` (B) `1 0 2 2` (C) `1 0 1 1` (D) `1 0 1 2` (E) `1 1 2 2`

8 Aşağıdaki kod parçası bittiğinde `toplam` kaç olur?

```
int toplam{0};
for (int n = 1; n <= 4; n++) {
    switch (n) {
        case 1: toplam += 1;
        case 2: toplam += 2;
        case 3: toplam += 3; break;
        case 4: toplam += 4;
        default: toplam += 10;
    }
}
```

(A) `28` (B) `10` (C) `24` (D) `42` (E) `18`

9 Aşağıdaki kod parçası ekrana ne yazdırır?

```
vector<int> a(3, 7);
vector<int> b{3, 7};
cout << a.size() << ' ' << b.size() << ' ' << a[0] + b[0] << '\n';
```

(A) `2 3 10` (B) `3 3 14` (C) `3 2 14` (D) `2 2 10` (E) `3 2 10`

10 Aşağıdaki program ekrana ne yazdırır?

```
void f(int a, int& b) {
    a++;
    b++;
    b = b + a;
}

int main() {
    int x{1}, y{1};
    f(x, y);
    f(y, y);
    cout << x << ' ' << y << '\n';
    return 0;
}
```

(A) `1 4` (B) `2 10` (C) `1 10` (D) `1 5` (E) `1 9`

B) Çıktı Tahmini

11 Aşağıdaki program ekrana ne yazdırır? Boşluklara dikkat et.

```cpp
#include <iostream>

using namespace std;

int sayac() {
    static int c{0};
    c += 2;
    return c;
}

int kopya(int x) {
    x *= 3;
    return x;
}

void referans(int& x) {
    x *= 3;
}

int main() {
    int n{1};
    for (int i = 0; i < 3; i++) {
        int s = sayac();
        kopya(n);
        if (i % 2 == 0) {
            referans(n);
        } else {
            n += s;
        }
        cout << i << ": s=" << s << " n=" << n << '\n';
    }
    cout << sayac() << '\n';
    return 0;
}
```

12 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
#include <vector>

using namespace std;

int main() {
    vector<int> v;
    for (int i = 1; i <= 5; i++) {
        v.push_back(i * i);
    }
    v.pop_back();
    v.at(1) = 40;
    int toplam{0};
    for (int x : v) {
        x = x / 2;
        toplam += x;
    }
    cout << v.size() << ' ' << v.front() << ' ' << v.back() << ' '
         << toplam << '\n';

    vector<int> w = v;
    w.push_back(99);
    cout << v.size() << ' ' << w.size() << '\n';
    cout << (v == w) << '\n';
    return 0;
}
```

13 Aşağıdaki program ekrana ne yazdırır? `substr(konum, uzunluk)` ve `npos` kurallarını hatırla.

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    string s{"programlama"};
    size_t k = s.find("ram");
    string t = s.substr(k, 4);
    cout << k << ' ' << t << '\n';
    cout << s.find('a') << ' ' << s.find('a', 6) << ' ' << s.rfind('a')
         << '\n';
    if (s.find('x') == string::npos) {
        cout << "yok\n";
    }
    s.erase(0, 3);
    s.replace(0, 2, "XY");
    cout << s << ' ' << s.size() << '\n';
    return 0;
}
```

14 Aşağıdaki program ekrana ne yazdırır? İzleme tablosu çıkarmak işini kolaylaştırır.

```cpp
#include <iostream>

using namespace std;

int main() {
    int toplam{0};
    int k{10};
    do {
        k -= 3;
        if (k % 2 == 0) {
            continue;
        }
        toplam += k;
    } while (k > 0);
    cout << toplam << ' ' << k << '\n';

    for (int i = 1; i <= 3; i++) {
        for (int j = 1; j <= 3; j++) {
            if (j == i) {
                break;
            }
            cout << i << j << ',';
        }
    }
    cout << '\n';
    return 0;
}
```

C) Kod Yazma

15 Kullanıcıdan önce öğrenci sayısını `n`, sonra `n` tane tamsayı notu okuyan `notlarOzet` programını yaz. Notları bir `vector<int>` içinde tut. Program ortalamayı (**iki ondalık basamakla**), en yüksek notu, en düşük notu ve **ortalamanın üstünde** kalan öğrenci sayısını yazdırsın. `n` okunamazsa ya da `1`'den küçükse sadece `Geçersiz sayı!` yazıp bitsin. Ortalama hesabında tamsayı bölmesi tuzağına düşme. Örnek çalışmalar:

```
Kaç öğrenci: 5
Notları gir: 70 85 90 55 100
Ortalama: 80.00
En yüksek: 100
En düşük: 55
Ortalamanın üstünde: 3
```

```
Kaç öğrenci: 0
Geçersiz sayı!
```

16 İki fonksiyon yazıp `asalIstatistik` programında kullan:

- `bool asalMi(int sayi)`: sayı asalsa `true` döndürür (`0` ve `1` asal değildir).
- `void asalIstatistik(int n, int& adet, int& toplam)`: `2` ile `n` arasındaki (`n` dahil) asal sayıların **adedini** ve **toplamını** referans parametrelerle çağırana verir.

`main` kullanıcıdan `n` okusun. `n` 2'den küçükse `Asal sayı yok.` yazıp bitsin; değilse asalları tek satırda, ardından adedi ve toplamı yazdırsın. Örnek çalışma:

```
n: 20
Asallar: 2 3 5 7 11 13 17 19
Adet: 8
Toplam: 77
```

17 Kullanıcıdan `getline` ile bir e-posta adresi okuyan `epostaCozumle` programını yaz. Adres **geçerli** sayılması için şu üç koşulu sağlamalı: tam **bir tane** `@` karakteri olmalı ve bu karakter ilk karakter olmamalı; `@`'den sonraki kısımda (alan adı) en az bir `.` bulunmalı; alan adındaki **son** `.` ise ne ilk ne de son karakter olmalı. Geçersizse sadece `Geçersiz e-posta!` yazdır. Geçerliyse kullanıcı adını, alan adını ve **uzantıyı** (alan adındaki **son** noktadan sonraki kısım) yazdır. `find`, `rfind` ve `substr` kullan; `string::npos` kontrolünü unutma. Örnek çalışmalar:

```
E-posta: ogrenci@mail.universite.edu.tr
Kullanıcı: ogrenci
Alan adı: mail.universite.edu.tr
Uzantı: tr
```

```
E-posta: ogrenci@universite
Geçersiz e-posta!
```

Çözümler

Cevap anahtarı

1 C	2 B	3 E	4 A	5 D	6 B	7 D	8 A
9 E	10 C						

1 Cevap: C

Dosyada hiç `#include` yok. `using namespace std;` yalnızca `std` içindeki adları öne ekleme sorumluluğundan kurtarır; o adların **bildirimini getirmez**. Bildirimi `<iostream>` getirir. Derleyici `cout`'u tanımadığı için **derleme** aşamasında durur ve çalıştırılabilir dosya oluşmaz. Hata mesajının tam metni derleyiciye göre değişir (g++ aşağıdaki gibi yazar; Apple clang `use of undeclared identifier` der), ama aşama her yerde aynıdır.

Derleyici çıktısı – program derlenmez

```
test1.cpp: In function 'int main()':
test1.cpp:4:5: error: 'cout' was not declared in this scope
    4 |     cout << "Merhaba\n";
      |     ^~~~
test1.cpp:1:1: note: 'std::cout' is defined in header '<iostream>'; this is probably fixable by
adding '#include <iostream>'
  +++ |+#include <iostream>
    1 | using namespace std;
```

- A) Ön işlemci yalnız yazılmış bir `#include` dosyasını bulamazsa "dosya yok" der. Burada ortada hiç `#include` yok.
- B) Bağlayıcıya hiç sıra gelmez; derleyici daha önce durur. Bağlama hatası, adın bildirilmiş ama tanımı bulunamamış olması demektir.
- D) `using namespace std;` bir bildirim getirmez, sadece `std::` yazma zahmetini kaldırır.
- E) Derleme başarısız olduğu için çalışacak bir program yoktur.

Konu: Bölüm 1.

2 Cevap: B

Tamsayı bölmesi sıfıra doğru **kırpar** ve `%` işleminin işareti **bölünenin** işaretidir:

- `-7 / 2` → `-3.5` kırpılır → **-3**.
- `-7 % 2` → bölünen negatif → **-1** (`-7 == (-3) * 2 + (-1)`).
- `2 % -7` → `2 / -7` sıfıra kırpılır (0), kalan **2**'dir. Bölünen pozitiftir, bu yüzden sonuç pozitiftir.
- `static_cast<double>(a) / b` → önce `a` `double` olur, bölme ondalıklı yapılır: **-3.5**.

Çıktı

```
-3 -1 2 -3.5
```

- A) Aşağı yuvarlama (`-4`) C++'ta olmaz; bölme sıfıra doğru kırpar.
- C) `2 % -7` için `-2` yazmak, işareti bölenden almaktır; işaret bölünenden gelir.
- D) `-7 % 2` için `1` Python'daki "hep pozitif kalan" davranışıdır; C++'ta kalan `-1`'dir.
- E) `static_cast<double>(a)` bölmeden **önce** uygulandığı için sonuç ondalıklıdır; `-3` olması için bölme bitince cast gerekirdi.

Konu: Bölüm 2.

3 Cevap: E

`i++` ifadesi `i`'nin **eski** değerini (3) verir, sonra `i` 4 olur: `j = 3 * 2 = 6`. Sonra `--i` önce azaltır (`i` yeniden 3), **yeni** değeri verir: `k = 3 + 6 = 9`. Son durum: `i = 3`, `j = 6`, `k = 9`.

```
Çıktı
3 6 9
```

- A) `--i` satırının `i`'yi azalttığını unutmak (`i` 4'te kalmış, `k = 4 + 6`).
- B) `i++` yerine `++i` varmış gibi hesaplamak: `j = 4 * 2 = 8`, `k = 3 + 8 = 11`.
- C) Hem `++` hem `--` için ön ek/son ek farkını yok sayıp `i`'yi 4'te bırakmak: `j = 8`, `k = 4 + 8`.
- D) `--i` sonucunu 2 sanmak (`k = 2 + 6`). `i` 4 iken tek azaltma 3 yapar.

Konu: Bölüm 2.

4 Cevap: A

`fixed` yokken `setprecision(3)` **anlamlı basamak** sayısıdır: `3.14159` → `3.14`. `fixed` gelince aynı sayı **virgülden sonra 3 basamak** olarak yazılır: `3.142`. `fixed` kalıcıdır; `setprecision(1)` ile `9.99` virgülden sonra bir basamağa yuvarlanır: `10.0`.

```
Çıktı
3.14 3.142 10.0
```

- B) `fixed`'tan önceki ilk yazdırma anlamlı basamakla yapıldığı için `3.14` çıkar, `3.142` değil.
- C) `fixed` basamak sayısının anlamını değiştirir; ikinci değer `3.142` olur. Ayrıca `fixed` varken `10` değil `10.0` yazılır.
- D) Yazdırma **yuvarlar**, kesmez; `9.99` bir basamakta `10.0` olur.
- E) `setprecision(1)` etkisiz değildir; `fixed` açıkken virgülden sonra tam bir basamak gösterir.

Konu: Bölüm 3.

5 Cevap: D

`cin >> n` yalnızca `7`'yi okur; Enter'ın bıraktığı `'\n'` tamponda kalır. `getline` satır sonuna kadar okur ve ilk karakter olarak o `'\n'`'ı görür: **boş** bir satır okur, `ad` boş kalır (boyut 0). `Ali Veli` hiç okunmaz. Çözüm: `cin >> n` ile `getline` arasına `cin.ignore(...)` koymak (Bölüm 3).

```
Çıktı
7[]0
```

- A) ve B) `getline`'ın ikinci satırı okuduğunu sanmak. Kalan `'\n'` yüzünden okumaz.
- C) `getline`'ı `cin >> ad` gibi düşünüp ilk sözcüğü (`Ali`) okuduğunu sanmak. `getline` satırın tamamını okur, ama bu satır boştur.
- E) `7` başarıyla okunur, `n` 0 olmaz.

Konu: Bölüm 3.

6 Cevap: B

C++'ta `else` her zaman kendinden önceki **en yakın** eşleşmemiş `if`'e bağlanır; girinti derleyici için anlamsızdır (**sarkan else**, dangling else). Burada `else`, içteki `if (b > 5)`'e aittir. `a > 5` doğru, `b > 5` yanlış: içteki `if`'in `else`'i çalışır ve `Y` yazılır. `cout << "Z\n"` her durumda çalışır.

Çıktı
```
YZ
```

g++ `-Wall` ile bu kod için uyarı verir:

Derleyici uyarısı – g++ -Wall -Wextra
```
test6.cpp: In function 'int main()':
test6.cpp:5:8: warning: suggest explicit braces to avoid ambiguous 'else' [-Wdangling-else]
    5 |     if (a > 5)
      |        ^
```

- A) `else`'i dıştaki `if`'e ait sanıp dıştaki koşul doğru olduğu için hiçbir şey yazılmadığını varsaymak.
- C) `b > 5` yanlış olduğu için `X` yazılmaz.
- D) `X` ve `Y` aynı çalışmada birlikte yazılamaz.
- E) Uyarı hata değildir; program derlenir.

Konu: Bölüm 4. İlgili kutu: İç içe if ve sarkan else (Bölüm 4.6).

7 Cevap: D

`&&` sol tarafı `||`'dan önce işlenir. `x-- > 1` → `1 > 1` yanlıştır ve `x` 0 olur. `&&` sol taraf yanlış olduğu için sağ tarafı hiç değerlendirmez (**kısa devre**): `y++` çalışmaz, `y` 1 kalır. Parantezin sonucu yanlış olduğundan `||` sağ tarafı değerlendirmek zorundadır: `z++ > 0` → `1 > 0` doğru, `z` 2 olur. `r = 1`. Ekran: `1 0 1 2`.

Çıktı
```
1 0 1 2
```

- A) `||` sonucunun yanlış olduğunu sanmak. `z++ > 0` doğrudur.
- B) `y++`'ın çalıştığını varsaymak. `&&` kısa devre yaptı.
- C) `||`'un da kısa devre yaptığını sanmak. Sol taraf yanlış olduğu için `z++` çalışır.
- E) `x--`'in çalışmadığını ve `y++`'ın çalıştığını varsaymak. Tam tersi oldu.

Konu: Bölüm 4.

8 Cevap: A

Her turda `switch` ilgili `case`'ten başlar ve `break` görene kadar **aşağı doğru düşer** (fall-through):

n	Çalışan satırlar	Tur sonu `toplam`
1	`case 1`, `case 2`, `case 3` (break)	6
2	`case 2`, `case 3` (break)	11
3	`case 3` (break)	14
4	`case 4`, `default`	28

Çıktı
```
28
```

g++ düşmeyi şöyle uyarır (bilerek düşüyorsan `[[fallthrough]];` yaz):

Derleyici uyarısı – g++ -Wall -Wextra
```
test8.cpp: In function 'int main()':
test8.cpp:7:28: warning: this statement may fall through [-Wimplicit-fallthrough=]
    7 |             case 1: toplam += 1;
      |                     ~~~~~~~^~~~
test8.cpp:8:13: note: here
    8 |             case 2: toplam += 2;
      |             ^~~~
test8.cpp:8:28: warning: this statement may fall through [-Wimplicit-fallthrough=]
    8 |             case 2: toplam += 2;
      |                     ~~~~~~~^~~~
test8.cpp:9:13: note: here
    9 |             case 3: toplam += 3; break;
      |             ^~~~
test8.cpp:10:28: warning: this statement may fall through [-Wimplicit-fallthrough=]
   10 |             case 4: toplam += 4;
      |                     ~~~~~~~^~~~
test8.cpp:11:13: note: here
   11 |             default: toplam += 10;
      |             ^~~~~~~
```

- B) Her `case`'te `break` olsaydı çıkan sonuçtur (1 + 2 + 3 + 4).
- C) `n = 4` için `case 4`'ü atlayıp yalnız `default`'u çalıştırmak (6 + 5 + 3 + 10).
- D) `case 3`'te de `break` yokmuş gibi düşmek (6 + 5 + 17 + 14).
- E) `n = 4` için `default`'a düşmeyi unutmak (6 + 5 + 3 + 4).

Konu: Bölüm 4.

---

9 Cevap: E

Küme parantezi ve parantez `vector` için **farklı** işler yapar: `vector<int> a(3, 7)` "3 tane 7" demektir (`7 7 7`, boyut 3). `vector<int> b{3, 7}` ise elemanları `3` ve `7` olan, **boyutu 2** olan bir `vector`'dür. Bu yüzden `a[0] + b[0] = 7 + 3 = 10`.

Çıktı
```
3 2 10
```

- A) Parantez ve küme parantezinin rollerini ters çevirmek.
- B) `b{3, 7}`'yi "3 tane 7" okumak (boyut 3, `b[0] = 7`).
- C) Boyutu doğru bulup `b[0]`'ı 7 sanmak.
- D) `a(3, 7)`'yi iki elemanlı sanmak.

Konu: Bölüm 7.

---

10 Cevap: C

`f(x, y)`: `a` bir **kopyadır**, `b` ise `y`'nin **takma adıdır**. `a++` → kopya 2. `b++` → `y` 2. `b = b + a` → `y = 2 + 2 = 4`. `x` hiç değişmez: `x = 1`, `y = 4`.

`f(y, y)`: `a` yine kopyadır (`y`'nin o anki değeri 4, `a++` ile 5), `b` aynı anda `y`'nin kendisidir. `b++` → `y` 5. `b = b + a` → `y = 5 + 5 = 10`. Kopya, çağrıdan sonra `y`'yi etkilemez. Ekran: `1 10`.

```
Çıktı
1 10
```

- A) İkinci çağrının `y`'yi değiştirmediğini sanmak.
- B) `a`'nın referans olduğunu sanıp `x`'in de değiştiğini varsaymak.
- D) `b = b + a` satırının `y`'yi değiştirmediğini sanmak (`y` yalnız `b++` ile artar: 5).
- E) İkinci çağrıda `a++` etkisini atlayıp `5 + 4` hesaplamak.

Konu: Bölüm 6.

11

`sayac()` içindeki `static int c` yalnızca **bir kez** (ilk çağrıda) başlatılır ve değeri çağrılar arasında korunur: 2, 4, 6, 8, ... `kopya(n)` kopyayı üçle çarpıp döndürür, ama dönüş değeri kullanılmadığı için `n` değişmez (değer ile geçirme). `referans(n)` değişkenin kendisini üçle çarpar.

i	`s`	`i % 2`	Yapılan	`n`
(başta)				1
0	2	0	`referans(n)`: `n *= 3`	3
1	4	1	`n += s`	7
2	6	0	`referans(n)`: `n *= 3`	21

Döngüden sonra `sayac()` dördüncü kez çağrılır ve `c` 8 olur.

```
Çıktı
0: s=2 n=3
1: s=4 n=7
2: s=6 n=21
8
```

12

`v`: 1, 4, 9, 16, 25 → `pop_back()` ile son eleman gider: 1, 4, 9, 16 → `at(1) = 40` ile 1, 40, 9, 16.

Aralık tabanlı `for (int x : v)` her elemanın **kopyasını** verir; `x = x / 2` kopyayı değiştirir, `v` değişmez. Kopyalar yarılanır: 0, 20, 4, 8 → `toplam = 32`. İlk satır: boyut 4, ilk 1, son 16, toplam 32.

`vector<int> w = v` **bağımsız** bir kopya üretir. `w.push_back(99)` yalnız `w`'yi büyütür: `v` 4, `w` 5 elemanlı. İki `vector` `==` ile eleman eleman ve boyutla karşılaştırılır; boyutlar farklı olduğu için sonuç yanlıştır ve `cout` `bool`'u `0` olarak yazar.

```
Çıktı
4 1 16 32
4 5
0
```

13

`programlama` harfleri: `p0 r1 o2 g3 r4 a5 m6 l7 a8 m9 a10`.

1. `find("ram")` → `ram` 4. indekste başlar, `k = 4`. `substr(4, 4)` 4. indeksten itibaren 4 karakter alır: `raml`.
2. `find('a')` → 5. `find('a', 6)` 6. indeksten başlayarak arar → 8. `rfind('a')` sondan arar → 10.

3. `'x'` yok, `find` `string::npos` döndürür; koşul doğrudur ve `yok` yazılır.
4. `erase(0, 3)` baştan 3 karakteri siler: `gramlama`. `replace(0, 2, "XY")` ilk iki karakteri (`gr`) `XY` ile değiştirir: `XYamlama`, uzunluk 8.

```
Çıktı
4 raml
5 8 10
yok
XYamlama 8
```

14

`do-while` bölümü. `continue` döngü gövdesinin kalanını atlar, ama `do-while`'da bir sonraki adım **koşula** bakmaktır.

Tur	`k -= 3` sonrası	`k % 2 == 0`?	`toplam`	Koşul `k > 0`
1	7	hayır	7	doğru
2	4	evet, `continue`	7	doğru
3	1	hayır	8	doğru
4	-2	evet, `continue`	8	yanlış, çık

İlk satır: `8 -2`.

İç içe `for`. İçteki döngü `j == i` olunca `break` ile kendini bitirir:

- `i = 1`: `j = 1` hemen `break`, hiçbir şey yazılmaz.
- `i = 2`: `j = 1` için `21,` yazılır; `j = 2` `break`.
- `i = 3`: `j = 1` için `31,`, `j = 2` için `32,` yazılır; `j = 3` `break`.

```
Çıktı
8 -2
21,31,32,
```

15

Yaklaşım: Önce `n`'i oku ve `!(cin >> n) || n <= 0` ile hem okuma hatasını hem de geçersiz sayıyı tek koşulda ele al. Notları `push_back` ile `vector`'e ekle, aynı turda toplamı biriktir. Ortalama için **bölünenlerden birini `double` yap**: `static_cast<double>(toplam) / n`; yoksa tamsayı bölmesi ondalık kısmı atar. En yüksek ve en düşük için ilk elemanla başla. Ortalamanın üstünü saymak için ortalamanın **bütün notlar okunduktan sonra** bilinmesi gerekir; bu yüzden ikinci bir `for` kullandık. Biçim: `fixed` + `setprecision(2)` (`<iomanip>`).

notlarOzet.cpp

```cpp
#include <iostream>
#include <iomanip>
#include <vector>

using namespace std;

int main() {
    int n{0};
    cout << "Kaç öğrenci: ";
    if (!(cin >> n) || n <= 0) {
        cout << "Geçersiz sayı!\n";
```

```
        return 0;
    }
    vector<int> puanlar;
    int toplam{0};
    cout << "Notları gir: ";
    for (int i = 0; i < n; i++) {
        int puan{0};
        cin >> puan;
        puanlar.push_back(puan);
        toplam += puan;
    }
    double ortalama = static_cast<double>(toplam) / n;
    int enYuksek = puanlar[0];
    int enDusuk = puanlar[0];
    int ustunde{0};
    for (int p : puanlar) {
        if (p > enYuksek) {
            enYuksek = p;
        }
        if (p < enDusuk) {
            enDusuk = p;
        }
        if (p > ortalama) {
            ustunde++;
        }
    }
    cout << fixed << setprecision(2);
    cout << "Ortalama: " << ortalama << '\n';
    cout << "En yüksek: " << enYuksek << '\n';
    cout << "En düşük: " << enDusuk << '\n';
    cout << "Ortalamanın üstünde: " << ustunde << '\n';
    return 0;
}
```

Çıktı

```
Kaç öğrenci: 5
Notları gir: 70 85 90 55 100
Ortalama: 80.00
En yüksek: 100
En düşük: 55
Ortalamanın üstünde: 3
```

`puanlar[0]` güvenlidir: `n >= 1` olduğunu denetledik ve döngü en az bir not ekledi. Toplam `400`, ortalama `80`; `85`, `90` ve `100` ortalamanın üstünde olduğu için sayaç 3 olur.

Puanlama (12): `n` denetimi ve geçersiz mesajı 2; notları `vector`'e okuma 2; ortalama (`double` bölme) 3; en yüksek ve en düşük 2; ortalamanın üstü sayacı 2; `fixed` + `setprecision(2)` ile biçim 1.

16

Yaklaşım: `asalMi` `0` ve `1` için hemen `false` döndürür; sonra `2`'den `b * b <= sayi` olana kadar bölenleri dener. Bir bölen bulunca erken `return false` yeter; döngü bölen bulmadan biterse `true`. `asalIstatistik` iki sonucu birden verdiği için `adet` ve `toplam` **referans** parametredir. Fonksiyon içinde önce sıfırlanmaları önemlidir, çünkü çağıranın değişkenleri sıfırdan başlamamış olabilir.

asalIstatistik.cpp

```
#include <iostream>

using namespace std;
```

```cpp
bool asalMi(int sayi) {
    if (sayi < 2) {
        return false;
    }
    for (int b = 2; b * b <= sayi; b++) {
        if (sayi % b == 0) {
            return false;
        }
    }
    return true;
}

void asalIstatistik(int n, int& adet, int& toplam) {
    adet = 0;
    toplam = 0;
    for (int i = 2; i <= n; i++) {
        if (asalMi(i)) {
            adet++;
            toplam += i;
        }
    }
}

int main() {
    int n{0};
    cout << "n: ";
    cin >> n;
    if (n < 2) {
        cout << "Asal sayı yok.\n";
        return 0;
    }
    cout << "Asallar:";
    for (int i = 2; i <= n; i++) {
        if (asalMi(i)) {
            cout << ' ' << i;
        }
    }
    cout << '\n';
    int adet{0};
    int toplam{0};
    asalIstatistik(n, adet, toplam);
    cout << "Adet: " << adet << '\n';
    cout << "Toplam: " << toplam << '\n';
    return 0;
}
```

Çıktı

```
n: 20
Asallar: 2 3 5 7 11 13 17 19
Adet: 8
Toplam: 77
```

İki fonksiyon da `main`'den **önce** tanımlandığı için ayrıca prototip gerekmedi (Bölüm 6). `asalIstatistik` parametrelerini `int adet, int toplam` (değer ile geçirme) yazsaydın, fonksiyon yalnız kopyaları değiştirir ve `main`'de `Adet: 0` görürdün; `&` işareti bunu önler.

Puanlama (12): `asalMi` doğruluğu (`0`/`1`, bölen sınırı, sonuç) 4; `asalIstatistik` imzası, referans parametreler ve sıfırlama 3; `n < 2` durumu 1; `main` içinde asalları yazdırma 2; adet ve toplamı doğru çağırıp yazdırma 2.

17

Yaklaşım: Önce `@`'i `find` ile ara. Üç ret koşulu var: bulunamadı (`npos`), ilk karakter (`at == 0`) ya da birden fazla `@` (`find` ile `rfind` farklı yeri gösteriyorsa en az iki tane vardır). Sonra `substr(at + 1)` ile alan adını al (ikinci parametre yoksa sona kadar alır). Alan adında **son** noktayı `rfind` ile ara: `npos` ise nokta yoktur, `0` ise nokta ilk karakterdir, `size() - 1` ise son karakterdir. Uzantı `substr(nokta + 1)`'dir. Tek `getline` kullandığımız için `cin >>` ile karışık bir okuma yok; araya `ignore` koymak gerekmez.

epostaCozumle.cpp

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    string eposta;
    cout << "E-posta: ";
    getline(cin, eposta);
    size_t at = eposta.find('@');
    if (at == string::npos || at == 0 || at != eposta.rfind('@')) {
        cout << "Geçersiz e-posta!\n";
        return 0;
    }
    string alan = eposta.substr(at + 1);
    size_t nokta = alan.rfind('.');
    if (nokta == string::npos || nokta == 0 || nokta == alan.size() - 1) {
        cout << "Geçersiz e-posta!\n";
        return 0;
    }
    cout << "Kullanıcı: " << eposta.substr(0, at) << '\n';
    cout << "Alan adı: " << alan << '\n';
    cout << "Uzantı: " << alan.substr(nokta + 1) << '\n';
    return 0;
}
```

Çıktı

```
E-posta: ogrenci@mail.universite.edu.tr
Kullanıcı: ogrenci
Alan adı: mail.universite.edu.tr
Uzantı: tr
```

Tuzaklar: `find` sonucunu `int`'e atayıp `-1` ile karşılaştırmak (`npos` `size_t` tipinde, çok büyük bir sayıdır; `size_t` ile karşılaştır). Koşulların sırası da önemlidir: `nokta == string::npos` en başta durduğu için kısa devre sayesinde `alan.size() - 1` (işaretsiz çıkarma) yalnızca nokta bulunduğunda hesaplanır.

Puanlama (12): `getline` ile okuma 1; `@` koşulları (yok, ilk karakter, birden fazla) 3; alan adını `substr` ile ayırma 2; nokta koşulları (`npos`, ilk, son) 3; kullanıcı adı ve uzantıyı `substr` ile doğru kesme 2; çıktı biçimi 1.

BÖLÜM 9

İşaretçiler ve Referanslar

Bu Bölümde Öğreneceklerin

- Her değişkenin bellekte bir adresi olduğunu ve bu adresin nasıl öğrenildiğini
- İşaretçi (pointer) nedir, başka bir değişkene işaretçiyle nasıl ulaşılır ve değiştirilir
- "Hiçbir yeri göstermeyen" işaretçinin ne olduğunu ve neden kontrol edilmesi gerektiğini
- Bölüm 6'da tanıdığın referansın bellekte ne olduğunu ve işaretçiden farkını
- Fonksiyona işaretçiyle geçirmeyi ve bunun referansla geçirmeden farkını
- İşaretçiyle bir dizinin elemanlarına nasıl ulaşılır ve dizide nasıl ilerlenir
- `const` ile "değiştirme" sözünün işaretçide ve referansta nasıl verildiğini
- Yok olmuş bir değişkeni gösteren işaretçinin neden tehlikeli olduğunu

Bu bölüm, kitabın en çok puan kaybettiren bölümüdür. Üç şeyi "sonra anlatacağız" diye erteledik. İlki, Bölüm 3'te `cin >> yas` yazınca `yas`'ın nasıl değiştiğiydi: `cin` sana değerin bir kopyasını değil, değişkenin kendisini alıyordu. İkincisi, Bölüm 6'da referans parametreyle (`int &a`) fonksiyonun çağıranın değişkenini değiştirebildiğini gördün, ama bunun bellekte ne anlama geldiğini söylemedik. Üçüncüsü, Bölüm 7'de dizilerin fonksiyona kopyalanmadan nasıl geçtiğiydi. Üçünün cevabı da aynı: bir şeyin **kendisine** değil, **yerine** ulaşmak. Çoğu öğrenci işaretçileri zor bulur, çünkü bellekte olanı kafasında canlandırmaz. Bu yüzden her örnekte bir bellek çizimi var. Sen de sınavda aynısını yap: kutuları çiz, okları çiz, satır satır izle.

9.1 Bellek ve adres

Bilgisayarın belleği (RAM), sıra numaralı bayt hücrelerinden oluşan çok uzun bir sıradır. Bu numaraya **adres** (address) denir. Bölüm 2'de değişkeni "adı olan kutu" diye anlatmıştık; kutunun bir de numarası vardır. `int yas{25};` yazınca derleyici 4 baytlık yer ayırır, adına `yas` der ve 25'i oraya yazar. Değişkenin adresi, kapladığı baytların ilkinin numarasıdır (örneğin `yas` 1000-1003 numaralı baytlarda duruyorsa adresi 1000'dür).

Bir değişkenin adresini **adres operatörü** (address-of operator) `&` verir: `&yas`. `cout` ile yazdırabilirsin:

adresGoster.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int yas{25};
    double boy{1.75};

    cout << "yas : değer = " << yas << ", adres = " << &yas << '\n';
    cout << "boy : değer = " << boy << ", adres = " << &boy << '\n';
    return 0;
}
```

Çıktı

```
yas : değer = 25, adres = 0x16f6ce6fc
boy : değer = 1.75, adres = 0x16f6ce6f0
```

Adresler onaltılık (hexadecimal) yazılır ve `0x` ile başlar. **Senin bilgisayarında adresler farklı çıkar**, hatta aynı programı iki kez çalıştırsan bile değişebilir: yerleştirmeyi işletim sistemi yapar. Sınavda adresin sayısal değeri sorulmaz; önemli olan her değişkenin bir adresi olmasıdır.

> **Sık Yapılan Hata: `&` her zaman "adres" demek değildir**
>
> `&` iki ayrı iş yapar ve hangisi olduğunu **bulunduğu yer** belirler. İfadede (`&yas`) adres operatörüdür. Birazdan göreceğin gibi tanımda tipin yanında (`int& r`) "referans" demektir. Yazdırırken de bir tuzak var: `char harf{'A'};` için `cout << &harf;` yazarsan `cout`, `char` adresini bir C tarzı metin sanır ve `'\0'` bulana kadar bellekte yürür. Bu **tanımsız davranıştır** (undefined behavior): çöp harfler çıkar, program çökebilir. `char` adresini görmek istiyorsan `cout << static_cast<const void*>(&harf);` yaz: `void*` "tipi belirsiz bir adres" demektir; `cout` böyle bir adresi metin sanmaz, sayı olarak yazar. `int` ve `double` için böyle bir sorun yoktur.

9.2 İşaretçi tanımlama ve dolaylama

İşaretçi (pointer), değeri başka bir değişkenin **adresi** olan değişkendir. Tanımlarken tipin yanına `*` konur. Üç adımda kullanılır:

Adım	Kod	Anlamı
1. Tanımla	`int* p;`	`p`, bir `int`'in adresini tutabilir
2. Adres ver	`p = &x;`	`p`'nin içine `x`'in adresi yazılır
3. Kullan	`*p`	`p`'nin gösterdiği yerdeki değer

`*p` işlemine **dolaylama** (dereference) denir: "`p`'nin içindeki adrese git, orada duran değişkenin kendisini kullan". `*p` hem okunabilir hem yazılabilir; `x` ile aynı kutudur.

ilkIsaretci.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int x{10};
    int* p{&x};                    // p, x'in adresini tutar

    cout << "x  = " << x << '\n';
    cout << "*p = " << *p << '\n';  // p'nin gösterdiği yerdeki değer
    cout << "p == &x : " << (p == &x) << '\n';
    *p = 20;                       // x'in kendisi değişir
    cout << "x  = " << x << '\n';
    *p = *p + 5;
    cout << "x  = " << x << '\n';
    return 0;
}
```

Çıktı

```
x  = 10
*p = 10
p == &x : 1
x  = 20
x  = 25
```

Bellek çizimi (adresler örnek): `p` de bir değişkendir, kendi kutusu vardır ve içinde `x`'in adresi yazılıdır.

```
int x{10};          int* p{&x};

   x  (adres 1000)              p  (adres 1008)
+------------------+         +------------------+
|        10        |<--------|       1000       |
+------------------+         +------------------+
                             p'nin içindeki sayı = x'in adresi
```

Satır satır izle: `int* p{&x};` sonrası `p` = `&x`. `*p = 20;` ile `x` 20 olur, `p` değişmez. `*p = *p + 5;` satırında sağdaki `*p` okur (20), soldaki `*p` yazar, `x` 25 olur.

`*` işareti üç iş yapar; hangisi olduğunu bulunduğu yer belirler:

Nerede?	Anlamı	Örnek
Tanımda, tipin yanında	"bu bir işaretçi"	`int* p;`
İfadede, tek başına ve önde	dolaylama: "gösterilen yer"	`*p = 3;` , `y = *p;`
İki değerin arasında	çarpma	`a * b`

> **● Sık Yapılan Hata: `p` ile `*p`'yi karıştırmak**
>
> `p` işaretçinin kendisidir (adres), `*p` gösterdiği değerdir. `x`'i değiştirmek için `*p = 20;` yazılır. `p = 20;` ise işaretçiye adres yerine **sayı** atamaya çalışır ve derlenmez (`invalid conversion from 'int' to 'int*'`). Aynı şekilde `double d{1.5}; int* q{&d};` da derlenmez: işaretçinin tipi, gösterdiği değişkenin tipiyle uyuşmalıdır.

`int* p, q;` tuzağı

`*` tipe değil, **o değişkene** aittir. Aynı satırda iki değişken tanımlarsan:

ikiTanim.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int x{7};
    int* q1, q2;                    // q1 işaretçi, q2 sıradan int!

    q1 = &x;
    q2 = x;
    cout << "sizeof(q1) = " << sizeof(q1) << ", sizeof(q2) = " << sizeof(q2)
         << '\n';
    cout << "*q1 = " << *q1 << ", q2 = " << q2 << '\n';
    return 0;
}
```

Çıktı
```
sizeof(q1) = 8, sizeof(q2) = 4
*q1 = 7, q2 = 7
```

`q2` işaretçi değil, sıradan bir `int`'tir (`sizeof` farkına bak). Yıldızı tipin yanına yazman (`int* q1`) ya da değişkenin yanına yazman (`int *q1`) anlamı değiştirmez; boşluğun yeri derleyici için fark yaratmaz. Yalnız bu, okuyan kişiyi yanıltır. İki işaretçi için `int *a, *b;` yaz; en temizi **her işaretçiyi ayrı satırda** tanımlamaktır. Kitapta yıldızı tipe bitişik yazıyoruz (`int* p`) ve satır başına tek işaretçi tanımlıyoruz.

> **Sınavda Çıkar**
>
> "`int* p, q;` tanımında `q` neyin tipindedir?" klasik bir sorudur. Cevap: `int`. `*`, yalnız hemen sağındaki ada yapışır. `int *p, *q, r;` yazılırsa `p` ve `q` işaretçi, `r` sıradan `int` olur.

9.3 İşaretçinin tipi

İşaretçinin içinde yalnızca bir adres vardır. Peki neden `int*`, `double*`, `char*` diye ayrı tipler var? Çünkü adres bir yerin başlangıcını söyler ama **orada ne olduğunu** söylemez. İşaretçinin tipi iki şeyi belirler: (1) `*p` yazdığında **kaç bayt** okunacak ve bu baytların nasıl yorumlanacağı; (2) `p + 1` yazdığında adresin **kaç bayt** ilerleyeceği (bunu işaretçi aritmetiğinde göreceksin).

Bugünkü bilgisayarlarda bütün işaretçiler **aynı boyuttadır**, çünkü hepsi bir adres tutar: `sizeof(char*)`, `sizeof(int*)` ve `sizeof(double*)` 64 bitlik sistemlerde (Windows, macOS ve Linux'ta kullandığın bilgisayarlar) **8** bayttır; az önceki `ikiTanim` programında `sizeof(q1)` 8 çıkmıştı. Eski 32 bitlik sistemlerde 4'tü. Gösterilen değerin boyutu ise tipe göre değişir: `int* pi` ve `double* pd` için `sizeof(*pi)` 4, `sizeof(*pd)` 8'dir.

9.4 Boş işaretçi: nullptr

Gösterecek bir şeyin henüz yoksa işaretçiye **hiçbir yeri göstermeyen** özel değeri verirsin: `nullptr`. Bu değere sahip işaretçiye **boş işaretçi** (null pointer) denir. `nullptr` C++11 ile geldi; eski kodlarda aynı iş için `NULL` ya da `0` yazıldığını göreceksin. Sen her zaman `nullptr` yaz: `NULL` aslında sıradan bir sıfırdır ve işaretçi mi sayı mı olduğu bağlamdan çıkarılır; `nullptr` ise yalnızca işaretçi olabilen ayrı bir tipe sahiptir. Kullanmadan önce `if (p == nullptr)` ya da kısaca `if (p)` ile kontrol etmek standart güvenlik adımıdır (`p` boş değilse koşul doğru sayılır).

Boş işaretçinin gösterdiği yere **erişmek** tanımsız davranıştır: standart bu durum için hiçbir sonuç öngörmez. Bugünkü masaüstü sistemlerde program çoğu zaman çöker. Aşağıdaki kod bunu gösterir. **Bu kodu çalıştırma**, çünkü sonucu sisteme göre değişir:

```cpp
#include <iostream>

using namespace std;

int main() {
    int* p{nullptr};                // henüz hiçbir yeri göstermiyor

    if (p == nullptr) {
        cout << "p henüz bir yeri göstermiyor." << endl;
    }
    cout << "Yazmadan önce" << endl;
    *p = 5;                         // TANIMSIZ DAVRANIŞ: nullptr'ın gösterdiği yere yazma
    cout << "Yazdıktan sonra" << endl;
    return 0;
}
```

Bu kodda `endl` kullandık: `endl` yazdıklarını hemen ekrana gönderir (Bölüm 3'te gördüğün fark), böylece program çökse bile son yazılar kaybolmaz. Kod çoğu sistemde `*p = 5;` satırında çöker ve son `cout` hiç çalışmaz. Çökme mesajı işletim sistemine göre değişir. Bu çökme aslında iyi haberdir, çünkü hata hemen ortaya çıkar. Daha kötüsü **başlatılmamış işaretçidir**:

```
int* p;            // ilk değer verilmedi: içinde rastgele bir adres var
*p = 5;            // TANIMSIZ DAVRANIŞ: bilinmeyen bir yere yazar
```

Bölüm 2'deki çöp değerin işaretçideki karşılığıdır: `p`'nin içinde o bellekte kalmış rastgele bir sayı vardır ve bu sayı adres sayılır. Program çökebilir, başka bir değişkeni sessizce bozabilir ya da hiçbir şey olmamış gibi çalışabilir; "benim bilgisayarımda çalıştı" bir kanıt değildir. Üstelik içinde `nullptr` değil çöp olduğu için `if (p)` ile de yakalayamazsın.

Özetle: geçerli adres gösteren işaretçi güvenlidir, `nullptr` kontrolle güvenli olur, başlatılmamış işaretçi hiçbir zaman güvenli olmaz. Altın kural: **İşaretçiyi tanımlarken ya geçerli bir adres ya da `nullptr` ver** (`int* p{nullptr};`).

9.5 Referanslar

Referansı Bölüm 6'da fonksiyon parametresi olarak tanıdın: `int &a` parametresi, çağıranın değişkeninin takma adıydı. Şimdi onu işaretçiyle yan yana koyup bellekte ne olduğuna bakalım. İşaretçi güçlüdür ama kullanması zahmetlidir: her yerde `&` ve `*` yazarsın, boş olabileceğini unutursun. Referans ise var olan bir değişkene verilen **ikinci addır** (takma ad, alias). Yalnız parametrede değil, sıradan bir değişken olarak da tanımlanabilir: tipin yanına `&` konur ve **tanımlanırken** hangi değişkene bağlanacağı söylenir:

referansTemel.cpp

```
#include <iostream>

using namespace std;

int main() {
    int x{10};
    int& r{x};                      // r, x'in ikinci adı

    cout << "x = " << x << ", r = " << r << '\n';
    cout << "aynı adres mi: " << (&r == &x) << '\n';
    r = 20;                         // x'i değiştirir
    cout << "x = " << x << ", r = " << r << '\n';

    int y{50};
    r = y;                          // r'yi y'ye bağlamaz! x = y demektir
    cout << "x = " << x << ", y = " << y << '\n';
    y = 99;
    cout << "x = " << x << ", r = " << r << '\n';
    return 0;
}
```

Çıktı

```
x = 10, r = 10
aynı adres mi: 1
x = 20, r = 20
x = 50, y = 50
x = 50, r = 50
```

Bellekte `r` adlı ayrı bir kutu **yoktur**: `x` ile `r` aynı kutunun iki etiketidir. Bu yüzden `&r` ile `&x` aynı adrestir. İşaretçide `*p` yazman gerekirken referansta bir şey yazmazsın; `r` ne zaman kullanılsa `x` kullanılmış gibidir.

Referansın üç kuralı vardır ve üçü de sınavda sorulur:

1. **Tanımlanırken ilk değer verilmelidir.** Boş referans yoktur.
2. **Bir kez bağlanır, sonra başka değişkene bağlanamaz.** Yukarıdaki `r = y;` satırı `r`'yi `y`'ye çevirmedi; `y`'nin **değerini** `x`'e kopyaladı. Sonra `y = 99;` yazınca `x`'in değişmemesi bunun kanıtıdır.
3. **Geçici değere bağlanamaz** (`const` referans hariç; birazdan göreceksin).

İlk kural ihlal edilirse (`int& r;`) ya da geçici değere bağlamaya çalışırsan (`int& s{5};`) program derlenmez; üçüncü kural `const` referansta gevşer, aşağıda göreceksin.

> **Not: Derleyiciye göre hata mesajı**
>
> Kitaptaki hata ve uyarı metinleri gerçek `g++` çıktısıdır. Mac'te `g++` yazınca çalışan program çoğu zaman Apple'ın `clang` derleyicisidir; mesajlar farklı sözcüklerle yazılır, hatta bazı uyarılar hiç çıkmayabilir. Bir hatanın **olup olmadığı** aynıdır, ifadesi değişir. Mesajın tamamını ezberleme; satır numarasına ve "nerede" sorusuna bak.

9.6 İşaretçi ve referansla fonksiyona geçirme

Bölüm 6'dan hatırla: argümanlar varsayılan olarak **değer ile** geçer. Fonksiyondaki parametre, çağıran taraftaki değişkenin **kopyasıdır**; `takas(x, y)` ancak parametreler referans (`int &a`) yapılınca çalışmıştı. Aynı işin ikinci bir yolu daha var: değişkenin kendisini değil **adresini** vermek. Önce bu yolu görelim, sonra Bölüm 6'daki referanslı çözümle karşılaştıralım.

İşaretçi ile geçirme

Değişkenin adresini geçirirsin. Fonksiyon adresin kopyasını alır, ama kopya da aynı yeri gösterdiği için `*a = ...` yazınca çağıran taraftaki değişken değişir.

takasIsaretci.cpp

```cpp
#include <iostream>

using namespace std;

void takas(int* a, int* b) {
    int gecici{*a};
    *a = *b;
    *b = gecici;
}

int main() {
    int x{3};
    int y{8};

    cout << "Önce  : x = " << x << ", y = " << y << '\n';
    takas(&x, &y);                  // değişkenlerin adreslerini ver
    cout << "Sonra : x = " << x << ", y = " << y << '\n';
    return 0;
}
```

Çıktı

```
Önce  : x = 3, y = 8
Sonra : x = 8, y = 3
```

`takas` çalışırken bellek şöyledir. `a` ve `b`, `takas`'ın yerel değişkenleridir (içlerinde adres var); `gecici` sıradan bir `int`'tir:

```
   main:  x [3] <------ a [&x]      takas'ın yerel değişkenleri:
          y [8] <------ b [&y]      a, b (adres tutar), gecici (sıradan int)
```

Adım adım: `gecici = *a;` ile `gecici` 3 olur; `*a = *b;` ile `x` 8 olur; `*b = gecici;` ile `y` 3 olur. Fonksiyondan dönünce `a`, `b` ve `gecici` yok olur, ama `x` ile `y` `main`'e aittir ve değişmiş hâlde kalır.

> **Sık Yapılan Hata: Çağırırken `&` unutmak**
>
> `void takas(int* a, int* b)` fonksiyonunu `takas(x, y);` diye çağırırsan `int` değerini işaretçi parametreye vermiş olursun; program derlenmez (`invalid conversion from 'int' to 'int*'`). Parametre `T*` ise çağırırken `&değişken` yazman gerekir. Referans parametrede ise böyle bir şey yazmazsın; fark, bir sonraki başlıkta.

Referans ile geçirme

Bölüm 6'daki çözüm buydu: parametreyi referans yaparsan çağıran taraftaki değişkene ikinci bir ad verirsin. Fonksiyonun gövdesinde hiçbir `*` ya da `&` yazmazsın, çağırırken de yazmazsın:

```
void takas(int& a, int& b) {        // a ve b, x ile y'nin ikinci adları
    int gecici{a};
    a = b;
    b = gecici;
}
// çağrı: takas(x, y);              // & yok, * yok; sonuç öncekiyle aynı
```

Bölüm 6'da bunun adının referans ile geçirme, kopyalayan geçirmenin adının ise değer ile geçirme olduğunu görmüştün. İşaretçi ile geçirmenin ayrı bir adı yoktur: işaretçi de bir değerdir, yani işaretçi geçirmek aslında adresin **değer ile** geçirilmesidir. Çıktıdan bakınca ikisi aynı sonucu verir; fark kullanım rahatlığında ve güvenlikte.

Artık `cin >> yas`'ın sırrı da çözüldü: `>>` operatörünün sağ tarafı bir referans parametredir. `cin`, `yas`'ın kopyasını değil kendisini alır ve içine yazar. İşaretçi gerekmediği için `&yas` yazmazsın (C kitabında `scanf("%d", &yas)` yazılırdı; orada işaretçi vermek zorunluydu).

Birden çok sonuç döndürme

Bölüm 6'daki çıkış parametrelerini hatırla: `return` tek değer döndürdüğü için sonuçların yazılacağı değişkenler referans parametre olarak verilir. Kısa bir tekrar:

bolVeKalan.cpp

```
#include <iostream>

using namespace std;

void bolVeKalan(int a, int b, int& bolum, int& kalan) {
    bolum = a / b;
    kalan = a % b;
}

int main() {
    int bolum{0};
    int kalan{0};

    bolVeKalan(17, 5, bolum, kalan);
    cout << "17 / 5 = " << bolum << ", kalan " << kalan << '\n';
    return 0;
}
```

```
Çıktı
17 / 5 = 3, kalan 2
```

Yeni olan şu: çağrıdan hangi argümanların değişeceği anlaşılmaz; bu yüzden bazı ekipler çıkış parametresi için işaretçi (`&bolum`) tercih eder. Sınavda hangisini istediği soruda yazar.

Kopyalamadan okumak: const referans

Bölüm 6'da büyük nesneleri `const string&` ile kopyalamadan geçirmeyi görmüştün. Değer ile geçirilen parametre kopyalandığı için, büyük bir `string` ya da `vector` her çağrıda baştan kopyalanır. Artık adresleri bildiğine göre bunu kanıtlayabiliriz; kopyalanıp kopyalanmadığını adresine bakarak anlarız:

kopyaMi.cpp

```cpp
#include <iostream>
#include <string>

using namespace std;

void karsilastir(string kopya, const string& ref, const string& asil) {
    cout << "değer ile  : kopya mı = " << (&kopya != &asil) << '\n';
    cout << "referansla : kopya mı = " << (&ref != &asil) << '\n';
}

int main() {
    string metin{"Bu uzun bir metin olsun diye yazıldı."};

    karsilastir(metin, metin, metin);
    return 0;
}
```

```
Çıktı
değer ile  : kopya mı = 1
referansla : kopya mı = 0
```

`const string&` hem kopyayı önler hem de fonksiyonun metni değiştirmesini yasaklar: fonksiyon "sadece okuyacağım" sözünü vermiştir ve derleyici o sözü denetler. Bir kural olarak al: **büyük nesneyi (`string`, `vector`) yalnız okuyacaksan `const T&` ile geçir.** `int`, `double`, `char`, `bool` gibi küçük tipler için kopyalamak zaten ucuzdur; onları değer ile geçir. `const` referansın bir ayrıcalığı daha var: geçici değere bağlanabilir. `const int& r{5};` geçerlidir, ama `int& r{5};` değildir.

Referans döndüren fonksiyon

Fonksiyon referans da döndürebilir. O zaman çağrı ifadesi bir değerin kopyası değil, **değişkenin kendisidir**; atamanın sol tarafına bile yazılabilir:

referansDondur.cpp

```cpp
#include <iostream>

using namespace std;

int& buyugu(int& a, int& b) {
    return (a > b) ? a : b;         // a ya da b'nin kendisi döner
}

int main() {
    int x{3};
    int y{8};
    buyugu(x, y) = 0;               // y büyük: y sıfırlanır
    buyugu(x, y) += 10;             // artık x büyük: x 13 olur
    cout << "x = " << x << ", y = " << y << '\n';
    return 0;
}
```

Çıktı

```
x = 13, y = 0
```

İlk çağrıda `y` büyük olduğu için `buyugu(x, y)` ifadesi `y`'nin ta kendisidir ve `= 0` onu sıfırlar. İkinci çağrıda `x` (3) `y`'den (0) büyüktür, `+= 10` bu kez `x`'e uygulanır. Bölüm 6'daki kural burada da geçerli: döndürülen referans, fonksiyon bittikten sonra da yaşayan bir değişkeni (burada `main`'deki `x` ya da `y`) göstermelidir; yerel değişkenin referansı döndürülmez. Bölüm 13'te `a[0] = 7;` yazımının da aynı yolla çalıştığını göreceksin.

İşaretçinin hâlâ gerektiği yer

Referans boş olamaz. Bir parametre **olmayabiliyorsa** ya da fonksiyon "bulamadım" diyebilecekse işaretçi kullanılır; `nullptr` "yok" demenin yoludur:

ilkNegatif.cpp

```cpp
#include <iostream>

using namespace std;

// ilk negatif elemanın adresini döndürür; yoksa nullptr
const int* ilkNegatifiBul(const int* dizi, int adet) {
    for (int i = 0; i < adet; i++) {
        if (dizi[i] < 0) {
            return &dizi[i];
        }
    }
    return nullptr;
}

int main() {
    int a[4] = {5, 8, -2, 7};
    const int* p = ilkNegatifiBul(a, 4);

    if (p != nullptr) {
        cout << "bulundu: " << *p << '\n';
    }
    return 0;
}
```

Çıktı
```
bulundu: -2
```

Fonksiyon dizinin kendisini değil bir işaretçi aldığı hâlde `dizi[i]` yazdık; bunun neden çalıştığını birazdan "Diziler ve işaretçiler" başlığında göreceksin. Özet tablo:

Durum	Ne kullan?	Neden
Küçük değer (`int`, `double`, `char`), değişmeyecek	değer ile (`int x`)	Kopya ucuz, en yalın
Parametre fonksiyonda değişecek, her zaman vardır	referans (`int& x`)	`*` ve `&` karmaşası yok, boş olamaz
Büyük nesne (`string`, `vector`), sadece okunacak	const referans (`const string& s`)	Kopya yok, değişmeyeceğinin sözü
Parametre "yok" olabilir ya da sonuç "bulunamadı" olabilir	işaretçi (`int*` ve `nullptr`)	Referans boş olamaz
Dizi ve eleman sayısı (C tarzı)	işaretçi (`int* a, int n`)	Dizi adresi olarak geçer
Sonradan başka nesneye yönelmesi gerek	işaretçi	Referans tek nesneye bağlıdır

Pratik kural: **önce referans, gerekirse işaretçi.**

> **İpucu: Okuyunca "bu değişir mi?" diye sor**
>
> Bir fonksiyonun başlığına bakıp ne yapacağını anlayabilirsin: `f(int x)` kendi kopyasıyla çalışır. `f(int& x)` ve `f(int* x)` değişkeni değiştirebilir. `f(const string& s)` ve `f(const int* p)` yalnız okur. Kendi fonksiyonlarını yazarken `const` koymak okuyana söz verir, derleyiciye de denetleme yetkisi verir.

9.7 İşaretçi aritmetiği

İşaretçilerle toplama ve çıkarma yapabilirsin, ama kuralı bilmelisin: `p + 1` adresi **1 bayt değil, 1 eleman** ilerletir. Eleman boyutu işaretçinin tipinden gelir (`char*` için 1, `int*` için 4, `double*` için 8 bayt; yani `sizeof(*p)`). İşaretçinin tipinin önemi tam burada ortaya çıkar. Bu işlemlere topluca **işaretçi aritmetiği** (pointer arithmetic) denir.

```
  a[0]   a[1]   a[2]   a[3]   a[4]
+------+------+------+------+------+
|  10  |  20  |  30  |  40  |  50  |
+------+------+------+------+------+
 1000   1004   1008   1012   1016        (örnek adresler)
  ^
  p          p + 1 = 1004 (1001 değil!),   p + 3 = 1012
```

İşlem	Anlamı	Sonuç
`p + n`, `p - n`	`n` eleman ileri/geri	işaretçi
`p++`, `p--`	bir eleman ileri/geri (`p`'yi değiştirir)	işaretçi
`p - q`	aralarındaki **eleman sayısı** (aynı dizide)	tamsayı
`p < q`, `p == q`	adres karşılaştırması	`bool`
`p + q`, `p * 2`, `p / 2`	anlamsız, **derlenmez**	

aritmetik.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int a[5] = {10, 20, 30, 40, 50};
    int* p{&a[0]};

    cout << *p << '\n';             // 10
    p++;                            // a[1]'e geç
    cout << *p << '\n';             // 20
    p = p + 2;                      // a[3]'e geç
    cout << *p << '\n';             // 40
    p--;                            // a[2]'ye dön
    cout << *p << '\n';             // 30
    cout << *(p + 1) << '\n';       // 40, p değişmez

    cout << a[3] << " " << *(a + 3) << '\n';      // ikisi de 40

    int* q{&a[4]};
    cout << "q - p = " << (q - p) << '\n';      // 4 - 2
    return 0;
}
```

Çıktı

```
10
20
40
30
40
40 40
q - p = 2
```

İki işaretçiyi **çıkarmak** eleman sayısını verir (`q - p` = 2), bayt farkını değil. Çıkarma ve karşılaştırma yalnız **aynı dizinin** elemanlarını gösteren işaretçiler için anlamlıdır. Dizinin **bir ötesini** gösteren işaretçi (`a + 5`) oluşturmak ve karşılaştırmak serbesttir (döngülerde kullanacaksın), ama onu `*` ile okumak ya da dizinin daha ötesine gitmek tanımsız davranıştır. Buna **sınırın bir ötesi** (one-past-the-end) adresi denir.

`*p++` ile `(*p)++` farkı

`++` sonek operatörü `*`'dan **daha yüksek önceliklidir**. Bu yüzden `*p++` "`p`'nin gösterdiğini kullan, sonra `p`'yi ilerlet" demektir; gösterilen değeri artırmaz:

İfade	Nasıl okunur	Ne yapar?
`*p++`	`*(p++)`	Önce `*p`'yi kullan, sonra `p`'yi bir eleman ilerlet
`(*p)++`	Parantez önce: `*p`, sonra `++`	`p` sabit kalır, gösterilen **değer** 1 artar (ifadenin değeri olarak artmadan önceki değer alınır)
`*++p`	`*(++p)`	Önce `p`'yi ilerlet, sonra yeni yerdeki değeri kullan

artirmaOncelik.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int a[4] = {10, 20, 30, 40};
    int* p{a};

    int ilk{*p++};                  // ilk = 10, sonra p -> a[1]
    cout << "ilk = " << ilk << ", *p = " << *p << '\n';
    (*p)++;                         // a[1]: 20 -> 21, p yerinde
    cout << "*p = " << *p << ", a[1] = " << a[1] << '\n';
    int ucuncu{*++p};               // önce p -> a[2], sonra oku
    cout << "ucuncu = " << ucuncu << '\n';
    cout << a[0] << " " << a[1] << " " << a[2] << " " << a[3] << '\n';
    return 0;
}
```

Çıktı

```
ilk = 10, *p = 20
*p = 21, a[1] = 21
ucuncu = 30
10 21 30 40
```

> **Sınavda Çıkar**
>
> `*p++` ile `(*p)++` ayrımı neredeyse her işaretçi sınavındadır. Takip yöntemi: iki şeyi ayrı ayrı yaz, "`p` nereyi gösteriyor?" ve "dizide ne var?". `*p++`'da **p** değişir, dizi değişmez. `(*p)++`'da **dizi** değişir, `p` değişmez. Parantez yoksa `++` önce `p`'ye yapışır.

9.8 Diziler ve işaretçiler

İşte Bölüm 7'deki sorunun cevabı: dizi adı, ifadelerde çoğu zaman **ilk elemanın adresine** dönüşür. Bu dönüşüme **dizi bozunması** (array-to-pointer decay) denir.

```
int a[5] = {10, 20, 30, 40, 50};

a           ≡  &a[0]       (ilk elemanın adresi)
a + i       ≡  &a[i]
*(a + i)    ≡  a[i]        (indeks yazımı, bu işlemin kısa yoludur)
```

`a[i]` yazımı derleyici için tam olarak `*(a + i)` demektir. İşaretçilere de indeks uygulanabilir: `p[i]` ile `*(p + i)` aynıdır.

Yukarıdaki `aritmetik` programında `a[3]` ile `*(a + 3)` aynı değeri yazdı. Bu eşdeğerlik işaretçiler için de geçerlidir: `p[i]` ile `*(p + i)` aynıdır (`p[-1]` bile yazabilirsin; `p`'nin bir önceki elemanı).

Diziyi dolaşmanın yollarından biri **gezen işaretçidir**. Dizinin bir ötesi burada işe yarar:

```cpp
for (int* p{a}; p < a + adet; p++) {    // a + adet: dizinin bir ötesi
    toplam += *p;                       // a[i] yerine *p
}
```

`p < a + adet` bitiş koşuludur ve `*p` o adrese hiç erişmez. Aynı işi indeksle de yapabilirsin; iki yazım eşdeğerdir. Bölüm 7'de gördüğün `vector` ve aralık tabanlı `for` (`for (int x : a)`) çoğu zaman en okunur yoldur. İşaretçi biçimini ise standart kütüphanenin içinde (Bölüm 15'te iteratörlerde) tekrar göreceksin: iteratör, işaretçinin genelleştirilmiş hâlidir.

Dizi adı ile işaretçi aynı şey değildir

Aralarındaki ilişki "dönüşüm"dür, eşitlik değil. Dizi adı bir **işaretçi değişkeni** olmadığı için `sizeof(a)` bütün dizinin boyutunu verir (`sizeof(p)` işaretçinin boyutunu, 8) ve `a`'ya **atama yapılamaz**, `p`'ye yapılır: dizinin yeri sabittir.

```
int main() {
    int a[3] = {1, 2, 3};
    int b[3] = {4, 5, 6};
    int* p{b};                      // olur: p değişken

    a = p;                          // hata: dizi adına atama
    a++;                            // hata: dizi adı ilerletilemez
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
diziAdiAtama.cpp: In function 'int main()':
diziAdiAtama.cpp:6:7: error: incompatible types in assignment of 'int*' to 'int [3]'
    6 |     a = p;                          // hata: dizi adına atama
      |     ~~^~~
diziAdiAtama.cpp:7:5: error: lvalue required as increment operand
    7 |     a++;                            // hata: dizi adı ilerletilemez
      |     ^
```

Fonksiyona dizi geçirmek

Dizi adı adrese dönüştüğü için fonksiyon **dizinin kopyasını değil, ilk elemanın adresini** alır. Parametrede `int a[]` yazmak `int* a` yazmakla aynıdır. Bölüm 7'de "dizi fonksiyona neden kopyalanmıyor?" diye sormuştun: sebep budur. Fonksiyon ana dizinin kendisini kullanır:

diziGecir.cpp

```
#include <iostream>

using namespace std;

void ikiyeKatla(int a[], int adet) {        // int* a ile aynı
    cout << "fonksiyonda: sizeof(a)     = " << sizeof(a) << '\n';
    for (int i = 0; i < adet; i++) {
        a[i] = a[i] * 2;
    }
}

int main() {
    int sayilar[4] = {1, 2, 3, 4};

    cout << "main'de    : sizeof(sayilar) = " << sizeof(sayilar) << '\n';
    ikiyeKatla(sayilar, 4);
    cout << sayilar[0] << " " << sayilar[1] << " " << sayilar[2] << " "
         << sayilar[3] << '\n';
    return 0;
}
```

Derleyici uyarısı – g++ -Wall -Wextra

```
diziGecir.cpp: In function 'void ikiyeKatla(int*, int)':
diziGecir.cpp:6:55: warning: 'sizeof' on array function parameter 'a' will return size of 'int*' [-
Wsizeof-array-argument]
    6 |     cout << "fonksiyonda: sizeof(a)     = " << sizeof(a) << '\n';
      |                                                       ~^~
diziGecir.cpp:5:21: note: declared here
    5 | void ikiyeKatla(int a[], int adet) {        // int* a ile aynı
      |                 ~~~~^~
```

Çıktı

```
main'de    : sizeof(sayilar) = 16
fonksiyonda: sizeof(a)     = 8
2 4 6 8
```

Fonksiyonda yapılan değişiklik `main`'deki diziyi değiştirdi, çünkü fonksiyon aynı belleği gösteriyordu. Fonksiyon yalnız **adresi** bildiği için dizinin kaç elemanlı olduğunu bilemez; uzunluğu (`adet`) ayrıca geçirmek zorundasın. `sizeof` tuzağı da buradan doğar: g++ uyarıyor, çünkü fonksiyondaki `a` bir dizi değil `int*`'dır ve `sizeof(a)` işaretçinin boyutunu verir (8 değeri 64 bitlik sistem içindir). Yani fonksiyon içinde `sizeof(a) / sizeof(a[0])` eleman sayısını **vermez**: 8 / 4 = 2 çıkar. Eleman sayısını her zaman dizinin tanımlandığı yerde hesapla, fonksiyona ayrı parametre olarak ver.

> **Sınavda Çıkar**
>
> "`void f(int a[])` içinde `sizeof(a)` kaçtır?" Cevap: dizinin boyutu **değil**, işaretçinin boyutu (64 bitte 8). Bir de: "fonksiyonda `a[0] = 99;` yapılırsa `main`'deki dizi değişir mi?" Evet; dizi kopyalanmaz, adresi geçer. Bu, `int` parametresinden farklıdır: `int` kopyalanır, dizi kopyalanmaz.

9.9 const ve işaretçiler

`const` işaretçide iki farklı şeyi sabitleyebilir: **gösterilen değeri** ya da **işaretçinin kendisini**. Ayrımı `const`'ın `*`'a göre yeri belirler:

Tanım	Gösterilen değer (`*p = ...`)	İşaretçi (`p = ...`)	Okunuşu
`int* p`	değişir	değişir	`int`'e işaretçi
`const int* p`	**değişmez**	değişir	sabit `int`'e işaretçi
`int* const p`	değişir	**değişmez**	`int`'e sabit işaretçi
`const int* const p`	**değişmez**	**değişmez**	sabit `int`'e sabit işaretçi

Ezber kuralı: `const` `*`'ın **solundaysa** gösterilen veri sabittir; `*`'ın **sağındaysa** işaretçi sabittir. Karışırsan tanımı **sağdan sola** oku: `int* const p` → "`p`, sabit bir işaretçidir, `int`'e işaret eder". `int const* p` yazımı da `const int* p` ile aynıdır; `const` `*`'ın solunda olduğu sürece hangi yana yazdığın fark etmez.

```cpp
int main() {
    int a{1};
    int b{2};
    const int* p1{&a};              // gösterilen değer sabit
    int* const p2{&a};              // işaretçi sabit

    p1 = &b;                        // olur: işaretçi değişebilir
    *p2 = 5;                        // olur: gösterilen değişebilir
    *p1 = 9;                        // HATA
    p2 = &b;                        // HATA
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
constHata.cpp: In function 'int main()':
constHata.cpp:9:9: error: assignment of read-only location '* p1'
    9 |     *p1 = 9;                        // HATA
      |     ~~~~^~~
constHata.cpp:10:8: error: assignment of read-only variable 'p2'
   10 |     p2 = &b;                        // HATA
      |     ~~~^~~
```

İki hata mesajı da “salt okunur” (read-only) ya da “const” diyor. Referanslarda ayrı bir `const` yeri yoktur: referans zaten sonradan başka değişkene bağlanamadığı için “sabit referans” kavramı anlamsızdır. `const int& r{a};` yazarsan `r = 7;` derlenmez; ama `a` kendi adıyla yine değişebilir (`a = 7;` geçerlidir). En çok kullanacağın biçimler `const T*` ve `const T&`’dır: fonksiyona “bu veriyi sadece okuyacağım” sözü vermek için. Derleyici o sözü senin yerine denetler.

Metin sabitleri de buraya girer. Çift tırnak içindeki `"Ali"` programın salt okunur bir bölgesinde durur; bu yüzden ona işaret eden işaretçiyi **`const char*`** olarak tanımlarsın:

```cpp
char dizi[] = "Ali";            // 4 baytlık kopya: 'A' 'l' 'i' '\0'
const char* metin = "Ali";      // işaretçi + salt okunur "Ali"
dizi[0] = 'E';                  // olur: dizi senin kopyan
metin = "Veli";                 // olur: işaretçi başka yere yönelebilir
```

Burada `sizeof(dizi)` 4, `sizeof(metin)` 8’dir: biri metnin kopyası, öbürü yalnız bir adrestir.

C++’ta metin tutmak için artık `string` kullanırsın (Bölüm 8); bu örnek yalnız eski kodda ve kütüphanelerde `const char*` gördüğünde ne olduğunu bilmen içindir. `char* p = "Ali";` yazarsan g++ uyarır, çünkü salt okunur metne değiştirilebilir işaretçi vermiş olursun; sonrasında `p[0] = 'E';` tanımsız davranıştır (çoğu zaman çökme).

> **● Sınavda Çıkar**
>
> “Aşağıdaki satırlardan hangisi derlenmez?” biçiminde bir `const` sorusu mutlaka çıkar. Yöntem: tanımı yaz (`const int* p`), sonra her satıra bak. `*p = ...` yazıyor ve `const` `*`’ın solundaysa **hata**. `p = ...` yazıyor ve `const` `*`’ın sağındaysa **hata**. İki tarafta da `const` varsa ikisi de hata.

İşaretçinin işaretçisi (İleri)

İşaretçi de bir değişken olduğuna göre onun da adresi vardır. O adresi tutan değişkene **işaretçinin işaretçisi** denir: `int** pp`. Her `*` bir basamak gösterir. İlk okumada atlayabilirsin; ama fonksiyonun bir işaretçiyi **değiştirmesi** gereken yerde karşına çıkar.

ciftIsaretci.cpp

```cpp
#include <iostream>

using namespace std;

void yonelt(int** hedef, int* yeni) {
    *hedef = yeni;                  // main'deki işaretçiyi değiştir
}

int main() {
    int x{5};
    int y{8};
    int* p{&x};                     // p, x'i gösterir
    int** pp{&p};                   // pp, p'yi gösterir

    **pp = 99;                      // pp -> p -> x
    cout << "x = " << x << '\n';
    yonelt(&p, &y);                 // p artık y'yi gösterir
    cout << "*p = " << *p << '\n';
    return 0;
}
```

Çıktı

```
x = 99
*p = 8
```

Kural basit: `takas`'ta bir `int`'i değiştirmek için `int*` (ya da `int&`) geçirmiştik; bir `int*`'ı değiştirmek için de `int**` (ya da `int*&`) geçirmek gerekir. Aynı kural, bir seviye yukarıda. Dinamik bellekten ayırdığın işaretçileri değiştirirken Bölüm 10'da yine karşılaşacaksın.

9.10 Sarkan işaretçi: yok olmuş değişkeni göstermek

Yerel değişken, fonksiyon bitince yok olur (Bölüm 6'daki çağrı yığını). Adresini ya da referansını döndürürsen çağıranın elinde **artık var olmayan bir yeri gösteren** işaretçi (ya da referans) kalır. Buna **sarkan işaretçi** (dangling pointer) denir:

yerelAdres.cpp

```cpp
#include <iostream>

using namespace std;

int* yanlisAdres() {
    int sayi{42};
    return &sayi;                   // HATA: sayi, fonksiyon bitince yok olur
}

int& yanlisReferans() {
    int sayi{42};
    return sayi;                    // HATA: aynı sorun, referansla
}

int main() {
    int* p{yanlisAdres()};
    int& r{yanlisReferans()};
    (void)p;                        // p ve r'yi kullanmak tanımsız davranıştır
    (void)r;
    return 0;
}
```

```
Derleyici uyarısı – g++ -Wall -Wextra
yerelAdres.cpp: In function 'int* yanlisAdres()':
yerelAdres.cpp:7:12: warning: address of local variable 'sayi' returned [-Wreturn-local-addr]
    7 |     return &sayi;                  // HATA: sayi, fonksiyon bitince yok olur
      |            ^~~~
yerelAdres.cpp:6:9: note: declared here
    6 |     int sayi{42};
      |         ^~~~
yerelAdres.cpp: In function 'int& yanlisReferans()':
yerelAdres.cpp:12:12: warning: reference to local variable 'sayi' returned [-Wreturn-local-addr]
   12 |     return sayi;                   // HATA: aynı sorun, referansla
      |            ^~~~
yerelAdres.cpp:11:9: note: declared here
   11 |     int sayi{42};
      |         ^~~~
```

Bu programı **çalıştırmadık**, çünkü `*p` ile ya da `r` ile okumak tanımsız davranıştır: o bellek sonraki çağrılarda başka işler için kullanılır; eski değer okunabilir, çöp de okunabilir, program da çökebilir. g++'ın `-Wreturn-local-addr` uyarısı bu hatayı yakalar; **uyarı olsa da asla yok sayma**. Aynı hata yerel bir dizide de olur: fonksiyonun içindeki bir diziyi `return dizi;` ile döndürürsen çağıran sarkan bir işaretçi alır.

Sarkan işaretçi yalnız fonksiyon dönüşünde oluşmaz: iç bloktaki değişkenin adresini blok dışına taşırsan (`p = &gecici;` ve blok biter) aynı durum doğar.

Sarkan işaretçiyi `if (p)` ile de yakalayamazsın: `p`'nin içinde hâlâ eski adres durur, boş değildir. Çözümler:

1. Sonucu **değer olarak** döndür: `int kareAl(int x)` kopyayı döndürür, kopya çağıranın olur. Çoğu durumda doğru yoldur.
2. Sonucu çağıranın verdiği yere yaz (çıkış parametresi, `bolVeKalan` gibi).
3. `static` yerel değişkenin adresini ya da referansını döndür: `static int adet{0};` gibi bir değişken program boyunca yaşar, bu yüzden dönüşten sonra da geçerlidir.
4. Kalıcı yer ayır: `new`, Bölüm 10'da.

> **● Sık Yapılan Hata: Yerel değişkene referans ya da işaretçi döndürmek**
>
> Fonksiyon `int*` ya da `int&` döndürüyorsa `return` ettiğin şeyin fonksiyon bitince yaşadığından emin ol. Hata bazen çalışıyormuş gibi görünür: yok olan değişkenin değeri bellekte henüz ezilmemiştir. Sonra başka kod eklediğinde ya da başka bilgisayarda program saçmalar. "Bende çalıştı" burada hiçbir şey kanıtlamaz.

● Bölüm Özeti – Bilmen Gerekenler

- Her değişkenin bir **adresi** vardır; `&x` adresi verir (adres yazdırınca değer bilgisayardan bilgisayara değişir).
- İşaretçi adres tutan değişkendir: `int* p{&x};`. `*p` gösterilen yere erişir (dolaylama), `*p = 20;` `x`'i değiştirir. `p` adrestir, `*p` değerdir.
- `int* p, q;` yalnız `p`'yi işaretçi yapar. 64 bitte tüm işaretçiler 8 bayttır; tip, `*p`'nin kaç bayt okuyacağını ve `p + 1`'in ne kadar ilerleyeceğini belirler.
- `nullptr` hiçbir yeri göstermez; `*p` ile erişmek tanımsız davranıştır, masaüstünde genellikle çökertir. Başlatılmamış işaretçi tanımsızdır. Tanımlarken adres ya da `nullptr` ver, kullanmadan önce `if (p)` ile kontrol et.
- **Referans** var olan değişkene verilen ikinci addır: tanımlanırken bağlanmak zorundadır, sonra başka değişkene bağlanamaz (`r = y;` değer kopyalar), boş olamaz, `&r == &x`.
- Değer ile geçirme kopyalar. Değişkeni değiştirmek için referans (`int&`) ya da işaretçi (`int*`) geçirilir; büyük nesneyi yalnız okumak için `const T&` kullanılır. Önce referans, gerekirse işaretçi.

- `p + 1` bir **eleman** ilerler (bayt değil); `p - q` eleman sayısını verir. `*p++` işaretçiyi ilerletir, `(*p)++` gösterilen değeri artırır.
- `a[i]` ≡ `*(a + i)`. Fonksiyona dizi geçince adres geçer (`int a[]` ≡ `int* a`), bu yüzden fonksiyon içinde `sizeof(a)` işaretçi boyutunu verir. Dizi adına atama yapılamaz.
- `const int* p`: değer sabit; `int* const p`: işaretçi sabit. `const char*` metin sabitleri içindir. `const T&` salt okunur referanstır.
- Yerel değişkenin adresini ya da referansını döndürmek sarkan işaretçi yaratır: tanımsız davranış. `int**` / `int*&` fonksiyonun işaretçiyi değiştirmesi gerektiğinde çıkar.

Sorular

A) Çoktan Seçmeli

1 `int* p, q;` tanımından sonra aşağıdakilerden hangisi doğrudur?

(A) `p` ve `q` ikisi de `int` işaretçisidir.
(B) `p` sıradan bir `int`'tir, `q` bir `int` işaretçisidir.
(C) `p` bir `int` işaretçisidir, `q` sıradan bir `int`'tir.
(D) `q`, `int` işaretçisinin işaretçisidir.
(E) Tanım derlenmez.

2 Aşağıdaki kod parçasından sonra `*(p - 1) + p[1]` ifadesinin değeri kaçtır?

```
int a[] = {10, 20, 30, 40, 50};
int* p = a + 2;
```

(A) 60 (B) 70 (C) 50 (D) 80 (E) 90

3 `int a{1}, b{2}; const int* p{&a}; int* const q{&a};` tanımlarından sonra aşağıdaki satırlardan hangisi **derlenmez**?

(A) `p = &b;` (B) `*q = 5;` (C) `a = 7;` (D) `cout << *p;` (E) `q = &b;`

4 `void f(int a[]) { ... }` fonksiyonu ana programdan `f(dizi);` diye çağrılıyor. Aşağıdakilerden hangisi doğrudur?

(A) `a`, `dizi`'nin kopyasıdır; fonksiyonda yapılan değişiklikler `dizi`'yi etkilemez.
(B) `a`, fonksiyon içinde bir işaretçidir; `sizeof(a)` dizinin boyutunu değil işaretçinin boyutunu verir.
(C) `int a[]` ile `int* a` farklı anlama gelir; ikisi aynı parametre olarak kullanılamaz.
(D) `sizeof(a) / sizeof(a[0])` her zaman dizinin eleman sayısını verir.
(E) Fonksiyon, dizinin kaç elemanlı olduğunu `a`'dan otomatik öğrenir.

5 `int x{5}, y{9}; int& r{x}; r = y;` satırlarından sonra aşağıdakilerden hangisi doğrudur?

A) `r`, artık `y`'nin ikinci adıdır; `x` 5 kalır.

B) Referanslar da işaretçiler gibi `nullptr` olabilir, bu yüzden `r` kullanmadan önce kontrol edilmelidir.

C) `r = y;` satırı derleme hatasıdır; referans bir kez bağlanınca ona atama yapılamaz.

D) `r`, hâlâ `x`'in ikinci adıdır ve `x` artık 9'dur.

E) `sizeof(r)` her zaman 8'dir, çünkü referans bir adres tutar.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
#include <iostream>

using namespace std;

int main() {
    int a[4] = {3, 6, 9, 12};
    int* p{a};
    int x{*p++};
    int y{(*p)++};
    int z{*++p};

    cout << x << " " << y << " " << z << '\n';
    cout << a[0] << " " << a[1] << " " << a[2] << " " << a[3] << '\n';
    cout << *p << '\n';
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>

using namespace std;

void artir(int& a, int b) {
    b = b * 2;
    a += b;
}

int main() {
    int x{2};
    int y{3};

    artir(x, y);
    artir(y, x);
    int& r{x};
    int* p{&y};
    r = r * 2;
    *p = *p + r;
    cout << x << " " << y << '\n';
    r = y;
    p = &x;
    *p += 1;
    cout << x << " " << y << " " << r << " " << *p << '\n';
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>

using namespace std;

int main() {
    int a[6] = {1, 3, 5, 7, 9, 11};
    int* p{a + 1};
    int* q{&a[5]};

    cout << q - p << '\n';
    cout << *(q - 2) << '\n';
    *(p + 1) = *p + *(q - 1);
    p += 2;
    cout << *p << " " << p[-1] << '\n';
    cout << (p == a + 3) << '\n';
    return 0;
}
```

C) Kod Yazma

9 Klavyeden 5 tamsayı okuyup **en büyük** olanı bulan `enBuyuk` programını yaz. Bulma işi `int* enBuyuguBul(int* a, int adet)` fonksiyonunda yapılsın: fonksiyon en büyük elemanın **adresini** döndürsün (`adet` 0 ya da negatifse `nullptr`). `main`, dönen adresi kullanarak en büyük sayıyı ve dizideki indeksini yazdırsın.

```
Sayıları gir: 8 -3 15 0 7
En büyük: 15 (indeks 2)
```

10 Klavyeden 3 tamsayı okuyup **küçükten büyüğe** sıralayan `ucluSirala` programını yaz. Sıralama `void ucluSirala(int& a, int& b, int& c)` fonksiyonunda, iki değişkeni yer değiştiren ayrı bir `void takas(int& a, int& b)` fonksiyonu kullanılarak yapılsın. Fonksiyonlar sonucu `main`'deki değişkenlerde bırakmalıdır (`return` ile sonuç döndürme yok).

```
Üç sayı gir: 9 4 6
Sıralı: 4 6 9
```

Çözümler

Cevap anahtarı

1 C	2 A	3 E	4 B	5 D

1 Cevap: C

`*` tipe değil, hemen sağındaki ada aittir: `p` işaretçi, `q` sıradan `int` olur. A, `int *p, *q;` yazılsaydı doğru olurdu. B tersini söylüyor. D yanlış, `q` işaretçi bile değildir. E yanlış, tanım geçerlidir.

2 Cevap: A

`p = a + 2` olduğundan `p`, `a[2]`'yi (30) gösterir. `*(p - 1)` = `a[1]` = 20; `p[1]` = `*(p + 1)` = `a[3]` = 40. Toplam 60. B (30 + 40), `-1`'i unutup ilk terimi `*p` sanınca çıkar. C (20 + 30), `p[1]`'i `p[0]` sanınca çıkar. D (40 + 40), ilk terimi `*(p + 1)` sanınca çıkar. E (50 + 40), ilk terimi `*(p + 2)` sanınca çıkar.

3 Cevap: E

`int* const q`: işaretçinin kendisi sabittir, yani `q = &b;` yazılamaz (E derlenmez). A olur, çünkü `p` gösterilen değeri sabitler ama kendisi sabit değildir. B olur: `q` sabit işaretçidir ama gösterdiği `int` sabit değildir. C olur, `a` sıradan değişken. D yalnız okuyor. (`*p = 5;` yazılsaydı o da derlenmezdi, ama seçeneklerde yok.)

4 Cevap: B

Dizi fonksiyona geçerken ilk elemanın adresine dönüşür; parametredeki `int a[]` aslında `int* a`'dır. Bu yüzden `sizeof(a)` işaretçi boyutudur (64 bitte 8). A yanlış, kopya yapılmaz, değişiklik `dizi`'yi etkiler. C yanlış, ikisi tamamen aynıdır. D yanlış, fonksiyon içinde 8 / 4 = 2 gibi yanlış bir sayı verir. E yanlış, uzunluk ayrıca parametre olarak geçirilmelidir.

5 Cevap: D

Referans bağlandığı değişkenin ikinci adıdır ve sonradan başka değişkene bağlanamaz. `r = y;` bir bağlama değil, **atamadır**: `y`'nin değeri (9) `r` üzerinden `x`'e yazılır. A bunu bağlama sanıyor, yanlış. B yanlış, referans boş olamaz; `nullptr` olabilen işaretçidir. C yanlış, satır derlenir ve çalışır. E yanlış, `sizeof(r)` bağlandığı değişkenin boyutunu verir (`int` için 4).

6

Önce `p` dizinin başını gösterir. Her satırdan sonra `p`'nin yerini ve dizinin içeriğini yaz:

Satır	p	Dizi	Açıklama
`int x{*p++};`	`a[1]`	3 6 9 12	`x` = 3 (eski `*p`), sonra `p` ilerler
`int y{(*p)++};`	`a[1]`	3 **7** 9 12	`y` = 6 (artmadan önceki değer), `a[1]` artar, `p` yerinde
`int z{*++p};`	`a[2]`	3 7 9 12	Önce `p` ilerler, sonra `z` = `a[2]` = 9

Son `cout` `p`'nin gösterdiği yeri, yani `a[2]` = 9'u yazar.

```
Çıktı
3 6 9
3 7 9 12
9
```

7

`artir`'ın ikinci parametresi **değer ile**, birincisi **referansla** geçer: `b` kopyadır, `a` çağıran taraftaki değişkendir.

1. `artir(x, y);` → `b` = 6, `x` = 2 + 6 = 8. `y` = 3.
2. `artir(y, x);` → `b` = 8 * 2 = 16, `y` = 3 + 16 = 19. `x` = 8.
3. `r` , `x`'in ikinci adı; `p` , `y`'yi gösterir. `r = r * 2;` → `x` = 16. `*p = *p + r;` → `y` = 19 + 16 = 35. İlk satır: `16 35` .
4. `r = y;` bağlama değil atamadır: `x` = 35. `p = &x;` ile `p` artık `x`'i gösterir, `*p += 1;` → `x` = 36. `r` ve `*p` de `x` olduğu için 36.

```
Çıktı
16 35
36 35 36 36
```

8

`a` = {1, 3, 5, 7, 9, 11}; `p` = `&a[1]` , `q` = `&a[5]` .

1. `q - p` = 5 - 1 = **4** (eleman sayısı).
2. `*(q - 2)` = `a[3]` = **7**.
3. `*(p + 1) = *p + *(q - 1);` → `a[2]` = `a[1]` + `a[4]` = 3 + 9 = 12. Dizi: 1 3 12 7 9 11.
4. `p += 2;` → `p` = `&a[3]` . `*p` = 7, `p[-1]` = `a[2]` = 12. İşaretçiye negatif indeks verilebilir.
5. `p == a + 3` doğru olduğu için `bool` değeri **1** yazılır.

```
Çıktı
4
7
7 12
1
```

9

Fonksiyon ilk elemanı aday seçer, kalanını gezen işaretçiyle tarar ve daha büyük bulduğu yerin adresini saklar. `adet` 0 ise gösterecek eleman yoktur, bu yüzden `nullptr` döndürür; `main` kullanmadan önce kontrol eder. İndeks için iki işaretçi çıkarılır: `p - sayilar` eleman sayısını verir.

enBuyuk.cpp

```cpp
#include <iostream>

using namespace std;

int* enBuyuguBul(int* a, int adet) {
    if (adet <= 0) {
        return nullptr;
    }
    int* enBuyuk{a};
    for (int* p{a + 1}; p < a + adet; p++) {
        if (*p > *enBuyuk) {
```

```
            enBuyuk = p;
        }
    }
    return enBuyuk;
}

int main() {
    int sayilar[5] = {0, 0, 0, 0, 0};

    cout << "Sayıları gir: ";
    for (int i = 0; i < 5; i++) {
        cin >> sayilar[i];
    }
    int* p{enBuyuguBul(sayilar, 5)};
    if (p != nullptr) {
        cout << "En büyük: " << *p << " (indeks " << (p - sayilar) << ")\n";
    }
    return 0;
}
```

Çıktı

```
Sayıları gir: 8 -3 15 0 7
En büyük: 15 (indeks 2)
```

10

`takas` iki referans alır ve değerlerini değiştirir. `ucluSirala` üç adımda sıralar: önce ilk ikiyi, sonra son ikiyi, sonra yine ilk ikiyi karşılaştırıp gerekirse takas eder. Parametreler referans olduğu için `main`'deki `x`, `y`, `z` doğrudan değişir; `return` gerekmez.

ucluSirala.cpp

```
#include <iostream>

using namespace std;

void takas(int& a, int& b) {
    int gecici{a};
    a = b;
    b = gecici;
}

void ucluSirala(int& a, int& b, int& c) {
    if (a > b) {
        takas(a, b);
    }
    if (b > c) {
        takas(b, c);
    }
    if (a > b) {
        takas(a, b);
    }
}

int main() {
    int x{0};
    int y{0};
    int z{0};

    cout << "Üç sayı gir: ";
    cin >> x >> y >> z;
```

```
    ucluSirala(x, y, z);
    cout << "Sıralı: " << x << " " << y << " " << z << '\n';
    return 0;
}
```

Çıktı

```
Üç sayı gir: 9 4 6
Sıralı: 4 6 9
```

BÖLÜM 10

Dinamik Bellek ve Akıllı İşaretçiler

Bu Bölümde Öğreneceklerin

- Programın belleği hangi bölgelerde kullandığını: yığın (stack) ve öbek (heap)
- Boyutu çalışma anında belli olan veriler için bellek nasıl istenir ve geri verilir
- Belleği geri vermeyi unutunca, iki kez geri verince ya da verdikten sonra kullanınca neler olduğunu
- `vector`'ün büyürken arka planda ne yaptığını
- RAII fikrini: belleği "sahibi olan" bir nesneye emanet etmeyi
- Belleği kendiliğinden bırakan akıllı işaretçileri: tek sahipli ve paylaşımlı
- Hangi durumda hangi aracın seçileceğini ve modern C++'ta neden çıplak `new` yazılmadığını

10.1 Neden dinamik bellek?

Bölüm 7'de dizinin boyutunun programı yazarken belli olması gerektiğini gördün. Bir derste kaç öğrenci olacağını bilmiyorsan `int notlar[100];` yazmak bir kumardır: 101. öğrenci sığmaz, 10 öğrencili sınıfta ise 90 yerlik bellek boşa gider. Çözüm, boyutu **program çalışırken** öğrenip o kadar yeri o anda istemektir. Buna **dinamik bellek ayırma** (dynamic memory allocation) denir.

`vector`'ün büyüyebildiğini de biliyorsun (Bölüm 7). Bu bir sihir değil: `vector`, içeride bu bölümde öğreneceğin araçlarla çalışır. Ne yaptığını bilmek hem sınav hem de doğru kod yazmak için önemlidir.

Çalışan bir program belleği üç ana bölgede kullanır:

Bölge	İçinde ne var?	Ömrü	Kim yönetir?	Boyut
Statik alan	global ve `static` değişkenler, string sabitleri	program boyunca	derleyici	derleme anında sabit
Yığın (stack)	yerel değişkenler, parametreler	kapsam (`{ }`) bitene kadar	otomatik	küçük (genelde 1–8 MB)
Öbek (heap)	`new` ile istediğin bloklar	sen geri verene kadar	**sen**	büyük (RAM'in elverdiği kadar)

Yerel değişkenlerin çağrı yığınında yaşadığını ve fonksiyon bitince yok olduğunu Bölüm 6'dan biliyorsun. Öbek üçüncü ve **elle yönetilen** bölgedir: yeri sen istersin, işin bitince sen geri verirsin.

Not: Yığın ve öbek karışmasın

Türkçe kaynakların çoğu hem *stack*'e hem *heap*'e "yığın" der. Bu kitapta *stack* = **yığın**, *heap* = **öbek**. Sınavda karışıklık olmasın diye parantez içinde İngilizcesini de yaz.

Yığın hızlı ama küçüktür. İçine çok büyük bir yerel dizi koyarsan (`int dev[10000000];` gibi, yaklaşık 40 MB) yığın dolar ve program daha çalışmaya başlarken çöker; buna **yığın taşması** (stack overflow) denir. Büyük ve boyutu değişken veri öbeğe aittir.

Sık Yapılan Hata: Değişken boyutlu dizi: `int a[n];`

```
int n{0};
cin >> n;
int a[n];              // C++ standardında YOK; g++ bir uzantı olarak kabul eder
```

Standart C++'ta dizi boyutu **derleme anında sabit** olmak zorundadır. g++ yukarıdaki satırı (C'den kalma bir uzantı olarak) kabul eder, ama `-Wpedantic` uyarı verir; Visual Studio'nun derleyicisi hiç kabul etmez. Üstelik böyle bir dizi yığında yer alır, `n` büyükse program çöker ve bunu yakalamanın yolu yoktur. Boyutu çalışma anında belli olan veri için bu bölümün araçlarını (ya da basitçe `vector`) kullan.

Bir işaretçi ile öbekteki blok arasındaki ilişkiyi çizelim:

```
   YIĞIN (stack)                        ÖBEK (heap)
+----------------+                  +----+----+----+----+----+
| n = 5          |                  | 10 | 20 | 30 | 40 | 50 |
| p = (adres) ---+----------------> +----+----+----+----+----+
+----------------+                   new int[5] ile alındı
main'in yerel                        delete[] p yazılana kadar yaşar
değişkenleri
```

`p` yığındadır; kapsam bitince yok olur. Gösterdiği blok ise **öbektedir** ve `p` yok olsa bile durmaya devam eder. Geri vermeden `p`'yi kaybedersen bloğa bir daha ulaşamazsın; buna **bellek sızıntısı** (memory leak) denir (aşağıda).

10.2 new ve delete

```
new Tip{ilkDeğer}       // öbekten bir Tip nesnesi ister, ADRESİNİ döndürür
delete işaretçi;        // o bloğu öbeğe geri verir
```

`new`, öbekten istediğin tipte bir nesne için yer ayırır, ilk değerini verir ve bloğun **adresini** döndürür. Adresi bir işaretçide (Bölüm 9) saklarsın. İşin bitince `delete` ile bloğu geri verirsin.

newTemel.cpp

```
#include <iostream>

using namespace std;

int main() {
    int yigindaki{10};              // yığında
    int* obekteki{new int{20}};     // öbekte; işaretçinin kendisi yığında

    cout << "yigindaki: " << yigindaki << '\n';
    cout << "obekteki : " << *obekteki << '\n';
    *obekteki += 5;                 // dolaylama ile değeri değiştir
    cout << "obekteki : " << *obekteki << '\n';

    delete obekteki;                // bloğu geri ver
    obekteki = nullptr;             // sarkan işaretçiyi temizle
    if (obekteki == nullptr) {
        cout << "obekteki artik bos\n";
    }
    return 0;
}
```

Çıktı

```
yigindaki: 10
obekteki : 20
obekteki : 25
obekteki artik bos
```

Satır satır:

- `new int{20}`: öbekte bir `int` için yer açar, içine 20 koyar, adresini döndürür. Dönen adres `int*` tipindedir; `malloc`'taki gibi cast ya da bayt hesabı yoktur, `new` tipi bilir.
- `*obekteki`: Bölüm 9'daki dolaylama; bloğun içindeki değere ulaşırsın.
- `delete obekteki;`: bloğu geri verir. **Her `new` için tam bir `delete`.**
- `obekteki = nullptr;`: `delete` işaretçinin içindeki adresi değiştirmez. Blok artık senin değildir ama adres hâlâ oradadır; işaretçiyi `nullptr` yaparsan yanlışlıkla kullanıldığında hata genelde hemen çöküşle belli olur. Eski adres ise sessizce yanlış veri okumana yol açabilirdi.

İlk değer vermenin üç hâli:

```
int* a{new int};        // ilk değersiz: içinde çöp değer var, okuma
int* b{new int{}};      // boş süslü parantez: 0
int* c{new int{7}};     // 7
```

`delete nullptr;` güvenlidir ve hiçbir şey yapmaz. Bir blok yalnızca bir kez, `new` **ile aldığın adresle** geri verilir.

> **Not: C'den geliyorsan: malloc ve free**
>
> C'de bellek `malloc` ve `free` ile alınırdı. C++'ta bunlar hâlâ vardır, ama **nesneler için kullanılmaz**: `malloc` yalnızca ham bayt verir, `new` ise nesnenin yapıcısını da çağırır. Şunu aklında tut: `new` ile alınanı `delete` ile, `malloc` ile alınanı `free` ile bırakırsın; karıştırmak tanımsız davranıştır. Bu kitapta `malloc` kullanmıyoruz. Bir fark daha var: `malloc` yer bulamazsa `NULL` döndürür, `new` ise yer bulamazsa `nullptr` **döndürmez**, bir **istisna** (exception) fırlatır (`std::bad_alloc`; Bölüm 14'te göreceksin). Yani `new`'dan sonra `if (p == nullptr)` kontrolü gereksizdir.

delete yazmazsan ne olur?

Bellek sızıntısını gözle görmek için küçük bir deney yapalım. Bir **nesne** ancak yok edilirken arkasını toplar (dosya kapatır, bağlantıyı keser, belleğini geri verir). Aşağıdaki `Kaynak`, doğarken ve yok olurken ekrana yazı yazar; böylece ne zaman temizlendiğini görürüz.

`struct`, birkaç değişkeni tek bir tipte toplar: her `Kaynak` nesnesinin içinde `no` adlı bir `int` vardır. Bir nesnenin üyesine `.` ile (`k.no`), nesneyi gösteren işaretçiden ise `->` ile (`k2->no`) ulaşırsın. `struct`'ı ve sınıfları Bölüm 11'de ayrıntılı göreceksin; şimdilik bu kadarı yeter. `struct Kaynak` içindeki `Kaynak(int n)` satırı nesne **doğarken**, `~Kaynak()` satırı nesne **yok edilirken** otomatik çalışan bloktur (yapıcı ve yıkıcı; ayrıntısı Bölüm 11'de). Şimdilik kalıp olarak al: "nesne yok olurken çalışan satırlar".

kaynakSizinti.cpp

```cpp
#include <iostream>

using namespace std;

struct Kaynak {
    int no;
    Kaynak(int n) { no = n; cout << "Kaynak " << no << " olustu\n"; }
    ~Kaynak() { cout << "Kaynak " << no << " yok edildi\n"; }
};

int main() {
    Kaynak* k1{new Kaynak{1}};
    Kaynak* k2{new Kaynak{2}};
    cout << "ikisi de kullanildi, k2->no = " << k2->no << '\n';

    delete k1;                      // k1 geri verildi
    cout << "main bitiyor\n";       // k2 için delete YOK
    return 0;
}
```

Çıktı

```
Kaynak 1 olustu
Kaynak 2 olustu
ikisi de kullanildi, k2->no = 2
Kaynak 1 yok edildi
main bitiyor
```

`k1` için `delete` yazdık: “yok edildi” satırı çıktı. `k2` için yazmadık: satır **hiç çıkmadı**, yani `k2`’nin temizlik kodu hiç çalışmadı. Program bitince işletim sistemi öbeği toplar, yani ekranda bir şey bozulmaz. Ama `Kaynak` bir dosya tutsaydı, yıkıcısı hiç çalışmadığı için dosyadaki bekleyen veriler yazılmadan kalabilirdi. Sunucu gibi günlerce çalışan programlarda unutulan her `delete`, belleği biraz biraz tüketir. (`k2->no`, `(*k2).no` ile aynıdır: işaretçiden üyeye ulaşmanın kısa yazılışı.)

10.3 new[] ve delete[]: dinamik dizi

Birden çok elemanı bitişik tutmak için `new Tip[adet]` yazılır. Eleman sayısı **değişken** olabilir; asıl iş budur:

```cpp
Tip* p{new Tip[adet]};    // adet elemanlık dizi, adresi ilk elemanın adresi
delete[] p;               // DİZİ için delete[] (köşeli parantez şart)
```

Aşağıdaki program önce kaç not gireceğini sorar, sonra tam o kadar yer ayırır:

dinamikOrtalama.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    int n{0};

    cout << "Kac not girilecek? ";
    if (!(cin >> n) || n <= 0) {
        cout << "Gecersiz sayi.\n";
        return 1;
    }
    int* notlar{new int[n]{}};      // n elemanlık dizi, hepsi 0
```

```
    cout << "Notlari gir: ";
    int toplam{0};
    for (int i{0}; i < n; i++) {
        cin >> notlar[i];          // sıradan dizi gibi kullan
        toplam += notlar[i];
    }
    cout << "Ortalama: " << static_cast<double>(toplam) / n << '\n';
    delete[] notlar;               // dizi: köşeli parantezli delete
    return 0;
}
```

Çıktı

```
Kac not girilecek? 5
Notlari gir: 72 85 90 60 95
Ortalama: 80.4
```

Dikkat edilecekler:

- `n`'yi okuduktan **sonra** ayırdık; önce kontrol (`n <= 0`), sonra `new`.
- `new int[n]{}`: sondaki boş süslü parantez bütün elemanları 0 yapar. Yazmazsan elemanlar çöp olur. `new int[4]{3, 1}` yazarsan ilk ikisi 3 ve 1, kalanlar 0 olur.
- `notlar[i]` ve `*(notlar + i)` aynı şeydir (Bölüm 9). Ama `notlar` bir **işaretçidir**, dizi değil: `sizeof(notlar)` eleman sayısını değil, işaretçinin boyutunu (64 bitlik sistemde 8 bayt) verir. Eleman sayısını ayrı bir değişkende (`n`) tutmak senin işindir; `vector` bunu kendisi yapar.
- `delete[]` yerine `delete` yazmak yanlıştır (aşağıda).

Sık Yapılan Hata: new[] ile alıp delete ile bırakmak

```
int* p{new int[10]};
delete p;              // YANLIŞ: new[] ile alınan dizi delete[] ile bırakılır
```

`new[]` ile alınan bloğu `delete` ile (köşeli parantezsiz) bırakmak **tanımsız davranıştır**: program çökebilir, öbek bozulabilir, ya da hiçbir şey olmamış gibi görünebilir. Derleyici her zaman uyarmaz; `g++ -Wall` ile bu basit örnekte uyarır, ama her durumda uyarıya güvenilemez. Kural eşleşmedir: `new` ↔ `delete`, `new[]` ↔ `delete[]`. Ters eşleşme (tek nesneyi `delete[]` ile bırakmak) da hatadır.

10.4 Bellek hataları

Dinamik bellek hataları genellikle **derleyiciyi geçer**. Program çalışır, çoğu zaman doğru sonuç da verir; hata günler sonra, başka bir yerde çöküş olarak ortaya çıkar. Aşağıdaki kodlar bilerek hatalıdır; çalıştırmıyoruz, çünkü sonuçları tanımsızdır ya da hiçbir belirti vermez.

1. Bellek sızıntısı (memory leak)

```
void sizinti() {
    int* p{new int[100]};
    p[0] = 1;
}                      // delete[] yok: p yok olur, 400 baytlık blok öbekte kalır
```

Bloğun adresini tutan son işaretçi kaybolunca blok program bitene kadar kullanılamaz kalır. Klasik yollar:

- `delete` yazmayı unutmak (yukarıdaki).
- Adresi kaybetmek: `p = new int{1}; p = new int{2};` İlk bloğun adresi `p`'den silindi.

- Erken çıkış: bir `if` içinde `return` edip önceden alınan blokları geri vermeyi unutmak.
- (Bölüm 14) İstisna fırlatılınca `delete` satırına hiç ulaşılmamak.

2. delete sonrası kullanım (sarkan işaretçi)

```
int* p{new int{7}};
delete p;
cout << *p << '\n';     // TANIMSIZ: p artık bizim olmayan bir yeri gösteriyor
```

`delete` sonrasında `p`'nin içindeki adres değişmez ama blok artık sana ait değildir; başka bir `new` o yeri başkasına verebilir. `*p` eski değeri de verebilir, çöp de, program da çökebilir. Böyle işaretçiye **sarkan işaretçi** (dangling pointer) denir; Bölüm 9'da yerel değişkenin adresini döndürünce de aynısını görmüştün.

3. Çift delete

```
int* p{new int{7}};
int* q{p};              // ikinci işaretçi aynı bloğu gösteriyor
delete p;
delete q;               // TANIMSIZ: aynı bloğu ikinci kez bırakmak
```

Öbeğin iç düzeni bozulur; çoğu sistemde program çöker, ama garantisi yoktur. Gizli tehlike: `p = nullptr;` yazsan bile `q` hâlâ sarkandır. `nullptr` alışkanlığı yalnızca **o işaretçiyi** korur.

4. Yanlış adresi delete etmek ve blok sınırını aşmak

Yığındaki bir değişkenin adresini (`delete &x;`) ya da bloğun ortasındaki bir adresi (`delete (p + 1);`) `delete` etmek, ve `new int[5]` bloğuna `p[5] = 1;` yazmak (geçerli indeksler 0..4) **tanımsız davranıştır**. Sınır aşımı komşu bloğun bilgilerini bozduğu için hata çoğu kez çok sonra, bir `delete`'te patlar.

> **Sınavda Çıkar**
>
> Bellek hatası soruları genelde **kod okuma** olarak gelir ve çalıştırılmaz: "Kaç blok sızar?", "`delete` sonrasında `*p` yazmak ne doğurur?" Çözüm yöntemi: Her `new` için "bu bloğun adresini şu an hangi işaretçiler tutuyor?" diye bak. Blok geri verilmeden önce son işaretçi başka adrese atanırsa ya da yok olursa **sızıntı** vardır. Geri verildikten sonra o bloğa erişim **sarkan işaretçidir**. İki işaretçi aynı bloğu gösteriyorsa blok **bir kez** geri verilir; ikinci `delete` çift silmedir.

> **İpucu: Bellek hatalarını makineye buldur**
>
> Bu hatalar gözle bulunması zor hatalardır; araçlar yardım eder. Programı hata ayıklama bilgisiyle derle: `g++ -std=c++17 -g -fsanitize=address program.cpp -o program` (AddressSanitizer: taşma, `delete` sonrası kullanım ve çift `delete`'i çalışma anında yakalar, satır numarasıyla bildirir). Linux'ta sızıntıları da raporlar. Windows'ta MinGW g++ çoğu kurulumda AddressSanitizer içermez; orada Dr. Memory kullanabilirsin. Linux'taki klasik araç `valgrind --leak-check=full ./program`'dır; Apple işlemcili Mac'lerde Valgrind çalışmaz, `leaks --atExit -- ./program` kullanılır. Bu araçlar sınavda sorulmaz ama laboratuvarda ve ödevde çok işine yarar.

10.5 Elle büyüyen dizi: vector içeride ne yapar?

Eleman sayısını baştan hiç bilmiyorsan (örneğin "0 girilene kadar sayı oku") diziyi doldukça büyütmen gerekir. C'deki `realloc` gibi bir "yerinde büyüt" aracı C++'ta `new[]` ile **yoktur**. Büyütmek şu 4 adımdır:

1. Daha büyük yeni bir dizi aç (`new`).
2. Eski elemanları yeni diziye kopyala.
3. Eski diziyi geri ver (`delete[]`).
4. İşaretçiyi yeni diziye çevir.

Her dolduğunda yalnızca 1 yer daha açarsan n eleman için toplam yaklaşık $n^2/2$ kopyalama yapılır. Bunun yerine **kapasite** (capacity) tutarsın: `kapasite` kadar yer ayırırsın, `adet` kadarını kullanırsın. `adet` kapasiteye ulaşınca kapasiteyi **iki katına** çıkarırsın; böylece büyütme sayısı yaklaşık $\log_2(n)$ olur. Aşağıdaki fonksiyon dolduğunda büyütür:

> **Not: İşaretçiye referans: int*&**
>
> `buyut` fonksiyonu, çağıranın `sayilar` işaretçisini **değiştirmek** zorundadır (yeni bloğu göstersin diye). Bölüm 9'daki gibi bir değişkeni fonksiyonda değiştirmek için referans ile geçirmek gerekir; değişken bir `int*` olduğu için parametre `int*&` (işaretçiye referans) olur. `kapasite` için de aynısı geçerlidir: `int&`. `adet` yalnızca okunduğundan değer ile geçer.

buyuyenDizi.cpp

```cpp
#include <iostream>

using namespace std;

// dizi dolunca iki kat büyük yeni dizi açar, elemanları taşır, eskisini bırakır
void buyut(int*& dizi, int& kapasite, int adet) {
    int yeniKapasite{kapasite * 2};
    int* yeni{new int[yeniKapasite]{}};

    for (int i{0}; i < adet; i++) {
        yeni[i] = dizi[i];          // eski elemanları kopyala
    }
    delete[] dizi;                  // eski bloğu geri ver
    dizi = yeni;                    // çağıranın işaretçisi yeni bloğu göstersin
    cout << "[kapasite " << kapasite << " -> " << yeniKapasite << "]\n";
    kapasite = yeniKapasite;
}

int main() {
    int kapasite{2};                // başlangıçta 2 elemanlık yer
    int adet{0};                    // şu an kaç eleman var
    int* sayilar{new int[kapasite]{}};
    int x{0};

    cout << "Sayilari gir (0 ile biter): ";
    while (cin >> x && x != 0) {
        if (adet == kapasite) {     // dizi dolu: büyüt
            buyut(sayilar, kapasite, adet);
        }
        sayilar[adet] = x;
        adet++;
    }
    int toplam{0};
    for (int i{0}; i < adet; i++) {
        toplam += sayilar[i];
    }
```

```
    cout << "Adet: " << adet << ", kapasite: " << kapasite;
    cout << ", toplam: " << toplam << '\n';
    delete[] sayilar;
    return 0;
}
```

Çıktı

```
Sayilari gir (0 ile biter): 4 8 15 16 23 42 0
[kapasite 2 -> 4]
[kapasite 4 -> 8]
Adet: 6, kapasite: 8, toplam: 108
```

İzleyelim: Kapasite 2 iken 4 ve 8 sığar. 15 gelince `adet == kapasite` olur, kapasite 4'e çıkar; 15 ve 16 yerleşir. 23 gelince kapasite 8'e çıkar; 23 ve 42 de sığar. `0` gelince okuma biter. Sonuçta 6 eleman, 8 kapasite vardır (`4+8+15+16+23+42 = 108`). "Eklemeden önce **dolu mu** diye bak" dinamik dizinin temel kalıbıdır.

`vector` tam olarak bunu yapar. `size()` = bizim `adet`, `capacity()` = bizim `kapasite`; `push_back` dolu olduğunu görünce yeni bloğu açıp elemanları taşıyan kısım, bizim `buyut`'umuzun yaptığı işin aynısıdır. `vector` bunu bizim yerimize, hatasız ve istisna güvenli yapar. Kapasitenin nasıl büyüdüğüne bakalım:

vectorBuyume.cpp

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    vector<int> v;
    size_t eskiKapasite{v.capacity()};

    cout << "baslangic: size " << v.size() << ", capacity " << eskiKapasite
         << '\n';
    for (int i{1}; i <= 20; i++) {
        v.push_back(i);
        if (v.capacity() != eskiKapasite) {     // kapasite değişti: büyüme oldu
            eskiKapasite = v.capacity();
            cout << "eleman " << v.size() << ": capacity " << eskiKapasite
                 << '\n';
        }
    }
    return 0;
}
```

Çıktı

```
baslangic: size 0, capacity 0
eleman 1: capacity 1
eleman 2: capacity 2
eleman 3: capacity 4
eleman 5: capacity 8
eleman 9: capacity 16
eleman 17: capacity 32
```

g++ (ve MinGW) `vector`'ü her dolduğunda kapasiteyi iki katına çıkarır: 1, 2, 4, 8, 16, 32. **Senin bilgisayarında farklı olabilir:** standart büyüme çarpanını belirlemez, Visual Studio'nun derleyicisi yaklaşık 1,5 kat büyütür. Sınavda "iki katına çıkar" demek yeterlidir, ama kesin sayıyı sormazlar. `v.reserve(100)` ise sayıyı önceden bildiğinde kapasiteyi baştan 100 yapar ve sonraki eklemelerde yeniden ayırma olmaz.

Büyütmenin bir bedeli de var: **eski blok geri verilir**. Eski bloğun içindeki bir elemanın adresini tutuyorsan işaretçin sarkan olur:

```
vector<int> v{1, 2, 3};
int* ilk{&v[0]};
v.push_back(4);         // kapasite doldu: vector yeni bloğa taşındı, eski blok bırakıldı
cout << *ilk << '\n';   // TANIMSIZ: ilk artık geri verilmiş bloğu gösteriyor
```

`vector`'e eleman eklerken kapasite aşılıp yeni blok açıldığında, eskiden aldığın işaretçileri ve referansları **geçersiz say** (iteratörler için de aynısı; Bölüm 15). Bunu bir kenara yaz: sınavda çok sevilen bir tuzaktır.

Elle yönettiğimiz bu dizinin zayıf yanlarını sayalım: `delete[]` yazmayı unutabiliriz; `buyut` içinde bir şey ters giderse (ya da bir istisna fırlarsa) eski blok sızabilir; diziyi ve `adet`, `kapasite` değişkenlerini hep birlikte taşımamız gerekir. Bunları tek bir yere toplayan şey **sınıftır** (Bölüm 11). Elle büyüyen dizi, `vector`'ün nasıl çalıştığını anlamak içindi; gerçek programda hazır `vector` kullanırsın. Peki bellek yönetimini hatasız yapan genel fikir nedir? RAII.

10.6 RAII: belleği bir nesneye emanet et

Bir `vector` ya da `string` kullanırken hiç `delete` yazmadın, ama bellek sızmadı. Neden? Çünkü bu nesneler **yok edilirken kendi belleklerini kendileri geri verir**. Yerel bir nesne, içinde bulunduğu `{ }` bloğunun sonunda otomatik olarak yok edilir. Aşağıdaki programda `Kaynak`'ı (sızıntı deneyindeki yapı) bu kez `new` olmadan, sıradan yerel değişken olarak kullanıyoruz:

kapsamSonu.cpp

```
#include <iostream>

using namespace std;

struct Kaynak {
    int no;
    Kaynak(int n) { no = n; cout << "Kaynak " << no << " olustu\n"; }
    ~Kaynak() { cout << "Kaynak " << no << " yok edildi\n"; }
};

int main() {
    Kaynak a{1};
    {
        Kaynak b{2};
        Kaynak c{3};
        cout << "ic blok bitiyor\n";
    }
    cout << "dis blok bitiyor\n";
    return 0;
}
```

Çıktı

```
Kaynak 1 olustu
Kaynak 2 olustu
Kaynak 3 olustu
ic blok bitiyor
Kaynak 3 yok edildi
Kaynak 2 yok edildi
dis blok bitiyor
Kaynak 1 yok edildi
```

Gördüğün gibi hiç `delete` yazmadık, ama temizlik kendiliğinden yapıldı ve **ters sırada**: en son oluşan (`c`) ilk yok edildi. Bu yöntemin adı **RAII**'dir (Resource Acquisition Is Initialization, "kaynak edinimi ilk değer vermedir"): kaynağı (bellek, dosya, bağlantı) bir nesnenin içinde edin, nesne yok olunca kaynak da kendiliğinden bırakılsın. Böylece geri vermeyi hatırlamak zorunda kalmazsın; erken `return` ya da istisna olsa bile nesne kapsam bitince yok edilir. C++'ın en önemli fikirlerinden biridir.

Sorun şu: `Kaynak* k{new Kaynak{1}};` yazdığında `k` yalnızca bir **işaretçidir** (yığında bir adres). Bu işaretçinin yok olması, gösterdiği öbek bloğunu bırakmaz; ham işaretçi RAII uygulamaz. Çözüm, işaretçiyi RAII yapan bir nesneye sarmaktır. İşte **akıllı işaretçi** (smart pointer) budur: içinde bir adres tutan, yok edilirken o adresi kendisi `delete` eden nesne.

10.7 unique_ptr: tek sahipli akıllı işaretçi

`unique_ptr`, `<memory>` başlığındadır. **Tek sahipli** (unique) bir işaretçidir: gösterdiği bloğun tek sahibi odur ve yok edilirken (kapsam bitince) bloğu kendisi `delete` eder. Oluşturmanın modern yolu `make_unique`'dir; `new` yazmadan bloğu açar ve sahibini kurar:

```
unique_ptr<Tip> p{make_unique<Tip>(yapıcı argümanları)};
auto p{make_unique<Tip>(...)};          // aynısı, kısa yazım
```

Kullanımı sıradan işaretçiye benzer: `*p` ile dolaylama, `p->üye` ile üye erişimi:

uniqueTemel.cpp

```
#include <iostream>
#include <memory>
#include <string>

using namespace std;

int main() {
    unique_ptr<int> p{make_unique<int>(42)};    // öbekte bir int, ilk değeri 42
    cout << *p << '\n';
    *p += 1;
    cout << *p << '\n';

    auto ad{make_unique<string>("Merhaba")};    // öbekte bir string
    cout << *ad << " (" << ad->size() << " karakter)\n";

    p.reset();                                  // bloğu şimdi geri ver
    if (p == nullptr) {
        cout << "p artik bos\n";
    }
    return 0;                                   // ad burada yok edilir: delete yok!
}
```

Çıktı

```
42
43
Merhaba (7 karakter)
p artik bos
```

Burada hiçbir yerde `delete` yazmadık. `ad` kapsamın sonunda yok edilirken öbekteki `string`'i de geri verir. `p.reset()` bloğu hemen geri verir ve `p`'yi boşaltır (`nullptr` yapar); bir `unique_ptr`'nin dolu mu boş mu olduğunu `if (p)` ile de sorabilirsin.

Sahipliğin taşınması (move)

"Tek sahipli" demek, bir bloğun aynı anda yalnız **bir** `unique_ptr` tarafından tutulması demektir. Bu yüzden `unique_ptr` **kopyalanamaz**:

uniqueKopya.cpp

```cpp
#include <iostream>
#include <memory>

using namespace std;

int main() {
    unique_ptr<int> p{make_unique<int>(5)};
    unique_ptr<int> q{p};           // kopya: iki sahip olurdu
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
uniqueKopya.cpp: In function 'int main()':
uniqueKopya.cpp:8:24: error: use of deleted function 'std::unique_ptr<_Tp, _Dp>::unique_ptr(const
std::unique_ptr<_Tp, _Dp>&) [with _Tp = int; _Dp = std::default_delete<int>]'
    8 |     unique_ptr<int> q{p};           // kopya: iki sahip olurdu
      |                        ^
In file included from /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/memory:80,
                 from uniqueKopya.cpp:2:
/opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/bits/unique_ptr.h:543:7: note: declared here
  543 |       unique_ptr(const unique_ptr&) = delete;
      |       ^~~~~~~~~~
uniqueKopya.cpp:8:24: note: use '-fdiagnostics-all-candidates' to display considered candidates
    8 |     unique_ptr<int> q{p};           // kopya: iki sahip olurdu
      |                        ^
```

Hata mesajının özü: `use of deleted function` (silinmiş fonksiyon kullanıldı), yani kopyalama bilerek yasaklanmıştır. Bu iyi bir şeydir: kopya mümkün olsaydı iki sahip aynı bloğu ikişer kez `delete` ederdi (çift silme).

> **Not: Apple clang farklı mesaj verir**
>
> macOS'ta `g++` yazınca çalışan program aslında Apple clang'dir. Aynı kodu clang da reddeder, ama mesaj farklıdır (`call to implicitly-deleted copy constructor ...`). Kitaptaki çıktı gerçek g++ içindir (Windows'ta MinGW da böyledir). Hata yine de aynı yerdedir: kopya satırı.

Sahipliği başka birine **devretmek** istiyorsan `std::move` kullanırsın (`<utility>`, çoğu başlıkla gelir). `std::move`, "bu değeri artık kullanmayacağım, sahipliği al" demektir. `using namespace std;` yazmış olsan da bu fonksiyonu `std::` ile tam adıyla yazmak yaygın bir alışkanlıktır; macOS'taki `g++` (Apple clang) yalın `move(a)` için `unqualified call to 'std::move'` uyarısı verir. Devirden sonra eski işaretçi **boş** (`nullptr`) kalır, yeni işaretçi sahip olur. Sahiplik fonksiyona da devredilebilir, fonksiyondan da dönebilir:

uniqueTasima.cpp

```cpp
#include <iostream>
#include <memory>

using namespace std;

struct Kaynak {
    int no;
    Kaynak(int n) { no = n; cout << "Kaynak " << no << " olustu\n"; }
    ~Kaynak() { cout << "Kaynak " << no << " yok edildi\n"; }
};
```

```
void goster(const Kaynak& k) {          // sadece KULLANIR, sahip olmaz
    cout << "goster: Kaynak " << k.no << '\n';
}

void devral(unique_ptr<Kaynak> k) {     // sahipliği ALIR
    cout << "devral basladi: " << k->no << '\n';
    cout << "devral bitiyor\n";
}                                       // k burada yok olur, blok geri verilir

unique_ptr<Kaynak> uret(int no) {
    return make_unique<Kaynak>(no);     // sahiplik çağırana geçer
}

int main() {
    auto a{make_unique<Kaynak>(1)};
    unique_ptr<Kaynak> b{std::move(a)}; // sahiplik a'dan b'ye taşındı
    if (a == nullptr) {
        cout << "a artik bos\n";
    }
    goster(*b);
    devral(std::move(b));               // sahiplik fonksiyona devredildi
    if (b == nullptr) {
        cout << "devraldan sonra b bos\n";
    }
    auto c{uret(2)};
    cout << "main bitiyor, c->no = " << c->no << '\n';
    return 0;
}
```

Çıktı

```
Kaynak 1 olustu
a artik bos
goster: Kaynak 1
devral basladi: 1
devral bitiyor
Kaynak 1 yok edildi
devraldan sonra b bos
Kaynak 2 olustu
main bitiyor, c->no = 2
Kaynak 2 yok edildi
```

Çıktıyı izle:

- `std::move(a)` ile `a` boşaldı, `b` bloğun sahibi oldu; blok **bir kez** oluştu, kopyalanmadı.
- `goster(*b)` yalnızca **bakar**: nesneyi referansla alır (Bölüm 9), sahipliğe karışmaz. Bir fonksiyon nesneyi sadece kullanacaksa `unique_ptr` değil **referans** (ya da gözlemci olarak ham işaretçi) al.
- `devral(std::move(b))`: sahiplik fonksiyona geçti, fonksiyon bitince `Kaynak 1` **orada** yok edildi, `main`'e dönmeden. `b` artık boş.
- `uret(2)`: fonksiyonun yerel `unique_ptr`'si `return` ile çağırana taşınır (açıkça `move` yazmak gerekmez). C'deki "fonksiyon yeni blok döndürür, çağıran `free` eder" kuralının güvenli hâlidir: sahiplik tipin içinde yazılıdır, yorum satırında değil.
- `main` bitince `c` yok edilir ve `Kaynak 2` geri verilir.

Erken çıkış ve RAII

`unique_ptr`'nin asıl kazancı, fonksiyondan hangi yoldan çıkılırsa çıkılsın bloğun bırakılmasıdır:

```
void islemHam(int deger) {
    Kaynak* k{new Kaynak{deger}};
    if (deger < 0) {
        return;             // SIZINTI: delete satırına ulaşılmadı
    }
    delete k;
}

void islem(int deger) {
    auto k{make_unique<Kaynak>(deger)};
    if (deger < 0) {
        return;             // k burada da yok edilir: sızıntı yok
    }
}                           // k burada yok edilir
```

unique_ptr ile dinamik dizi

`unique_ptr` diziyi de taşır: tip `unique_ptr<int[]>` olur ve `make_unique<int[]>(n)` `n` elemanlık diziyi **sıfırlanmış** olarak açar. Bırakırken doğru olan `delete[]`'i kendisi çağırır, yani eşleşme hatası yapamazsın. Dizi fonksiyondan da döndürülebilir (aşağıda Soru 10):

```
auto a{make_unique<int[]>(5)};      // 5 eleman, hepsi 0
a[2] = 7;                           // sıradan dizi gibi kullan
unique_ptr<int[]> b{std::move(a)};  // sahiplik taşınır; delete[] yazılmaz
```

Dikkat: `unique_ptr<int[]>` da bir işaretçidir; **eleman sayısını bilmez** (`n`'i sen tutarsın), `size()` ya da `push_back`'i yoktur. Boyutu değişebilecek bir dizi gerekiyorsa `vector` yazmaya devam et; `make_unique<int[]>` boyutu hiç değişmeyen, öbekte duran bir blok gerektiğinde ya da C'den kalma bir arayüz için kullanılır.

● Sık Yapılan Hata: Aynı ham adresi iki akıllı işaretçiye vermek

```
int* ham{new int{5}};
unique_ptr<int> a{ham};
unique_ptr<int> b{ham};     // YANLIŞ: iki "tek" sahip, aynı blok
```

Derleyici bunu kabul eder, ama `a` ve `b` yok olurken ikisi de `delete` eder: **çift silme**. Aynı hata `delete p.get();` yazmaktır: `get()` sahiplik vermeden ham adresi verir (gözlemlemek içindir), onu `delete` etme ve başka bir akıllı işaretçiye de verme. Kural: ham adres alınırsa **hemen** bir akıllı işaretçiye ver ve bir daha ham adresi saklama; en iyisi `make_unique` kullan, o zaman ham adres hiç görünmez.

● Sınavda Çıkar

`unique_ptr` soruları dört kalıba iner: (1) **kopyalanamaz**, kopyalamaya çalışan kod derlenmez; (2) `move` ile sahiplik taşınır ve **eski işaretçi boş** olur; (3) kapsam bitince bloğun `delete` edilmesini `unique_ptr` yapar, yok edilme sırası oluşturulmanın **tersidir**; (4) fonksiyon `unique_ptr`'yi **değer ile** alıyorsa çağıran `move` yazmak zorundadır ve çağrıdan sonra elindeki boştur.

10.8 shared_ptr: paylaşımlı sahiplik (kısa)

Bazen bir nesneyi gerçekten birden çok yerin paylaşması gerekir ve "son kullanan temizlesin" istersin. `shared_ptr` bunu yapar: içeride bir **sayaç** (referans sayacı, reference count) tutar. Kopyalanınca sayaç artar, bir kopya yok olunca azalır; sayaç **0'a inince** blok geri verilir. Oluşturmanın yolu `make_shared`'dir; sayaç `use_count()` ile görülür:

paylasimliSayac.cpp

```cpp
#include <iostream>
#include <memory>

using namespace std;

struct Kaynak {
    int no;
    Kaynak(int n) { no = n; cout << "Kaynak " << no << " olustu\n"; }
    ~Kaynak() { cout << "Kaynak " << no << " yok edildi\n"; }
};

int main() {
    auto a{make_shared<Kaynak>(5)};
    cout << "sayac: " << a.use_count() << '\n';
    {
        shared_ptr<Kaynak> b{a};        // kopya: sayaç artar
        auto c{a};                      // bir kopya daha
        cout << "ic blokta sayac: " << a.use_count() << '\n';
    }                                   // b ve c yok olur: sayaç azalır
    cout << "ic bloktan sonra sayac: " << a.use_count() << '\n';
    a.reset();                          // son sahip bıraktı: blok geri verilir
    cout << "reset sonrasi\n";
    return 0;
}
```

Çıktı

```
Kaynak 5 olustu
sayac: 1
ic blokta sayac: 3
ic bloktan sonra sayac: 1
Kaynak 5 yok edildi
reset sonrasi
```

`b` ve `c` yok olunca blok **silinmedi**, çünkü `a` hâlâ sahipti (sayaç 1). `a.reset()` ile sayaç 0'a indi ve `Kaynak 5` yok edildi. `unique_ptr`'den farkı: `shared_ptr` kopyalanabilir; karşılığında sayaç tutmanın küçük bir maliyeti vardır.

`shared_ptr` bir ilaç gibidir: gerektiğinde kullanılır, her derde deva değildir. İki nesne birbirini `shared_ptr` ile tutarsa (A, B'yi; B de A'yı) sayaçlar hiçbir zaman 0'a inmez ve ikisi de **sızar**. Bu döngüyü kırmak için sayaca katılmayan `weak_ptr` vardır; ileri bir konu olduğundan bu kitapta işlemiyoruz.

10.9 Ne zaman hangisi?

Bellek gerektiğinde şu sırayla düşün; ilk uyan çözümde dur:

İhtiyaç	Seç	Neden
Bir değer, ömrü kapsamla sınırlı	sıradan yerel değişken (yığın)	en hızlı, ücretsiz temizlik
Metin	`string`	RAII, kendi büyür
Çok sayıda eleman, boyut değişebilir	`vector`	RAII, kendi büyür; **varsayılan tercih**
Kapsamı aşan tek bir nesne, tek sahip	`unique_ptr` + `make_unique`	RAII, kopya yok, maliyetsiz
Gerçekten birden çok sahip	`shared_ptr` + `make_shared`	son sahip temizler
Sadece kullanmak (sahip olmadan)	referans ya da ham işaretçi	sahipliğe karışmaz
`new` / `delete` doğrudan	neredeyse **hiç**	eski kodu okurken, sınavda, ileride kendi sınıfını yazarken

Modern C++ kuralı: çıplak `new` ve `delete` yazma. Öbeğe elle ulaşman gerekmiyorsa `vector`, `string` ya da yığındaki değişken; öbekteki bir nesne gerekiyorsa `make_unique`; paylaşılıyorsa `make_shared`. Böylece ne sızıntı, ne çift silme, ne de `delete[]` ile `delete` karışması yaşarsın. Peki neden `new` ve `delete`'i öğrendik? Çünkü (1) `vector` ve akıllı işaretçilerin içte ne yaptığını anlamak için, (2) eski kodu okurken, (3) Bölüm 11'de kendi sınıfının içinde bellek yöneteceğinde ve (4) sınavda soruluyor.

> **İpucu: Çıplak new görürsen sor**
>
> Kodda `new` gördüğünde şu iki soruyu sor: "Bunun `delete`'i nerede?" ve "Bunun yerine `vector`/`make_unique` olamaz mı?" Yeni yazdığın kodda cevap neredeyse hep ikincisidir. Yeni bir blok açarken ilk satırda sahibini (akıllı işaretçi ya da `vector`) kur; ham işaretçi yalnızca bir şeye **bakmak** için kullanılır, sahiplik taşımaz.

Bölüm Özeti – Bilmen Gerekenler

- Yerel değişkenler **yığında** (otomatik, küçük), `new` ile istenenler **öbekte** (elle yönetilir, büyük) yaşar. Büyük yerel dizi yığın taşırır; `int a[n];` standart C++ değildir.
- `new Tip{değer}` ↔ `delete p`; `new Tip[n]{}` ↔ `delete[] p`. Eşleşme bozulursa tanımsız davranış. `new` yer bulamazsa `nullptr` değil **istisna** fırlatır.
- Hatalar: **sızıntı** (adres kaybı, `delete` unutmak, erken `return`), **sarkan işaretçi** (`delete` sonrası kullanım), **çift `delete`**, yanlış adresi `delete`, blok sınırını aşmak. Hepsi tanımsız ya da sinsi hatadır. `delete p; p = nullptr;` yalnız `p`'yi korur.
- Dinamik dizi büyütme: yeni (iki kat) dizi aç, kopyala, eskiyi `delete[]`, işaretçiyi çevir. `vector` bunu yapar: `size()` ≤ `capacity()`; büyüyünce eski adresler geçersiz olur.
- **RAII**: kaynağı bir nesneye bağla; nesne yok olunca kaynak bırakılır. Yerel nesneler kapsam sonunda **ters sırada** yok edilir.
- `unique_ptr` (`make_unique`): tek sahip, **kopyalanamaz**, `move` ile taşınır (eski işaretçi boş kalır). `make_unique<int[]>(n)` diziyi sıfırlı açar.
- `shared_ptr` (`make_shared`): sayaçlı paylaşım, `use_count()`; sayaç 0 olunca blok geri verilir. Döngüsel sahiplik sızıntı yapar.
- Seçim sırası: yerel değişken → `string`/`vector` → `unique_ptr` → `shared_ptr` → (neredeyse hiç) `new`/`delete`. **Çıplak `new` yazma.**

Sorular

A) Çoktan Seçmeli

1 `int* d{new int[10]};` satırıyla alınan bloğu doğru biçimde geri vermenin yolu hangisidir?

(A) `delete d;` (B) `delete[] d;` (C) `free(d);` (D) `delete d[10];` (E) `d = nullptr;`

2 Aşağıdaki satırlardan sonra program bitiyor (başka satır yok). Kaç öbek bloğu sızmıştır?

```
int* a{new int{1}};
int* b{new int{2}};
a = b;
delete b;
int* c{new int[3]{}};
```

(A) 0 (B) 1 (C) 2 (D) 3 (E) 4

3 Aşağıdaki iki satırdan sonra hangisi **doğrudur**?

```
auto p{make_unique<int>(10)};
auto q{std::move(p)};
```

(A) `q` bloğun sahibidir, `p` boştur (`nullptr`).

(B) `p` ve `q` ikisi de sahiptir; blok iki kez silinir.

(C) Satır derlenmez, çünkü `unique_ptr` taşınamaz.

(D) `p` hâlâ 10'u gösterir, `q` boştur.

(E) 10 değeri iki ayrı bloğa kopyalanır; `p` ve `q` bağımsızdır.

4 Aşağıdaki kod ekrana ne yazdırır?

```
auto a{make_shared<int>(5)};
auto b{a};
{
    auto c{a};
    auto d{b};
}
auto e{a};
shared_ptr<int> f;
f = b;
cout << a.use_count();
```

(A) 1 (B) 2 (C) 3 (D) 4 (E) 5

5 Eleman sayısı çalışma anında belli olan ve sonradan büyüyebilecek bir `int` listesini saklamak için modern C++'ta **en uygun** yol hangisidir?

(A) `int* d{(int*)malloc(n * sizeof(int))};`

(B) `int* d{new int[n]};` ve sonunda `delete[] d;`

(C) `int d[n];`

(D) `shared_ptr<int> d{new int[n]};`

(E) `vector<int> d(n);`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
#include <iostream>
#include <memory>

using namespace std;

struct Kaynak {
    int no;
    Kaynak(int n) {
        no = n;
        cout << "+" << no << '\n';
    }
    ~Kaynak() {
        cout << "-" << no << '\n';
    }
};

int main() {
    auto x{make_unique<Kaynak>(1)};
    {
        auto y{make_unique<Kaynak>(2)};
        auto z{make_unique<Kaynak>(3)};
        cout << "ic\n";
    }
    auto w{std::move(x)};
    cout << (x == nullptr) << '\n';
    cout << "son\n";
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
#include <memory>

using namespace std;

void goster(shared_ptr<int> p) {
    cout << p.use_count() << ' ';
}

int main() {
    auto a{make_shared<int>(7)};
    goster(a);
    auto b{a};
    goster(b);
    cout << a.use_count() << ' ';
    b.reset();
    cout << a.use_count() << ' ' << *a << '\n';
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>

using namespace std;

int main() {
    int* d{new int[4]{3, 1}};
    d[2] = d[0] + d[1];
    int toplam{0};
    for (int i{0}; i < 4; i++) {
        toplam += d[i];
    }
    int* e{new int[6]{}};
    for (int i{0}; i < 4; i++) {
        e[i] = d[i] * 2;
    }
    delete[] d;
    d = e;
    d[4] = toplam;
    d[5] = d[4] - d[0];
    cout << toplam << '\n';
    for (int i{0}; i < 6; i++) {
        cout << d[i] << ' ';
    }
    cout << '\n';
    delete[] d;
    return 0;
}
```

C) Kod Yazma

9 Klavyeden **0 girilene kadar** tamsayı okuyan `enBuyukSay` programını yaz (kaç sayı gireceği belli değil; 0 diziye eklenmez). Sayıları `new[]` ile **2 elemanlık** başlayan, dolunca kapasitesi **ikiye katlanan** dinamik dizide sakla (büyütme işini yapan bir `buyut` fonksiyonu yaz). Okuma bitince girilen sayı adedini, **en büyük** sayıyı ve onun kaç kez girildiğini yazdır. Diziyi `delete[]` ile geri ver.

```
Sayilari gir (0 ile biter): 5 12 7 12 3 12 0
Girilen sayi: 6
En buyuk: 12 (3 kez)
```

10 Klavyeden bir `n` okuyup ilk `n` **Fibonacci** sayısını (1, 1, 2, 3, 5, 8, ...) hesaplayan `fibonacciDizisi` programını yaz. `unique_ptr<int[]> fibonacci(int n)` fonksiyonu sayıları `make_unique<int[]>` ile açtığı dizide tutup **sahipliği çağırana** döndürsün; `main` sayıları ve toplamlarını yazdırsın. `n` geçerli değilse (`n <= 0` ya da okuma başarısız) `Gecersiz sayi.` yazıp çık. Programda `new` ve `delete` kullanma.

```
Kac eleman? 8
1 1 2 3 5 8 13 21
Toplam: 54
```

Çözümler

Cevap anahtarı

1 B	2 C	3 A	4 D	5 E

1 Cevap: B

`new[]` ile alınan dizi **`delete[]`** ile bırakılır. A yanlış: köşeli parantezsiz `delete` tanımsız davranıştır. C yanlış: `free` yalnızca `malloc` ailesi içindir; `new` ile alınanı `free` ile bırakmak da tanımsız davranıştır. D derlenmez: `d[10]` bir `int`'tir ve `delete` yalnızca işaretçi ister. E yalnızca işaretçiyi boşaltır, bloğu geri vermez; blok **sızar**.

2 Cevap: C

İzleyelim: `a` birinci bloğu (değer 1), `b` ikinci bloğu (değer 2) gösterir. `a = b;` ile `a`'nın eski adresi silinir; birinci bloğu gösteren başka işaretçi kalmadı, o blok **sızdı** (1). `delete b;` ikinci bloğu geri verir (`a` ve `b` artık sarkandır ama kullanılmıyor, sızıntı değil). `c`'nin 3 elemanlık bloğu hiç `delete[]` edilmedi: **sızdı** (2). Toplam **2** blok. A yanlış: iki sızıntı var. B bunlardan birini kaçırıyor. D ve E sayıyı abartıyor: `delete b` ikinci bloğu zaten geri verdiği için o sızıntı değildir.

3 Cevap: A

`std::move(p)` sahipliği `q`'ya taşır; `p` boş (`nullptr`) kalır, `q` değeri 10 olan bloğun tek sahibidir. B yanlış: sahip tektir, blok bir kez silinir. C yanlış: `unique_ptr` **kopyalanamaz** ama `move` ile **taşınabilir**. D tersini söylüyor. E yanlış: değer kopyalanmaz, blok olduğu gibi devredilir.

4 Cevap: D

Sayaç sahip sayısını tutar. `a` 1; `b{a}` 2; iç blokta `c` ve `d` sayacı 4'e çıkarır ama blok bitince ikisi de yok olur, sayaç 2'ye döner; `e{a}` 3; `f = b` 4. Sonuçta `a`, `b`, `e` ve `f` aynı bloğu tutar: **4**. Tuzak: iç bloktaki `c` ve `d`'yi saymamak gerekir; onlar `cout` satırından önce yok olmuştur.

5 Cevap: E

Boyutu çalışma anında belli olan ve büyüyebilen liste için `vector` en uygun araçtır: belleği kendisi yönetir, büyür, kapsam bitince bırakır. A C stilidir, `malloc` nesneler için uygun değildir ve cast gerektirir. B çalışır ama elle yönetim gerektirir (`delete[]` unutulursa sızar) ve büyümez. C standart C++ değildir (değişken uzunluklu dizi) ve yığın taşabilir. D hatalıdır: `shared_ptr<int>` tek `int` için `delete` çağırır, `new int[n]` ise `delete[]` ister (tanımsız davranış).

6

Adım adım izle:

1. `x` oluşur: `+1`.
2. İç blokta `y` ve `z` oluşur: `+2`, `+3`; sonra `ic`.
3. İç blok biter, yerel nesneler **oluşturulmanın tersi sırada** yok edilir: önce `z` (`-3`), sonra `y` (`-2`).
4. `w{std::move(x)}`: sahiplik `x`'ten `w`'ye taşınır, yeni nesne **oluşmaz** (`+1` tekrar yazılmaz). `x == nullptr` doğrudur, `cout` bunu `1` olarak yazar.
5. `son`; `main` biterken `w` yok edilir: `-1`.

Çıktı
```
+1
+2
+3
ic
-3
-2
1
son
-1
```

7

Tuzak: Fonksiyona `shared_ptr`'yi **değer ile** geçirmek kopya demektir; çağrı süresince sayaç bir artar.

Adım	Sayaç
`goster(a)` içinde (`a` + parametre kopyası)	2
`auto b{a}` sonrası, `goster(b)` içinde (`a`, `b`, kopya)	3
`goster(b)` döndü; `a.use_count()` (`a`, `b`)	2
`b.reset()` sonrası (yalnız `a`)	1

Son olarak `*a` 7 yazar.

Çıktı
```
2 3 2 1 7
```

8

`new int[4]{3, 1}` dizisi `{3, 1, 0, 0}` olur (eksik ilk değerler 0'dır). `d[2] = 3 + 1 = 4` → `{3, 1, 4, 0}`; `toplam = 8`.

`e` 6 elemanlık, sıfırlı dizidir. İlk 4 eleman `d[i] * 2` ile `{6, 2, 8, 0, 0, 0}` olur. Eski `d` bırakılır, `d = e;` ile `d` yeni diziyi gösterir. `d[4] = 8`; `d[5] = d[4] - d[0] = 8 - 6 = 2`. Dizi `{6, 2, 8, 0, 8, 2}`. Sondaki `delete[] d;` (aynı zamanda `e`'nin bloğunu bırakır) yeni diziyi geri verir; eski `d` bloğu zaten bırakılmıştı, sızıntı ya da çift silme yoktur.

Çıktı
```
8
6 2 8 0 8 2
```

9

Yaklaşım: Diziyi 2 elemanla başlat; `adet == kapasite` olunca `buyut` yeni (iki kat) dizi açıp kopyalasın, eskiyi `delete[]` etsin. Fonksiyon çağıranın işaretçisini değiştirdiği için `int*&` alır. Sonra bir geçişle en büyüğü bul, ikinci geçişle say. İlk girilen sayı `0` ise dizi boştur ve "en büyük" diye bir şey yoktur; bu durumu en büyüğü aramadan önce ayrıca kontrol et.

enBuyukSay.cpp
```cpp
#include <iostream>

using namespace std;

void buyut(int*& dizi, int& kapasite, int adet) {
```

```
    int yeniKapasite{kapasite * 2};
    int* yeni{new int[yeniKapasite]{}};

    for (int i{0}; i < adet; i++) {
        yeni[i] = dizi[i];
    }
    delete[] dizi;
    dizi = yeni;
    kapasite = yeniKapasite;
}

int main() {
    int kapasite{2};
    int adet{0};
    int* sayilar{new int[kapasite]{}};
    int x{0};

    cout << "Sayilari gir (0 ile biter): ";
    while (cin >> x && x != 0) {
        if (adet == kapasite) {
            buyut(sayilar, kapasite, adet);
        }
        sayilar[adet] = x;
        adet++;
    }
    if (adet == 0) {                    // ilk sayı 0: en büyük yok
        cout << "Hic sayi girilmedi\n";
        delete[] sayilar;
        return 0;
    }
    int enBuyuk{sayilar[0]};
    for (int i{1}; i < adet; i++) {
        if (sayilar[i] > enBuyuk) {
            enBuyuk = sayilar[i];
        }
    }
    int kez{0};
    for (int i{0}; i < adet; i++) {
        if (sayilar[i] == enBuyuk) {
            kez++;
        }
    }
    cout << "Girilen sayi: " << adet << '\n';
    cout << "En buyuk: " << enBuyuk << " (" << kez << " kez)\n";
    delete[] sayilar;
    return 0;
}
```

Çıktı

```
Sayilari gir (0 ile biter): 5 12 7 12 3 12 0
Girilen sayi: 6
En buyuk: 12 (3 kez)
```

Sayılar 5, 12, 7, 12, 3, 12: altı sayı, en büyük 12, üç kez girilmiş.

10

Yaklaşım: `make_unique<int[]>(n)` diziyi sıfırlı açar; ilk iki eleman 1, sonrakiler önceki ikisinin toplamıdır. Dizi `return a;` ile çağırana taşınır; `main` `delete[]` yazmaz, `fib` kapsam sonunda kendiliğinden bırakılır. `n == 1` durumunda `a[1]` dizinin dışına çıkmasın diye ikinci elemanı `n > 1` koşuluyla yazıyoruz.

fibonacciDizisi.cpp

```cpp
#include <iostream>
#include <memory>

using namespace std;

unique_ptr<int[]> fibonacci(int n) {
    auto a{make_unique<int[]>(n)};

    a[0] = 1;
    if (n > 1) {
        a[1] = 1;
    }
    for (int i{2}; i < n; i++) {
        a[i] = a[i - 1] + a[i - 2];
    }
    return a;
}

int main() {
    int n{0};

    cout << "Kac eleman? ";
    if (!(cin >> n) || n <= 0) {
        cout << "Gecersiz sayi.\n";
        return 1;
    }
    auto fib{fibonacci(n)};
    int toplam{0};
    for (int i{0}; i < n; i++) {
        cout << fib[i];
        cout << (i + 1 < n ? ' ' : '\n');
        toplam += fib[i];
    }
    cout << "Toplam: " << toplam << '\n';
    return 0;
}
```

Çıktı

```
Kac eleman? 8
1 1 2 3 5 8 13 21
Toplam: 54
```

$1 + 1 + 2 + 3 + 5 + 8 + 13 + 21 = 54$.

BÖLÜM 11

Sınıflar ve Nesneler

Bu Bölümde Öğreneceklerin

- Birbirine ait verileri tek pakette toplamayı (struct ve class)
- Sınıf ile nesne farkını, üye değişken ve üye fonksiyonu
- Verinin dışarıdan korunmasını: public, private, kapsülleme
- Nesne doğarken çalışan yapıcıyı, yok olurken çalışan yıkıcıyı ve çağrı sırasını
- Nesnenin kendisini gösteren this'i ve const üye fonksiyonları
- Bütün nesnelerin paylaştığı static üyeleri
- Nesne dizileri kurmayı ve nesneleri fonksiyonlara geçirmeyi
- Sınıfı başlık ve kaynak dosyalarına bölmeyi, enum class'ı

11.1 Neden sınıf? struct ile veri gruplama

Şimdiye kadar veriler (değişkenler, `vector`'ler) ve onlarla çalışan fonksiyonlar ayrıydı. Öğrencilerin adını, numarasını ve ortalamasını tutmak için Proje 1'deki gibi üç paralel `vector` açardın. Bu düzen kırılgandır: adları sıralayıp diğer iki vector'ü sıralamayı unutursan Ali'nin notu Ayşe'ye geçer; yeni bir bilgi eklemek yeni bir vector demektir. Çözüm, birbirine ait verileri **tek bir tip** olarak paketlemektir. İlk araç **yapı** (`struct`) türüdür; içindeki değişkenlere **üye değişken** (member variable) denir ve tanımın sonuna `;` konur:

structOgrenci.cpp

```
#include <iostream>
#include <string>
using namespace std;

struct Ogrenci {
    string ad;
    int no;
    double ortalama;
};

void yazdir(const Ogrenci& o) {         // const &: Bölüm 6'dan tanıdık
    cout << o.no << " - " << o.ad << " - " << o.ortalama << '\n';
}

int main() {
    Ogrenci a{"Ali", 101, 72.5};        // üyeler tanım sırasıyla dolar
    Ogrenci b = a;                      // b, a'nın bağımsız kopyası
    b.ad = "Can";
    b.ortalama = 64.0;
    yazdir(a);
    yazdir(b);
    return 0;
}
```

```
Çıktı
101 - Ali - 72.5
101 - Can - 64
```

`Ogrenci` artık `int` gibi bir tip; `Ogrenci a{...};` bu tipte bir **nesne** (object) oluşturur. Üyelere **nokta operatörüyle** ulaşılır (`a.ad`). `b = a` bütün üyeleri kopyalar; `b`'yi değiştirmek `a`'yı etkilemez, tıpkı `int` gibi. Yapının zayıflığı: her şey dışarıdan değiştirilebilir. `a.ortalama = 250;` yazan çıkarsa derleyici sesini çıkarmaz. Verisini koruyan ve "bilgilerini yazdır" gibi işleri kendisi bilen bir paket için `class`'a geçeceğiz.

11.2 Sınıf, nesne ve üye fonksiyon

Sınıf (class), nesnelerin nasıl olacağını tarif eden şablondur; **nesne** ise o şablondan üretilmiş somut örnektir. Kurabiye kalıbı tek, kurabiye çok: her nesnenin yapısı aynı, değerleri farklıdır. Sınıf yalnızca veri değil **davranış** da taşır: sınıfın içindeki fonksiyona **üye fonksiyon** (member function) denir. Bu kitapta "metot" demiyoruz.

ilkSinif.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

class Ogrenci {
public:                                 // şimdilik herkese açık
    string ad;
    int no;
    double ortalama;

    void bilgiYazdir() {                // üye fonksiyon
        cout << no << " - " << ad << " - " << ortalama << '\n';
    }
};

int main() {
    Ogrenci a{"Ali", 101, 72.5};
    Ogrenci b{"Ayşe", 102, 88.0};
    a.bilgiYazdir();
    b.bilgiYazdir();
    Ogrenci* p = &b;                    // nesneye işaretçi
    p->ortalama = 90.5;                 // (*p).ortalama ile aynı
    p->bilgiYazdir();
    return 0;
}
```

```
Çıktı
101 - Ali - 72.5
102 - Ayşe - 88
102 - Ayşe - 90.5
```

- Üye fonksiyon **hangi nesne üzerinden çağrıldıysa** onun üyelerini kullanır: `a.bilgiYazdir()` içindeki `no`, `a`'nın `no`'sudur. Çağrı noktası "kimin?" sorusunu cevaplar.
- Nesneye işaretçiyle `.` yerine `->` yazılır (Bölüm 10'daki gibi).
- Üye fonksiyon, sınıfın daha aşağıda tanımlanan üyelerini de görür; sıra önemsizdir.

> **Not: struct ile class farkı**
>
> C++'ta tek dil farkı **varsayılan erişimdir**: `struct` üyeleri varsayılan `public`, `class` üyeleri `private`'tır. Yapıcı, yıkıcı, üye fonksiyon ikisinde de vardır. Alışkanlık: yalnızca veri taşıyan basit paketlere `struct`, davranışı ve korunan verisi olana `class`.

> **Sınavda Çıkar: struct ve class**
>
> "`struct` ile `class` farkı nedir?" sorusunun C++ cevabı varsayılan erişimdir. "Biri değer, öbürü referans tipidir" diyen cevap C#'a aittir; C++'ta ikisi de değer gibi davranır.

11.3 Erişim belirteçleri ve kapsülleme

Üyeler herkese açıksa biri `hesap.bakiye = -5000;` yazıp nesneyi geçersiz duruma düşürebilir. **Kapsülleme** (encapsulation), veriyi dış dünyadan gizleyip ona yalnızca sınıfın izin verdiği yoldan erişilmesini sağlamaktır; bankamatikte kasayı açıp parayı kendin alamazsın, düğmelerle kurallara uygun işlem yaparsın. Gizlemeyi **erişim belirteçleri** (access specifiers) yapar:

Belirteç	Sınıfın kendi üyeleri	Sınıfın dışı (`main`, diğer fonksiyonlar)
`public`	Erişir	Erişir
`private`	Erişir	**Erişemez**
`protected`	Erişir	Erişemez (türetilmiş sınıf farkı Bölüm 12'de)

Belirteç, bir sonrakini görene ya da sınıf bitene kadar geçerlidir. Yazmayı unutmak, ilk sınıfını yazan herkesin yaşadığı hatadır:

privateErisim.cpp

```cpp
class Hesap {
    double bakiye{100};                // belirteç yok: class'ta private
};

int main() {
    Hesap h;
    h.bakiye = -5000;
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
privateErisim.cpp: In function 'int main()':
privateErisim.cpp:7:7: error: 'double Hesap::bakiye' is private within this context
    7 |     h.bakiye = -5000;
      |       ^~~~~~
privateErisim.cpp:2:12: note: declared private here
    2 |     double bakiye{100};                // belirteç yok: class'ta private
      |            ^~~~~~
```

Kural: **üye değişkenler** `private`, dışarıya sunulan işlemler `public`. Dışarıdan okuma gerekiyorsa **getter**, değiştirme gerekiyorsa **setter** yazarsın; setter'ın asıl değeri **doğrulama** yapabilmesidir:

kapsulleme.cpp

```cpp
#include <iostream>
using namespace std;

class Ogrenci {
private:
    double ortalama{0};                // üye ilk değeri (yapıcıda göreceğiz)

public:
    double ortalamaAl() const { return ortalama; }      // getter

    void ortalamaAyarla(double yeni) {                  // setter
        if (yeni < 0 || yeni > 100) {
            cout << "Geçersiz ortalama: " << yeni << '\n';
            return;                    // alanı değiştirmeden çık
        }
        ortalama = yeni;
    }

    double ortalamaFarki(const Ogrenci& diger) const {
        return ortalama - diger.ortalama;       // başka nesnenin private'ı
    }
};

int main() {
    Ogrenci o, p;
    o.ortalamaAyarla(78.5);
    o.ortalamaAyarla(250);
    p.ortalamaAyarla(30);
    cout << o.ortalamaAl() << " " << p.ortalamaAl() << '\n';
    cout << "Fark: " << o.ortalamaFarki(p) << '\n';
    return 0;
}
```

Çıktı

```
Geçersiz ortalama: 250
78.5 30
Fark: 48.5
```

Doğrulama, geçersiz `250` değerinin alana hiç yazılmasını engelledi. Sonlardaki `const`'u birazdan anlatacağız; şimdilik kalıp olarak al. `ortalamaFarki` içinde `diger.ortalama` yazılabildiğine dikkat: `private`, **sınıfa** özeldir, nesneye değil.

Sınavda Çıkar: private neye özeldir

Tuzak: erişimi sınıf belirler, nesne değil. Aynı sınıfın üye fonksiyonu, aynı sınıftan **başka** bir nesnenin `private` üyesini de okuyabilir (`diger.ortalama`). Sınıfın dışındaki `main` ya da serbest fonksiyonlar okuyamaz.

İpucu: Her üyeye get/set yazma

Her üyeye mekanik getter ve setter yazmak kapsülleme değildir; veriyi daha uzun bir yoldan açık bırakmaktır. Okunması gerekene getter, değişmesi gereken ve **doğrulanabilene** setter yaz. `bakiyeAyarla(5000)` yerine `yatir` ve `cek` gibi anlamlı işlemler yaz. Başka kitaplarda `getAd()` / `setAd()` göreceksin; biz `adAl()` / `adAyarla()` yazıyoruz.

11.4 Yapıcı (constructor)

`private` üyeleri dışarıdan değiştiremiyorsak nesneye ilk değerini kim verecek? Verilmezse `bakiye` çöp değerle doğar. Çözüm **yapıcı** (constructor): nesne **oluşturulurken** otomatik çalışan özel bir üye fonksiyondur. Nesne yaşamına ilk anda geçerli bir durumla başlar; "önce nesne oluştur, sonra alanları doldurmayı unutma" derdi biter. İki kuralı vardır: adı **sınıfın adıyla aynıdır** ve **dönüş tipi yoktur** (`void` bile yazılmaz).

hesapYapici.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

class Hesap {
private:
    string sahip;
    double bakiye;

public:
    Hesap(string yeniSahip, double ilkBakiye) {     // yapıcı
        sahip = yeniSahip;
        bakiye = ilkBakiye >= 0 ? ilkBakiye : 0;    // geçersizse 0
        cout << sahip << " için hesap açıldı\n";
    }

    void bilgiYazdir() const {
        cout << sahip << ": " << bakiye << " TL\n";
    }
};

int main() {
    Hesap a{"Ali", 500};               // yapıcı bu satırda çalışır
    Hesap b("Ayşe", -20);              // parantezle de yazılır
    a.bilgiYazdir();
    b.bilgiYazdir();
    return 0;
}
```

Çıktı

```
Ali için hesap açıldı
Ayşe için hesap açıldı
Ali: 500 TL
Ayşe: 0 TL
```

Yapıcıyı **sen çağırmıyorsun**; nesne tanımlanırken `{...}` (ya da `(...)`) içine yazdığın değerler yapıcıya argüman olarak gider. Kitap boyunca süslü parantezi tercih ediyoruz; küçük bir fayda: `int` parametreye `2.5` gibi veri kaybettiren bir değer verirsen derleyici itiraz eder. `Ayşe` hesabının `-20` ile açılmaya çalışıldığına, ama yapıcının bakiyeyi `0`'a çektiğine dikkat: asıl iş budur, **hiçbir nesne geçersiz doğmaz**.

Varsayılan yapıcı ve kaybolması

Hiç yapıcı yazmadığında derleyici sana **varsayılan yapıcıyı** (default constructor; parametresiz yapıcı) kendisi üretir. Bu yüzden önceki örneklerde `Ogrenci o, p;` yazabildik. Tuzak şu: **kendin herhangi bir yapıcı yazdığın an derleyici bunu üretmeyi bırakır.**

varsayilanYok.cpp

```cpp
class Kutu {
public:
    int en;
    Kutu(int e) { en = e; }            // kendi yapıcımızı yazdık
};

int main() {
    Kutu k;                            // parametresiz yapıcı yok artık
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
varsayilanYok.cpp: In function 'int main()':
varsayilanYok.cpp:8:10: error: no matching function for call to 'Kutu::Kutu()'
    8 |     Kutu k;                            // parametresiz yapıcı yok artık
      |          ^
  * there are 3 candidates
    * candidate 1: 'Kutu::Kutu(int)'
      varsayilanYok.cpp:4:5:
          4 |     Kutu(int e) { en = e; }            // kendi yapıcımızı yazdık
            |     ^~~~
      * candidate expects 1 argument, 0 provided
    * candidate 2: 'constexpr Kutu::Kutu(const Kutu&)'
      varsayilanYok.cpp:1:7:
          1 | class Kutu {
            |       ^~~~
      * candidate expects 1 argument, 0 provided
    * candidate 3: 'constexpr Kutu::Kutu(Kutu&&)'
      * candidate expects 1 argument, 0 provided
varsayilanYok.cpp:8:10: warning: unused variable 'k' [-Wunused-variable]
    8 |     Kutu k;                            // parametresiz yapıcı yok artık
      |          ^
```

Parametresiz de nesne oluşturmak istiyorsan ya ikinci bir yapıcı yazarsın (yapıcılar da **aşırı yüklenebilir**, Bölüm 6'daki kural aynen geçerli) ya da `= default` ile "derleyicinin üreteceği hâli bana ver" dersin:

yapiciYukleme.cpp

```cpp
#include <iostream>
using namespace std;

class Kutu {
private:
    int en;
    int boy;

public:
    Kutu(int e, int b) : en{e}, boy{b} {}       // asıl yapıcı
    Kutu(int kenar) : Kutu{kenar, kenar} {}     // kare kutu
    Kutu() : Kutu{1, 1} {}                      // varsayılan: 1x1

    int alan() const { return en * boy; }
};

int main() {
    cout << Kutu{}.alan() << " " << Kutu{5}.alan() << " ";
    cout << Kutu{3, 4}.alan() << '\n';
    return 0;
}
```

```
Çıktı
1 25 12
```

İki yeni şey var. `: en{e}, boy{b}` kısmına birazdan geleceğiz. `Kutu(int kenar) : Kutu{kenar, kenar}` ise bir yapıcının işi **başka bir yapıcıya devretmesidir** (**devreden yapıcı**, delegating constructor): ortak iş tek yerde kalır, her yapıcıda tekrar yazılmaz. Gövdesi boş bir `Kutu() = default;` da yazabilirdin; ama bu durumda `en` ve `boy` çöp değerle başlardı. Bu yüzden 1x1 değerini veren devreden yapıcıyı tercih ettik. `= default` ifadesini Bölüm 12'de `virtual ~Sekil() = default;` olarak tekrar göreceksin.

> **Sık Yapılan Hata: Parantezli boş nesne tanımı**
>
> `Hesap h();` yazıp parametresiz nesne oluşturduğunu sanma: C++ bunu "`h` adlı, parametresiz, `Hesap` döndüren **fonksiyon bildirimi**" diye okur ve `h.bilgiYazdir()` anlamsız bir hata verir. Parantezsiz (`Hesap h;`) ya da `Hesap h{};` yaz. Bu, "en can sıkıcı ayrıştırma" (most vexing parse) tuzağıdır.

Üye ilk değer listesi

Yapıcıların `:` ile başlayan kısmı **üye ilk değer listesidir** (member initializer list). Yapıcının gövdesinde `sahip = yeniSahip;` yazmak **atamadır**: üye önce (boş) doğar, sonra değer atanır. Listede yazmak ise üyeyi **doğduğu anda** istenen değerle oluşturur. `int` için fark ihmal edilebilir; ama bazı üyeler için liste **zorunludur**:

- `const` üyeler ve referans üyeler: doğduktan sonra atanamazlar.
- Varsayılan yapıcısı olmayan sınıf tipli üyeler: argüman vermen gerekir.

`const int no;` üyesi olan bir sınıfta yapıcı gövdesine `no = n;` yazarsan derleme hatası alırsın: atama, değiştirmeye çalışmaktır. Doğrusu `Ogrenci(int n) : no{n} {}` yazmaktır. Kural olarak **bütün üyeleri listede başlat**; gövdeyi doğrulama gibi ek işlere ayır. Bir tuzak daha var: üyeler listede yazdığın sıraya göre değil, **sınıfta tanımlandıkları sıraya** göre başlatılır. Sınıfta `alt`, `ust`'tan önce tanımlıysa `Aralik(int a) : ust{a}, alt{ust - 5} {}` yanıltıcıdır (g++ `-Wreorder` uyarısı verir): `alt` önce başlatıldığı için henüz değeri olmayan `ust`'u okur ve çöp değer alır. Listeyi sınıftaki sıraya göre yaz, uyarıyı ciddiye al.

Üyeye **sınıfın içinde** de ilk değer verebilirsin: kapsülleme örneğindeki `double ortalama{0};` satırı böyleydi. Buna **sınıf içi üye ilk değeri** (default member initializer) denir. Yapıcının listesi o üyeye değer vermezse bu değer kullanılır; listede değer verilirse listedeki kazanır. Böylece hangi yapıcı çalışırsa çalışsın üye çöp değerle kalmaz. Yukarıdaki `Kutu` sınıfında `int en{1};` ve `int boy{1};` yazsaydın `Kutu() = default;` de güvenli olurdu.

11.5 Yıkıcı (destructor)

Yapıcı nesne doğarken çalışır; **yıkıcı** (destructor) nesne **yok edilirken** otomatik çalışır. Adı `~` ile başlayan sınıf adıdır (`~Hesap`), parametre almaz, dönüş tipi yoktur, sınıfta bir tane olur. Bölüm 10'da RAII'yi anlatırken "nesne yok olurken çalışan satırlar" demiştik; asıl işi, nesnenin sahip olduğu **kaynağı** (bellek, dosya, bağlantı) geri vermektir. İlk bakışta ne zaman çalıştığını görmek için içine mesaj yazan bir sınıf yeter. Bu örnek aynı zamanda **çağrı sırasının** kuralını gösterir:

yikiciSirasi.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

class Iz {
private:
    string ad;

public:
```

```
    Iz(string yeniAd) : ad{yeniAd} { cout << "+" << ad << '\n'; }
    ~Iz() { cout << "-" << ad << '\n'; }
};

Iz genel{"genel"};                      // main'den önce doğar

int main() {
    cout << "main basladi\n";
    Iz a{"a"};
    {
        Iz b{"b"};
        Iz c{"c"};
        cout << "blok sonu\n";
    }
    Iz* p{new Iz{"p"}};                 // öbekte nesne
    cout << "delete oncesi\n";
    delete p;                           // yıkıcıyı çalıştırır
    Iz d{"d"};
    cout << "main bitiyor\n";
    return 0;
}
```

Çıktı

```
+genel
main basladi
+a
+b
+c
blok sonu
-c
-b
+p
delete oncesi
-p
+d
main bitiyor
-d
-a
-genel
```

Kuralları çıktıdan oku:

- **Yerel nesneler** tanımlandıkları sırayla doğar, kapsam bittiğinde **ters sırayla** yok edilir (`c`, sonra `b`; `main` bitince `d`, sonra `a`). Bu, Bölüm 10'daki RAII sırasıdır.
- `new` ile oluşturulan nesne, kapsam bitince yok olmaz; yıkıcı yalnızca **`delete` yazdığın anda** çalışır. `delete` unutulursa yıkıcı hiç çalışmaz, bellek sızar. `unique_ptr` bu yüzden iyidir: yıkıcısı `delete` işini senin yerine yapar.
- **Global nesne**, `main`'den önce doğar ve `main` bittikten sonra, en son yok edilir.
- **Üye nesneler:** Bir sınıfın üyesi başka bir sınıfın nesnesiyse (örneğin `Kutu`'nun içinde bir `Iz ic;` üyesi), önce üyeler doğar, sonra dıştaki sınıfın yapıcı gövdesi çalışır (**içten dışa**). Yok edilirken tersi olur: önce dıştaki sınıfın yıkıcı gövdesi, sonra üyelerin yıkıcıları çalışır (**dıştan içe**). Bina temelden kurulur, çatıdan yıkılır.

Yıkıcı, temizlik gerektiren sınıflarda yazılır. Öbekte dizi tutan bir `Tampon` sınıfı, yapıcısında `veri{new int[n]{}}` ile belleği alır, yıkıcısında `delete[] veri;` ile geri verir; sınıfı kullanan kod hiç `delete[]` yazmaz, nesne kapsamdan çıkınca bellek kendiliğinden bırakılır. Kaynağı sınıfa bağlamak RAII'nin kendisidir.

> **Sık Yapılan Hata: Yıkıcılı sınıfı kopyalamak**
>
> `new[]` ile bellek tutan `Tampon` gibi bir sınıfta `Tampon u{t};` ya da `Tampon u = t;` yazarsan derleyicinin ürettiği kopya, `veri` **işaretçisini** olduğu gibi kopyalar; iki nesne aynı öbek bloğunu gösterir ve ikisinin yıkıcısı aynı bloğa `delete[]` uygular. Bu bir çift silmedir ve **tanımsız davranıştır**. Kopyalamanın neden böyle olduğunu ve çözümünü (kopya yapıcı, üçler kuralı) Bölüm 13'te göreceksin. Şimdilik kural: kaynak tutan sınıfı kopyalama; ya da kaynağı `vector` / `unique_ptr` ile tut, o zaman yıkıcıyı hiç yazman gerekmez.

11.6 this: nesnenin kendisi

Üye fonksiyon `a.bilgiYazdir()` diye çağrıldığında içindeki `no` nasıl `a`'nın `no`'su oluyor? Derleyici her (static olmayan) üye fonksiyona gizli bir parametre ekler: fonksiyonun çağrıldığı nesneyi gösteren işaretçi, yani **`this`**. Tipi `Sinif*`'dır (`const` üye fonksiyonda `const Sinif*`). `no` yazmak aslında `this->no` yazmaktır; çoğu zaman `this`'i yazmana gerek kalmaz. Gerektiği iki yer vardır.

1. Parametre adı üye adıyla aynıysa. İç kapsamdaki ad, dıştakini gölgeler (Bölüm 6). `Nokta` sınıfında `void xAyarla(int x)` yazarsan parametre `x`, üye `x`'i gölgeler; üyeye `this->x = x;` ile yazarsın (sol taraf üye, sağ taraf parametre). `this->x = x;` yerine yalnızca `x = x;` yazsaydın parametreyi kendisine atardın; üye değişmezdi. Bu yüzden bu kitapta çoğunlukla farklı adlar seçiyoruz (`yeniAd`, `ilkBakiye`).

2. Nesnenin kendisini döndürmek. `*this`, çağrıldığı nesnenin kendisidir. Üye fonksiyon `Sinif&` döndürüp `return *this;` derse çağrılar zincirlenir (`cout << a << b` aynı fikirdir). Sayaç sınıfında `Sayac& artir() { sayi++; return *this; }` ve `Sayac& ekle(int n) { sayi += n; return *this; }` yazarsan `s.artir().artir().ekle(10).artir();` (toplam 13 artar) soldan sağa işler: her çağrı `s`'nin kendisini geri verir, bir sonraki çağrı yine `s` üzerinde çalışır.

> **Sık Yapılan Hata: Zincirde & unutmak**
>
> Dönüş tipini `Sayac&` yerine `Sayac` yazarsan `return *this;` nesnenin **kopyasını** döndürür. `s.artir().artir()` içindeki ikinci `artir()` kopyaya uygulanır, `s` yalnızca bir kez artar. Derleyici ses çıkarmaz; sonuç sessizce yanlış olur. Zincirleme için dönüş tipi daima `Sinif&`.

11.7 const üye fonksiyon

Daha önce gördüğün `double ortalamaAl() const` ve `void bilgiYazdir() const` satırlarındaki parantezden sonraki `const`, derleyiciye verilen bir **sözdür**: "bu fonksiyon nesnenin hiçbir üyesini değiştirmez." Söz tutulmazsa derleyici hata verir. Asıl faydası şuradadır: **`const` nesne** ya da **`const &` parametre** üzerinden yalnızca `const` üye fonksiyonlar çağrılabilir. Fonksiyona `const Ogrenci&` ile bir nesne gönderdiğinde, onun üzerinde değiştirme yapmayacağını derleyici sana garanti eder.

constHata.cpp

```cpp
#include <iostream>
using namespace std;

class Dikdortgen {
private:
    int en{3};
    int boy{4};

public:
    int alan() const { return en * boy; }
    void buyut(int kat) { en *= kat; boy *= kat; }      // const değil
};
```

```
void incele(const Dikdortgen& d) {
    cout << d.alan() << '\n';          // tamam: alan() const
    d.buyut(2);                        // hata: buyut() const değil
}

int main() {
    Dikdortgen d;
    incele(d);
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
constHata.cpp: In function 'void incele(const Dikdortgen&)':
constHata.cpp:16:12: error: passing 'const Dikdortgen' as 'this' argument discards qualifiers [-
fpermissive]
   16 |     d.buyut(2);                        // hata: buyut() const değil
      |     ~~~~~~~^~
constHata.cpp:11:10: note:   in call to 'void Dikdortgen::buyut(int)'
   11 |     void buyut(int kat) { en *= kat; boy *= kat; }      // const değil
      |          ^~~~
```

Hata mesajının özü: `const` bir nesne üzerinden `const` olmayan fonksiyon çağırdın (`discards qualifiers`: "nitelikleri atıyor"). İki yönlü bir kuraldır:

	`const` üye fonksiyon	`const` olmayan üye fonksiyon
`const` nesne / `const &` üzerinden çağrılır mı?	Evet	**Hayır**
Üyeleri değiştirebilir mi?	**Hayır**	Evet
Normal nesne üzerinden çağrılır mı?	Evet	Evet

Bu yüzden kural: **nesneyi değiştirmeyen her üye fonksiyonu `const` yaz** (getter'lar, `alan()`, `bilgiYazdir()`). Sonradan `const &` parametre alan bir fonksiyondan o fonksiyonu çağırmak gerektiğinde `const` eksikliği seni zorlamasın. Bu bir "const doğruluğu" (const correctness) alışkanlığıdır.

> **Sınavda Çıkar: const üye fonksiyon**
>
> "`const` üye fonksiyon neyi garanti eder?" → nesnenin üyelerini değiştirmediğini. Ek tuzak: `const` yazılmış bir fonksiyonun içinde üyeye atama yapmak da derleme hatasıdır. Hem `const` olan hem olmayan hâli aynı adla yazılabilir (aşırı yükleme); `const` nesne için `const` olanı seçilir.

11.8 static üyeler

Bir `Ogrenci` nesnesi oluşturuldukça sayan bir sayaç istiyorsun. Sayaç her öğrencinin değil, **sınıfın** malı olmalı: bir tane olmalı ve bütün nesneler onu paylaşmalı. Üye değişkenin başına `static` yazarsan, nesne başına bir kopya yerine **tüm sınıf için tek bir kopya** olur. `static` üye fonksiyon ise nesne olmadan, `Sinif::fonksiyon()` biçiminde çağrılır ve `this`'i yoktur; bu yüzden yalnızca `static` üyelere erişebilir.

staticUye.cpp

```
#include <iostream>
#include <string>
using namespace std;

class Ogrenci {
private:
    string ad;
```

```
    int no;
    static int sonNo;                   // sınıfta yalnızca BİLDİRİM

public:
    Ogrenci(string yeniAd) : ad{yeniAd}, no{++sonNo} {}

    void bilgiYazdir() const { cout << no << " - " << ad << '\n'; }

    static int toplamAl() { return sonNo; }     // this yok
};

int Ogrenci::sonNo{0};                  // TANIM: sınıfın dışında, bir kez

int main() {
    cout << "Baslangic: " << Ogrenci::toplamAl() << '\n';
    Ogrenci a{"Ali"};
    Ogrenci b{"Ayşe"};
    Ogrenci c{"Can"};
    a.bilgiYazdir();
    c.bilgiYazdir();
    cout << "Toplam: " << Ogrenci::toplamAl() << " " << a.toplamAl() << '\n';
    return 0;
}
```

Çıktı

```
Baslangic: 0
1 - Ali
3 - Can
Toplam: 3 3
```

- `sonNo` üç nesne için **tek** değişkendir; her yapıcı onu bir artırıp sonucu `no`'ya yazar. Böylece numaralar 1, 2, 3 olur.
- `static` veri üyesinin sınıfın içindeki satırı yalnızca **bildirimdir**; belleği ayıran **tanım**, sınıfın dışındaki `int Ogrenci::sonNo{0};` satırıdır ve programda **bir kez** yazılır. `::` kapsam çözümleme operatörüdür (Bölüm 6'da genel değişken için görmüştün); burada "`Ogrenci` sınıfının `sonNo`'su" demektir.
- Çağrı `Ogrenci::toplamAl()` olarak hiç nesne olmadan yapılabilir (ilk satır). `a.toplamAl()` de çalışır, ama sınıf adıyla yazmak okuyana bunun `static` olduğunu söyler.

C++17'den beri `inline static int sonNo{0};` yazarsan sınıfın içinde hem bildirim hem tanım olur ve dışarıdaki satır gerekmez. Burada `inline`, başlık dosyasına yazılan bu tanımın birden çok `.cpp` dosyasında tekrar tanım hatası yaratmasını önler; ayrıntısı ileri düzey bir konudur, şimdilik kalıp olarak al.

● Sık Yapılan Hata: static üyenin tanımını unutmak

Sınıf içinde `static int sonNo;` yazıp dışarıdaki `int Ogrenci::sonNo{0};` satırını unutursan derleme geçer, **bağlama** aşamasında hata alırsın (bağlama hatası; Bölüm 1). g++/MinGW mesajı şuna benzer:

```
undefined reference to `Ogrenci::sonNo'
```

Mesaj satır numarası vermez; `.cpp` dosyalarına bölerken özellikle sık görülür. Tanımı `.cpp` dosyasına yaz, başlık dosyasına değil; başlıkta yazarsan o başlığı çağıran her dosyada tanım tekrarlanır (`multiple definition`).

11.9 Nesne dizisi ve `vector<Sinif>`

Sınıf yazmanın asıl kazancı, Bölüm 7'deki paralel `vector`'lerden kurtulmaktır: üç `vector` yerine **tek bir** `vector<Urun>`. Nesne `vector`'ünü `int` `vector`'ü gibi kullanırsın; elemanlara ekleme `push_back` ile ya da yapıcı argümanlarını doğrudan veren `emplace_back` ile yapılır (nesneyi `vector`'ün içinde, yerinde oluşturur):

nesneVector.cpp

```cpp
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
using namespace std;

class Urun {
private:
    string ad;
    double fiyat;
    int adet;

public:
    Urun(string yeniAd, double yeniFiyat, int yeniAdet)
        : ad{yeniAd}, fiyat{yeniFiyat}, adet{yeniAdet} {}

    double deger() const { return fiyat * adet; }
    void zamYap(double yuzde) { fiyat += fiyat * yuzde / 100; }
};

double toplamDeger(const vector<Urun>& liste) {
    double toplam{0};
    for (const Urun& u : liste) {
        toplam += u.deger();
    }
    return toplam;
}

int main() {
    vector<Urun> liste;
    liste.push_back(Urun{"Defter", 25.0, 10});
    liste.emplace_back("Kalem", 7.5, 40);
    liste.emplace_back("Silgi", 4.0, 25);

    cout << fixed << setprecision(2);
    cout << "Toplam: " << toplamDeger(liste) << '\n';

    for (auto u : liste) {              // u, elemanın KOPYASI
        u.zamYap(10);
    }
    cout << "Kopyaya zam: " << toplamDeger(liste) << '\n';

    for (auto& u : liste) {             // u, elemanın kendisi
        u.zamYap(10);
    }
    cout << "Referansa zam: " << toplamDeger(liste) << '\n';
    return 0;
}
```

Çıktı

```
Toplam: 650.00
Kopyaya zam: 650.00
Referansa zam: 715.00
```

İlk zam döngüsü sonucu **hiç değiştirmedi**: `for (auto u : liste)` her elemanın bir **kopyasını** `u` yapar, `zamYap` kopyaya uygulanır, döngü bitince kopya atılır. Elemanı değiştireceksen `auto&`, yalnızca okuyacaksan `const auto&` (kopya maliyeti de yok) yaz. Bölüm 7'deki aralık tabanlı `for` kuralı nesnelerde de aynen geçerlidir.

Ham dizi de olur (`Urun u[3];`), ama dizideki her eleman için **varsayılan yapıcı** çağrılır; sınıfta yoksa derlenmez. `vector<Urun> v(3);` da aynı sebepten varsayılan yapıcı ister. `Urun` gibi varsayılan yapıcısı olmayan sınıf için ya elemanları tek tek `emplace_back` ile ekle ya da `Urun u[2]{Urun{"a", 1, 1}, Urun{"b", 2, 2}};` yaz.

11.10 Nesneyi fonksiyona geçirmek: const & ya da &

Nesneler `int`'ten büyüktür (içinde `string`, `vector` olabilir). Bölüm 6'daki kuralı sınıflara uygula:

- Fonksiyon nesneyi yalnızca **okuyacaksa**: `const Hesap&`. Kopya yapılmaz, değiştirme de engellenir.
- Fonksiyon nesneyi **değiştirecekse**: `Hesap&`.
- **Değer ile geçirme** (`Hesap h`) nesnenin kopyasını alır: ağır olabilir ve fonksiyonun yaptığı değişiklik çağıranın nesnesine ulaşmaz.

Bakiyesi 500 olan `a` için `void kopyayaYatir(Hesap h) { h.yatir(100); }` çağrısından sonra bakiye hâlâ 500'dür (değişen `h` kopyadır); `void gercegeYatir(Hesap& h)` aynı gövdeyle çağrılınca bakiye 600 olur. Soru 7'de bunu kendin izleyeceksin.

> **Sınavda Çıkar: Nesne parametresi**
>
> Soru kalıbı: "`void f(Hesap h) { h.yatir(100); }` çağrıldıktan sonra çağıranın bakiyesi ne olur?" Cevap: **değişmez**, çünkü `h` kopyadır. Referans (`Hesap&`) alsaydı değişirdi. Okuma amaçlı nesne parametresi için doğru yazım `const Hesap&`'dir; "bir nesneyi neden `const &` ile geçiririz?" sorusunun cevabı: **gereksiz kopyayı önlemek ve yanlışlıkla değiştirmeyi engellemek**. Not: sınıf nesnelerini `==` ile karşılaştıramazsın; bunun için operatör aşırı yükleme gerekir (Bölüm 13).

11.11 Sınıfı .h ve .cpp dosyalarına bölmek

Gerçek projelerde sınıf `main`'in üstüne yazılmaz. Sınıfın **bildirimi** (hangi üyeler var) bir **başlık dosyasına** (`.h`), üye fonksiyonların **tanımları** (gövdeler) bir **kaynak dosyasına** (`.cpp`) yazılır. `main` yalnızca başlığı `#include` eder; böylece sınıfı kullananlar gövdeleri görmek zorunda kalmaz ve bir gövde değişince yalnızca o `.cpp` yeniden derlenir. Üç dosyalı bir örnek (çalıştırılan blok değildir, kendi bilgisayarında dene):

```
// hesap.h
#ifndef HESAP_H
#define HESAP_H

#include <string>

class Hesap {
private:
    std::string sahip;
    double bakiye;
    static int adet;

public:
    Hesap(const std::string& yeniSahip, double ilkBakiye);
    void yatir(double miktar);
    double bakiyeAl() const;
    static int adetAl();
};

#endif
```

```cpp
// hesap.cpp
#include "hesap.h"

int Hesap::adet{0};                    // static üyenin tanımı

Hesap::Hesap(const std::string& yeniSahip, double ilkBakiye)
    : sahip{yeniSahip}, bakiye{ilkBakiye} {
    adet++;
}

void Hesap::yatir(double miktar) {
    if (miktar > 0) {
        bakiye += miktar;
    }
}

double Hesap::bakiyeAl() const { return bakiye; }      // const burada da
int Hesap::adetAl() { return adet; }                   // static yazılmaz
```

```cpp
// main.cpp
#include <iostream>
#include "hesap.h"
using namespace std;

int main() {
    Hesap a{"Ali", 100};
    Hesap b{"Ayşe", 40};
    a.yatir(50);
    cout << a.bakiyeAl() << " " << Hesap::adetAl() << '\n';
    return 0;
}
```

Derleme, üç dosyayı birlikte verince **derleyici** her `.cpp`'yi ayrı nesne dosyasına çevirir, **bağlayıcı** (linker) birleştirir:

```
g++ -std=c++17 -Wall -Wextra main.cpp hesap.cpp -o hesapProgrami
./hesapProgrami
```

Program `150 2` yazar (100 + 50 = 150 bakiye; iki hesap açıldı). Windows'ta çalıştırma komutu `hesapProgrami.exe`'dir. Dikkat edilecekler:

- **Koruma satırları** (`#ifndef HESAP_H` / `#define` / `#endif`, include guard): aynı başlık bir dosyaya iki yoldan girerse sınıf tanımı iki kez gelir ve "redefinition" hatası alırsın. Koruma, başlığın yalnızca bir kez işlenmesini sağlar (**ön işlemci** işi). Tek satırlık `#pragma once` de çalışır.
- **Başlıkta `using namespace std;` yazılmaz** (Bölüm 1): başlığı çağıran herkesin dosyasına bu satırı sokarsın. Bu yüzden başlıkta `std::string` yazdık. `.cpp` dosyasında istersen kullanabilirsin.
- Tanımda `const` bildirimdeki gibi tekrarlanır (`double Hesap::bakiyeAl() const`), `static` ise tekrarlanmaz; aksi hâlde "no declaration matches" gibi hatalar alırsın.
- Kendi başlığını `"hesap.h"` ile (tırnak), standart olanı `<string>` ile (açılı) çağır.
- Tanımda `Hesap::` önekini unutursan yazdığın şey sınıfın üyesi değil, **sıradan bir fonksiyondur**. Tanımsız üye fonksiyon ise bağlama hatasıdır; derleme komutuna `hesap.cpp`'yi yazmayı unutunca da aynısı çıkar. g++/MinGW mesajı (macOS'ta farklı görünür) şuna benzer:

```
undefined reference to `Hesap::yatir(double)'
```

11.12 enum class: adlandırılmış seçenekler

Bir hesabın durumu “aktif”, “pasif” ya da “dondurulmuş” olsun. `int` ile `0, 1, 2` tutmak okunmaz (sihirli sayı) ve `5` gibi anlamsız değerlere de izin verir. **`enum class`** (kapsamlı numaralandırma), adlandırılmış sabitlerden oluşan yeni bir tip tanımlar:

enumSinif.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

enum class Durum { Aktif, Pasif, Dondurulmus };

string durumAdi(Durum d) {
    switch (d) {
        case Durum::Aktif: return "aktif";
        case Durum::Pasif: return "pasif";
        case Durum::Dondurulmus: return "dondurulmus";
    }
    return "bilinmiyor";
}

int main() {
    Durum d{Durum::Pasif};
    cout << durumAdi(d) << ' ' << static_cast<int>(d) << '\n';
    d = Durum::Dondurulmus;
    cout << durumAdi(d) << ' ' << static_cast<int>(d) << '\n';
    return 0;
}
```

Çıktı

```
pasif 1
dondurulmus 2
```

Seçeneklere her zaman tip adıyla ulaşılır (`Durum::Aktif`); aynı adı başka bir `enum class` da kullanabilir. Seçenekler içeride `0, 1, 2` diye numaralanır, ama **otomatik `int`'e dönüşmezler**. Sayıya çevirmek istersen `static_cast<int>` yazarsın (Bölüm 2). C'den kalma eski `enum` (class'sız) seçenek adlarını doğrudan ortama döker ve `int`'e sessizce dönüşür; yeni kodda `enum class` kullan.

Dönüşümü zorlamaya çalışırsan (`int n{Durum::Pasif};`) derleyici hata verir; `cout << Durum::Aktif;` da derlenmez, çünkü `cout` bu tipi tanımaz.

Bölüm Özeti – Bilmen Gerekenler

- `struct` ile `class` farkı yalnızca **varsayılan erişimdir** (`struct`: `public`, `class`: `private`). **Sınıf** şablon, **nesne** ondan üretilen örnektir; üye değişken ve **üye fonksiyon** nesneye aittir, `nesne.uye` ya da `isaretci->uye` ile ulaşılır.
- **Kapsülleme**: üye değişkenler `private`, işlemler `public`. Erişimi **sınıf** belirler, nesne değil. Setter'ın değeri doğrulama yapabilmesidir; her üyeye mekanik get/set yazma.
- **Yapıcı**: adı sınıfla aynı, dönüş tipi yok, nesne doğarken çalışır. Kendi yapıcını yazdıysan **varsayılan yapıcı kaybolur** (`Kutu k;` derlenmez); `= default` onu geri getirir, ama üyelere ilk değer vermediysen çöp değer kalır. `Hesap h();` nesne değil fonksiyon bildirimidir.
- **Üye ilk değer listesi** (`: ad{a}, no{n}`): `const` ve referans üyeler için zorunludur. Başlatma sırası sınıftaki tanım sırasıdır.

- **Yıkıcı** (`~Sinif()`): yerel nesneler **ters sırayla** yok edilir; `new` ile açılan nesnede yıkıcı `delete` anında çalışır; global nesne `main`'den sonra yok edilir. Kaynağı yıkıcıda geri ver (RAII); kaynak tutan sınıfı kopyalama (Bölüm 13).
- `this` çağrılan nesneyi gösteren işaretçidir; `return *this` (`Sinif&`) zincirleme sağlar, `Sinif` döndürmek kopya demektir.
- `const` **üye fonksiyon** üyeleri değiştirmez; `const` nesne ve `const &` üzerinden yalnızca onlar çağrılır. Değiştirmeyen her üye fonksiyonu `const` yaz.
- `static` **üye**: bütün nesneler için tek kopya; veri üyesinin tanımı sınıf dışında bir kez yazılır (`int S::x{0};`, ya da C++17 `inline static`). `static` üye fonksiyonun `this`'i yoktur, `Sinif::f()` ile çağrılır.
- `vector<Sinif>`: `emplace_back(yapıcı argümanları)`. Elemanı değiştirmek için `auto&`; `auto` kopya üzerinde çalışır. Nesne parametresi: okuma için `const Sinif&`, değiştirme için `Sinif&`, değer ile geçirme kopyadır.
- `.h` bildirim (koruma satırları, `std::` önekli, `using namespace` yok), `.cpp` tanım (`Sinif::`, `const` tekrarlanır, `static` tekrarlanmaz); `g++ main.cpp hesap.cpp`. `enum class`: `Durum::Aktif`, `int`'e kendiliğinden dönüşmez, `static_cast<int>` gerekir.

Sorular

A) Çoktan Seçmeli

1 Aşağıdaki sınıf için hangi satır **derlenir**?

```
class Kutu {
public:
    int en;
    Kutu(int e) : en{e} {}
};
```

(A) `Kutu k;`
(B) `Kutu k{7};`
(C) `Kutu k{};`
(D) `Kutu dizi[3];`
(E) `Kutu k = Kutu();`

2 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
using namespace std;

class Sayac {
public:
    int sayi{0};
    Sayac artirDeger() { sayi++; return *this; }
    Sayac& artirRef() { sayi++; return *this; }
};

int main() {
    Sayac s;
    s.artirDeger().artirDeger().artirDeger();
    cout << s.sayi << ' ';
    Sayac t;
    t.artirRef().artirRef().artirRef();
    cout << t.sayi;
    return 0;
}
```

A) `3 3` B) `1 1` C) `3 1` D) `1 3` E) `0 3`

3 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
using namespace std;

class Sayac {
public:
    static int adet;
    Sayac() { adet++; }
    ~Sayac() { adet--; }
};
int Sayac::adet{0};

int main() {
    Sayac a;
    {
        Sayac b;
        Sayac c;
    }
    Sayac d;
    cout << Sayac::adet;
    return 0;
}
```

A) `0` B) `1` C) `2` D) `3` E) `4`

4 Aşağıdaki sınıf ve fonksiyon için hangi satır **derleme hatası** verir?

```
class Hesap {
private:
    double bakiye{0};
public:
    void yatir(double m) { bakiye += m; }
    double bakiyeAl() const { return bakiye; }
};

void f(const Hesap& h, Hesap& g) {
    // ... buraya aşağıdaki şıklardan biri yazılıyor
}
```

A `h.yatir(5);`

B `g.yatir(5);`

C `cout << h.bakiyeAl();`

D `cout << g.bakiyeAl();`

E `const Hesap& r{g}; cout << r.bakiyeAl();`

5 Aşağıdakilerden hangisi **doğrudur**?

```
enum class Renk { Kirmizi, Yesil, Mavi };
Renk r{Renk::Yesil};
```

A `int n{r};` satırı derlenir ve `n` 1 olur.

B `cout << r;` satırı `1` yazdırır.

C `if (r == 1)` ifadesi derlenir.

D `Renk s = 1;` satırı derlenir ve `s`, `Yesil` olur.

E `int n{static_cast<int>(r)};` satırı derlenir ve `n` 1 olur.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
#include <iostream>
#include <string>
using namespace std;

struct Iz {
    string ad;
    Iz(string a) : ad{a} { cout << "+" << ad << '\n'; }
    ~Iz() { cout << "-" << ad << '\n'; }
};

struct Kutu {
    Iz ic;
    Kutu() : ic{"ic"} { cout << "+kutu\n"; }
    ~Kutu() { cout << "-kutu\n"; }
};

Iz genel{"genel"};

int main() {
    cout << "basla\n";
    Kutu k;
    Iz* p{new Iz{"oge"}};
    {
        Iz y{"yerel"};
    }
    delete p;
    cout << "bitis\n";
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
using namespace std;

class Sayac {
private:
    int sayi{0};
    static int toplam;

public:
    void artir() { sayi++; toplam++; }
    int sayiAl() const { return sayi; }
    static int toplamAl() { return toplam; }
};

int Sayac::toplam{0};

void kopyaArtir(Sayac s) { s.artir(); }
void gercekArtir(Sayac& s) { s.artir(); }
void yazdir(const Sayac& s) { cout << s.sayiAl() << ' '; }

int main() {
    Sayac x;
    kopyaArtir(x);
    yazdir(x);
    gercekArtir(x);
    gercekArtir(x);
    yazdir(x);
    cout << Sayac::toplamAl() << '\n';
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Oyuncu {
private:
    string ad;
    int puan;

public:
    Oyuncu(string a, int p) : ad{a}, puan{p} {}
    void puanEkle(int n) { puan += n; }
    int puanAl() const { return puan; }
};

int main() {
    vector<Oyuncu> takim;
    takim.emplace_back("Ali", 10);
    takim.emplace_back("Ece", 25);
    takim.emplace_back("Can", 15);

    for (auto o : takim) {
        o.puanEkle(100);
    }
    for (auto& o : takim) {
        if (o.puanAl() < 20) {
            o.puanEkle(5);
        }
    }
    for (const Oyuncu& o : takim) {
        cout << o.puanAl() << ' ';
    }
    cout << '\n';
    return 0;
}
```

C) Kod Yazma

9 `BankaHesabi` adlı bir sınıf yaz. `private` üyeleri `sahip` (`string`) ve `bakiye` (`double`) olsun. Yapıcı sahibi ve ilk bakiyeyi üye ilk değer listesiyle alsın; ilk bakiye negatifse `0` yapsın. `yatir(double)` yalnızca pozitif tutarı kabul etsin. `bool cek(double)` tutar pozitif ve bakiyeyi aşmıyorsa parayı düşüp `true`, aksi hâlde `false` döndürsün. `sahipAl()` ve `bakiyeAl()` `const` olsun.

`main` şunları yapsın: klavyeden sahip adını (tek sözcük), ilk bakiyeyi, yatırılacak tutarı ve çekilecek tutarı oku; hesabı oluştur, yatır, çekmeyi dene (başarısızsa `Yetersiz bakiye.` yaz). Sonra hesabı `Ad: bakiye TL` biçiminde (iki ondalıkla) yazdır.

```
Sahip, ilk bakiye, yatirilacak, cekilecek: Ayse 500 250 900
Yetersiz bakiye.
Ayse: 750.00 TL
```

10 Bir `Dikdortgen` sınıfı yaz. Yapıcı `en` ve `boy` alsın; sıfır ya da negatif gelen kenarı `1` yapsın. `alan()`, `cevre()` ve `kareMi() const` üye fonksiyon olsun. `main` önce `n` sayısını, sonra `n` tane `en boy` çiftini okuyup `vector<Dikdortgen>` içinde saklasın; her dikdörtgenin alanını ve çevresini yazdırsın, en büyük alanı ve kare sayısını bulsun.

```
Kac dikdortgen? 3
3 4
5 5
6 6
Dikdortgen 1: alan=12 cevre=14
Dikdortgen 2: alan=25 cevre=20
Dikdortgen 3: alan=36 cevre=24
En buyuk alan: 36
Kare sayisi: 2
```

Çözümler

Cevap anahtarı

1	2	3	4	5
B	D	C	A	E

1 Cevap: B

`Kutu(int)` yazdığımız için derleyici varsayılan yapıcıyı **üretmez**. `Kutu k{7};` argümanı olan yapıcıyı çağırır, derlenir. A, C, D ve E parametresiz yapıcı ister: `Kutu k;`, `Kutu k{};`, dizinin her elemanı (`dizi[3]`) ve `Kutu()` ifadesi. Hepsi "no matching function" hatası verir. Çözüm `Kutu() : en{0} {}` gibi bir yapıcı eklemektir; ya da üyeye `int en{0};` diye ilk değer verip `Kutu() = default;` yazabilirsin.

2 Cevap: D

`artirDeger()` dönüş tipi `Sayac` olduğu için `return *this;` **kopya** döndürür. Zincirde yalnızca ilk çağrı `s`'yi artırır (`s.sayi` 1 olur); ikinci ve üçüncü çağrılar kopyalar üzerinde çalışır. `artirRef()` ise `Sayac&` döndürür, üç çağrı da `t`'ye uygulanır (3). Çıktı `1 3`. A iki zincirin de üç artış yaptığını sanar. B iki zincirin de yalnızca bir artış yaptığını sanar. C, ilk zincirin üç artışını `s`'ye, ikincinin tek artışını `t`'ye yaptığını sanar. E, değer döndüren çağrının `s`'yi hiç artırmadığını sanar.

3 Cevap: C

`adet` sınıfın **tek** değişkenidir. `a` doğar: 1. İç blokta `b` ve `c`: 3. Blok biter, ikisinin yıkıcısı çalışır: 1. `d` doğar: 2. `cout` anında `a` ve `d` yaşıyor: **2**. A (0) ve B (1), sayacın gereğinden fazla düştüğünü sanar. D (3) iç bloktaki iki nesneden birinin, E (4) ikisinin de hâlâ yaşadığını sanar.

4 Cevap: A

`h`, `const Hesap&` olduğu için üzerinden yalnızca `const` üye fonksiyon çağrılabilir. `yatir` `const` değildir: A hata verir. B'de `g` `const` olmayan referanstır, `yatir` çağrılabilir. C ve D'de `bakiyeAl()` `const`'tır, ikisinden de çağrılır. E'de `g`'den `const` bir referans oluşturmak serbesttir (yetki daraltılabilir); `r.bakiyeAl()` `const` olduğu için çalışır.

5 Cevap: E

`enum class` değerleri **kendiliğinden `int`'e dönüşmez**; `static_cast<int>(r)` ile açıkça çevrilir ve `Yesil` 1 numaradır (`Kirmizi` 0). A (`int n{r};`) ve C (`r == 1`) örtük dönüşüm gerektirir, derlenmez. B: `cout << r` için `Renk` türünün `<<` operatörü yoktur, derlenmez (sayıya çevirmen gerekir). D: `Renk s = 1;` tamsayıdan `enum class`'a örtük dönüşüm yoktur, derlenmez.

6

Sırayla izle:

1. `genel` global nesnesi `main`'den **önce** oluşur: `+genel`.
2. `main` başlar: `basla`.
3. `Kutu k`: önce üyeler oluşur (`ic`), sonra `Kutu`'nun gövdesi çalışır: `+ic`, `+kutu`.
4. `new Iz{"oge"}`: `+oge`. İç blokta `y` oluşur ve blok biter bitmez yok edilir: `+yerel`, `-yerel`.
5. `delete p`: `-oge`. Sonra `bitis`.
6. `main` biterken `k` yok edilir: önce `Kutu`'nun yıkıcısı, **sonra** üyeler: `-kutu`, `-ic`. En son global nesne: `-genel`.

Yapıcılarda içten dışa, yıkıcılarda dıştan içe gidilir.

Çıktı

```
+genel
basla
+ic
+kutu
+oge
+yerel
-yerel
-oge
bitis
-kutu
-ic
-genel
```

7

Tuzak, `toplam`'ın `static` olması: kopyaya yapılan artış da **sınıfın** sayacını artırır; ama kopyanın `sayi`'si çağıranı etkilemez.

`kopyaArtir(x)` sonrası `x.sayi` 0, `toplam` 1; her `gercekArtir(x)` ikisini de birer artırır (`x.sayi` 2, `toplam` 3).

`yazdir(x)` ilk seferde `0`, ikincisinde `2` yazar; sonda `toplamAl()` 3 döndürür.

Çıktı

```
0 2 3
```

8

İlk döngüde `auto o` kopyadır: `+100` kopyalara gider, `takim` aynı kalır (`10, 25, 15`). İkinci döngüde `auto&` asıl elemanı verir ve 20'den küçüklere 5 eklenir: Ali 15, Ece 25 (değişmez), Can 20. Son döngü `15 25 20` yazar.

Çıktı

```
15 25 20
```

9

Yaklaşım: üyeleri sınıf içindeki sırayla (`sahip`, `bakiye`) listede başlat; `cek` önce tutarı kontrol edip `bool` döndürsün.

bankaHesabi.cpp

```cpp
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

class BankaHesabi {
private:
    string sahip;
    double bakiye;

public:
    BankaHesabi(string yeniSahip, double ilkBakiye)
        : sahip{yeniSahip}, bakiye{ilkBakiye >= 0 ? ilkBakiye : 0} {
    }

    void yatir(double miktar) {
```

```
        if (miktar > 0) {
            bakiye += miktar;
        }
    }

    bool cek(double miktar) {
        if (miktar <= 0 || miktar > bakiye) {
            return false;
        }
        bakiye -= miktar;
        return true;
    }

    string sahipAl() const { return sahip; }
    double bakiyeAl() const { return bakiye; }
};

int main() {
    string ad;
    double ilk{0};
    double yatirilacak{0};
    double cekilecek{0};
    cout << "Sahip, ilk bakiye, yatirilacak, cekilecek: ";
    cin >> ad >> ilk >> yatirilacak >> cekilecek;

    BankaHesabi hesap{ad, ilk};
    hesap.yatir(yatirilacak);
    if (!hesap.cek(cekilecek)) {
        cout << "Yetersiz bakiye.\n";
    }

    cout << fixed << setprecision(2);
    cout << hesap.sahipAl() << ": " << hesap.bakiyeAl() << " TL\n";
    return 0;
}
```

Çıktı

```
Sahip, ilk bakiye, yatirilacak, cekilecek: Ayse 500 250 900
Yetersiz bakiye.
Ayse: 750.00 TL
```

500 + 250 = 750 olur; 900 çekilemez (`cek` `false` döndürür), bakiye değişmez.

10

Yaklaşım: yapıcıda her kenarı koşullu ifadeyle düzelt; `vector`'e `emplace_back` ile ekle; en büyük alan ve kare sayısı için tek geçiş yeter.

dikdortgenler.cpp

```
#include <iostream>
#include <vector>
using namespace std;

class Dikdortgen {
private:
    int en;
    int boy;

public:
    Dikdortgen(int e, int b) : en{e > 0 ? e : 1}, boy{b > 0 ? b : 1} {}
```

```
    int alan() const { return en * boy; }
    int cevre() const { return 2 * (en + boy); }
    bool kareMi() const { return en == boy; }
};

int main() {
    int n{0};
    cout << "Kac dikdortgen? ";
    cin >> n;

    vector<Dikdortgen> liste;
    for (int i{0}; i < n; i++) {
        int en{0};
        int boy{0};
        cin >> en >> boy;
        liste.emplace_back(en, boy);
    }

    int enBuyukAlan{0};
    int kareSayisi{0};
    for (int i{0}; i < static_cast<int>(liste.size()); i++) {
        cout << "Dikdortgen " << i + 1 << ": alan=" << liste[i].alan()
             << " cevre=" << liste[i].cevre() << '\n';
        if (liste[i].alan() > enBuyukAlan) {
            enBuyukAlan = liste[i].alan();
        }
        if (liste[i].kareMi()) {
            kareSayisi++;
        }
    }
    cout << "En buyuk alan: " << enBuyukAlan << '\n';
    cout << "Kare sayisi: " << kareSayisi << '\n';
    return 0;
}
```

Çıktı

```
Kac dikdortgen? 3
3 4
5 5
6 6
Dikdortgen 1: alan=12 cevre=14
Dikdortgen 2: alan=25 cevre=20
Dikdortgen 3: alan=36 cevre=24
En buyuk alan: 36
Kare sayisi: 2
```

Alanlar 12, 25, 36; en büyüğü 36. `5x5` ve `6x6` kare olduğu için kare sayısı 2’dir.

BÖLÜM 12

Kalıtım ve Polimorfizm

Bu Bölümde Öğreneceklerin

- Bir sınıfın, başka bir sınıfın özelliklerini nasıl devraldığını
- Bir sınıfın üyelerinin kime açık olduğunu: herkese mi, yalnızca devralan sınıfa mı
- Bir nesne yaratılırken ve yok edilirken yapıcıların ve yıkıcıların hangi sırayla çalıştığını
- Devralan sınıfın, devraldığı bir işi kendi işiyle nasıl değiştirdiğini
- Aynı satırın, nesnenin gerçekte ne olduğuna göre farklı işler yapabilmesini
- Farklı türden nesneleri tek bir listede bir arada tutmayı
- Nesnesi üretilemeyen, yalnızca temel olarak kullanılan sınıfları
- Temel sınıfın yıkıcısının neden "sanal" (virtual) yapılması gerektiğini
- Nesne dilimleme tuzağını ve ondan nasıl korunacağını

12.1 Kalıtım nedir?

Bölüm 11'de sınıfları yazdık: veriyi ve o veriyle çalışan fonksiyonları bir paket yaptık. Gerçek programlarda birbirine çok benzeyen sınıflar çıkar. Bir hayvan barınağı programı düşün: `Kedi` sınıfında ad, yaş, `uyu()` ve `miyavla()` var; `Kopek` sınıfında ad, yaş, `uyu()` ve `havla()` var. Ad, yaş ve `uyu()` iki sınıfta da aynı. Bunları iki yere kopyalarsan bir hatayı iki yerde düzeltmen gerekir; üçüncü bir hayvan eklediğinde üç yerde.

Kalıtım (inheritance), bir sınıfın başka bir sınıfın üyelerini **devralmasıdır**. Ortak üyeleri bir kez yazdığın sınıfa **temel sınıf** (base class), onu devralan sınıfa **türetilmiş sınıf** (derived class) denir. Türetilmiş sınıf temel sınıfın her şeyine sahiptir, üstüne kendi üyelerini ekler. Kalıtımın kullanılacağı yeri bulmak için tek bir test yeterli: "**X bir Y'dir**" cümlesi doğru mu? Kedi bir hayvandır, öyleyse `Kedi`, `Hayvan`'dan türetilebilir. Araba bir motor değildir, motora sahiptir; orada kalıtım değil, üye değişken kullanırsın.

kalitimIlk.cpp

```
#include <iostream>
#include <string>

using namespace std;

class Hayvan {
public:
    Hayvan(string adi) : ad{adi} {}

    void tanit() const { cout << ad << " bir hayvan.\n"; }
    void uyu() const { cout << ad << " uyuyor.\n"; }

protected:
    string ad;
};

class Kedi : public Hayvan {
public:
    Kedi(string adi) : Hayvan{adi} {}
```

```
    void miyavla() const { cout << ad << ": Miyav!\n"; }
};

class Kopek : public Hayvan {
public:
    Kopek(string adi) : Hayvan{adi} {}

    void havla() const { cout << ad << ": Hav hav!\n"; }
};

int main() {
    Kedi kedi{"Pamuk"};
    Kopek kopek{"Karabaş"};
    kedi.tanit();
    kedi.miyavla();
    kopek.uyu();
    kopek.havla();
    return 0;
}
```

Çıktı

```
Pamuk bir hayvan.
Pamuk: Miyav!
Karabaş uyuyor.
Karabaş: Hav hav!
```

Yeni olan satırlar: `class Kedi : public Hayvan` ifadesi "`Kedi`, `Hayvan`'dan türetilmiştir" demektir. `Kedi` içinde `tanit()` ve `uyu()` yazılmadı ama çalışıyor; `miyavla()` ise `Hayvan`'dan devralınan `ad` değişkenini kullanıyor. `Hayvan{adi}` ise temel sınıfın yapıcısının çağrılmasıdır (aşağıda anlatılıyor). Yeni bir `Kus` sınıfı için ad ve `uyu()` kodunu tekrar yazmazsın; `class Kus : public Hayvan` yazıp yalnızca kuşa özgü şeyleri eklersin.

> **İpucu: Kalıtımdan önce "bir mi, sahip mi?" diye sor**
>
> "X bir Y'dir" cümlesi doğruysa kalıtım (**is-a**), "X bir Y'ye sahiptir" cümlesi doğruysa üye değişken (**has-a**) kullan. `Daire` bir `Sekil`'dir (kalıtım); `Araba` bir `Motor`'a sahiptir (üye değişken). Yalnızca birkaç satır kod tekrarlamamak için kalıtım kurma. Kalıtım iki sınıfı sıkı bağlar; temel sınıfı değiştirdiğinde bütün türetilmiş sınıflar etkilenir.

12.2 Erişim: `public`, `protected`, `private`

Bölüm 11'de `public` ve `private` erişim belirteçlerini (access specifier) gördün. Kalıtımla birlikte üçüncüsü devreye girer: **`protected`** (korumalı). `protected` üye dışarıdan erişilemez, ama **türetilmiş sınıflar** ona erişebilir.

Üyenin erişimi	Kendi sınıfı	Türetilmiş sınıf	Sınıfın dışı (`main` vb.)
`public`	evet	evet	evet
`protected`	evet	evet	**hayır**
`private`	evet	**hayır**	**hayır**

Yani türetilmiş sınıf temel sınıfın `private` üyelerini **devralır** (nesnenin içinde dururlar) ama onlara doğrudan **erişemez**. Temel sınıfın `public` ya da `protected` fonksiyonları üzerinden dolaylı yoldan ulaşabilir.

```
class Hayvan {
public:
    Hayvan() : kilo{4}, yas{3} {}
```

```
protected:
    int kilo;

private:
    int yas;
};

class Kedi : public Hayvan {
public:
    int yasVer() const { return yas; }
    int kiloVer() const { return kilo; }
};

int main() {
    Kedi kedi;
    return kedi.kilo;
}
```

Derleyici çıktısı – program derlenmez

```
erisimHata.cpp: In member function 'int Kedi::yasVer() const':
erisimHata.cpp:14:33: error: 'int Hayvan::yas' is private within this context
   14 |     int yasVer() const { return yas; }
      |                                 ^~~
erisimHata.cpp:9:9: note: declared private here
    9 |     int yas;
      |         ^~~
erisimHata.cpp: In function 'int main()':
erisimHata.cpp:20:17: error: 'int Hayvan::kilo' is protected within this context
   20 |     return kedi.kilo;
      |                 ^~~~
erisimHata.cpp:6:9: note: declared protected here
    6 |     int kilo;
      |         ^~~~
```

Derleyici iki hata verdi: `yas`, `Hayvan`'ın `private` üyesi olduğu için `Kedi` içinden okunamıyor; `kilo` ise `protected` olduğu için `Kedi` içinde serbest (`kiloVer` derleniyor), ama `main` içinde `kedi.kilo` yazılamıyor.

Gerçek tasarımda veri üyelerini genellikle `private` tutar, türetilmiş sınıfların ihtiyacını `protected` fonksiyonlarla karşılarsın (Bölüm 11'deki kapsülleme). Biz örnekleri kısa tutmak için bazı üyeleri `protected` yaptık.

Kalıtımın türü: neden `public`?

`class Kedi : public Hayvan` satırındaki `public`, kalıtımın **türünü** belirler. C++'ta `public`, `protected` ve `private` kalıtım vardır; biz yalnızca "Kedi bir hayvandır" ilişkisini kuran `public` kalıtımı kullanacağız. Çok yapılan hata: `class` ile tanımlanan sınıfta türü **yazmazsan** varsayılan `private` olur.

> **Sık Yapılan Hata: `public` yazmayı unutmak**
>
> `class Kedi : Hayvan` yazarsan kalıtım `private` olur (struct'ta ise `public`). Bu durumda `Kedi` nesnesini `Hayvan*` türünde bir işaretçiye atayamazsın, `kedi.tanit()` bile dışarıdan çağrılamaz; çünkü "Kedi bir Hayvan'dır" bilgisi dışarıdan gizlenmiştir.
>
> Derleyici "`Hayvan` erişilemez bir temel sınıftır" ya da benzeri bir şey söyler; sebebi her zaman aynıdır: `public` eksik.

> **Not: Derleyici mesajları**
>
> Kitaptaki hata mesajları gerçek g++ ile üretildi. macOS'ta `g++` yazınca çalışan program aslında Apple clang'dir; hata metinleri farklı çıkabilir, ama hatanın nedeni ve satırı aynıdır.

12.3 Yapıcı ve yıkıcı sırası

Türetilmiş bir nesne yaratıldığında içinde bir temel sınıf parçası da vardır. Bu parçanın da yapıcısı çalışmalıdır; çünkü `ad` gibi üyelerin ilk değerini o verir. Kural şu:

1. Önce **temel sınıfın** yapıcısı çalışır (yukarıdan aşağı, en üstteki sınıftan başlayarak).
2. Sonra türetilmiş sınıfın **üye nesnelerinin** yapıcıları çalışır.
3. En son **türetilmiş sınıfın** yapıcı gövdesi çalışır.

Yıkıcılar tam **ters sırayla** çalışır: önce türetilmiş sınıfın yıkıcısı, sonra üye nesneler, en son temel sınıfın yıkıcısı. Bir evi inşa ederken önce temeli atar, sonra duvarları örersin; yıkarken önce duvarları yıkar, temeli en son bırakırsın.

yapiciSirasi.cpp

```cpp
#include <iostream>

using namespace std;

class Motor {
public:
    Motor() { cout << "  Motor yapıcı\n"; }
    ~Motor() { cout << "  Motor yıkıcı\n"; }
};

class Arac {
public:
    Arac() { cout << "  Arac yapıcı\n"; }
    ~Arac() { cout << "  Arac yıkıcı\n"; }
};

class Araba : public Arac {
public:
    Araba() { cout << "  Araba yapıcı\n"; }
    ~Araba() { cout << "  Araba yıkıcı\n"; }

private:
    Motor motor;
};

int main() {
    cout << "bloğa giriliyor\n";
    {
        Araba araba;
        cout << "  blok içinde\n";
    }
    cout << "blok bitti\n";
    return 0;
}
```

Çıktı

```
bloğa giriliyor
  Arac yapıcı
  Motor yapıcı
  Araba yapıcı
  blok içinde
  Araba yıkıcı
  Motor yıkıcı
  Arac yıkıcı
blok bitti
```

`Araba` yapıcısına hiçbir şey yazmadığımız hâlde önce `Arac` yapıcısı çalıştı: Derleyici temel sınıfın **varsayılan yapıcısını** kendiliğinden çağırır. Sonra üye nesne `motor` yaratıldı, en son `Araba`'nın kendi gövdesi çalıştı. Blok bitince sıra tersine döndü. Süslü parantezli blok, `araba` nesnesinin ömrünü sınırlıyor; `araba` blok sonunda yok edilir (Bölüm 11'deki yığın nesnesi ömrü).

> **Sınavda Çıkar: Yapıcı sırası: içten dışa, yıkıcı sırası: dıştan içe**
>
> Sınavlarda sıradan bir `Cocuk` nesnesi için çıktı sorulur. Ezberlenecek kural: **Yapıcılar temel sınıftan türetilmişe, yıkıcılar türetilmişten temele.** Sınıfın üye nesneleri varsa, onlar temel sınıf yapıcısından sonra, kendi gövdesinden önce yaratılır ve ters sırada yok edilir. Üye nesneler tanımlandıkları sırayla yaratılır (üye ilk değer listesindeki yazım sırasıyla değil).

Temel sınıf yapıcısına argüman geçirmek

Temel sınıfın yapıcısı parametre alıyorsa türetilmiş sınıf, bu argümanları **üye ilk değer listesinde** (member initializer list) kendisi vermelidir. `Hayvan{adi}` yazdığımız yer burasıydı.

tabanYapici.cpp

```cpp
#include <iostream>
#include <string>

using namespace std;

class Calisan {
public:
    Calisan(string adi, double aylik) : ad{adi}, maas{aylik} {}

    void yazdir() const { cout << ad << ": " << maas << '\n'; }

protected:
    string ad;
    double maas;
};

class Mudur : public Calisan {
public:
    Mudur(string adi, double aylik, int ekip)
        : Calisan{adi, aylik}, ekipSayisi{ekip} {}

    void ekipYazdir() const { cout << ad << ", ekip: " << ekipSayisi << '\n'; }

private:
    int ekipSayisi;
};

int main() {
    Mudur mudur{"Ayşe", 52000, 8};
    mudur.yazdir();
    mudur.ekipYazdir();
    return 0;
}
```

Çıktı

```
Ayşe: 52000
Ayşe, ekip: 8
```

`Mudur` yapıcısı, `adi` ve `aylik` değerlerini `Calisan{adi, aylik}` ile yukarı iletti. `yazdir()` fonksiyonunu `Mudur` içinde yazmadık; `Calisan`'dan geliyor.

Sık Yapılan Hata: Temel sınıfın yapıcısını çağırmayı unutmak

Temel sınıfta **varsayılan yapıcı yoksa** (yani her yapıcı parametre istiyorsa) türetilmiş sınıf, üye ilk değer listesinde temel sınıfın yapıcısını çağırmak zorundadır. Çağırmazsan derleyici varsayılan yapıcıyı arar, bulamaz ve program derlenmez.

Örneğin `Taban` yalnızca `Taban(int x)` yapıcısına sahipse `Alt() {}` yazmak derleme hatasıdır ("no matching function for call to `Taban::Taban()`"). Çözüm: `Alt() : Taban{0} {}`.

12.4 Fonksiyonu yeniden tanımlama

Türetilmiş sınıf, temel sınıftan gelen bir fonksiyonu **kendi sürümüyle değiştirmek** isteyebilir. Kedinin `sesCikar()` fonksiyonu "..." yazmasın, "Miyav" yazsın. Bunun için türetilmiş sınıfta **aynı adı ve aynı parametreleri** taşıyan bir fonksiyon yazarsın. Buna fonksiyonu **yeniden tanımlama** (redefinition) denir. Fonksiyon `virtual` değilse yeniden tanımlama, temel sınıf fonksiyonunu **gizler** (hide).

yenidenTanimlama.cpp

```cpp
#include <iostream>

using namespace std;

class Hayvan {
public:
    void sesCikar() const { cout << "...\n"; }
};

class Kedi : public Hayvan {
public:
    void sesCikar() const { cout << "Miyav\n"; }
};

int main() {
    Kedi kedi;
    kedi.sesCikar();
    kedi.Hayvan::sesCikar();

    Hayvan* hayvan = &kedi;
    hayvan->sesCikar();
    return 0;
}
```

Çıktı

```
Miyav
...
...
```

Üç çağrı, iki ayrı sonuç verdi:

- `kedi.sesCikar()`: `kedi` değişkeninin tipi `Kedi`, bu yüzden `Kedi`'nin sürümü çalıştı.
- `kedi.Hayvan::sesCikar()`: Fonksiyon adının başına `Hayvan::` yazarak temel sınıfın sürümünü **açıkça** çağırdın. Türetilmiş sınıfın kendi sürümü, temel sınıfınkini genişletmek isterse bu kalıbı kullanır: `Hayvan::sesCikar();`.
- `hayvan->sesCikar()`: Burada `hayvan`, `Hayvan*` tipinde bir işaretçi (Bölüm 9), ama gösterdiği nesne bir `Kedi`. Yine de `Hayvan`'ın sürümü çalıştı, çünkü fonksiyon `virtual` değil. Derleyici sürümü **işaretçinin tipine** bakarak, daha program çalışmadan seçti.

Üçüncü satır önemli: Temel sınıf işaretçisiyle türetilmiş nesne gösterebilirsin (bir kedi bir hayvandır); buna **yukarı dönüşüm** (upcasting) denir ve cast istemez. Ama `virtual` olmadan yalnızca temel sınıfın bildiği fonksiyonlar, temel sınıfın sürümleriyle çalışır. Biz ise tersini istiyoruz: Nesne kediyse, işaretçi `Hayvan*` olsa bile kedi miyavlasın.

12.5 `virtual`, `override` ve dinamik bağlama

Temel sınıftaki fonksiyonun başına `virtual` yazarsan o fonksiyon **sanal fonksiyon** (virtual function) olur. Türetilmiş sınıf onu aynı imzayla yeniden yazdığında (`override`, ezme), çağrıyı hangi sürümün karşılayacağına derleyici değil, program **çalışırken nesnenin gerçek tipi** karar verir. Buna **dinamik bağlama** (dynamic binding) denir. Çağrının daha derleme sırasında, işaretçi ya da referans tipine göre bağlanmasına ise **statik bağlama** (static binding) denir.

dinamikBaglama.cpp

```cpp
#include <iostream>

using namespace std;

class Hayvan {
public:
    virtual void sesCikar() const { cout << "...\n"; }
};

class Kedi : public Hayvan {
public:
    void sesCikar() const override { cout << "Miyav\n"; }
};

class Kopek : public Hayvan {
public:
    void sesCikar() const override { cout << "Hav hav\n"; }
};

void konustur(const Hayvan& hayvan) {
    hayvan.sesCikar();
}

int main() {
    Hayvan hayvan;
    Kedi kedi;
    Kopek kopek;
    konustur(hayvan);
    konustur(kedi);
    konustur(kopek);

    Hayvan* isaretci = &kopek;
    isaretci->sesCikar();
    isaretci = &kedi;
    isaretci->sesCikar();
    return 0;
}
```

Çıktı

```
...
Miyav
Hav hav
Hav hav
Miyav
```

`konustur` fonksiyonunun parametresi `const Hayvan&`. Ona üç ayrı nesne verdik ve her seferinde o nesnenin kendi `sesCikar()` sürümü çalıştı. Aynı şekilde `isaretci` önce bir köpeği, sonra bir kediyi gösterirken `isaretci->sesCikar()` satırı hiç değişmediği hâlde iki farklı sonuç verdi. İşte **çok biçimlilik** (polymorphism): Aynı çağrı, nesnenin gerçek tipine göre farklı davranır. `konustur` yarın yazacağın `Kus` sınıfıyla da, **tek satır değişmeden** çalışacak; bu yüzden nesneye yönelik programlamanın en güçlü aracı sayılır.

Kodda iki şey daha var:

- `const` (fonksiyon parantezinden sonraki): Fonksiyonun nesneyi değiştirmeyeceğini söyler. Bu, fonksiyon imzasının bir parçasıdır; türetilmiş sınıfta ezerken de aynen yazılmalıdır.
- `override`: Fonksiyon parantezinin ve `const`'un ardından yazılır, derleyiciye "bu fonksiyon temel sınıftaki bir sanal fonksiyonu eziyor, kontrol et" der. Aşağıda neden vazgeçilmez olduğunu göreceksin.

Statik tip ve dinamik tip

`Hayvan* isaretci = &kopek;` satırında aynı nesne için iki ayrı tip var:

Kavram	Örnekte	Neyi belirler?
Statik tip (değişkenin tanımlı tipi)	`Hayvan*`	Hangi üyeleri **çağırabileceğini**, derleme zamanında
Dinamik tip (nesnenin gerçek tipi)	`Kopek`	Sanal bir fonksiyonun **hangi sürümünün** çalışacağını, çalışma zamanında

Derleyici yalnızca statik tipe bakar. `Hayvan`'da olmayan bir fonksiyonu, nesne gerçekten kedi olsa bile `Hayvan*` üzerinden çağıramazsın:

Örneğin `Kedi`'ye `tirmala()` eklediysen, `Hayvan* hayvan = &kedi;` ile `hayvan->tirmala()` yazamazsın: Derleyici "`Hayvan` sınıfında `tirmala` adlı üye yok" der. Çözüm, fonksiyonu `Hayvan`'a da (sanal olarak) eklemek ya da nesneyi `Kedi` tipiyle kullanmaktır.

> **Sınavda Çıkar: Dinamik bağlamanın üç şartı**
>
> Bir çağrının dinamik bağlanması için **üç şart birlikte** gerekir. Biri eksikse çağrı statik bağlanır, yani temel sınıfın sürümü çalışır:
>
> 1. Fonksiyon temel sınıfta **`virtual`** olarak tanımlanmış olmalı.
> 2. Türetilmiş sınıf onu **aynı imzayla** (aynı ad, aynı parametreler, aynı `const`) ezmiş olmalı.
> 3. Çağrı bir **işaretçi ya da referans** üzerinden yapılmalı. Nesnenin kendisi (değer) üzerinden yapılan çağrı hep statik bağlanır.
>
> Sınav sorusunda önce şunu sor: "Fonksiyon `virtual` mı?" Cevap hayırsa gerisini düşünme; işaretçinin tipine bak.

Neden `override` yazmalı?

`override` zorunlu değildir; `virtual` bir fonksiyonu ezmek için türetilmiş sınıfta yalnızca aynı imzayı yazman yeter. Ama imzayı **yanlış** yazarsan (örneğin `const`'u unutursan) derleyici bunu yeni, ayrı bir fonksiyon sayar. Ezme gerçekleşmez, temel sınıfın sürümü çalışmaya devam eder ve **hata mesajı gelmez** (`g++ -Wall` ile yalnızca bir uyarı verir). `override` yazarsan derleyici "ezdiğin bir şey yok" diye hata verir:

overrideHata.cpp

```cpp
#include <iostream>

using namespace std;

class Hayvan {
public:
    virtual void sesCikar() const { cout << "...\n"; }
};

class Kedi : public Hayvan {
public:
    void sesCikar() override { cout << "Miyav\n"; }
};

int main() {
    Kedi kedi;
    kedi.sesCikar();
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
overrideHata.cpp:12:10: error: 'void Kedi::sesCikar()' marked 'override', but does not override
   12 |     void sesCikar() override { cout << "Miyav\n"; }
      |          ^~~~~~~~
overrideHata.cpp:7:18: warning: 'virtual void Hayvan::sesCikar() const' was hidden [-Woverloaded-
virtual=]
    7 |     virtual void sesCikar() const { cout << "...\n"; }
      |                  ^~~~~~~~
overrideHata.cpp:12:10: note:   by 'void Kedi::sesCikar()'
   12 |     void sesCikar() override { cout << "Miyav\n"; }
      |          ^~~~~~~~
```

`Hayvan`'ın fonksiyonu `const` idi, `Kedi`'ninki değil; bu iki ayrı imzadır. `override` sayesinde hata derleme sırasında yakalandı. Çıktıdaki `warning` satırı ayrı bir uyarıdır: `Kedi`'nin fonksiyonunun `Hayvan`'ınkini gizlediğini bildirir. Asıl hata `error` satırıdır. Kural basit: **Bir sanal fonksiyonu ezerken her zaman `override` yaz.** (Türetilmiş sınıfta `virtual` yazmak da gerekmez; zaten sanaldır.)

> **Sık Yapılan Hata: `virtual` yazmayı unutmak**
>
> Temel sınıfta `virtual` yazmazsan türetilmiş sınıftaki aynı imzalı fonksiyon yalnızca temel sınıfın fonksiyonunu **gizler**. `Hayvan*` ya da `Hayvan&` üzerinden çağrıda her zaman `Hayvan`'ın sürümü çalışır. Çok biçimlilik sessizce bozulur; program derlenir, hata vermez, yanlış çıktı üretir. Ezilmesini istediğin her fonksiyonun temel sınıfta `virtual` olduğundan emin ol.

12.6 Çok biçimli koleksiyon: `vector<unique_ptr<Sekil>>`

Çok biçimliliğin asıl işe yaradığı yer, farklı türden nesneleri **tek bir koleksiyonda** tutmaktır. Bir `Sekil` listesi düşün: İçinde daire, dikdörtgen, kare olsun; döngü hangisinin ne olduğunu bilmeden hepsinin alanını hesaplasın.

Burada küçük bir engel var. `vector<Sekil>` yazarsan her eleman gerçekten bir `Sekil` nesnesi olur ve bir `Daire` koyduğunda `Daire`'ye ait kısım **kesilip atılır** (aşağıda "nesne dilimleme" başlığında anlatılıyor). Çözüm, nesnelerin kendisini değil onlara **işaretçileri** tutmaktır. Nesneler öbekte (heap) yaratılır, `vector` ise onları gösteren işaretçileri taşır. Ham işaretçi yerine Bölüm 10'da gördüğün `unique_ptr` akıllı işaretçisini (smart pointer) kullanırsak `delete` yazmak gerekmez; `vector` yok edilirken nesneler de kendiliğinden silinir.

Satır satır ne yapacağımızı sonraki başlıktaki programda göreceksin; şimdilik üç kalıbı not et:

- `make_unique<Daire>(1)` bir `unique_ptr<Daire>` üretir. `push_back` bunu `unique_ptr<Sekil>`'e kendiliğinden dönüştürür (yukarı dönüşümün akıllı işaretçi sürümü).
- Döngüde `for (const auto& sekil : sekiller)` yaz: **referans** kullan. `unique_ptr` kopyalanamaz (nesnenin tek sahibi olmalı); `for (auto sekil : sekiller)` yazarsan derleyici hata verir.
- `sekil->alan()` çağrısı her elemanın gerçek tipine göre ilerler. Yarın listeye `Ucgen` eklersen döngüde **tek karakter** değiştirmen gerekmez.

Bir şeye daha ihtiyacımız var. `Sekil` sınıfı soyut bir kavramdır: "Şekil"in alanı sıfır değil, **tanımsızdır** ve kimse `Sekil sekil;` yazıp anlamsız bir nesne yaratmamalı. Bunu dile anlatmanın yolu soyut sınıflardır.

12.7 Saf sanal fonksiyon ve soyut sınıf

Gövdesi olmayan, yalnızca imzası bulunan sanal fonksiyona **saf sanal fonksiyon** (pure virtual function) denir. Yazımı, imzanın sonuna `= 0` eklemektir. En az bir saf sanal fonksiyonu olan sınıf **soyut sınıf** (abstract class) olur: Kendisinden nesne **yaratılamaz**, yalnızca türetilmek için vardır. Saf sanal fonksiyon, "her türetilmiş sınıf bu fonksiyonu **yazmak zorunda**" demektir.

soyutSinif.cpp

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <vector>

using namespace std;

const double PI{3.14159265358979};

class Sekil {
public:
    virtual ~Sekil() = default;

    virtual double alan() const = 0;
    virtual double cevre() const = 0;

    void bilgi() const {
        cout << ad << ": alan=" << alan() << ", çevre=" << cevre() << '\n';
    }

protected:
    Sekil(string adi) : ad{adi} {}

private:
    string ad;
};

class Daire : public Sekil {
public:
    Daire(double yaricap) : Sekil{"Daire"}, r{yaricap} {}

    double alan() const override { return PI * r * r; }
    double cevre() const override { return 2 * PI * r; }

private:
    double r;
};

class Dikdortgen : public Sekil {
public:
```

```
    Dikdortgen(double en, double boy) : Sekil{"Dikdörtgen"}, e{en}, b{boy} {}

    double alan() const override { return e * b; }
    double cevre() const override { return 2 * (e + b); }

private:
    double e, b;
};

int main() {
    vector<unique_ptr<Sekil>> sekiller;
    sekiller.push_back(make_unique<Daire>(1.5));
    sekiller.push_back(make_unique<Dikdortgen>(2, 5));
    for (const auto& sekil : sekiller) {
        sekil->bilgi();
    }
    return 0;
}
```

Çıktı

```
Daire: alan=7.06858, çevre=9.42478
Dikdörtgen: alan=10, çevre=14
```

`Sekil::bilgi()` fonksiyonu normal (gövdeli, sanal olmayan) bir fonksiyon. İçinde `alan()` ve `cevre()` çağrıları yapıyor; ama bu çağrılar `this->alan()` demektir ve `this` aslında bir `Daire` ya da `Dikdortgen`'i gösterdiği için doğru sürüm seçilir. Üst sınıfın içinden yapılan sanal çağrılar da dinamik bağlanır; sınavların en sevdiği tuzaklardan biri budur.

Soyut sınıfın kuralları:

- Soyut sınıftan nesne **yaratılamaz**, ama soyut sınıf tipinde **işaretçi ve referans** tanımlanabilir (`Sekil*`, `const Sekil&`, `unique_ptr<Sekil>`). Çok biçimlilik de zaten bunlar üzerinden çalışır.
- Soyut sınıfın **üye değişkenleri, yapıcısı ve gövdeli fonksiyonları** olabilir. Yapıcı, türetilmiş sınıfın üye ilk değer listesinden çağrılır. Nesne yaratılamayacağı için yapıcıyı genellikle `protected` yazarız.
- Soyut olmayan (somut) bir türetilmiş sınıf, temel sınıftaki **bütün** saf sanal fonksiyonları ezmek zorundadır. Birini bile ezmezse o sınıf da soyut kalır.
- Hiç üye değişkeni olmayan ve yalnızca saf sanal fonksiyonlardan oluşan soyut sınıflar, başka dillerdeki **arayüz** (interface) kavramının C++'taki karşılığıdır. C++'ta ayrı bir `interface` anahtar sözcüğü yoktur.

Soyut sınıftan nesne yaratmaya çalışırsan (`Sekil sekil;`) derleyici "cannot declare variable 'sekil' to be of abstract type 'Sekil'" der.

> **Sık Yapılan Hata: Saf sanal fonksiyonu ezmeyi unutmak**
>
> `Sekil`'in `alan()` ve `cevre()` diye iki saf sanal fonksiyonu varken `Kare` yalnızca `alan()`'ı yazarsa `Kare` de soyut kalır. Çoğu zaman hata, `Kare` sınıfını tanımlarken değil, onun bir nesnesini yaratmaya çalıştığın satırda patlar. Ya eksik fonksiyonu yaz ya da `Kare`'yi bilerek soyut bırak. Ayrıca fonksiyonun imzası da birebir tutmalıdır; `alan()` yerine `const` unuttuğun bir `alan()` yazarsan o fonksiyon saf sanalı **ezmez**.

12.8 Sanal yıkıcı neden gerekli?

Bir önceki başlıktaki soyut `Sekil` sınıfında `virtual ~Sekil() = default;` satırı vardı. Şimdi nedenine gelelim. Çok biçimliliğin tipik kullanımı, türetilmiş nesneyi **temel sınıf işaretçisiyle** tutmaktır. Nesneyi silme zamanı geldiğinde de silme işlemi o temel sınıf işaretçisi üzerinden yapılır (`delete` ya da `unique_ptr`'nin yıkıcısı). Temel sınıfın yıkıcısı `virtual` **değilse** silme işlemi de statik bağlanır: Yalnızca temel sınıfın yıkıcısı çalışır, türetilmiş sınıfın yıkıcısı hiç çağrılmaz. Standart bu durumu **tanımsız davranış** (undefined behavior) sayar:

```
class Kaynak {
public:
    ~Kaynak() { cout << "Kaynak temizlendi\n"; }   // virtual DEĞİL
};

class TamponluKaynak : public Kaynak {
    int* tampon = new int[1000];
public:
    ~TamponluKaynak() { delete[] tampon; }          // hiç çalışmayabilir
};

Kaynak* kaynak = new TamponluKaynak();
delete kaynak;      // tanımsız davranış; tampon sızabilir (bellek sızıntısı)
```

Pratikte çoğu derleyicide `TamponluKaynak` yıkıcısı çalışmaz ve `tampon` için ayrılan bellek geri verilmez: **bellek sızıntısı** (memory leak). Üstelik program tanımsız davranışa girdiği için başka sonuçlar da olabilir. Çözüm tek satır: Temel sınıfın yıkıcısını `virtual` yap. Aynı programı düzeltelim ve bu sefer akıllı işaretçiyle çalıştıralım:

sanalYikici.cpp

```
#include <iostream>
#include <memory>

using namespace std;

class Kaynak {
public:
    virtual ~Kaynak() { cout << "Kaynak temizlendi\n"; }
};

class TamponluKaynak : public Kaynak {
public:
    TamponluKaynak() : tampon{new int[1000]} {
        cout << "Tampon ayrıldı\n";
    }

    ~TamponluKaynak() override {
        delete[] tampon;
        cout << "Tampon serbest bırakıldı\n";
    }

private:
    int* tampon;
};

int main() {
    {
        unique_ptr<Kaynak> kaynak = make_unique<TamponluKaynak>();
        cout << "kaynak kullanılıyor\n";
    }
    cout << "blok bitti\n";
    return 0;
}
```

Çıktı

```
Tampon ayrıldı
kaynak kullanılıyor
Tampon serbest bırakıldı
Kaynak temizlendi
blok bitti
```

`unique_ptr<Kaynak>` yok edilirken içindeki `Kaynak*` üzerinden silme yapıldı; yıkıcı `virtual` olduğu için önce `TamponluKaynak` yıkıcısı (tamponu geri verdi), sonra `Kaynak` yıkıcısı çalıştı. Sıra, "yıkıcılar tersine" kuralıyla aynı.

> **Sınavda Çıkar: Sanal yıkıcı kuralı**
>
> **Bir sınıfta en az bir `virtual` fonksiyon varsa yıkıcısını da `virtual` yap.** Böyle bir sınıf zaten temel sınıf olmak için tasarlanmıştır ve nesneleri temel sınıf işaretçisiyle silinecektir. Yıkıcı gövdesi boşsa `virtual ~Sinif() = default;` yazmak yeter. Temel sınıfın yıkıcısı `virtual` ise türetilmiş sınıfların yıkıcıları da otomatik `virtual` olur; yine de `override` yazabilirsin. Sınavda "temel sınıf yıkıcısı sanal değilse ne olur?" sorusunun cevabı: tanımsız davranış, türetilmiş sınıf yıkıcısı çalışmayabilir, bellek sızabilir.

12.9 Nesne dilimleme tuzağı

Temel sınıf tipinde bir **nesneye** (işaretçi ya da referansa değil) türetilmiş bir nesne atarsan, türetilmiş sınıfa ait her şey **dilimlenip atılır**: Geriye yalnızca temel sınıf parçası kalır. Buna **nesne dilimleme** (object slicing) denir. Aynı şey bir fonksiyona temel sınıf tipinde **değer ile geçirme** (pass by value) yaptığında ve `vector<Hayvan>` içine türetilmiş nesne eklediğinde de olur.

dilimleme.cpp

```cpp
#include <iostream>
#include <string>
#include <vector>

using namespace std;

class Hayvan {
public:
    Hayvan(string adi) : ad{adi} {}
    virtual ~Hayvan() = default;

    virtual string ses() const { return "..."; }
    string adi() const { return ad; }

private:
    string ad;
};

class Kopek : public Hayvan {
public:
    Kopek(string adi) : Hayvan{adi} {}

    string ses() const override { return "Hav hav"; }
};

void degerIle(Hayvan hayvan) {
    cout << "değer ile:    " << hayvan.adi() << " " << hayvan.ses() << '\n';
}

void referansIle(const Hayvan& hayvan) {
    cout << "referans ile: " << hayvan.adi() << " " << hayvan.ses() << '\n';
}
```

```
int main() {
    Kopek kopek{"Karabaş"};
    degerIle(kopek);
    referansIle(kopek);

    vector<Hayvan> dilimli;
    dilimli.push_back(kopek);
    cout << "vector<Hayvan>: " << dilimli[0].ses() << '\n';
    return 0;
}
```

Çıktı

```
değer ile:    Karabaş ...
referans ile: Karabaş Hav hav
vector<Hayvan>: ...
```

Dilimleme sessizce olur: Derleyici uyarı vermez, program çökmez, köpek yalnızca "..." diye ses çıkarır. Çünkü `Hayvan` tipindeki bir değişken **gerçekten** bir `Hayvan` nesnesidir; yalnızca `Hayvan` kadar yer ayrılır. `Kopek`'e ait kısım kopyalanmaz ve nesnenin sanal fonksiyon tablosu da temel sınıfı gösterir. Bir ayrıntı daha: `Hayvan h = kopek;` gibi bir satırda yeni nesne senin yazdığın yapıcılardan biriyle değil, derleyicinin kendiliğinden ürettiği **kopyalama** işlemiyle doğar; üyeler tek tek kopyalanır, yapıcı gövdelerindeki satırlar (örneğin bir `cout`) çalışmaz. Bu işlemin adı kopya yapıcıdır, ayrıntısını Bölüm 13'te göreceksin. Korunmanın yolu belli:

- Polimorfik sınıfları fonksiyona `const Sınıf&` (referans ile geçirme, Bölüm 6) ya da işaretçi ile ver.
- Koleksiyonda `vector<Hayvan>` değil `vector<unique_ptr<Hayvan>>` kullan.
- Temel sınıfı soyut yaparsan dilimleme zaten derlenmez: `Hayvan` tipinde nesne yaratılamaz.

> **Sık Yapılan Hata: Çok biçimli sınıfı değerle geçirmek ya da `vector<Temel>` kullanmak**
>
> `void goster(Hayvan h)` ve `vector<Hayvan>` yazmak, çok biçimliliği kapatmanın en kolay yoludur. Program derlenir, çalışır, ama türetilmiş sınıfların davranışı kaybolur. Temel sınıf tipini parametrede her zaman `const Hayvan&`, koleksiyonda ise akıllı işaretçi olarak yaz.

(İleri) Yapıcının içinden sanal fonksiyon çağırmak

Temel sınıfın yapıcısı çalışırken türetilmiş kısım **henüz yaratılmamıştır**. Bu yüzden yapıcıda (ve yıkıcıda) çağrılan sanal fonksiyon dinamik bağlanmaz: O anda hangi sınıfın yapıcısı çalışıyorsa onun sürümü seçilir. Yapıcıda ve yıkıcıda sanal fonksiyon çağırmamak iyi bir alışkanlıktır.

12.10 `final` (kısa)

Bir sanal fonksiyonu ya da bir sınıfı "burada dur, daha fazla türetme/ezme olmasın" diye kilitlemek için `final` yazılır. `class Kare final : public Sekil` ifadesi `Kare`'den türetmeyi yasaklar; `double alan() const final` ifadesi o fonksiyonun daha ileri ezilmesini yasaklar.

```
class Kare final : public Sekil { /* ... */ };    // Kare'den türetilemez
class BuyukKare : public Kare { };                 // HATA: Kare final
```

12.11 Çoklu kalıtım (kısa)

C++'ta bir sınıf **birden fazla** temel sınıftan türetilebilir: `class Amfibi : public Kara, public Su { };`. Buna **çoklu kalıtım** (multiple inheritance) denir. Güçlüdür, ama tehlikelidir: İki temel sınıf da aynı bir üst sınıftan türemişse, türetilmiş nesne o üst sınıfın **iki ayrı kopyasını** taşır ve çağrılar belirsiz kalır. Bu sorunun adı **elmas problemi** (diamond problem); `virtual` kalıtımla çözülür, ama bu kitabın kapsamı dışındadır. Başlangıçta tek kalıtım ve saf sanal fonksiyonlardan oluşan arayüz sınıfları ile ilerlemek yeterlidir.

(İleri) Perde arkası: sanal fonksiyon tablosu

Derleyici, sanal fonksiyonu olan her sınıf için bir **sanal fonksiyon tablosu** (vtable) hazırlar. Her nesneye gizli bir işaretçi eklenir; `p->sesCikar()` çağrısı bu işaretçi ve tablo üzerinden doğru fonksiyona ulaşır. Dinamik bağlamanın küçük bir maliyeti bu yüzden vardır: nesne bir işaretçi kadar büyür ve çağrı bir adım dolaylı olur.

Bölüm Özeti – Bilmen Gerekenler

- **Kalıtım:** `class Turetilmis : public Temel`. "X bir Y'dir" ilişkisi için kullan; `public` yazmayı unutma (`class`'ta varsayılan `private`).
- `private` üyeye türetilmiş sınıf erişemez, `protected` üyeye erişir ama dışarısı erişemez.
- **Yapıcı sırası:** temel sınıf, üye nesneler, türetilmiş gövde. **Yıkıcı sırası** tam tersi. Temel sınıfın yapıcısı parametre istiyorsa üye ilk değer listesinde çağır: `Alt() : Temel{x} {}`.
- `virtual` yoksa yeniden tanımlama yalnızca gizler ve işaretçinin tipine göre statik bağlanır. `virtual` + `override` + işaretçi/referans = **dinamik bağlama** (çok biçimlilik).
- Ezerken her zaman `override` yaz; imza (`const` dahil) uyuşmazsa derleyici hata verir.
- **Saf sanal fonksiyon** `= 0` ile yazılır; en az bir tane içeren sınıf **soyut sınıf** olur ve nesnesi yaratılamaz. Türetilmiş sınıf hepsini ezmezse o da soyut kalır.
- Çok biçimli koleksiyon: `vector<unique_ptr<Temel>>`. Döngüde `const auto&` kullan.
- Çok biçimli sınıfta yıkıcı `virtual` olmalı; yoksa temel işaretçisiyle `delete` tanımsız davranıştır.
- **Nesne dilimleme:** Temel tipte nesneye/değer parametresine atama türetilmiş kısmı keser. `const Temel&` ya da akıllı işaretçi kullan.
- Yapıcı ve yıkıcı içinde sanal çağrı dinamik bağlanmaz. `final` türetmeyi/ezmeyi kapatır. Çoklu kalıtım vardır; elmas problemi çıkarabilir.

Sorular

A) Çoktan Seçmeli

1 `class Sekil { public: virtual double alan() const = 0; virtual ~Sekil() = default; };` tanımlı ve bu sınıfı doğru biçimde uygulayan bir `class Daire : public Sekil` var. Aşağıdaki satırlardan hangisi **derlenmez**?

(A) `Sekil* p = new Daire{};`

(B) `Sekil dizi[2];`

(C) `Sekil* dizi[2]{};`

(D) `const Sekil& r = Daire{};`

(E) `unique_ptr<Sekil> u{make_unique<Daire>()};`

2 Aşağıdaki kod ne yazdırır?

```
class A { public: virtual string f() const { return "A"; }
                  string g() const { return "a"; } };
class B : public A { public: string f() const override { return "B"; }
                             string g() const { return "b"; } };
void yaz(A x) { cout << x.f() << x.g(); }
// main içinde:
B b;
yaz(b);
A& r = b;
cout << r.f() << r.g();
```

(A) `BbBb` (B) `AaAa` (C) `BbBa` (D) `AbBb` (E) `AaBa`

3 `Hayvan* p = new Kedi{}; delete p;` satırında `Hayvan` sınıfının yıkıcısı `virtual` **değilse** ne olur?

(A) Tanımsız davranış oluşur; `Kedi` yıkıcısı çalışmayabilir ve bellek sızabilir.

(B) `Kedi` yıkıcısı, ardından `Hayvan` yıkıcısı çalışır.

(C) Yalnızca `Kedi` yıkıcısı çalışır.

(D) Program derlenmez.

(E) `delete` bir istisna fırlatır.

4 Aşağıdaki program ekrana ne yazdırır?

```
class A { public: A() { cout << "1"; } virtual ~A() { cout << "2"; } };
class B : public A { public: B() { cout << "3"; } ~B() { cout << "4"; } };
class C { public: C() { cout << "5"; } ~C() { cout << "6"; } };
class D : public B {
    C c;
public:
    D() { cout << "7"; }
    ~D() { cout << "8"; }
};
int main() { D d; return 0; }
```

(A) `13572468` (B) `13758642` (C) `75318642` (D) `13578642` (E) `13578462`

5 Kalıtım ve çok biçimlilikle ilgili aşağıdakilerden hangisi **yanlıştır**?

(A) `override` yazılmış bir fonksiyon, temel sınıftaki bir sanal fonksiyonla aynı imzaya sahip değilse derleme hatası oluşur.

(B) Türetilmiş bir nesnenin adresi, temel sınıf işaretçisine cast olmadan atanabilir.

(C) Soyut bir sınıfın yapıcısı olamaz.

(D) `final` ile işaretlenmiş bir sınıftan yeni bir sınıf türetilemez.

(E) Bir sınıfın en az bir saf sanal fonksiyonu varsa o sınıf soyuttur.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır? (`<iostream>` ve `using namespace std;` var.)

```
class Kutu {
public:
    Kutu() { cout << "K+ "; }
    virtual ~Kutu() { cout << "K- "; }
    virtual void yaz() const { cout << "Kutu "; }
};

class Hediye : public Kutu {
public:
    Hediye() { cout << "H+ "; }
    ~Hediye() override { cout << "H- "; }
    void yaz() const override { Kutu::yaz(); cout << "Hediye "; }
};

int main() {
    Kutu* p = new Hediye();
    p->yaz();
    delete p;
    cout << '\n';
    {
        Hediye h;
        Kutu k = h;
        k.yaz();
        cout << '\n';
    }
    cout << "bitti\n";
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```
class Hayvan {
public:
    virtual string ses() const { return "..."; }
    string tur() const { return "Hayvan"; }
};

class Kedi : public Hayvan {
public:
    string ses() const override { return "Miyav"; }
    string tur() const { return "Kedi"; }
};

void a(Hayvan h) { cout << h.ses() << " " << h.tur() << '\n'; }
void b(const Hayvan& h) { cout << h.ses() << " " << h.tur() << '\n'; }
void c(const Hayvan* h) { cout << h->ses() << " " << h->tur() << '\n'; }

int main() {
    Kedi kedi;
    a(kedi);
    b(kedi);
    c(&kedi);
    cout << kedi.ses() << " " << kedi.tur() << '\n';
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```
class Kisi {
public:
    Kisi(string adi) : ad{adi} {}
    virtual ~Kisi() = default;

    virtual string unvan() const = 0;
    string tanit() const { return unvan() + " " + ad; }

private:
    string ad;
};

class Ogrenci : public Kisi {
public:
    Ogrenci(string adi) : Kisi{adi} {}
    string unvan() const override { return "Ogr."; }
};

class Hoca : public Kisi {
public:
    Hoca(string adi) : Kisi{adi} {}
    string unvan() const override { return "Dr."; }
};

class Asistan : public Hoca {
public:
    Asistan(string adi) : Hoca{adi} {}
    string unvan() const override { return "Ars. " + Hoca::unvan(); }
};

int main() {
    vector<unique_ptr<Kisi>> liste;
    liste.push_back(make_unique<Ogrenci>("Ece"));
    liste.push_back(make_unique<Hoca>("Kaya"));
    liste.push_back(make_unique<Asistan>("Mert"));
    for (const auto& kisi : liste) {
        cout << kisi->tanit() << '\n';
    }
    return 0;
}
```

C) Kod Yazma

9 Bir kütüphanenin gecikme cezası programını yaz. Soyut `Materyal` sınıfı başlığı saklasın ve `string tur()` ile `double ceza(int gun)` saf sanal fonksiyonlarına sahip olsun. `Kitap` günlük 2 TL, `Dergi` günlük 1 TL (en çok 20 TL), `Dvd` ilk 2 gün ücretsiz, sonrası günlük 5 TL ceza alsın. `main` içinde `vector<unique_ptr<Materyal>>` ile şu materyalleri ve gecikmelerini kullan: Sefiller (kitap, 10 gün), Bilim (dergi, 30 gün), Matrix (dvd, 4 gün), Dune (kitap, 3 gün). Başlık 10, tür 7, ceza 7 karakter genişliğinde (ceza iki ondalıklı) olsun:

```
Sefiller  Kitap    20.00
Bilim     Dergi    20.00
Matrix    DVD      10.00
Dune      Kitap     6.00
Toplam ceza: 56.00 TL
```

10 `Hesap` sınıfını yaz: `protected` `bakiye` üyesi, `virtual void ayBitir()` (her ay 5 TL hesap işletim ücreti keser) ve `bakiyeVer()` olsun. `VadeliHesap` bu fonksiyonu ezsin: önce temel sınıfın sürümünü çağırsın, sonra kalan bakiyeye %1 faiz eklesin. `main` içinde ikisinden de 1000 TL bakiyeli birer hesap aç, `vector<unique_ptr<Hesap>>` içinde tut ve 3 ay boyunca her ay sonunda iki bakiyeyi yazdır:

```
1. ay: 995.00 1004.95
2. ay: 990.00 1009.95
3. ay: 985.00 1015.00
```

Çözümler

Cevap anahtarı

1	2	3	4	5
B	E	A	D	C

1 Cevap: B

Soyut sınıf tipinde **nesne** (ve nesne dizisi) yaratılamaz: `Sekil dizi[2];` iki ayrı `Sekil` nesnesi demektir. C tuzaktır: `Sekil* dizi[2]{}` bir **işaretçi dizisi**dir (elemanlar `nullptr`), geçerlidir. A, D ve E türetilmiş somut sınıftan nesne yaratıp soyut tipte işaretçi/referans/akıllı işaretçiye bağlar; hepsi derlenir.

2 Cevap: E

`yaz(A x)` değerle alır: `b` nesnesi `A`'ya dilimlenir, bu yüzden `x.f()` → `A`, `x.g()` → `a`. Sonra `A& r = b;` referansı gerçek `B` nesnesini gösterir: `f()` sanal olduğu için `B`, `g()` sanal olmadığı için referansın tipine göre `a`. Sonuç `AaBa`. A (`BbBb`) dilimlemeyi hiç görmez ve `g()`'yi de sanal sanar. B (`AaAa`) referans üzerinden de `f()`'nin temel sınıf sürümünün çalışacağını sanar. C (`BbBa`) `yaz` içinde dilimlemenin olmadığını sanar. D (`AbBb`) `g()`'nin de sanal olduğunu sanar.

3 Cevap: A

Temel sınıf yıkıcısı sanal değilse temel işaretçisi üzerinden silmek **tanımsız davranıştır**; pratikte yalnızca `Hayvan` yıkıcısı çalışır, `Kedi`'nin ayırdığı kaynaklar sızar. B yıkıcı `virtual` olsaydı doğru olurdu. C yanlıştır: `Kedi` yıkıcısı hiç çalışmaz; yalnızca temel sınıfın yıkıcısı çalışır. D yanlıştır; derleme sorunsuzdur (derleyici en fazla uyarı verir). E de yanlıştır; `delete` istisna fırlatmaz.

4 Cevap: D

Yapıcılar: önce temel sınıflar (`A` → 1, `B` → 3), sonra `D`'nin üye nesnesi `C` (5), en son `D`'nin gövdesi (7): `1357`. Yıkıcılar tersine: `D` (8), üye nesne `C` (6), `B` (4), `A` (2): `8642`. Toplam `13578642`. A, yıkıcıları da yapıcılarla aynı sırada sanır. B, üye nesneyi gövdeden sonra yaratır. C, yapıcı sırasını tersine çevirir. E, sonda `4` ile `6`'nın yerini karıştırır.

5 Cevap: C

Soyut sınıfın yapıcısı **olabilir**; hatta çoğu zaman vardır, türetilmiş sınıfların üye ilk değer listesinden çağrılır (yapıcıyı `protected` yazmak yaygındır). Diğer şıklar doğrudur: A `override` kontrolü, B yukarı dönüşüm, D `final`, E soyut sınıfın tanımıdır.

6

- `new Hediye()`: Önce `Kutu` yapıcısı, sonra `Hediye` yapıcısı: `K+ H+`.
- `p->yaz()`: `yaz` sanal; `Hediye` sürümü çalışır, içinde `Kutu::yaz()` çağırır: `Kutu Hediye`.
- `delete p`: Yıkıcı sanal olduğu için önce `Hediye`, sonra `Kutu`: `H- K-`.
- Blokta `Hediye h;` yine `K+ H+` yazar. `Kutu k = h;` **dilimlemedir**: `k` gerçekten bir `Kutu` olur ve kopya yapıcıyı derleyici yazdığı için ekrana bir şey çıkmaz. `k.yaz()` → `Kutu`.
- Blok biter, nesneler tanımlanma sırasının tersiyle yok edilir: önce `k` (`K-`), sonra `h` (`H- K-`).
- Bloktaki `cout << '\n'` satırı yıkıcı çıktılarından önce çalışır; bu yüzden `bitti`, `K- H- K-` satırının hemen ardına aynı satıra yazılır.

Çıktı
```
K+ H+ Kutu Hediye H- K-
K+ H+ Kutu
K- H- K- bitti
```

7

Her çağrıda iki karar var: `ses()` sanaldır (nesnenin gerçek tipine, ama yalnızca işaretçi/referansta), `tur()` sanal değildir (her zaman ifadenin tipine).

- `a(kedi)`: Değerle geçirme **dilimler**; `h` gerçekten bir `Hayvan` olur: `... Hayvan`.
- `b(kedi)`: Referans gerçek nesneyi gösterir, `ses()` → `Miyav`; ama referansın tipi `Hayvan` olduğu için `tur()` → `Hayvan`.
- `c(&kedi)`: İşaretçi için de aynı: `Miyav Hayvan`.
- `kedi.tur()`: İfadenin tipi `Kedi`, yeniden tanımlanan sürüm çalışır: `Miyav Kedi`.

Çıktı
```
... Hayvan
Miyav Hayvan
Miyav Hayvan
Miyav Kedi
```

8

`tanit()` yalnızca `Kisi` içinde yazılı, ama içindeki `unvan()` çağrısı dinamik bağlanır.

- `Ogrenci("Ece")`: `Ogr. Ece`.
- `Hoca("Kaya")`: `Dr. Kaya`.
- `Asistan("Mert")`: `unvan()` önce `"Ars. "` yazar, `Hoca::unvan()` ile `"Dr."` ekler: `Ars. Dr. Mert`.

Çıktı
```
Ogr. Ece
Dr. Kaya
Ars. Dr. Mert
```

9

`Materyal`, `tur()` ve `ceza()` fonksiyonlarını saf sanal bırakır. `satir()` gövdeli ortak fonksiyondur ve dinamik bağlama sayesinde her materyalin kendi cezasını yazar. DVD için `(gun - 2) * 5` yalnızca `gun > 2` iken hesaplanır.

kutuphane.cpp
```cpp
#include <iomanip>
#include <iostream>
#include <memory>
#include <string>
#include <vector>

using namespace std;

class Materyal {
public:
    Materyal(string baslik) : baslik{baslik} {}
    virtual ~Materyal() = default;

    virtual string tur() const = 0;
    virtual double ceza(int gun) const = 0;
```

```
    void satir(int gun) const {
        cout << left << setw(10) << baslik << setw(7) << tur();
        cout << right << setw(7) << ceza(gun) << '\n';
    }

private:
    string baslik;
};

class Kitap : public Materyal {
public:
    Kitap(string baslik) : Materyal{baslik} {}
    string tur() const override { return "Kitap"; }
    double ceza(int gun) const override { return gun * 2; }
};

class Dergi : public Materyal {
public:
    Dergi(string baslik) : Materyal{baslik} {}
    string tur() const override { return "Dergi"; }
    double ceza(int gun) const override { return gun > 20 ? 20 : gun; }
};

class Dvd : public Materyal {
public:
    Dvd(string baslik) : Materyal{baslik} {}
    string tur() const override { return "DVD"; }
    double ceza(int gun) const override { return gun > 2 ? (gun - 2) * 5 : 0; }
};

int main() {
    vector<unique_ptr<Materyal>> raf;
    vector<int> gecikme{10, 30, 4, 3};
    raf.push_back(make_unique<Kitap>("Sefiller"));
    raf.push_back(make_unique<Dergi>("Bilim"));
    raf.push_back(make_unique<Dvd>("Matrix"));
    raf.push_back(make_unique<Kitap>("Dune"));

    double toplam{0};
    cout << fixed << setprecision(2);
    for (size_t i = 0; i < raf.size(); i++) {
        raf[i]->satir(gecikme[i]);
        toplam += raf[i]->ceza(gecikme[i]);
    }
    cout << "Toplam ceza: " << toplam << " TL\n";
    return 0;
}
```

Çıktı

```
Sefiller  Kitap    20.00
Bilim     Dergi    20.00
Matrix    DVD      10.00
Dune      Kitap     6.00
Toplam ceza: 56.00 TL
```

10

`VadeliHesap::ayBitir()` önce `Hesap::ayBitir()` ile işletim ücretini keser, sonra faizi ekler. `bakiye` `protected` olduğu için türetilmiş sınıf ona doğrudan erişir. Liste (`vector`) `Hesap` tipinde akıllı işaretçiler tuttuğu için `ayBitir()` çağrısı her hesabın kendi sürümüne gider.

hesaplar.cpp

```cpp
#include <iomanip>
#include <iostream>
#include <memory>
#include <vector>

using namespace std;

class Hesap {
public:
    Hesap(double bakiye) : bakiye{bakiye} {}
    virtual ~Hesap() = default;

    virtual void ayBitir() { bakiye -= 5; }
    double bakiyeVer() const { return bakiye; }

protected:
    double bakiye;
};

class VadeliHesap : public Hesap {
public:
    VadeliHesap(double bakiye) : Hesap{bakiye} {}

    void ayBitir() override {
        Hesap::ayBitir();
        bakiye *= 1.01;
    }
};

int main() {
    vector<unique_ptr<Hesap>> hesaplar;
    hesaplar.push_back(make_unique<Hesap>(1000));
    hesaplar.push_back(make_unique<VadeliHesap>(1000));
    cout << fixed << setprecision(2);
    for (int ay = 1; ay <= 3; ay++) {
        for (auto& hesap : hesaplar) {
            hesap->ayBitir();
        }
        cout << ay << ". ay: " << hesaplar[0]->bakiyeVer() << " "
             << hesaplar[1]->bakiyeVer() << '\n';
    }
    return 0;
}
```

Çıktı

```
1. ay: 995.00 1004.95
2. ay: 990.00 1009.95
3. ay: 985.00 1015.00
```

BÖLÜM 13

Operatör Aşırı Yükleme ve Şablonlar

Bu Bölümde Öğreneceklerin

- Kendi sınıfın için toplama, karşılaştırma gibi operatörleri nasıl tanımlayacağını
- Aynı operatörün sınıfın içinde mi, dışında mı yazılacağına nasıl karar vereceğini
- Nesneyi `cout` ile yazdırabilmek için ne gerektiğini ve `friend`'in ne işe yaradığını
- Bir nesneyi kopyalamanın ne zaman yanlış gidebileceğini
- Kopya yapıcı, atama operatörü ve yıkıcının neden üçlü bir takım olduğunu
- Aynı kodu her tip için yeniden yazmak yerine fonksiyon şablonu yazmayı
- Her tiple çalışan sınıflar (şablon sınıflar) yazmayı
- Şablon hata mesajlarının neden uzun olduğunu ve nasıl okunacağını

13.1 Neden operatör aşırı yükleme?

`int` ve `double` ile `a + b`, `a == b`, `a < b` yazmak çok doğal. Peki iki **süreyi** toplamak istersen? Bölüm 11'de gördüğün gibi `Sure` adında kendi sınıfını yazabilirsin; ama derleyici bu sınıfın nesnelerini nasıl toplayacağını bilmez:

toplaYok.cpp

```
class Sure {
public:
    Sure(int dakika) : dakika_{dakika} {}
    int dakika() const { return dakika_; }
private:
    int dakika_;
};

int main() {
    Sure ders{90};
    Sure mola{45};
    Sure toplam = ders + mola;
    return toplam.dakika();
}
```

Derleyici çıktısı – program derlenmez

```
toplaYok.cpp: In function 'int main()':
toplaYok.cpp:12:24: error: no match for 'operator+' (operand types are 'Sure' and 'Sure')
   12 |     Sure toplam = ders + mola;
      |                   ~~~~ ^ ~~~~
      |                   |      |
      |                   Sure   Sure
```

Çözüm olarak `ders.topla(mola)` gibi bir üye fonksiyon yazabilirsin; ama üç süreyi toplamak `a.topla(b).topla(c)` olur ve okunması zorlaşır. C++ sana başka bir yol verir: **operatör aşırı yükleme** (operator overloading). Operatörün senin sınıfın için ne yapacağını, adı `operator` ile başlayan özel bir fonksiyonla tanımlarsın. Böylece `ders + mola` yazabilirsin. Bölüm 6'da aynı adlı fonksiyonları farklı parametrelerle **aşırı yüklemeyi** (overloading) görmüştün; bu onun operatörlere uygulanmış hâlidir.

Önemli bir gerçek: operatör aşırı yükleme **yeni bir yetenek katmaz**. `ders + mola` yazdığında derleyici bunu `ders.operator+(mola)` olarak okur; yani sıradan bir fonksiyon çağrısıdır. Tek kazancın okunaklılıktır. Bu yüzden kural basit: **operatörü yalnızca anlamı herkesçe belli olduğu yerde tanımla.** Süreleri `+` ile toplamak doğal; sürelerin `*` ile bir şey yapması ise kimseyi şaşırtmamalı. `+` operatörünün çıkarma yapması ya da `<<` operatörünün dosya silmesi gibi sürprizler yapma.

> **Sınavda Çıkar: Aşırı yüklemenin kuralları**
>
> - Yeni operatör icat edemezsin (`**` yok); yalnızca var olanları aşırı yüklersin.
> - Operatörün **önceliği** ve **işlenen** (operand) sayısı değişmez: `*`, `+`'dan önce çalışır; iki işlenenli `+` iki işlenenli kalır (tek işlenenli `+x` ayrı bir operatördür).
> - İşlenenlerden **en az biri** kendi tipin (sınıf) olmalı. `int + int`'in anlamını değiştiremezsin.
> - Aşırı yüklenemeyen operatörler: `::`, `.`, `.*`, `?:` ve `sizeof`.
> - `=`, `[]`, `()` ve `->` operatörleri yalnızca **üye fonksiyon** olarak tanımlanabilir.
> - `&&` ve `||` aşırı yüklenebilir ama bunu yapma: aşırı yüklenmiş hâlleri **kısa devre değerlendirmeyi** (short-circuit evaluation) kaybeder; her iki işlenen de hesaplanır.

13.2 Üye fonksiyon olarak operatörler

Başlayalım: `+`, `+=`, `==`, `!=` ve `<` operatörlerini `Sure` sınıfının üye fonksiyonu olarak yazalım. Soldaki işlenen, fonksiyonun çağrıldığı nesnedir (`*this`); sağdaki işlenen parametre olur.

sureOperator.cpp

```cpp
#include <iostream>

using namespace std;

class Sure {
public:
    Sure(int saat, int dakika) : toplamDakika_{saat * 60 + dakika} {}

    int saat() const { return toplamDakika_ / 60; }
    int dakika() const { return toplamDakika_ % 60; }

    Sure& operator+=(const Sure& diger) {
        toplamDakika_ += diger.toplamDakika_;
        return *this;
    }

    Sure operator+(const Sure& diger) const {
        Sure sonuc = *this;
        sonuc += diger;
        return sonuc;
    }

    bool operator==(const Sure& diger) const {
        return toplamDakika_ == diger.toplamDakika_;
    }

    bool operator!=(const Sure& diger) const { return !(*this == diger); }
```

```
    bool operator<(const Sure& diger) const {
        return toplamDakika_ < diger.toplamDakika_;
    }

    void yaz() const {
        cout << saat() << " saat " << dakika() << " dk";
    }

private:
    int toplamDakika_;
};

int main() {
    Sure ders{1, 30};
    Sure mola{0, 45};

    Sure toplam = ders + mola;
    cout << "ders + mola = ";
    toplam.yaz();
    cout << '\n';

    ders += mola;
    cout << "ders += mola -> ";
    ders.yaz();
    cout << '\n';

    cout << "ders == toplam: " << (ders == toplam) << '\n';
    cout << "mola < ders: " << (mola < ders) << '\n';
    cout << "mola != ders: " << (mola != ders) << '\n';
    return 0;
}
```

Çıktı

```
ders + mola = 2 saat 15 dk
ders += mola -> 2 saat 15 dk
ders == toplam: 1
mola < ders: 1
mola != ders: 1
```

Kodda dikkat edilecek üç yer var:

1. **Parametre `const Sure&`:** Sağdaki işlenen referansla geçer (Bölüm 9'daki referans ile geçirme), yani nesne kopyalanmaz; `const` ise fonksiyonun onu değiştirmeyeceğini söyler.
2. **Parantezden sonraki `const`:** `bool operator==(...) const` demek "bu fonksiyon çağrıldığı nesneyi değiştirmez" demektir. Yukarıdaki `Sure` sınıfının `saat() const` fonksiyonuyla aynı mantık (`const` üye fonksiyon, Bölüm 11). Karşılaştırma operatörleri ve `+` hiçbir işlenenini değiştirmemeli.
3. **Dönüş tipleri farklı: `+` yeni bir nesneyi değer olarak** döndürür (`a + b` ne `a`'yı ne `b`'yi değiştirir). `+=` ise soldaki nesnenin kendisini değiştirir ve `*this`'i **referansla** (`Sure&`) döndürür; böylece `(a += b) += c` gibi zincirler çalışır.

İpucu: `+`'yı `+=` üzerinden yaz

Yukarıda `operator+` kendi hesabını yapmadı; nesneyi kopyaladı ve `+=` ile topladı. Bu kalıp çok yaygındır: mantığı tek yerde (`+=`) tutarsın, `+` onu kullanır. Aynı şekilde `!=` operatörünü `==`'in tersi olarak yazdık; `>`, `<=`, `>=` de `<` üzerinden yazılabilir (`a > b` yerine `b < a`).

Sık Yapılan Hata: `const` unutulan karşılaştırma operatörü

`operator==`'e `const` yazmazsan, `const` bir nesneyi ya da `const Sure&` parametresini karşılaştırırken derlenmez. Fonksiyonlara nesneleri `const` referansla geçirmek zaten kuraldır, yani bu hata hemen karşına çıkar:

```
class Sure {
public:
    Sure(int dakika) : dakika_{dakika} {}
    bool operator==(const Sure& d) { return dakika_ == d.dakika_; }
private:
    int dakika_;
};

bool ayniMi(const Sure& a, const Sure& b) { return a == b; }

int main() {
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
constYok.cpp: In function 'bool ayniMi(const Sure&, const Sure&)':
constYok.cpp:9:57: error: passing 'const Sure' as 'this' argument discards qualifiers [-
fpermissive]
    9 | bool ayniMi(const Sure& a, const Sure& b) { return a == b; }
      |                                                         ^
constYok.cpp:4:10: note:   in call to 'bool Sure::operator==(const Sure&)'
    4 |     bool operator==(const Sure& d) { return dakika_ == d.dakika_; }
      |          ^~~~~~~~
```

Çözüm: fonksiyonun başlığına `const` ekle: `bool operator==(const Sure& diger) const`.

Üye olmayan fonksiyon olarak operatör

Üye fonksiyonda sol işlenen her zaman senin sınıfındır (`*this`). `ders * 3` böyle yazılır; ama `3 * ders` için sol işlenen `int`'tir; `int` bir sınıf değil temel bir tip olduğu için ona üye fonksiyon ekleyemezsin. Kural basit: **sol işlenen senin sınıfınsa üye yap; değilse sınıfın dışında, sıradan bir fonksiyon olarak yaz.** Örneğin `operator*(int kat, const Sure& s)` sınıfın dışına yazılır. Aynı kural `cout << ders` için de geçerlidir (aşağıda).

13.3 `<<` ile `cout`'a yazdırma

Şimdiye kadar süreyi ekrana `yaz()` fonksiyonuyla yazdırdık. Oysa `int` için `cout << sayi` yazıyorduk. Aynısını `Sure` için yapmak isteriz: `cout << ders`. Burada `<<` operatörünün **sol işleneni `cout`'tur**, yani `ostream` tipinde bir akış (stream) nesnesidir. Üye fonksiyon olsaydı `ostream` sınıfının içine yazılması gerekirdi ve o sınıf senin değil. Demek ki `operator<<` **üye olmayan** bir fonksiyon olmak zorunda. Kalıbı:

```
ostream& operator<<(ostream& os, const Sure& s);
```

- İlk parametre `ostream&`: akış **referansla** geçer, çünkü akış nesneleri kopyalanamaz.
- Fonksiyon `os`'a yazar ve **`os`'u referansla geri döndürür.** `cout << a << b` aslında `(cout << a) << b` demektir; ilk `<<` yazdıktan sonra `cout`'u döndürdüğü için ikincisi çalışır. Zincirleme (chaining) bu yüzden mümkündür.

Fonksiyon `Sure`'nin özel üyelerine ihtiyaç duyuyorsa, sınıf ona izin vermelidir. Bunun yolu `friend` anahtar sözcüğüdür: sınıfın içine fonksiyonun bildirimini `friend` ile yazarsan, o fonksiyon sınıfın **arkadaş fonksiyonu** (friend function) olur ve özel üyelere erişebilir. Arkadaş fonksiyon üye değildir; yalnızca erişim izni almıştır.

sureYazdir.cpp

```cpp
#include <iostream>

using namespace std;

class Sure {
public:
    Sure(int saat, int dakika) : toplamDakika_{saat * 60 + dakika} {}

    Sure operator+(const Sure& diger) const {
        return Sure{0, toplamDakika_ + diger.toplamDakika_};
    }

    friend ostream& operator<<(ostream& os, const Sure& s);

private:
    int toplamDakika_;
};

ostream& operator<<(ostream& os, const Sure& s) {
    os << s.toplamDakika_ / 60 << " saat " << s.toplamDakika_ % 60 << " dk";
    return os;
}

int main() {
    Sure ders{1, 30};
    Sure mola{0, 45};
    cout << "Ders: " << ders << '\n';
    cout << "Toplam: " << ders + mola << '\n';
    return 0;
}
```

Çıktı

```
Ders: 1 saat 30 dk
Toplam: 2 saat 15 dk
```

`cout << "Toplam: " << ders + mola` satırında `+`, `<<`'dan önce çalıştığı için önce iki süre toplandı, sonra sonuç yazdırıldı. `friend` yerine sınıfa `saat()` ve `dakika()` gibi açık (public) üyeler eklesen, `operator<<` onları kullanır ve arkadaşlığa gerek kalmazdı. Aynı kalıpla `istream& operator>>(istream& is, Sure& s)` yazarsan `cin >> s` de çalışır; burada `s` referans olmalı ve `const` olmamalıdır, çünkü okuduğun değer içine yazılacaktır.

> **Not: `friend` kapsüllemeyi bozar mı?**
>
> `friend`, sınıfın kendi isteğiyle verdiği bir izindir; `private` bölümünü herkese açmaz. Yine de her yardımcı fonksiyonu arkadaş yapma: önce açık `get` fonksiyonlarıyla çözülüp çözülmediğine bak. Arkadaşlık tek yönlüdür ve miras alınmaz.

Sık Yapılan Hata: `operator<<` zincirleme yazmaya izin vermiyor

`operator<<` içinde `os`'u döndürmeyi unutursan (`void` yazarsan) tek `cout << a;` çalışır, ama `cout << a << b;` çalışmaz: ilk `<<` işleminin sonucu `void` olur ve `void << b` diye bir işlem yoktur.

```
void operator<<(ostream& os, const Sure& s) {       // os döndürülmüyor
    os << s.dakika_ << " dk";
}
// cout << a << b;   ->  (cout << a) bir void; "void << b" diye işlem yok
```

g++ bu satırda `no match for 'operator<<' (operand types are 'void' and 'Sure')` der ve ardından kütüphanedeki onlarca `<<` adayını da sıralar; tek satırlık hata yüzlerce satırlık mesaja dönüşür. Dönüş tipini `ostream&` yerine `ostream` yazmak da derlenmez: akışı kopyalamaya çalışır.

13.4 Kopyalama: kopya yapıcı ve atama operatörü

Şimdi operatör aşırı yüklemenin en önemli ve en tehlikeli kullanımına geldik: nesneyi **kopyalamak**. `Sure` gibi yalnızca bir `int` tutan sınıflarda sorun yok; derleyicinin kendiliğinden yazdığı kopyalama zaten işini görür. Sorun, sınıfın `new` ile bellek ayırdığı zaman başlar.

Derleyicinin yazdığı kopya: sığ kopya

Bölüm 10'da `new int[n]` ile öbekte (heap) dizi ayırmayı gördün. Bu diziyi bir sınıfın içine koyalım: nesnenin kendisi yalnızca boyutu ve bir **işaretçiyi** tutar, asıl veri öbektedir.

Derleyici, sen yazmadıysan her sınıf için iki kopyalama fonksiyonu üretir: **kopya yapıcı** (copy constructor) ve **kopya atama operatörü** (copy assignment operator). İkisi de üyeleri **tek tek aynen kopyalar**. İşaretçi üye için bu, adresin kopyalanması demektir; iki nesne **aynı öbek bölgesine** bakar. Buna **sığ kopya** (shallow copy) denir. Sonuçları kötüdür (yıkıcısı `delete[] veri_;` yapan bir `Dizi` sınıfı düşün):

```
{
    Dizi a{3};
    a[0] = 7;
    Dizi b = a;        // sığ kopya: a.veri_ ve b.veri_ AYNI adresi tutar
    b[0] = 100;        // a[0] da 100 oldu: iki nesne aynı diziyi paylaşıyor
}   // b yok edilir: delete[] veri_;  sonra a yok edilir: aynı adres tekrar delete[]
    // -> çift delete: tanımsız davranış
```

Önce iki nesne bir diziyi **paylaşır** (birinde yapılan değişiklik ötekini bozar). Sonra ikisi yok edilirken aynı bellek iki kez serbest bırakılır. Çift `delete` **tanımsız davranıştır** (Bölüm 10); bu yüzden çıktısını göstermiyoruz: program çökebilir, çökmeyebilir, bir sonraki `new`'in verisini bozabilir.

Sık Yapılan Hata: Yıkıcı olan sınıfta sığ kopya

`new` kullanan, yıkıcısı `delete` yapan bir sınıfta kopya yapıcıyı ve atama operatörünü yazmazsan derleyicinin sığ kopyası devreye girer. Tek bir `Dizi b = a;` satırı çift `delete` demektir. Sınavda "bu kodun hatası nedir?" sorusunun cevabı çoğu zaman budur.

Derin kopya: kendi kopya yapıcın

Çözüm: kopyalamayı kendin yazmak ve işaretçinin gösterdiği veriyi de kopyalamak. Buna **derin kopya** (deep copy) denir: yeni nesne kendi öbek bölgesini ayırır ve içeriği oraya kopyalar. Aşağıdaki `Dizi` sınıfı yapıcı, yıkıcı, kopya yapıcı ve atama operatörünü içerir. Her biri bir mesaj yazdırır; böylece hangi satırın hangi fonksiyonu çağırdığını gözle görürsün:

derinKopya.cpp

```cpp
#include <iostream>

using namespace std;

class Dizi {
public:
    Dizi(int boyut) : boyut_{boyut}, veri_{new int[boyut]{}} {
        cout << "yapici(" << boyut_ << ")\n";
    }

    Dizi(const Dizi& diger)
        : boyut_{diger.boyut_}, veri_{new int[diger.boyut_]} {
        for (int i = 0; i < boyut_; i++) {
            veri_[i] = diger.veri_[i];
        }
        cout << "kopya yapici\n";
    }

    Dizi& operator=(const Dizi& diger) {
        cout << "atama\n";
        if (this == &diger) {
            return *this;                    // kendine atama: bir şey yapma
        }
        int* yeniVeri = new int[diger.boyut_];
        for (int i = 0; i < diger.boyut_; i++) {
            yeniVeri[i] = diger.veri_[i];
        }
        delete[] veri_;                      // eski bellek iade edilir
        veri_ = yeniVeri;
        boyut_ = diger.boyut_;
        return *this;
    }

    ~Dizi() {
        cout << "yikici(" << boyut_ << ")\n";
        delete[] veri_;
    }

    int& operator[](int i) { return veri_[i]; }
    int operator[](int i) const { return veri_[i]; }

private:
    int boyut_;
    int* veri_;
};

void goster(Dizi d) {
    cout << "goster: " << d[0] << '\n';
}

int main() {
    Dizi a{3};
    a[0] = 7;

    Dizi b = a;                  // kopya yapıcı
    b[0] = 100;
    cout << a[0] << " " << b[0] << '\n';

    Dizi c{5};
    c = a;                       // atama operatörü
    Dizi& takma = a;
    a = takma;                   // dolaylı kendine atama
```

```
    goster(a);                  // değer ile geçirme: kopya yapıcı
    cout << "bitti\n";
    return 0;
}
```

Çıktı

```
yapici(3)
kopya yapici
7 100
yapici(5)
atama
atama
kopya yapici
goster: 7
yikici(3)
bitti
yikici(3)
yikici(3)
yikici(3)
```

Önce iki küçük yeniliğe bak: `operator[]`, `a[0]` yazımını sınıfa öğretir. İlk sürüm `int&` döndürdüğü için elemanın kendisini verir; `a[0] = 7;` ile elemana yazabilirsin. `const` olan ikinci sürüm (Bölüm 11'deki `const` üye fonksiyon) kopyasını döndürür; `const Dizi&` üzerinden yalnız okumaya izin verir. İkisini birlikte yazmak `vector`'ün de yaptığı şeydir.

Çıktıyı satır satır oku:

- `Dizi b = a;` satırı **kopya yapıcıyı** çağırdı ve bu sefer `b[0] = 100` yazmak `a[0]`'ı bozmadı: `7 100`.
- `c = a;` satırında `c` zaten vardı, bu yüzden **atama operatörü** çalıştı (kopya yapıcı değil). `c`'nin eski 5 elemanlık dizisi `delete[]` ile iade edildi, yerine `a`'nın 3 elemanlık kopyası geldi.
- `a = takma;` aynı nesneye atama. `this == &diger` kontrolü bu durumda gereksiz kopyayı atlar. Bu kodda kontrol olmasa da sonuç doğru kalırdı, çünkü yeni bellek önce ayrılıp kopyalanıyor. Ama eski belleği önce silen bir atama yazsaydın, aynı nesneye atamada silinmiş belleği kopyalardın.
- `goster(a)` çağrısında parametre **değer ile geçtiği** için `d` bir kopyadır ve kopya yapıcı çalıştı. Fonksiyon bitince `d` yok oldu (`yikici(3)`). Parametreyi `const Dizi&` yapsaydın kopya olmazdı; `d[0]` de `const` sürümü çağırırdı. O sürüm olmasaydı `const` nesnede `d[0]` derlenmezdi (`discards qualifiers`).
- `main` bitince nesneler **oluşturulma sırasının tersiyle** yok edilir: `c`, `b`, `a`. Üçünün boyutu da 3.

İki ayrıntı: kopya yapıcının parametresi **referans olmak zorundadır** (`const Dizi&`); değer olarak alan bir kopya yapıcı derlenmez (g++ `invalid constructor` hatası verir), çünkü parametreyi kopyalamak için yine kopya yapıcıyı çağırmak gerekir ve bu sonsuz bir zincir olurdu. Atama operatörü ise kendi nesnesini `Dizi&` olarak döndürür (`return *this;`), böylece `x = y = z` zinciri çalışır.

Hangi satır hangisini çağırır, sınavlarda sık sorulur:

Satır	Çağrılan fonksiyon
`Dizi b = a;`	kopya yapıcı (`b` **yeni** doğuyor; `=` burada atama değil)
`Dizi b(a);` ve `Dizi b{a};`	kopya yapıcı
`b = a;` (`b` zaten var)	kopya atama operatörü
`f(a)` (parametre `Dizi d`)	kopya yapıcı
`f(a)` (parametre `const Dizi& d`)	hiçbiri

Sınavda Çıkar: Üçler kuralı (rule of three)

Bir sınıfta **yıkıcı**, **kopya yapıcı** ve **kopya atama operatörü**nden birini kendin yazmak zorunda kaldıysan, büyük olasılıkla üçünü de yazman gerekir. Nedeni: ilki yazmanı gerektiren şey (örneğin `new`/`delete`) diğer ikisinde de aynen gerekir. Birini yazıp ikisini unutursan derleyicinin sığ kopyası devreye girer.

- Atama operatörünün kalıbı: **kendine atamayı** kontrol et, yeniyi ayır ve kopyala, eskiyi `delete[]` et, `*this` döndür.

İpucu: En iyisi: bellek yönetimini kendin yapma

Üçler kuralı, sınıfın kendi kaynağını (`new` ile bellek) yönettiği durumlar içindir. Üyelerin `vector` ya da `string` gibi hazır sınıflarsa onların kopyalaması zaten derindir; derleyicinin yazdığı kopya doğru çalışır ve üçünü de yazman gerekmez. Bu yaklaşıma **sıfırlar kuralı** (rule of zero) denir. Yeni bir sınıfta ham `new`/`delete` yerine `vector` kullanabiliyorsan kullan. `Dizi` örneği mekanizmayı anlaman içindi (taşıma yapıcısı ve taşıma atamasını içeren "beşler kuralı" C++11 ile geldi, bu kitabın kapsamı dışında).

Kopyayı yasaklamak: `= delete`

Bazı nesnelerin kopyası anlamsızdır: bir banka hesabının kopyası aynı hesap numarasını taşır, iki "aynı" hesap ortaya çıkar. Böyle bir sınıfta derin kopya yazmak yerine kopyayı **tamamen yasaklayabilirsin**. Kopya yapıcının ve kopya atama operatörünün bildiriminin sonuna `= delete` yazarsın; derleyici artık onları üretmez ve kopyalamaya çalışan her satırı derleme hatasıyla durdurur. Böyle işaretlenmiş fonksiyona **silinmiş fonksiyon** (deleted function) denir:

kopyaYasak.cpp

```cpp
#include <iostream>

using namespace std;

class Kimlik {
public:
    Kimlik(int no) : no_{no} {}
    Kimlik(const Kimlik&) = delete;              // kopya yapıcı yok
    Kimlik& operator=(const Kimlik&) = delete;   // kopya atama yok
    int no() const { return no_; }

private:
    int no_;
};

int main() {
    Kimlik a{7};
    Kimlik b = a;           // kopya yapıcı silinmiş: derlenmez
    cout << b.no() << '\n';
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
kopyaYasak.cpp: In function 'int main()':
kopyaYasak.cpp:18:16: error: use of deleted function 'Kimlik::Kimlik(const Kimlik&)'
   18 |     Kimlik b = a;          // kopya yapıcı silinmiş: derlenmez
      |                ^
kopyaYasak.cpp:8:5: note: declared here
    8 |     Kimlik(const Kimlik&) = delete;             // kopya yapıcı yok
      |     ^~~~~~
kopyaYasak.cpp:18:16: note: use '-fdiagnostics-all-candidates' to display considered candidates
   18 |     Kimlik b = a;          // kopya yapıcı silinmiş: derlenmez
      |                ^
```

Mesajdaki `use of deleted function` ("silinmiş fonksiyonun kullanımı") ifadesini tanı; macOS'taki `g++` (Apple clang) aynı hatayı `call to deleted constructor` diye yazar. Kopyası yasak nesneyi fonksiyona yine `const Kimlik&` ile geçirebilirsin, çünkü referans kopya oluşturmaz. Bölüm 10'daki `unique_ptr`'nin kopyalanamamasının nedeni de budur: onun kopya yapıcısı `= delete` ile silinmiştir.

13.5 Fonksiyon şablonu

İki sayıdan büyüğünü döndüren bir fonksiyon yazalım. `int` için yazarsın, sonra `double` lazım olur, sonra `string`. Fonksiyonları aşırı yüklemek (Bölüm 6) çalışır, ama gövdelerin **birebir aynı** olduğuna bak:

```
int enBuyuk(int a, int b) { return (a < b) ? b : a; }
double enBuyuk(double a, double b) { return (a < b) ? b : a; }
string enBuyuk(string a, string b) { return (a < b) ? b : a; }
```

Her yeni tip için aynı kodu kopyalayıp yapıştırmak hem sıkıcı hem hataya açık (bir kopyayı düzeltip ötekini unutursun). **Şablon** (template) bu sorunu çözer: tipi bir **yer tutucu** yaparsın, derleyici gerektiğinde gerçek tiple kodu kendisi üretir. Fonksiyonlar için olan şablona **fonksiyon şablonu** (function template) denir:

sablonFonk.cpp

```
#include <iostream>
#include <string>

using namespace std;

class Sure {
public:
    Sure(int dakika) : dakika_{dakika} {}
    bool operator<(const Sure& d) const { return dakika_ < d.dakika_; }
    int dakika() const { return dakika_; }

private:
    int dakika_;
};

template <typename T>
T enBuyuk(T a, T b) {
    return (a < b) ? b : a;
}

template <typename T, typename U>
void ciftYaz(const T& ilk, const U& ikinci) {
    cout << ilk << " - " << ikinci << '\n';
}

int main() {
    cout << enBuyuk(3, 8) << '\n';
    cout << enBuyuk(2.5, 1.5) << '\n';
```

```
    cout << enBuyuk(string("elma"), string("armut")) << '\n';
    cout << enBuyuk<double>(3, 4.5) << '\n';
    cout << enBuyuk(Sure{45}, Sure{90}).dakika() << '\n';
    ciftYaz(42, "elma");
    ciftYaz(3.5, 'A');
    return 0;
}
```

Çıktı

```
8
2.5
elma
4.5
90
42 - elma
3.5 - A
```

Şablonun parçaları:

- `template <typename T>` satırı "aşağıdaki fonksiyon bir **şablon parametresi** `T` alıyor" der. `T` bir tip adıdır; istediğin adı verebilirsin ama gelenek `T`'dir. `typename` yerine `class` da yazılabilir; ikisi aynı işi görür.
- Fonksiyonu çağırdığında derleyici `T`'yi argümanlardan kendisi bulur. Buna **tip çıkarımı** (type deduction) denir: `enBuyuk(3, 8)` için `T = int`. Tipi kendin de söyleyebilirsin: `enBuyuk<double>(3, 4.5)` için `T = double` ve `3`, `double`'a dönüştürülür.
- Birden fazla şablon parametresi olabilir (`ciftYaz`); her biri ayrı çıkarılır. Türün adını `Sure` gibi kendi sınıfın da olabilir; `enBuyuk(Sure{45}, Sure{90})` çalıştı.
- Derleyici şablonu her **farklı tip** için bir kez gerçek bir fonksiyona dönüştürür. Bu işleme **örnekleme** (instantiation) denir. Şablon, kullanılmadığı sürece hiç kod üretmez.

Sık Yapılan Hata: Çelişen tip çıkarımı

`enBuyuk(3, 4.5)` yazarsan `a` parametresi için `T = int`, `b` için `T = double` çıkar. Derleyici ikisini uzlaştırmaz; kendiliğinden `int`'i `double` yapmaz. Çözüm: ya tipi kendin yaz (`enBuyuk<double>(3, 4.5)`) ya da iki argümanı aynı tipe getir (`enBuyuk(3.0, 4.5)`).

```
template <typename T>
T enBuyuk(T a, T b) {
    return (a < b) ? b : a;
}

int main() {
    auto sonuc = enBuyuk(3, 4.5);
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
cikarimCeliski.cpp: In function 'int main()':
cikarimCeliski.cpp:7:25: error: no matching function for call to 'enBuyuk(int, double)'
    7 |     auto sonuc = enBuyuk(3, 4.5);
      |                  ~~~~~~~^~~~~~~~
  * there is 1 candidate
    * candidate 1: 'template<class T> T enBuyuk(T, T)'
      cikarimCeliski.cpp:2:3:
          2 | T enBuyuk(T a, T b) {
            |   ^~~~~~~
      * template argument deduction/substitution failed:
        *   deduced conflicting types for parameter 'T' ('int' and 'double')
          cikarimCeliski.cpp:7:25:
              7 |     auto sonuc = enBuyuk(3, 4.5);
                |                  ~~~~~~~^~~~~~~~
```

Şablon, tipin gövdede kullanılan her işlemi **desteklemesini** bekler. `enBuyuk` yalnızca `<` kullandığı için `<` operatörü olan her tiple çalışır: `string`'in `<` operatörü hazırdır, `Sure` için ise biz `operator<` yazdık (yukarıdaki program). İşte operatör aşırı yüklemeyle şablonların buluştuğu yer burasıdır: `<` olmayan bir tiple çağırırsan hata alırsın (aşağıda göreceksin).

> **Not: Neden `enBuyuk("elma", "armut")` yazma?**
>
> Sabit metinleri doğrudan verirsen `T`, `const char*` olur ve `<` iki **adresi** karşılaştırır, metinleri değil. Sonuç anlamsızdır. Metin karşılaştırmak için `string(...)` ile sar, ya da `T`'nin `string` olmasını sağla.

> **Sınavda Çıkar: Şablon hakkında sık sorulanlar**
>
> - Şablon bir **fonksiyon değil, fonksiyon üretme tarifidir**; derleyici kullanılan her tip için ayrı fonksiyon üretir (`enBuyuk<int>`, `enBuyuk<double>` farklı fonksiyonlardır).
> - Aynı `T` iki parametrede geçiyorsa argümanların tipi **tam aynı** olmalıdır; farklıysa tip çıkarımı başarısız olur (`enBuyuk(3, 4.5)`).
> - Şablon tanımı, onu kullanan koddan **önce görülebilir** olmalıdır; bu yüzden şablonlar pratikte başlık dosyalarında yazılır (ayrı bir `.cpp`'ye koyarsan bağlayıcı hatası alırsın).

13.6 Sınıf şablonu

Fonksiyonlar gibi sınıflar da şablon olabilir. İçinde bir `int` tutan basit bir `Kutu` sınıfı düşün. `double` tutan kutu için sınıfı kopyalamak yerine, tutulan tipi şablon parametresi yaparsın. Bu bir **sınıf şablonu**dur (class template). Kullanırken tipi açıkça yazarsın: `Kutu<int>`, `Kutu<string>`.

kutuSablon.cpp

```cpp
#include <iostream>
#include <string>

using namespace std;

template <typename T>
class Kutu {
public:
    Kutu(const T& deger) : deger_{deger} {}

    const T& al() const { return deger_; }
    void koy(const T& yeni);
```

```
    friend ostream& operator<<(ostream& os, const Kutu& k) {
        return os << "[" << k.deger_ << "]";   // sınıf içinde tanımlanan friend
    }

private:
    T deger_;
};

template <typename T>
void Kutu<T>::koy(const T& yeni) {
    deger_ = yeni;
}

int main() {
    Kutu<int> sayi{42};
    Kutu<string> metin{"merhaba"};
    Kutu<double> oran{0.75};

    cout << sayi << " " << metin << " " << oran << '\n';

    sayi.koy(sayi.al() + 1);
    metin.koy("dunya");
    cout << sayi << " " << metin << '\n';

    Kutu otomatik{2.5};              // C++17: tip yapıcıdan çıkarılır
    cout << otomatik << '\n';
    return 0;
}
```

Çıktı

```
[42] [merhaba] [0.75]
[43] [dunya]
[2.5]
```

Dikkat edilecekler:

- Sınıfın başına `template <typename T>` yazarsın; sınıf içinde `T` herhangi bir tip gibi kullanılır.
- Üye fonksiyonu **sınıfın dışında** tanımlarsan her seferinde şablon başlığını tekrar yazar ve sınıf adını `Kutu<T>::` biçiminde yazarsın. Sınıf içindeki sınıf adı (`Kutu`) zaten `Kutu<T>` anlamına geldiği için içeride `<T>` yazmana gerek yok.
- `Kutu otomatik{2.5};` satırında tip yazılmadı: C++17'den itibaren derleyici yapıcıya verdiğin argümandan `T = double` çıkarır.
- Her kullanılan tip için derleyici ayrı bir sınıf üretir: `Kutu<int>` ve `Kutu<string>` birbirinden bağımsız iki farklı tiptir.

Yığın şablonu: `Yigin<T>`

Sınıf şablonunun klasik örneği **yığın** (stack) **veri yapısı**dır: son giren ilk çıkar. Bu, bellek bölgesi olan yığınla **aynı şey değil**; yalnızca adı benzer. Tabaklar üst üste dizilir; en son konan tabağı alırsın. İşlemleri: `ekle` (koy), `cikar` (en üsttekini al), `ust` (en üsttekine bak), `bos`, `boyut`. Eleman saklamak için `vector<T>` kullanacağız (Bölüm 7); böylece kopyalama ve bellek işleri `vector`'e kalır, üçler kuralı gerekmez.

yiginSablon.cpp

```
#include <iostream>
#include <string>
#include <vector>

using namespace std;
```

```cpp
template <typename T>
class Yigin {
public:
    void ekle(const T& eleman) { elemanlar_.push_back(eleman); }

    bool cikar(T& hedef) {
        if (elemanlar_.empty()) {
            return false;
        }
        hedef = elemanlar_.back();
        elemanlar_.pop_back();
        return true;
    }

    const T& ust() const { return elemanlar_.back(); }   // bos() değilken çağır
    bool bos() const { return elemanlar_.empty(); }
    int boyut() const { return static_cast<int>(elemanlar_.size()); }

private:
    vector<T> elemanlar_;
};

bool parantezDengeli(const string& metin) {
    Yigin<char> acik;
    for (char c : metin) {
        if (c == '(' || c == '[') {
            acik.ekle(c);
        } else if (c == ')' || c == ']') {
            char son;
            if (!acik.cikar(son)) {
                return false;
            }
            if ((c == ')' && son != '(') || (c == ']' && son != '[')) {
                return false;
            }
        }
    }
    return acik.bos();
}

int main() {
    Yigin<int> sayilar;
    sayilar.ekle(10);
    sayilar.ekle(20);
    sayilar.ekle(30);
    cout << "ust: " << sayilar.ust() << ", boyut: " << sayilar.boyut() << '\n';

    int deger;
    while (sayilar.cikar(deger)) {
        cout << deger << " ";
    }
    cout << "| bos mu: " << sayilar.bos() << '\n';

    cout << parantezDengeli("(a[1] + b) * (c)") << " ";
    cout << parantezDengeli("(a[1)]") << " ";
    cout << parantezDengeli("((a)") << '\n';
    return 0;
}
```

Çıktı

```
ust: 30, boyut: 3
30 20 10 | bos mu: 1
1 0 0
```

`Yigin<char>` ile parantez dengesi kontrolü, yığının en bilinen kullanım yeridir: her açan paranteze `ekle`, her kapayana `cikar`; son çıkan açılış kapanışla eşleşmiyorsa ya da yığın boşken kapanış geldiyse ya da sonda yığın boş değilse ifade dengesizdir. Bölüm 15'te STL'in hazır `stack` ve `vector` şablonlarını göreceksin; burada onların nasıl yazıldığını anlamak için kendimiz yazdık. Pop işleminin boş yığında ne yapacağını burada `bool` döndürerek çözdük; C++'ın bunun için daha doğal bir yolu var, **istisna** (exception): Bölüm 14'te göreceksin.

> **İpucu: Şablonu önce somut tiple yaz**
>
> Şablon yazarken önce `int` ile çalışan sıradan bir sınıf yaz, çalıştığından emin ol, sonra `int`'leri `T` ile değiştir. Hata yaparsan, mesajı somut sınıfta okumak çok daha kolaydır.

13.7 Şablon hata mesajları

Şablonlarda en çok yorulacağın yer hata mesajlarıdır. Yukarıdaki `enBuyuk` şablonunu `<` operatörü olmayan bir sınıfla çağıralım:

sablonHata.cpp

```
template <typename T>
T enBuyuk(T a, T b) {
    return (a < b) ? b : a;
}

class Nokta {
public:
    int x;
    int y;
};

int main() {
    Nokta p{1, 2};
    Nokta q{3, 4};
    enBuyuk(p, q);
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
sablonHata.cpp: In instantiation of 'T enBuyuk(T, T) [with T = Nokta]':
sablonHata.cpp:15:12:   required from here
   15 |     enBuyuk(p, q);
      |     ~~~~~~~^~~~~~
sablonHata.cpp:3:15: error: no match for 'operator<' (operand types are 'Nokta' and 'Nokta')
    3 |     return (a < b) ? b : a;
      |            ~~~^~~~
```

Hata, `enBuyuk` şablonunun **içinde** (`a < b` satırında) çıkıyor, ama senin yazdığın hata `main`'deki çağrı. Derleyici bu yüzden iki bilgi verir: hatanın şablonun hangi satırında olduğunu ve hangi tiple örneklendiğinde olduğunu (`In instantiation of ... [with T = Nokta]` satırı). Mesajı okurken önce **ilk `error:` satırına**, sonra `with T = ...` kısmına bak: sorun çoğu zaman "bu tip bu operatörü desteklemiyor"dur.

Not: Mesajlar neden uzar?

Şablon hataları üç sebeple uzar. Birincisi, hata şablonun derinliklerinde çıkar ve derleyici seni oraya götüren **tüm çağrı zincirini** yazar. İkincisi, `<` gibi bir operatör için tüm adayları (kütüphanedeki yüzlerce aşırı yüklenmiş sürüm dahil) sıralar. Üçüncüsü, tipler kısaltılmadan yazılır: `Kutu<vector<string>>` gibi tiplerde bir satır ekrana sığmaz. Burada küçük bir örnekle durduk; `<iostream>` ve `<vector>` ekleyip karmaşık bir şablon çağırınca mesaj yüzlerce satıra çıkar. Alışkanlık: **ilk `error:` satırını** oku, gerisini sadece gerekirse incele. Apple'ın "g++" komutu aslında clang olduğu için macOS'ta mesaj biçimi farklı olabilir; bu bölümdeki çıktılar gerçek g++ ile alındı.

C++20/23 Notu: Kavramlar (concepts) ve `<=>`

C++20'de şablon parametresine koşul koyabilirsin; koşul sağlanmazsa hata, şablonun derinlerinde değil çağrı satırında ve kısa bir cümleyle çıkar. `<=>` ("uzay gemisi") operatörünü `= default` yazmak da `<`, `>`, `<=`, `>=` operatörlerini tek satırda üretir (`==` için ayrıca `bool operator==(const Sure&) const = default;` gerekir). Bu kitap C++17 kullandığı için örnek çalıştırılmıyor:

```
template <std::totally_ordered T>            // <concepts> başlığı
T enBuyuk(T a, T b) { return (a < b) ? b : a; }

auto operator<=>(const Sure&) const = default;   // Sure sınıfının içinde
```

Bölüm Özeti – Bilmen Gerekenler

- **Operatör aşırı yükleme:** `operator+`, `operator==`, `operator<` gibi özel adlı fonksiyonlarla operatörü kendi sınıfına öğretirsin; `a + b`, `a.operator+(b)` demektir. Yeni operatör icat edilmez, öncelik değişmez, en az bir işlenen kendi tipin olmalı.
- Aşırı yüklenemeyenler: `::`, `.`, `.*`, `?:`, `sizeof`. `=`, `[]`, `()`, `->` yalnızca üye olabilir. `&&`/`||` aşırı yüklenirse kısa devre kaybolur.
- Üye fonksiyonda sol işlenen `*this`'tir. Sol işlenen başka tipse (`3 * ders`, `cout << ders`) üye olmayan fonksiyon yazılır.
- Karşılaştırma ve `+` operatörleri `const` olmalı, parametreyi `const T&` almalı. `+` yeni nesne döndürür, `+=` ise `T&` ile `*this` döndürür; `+` genelde `+=` üzerinden yazılır.
- `ostream& operator<<(ostream& os, const T& x)` kalıbı: üye değil, `os`'u referansla döndürür (zincirleme için). Özel üyelere erişim gerekirse `friend`.
- Derleyicinin yazdığı kopya **sığ kopyadır** (işaretçi üye adresle kopyalanır). Yıkıcısı `delete` yapan sınıfta bu **çift delete** demektir. **Derin kopya** için kopya yapıcıyı ve atama operatörünü kendin yaz.
- `Dizi b = a;` kopya yapıcıyı, `b = a;` (b var) atama operatörünü çağırır; değer ile geçirme de kopya yapıcıyı çağırır. Kopya yapıcının parametresi referans olmak zorundadır. Atama operatöründe kendine atamayı kontrol et ve `*this` döndür.
- **Üçler kuralı:** yıkıcı, kopya yapıcı ve atama operatöründen birine ihtiyaç duyuyorsan üçüne de ihtiyacın var. `vector`/`string` üyeler varsa hiçbirini yazma (sıfırlar kuralı). Kopyayı yasaklamak için `= delete`.
- **Fonksiyon şablonu:** `template <typename T> T f(T a, T b)`; derleyici kullanılan her tip için örnekler. Aynı `T` iki parametrede farklı tiplerle çıkarsa hata verir; `f<double>(3, 4.5)` ile tip açıkça verilir. Tipin gövdede kullanılan operatörleri desteklemesi gerekir.
- **Sınıf şablonu:** `template <typename T> class Kutu`; kullanırken tip yazılır (`Kutu<int>`), C++17'de yapıcıdan çıkarılabilir. Üyeyi dışarıda tanımlarken `template <typename T> ... Kutu<T>::f()`. `Yigin<T>` son giren ilk çıkar.

- Şablon tanımları başlık dosyasında olur. Şablon hata mesajlarını okurken ilk `error:` satırına ve `with T = ...` kısmına bak.

Sorular

A) Çoktan Seçmeli

1 Operatör aşırı yükleme ile ilgili aşağıdaki ifadelerden hangisi **doğrudur**?

(A) `int` tipinin `+` operatörünün anlamı, iki `int` toplanırken değiştirilebilir.

(B) `**` ile üs alma operatörü tanımlanabilir.

(C) `operator[]` yalnızca üye fonksiyon olarak yazılabilir.

(D) `?:` operatörü sınıflar için aşırı yüklenebilir.

(E) Aşırı yüklenen `+` operatörünün önceliği `*` operatöründen yüksek hâle gelir.

2 `Para` sınıfı için `cout << p1 << p2 << '\n';` satırının çalışabilmesi için `operator<<` aşağıdakilerden hangisi gibi bildirilmelidir?

(A) `void operator<<(ostream& os, const Para& p)`

(B) `ostream operator<<(ostream os, const Para& p)`

(C) `ostream& operator<<(const Para& p)` (`Para`'nın üye fonksiyonu)

(D) `ostream& operator<<(ostream& os, const Para& p)` (üye olmayan fonksiyon)

(E) `Para& operator<<(ostream& os, const Para& p)` (üye olmayan fonksiyon)

3 Aşağıdaki program ekrana ne yazdırır?

```
#include <iostream>

using namespace std;

class Iz {
public:
    Iz() {}
    Iz(const Iz&) { cout << "K"; }
    Iz& operator=(const Iz&) { cout << "A"; return *this; }
};

void f(Iz) {}
void g(const Iz&) {}

int main() {
    Iz a;
    Iz b = a;
    Iz c(b);
    c = a;
    f(a);
    g(a);
    Iz d{a};
    cout << '\n';
    return 0;
}
```

(A) `KKAAK` (B) `KKAKK` (C) `KAAKK` (D) `KKAK` (E) `KAKKK`

4 Aşağıdaki şablon verilmiştir. `kucuk(3, 4.5)` çağrısı derlenmemektedir. Sorunu gidermenin **doğru** yolu hangisidir?

```
template <typename T>
T kucuk(T a, T b) {
    return (a < b) ? a : b;
}
```

(A) Şablon başlığını `template <typename T, typename T>` yapmak

(B) Çağrıyı `kucuk<int, double>(3, 4.5)` yapmak

(C) `typename` yerine `class` yazmak

(D) Fonksiyonun başına `inline` eklemek

(E) Çağrıyı `kucuk<double>(3, 4.5)` yapmak

5 `Tampon` sınıfı yapıcısında `new char[n]` ile bellek ayırmakta ve yıkıcısında `delete[]` yapmaktadır; kopya yapıcı ve atama operatörü yazılmamıştır. `Tampon a{10}; Tampon b = a;` satırlarından sonra aşağıdakilerden hangisi doğrudur?

(A) Sığ kopya yapılır; `a` ve `b` yok edilirken aynı belleğe iki kez `delete[]` uygulanır. Kopya yapıcı ve atama operatörü de yazılmalıdır.

(B) Kopya yapıcı olmadığı için derleme hatası alınır.

(C) Derleyici derin kopya yapar, sorun olmaz.

(D) Yalnızca atama operatörü yazılmalıdır; `Tampon b = a;` zaten atama operatörünü çağırır.

(E) `b = a;` yazılırsa program çalışır ama `Tampon b = a;` yazılırsa çalışmaz.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır? (Son satırdaki yıkıcı çıktılarını da unutma.)

```
#include <iostream>

using namespace std;

class Iz {
public:
    Iz(int no) : no_{no} { cout << "Y" << no_ << " "; }
    Iz(const Iz& d) : no_{d.no_ + 10} { cout << "K" << no_ << " "; }
    Iz& operator=(const Iz& d) {
        no_ = d.no_ + 20;
        cout << "A" << no_ << " ";
        return *this;
    }
    ~Iz() { cout << "X" << no_ << " "; }
    int no() const { return no_; }

private:
    int no_;
};

void f(Iz x) {
    cout << "f" << x.no() << " ";
}

int main() {
    Iz a{1};
    Iz b = a;
    b = a;
    f(a);
    cout << '\n';
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
#include <string>

using namespace std;

template <typename T>
T topla(T a, T b) {
    return a + b;
}

int main() {
    cout << topla(7, 2) / 2 << ' ';
    cout << topla<double>(7, 2) / 2 << ' ';
    cout << topla<int>(2.9, 3.9) << ' ';
    cout << topla(string("ab"), string("cd")) << '\n';
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>

using namespace std;

class Nokta {
public:
    Nokta(int x, int y) : x_{x}, y_{y} {}

    Nokta operator+(const Nokta& d) const { return Nokta{x_ + d.x_, y_ + d.y_}; }
    Nokta& operator+=(const Nokta& d) {
        x_ += d.x_;
        y_ += d.y_;
        return *this;
    }
    bool operator==(const Nokta& d) const { return x_ == d.x_ && y_ == d.y_; }

    friend ostream& operator<<(ostream& os, const Nokta& n) {
        return os << "(" << n.x_ << "," << n.y_ << ")";
    }

private:
    int x_;
    int y_;
};

int main() {
    Nokta a{1, 2};
    Nokta b{3, 4};
    Nokta c = a + b;
    a += b;
    (a += b) += b;
    (c + b) += b;
    cout << a << " " << c << " " << (a == c) << '\n';
    return 0;
}
```

C) Kod Yazma

9 Pay ve paydası olan bir `Kesir` sınıfı yaz. Yapıcı kesri **sadeleştirsin** (`<numeric>` içindeki `gcd` işine yarar) ve paydayı pozitif tutsun. Sınıf şu operatörleri içersin: `+`, `*`, `<` ve `cout` için `<<` (`3/8` biçiminde yazsın). Kullanıcıdan iki kesri (pay ve payda ikilileri olarak) oku; toplamlarını, çarpımlarını ve birincinin ikinciden küçük olup olmadığını yazdır. Paydalar sıfır girilmeyecek. Örnek:

```
Birinci kesir (pay payda): 1 2
Ikinci kesir (pay payda): 3 4
1/2 + 3/4 = 5/4
1/2 * 3/4 = 3/8
1/2 < 3/4 : evet
```

10 Bu bölümdeki `Yigin<T>` gibi, ama **ilk giren ilk çıkar** çalışan bir `Kuyruk<T>` sınıf şablonu yaz. Elemanları `vector<T>` içinde tutsun; `ekle(const T&)`, `cikar(T&)` (kuyruk boşsa `false`), `bos()` ve `boyut()` fonksiyonları olsun. `main`'de bir `Kuyruk<string>` ile üç ismi ekleyip sırayla çağır; bir `Kuyruk<int>` ile 1'den 5'e kadar sayıları ekleyip çıkararak toplamlarını yazdır. Örnek çıktı:

```
Siradaki: Ali
Siradaki: Ece
Siradaki: Can
Kalan: 0
Toplam: 15
```

Çözümler

Cevap anahtarı

1 C	2 D	3 B	4 E	5 A

1 Cevap: C

`=`, `[]`, `()` ve `->` yalnızca üye fonksiyon olarak yazılabilir. A: en az bir işlenen sınıf tipinde olmalıdır, yerleşik `int + int` değiştirilemez. B: yeni operatör icat edilemez. D: `?:` aşırı yüklenemeyen operatörlerdendir (`::`, `.`, `.*`, `sizeof` ile birlikte). E: aşırı yükleme önceliği değiştirmez; `*` yine `+`'dan önce çalışır.

2 Cevap: D

`cout << p1 << p2` işleminde sol işlenen `cout` olduğu için fonksiyon `Para`'nın üyesi olamaz (C yanlış; üye olsaydı `p1 << cout` yazılırdı). Zincirleme için akışı referansla döndürmek gerekir: `(cout << p1)` yine `cout` olmalıdır. A `void` döndürdüğü için zincir kırılır. B akışı değer olarak alır ve döndürür; akış nesneleri kopyalanamaz, derlenmez. E `Para&` döndürdüğü için `Para << Para` anlamsızlaşır, `'\n'` yazdırılamaz.

3 Cevap: B

Satır satır: `Iz b = a;` kopya yapıcı (K); `Iz c(b);` kopya yapıcı (K); `c = a;` c zaten var, atama operatörü (A); `f(a)` değer ile geçirme, kopya yapıcı (K); `g(a)` referans, hiçbir şey çağrılmaz; `Iz d{a};` kopya yapıcı (K). Sonuç `KKAKK`. A atamayı iki kez sayar, C ve E sıralamayı karıştırır, D `d{a}` satırını unutur.

Çıktı

```
KKAKK
```

4 Cevap: E

`kucuk(3, 4.5)`'te `a` için `T = int`, `b` için `T = double` çıkar; çelişki var. `kucuk<double>(3, 4.5)` tipi kendin söylersin ve `3`, `double`'a dönüştürülür. A: aynı adı iki şablon parametresine veremezsin. B: şablonun tek parametresi var, iki tip verilemez, derlenmez. C: `typename` ile `class` aynı işi görür, çelişkiyi çözmez. D: `inline` tip çıkarımıyla ilgisizdir.

5 Cevap: A

Derleyicinin ürettiği kopya yapıcı üyeleri aynen kopyalar; işaretçi adresle kopyalandığı için `a` ve `b` aynı diziyi paylaşır (sığ kopya). Yıkıcı iki nesne için de aynı belleğe `delete[]` uygular: çift `delete`, tanımsız davranış. Bu yüzden üçler kuralı gereği kopya yapıcı ve atama operatörü de yazılmalıdır. B: derleyici kopya yapıcıyı kendisi üretir, derlenir. C: derleyici derin kopya yapmaz. D: `Tampon b = a;` atama değil, kopya yapıcıdır; ikisi de gerekir. E: ikisi de aynı sorunu yaratır.

6

Adım adım:

Satır	Çağrılan	Yazdırılan
`Iz a{1};`	yapıcı	`Y1`
`Iz b = a;`	kopya yapıcı (`b.no_` = 1 + 10)	`K11`
`b = a;`	atama (`b.no_` = 1 + 20)	`A21`
`f(a)`	kopya yapıcı: `x.no_` = 11	`K11`, sonra `f11`
`f` biter	`x` yok edilir	`X11`
`main` biter	önce sonradan oluşan `b`, sonra `a`	`X21`, `X1`

`'\n'` `f(a)` çağrısından sonra yazıldığı için `X11` ilk satırda kalır; `main` sonundaki yıkıcılar (`X21 X1`) ikinci satırda görünür.

Çıktı
```
Y1 K11 A21 K11 f11 X11
X21 X1
```

7

- `topla(7, 2)` → `T = int`, sonuç 9; `9 / 2` tamsayı bölmesi, yani 4.
- `topla<double>(7, 2)` → 7 ve 2 `double`'a çevrilir, 9.0; `9.0 / 2` = 4.5.
- `topla<int>(2.9, 3.9)` → argümanlar `int`'e çevrilirken ondalık kısım atılır: 2 + 3 = 5. (macOS'taki `g++`, yani Apple clang, bu satırda veri kaybı için uyarı verir; g++ susar.)
- `topla(string("ab"), string("cd"))` → `string` için `+` birleştirmedir: `abcd`.

Çıktı
```
4 4.5 5 abcd
```

8

- `c = a + b` → `c` = (4,6); `a` değişmez, hâlâ (1,2).
- `a += b` → `a` = (4,6).
- `(a += b) += b`: iç `+=` `a`'nın kendisini döndürür (`Nokta&`), yani `a` iki kez artar: önce (7,10), sonra (10,14).
- `(c + b) += b`: `c + b` **geçici** bir nesnedir; `+=` onu değiştirir, `c` aynen (4,6) kalır.
- `a == c` → (10,14) ile (4,6) eşit değil, `0` yazılır (`bool` değeri `cout` ile `0`/`1` yazılır).

Çıktı
```
(10,14) (4,6) 0
```

9

Yapıcı `gcd` ile sadeleştirir; payda negatifse pay ile paydanın işaretini çevirir. İki kesri karşılaştırmak için çapraz çarpım kullanılır: `a/b < c/d` ancak `a*d < c*b` iken doğrudur (paydalar pozitif olduğu için).

kesir.cpp
```cpp
#include <iostream>
#include <numeric>

using namespace std;

class Kesir {
```

```
public:
    Kesir(int pay, int payda) : pay_{pay}, payda_{payda} {
        if (payda_ < 0) {
            pay_ = -pay_;
            payda_ = -payda_;
        }
        int ortak = gcd(pay_, payda_);
        if (ortak > 1) {
            pay_ /= ortak;
            payda_ /= ortak;
        }
    }

    Kesir operator+(const Kesir& d) const {
        return Kesir{pay_ * d.payda_ + d.pay_ * payda_, payda_ * d.payda_};
    }

    Kesir operator*(const Kesir& d) const {
        return Kesir{pay_ * d.pay_, payda_ * d.payda_};
    }

    bool operator<(const Kesir& d) const {
        return pay_ * d.payda_ < d.pay_ * payda_;
    }

    friend ostream& operator<<(ostream& os, const Kesir& k) {
        return os << k.pay_ << "/" << k.payda_;
    }

private:
    int pay_;
    int payda_;
};

int main() {
    int p1{0}, d1{1}, p2{0}, d2{1};
    cout << "Birinci kesir (pay payda): ";
    cin >> p1 >> d1;
    cout << "Ikinci kesir (pay payda): ";
    cin >> p2 >> d2;

    Kesir a{p1, d1};
    Kesir b{p2, d2};
    cout << a << " + " << b << " = " << a + b << '\n';
    cout << a << " * " << b << " = " << a * b << '\n';
    cout << a << " < " << b << " : " << ((a < b) ? "evet" : "hayir") << '\n';
    return 0;
}
```

Çıktı

```
Birinci kesir (pay payda): 1 2
Ikinci kesir (pay payda): 3 4
1/2 + 3/4 = 5/4
1/2 * 3/4 = 3/8
1/2 < 3/4 : evet
```

10

`Yigin<T>`'ten tek farkı, elemanın **baştan** alınmasıdır: `vector`'ün başından silmek için `v.erase(v.begin())` yazılır: `begin()` ilk elemanın konumunu verir, `erase` o elemanı siler ve kalanlar bir sola kayar (ikisini de Bölüm 15'te ayrıntılı göreceksin). (Büyük kuyruklarda bu yavaştır; Bölüm 15'te hazır `queue` şablonunu göreceksin.)

kuyruk.cpp

```cpp
#include <iostream>
#include <string>
#include <vector>

using namespace std;

template <typename T>
class Kuyruk {
public:
    void ekle(const T& eleman) { elemanlar_.push_back(eleman); }

    bool cikar(T& hedef) {
        if (elemanlar_.empty()) {
            return false;
        }
        hedef = elemanlar_.front();
        elemanlar_.erase(elemanlar_.begin());
        return true;
    }

    bool bos() const { return elemanlar_.empty(); }
    int boyut() const { return static_cast<int>(elemanlar_.size()); }

private:
    vector<T> elemanlar_;
};

int main() {
    Kuyruk<string> sira;
    sira.ekle("Ali");
    sira.ekle("Ece");
    sira.ekle("Can");

    string ad;
    while (sira.cikar(ad)) {
        cout << "Siradaki: " << ad << '\n';
    }
    cout << "Kalan: " << sira.boyut() << '\n';

    Kuyruk<int> sayilar;
    for (int i = 1; i <= 5; i++) {
        sayilar.ekle(i);
    }
    int toplam{0};
    int sayi{0};
    while (sayilar.cikar(sayi)) {
        toplam += sayi;
    }
    cout << "Toplam: " << toplam << '\n';
    return 0;
}
```

Çıktı

```
Siradaki: Ali
Siradaki: Ece
Siradaki: Can
Kalan: 0
Toplam: 15
```

PROJE 2

Proje: Banka Hesabı Simülasyonu

Bu proje, 9. bölümden 13. bölüme kadar öğrendiğin her şeyi **tek bir programda** birleştiriyor. Küçük bir bankanın hesap sistemini yazacaksın: hesap açılacak, para yatırılıp çekilecek, hesaplar arasında transfer yapılacak, ay sonunda faizler işletilecek. Programı bir kerede vermeyeceğiz; **dört adımda** büyüteceğiz ve her adımın sonunda derlenip çalışan bir program olacak. Her adımı önce kendin yazmayı dene, takıldığın yerde açıklamalara bak.

Bölüm	Projede nerede?
9 · İşaretçiler	Hesabı bulamayınca `nullptr` döndüren `bul`, sahip olmayan `Hesap*`
10 · Dinamik bellek	`make_unique`, `unique_ptr`: hesapların sahibi `Banka`
11 · Sınıflar	`private` veri, yapıcı, `const` üye fonksiyon, `enum class`
12 · Kalıtım	`Hesap` soyut sınıfı, `VadesizHesap` ve `VadeliHesap`, `virtual` / `override`
13 · Operatör aşırı yükleme	`cout << hesap`

Öncesinden de şunları kullanacaksın: `getline` ile güvenli girdi okuma (Bölüm 8'deki not projesinin fonksiyonları aynen geri geliyor), `setw` ile hizalama (Bölüm 3), `vector` (Bölüm 7) ve `to_string` (Bölüm 8).

> **Bu Bölümde Öğreneceklerin**
>
> Bu projeyi bitirince:
>
> - ortak davranışı bir temel sınıfta, farklı kuralları türetilmiş sınıflarda yazabilirsin;
> - farklı türden nesneleri tek bir `vector` içinde tutup hepsine aynı çağrıyla ulaşabilirsin;
> - parayı ondalık sayı yerine tamsayı olarak tutmanın nedenini anlatabilirsin.

Amaç

Bankada iki tür hesap var. **Vadesiz hesap** eksiye düşebilir (en çok 1000 TL), düşerse ay sonunda borcun %5'i faiz olarak kesilir. **Vadeli hesap** eksiye düşemez, ay sonunda yıllık orana göre faiz kazanır, ama erken çekimde %1 komisyon öder. Kullanıcı ne yazarsa yazsın program **çökmemeli**: yetersiz bakiye, olmayan hesap numarası, harf yazılmış sayı... Her hatalı durum anlaşılır bir mesajla bildirilir ve menü çalışmaya devam eder.

Gereksinimler

İşlevler (menü seçenekleri):

1. **Hesap aç:** Tür (1 vadesiz, 2 vadeli) ve ad soyad sorulur; vadeli hesapta yıllık faiz yüzdesi de sorulur. Banka sırayla `1001`, `1002`, ... numaraları verir.
2. **Yatır:** Hesap numarası ve tutar (TL) sorulur.
3. **Çek:** Çekilecek tutar ve komisyonu, hesabın çekilebilir sınırını aşıyorsa istek reddedilir.
4. **Transfer:** Gönderen hesapta para yetmezse **hiçbir hesap değişmez**.
5. **Döküm:** Bir hesabın işlem geçmişi, her işlemden sonraki bakiyeyle.

6. **Ay sonu:** Bütün hesaplar kendi kuralıyla faiz işletir: vadeli hesaba aylık faiz eklenir (yıllık yüzde / 12), eksideki vadesiz hesaptan borcun %5'i kesilir.
7. **Listele:** Bütün hesaplar ve bankadaki toplam para.
8. **Çıkış.**

Teknik kurallar:

- `Hesap` **soyut** olmalı; hesap türleri ondan türemeli. Hesaplar `vector<unique_ptr<Hesap>>` içinde tutulmalı.
- Tutarlar menüde **tam TL** olarak (1–100000) istenir; içeride kuruş olarak saklanır.
- Bu projede **istisna kullanılmaz**: istisnaları Bölüm 14'te öğreneceksin. "Yetersiz bakiye" gibi durumları şimdilik `bool` ya da bir `enum class` dönüşüyle bildireceğiz; Bölüm 14'te bu projeyi istisnalarla yeniden yazmak iyi bir alıştırma olacak.
- Bütün girdiler `getline` ile satır olarak okunur; `cin >>` kullanılmaz.

Tasarım

Sınıflar ve sorumlulukları

Sınıf	Tür	Önemli üyeler	Sorumluluğu
`Hesap`	**Soyut** temel sınıf	`no`, `sahip`, `bakiye`, `gecmis`; `yatir`, `cek`, `dokum`; saf sanal `tur`, `cekilebilir`, `aySonu`; sanal `komisyon`	Ortak davranış: bakiye değişimi, kayıt
`VadesizHesap`	`Hesap`'tan türer	`EK_HESAP_LIMITI`	Ek hesap kuralı, eksiye faiz
`VadeliHesap`	`Hesap`'tan türer	`yillikYuzde`	Eksiye düşmeme kuralı, ay sonu faizi, erken çekim komisyonu
`TransferSonucu`	`enum class`	`Tamam`, `HesapYok`, `AyniHesap`, `YetersizBakiye`	Transferin neden olmadığını anlatır
`Banka`	Sınıf	`hesaplar`, `sonNo`; `hesapAc`, `bul`, `transfer`, `aySonu`, `listele`	Hesapları saklar, numara verir, transferi yönetir
Serbest fonksiyonlar		`paraYaz`, `satirOku`, `sayiOku`, `metinOku`, menü ekranları	Kullanıcıyla konuşur

Yazım sırası: `Hesap` ve `VadesizHesap` → `VadeliHesap` → `Banka` → menü. Menü en sona kalır; çünkü sınıflar doğru çalışmadan üstüne ekran yazmanın anlamı yok.

Para neden `double` değil?

Bölüm 2'de `0.1 + 0.2`'nin tam `0.3` olmadığını görmüştün. Bankada kuruş hatası kabul edilmez; bu yüzden parayı **kuruş cinsinden** `long long` tutuyoruz: 12.50 TL, `1250` olur. Buna sabit noktalı gösterim (fixed-point representation) denir. Ekrana yazarken `paraYaz` kuruşu tekrar `12.50` biçimine çevirir.

● Sık Yapılan Hata: Parayı double ile toplama

On kez `0.1` toplamak `1.0` etmez:

```cpp
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    double toplam = 0;
    for (int i = 0; i < 10; i++) {
        toplam += 0.1;
    }
    cout << boolalpha << setprecision(17);
    cout << toplam << ", 1.0'a eşit mi? " << (toplam == 1.0) << '\n';
    return 0;
}
```

Çıktı

```
0.99999999999999989, 1.0'a eşit mi? false
```

Tamsayıda ise `10 * 10 == 100` her zaman doğrudur. Faiz hesabındaki bölmeler (`/ 100`, `/ 1200`) tamsayı bölmesi olduğundan kuruşun altı **atılır** (Bölüm 2); tutarlı ve tahmin edilebilir bir kural.

Hata durumları

Durum	Nasıl bildirilir?	Kim bildirir?
Çekim (komisyon dahil) çekilebilir tutarı aşıyor	`cek` `false` döner	`Hesap::cek`
Tutar sıfır ya da negatif	`yatir` / `cek` `false` döner	`Hesap` (menü zaten 1–100000 ister)
Olmayan hesap numarası	`bul` `nullptr` döner	`Banka::bul`
Transfer olmadı	`TransferSonucu` değeri döner	`Banka::transfer`
Harf, boş satır, aralık dışı sayı	Uyarı yazıp **tekrar sorar**	`sayiOku`

Sahiplik: kim neyi siler?

Hesap nesneleri öbekte (heap) yaşar ve **tek sahibi** `Banka`'nın `vector<unique_ptr<Hesap>>`'ıdır. Bankanın dışına verdiğimiz `Hesap*` işaretçileri yalnız **gösterir**, silme görevi onlarda değildir (sahip olmayan işaretçi, non-owning pointer). Bölüm 10'daki kuralı hatırla: ham adres sahiplik vermez; onu `delete` etmezsin ve başka bir akıllı işaretçiye de vermezsin.

```
Banka
 └─ hesaplar : vector<unique_ptr<Hesap>>
      [0] ──► VadesizHesap 1001   (öbekte; Banka silince yıkıcı çalışır)
      [1] ──► VadeliHesap  1002

 menüdeki Hesap* h  ──►  yalnız gösterir, silmez
```

● İpucu: Önce sınıfları, sonra menüyü yaz

Menüyle başlamak cazip gelir ama sınıflar yanlışsa menü de yanlış çalışır. Her adımın sonunda küçük bir `main` ile sınıfı dene: birkaç hesap aç, para yatır, çek, çıktıya bak. Menüyü yalnız hepsi çalışınca ekle.

Adım adım geliştirme

Dosyayı `banka.cpp` olarak kaydet; derle ve çalıştır. Windows'ta Türkçe harfler bozuk görünürse Bölüm 3'teki gibi önce `chcp 65001` yaz.

```
g++ -std=c++17 -Wall -Wextra banka.cpp -o banka
./banka
```

Adım 1: Temel sınıf `Hesap` ve `VadesizHesap`

Önce programın **çekirdeğini** yazıyoruz: ortak davranışı taşıyan `Hesap` ve ilk türetilmiş sınıf. Menü, girdi, `Banka` yok; yalnız bir `main` sınıfı deniyor.

`Hesap`'ı satır satır okuyalım:

- **Kapsülleme.** `no`, `sahip`, `bakiye` ve `gecmis` `private` (Bölüm 11): dışarıdan okumak için `noAl`, `sahipAl`, `bakiyeAl` adlı `const` üye fonksiyonlar (getter) var; dışarıdan yazmak için **hiçbir yol yok**. Bakiyeyi değiştirmenin tek yolu `yatir` ve `cek`'tir. Böylece her hareket geçmişe yazılır ve kimse bakiyeyi sessizce değiştiremez.
- **`protected` yardımcı.** `degistir` hem bakiyeyi değiştirir hem geçmişe satır ekler. `protected` olduğu için dışarıdan çağrılamaz, ama türetilmiş sınıflar çağırabilir (Bölüm 12): ay sonu faizi bu yolla işlenecek. Alt sınıflar `bakiye`'ye **doğrudan** erişemez; `private` üye türetilmiş sınıfa da kapalıdır. Okumak için `bakiyeAl()`, değiştirmek için `degistir` kullanırlar.
- **Yapıcı.** Üye ilk değer listesi (Bölüm 11) `no` ve `sahip`'i doldurur, gövde geçmişin ilk satırını yazar.
- **Sanal yıkıcı.** `virtual ~Hesap() = default;` (Bölüm 12): nesneler temel sınıf işaretçisiyle (`unique_ptr<Hesap>`) silineceği için şart.
- **Saf sanal fonksiyonlar.** `tur`, `cekilebilir` ve `aySonu = 0` ile bildirildi (Bölüm 12): `Hesap` artık **soyut sınıf**tır, `Hesap h;` yazılamaz. Her hesap türü bunları kendi kuralıyla yazmak zorundadır.
- **Gövdesi olan sanal fonksiyon.** `komisyon` saf değil: gövdesi `0` döndürüyor. Komisyon almayan türler hiçbir şey yazmaz; alan tür `override` eder. Parametre adını yoruma almamız (`/*tutar*/`) "bu parametreyi kullanmıyorum" demektir; yoksa `-Wextra` uyarı verir.
- **`cek` bir `bool` döndürür.** İstisnaları Bölüm 14'te göreceksin; şimdilik "yetersiz bakiye" `false` ile bildirilir. `cek` içindeki `komisyon(tutar)` ve `cekilebilir()` çağrıları **sanal**: hesabın gerçek türü hangisiyse onun kuralı çalışır (dinamik bağlama, Bölüm 12). `cek` yazılırken türler daha yazılmamıştı bile!
- **`paraYaz`.** `to_string` ile (Bölüm 8) kuruşu `lira.kuruş` metnine çevirir; `kr < 10` ise araya `0` koyar (`5` kuruş `.05` olmalı).
- **`operator<<`.** Bölüm 13'teki kalıp: `ostream&` alır, `ostream&` döndürür. Parametre `const Hesap&` olduğu için **her** hesap türüyle çalışır ve `h.tur()` çağrısı sanaldır, yani satırda doğru tür adı yazar. Sütunlar Bölüm 3'teki `setw` ile hizalanır; `left` ve `right` kalıcıdır, bu yüzden fonksiyon sonunda `right`'a döner.

`VadesizHesap` (`: public Hesap`, Bölüm 12) üç saf sanal fonksiyonu `override` eder (kendi eklediği hiçbir üye değişkeni yok; kurallar sabit değerlerde). Temel sınıf yapıcısı üye ilk değer listesinde çağrılır: `Hesap(no, sahip)`. Eksiye düşen hesabın çekilebilir tutarı `bakiye + 1000 TL` olur; ek hesap faizi ay sonunda borcun %5'idir.

hesapSinifi.cpp

```cpp
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>

using namespace std;

const long long TL = 100;   // 1 TL = 100 kuruş; paralar kuruş olarak tutulur
```

```
// 123456 kuruş -> "1234.56"; eksi bakiyede "-12.30"
string paraYaz(long long kurus) {
    string isaret = kurus < 0 ? "-" : "";
    long long mutlak = kurus < 0 ? -kurus : kurus;
    long long kr = mutlak % 100;
    return isaret + to_string(mutlak / 100) + "." +
           (kr < 10 ? "0" : "") + to_string(kr);
}

class Hesap {
private:
    int no;
    string sahip;
    long long bakiye{0};                // kuruş
    vector<string> gecmis;              // işlem geçmişi

protected:
    // Bakiyeyi değiştirir ve geçmişe yazar; yalnız Hesap ve türevleri çağırır
    void degistir(long long fark, const string& aciklama) {
        bakiye += fark;
        gecmis.push_back(aciklama + ": " + (fark > 0 ? "+" : "") +
                         paraYaz(fark) + " -> bakiye " + paraYaz(bakiye));
    }

public:
    Hesap(int yeniNo, const string& yeniSahip) : no{yeniNo}, sahip{yeniSahip} {
        gecmis.push_back("Hesap açıldı");
    }
    virtual ~Hesap() = default;

    int noAl() const { return no; }
    const string& sahipAl() const { return sahip; }
    long long bakiyeAl() const { return bakiye; }

    // Her türün kendi kuralı: türetilmiş sınıflar yazmak zorunda
    virtual string tur() const = 0;
    virtual long long cekilebilir() const = 0;
    virtual void aySonu() = 0;
    // Varsayılanı olan sanal fonksiyon: komisyon yoksa 0
    virtual long long komisyon(long long /*tutar*/) const { return 0; }

    bool yatir(long long tutar, const string& aciklama) {
        if (tutar <= 0) {
            return false;
        }
        degistir(tutar, aciklama);
        return true;
    }

    // Yetersiz bakiyede istisna yerine false döner (istisna: Bölüm 14)
    bool cek(long long tutar, const string& aciklama) {
        if (tutar <= 0) {
            return false;
        }
        long long kesinti = komisyon(tutar);
        if (tutar + kesinti > cekilebilir()) {
            return false;
        }
        degistir(-tutar, aciklama);
        if (kesinti > 0) {
            degistir(-kesinti, "Çekim komisyonu");
        }
        return true;
    }
```

```
    void dokum() const {
        cout << "Döküm: " << sahip << " (" << no << ")\n";
        for (const string& satir : gecmis) {
            cout << "  " << satir << '\n';
        }
    }
};

// Üye olmayan operatör: hangi türden olursa olsun her hesabı aynı biçimde yazar
ostream& operator<<(ostream& os, const Hesap& h) {
    os << left << setw(6) << h.noAl() << setw(9) << h.tur()
       << setw(12) << h.sahipAl() << right << setw(12)
       << paraYaz(h.bakiyeAl()) << " TL";
    return os;
}

const long long EK_HESAP_LIMITI = 1000 * TL;   // eksiye en çok 1000 TL
const int EK_HESAP_FAIZ_YUZDE = 5;               // aylık, borç üzerinden

class VadesizHesap : public Hesap {
public:
    VadesizHesap(int no, const string& sahip) : Hesap(no, sahip) {}

    string tur() const override { return "Vadesiz"; }

    long long cekilebilir() const override {
        long long limit = bakiyeAl() + EK_HESAP_LIMITI;
        return limit > 0 ? limit : 0;
    }

    void aySonu() override {
        if (bakiyeAl() < 0) {
            long long faiz = -bakiyeAl() * EK_HESAP_FAIZ_YUZDE / 100;
            if (faiz > 0) {
                degistir(-faiz, "Ek hesap faizi");
            }
        }
    }
};

int main() {
    VadesizHesap h(1001, "Ayse Demir");
    h.yatir(2000 * TL, "Para yatırma");
    cout << boolalpha;
    cout << "3500 TL çek: " << h.cek(3500 * TL, "Para çekme") << '\n';
    cout << "2500 TL çek: " << h.cek(2500 * TL, "Para çekme") << '\n';
    cout << "600 TL çek : " << h.cek(600 * TL, "Para çekme") << '\n';
    h.aySonu();
    cout << h << '\n';
    h.dokum();
    return 0;
}
```

Çıktı

```
3500 TL çek: false
2500 TL çek: true
600 TL çek : false
1001  Vadesiz  Ayse Demir       -525.00 TL
Döküm: Ayse Demir (1001)
  Hesap açıldı
  Para yatırma: +2000.00 -> bakiye 2000.00
  Para çekme: -2500.00 -> bakiye -500.00
  Ek hesap faizi: -25.00 -> bakiye -525.00
```

Satır satır doğrula: bakiye 2000 TL iken çekilebilir tutar 3000 TL; 3500 TL isteği **reddedildi** (`false`). 2500 TL çekilebilir; bakiye `-500` olur. Şimdi çekilebilir tutar 500 TL kaldığı için 600 TL yine reddedildi. Ay sonunda borç 500 TL'nin %5'i, yani **25 TL** kesildi ve bakiye `-525.00` oldu. Hesap yalnız `degistir` ile değiştiği için geçmişte her adım var.

Sık Yapılan Hata: Bakiyeyi dışarıdan değiştirmek

`bakiye` `private` olduğu için şu kod derlenmez; bu tam istediğimiz davranış:

```
#include <iostream>

using namespace std;

class Hesap {
private:
    long long bakiye{0};
public:
    long long bakiyeAl() const { return bakiye; }
};

int main() {
    Hesap h;
    h.bakiye = 5000000;
    cout << h.bakiyeAl() << '\n';
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
bakiyeGizli.cpp: In function 'int main()':
bakiyeGizli.cpp:14:7: error: 'long long int Hesap::bakiye' is private within this context
   14 |     h.bakiye = 5000000;
      |       ^~~~~
bakiyeGizli.cpp:7:15: note: declared private here
    7 |     long long bakiye{0};
      |               ^~~~~
bakiyeGizli.cpp:14:7: note: field 'long long int Hesap::bakiye' can be accessed via 'long long
int Hesap::bakiyeAl() const'
   14 |     h.bakiye = 5000000;
      |       ^~~~~
      |       bakiyeAl()
```

Mesajdaki `private within this context` (yani "bu bağlamda `private`") ifadesini tanı. macOS'taki `g++` komutu aslında Apple clang'dir ve cümlesi farklıdır; kitaptaki çıktılar gerçek g++'dan alınmıştır.

Adım 2: `VadeliHesap` ve çok biçimli koleksiyon

İkinci türü ekliyoruz. `VadeliHesap` eksiye düşemez (`cekilebilir` yalnız bakiyeyi verir), erken çekimde `komisyon`'u **ezer** (override) ve ay sonunda `bakiye * yillikYuzde / 1200` kadar faiz kazanır (yıllık yüzde, 12 aya ve 100'e bölünür: `/ 1200`).

Asıl yenilik `main`'de: iki farklı türden hesabı **tek bir** `vector<unique_ptr<Hesap>>` içinde tutuyoruz (Bölüm 12). `make_unique<VadesizHesap>(...)` bir `unique_ptr<VadesizHesap>` üretir, `push_back` bunu `unique_ptr<Hesap>`'a dönüştürür. Aşağıda yalnız yeni kısımlar görünüyor; Adım 1'deki kod aynen duruyor. `unique_ptr` ve `make_unique` için dosyanın başına `#include <memory>` eklemeyi unutma (Bölüm 10).

```cpp
#include <memory>

// ...

class VadeliHesap : public Hesap {
private:
    int yillikYuzde;                    // 36 = yıllık %36

public:
    VadeliHesap(int no, const string& sahip, int yuzde)
        : Hesap(no, sahip), yillikYuzde{yuzde} {}

    string tur() const override { return "Vadeli"; }

    long long cekilebilir() const override { return bakiyeAl(); }   // eksiye düşmez

    // Erken çekim: çekilen tutarın %1'i
    long long komisyon(long long tutar) const override { return tutar / 100; }

    void aySonu() override {
        long long faiz = bakiyeAl() * yillikYuzde / 1200;   // yıllık / 12 / 100
        if (faiz > 0) {
            degistir(faiz, "Aylık faiz");
        }
    }
};

// ...

int main() {
    vector<unique_ptr<Hesap>> hesaplar;
    hesaplar.push_back(make_unique<VadesizHesap>(1001, "Ayse Demir"));
    hesaplar.push_back(make_unique<VadeliHesap>(1002, "Mehmet Kaya", 36));

    hesaplar[0]->yatir(300 * TL, "Para yatırma");
    hesaplar[1]->yatir(8500 * TL, "Para yatırma");

    for (const auto& h : hesaplar) {
        cout << *h << "  limit " << paraYaz(h->cekilebilir())
             << ", 1000 TL'nin komisyonu "
             << paraYaz(h->komisyon(1000 * TL)) << '\n';
    }

    hesaplar[0]->cek(800 * TL, "Para çekme");     // eksiye düşer
    hesaplar[1]->cek(1000 * TL, "Para çekme");    // komisyon kesilir
    for (const auto& h : hesaplar) {
        h->aySonu();                              // aynı çağrı, farklı kural
    }

    cout << "--- ay sonundan sonra ---\n";
    for (const auto& h : hesaplar) {
        cout << *h << '\n';
    }
    hesaplar[1]->dokum();
    return 0;
}
```

Çıktı

```
1001  Vadesiz  Ayse Demir        300.00 TL  limit 1300.00, 1000 TL'nin komisyonu 0.00
1002  Vadeli   Mehmet Kaya      8500.00 TL  limit 8500.00, 1000 TL'nin komisyonu 10.00
--- ay sonundan sonra ---
1001  Vadesiz  Ayse Demir       -525.00 TL
1002  Vadeli   Mehmet Kaya      7714.70 TL
Döküm: Mehmet Kaya (1002)
  Hesap açıldı
  Para yatırma: +8500.00 -> bakiye 8500.00
  Para çekme: -1000.00 -> bakiye 7500.00
  Çekim komisyonu: -10.00 -> bakiye 7490.00
  Aylık faiz: +224.70 -> bakiye 7714.70
```

`for` döngülerindeki `h->aySonu()` çağrısı **her hesapta aynı yazılıyor**, ama vadesiz hesap borcuna faiz **keserken** vadeli hesap faiz **ekliyor**. Çok biçimliliğin bu projedeki işi budur: yarın üçüncü bir hesap türü eklesen bu döngüye dokunman gerekmez.

Rakamları kontrol et. Vadesiz: 300 TL yatırıldı, 800 TL çekilince bakiye `-500` oldu (çekilebilir sınır 1300 TL'ydi). Ay sonu faizi 500 × 5 / 100 = 25 TL, bakiye `-525.00`. Vadeli: 8500 TL yatırıldı, 1000 TL çekildi; komisyon 1000 TL'nin %1'i, yani 10 TL; bakiye 7490 TL. Ay sonu faizi 7490 × 36 / 1200 = 224.70 TL, bakiye `7714.70`. Geçmişte komisyon **ayrı bir satır** olarak görünüyor.

Sık Yapılan Hata: Döngüde unique_ptr'yi kopyalamak

`for (auto h : hesaplar)` yazarsan her turda `unique_ptr` **kopyalanmaya** çalışılır; tek sahipli işaretçinin kopyası olmaz ve program derlenmez (Bölüm 10). Çare: `const auto&` ile referans almak, projedeki her döngüde olduğu gibi.

```cpp
#include <iostream>
#include <memory>
#include <vector>

using namespace std;

int main() {
    vector<unique_ptr<int>> v;
    v.push_back(make_unique<int>(5));
    for (auto p : v) {
        cout << *p << '\n';
    }
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
kopyaTuzak.cpp: In function 'int main()':
kopyaTuzak.cpp:10:19: error: use of deleted function 'std::unique_ptr<_Tp,
_Dp>::unique_ptr(const std::unique_ptr<_Tp, _Dp>&) [with _Tp = int; _Dp =
std::default_delete<int>]'
   10 |     for (auto p : v) {
      |                   ^
In file included from /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/memory:80,
                 from kopyaTuzak.cpp:2:
/opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/bits/unique_ptr.h:543:7: note: declared here
  543 |       unique_ptr(const unique_ptr&) = delete;
      |       ^~~~~~~~~~
kopyaTuzak.cpp:10:19: note: use '-fdiagnostics-all-candidates' to display considered candidates
   10 |     for (auto p : v) {
      |                   ^
```

Mesajın özü "silinmiş kopya yapıcıyı kullanmaya çalışıyorsun" demektir. Uzun mesajın ilk `error` satırını oku, gerisini atla.

Sınavda Çıkar: Hangi komisyon çalışır?

`VadeliHesap v(1002, "Mehmet", 36); Hesap& r = v;` olsun. `r.komisyon(1000 * TL)` kaç kuruş verir? `komisyon` `virtual` olduğu için çağrı **nesnenin gerçek türüne** (dinamik tip) göre bağlanır: `VadeliHesap::komisyon`, yani `1000 * TL / 100 = 1000` kuruş (10 TL). `virtual` yazılmasaydı çağrı referansın **statik tipine** bağlanır ve `Hesap::komisyon` çalışıp `0` verirdi. Bu yüzden temel sınıftaki sanal fonksiyon bildirimi ve türetilmiş sınıftaki `override` ikisi birlikte aranır.

Adım 3: `Banka` sınıfı ve transfer

Hesapları bir arada tutan, numara dağıtan ve transferi yöneten sınıf. İşte yeni parçalar:

- **`enum class TransferSonucu`** (Bölüm 11): transferin dört olası sonucunu adlandırır. `bool` iki sonuç verir; burada dört neden var (başarılı, hesap yok, aynı hesap, yetersiz bakiye) ve çağıran hangisi olduğunu bilmeli. `sonucMetni` bunu Bölüm 4'teki `switch` ile metne çevirir. `enum class` değerleri `TransferSonucu::Tamam` biçiminde yazılır; her `case` yazıldığı için `default` gerekmez, ama g++ fonksiyon sonunda `return` göremezse uyarı verir, o yüzden sona boş bir `return` ekledik.
- **`hesapAc`**: türe göre `make_unique` ile nesneyi yaratır, vektöre ekler ve `hesaplar.back().get()` ile **sahiplik vermeyen** bir `Hesap*` döndürür (Bölüm 10'daki `get()`). `sonNo` yalnız hesap oluşunca artar. Geçersiz türde `nullptr` döner.
- **`bul`**: hesapları tek tek gezer (`const auto&`, Bölüm 12), bulursa `Hesap*`, bulamazsa `nullptr` (Bölüm 9) döndürür. Çağıranın **kontrol etmesi** şart: `nullptr`'ı dolaylamak programı çökertir.
- **`transfer`**: önce her iki hesabı bulur (`nullptr` kontrolü), sonra `kaynak->cek` çağırır. Para yetmezse `yatir` **hiç çalışmaz**: iki hesap da değişmeden kalır.
- **`aySonu`** tek satır: her hesabın kendi `aySonu`'nu çağırır (çok biçimlilik).
- **`listele`** `const` bir üye fonksiyondur (Bölüm 11): hesapları yazar ve toplam parayı verir. `cout << *h` ile `unique_ptr`'nin gösterdiği nesneyi dolaylayıp Bölüm 13'teki `operator<<`'a veririz.

```
enum class TransferSonucu { Tamam, HesapYok, AyniHesap, YetersizBakiye };

string sonucMetni(TransferSonucu s) {
    switch (s) {
        case TransferSonucu::Tamam:
            return "Transfer tamamlandı.";
        case TransferSonucu::HesapYok:
            return "Hesap bulunamadı.";
        case TransferSonucu::AyniHesap:
            return "Aynı hesaba transfer yapılamaz.";
        case TransferSonucu::YetersizBakiye:
            return "Yetersiz bakiye, hiçbir hesap değişmedi.";
    }
    return "";
}

class Banka {
private:
    vector<unique_ptr<Hesap>> hesaplar;     // hesapların tek sahibi Banka
    int sonNo{1000};

public:
    // 1 = vadesiz, 2 = vadeli; geçersiz türde nullptr döner
    Hesap* hesapAc(int tur, const string& sahip, int yuzde) {
        int no = sonNo + 1;
```

```
        if (tur == 1) {
            hesaplar.push_back(make_unique<VadesizHesap>(no, sahip));
        } else if (tur == 2) {
            hesaplar.push_back(make_unique<VadeliHesap>(no, sahip, yuzde));
        } else {
            return nullptr;
        }
        sonNo = no;
        return hesaplar.back().get();       // sahiplik vermeyen ham adres
    }

    Hesap* bul(int no) {
        for (const auto& h : hesaplar) {
            if (h->noAl() == no) {
                return h.get();
            }
        }
        return nullptr;
    }

    TransferSonucu transfer(int kaynakNo, int hedefNo, long long tutar) {
        Hesap* kaynak = bul(kaynakNo);
        Hesap* hedef = bul(hedefNo);
        if (kaynak == nullptr || hedef == nullptr) {
            return TransferSonucu::HesapYok;
        }
        if (kaynak == hedef) {
            return TransferSonucu::AyniHesap;
        }
        if (!kaynak->cek(tutar, "Giden transfer")) {   // para yetmezse burada biter
            return TransferSonucu::YetersizBakiye;
        }
        hedef->yatir(tutar, "Gelen transfer");
        return TransferSonucu::Tamam;
    }

    void aySonu() {
        for (const auto& h : hesaplar) {
            h->aySonu();                    // her hesap kendi kuralını işletir
        }
    }

    void listele() const {
        long long toplam = 0;
        for (const auto& h : hesaplar) {
            cout << *h << '\n';
            toplam += h->bakiyeAl();
        }
        cout << "Toplam: " << paraYaz(toplam) << " TL, " << hesaplar.size()
             << " hesap\n";
    }
};

// ...

int main() {
    Banka banka;
    banka.hesapAc(1, "Ayse Demir", 0);
    banka.hesapAc(2, "Mehmet Kaya", 36);
    banka.bul(1001)->yatir(2000 * TL, "Para yatırma");
    banka.bul(1002)->yatir(10000 * TL, "Para yatırma");

    cout << sonucMetni(banka.transfer(1002, 1001, 1500 * TL)) << '\n';
    cout << sonucMetni(banka.transfer(1001, 1002, 9000 * TL)) << '\n';
    cout << sonucMetni(banka.transfer(1001, 1001, 100 * TL)) << '\n';
```

```
    cout << sonucMetni(banka.transfer(1001, 1999, 100 * TL)) << '\n';
    cout << "bul(1005) nullptr mı? " << (banka.bul(1005) == nullptr) << '\n';

    banka.aySonu();
    banka.listele();
    return 0;
}
```

Çıktı

```
Transfer tamamlandı.
Yetersiz bakiye, hiçbir hesap değişmedi.
Aynı hesaba transfer yapılamaz.
Hesap bulunamadı.
bul(1005) nullptr mı? 1
1001  Vadesiz  Ayse Demir       3500.00 TL
1002  Vadeli   Mehmet Kaya      8739.55 TL
Toplam: 12239.55 TL, 2 hesap
```

Kontrol: ilk transferde Mehmet (vadeli) 1500 TL gönderdi ve **1500'ün %1'i olan 15 TL komisyon** da ödedi: 10000 − 1500 − 15 = 8485 TL. Ayşe 2000 + 1500 = 3500 TL oldu. İkinci transfer (Ayşe'den 9000 TL) reddedildi, çünkü Ayşe'nin çekilebilir sınırı 4500 TL; **hiçbir bakiye değişmedi**. Aynı hesaba transfer ve olmayan hesap da ayrı mesajlar verdi. Ay sonunda Mehmet'e 8485 × 36 / 1200 = 254.55 TL faiz eklendi (8739.55); Ayşe'nin bakiyesi pozitif olduğu için kesinti yok. Toplam 12239.55 TL: 12000 TL değil, çünkü 15 TL komisyon bankaya gitti ve 254.55 TL faiz eklendi.

> **Sınavda Çıkar: Transferde sıra neden önemli?**
>
> `transfer` içinde önce `yatir`, sonra `cek` çağrılsaydı ve `cek` başarısız olsaydı, alıcıya **yoktan para** verilmiş olurdu. Doğru sıra: önce kaynak hesaptan `cek`, başarılıysa hedefe `yatir`. Sınavda "hata olursa hiçbir şey değişmemeli" cümlesi geçiyorsa önce **başarısız olabilecek** işlemi yap.

> **Sık Yapılan Hata: Hesap* ve nullptr kontrolü**
>
> `banka.bul(1005)->yatir(...)` yazarsan, 1005 numaralı hesap yokken `nullptr` dolaylanır ve program çöker (Bölüm 9). `bul`'un döndürdüğü işaretçiyi **her zaman** `nullptr` ile karşılaştır; menü fonksiyonlarındaki `hesapSec` bu yüzden var. Adım 3'teki deneme `main`'inde, hesapların var olduğunu bildiğimiz için doğrudan `->` kullandık; gerçek programda bu kısayol yok.

Adım 4: Güvenli girdi ve menü

Son adımda kullanıcıyla konuşuyoruz. Girdi okuma fonksiyonları Bölüm 8'deki not projesinden **aynen** geliyor (`satirOku`, `sayiOku`, `metinOku`): her şey `getline` ile satır olarak okunur, sayı `isdigit` ile denetlenip `stoi` ile çevrilir. Tek fark: tutarlar daha büyük olabildiği için karakter sınırı 4'ten **6'ya** çıktı. `exit`'in kullanımı da aynı: girdi biterse program kapanır. Bu yüzden dosyanın başına not projesindeki gibi `#include <cctype>` (`isdigit` için) ve `#include <cstdlib>` (`exit` için) eklenir.

Menü kısmında yeni bir şey yok, ama dikkat edeceğin üç nokta var:

- **Her ekran bir fonksiyon.** `hesapAcEkrani`, `yatirEkrani`, `cekEkrani`, `transferEkrani`, `dokumEkrani` ayrı fonksiyonlar; `main` yalnız `switch` ile bunları çağırır. `Banka`, referansla (`Banka&`) geçirilir (Bölüm 6, 9): kopyalanırsa `main`'deki banka değişmezdi. Zaten `vector<unique_ptr<...>>` kopyalanamaz, yani yanlışlıkla değer ile geçirirsen derleyici seni durdurur.
- **`hesapSec` yardımcısı.** Numarayı sorar, `bul` ile arar, bulamazsa mesaj yazar ve `nullptr` döndürür; çağıran ekranlar `nullptr` ise hemen `return` eder.

- **`transferEkrani` numaraları doğrudan sorar.** Çünkü olmayan hesabı `Banka::transfer` kendisi `HesapYok` ile bildiriyor; sonuç mesajını `sonucMetni` yazıyor.

`cekEkrani` reddedilen çekimde sebebi gösterir: **istenen + komisyon** ile **limit**. Vadeli hesapta 8450 TL isteyen biri komisyonla 8534.50 TL'ye ihtiyaç duyar, bunu bilmezse "bakiyem 8485, neden olmadı?" der.

```
#include <cctype>
#include <cstdlib>

// ...

// Mesajı yazar, bir satır okur; girdi tümüyle biterse programı kapatır
string satirOku(const string& mesaj) {
    string satir;
    cout << mesaj;
    if (!getline(cin, satir)) {
        cout << "\nGirdi bitti, program kapatılıyor.\n";
        exit(1);
    }
    return satir;
}

// alt-ust aralığında geçerli bir tam sayı girilene kadar tekrar sorar
int sayiOku(const string& mesaj, int alt, int ust) {
    while (true) {
        string satir = satirOku(mesaj);
        bool gecerli = !satir.empty() && satir.size() <= 6;
        for (char c : satir) {
            if (!isdigit(static_cast<unsigned char>(c))) {
                gecerli = false;
            }
        }
        if (gecerli) {                      // artık stoi güvenle çalışır
            int deger = stoi(satir);
            if (deger >= alt && deger <= ust) {
                return deger;
            }
        }
        cout << "  Geçersiz! " << alt << "–" << ust
             << " arası bir tam sayı gir.\n";
    }
}

// Boş bırakılamayan bir satır okur (ad soyad için)
string metinOku(const string& mesaj) {
    string metin;
    do {
        metin = satirOku(mesaj);
    } while (metin.empty());
    return metin;
}

// ...

void menuYazdir() {
    cout << "======== BANKA ========\n"
         << "1) Hesap aç  2) Yatır  3) Çek  4) Transfer\n"
         << "5) Döküm  6) Ay sonu  7) Listele  0) Çıkış\n";
}

// TL olarak sorar, kuruşa çevirip döndürür
long long tutarOku() {
    return sayiOku("Tutar (TL): ", 1, 100000) * TL;
}
```

```
// Hesap numarasını sorar; bulamazsa uyarır ve nullptr döndürür
Hesap* hesapSec(Banka& banka) {
    int no = sayiOku("Hesap no: ", 1, 999999);
    Hesap* h = banka.bul(no);
    if (h == nullptr) {
        cout << "  Hesap bulunamadı: " << no << '\n';
    }
    return h;
}

void hesapAcEkrani(Banka& banka) {
    int tur = sayiOku("Tür (1=Vadesiz, 2=Vadeli): ", 1, 2);
    string ad = metinOku("Ad soyad: ");
    int yuzde = 0;
    if (tur == 2) {
        yuzde = sayiOku("Yıllık faiz (%): ", 1, 100);
    }
    Hesap* h = banka.hesapAc(tur, ad, yuzde);
    if (h != nullptr) {
        cout << "  Açıldı: " << *h << '\n';
    }
}

void yatirEkrani(Banka& banka) {
    Hesap* h = hesapSec(banka);
    if (h == nullptr) {
        return;
    }
    h->yatir(tutarOku(), "Para yatırma");
    cout << "  Bakiye: " << paraYaz(h->bakiyeAl()) << " TL\n";
}

void cekEkrani(Banka& banka) {
    Hesap* h = hesapSec(banka);
    if (h == nullptr) {
        return;
    }
    long long tutar = tutarOku();
    if (h->cek(tutar, "Para çekme")) {
        cout << "  Bakiye: " << paraYaz(h->bakiyeAl()) << " TL\n";
    } else {
        cout << "  Yetersiz bakiye! İstenen (komisyon dahil) "
             << paraYaz(tutar + h->komisyon(tutar)) << " TL, limit "
             << paraYaz(h->cekilebilir()) << " TL.\n";
    }
}

void transferEkrani(Banka& banka) {
    int kaynak = sayiOku("Gönderen hesap: ", 1, 999999);
    int hedef = sayiOku("Alıcı hesap: ", 1, 999999);
    long long tutar = tutarOku();
    cout << "  " << sonucMetni(banka.transfer(kaynak, hedef, tutar)) << '\n';
}

void dokumEkrani(Banka& banka) {
    Hesap* h = hesapSec(banka);
    if (h != nullptr) {
        h->dokum();
    }
}

// ...

int main() {
```

```
    Banka banka;
    menuYazdir();
    bool devam = true;
    while (devam) {
        int secim = sayiOku("\nSeçim: ", 0, 7);
        switch (secim) {
            case 1: hesapAcEkrani(banka); break;
            case 2: yatirEkrani(banka); break;
            case 3: cekEkrani(banka); break;
            case 4: transferEkrani(banka); break;
            case 5: dokumEkrani(banka); break;
            case 6:
                banka.aySonu();
                cout << "  Ay sonu işlemleri yapıldı.\n";
                break;
            case 7: banka.listele(); break;
            case 0: devam = false; break;
        }
    }
    cout << "İyi günler!\n";
    return 0;
}
```

Menü yalnızca bir kez yazılıyor (kullanıcı her turda yeniden görmek isterse `menuYazdir()` çağrısını döngünün içine al). `switch`'te `default` yok, çünkü `sayiOku(…, 0, 7)` zaten 0–7 dışındaki her şeyi tekrar sordurur. Böylece `case` listesi, geçerli menü seçeneklerinin **tam listesi** olur.

Örnek çalışma

Aşağıdaki oturumda iki hesap açılıp para yatırılıyor. Sonra sırayla şunlar deneniyor: limiti aşan çekim (reddedilir), tutar yerine harf ve eksi sayı (tekrar sorar), olmayan hesap, transfer, yetersiz bakiyeli transfer, aynı hesaba transfer, komisyon yüzünden reddedilen vadeli çekim, ek hesaba düşen bir çekim, ay sonu, döküm, listeleme, geçersiz menü seçimi ve çıkış.

Çıktı

```
======== BANKA ========
1) Hesap aç  2) Yatır  3) Çek  4) Transfer
5) Döküm  6) Ay sonu  7) Listele  0) Çıkış

Seçim: 1
Tür (1=Vadesiz, 2=Vadeli): 1
Ad soyad: Ayse Demir
  Açıldı: 1001  Vadesiz  Ayse Demir          0.00 TL

Seçim: 1
Tür (1=Vadesiz, 2=Vadeli): 2
Ad soyad: Mehmet Kaya
Yıllık faiz (%): 36
  Açıldı: 1002  Vadeli   Mehmet Kaya         0.00 TL

Seçim: 2
Hesap no: 1001
Tutar (TL): 2000
  Bakiye: 2000.00 TL

Seçim: 2
Hesap no: 1002
Tutar (TL): 10000
  Bakiye: 10000.00 TL

Seçim: 3
```

```
Hesap no: 1001
Tutar (TL): 3500
  Yetersiz bakiye! İstenen (komisyon dahil) 3500.00 TL, limit 3000.00 TL.

Seçim: 3
Hesap no: 1001
Tutar (TL): abc
  Geçersiz! 1-100000 arası bir tam sayı gir.
Tutar (TL): -50
  Geçersiz! 1-100000 arası bir tam sayı gir.
Tutar (TL): 2500
  Bakiye: -500.00 TL

Seçim: 2
Hesap no: 1005
  Hesap bulunamadı: 1005

Seçim: 4
Gönderen hesap: 1002
Alıcı hesap: 1001
Tutar (TL): 1500
  Transfer tamamlandı.

Seçim: 4
Gönderen hesap: 1001
Alıcı hesap: 1002
Tutar (TL): 5000
  Yetersiz bakiye, hiçbir hesap değişmedi.

Seçim: 4
Gönderen hesap: 1001
Alıcı hesap: 1001
Tutar (TL): 100
  Aynı hesaba transfer yapılamaz.

Seçim: 3
Hesap no: 1002
Tutar (TL): 8450
  Yetersiz bakiye! İstenen (komisyon dahil) 8534.50 TL, limit 8485.00 TL.

Seçim: 3
Hesap no: 1001
Tutar (TL): 1800
  Bakiye: -800.00 TL

Seçim: 6
  Ay sonu işlemleri yapıldı.

Seçim: 5
Hesap no: 1001
Döküm: Ayse Demir (1001)
  Hesap açıldı
  Para yatırma: +2000.00 -> bakiye 2000.00
  Para çekme: -2500.00 -> bakiye -500.00
  Gelen transfer: +1500.00 -> bakiye 1000.00
  Para çekme: -1800.00 -> bakiye -800.00
  Ek hesap faizi: -40.00 -> bakiye -840.00

Seçim: 7
1001  Vadesiz  Ayse Demir       -840.00 TL
1002  Vadeli   Mehmet Kaya      8739.55 TL
Toplam: 7899.55 TL, 2 hesap

Seçim: 9
```

```
  Geçersiz! 0-7 arası bir tam sayı gir.

Seçim: 0
İyi günler!
```

Sonucu elle kontrol edelim:

- Ayşe'nin vadesiz hesabında 2000 TL varken çekilebilir sınır 3000 TL'dir; 3500 TL isteği reddedildi. `abc` ve `-50` tutar sorusunu tekrarlattı; `2500` kabul edildi (bakiye -500).
- Transferden sonra Mehmet'te 10000 − 1500 − 15 (komisyon) = 8485, Ayşe'de −500 + 1500 = 1000 TL kaldı. Ayşe'nin 5000 TL'lik transferi 2000 TL'lik sınırı aştığı için reddedildi.
- Mehmet'in 8450 TL çekimi, komisyonla 8534.50 TL gerektirdiğinden (bakiye 8485) reddedildi.
- Ayşe 1800 TL çekince bakiyesi −800 oldu. Ay sonunda ondan 800 × 5 / 100 = 40 TL kesildi (−840); Mehmet'e 8485 × 36 / 1200 = 254.55 TL eklendi (8739.55).
- Döküm Ayşe'nin bütün hareketlerini bakiyeyle birlikte gösteriyor. Toplam: 8739.55 − 840 = 7899.55 TL.
- `9` menü seçimi `switch`'e ulaşmadan `sayiOku` tarafından reddedildi; hatalı işlemlerin hiçbiri bakiyeleri değiştirmedi.

Sınavda Çıkar: Hesabın türü ne zaman bellidir?

`Hesap* h = banka.bul(1002);` satırında `h`'nin **statik tipi** `Hesap*`, gösterdiği nesnenin **dinamik tipi** `VadeliHesap`'tır. `h->aySonu()` hangisine göre bağlanır? Dinamik tipe; çünkü `aySonu` `virtual`. Aynı çağrı vadesiz hesapta başka kod çalıştırır. Sınavda bu cümleyi "çağrılan fonksiyonu çalışma zamanında nesnenin gerçek türü belirler" diye kur.

Not: Türkçe harfler ve hizalama

`setw` genişliği **bayt** sayar (Bölüm 8 projesindeki not). Bu yüzden örnekteki adlar ASCII harflerle yazıldı; `Ayşe` gibi bir ad tabloda bir sütun kaydırırdı. Çözüm yolu Proje 1'in (not sistemi) "Kendin Geliştir" kutusunda.

Bölüm Özeti – Bilmen Gerekenler

- `Hesap` soyut bir temel sınıftır; ortak davranış (`yatir`, `cek`, `dokum`) orada, farklı kurallar (`cekilebilir`, `aySonu`, `komisyon`) türetilmiş sınıflarda `override` ile yazılır.
- `private` veri yalnız üye fonksiyonlarla değişir; türetilmiş sınıflar `protected` yardımcıyı (`degistir`) kullanır.
- Hesaplar `vector<unique_ptr<Hesap>>` içinde tutulur; `Banka` sahibidir, dışarıya yalnız sahip olmayan `Hesap*` verilir ve `bul` bulamazsa `nullptr` döndürür.
- Para kuruş olarak `long long` tutulur; tamsayı bölmesi kuruşun altını atar.
- İstisna yok: başarısızlık `bool` ya da `enum class` dönüşüyle bildirilir (istisnalar Bölüm 14'te).

Kendin Geliştir

- **Yeni hesap türü:** Günlük en fazla 2000 TL çekim izni olan, faiz almayan bir `OgrenciHesabi` ekle. Yalnız yeni bir alt sınıf ve `Banka::hesapAc`'ta bir kol yeter: `Banka`'nın geri kalanına dokunman gerekmez.
- **Kuruşlu tutar:** `tutarOku` şimdi tam TL istiyor. `150.50` gibi girdiyi okuyup kuruşa çevir (`find`, `substr`, Bölüm 8; kuruş kısmını ayrı doğrula).
- **Menüyü tekrar göster:** Çıktı kalabalık olmasın diye menü bir kez yazılıyor. `8` seçeneğiyle "menüyü göster" ekle (`sayiOku` üst sınırını unutma).
- **İşlem geçmişine sınır:** `gecmis` sınırsız büyüyor. Son 20 işlemi tutacak şekilde en eskiyi sil (`gecmis.erase(gecmis.begin())` ilk elemanı siler; ayrıntısı Bölüm 15'te).
- **Kopya yapıcıyı kapat:** `Hesap` kopyalanırsa iki hesap aynı numarayı taşırdı. Bölüm 13'teki `= delete` ile kopya yapıcıyı yasakla ve ne olduğunu gör.
- **Hesap kapatma:** Bakiyesi sıfır olan hesabı silen bir seçenek yaz. `vector`'den `unique_ptr` silmek, sahip olunan nesneyi de yok eder: yıkıcı çalışır.
- **PIN:** `Hesap`'a PIN ekle; üç yanlış denemede hesabı kilitle.
- **İstisnalarla yeniden yaz (Bölüm 14):** `cek` ve `Banka::transfer` hata durumunda `bool`/`enum` yerine kendi istisna sınıfını (`YetersizBakiyeHatasi`) fırlatsın, menüdeki `try`/`catch` mesajı yazsın. `TransferSonucu` gereksiz kalacak: neden?
- **Dosyaya kaydet (Bölüm 16):** Çıkarken hesapları bir metin dosyasına yaz, açılışta oku.

BÖLÜM 14

İstisna Yönetimi

Bu Bölümde Öğreneceklerin

- Hata kodu döndürmenin neden yetersiz kaldığını ve istisnanın (exception) buna nasıl çözüm getirdiğini
- Hatayı fırlatmayı, yakalamayı ve ele almayı
- Standart kütüphanenin hazır istisna sınıflarını ve hata mesajını okumayı
- Birden çok yakalama bloğu yazarken sıranın neden önemli olduğunu
- Yakalanmayan bir istisnada programın nasıl sonlandığını
- Kendi istisna sınıfını yazmayı
- Hata fırlatıldığında yerel nesnelerin kendiliğinden temizlenmesini
- Banka projesindeki hesap sınıfını hatalara karşı korumayı

14.1 Hata kodu yaklaşımı ve sorunları

Şimdiye kadar yazdığın fonksiyonlar hata olunca ne yaptı? Büyük ihtimalle ya bir mesaj yazdırdın ya da "olmaz" anlamına gelen özel bir değer döndürdün. Bu yönteme **hata kodu** (error code) yaklaşımı denir. Bölme fonksiyonuyla deneyelim: sıfıra bölme olursa `-1` döndürsün.

hataKodu.cpp

```cpp
#include <iostream>

using namespace std;

int bol(int a, int b) {
    if (b == 0) {
        return -1;      // hata kodu
    }
    return a / b;
}

int main() {
    cout << "10 / 2 = " << bol(10, 2) << '\n';
    cout << "10 / 0 = " << bol(10, 0) << '\n';
    cout << "-10 / 10 = " << bol(-10, 10) << '\n';
    return 0;
}
```

Çıktı

```
10 / 2 = 5
10 / 0 = -1
-10 / 10 = -1
```

Son iki satır aynı sayıyı yazdı, ama biri hata, diğeri gerçek sonuç. Hata kodu yaklaşımının sorunları şunlar:

1. **Sonuçla karışır.** `-1` hem "hata oldu" hem de geçerli bir sonuç olabilir.

2. **Kontrol etmeyi unutmak kolaydır.** Çağıran dönüş değerine bakmazsa derleyici ses çıkarmaz; program yanlış değerle devam eder.
3. **Hata her basamaktan elle taşınır.** `A`, `B`'yi, `B` de `C`'yi çağırıyor ve hata `C`'de olduysa `B` hata kodunu `A`'ya iletmek zorundadır; asıl iş kodu `if` yığınında kaybolur.
4. **Yapıcı hata kodu döndüremez.** Bölüm 11'deki yapıcının dönüş tipi yoktur; nesne kurulurken bir şey ters giderse bildirmenin yolu kalmaz.

C++'ın çözümü **istisnadır** (exception). İstisna, hatayı dönüş değerinden ayırıp ayrı bir kanaldan gönderir: hata olduğunda fonksiyon normal yoldan **dönmez**, hata doğrudan hatayı ele alabilecek koda **fırlatılır**. Aradaki fonksiyonların hiçbirinin kontrol kodu yazması gerekmez.

14.2 throw, try ve catch

Üç anahtar sözcük vardır:

- `throw` hatayı fırlatır (bildirir).
- `try` içine, hata fırlatabilecek kodu yazarsın.
- `catch` hatayı yakalar ve ele alır.

ilkIstisna.cpp

```cpp
#include <iostream>
#include <stdexcept>

using namespace std;

int bol(int a, int b) {
    if (b == 0) {
        throw invalid_argument("bölen sıfır olamaz");
    }
    return a / b;
}

int main() {
    try {
        cout << "10 / 2 = " << bol(10, 2) << '\n';
        cout << "10 / 0 = " << bol(10, 0) << '\n';
        cout << "Bu satır hiç çalışmaz\n";
    } catch (const invalid_argument& hata) {
        cout << "Hata yakalandı: " << hata.what() << '\n';
    }
    cout << "Program devam ediyor\n";
    return 0;
}
```

Çıktı

```
10 / 2 = 5
10 / 0 = Hata yakalandı: bölen sıfır olamaz
Program devam ediyor
```

Adım adım ne oldu:

1. `throw invalid_argument("...")` ifadesi bir **istisna nesnesi** oluşturup fırlatır. `invalid_argument` standart kütüphanedeki hazır bir sınıftır; `<stdexcept>` başlığında bulunur ve yapıcısı bir mesaj metni alır.
2. `bol(10, 0)` içinde `throw` çalışınca fonksiyon `return` etmeden **terk edilir**. `return a / b;` hiç çalışmaz.

3. İstisna `main`'e geçer. `try` bloğunun **geri kalanı atlanır**: `"Bu satır hiç çalışmaz"` yazılmadı. Çıktıdaki yarım kalmış `10 / 0 =` kısmına da dikkat et: `<<` zinciri soldan sağa işlenir, o metin `bol(10, 0)` çağrılmadan **önce** `cout`'a yazıldı. Hata, satırın ortasında kesti.
4. Türü uyan `catch` bloğu çalışır. `hata.what()` istisna nesnesinin içindeki mesajı verir.
5. `catch` bitince program `try-catch` yapısının **altından** devam eder. İstisnanın fırlatıldığı yere geri dönülmez.

`catch (const invalid_argument& hata)` yazımını ezberle: istisnayı **`const` referans** ile yakalarsın; nedenini birazdan göreceksin (nesne dilimleme).

Kısa kurallar: `try` hatasız biterse `catch` bloklarına hiç uğranmaz. `catch`, `try` bloğunun hemen ardından gelir ve (`if`'in aksine) tek satır olsa bile **süslü parantez** ister. İstisna nesnesine ihtiyacın yoksa adı yazmayabilirsin: `catch (const invalid_argument&)`.

> **Sık Yapılan Hata: `try` içindeki değişken dışarıdan görünmez**
>
> `try { int deger{stoi(metin)}; } catch (...) { }` yazıp `deger`'i `try`'ın altında kullanmaya çalışırsan derleme hatası alırsın. Her blok gibi `try` bloğu da kendi kapsamını (scope) oluşturur. Değişkeni dışarıda kullanacaksan `try`'dan **önce** tanımla: `int deger{0}; try { deger = stoi(metin); }`.

Yakalanmayan istisna

Fırlatılan istisnayı hiçbir `catch` yakalamazsa istisna fonksiyonları tek tek **terk ederek** yukarı çıkar. `main`'i de geçerse C++ çalışma zamanı `std::terminate` fonksiyonunu çağırır ve program anında sonlanır. Metni sayıya çeviren `stoi` fonksiyonuna sayı olmayan bir metin verelim:

cokme.cpp

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    string metin{"doksan"};
    cout << "Çevriliyor...\n";
    int puan{stoi(metin)};
    cout << "Puan: " << puan << '\n';
    return 0;
}
```

Çıktı – program çöker (Aborted)

```
Çevriliyor...
terminate called after throwing an instance of 'std::invalid_argument'
  what():  stoi
```

Çıktıyı oku: `Çevriliyor...` yazıldı; `stoi` satırında `std::invalid_argument` fırlatıldı, yakalayan olmadığı için program `terminate called after throwing an instance of 'std::invalid_argument'` mesajıyla sonlandı. `what():` kısmı istisnanın mesajıdır. `Puan: ...` satırı hiç çalışmadı. Bu metin g++'ya özgüdür; Visual Studio'da aynı program bir hata penceresi açar.

> **Not: Apple clang'de mesaj farklıdır**
>
> macOS'ta `g++` komutu aslında Apple clang'dir. Aynı programda mesaj `libc++abi: terminating due to uncaught exception of type std::invalid_argument: stoi: no conversion` gibi görünür; davranış aynıdır. Kitaptaki çıktılar gerçek g++ ile üretildi.

14.3 std::exception hiyerarşisi

Standart kütüphanedeki istisna sınıflarının hepsi bir **kalıtım ağacında** toplanır (Bölüm 12). Ağacın kökü `std::exception` sınıfıdır ve hepsinde `what()` adlı bir **sanal fonksiyon** (virtual function) bulunur:

```
std::exception
├── std::logic_error          (programcının hatası, kod doğru olsa olmazdı)
│   ├── std::invalid_argument     geçersiz argüman
│   ├── std::out_of_range         geçerli aralığın dışına erişim
│   ├── std::length_error         izin verilen uzunluk aşıldı
│   └── std::domain_error         değer fonksiyonun tanım kümesi dışında
├── std::runtime_error        (çalışırken ortaya çıkan, önceden kestirilemeyen hata)
│   ├── std::range_error
│   ├── std::overflow_error
│   └── std::underflow_error
├── std::bad_alloc            new ile bellek ayrılamadı
└── std::bad_cast             geçersiz dynamic_cast (referansla)
```

`logic_error` ve `runtime_error` ailelerinin hepsi `<stdexcept>` başlığındadır. Kabaca ayrım şudur: **`logic_error`** "bu çağrıyı yapmamalıydın" der (negatif yaş, olmayan eleman numarası); **`runtime_error`** "kodun doğruydu ama dışarıdaki bir şey ters gitti" der (dosya yok, hesapta para yetmiyor). En çok işine yarayacaklar `invalid_argument`, `out_of_range`, `runtime_error` ve `overflow_error`'dır; son ikisini genellikle **sen** fırlatırsın.

what() ve çok biçimlilik

`what()` hatanın açıklamasını `const char*` olarak döndürür. Sınıfın yapıcısına verdiğin mesaj olduğu gibi gelir. Hazır istisnaları kütüphane fırlatırsa mesajı da kütüphane yazar ve **derleyiciye göre değişir**: aynı hata g++'da bir, MSVC'de başka bir metin verebilir. Bu yüzden mesajın metnine bakarak program akışını karara bağlama; yalnızca kullanıcıya ya da günlüğe (log) yazdır.

`what()` bir sanal fonksiyon olduğu için çok biçimlilik (polymorphism) işler: `void yazdir(const exception& e)` gibi bir fonksiyona `invalid_argument` da gönderebilirsin, `runtime_error` da; içindeki `e.what()` her seferinde **gerçek türün** mesajını verir.

Aynı nedenle `catch` **referansla** yazılır. Bölüm 12'de gördüğün **nesne dilimleme** (object slicing) burada da geçerlidir: istisnayı değerle, yani `catch (exception e)` ile yakalarsan türetilmiş sınıfa ait kısım kopyalanırken kesilir ve `what()` artık gerçek mesajı vermez. Kendin dene:

dilimleme.cpp

```cpp
#include <iostream>
#include <stdexcept>

using namespace std;

int main() {
    try {
        throw runtime_error("dosya bozuk");
    } catch (exception e) {
        cout << "mesaj: " << e.what() << '\n';
    }
    return 0;
}
```

Derleyici uyarısı – g++ -Wall -Wextra

```
dilimleme.cpp: In function 'int main()':
dilimleme.cpp:9:24: warning: catching polymorphic type 'class std::exception' by value [-Wcatch-
value=]
    9 |     } catch (exception e) {
      |                        ^
```

Çıktı

```
mesaj: std::exception
```

Mesaj `dosya bozuk` değil, `std::exception` yazdı: `runtime_error`'a ait kısım kopyalanırken kesildi. (Bu tam metin g++'a özgüdür; MSVC'de başka bir ifade çıkar, ama gerçek mesaj orada da kaybolur.) g++ bu yüzden `-Wall` ile seni uyarır; macOS'taki `g++` (Apple clang) ise bu uyarıyı vermez, yani orada hata sessizce geçer. Kalıp her zaman `catch (const TurAdi& ad)`.

> **Sınavda Çıkar: `catch` parametresi**
>
> "İstisna yakalamanın doğru yolu hangisi?" sorusunun cevabı hep **`const` referanstır**: `catch (const exception& e)`. Değerle yakalama kopya ve dilimleme demektir; işaretçiyle yakalama yalnızca `throw new ...` ile çalışır ve sızıntı riski taşır.

14.4 stoi ve at() istisnaları

Kütüphanenin istisna fırlatan iki tanıdık fonksiyonu var. `stoi` (string to int) metni `int`'e çevirir ve iki ayrı istisna fırlatabilir:

- **`invalid_argument`**: metnin başında hiç sayı yok (`"abc"`, `""`).
- **`out_of_range`**: sayı var ama `int`'e sığmıyor (`"99999999999"`).

Bir de beklenmedik bir davranışı vardır: metnin **başındaki** sayıyı okur, kalan kısmı sessizce bırakır. `"12abc"` hata vermez, `12` döner. `vector`'ün `at()` fonksiyonu (Bölüm 7) ise `vector` sınırı dışına çıkıldığında `out_of_range` fırlatır (`[]` ise denetim yapmaz, tanımsız davranıştır).

stoilstisna.cpp

```cpp
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

using namespace std;

void dene(const string& metin) {
    try {
        int sayi{stoi(metin)};
        cout << "\"" << metin << "\" -> " << sayi << '\n';
    } catch (const invalid_argument& e) {
        cout << "\"" << metin << "\" -> invalid_argument: " << e.what() << '\n';
    } catch (const out_of_range& e) {
        cout << "\"" << metin << "\" -> out_of_range: " << e.what() << '\n';
    }
}

int main() {
    for (string metin : {"42", "  -7", "12abc", "abc", "99999999999"}) {
        dene(metin);
    }
    vector<int> v{10, 20, 30};
```

```
    try {
        cout << v.at(1) << '\n';
        cout << v.at(5) << '\n';
    } catch (const out_of_range&) {
        cout << "at() hata verdi\n";
    }
    return 0;
}
```

Çıktı

```
"42" -> 42
"  -7" -> -7
"12abc" -> 12
"abc" -> invalid_argument: stoi
"99999999999" -> out_of_range: stoi
20
at() hata verdi
```

Okuma notları: baştaki boşluklar atlanır (`"  -7"` → `-7`). `"12abc"` için `12` döner; kullanıcı girdisini doğrularken bu yüzden `stoi`'nin ikinci parametresini kullanmak gerekir (aşağıda). `what()` mesajı g++'a göredir (`stoi`); başka derleyicide farklı bir metin çıkar, bu yüzden programın mantığını mesaja bağlama. `v.at(5)` satırında `at()` fırlattığı için o satırda hiçbir şey yazılmadı ve `catch` çalıştı.

`stoi`'nin ikinci parametresi `size_t*` tipindedir: kaç karakterin okunduğunu o değişkene yazar. Metnin tamamının sayı olup olmadığını böyle anlarsın (Soru 9'da kullanacaksın):

```
size_t okunan{0};
int sayi{stoi(metin, &okunan)};
if (okunan != metin.size()) {
    throw invalid_argument("metnin tamamı sayı değil: " + metin);
}
```

`"12abc"` için `stoi` hata vermez ama `okunan` 2 olur, `metin.size()` ise 5; böylece **kendi** `invalid_argument`'imizi fırlatırız.

14.5 Birden çok catch ve sıra tuzağı

Bir `try` bloğundan farklı türde istisnalar çıkabilir; `stoiIstisna` programında `invalid_argument` ve `out_of_range` için iki ayrı `catch` yazmıştık. **Yukarıdan aşağıya bakılır ve türü uyan ilk `catch` çalışır**, diğerleri atlanır.

Bir uyarı: C++'ta tamsayı sıfıra bölme için hazır bir istisna **yoktur**. `100 / bolen` ifadesinde `bolen` sıfırsa bu istisna değil **tanımsız davranıştır**; bir bilgisayarda program çöker, bir başkasında saçma bir sonuç üretir. Bölmeden önce `bolen != 0` kontrolünü sen yapmalısın (ya da bulursa kendi `throw`'unla bildirmelisin).

Sıra tuzağı

Kalıtım ağacında **üst sınıf türündeki `catch`, alt sınıfları da yakalar** (çünkü `out_of_range` bir `logic_error`'dır). Bu yüzden sıra önemlidir: **önce özel (alt sınıf), sonra genel (üst sınıf)**. Genel olan önce yazılırsa özel olana sıra hiç gelmez:

catchSirasi.cpp

```cpp
#include <iostream>
#include <stdexcept>

using namespace std;

int main() {
    try {
        throw out_of_range("indeks 9, boyut 3");
    } catch (const logic_error&) {
        cout << "logic_error bloğu\n";
    } catch (const out_of_range&) {
        cout << "out_of_range bloğu\n";
    }
    return 0;
}
```

Derleyici uyarısı – g++ -Wall -Wextra

```
catchSirasi.cpp: In function 'int main()':
catchSirasi.cpp:11:7: warning: exception of type 'std::out_of_range' will be caught by earlier
handler [-Wexceptions]
   11 |     } catch (const out_of_range&) {
      |       ^~~~~
catchSirasi.cpp:9:7: note: for type 'std::logic_error'
    9 |     } catch (const logic_error&) {
      |       ^~~~~
```

Çıktı

```
logic_error bloğu
```

`out_of_range` fırlatıldı ama `logic_error` bloğu çalıştı; ikinci `catch` ölü koddur. Başka bazı dillerin aksine C++ bunu **hata saymaz**, g++ yalnızca uyarı verir ve program derlenir. Yani derleyici seni durdurmaz; sırayı kendin doğru kurmak zorundasın. Doğrusu iki bloğun yerini değiştirmektir.

> **Sınavda Çıkar: catch sırası**
>
> Çıktı tahmininde `catch` blokları sırasıyla verilirse şunu sor: **"Fırlatılan türün, ondan önce yazılmış hangi `catch` türüyle akrabalığı (alt sınıf) var?"** İlk akraba blok çalışır. Ağaç: `exception` → `logic_error` → `invalid_argument`, `out_of_range`. `invalid_argument` ile `out_of_range` **kardeştir**, biri diğerini yakalamaz. `catch (const exception&)` her standart istisnayı yakalar, o yüzden en sona yazılır. Sınavda sevilen tuzak: ilk `catch`'in genel olması.

14.6 catch (...) ve yeniden fırlatma

Üç nokta `...` her türlü istisnayı yakalar. Ama içeride istisna nesnesine ulaşamazsın; yani `what()` çağıramazsın. Bu yüzden `catch (...)` yalnızca "son çare" bloğudur ve her zaman **en sona** yazılır:

uc_nokta.cpp

```cpp
#include <iostream>
#include <stdexcept>

using namespace std;

int main() {
    for (int secim{0}; secim <= 2; secim++) {
        try {
```

```
            if (secim == 1) {
                throw runtime_error("çalışma zamanı hatası");
            }
            if (secim == 2) {
                throw 42;           // C++'ta herhangi bir tip fırlatılabilir
            }
            cout << secim << ": hata yok\n";
        } catch (const exception& e) {
            cout << secim << ": exception -> " << e.what() << '\n';
        } catch (...) {
            cout << secim << ": bilinmeyen tür\n";
        }
    }
    return 0;
}
```

Çıktı

```
0: hata yok
1: exception -> çalışma zamanı hatası
2: bilinmeyen tür
```

`throw 42;` geçerlidir: C++ her tipte değer fırlatmana izin verir, ama bu **kötü stildir**. `int` fırlatırsan kimse `what()` çağıramaz, hangi hata olduğu bilinmez. Her zaman `std::exception`'dan türeyen bir sınıf fırlat.

throw; ile yeniden fırlatma

Bazen hatayı bir yerde yakalayıp **bir şey yapmak** (örneğin günlüğe yazmak), ama asıl ele alma işini daha üstteki koda bırakmak istersin. Bunun için `catch` bloğunda argümansız `throw;` yazarsın: yakalanan istisnayı olduğu gibi tekrar fırlatır.

```
void orta() {
    try {
        alt();                      // alt() out_of_range fırlatıyor
    } catch (const exception& e) {
        cout << "günlüğe yazdım: " << e.what() << '\n';
        throw;                      // aynı istisna yukarı gider
    }
}
```

`orta` istisnayı `const exception&` ile yakaladı ama `throw;` yazdığı için fırlatılan şey **asıl nesnenin kendisidir**: `orta`'yı çağıran kod onu hâlâ `out_of_range` olarak yakalayabilir.

> **Sık Yapılan Hata: `throw e;` ile `throw;` aynı değildir**
>
> `catch (const exception& e) { throw e; }` yazarsan `e`'nin **kopyası** (türü `exception`) yeni bir istisna olarak fırlatılır; asıl tür dilimlenerek kaybolur. Yukarıdaki `orta` örneğinde `throw e;` yazsaydın, `orta`'yı çağıran kodun `catch (const out_of_range&)` bloğu bu istisnayı yakalayamazdı; onu yalnızca genel `catch (const exception&)` yakalardı. Yeniden fırlatırken **her zaman `throw;`** yaz.

14.7 Kendi istisna sınıfını yazmak

Hazır sınıflar her durumu anlatmaya yetmez. "Bakiye yetersiz", "stok bitti" gibi programına özgü bir hata için **kendi istisna sınıfını** yazarsın. İki nedeni var: `catch` hatayı **türüne bakarak** ayırt edebilir ve istisna, mesajın yanında hatayla ilgili **veri** de taşıyabilir.

Kural basit: `std::runtime_error`'dan (ya da duruma uygun başka bir standart sınıftan) türet ve mesajı üst sınıfın yapıcısına ilet. `what()` fonksiyonunu yeniden yazman gerekmez, `runtime_error` zaten mesajı saklar.

Kalıp üç parçadır: `class YetersizBakiyeHatasi : public runtime_error;` yapıcıda `: runtime_error("mesaj")` ile mesajı üst sınıfın yapıcısına iletmek (üye ilk değer listesi, member initializer list; Bölüm 12'deki temel sınıf yapıcısı kuralı); ve istersen hatayla ilgili **veri** tutan bir üye ile onu okuyan fonksiyon (`eksik()`). Tam yazılışı aşağıdaki banka projesinde var. Fırlatmak sıradan bir ifadedir: `throw YetersizBakiyeHatasi(100);`. Yakalayan kod hem `e.what()` ile mesajı hem `e.eksik()` ile sayıyı kullanabilir.

Kendi sınıfın bir `runtime_error` olduğu için, `catch (const runtime_error&)` ya da `catch (const exception&)` de onu yakalar. Sıra kuralı burada da geçerli: önce `YetersizBakiyeHatasi`, sonra genel türler.

> **İpucu: Ne zaman kendi sınıf, `what()` yeniden yazmak**
>
> Hazır tür durumu anlatıyorsa onu kullan; kendi sınıfını, çağıranın **türe bakarak** farklı davranması ya da hatayla birlikte **veri taşıman** gerektiğinde yaz. Mesajı yapıcıda hazırlayıp `runtime_error`'a vermek en kısa yoldur. Doğrudan `exception`'dan türetirsen mesajı `what()` fonksiyonunu kendin yazarak vermelisin: `const char* what() const noexcept override { return "..."; }`. Yazmazsan `what()` yalnızca `std::exception` der. İmzadaki `const noexcept` kısmı aynen olmalı, yoksa üst sınıfın fonksiyonunu ezmiş (override) olmazsın.

14.8 Banka projesine throw eklemek

Proje 2'deki banka uygulamasında `Hesap` sınıfı geçersiz işleme "olmaz" derken `bool` döndürüyordu: `cek` yetersiz bakiyede `false` veriyordu ve çağıranın bunu kontrol etmesi gerekiyordu. Şimdi aynı fikri istisnayla kuralım. Örneği kısa tutmak için bu sefer tutarları `double` tutuyoruz; gerçek bir uygulamada Proje 2'deki gibi parayı kuruş cinsinden `long long` ile tut. Tasarım kararları:

- Negatif ya da sıfır tutar **geçersiz argümandır** → `invalid_argument`.
- Bakiyeden fazla çekmek → kendi sınıfımız `YetersizBakiyeHatasi`, eksik miktarı taşır.
- Negatif bakiyeyle **hesap açılamaz**: yapıcı `throw` ederse nesne **hiç oluşmaz**, yarım yamalak hesap ortada dolaşmaz.
- Hesap sınıfı hatayı **ekrana yazmaz**, yalnızca fırlatır; göstermek çağıran kodun kararıdır.

bankaIstisna.cpp

```
#include <iomanip>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

using namespace std;

class YetersizBakiyeHatasi : public runtime_error {
public:
    YetersizBakiyeHatasi(double eksikTutar)
        : runtime_error("Yetersiz bakiye"), eksikMiktar{eksikTutar} {}

    double eksik() const { return eksikMiktar; }

private:
    double eksikMiktar;
};

class BankaHesabi {
public:
    BankaHesabi(const string& sahip, double ilkBakiye)
```

```
        : sahipAdi{sahip}, bakiye{ilkBakiye} {
        if (ilkBakiye < 0) {
            throw invalid_argument("Başlangıç bakiyesi negatif olamaz");
        }
    }

    void yatir(double tutar) {
        if (tutar <= 0) {
            throw invalid_argument("Yatırılacak tutar pozitif olmalı");
        }
        bakiye += tutar;
    }

    void cek(double tutar) {
        if (tutar <= 0) {
            throw invalid_argument("Çekilecek tutar pozitif olmalı");
        }
        if (tutar > bakiye) {
            throw YetersizBakiyeHatasi(tutar - bakiye);
        }
        bakiye -= tutar;
    }

    double bakiyeGetir() const { return bakiye; }

private:
    string sahipAdi;
    double bakiye;
};

struct Islem {
    char tur;           // 'y' = yatır, 'c' = çek
    double tutar;
};

int main() {
    cout << fixed << setprecision(2);

    try {
        BankaHesabi hatali{"Ali", -50};
        cout << "Bu satıra gelinmez\n";
    } catch (const invalid_argument& e) {
        cout << "Hesap açılamadı: " << e.what() << '\n';
    }

    BankaHesabi hesap{"Elif", 500};
    vector<Islem> islemler{
        {'c', 200}, {'y', -30}, {'c', 400}, {'y', 150}, {'c', 400}
    };
    for (const Islem& islem : islemler) {
        try {
            if (islem.tur == 'y') {
                hesap.yatir(islem.tutar);
            } else {
                hesap.cek(islem.tutar);
            }
            cout << "Tamam, bakiye: " << hesap.bakiyeGetir() << " TL\n";
        } catch (const YetersizBakiyeHatasi& e) {
            cout << "Reddedildi: " << e.what()
                 << ", eksik " << e.eksik() << " TL\n";
        } catch (const invalid_argument& e) {
            cout << "Geçersiz işlem: " << e.what() << '\n';
```

```
        }
    }
    return 0;
}
```

Çıktı

```
Hesap açılamadı: Başlangıç bakiyesi negatif olamaz
Tamam, bakiye: 300.00 TL
Geçersiz işlem: Yatırılacak tutar pozitif olmalı
Reddedildi: Yetersiz bakiye, eksik 100.00 TL
Tamam, bakiye: 450.00 TL
Tamam, bakiye: 50.00 TL
```

Gerçek çıktıyı satır satır izle. `hatali` nesnesi yapıcıdaki `throw` yüzünden oluşmadı; `Bu satıra gelinmez` yazılmadı. Yapıcısı tamamlanmamış bir nesne hiç doğmamış sayılır; bu yüzden onun **yıkıcısı da çağrılmaz**. Yapıcı `throw` etmeden önce `new` ile bellek aldıysa onu bırakacak kimse kalmaz; üyeleri baştan `vector`, `string` ya da `unique_ptr` ile tutarsan bu sorun olmaz, çünkü tamamlanmış üyelerin yıkıcıları yine çalışır. `-30` yatırma geçersiz argüman olarak reddedildi ve bakiye değişmedi. 400 TL çekme, bakiye 300 iken `YetersizBakiyeHatasi` fırlattı; `cek` içinde `bakiye -= tutar` satırına **gelinmediği** için bakiye bozulmadı. Bu önemli bir tasarım ilkesidir: **kontrolü ve fırlatmayı değişiklikten önce yap**; hata olunca nesne önceki tutarlı durumunda kalır. Sınıf, tutarsız bir duruma düşmeden hatayı bildirmiş olur (buna "güçlü istisna güvencesi" denir).

14.9 Yığın çözülmesi ve RAII

İstisna fırlatıldığında C++ ne yapar? `throw`'un bulunduğu fonksiyondan başlayarak çağrı yığınında (call stack) **geriye doğru** ilerler ve uyan bir `catch` arar. Her fonksiyonu terk ederken, o fonksiyonda tanımlı **yerel nesnelerin yıkıcılarını** (destructor) çağırır. Buna **yığın çözülmesi** (stack unwinding) denir.

yiginCozulmesi.cpp

```
#include <iostream>
#include <stdexcept>
#include <string>

using namespace std;

class Iz {
public:
    Iz(const string& adi) : ad{adi} { cout << "  + " << ad << " oluştu\n"; }
    ~Iz() { cout << "  - " << ad << " yok edildi\n"; }

private:
    string ad;
};

void c() {
    Iz iz{"c"};
    throw runtime_error("c'de hata oldu");
    cout << "  c bitti\n";
}

void b() {
    Iz iz{"b"};
    c();
    cout << "  b bitti\n";
}

int main() {
    Iz iz{"main"};
```

```
    try {
        b();
    } catch (const exception& e) {
        cout << "yakalandı: " << e.what() << '\n';
    }
    cout << "main bitiyor\n";
    return 0;
}
```

Çıktı

```
  + main oluştu
  + b oluştu
  + c oluştu
  - c yok edildi
  - b yok edildi
yakalandı: c'de hata oldu
main bitiyor
  - main yok edildi
```

Çıktıya bak: `c` içindeki `throw` çalışınca `c bitti` ve `b bitti` satırları **atlandı**, ama `Iz c` ve `Iz b` nesneleri yıkıcıları çalışarak **tersine sırada** (önce `c`, sonra `b`) yok edildi ve bunların hepsi `catch` bloğu çalışmadan **önce** oldu. `main`'deki `Iz main` ise `try`'ın dışında olduğu için programın sonunda yok edildi.

Bu davranış C++'ın en önemli tasarım ilkelerinden birinin temelidir: **RAII** (Resource Acquisition Is Initialization, "kaynak edinimi ilk değer vermedir"). Fikir şu: bir kaynağı (bellek, dosya, kilit, bağlantı) bir sınıfın **yapıcısında** edin, **yıkıcısında** bırak. Nesne yığında yaşadığı için ister normal yoldan, ister istisnayla çıkılsın, yıkıcı **kesinlikle** çalışır ve kaynak sızmaz. Kaynağı elle bırakmak (`delete`, `close`) zorunda kalırsan, araya giren bir istisna o satırı atlatır:

raiiKarsilastirma.cpp

```
#include <iostream>
#include <memory>
#include <stdexcept>
#include <string>

using namespace std;

class Dosya {
public:
    Dosya(const string& adi) : ad{adi} { cout << "  " << ad << " açıldı\n"; }
    ~Dosya() { cout << "  " << ad << " kapandı\n"; }

private:
    string ad;
};

void isleme(const string& adi) {
    throw runtime_error("işlem sırasında hata: " + adi);
}

void elleYonetim() {
    Dosya* d{new Dosya("A")};
    isleme("A");
    delete d;                       // buraya hiç gelinmez
}

void nesneYonetimi() {
    Dosya d{"B"};
    isleme("B");
}
```

```
void akilliIsaretci() {
    unique_ptr<Dosya> d{make_unique<Dosya>("C")};
    isleme("C");
}

int main() {
    try {
        elleYonetim();
    } catch (const exception& e) {
        cout << "  yakalandı: " << e.what() << '\n';
    }
    try {
        nesneYonetimi();
    } catch (const exception& e) {
        cout << "  yakalandı: " << e.what() << '\n';
    }
    try {
        akilliIsaretci();
    } catch (const exception& e) {
        cout << "  yakalandı: " << e.what() << '\n';
    }
    return 0;
}
```

Çıktı

```
  A açıldı
  yakalandı: işlem sırasında hata: A
  B açıldı
  B kapandı
  yakalandı: işlem sırasında hata: B
  C açıldı
  C kapandı
  yakalandı: işlem sırasında hata: C
```

`A` açıldı ama **hiç kapanmadı**: `delete d;` satırı istisna yüzünden atlandı ve `new` ile alınan bellek ile dosya sızdı (bellek sızıntısı, memory leak). `B` bir yerel nesne olduğu için yıkıcısı çalıştı. `C`'de `unique_ptr` (akıllı işaretçi, Bölüm 10) kendi yıkıcısında `delete` çağırdığı için o da temiz kapandı. Çıkarılacak ders: **ham `new`/`delete` yerine yerel nesne ya da akıllı işaretçi kullan**; istisna güvenli kod yazmanın yolu budur.

> **Sınavda Çıkar: yığın çözülmesi**
>
> Üç şeyi ezberle: (1) çözülme sırasında yıkıcılar **ters sırada** çağrılır, (2) `new` ile alınmış ham işaretçinin gösterdiği nesne için **yıkıcı çağrılmaz** (işaretçinin kendisi bir `int` gibidir, yok edilmesi hiçbir şey yapmaz), (3) yakalayan `catch` yoksa çözülme yapılıp yapılmayacağı **standart tarafından garanti edilmez**; program `terminate` ile sonlanır. Çıktı tahmini sorularında yıkıcı mesajları `catch` mesajından **önce** gelir.

> **Sık Yapılan Hata: Yıkıcıdan istisna fırlatmak**
>
> Yıkıcı normalde `noexcept`'tir. Yığın çözülürken bir yıkıcı istisna fırlatırsa aynı anda iki istisna olur ve C++ hemen `std::terminate` çağırır. Yıkıcının içinde hata olabilecek iş varsa (dosya kapatma gibi) hatayı yıkıcının içinde `try-catch` ile yakala ve yut, dışarı fırlatma.

14.10 noexcept

Bir fonksiyonun hata fırlatmayacağını biliyorsan imzasına `noexcept` yazabilirsin. Bu hem okuyana hem derleyiciye bir sözdür:

```
int topla(int a, int b) noexcept {
    return a + b;          // hata fırlatmayacağına söz verdim
}
```

Sözü tutmazsan ceza ağırdır: `noexcept` bir fonksiyonun dışına istisna çıkmaya çalışırsa standart yığın çözülmesinin yapılıp yapılmayacağını derleyiciye bırakır. g++ ve Apple clang'de çözülme yapılmaz; doğrudan `std::terminate` çağrılır ve çağıranın `catch` bloğu hiç devreye girmez.

noexceptCokme.cpp

```
#include <iostream>
#include <stdexcept>

using namespace std;

void riskliIs() {
    throw runtime_error("beklenmedik hata");
}

void sozVerdim() noexcept {
    cout << "sozVerdim başladı\n";
    riskliIs();
}

int main() {
    try {
        sozVerdim();
    } catch (const exception& e) {
        cout << "yakalandı: " << e.what() << '\n';
    }
    return 0;
}
```

Çıktı – program çöker (Aborted)

```
sozVerdim başladı
terminate called after throwing an instance of 'std::runtime_error'
  what():  beklenmedik hata
```

Çıktının `terminate` satırları g++'ya özgüdür; Apple clang aynı programda `libc++abi: terminating due to uncaught exception of type std::runtime_error: beklenmedik hata` yazar. `main`'deki `catch` bloğu hiç çalışmadı, çünkü istisna `noexcept` sınırını aşamadı.

Kural olarak: yıkıcılar zaten `noexcept` sayılır, günlük fonksiyonlarına gereksiz yere `noexcept` yazma. Yanlış söz vermek, hiç söz vermemekten kötüdür.

14.11 İstisna mı, hata kodu mu?

İstisna her derde deva değildir. Hata **sık ve beklenen** ise (kullanıcının yanlış tuşa basması) dönüş değeri ya da `if` kullan: istisna fırlatıp yakalamak, aynı kontrolü `if` ile yapmaktan **çok daha yavaştır**. Hata **gerçekten istisnaysa** (dosya yok, bakiye yetersiz, bellek bitti), hatanın birkaç fonksiyon yukarıda ele alınması gerekiyorsa ya da hata **yapıcıda** ise istisna kullan. Programcı hatalarını (kodun yanlış çağrılması) ise kullanıcıya göstermek yerine düzeltmen gerekir.

Önemli bir ayrıntı: `cin` ile sayı okurken yanlış girdi **varsayılan olarak istisna fırlatmaz**; Bölüm 3'te gördüğün gibi akış (stream) hata durumuna geçer ve `cin.fail()` ile kontrol edilir. Kullanıcı girdisi "beklenen hata" kategorisindedir. `getline` ile satırı alıp `stoi` ile çevirmek ise hataları `try-catch` ile işlemene izin verir; ikisini de bilmen gerekir.

Sık Yapılan Hata: Boş catch: hatayı gizlemek

İstisnayı yakalayıp hiçbir şey yapmamak, hatayı halının altına süpürmektir. Program çökmez ama **yanlış sonuç üretir** ve bunu kimseye söylemez:

```
vector<string> veriler{"10", "yirmi", "30"};
int toplam{0};
for (const string& v : veriler) {
    try {
        toplam += stoi(v);
    } catch (...) {
        // boş catch: hata sessizce yutuldu
    }
}
cout << "Toplam: " << toplam << '\n';      // 40 yazar; yirmi sayılmadı
```

`yirmi` için `stoi` istisna fırlatır, boş `catch` yutar ve toplam `40` çıkar. Ekranda hiçbir uyarı yok; böyle hatalar haftalarca fark edilmeyebilir. En azından hatayı görünür kıl: mesaj yazdır, günlüğe kaydet ya da yeniden fırlat. Ve `catch (...)` ile her şeyi yakalama; beklediğin **özel** türü yakala.

Bölüm Özeti – Bilmen Gerekenler

- **Hata kodu** yaklaşımında hata ile geçerli sonuç karışabilir, kontrol unutulabilir ve yapıcı hata döndüremez. **İstisna** hatayı ayrı bir kanaldan, doğrudan ele alabilecek koda iletir.
- `throw nesne;` fırlatır; `try { ... } catch (const Tur& ad) { ... }` yakalar. İstisna olunca `try`'ın geri kalanı atlanır, uyan ilk `catch` çalışır, program `try`-`catch`'in altından devam eder.
- İstisnayı her zaman **`const` referansla** yakala; değerle yakalamak nesne dilimlemeye yol açar (`what()` gerçek mesajı vermez).
- Kök sınıf `std::exception`; `what()` mesajı verir. `logic_error` ailesi (`invalid_argument`, `out_of_range`, `length_error`) ve `runtime_error` ailesi (`overflow_error`, ...) `<stdexcept>` başlığındadır. Hazır istisnaların `what()` metni derleyiciye göre değişir.
- `stoi`: sayı yoksa `invalid_argument`, `int`'e sığmıyorsa `out_of_range` fırlatır; `"12abc"` hata vermez, `12` döner. `vector::at()` sınır dışında `out_of_range` fırlatır, `[]` fırlatmaz.
- Birden çok `catch`'te **uyan ilk** çalışır; sıra **özelden genele** olmalıdır. C++ yanlış sırayı hata saymaz, yalnızca uyarı verir. `catch (...)` her şeyi yakalar ama nesneye ulaşamaz; en sona yazılır.
- Bir `catch` içinde istisnayı olduğu gibi yukarı göndermek için `throw;` yaz; `throw e;` kopyalar ve türü dilimler.
- Yakalanmayan istisna `std::terminate` ile programı sonlandırır. Kendi istisna sınıfın: `runtime_error`'dan türet, mesajı üst sınıfa ilet; istersen veri üyeleri ekle.
- **Yığın çözülmesi:** `throw` ile yakalayan `catch` arasındaki fonksiyonlar terk edilirken yerel nesnelerin yıkıcıları **ters sırada** çağrılır. **RAII:** kaynağı yapıcıda al, yıkıcıda bırak; ham `new`/`delete` yerine yerel nesne ya da akıllı işaretçi kullan.
- `noexcept` fonksiyonun hata fırlatmayacağına söz verir; söz tutulmazsa doğrudan `terminate`. Yıkıcıdan istisna fırlatma.
- Sık ve beklenen hata için `if`/dönüş değeri, gerçekten istisnai durum için istisna. Boş `catch` yazma.

Sorular

A) Çoktan Seçmeli

1 Aşağıdakilerden hangisi `std::runtime_error`'ın alt sınıfıdır?

(A) `invalid_argument`
(B) `out_of_range`
(C) `overflow_error`
(D) `bad_alloc`
(E) `length_error`

2 Bir `try` bloğunda `throw out_of_range("x");` satırı çalışıyor. Bu `try`'ın `catch` blokları sırasıyla `runtime_error`, `invalid_argument`, `logic_error` ve `exception` türünde, hepsi `const` referansla yazılmış. Hangi blok çalışır?

(A) `catch (const logic_error&)` bloğu
(B) `catch (const runtime_error&)` bloğu
(C) `catch (const invalid_argument&)` bloğu
(D) `catch (const exception&)` bloğu
(E) Hiçbiri; program `terminate` ile sonlanır

3 Bir fonksiyonda yerel bir `Dosya d;` nesnesi ve `Dosya* p{new Dosya};` ile alınmış bir ham işaretçi vardır. Sonra, `delete p;` satırından önce bir `throw` çalışıyor ve istisna yakalanıyor. Hangisi doğrudur?

(A) Hem `d`'nin hem `*p`'nin yıkıcısı çağrılır.
(B) İkisinin de yıkıcısı çağrılmaz.
(C) Yalnızca `*p`'nin yıkıcısı çağrılır.
(D) `d`'nin yıkıcısı çağrılır, `*p` ise `delete` edilmediği için yok edilmez (sızar).
(E) Yıkıcılar `catch` bloğu bittikten sonra çağrılır.

4 Fırlatılan bir istisnayı hiçbir `catch` yakalamazsa ne olur?

(A) Program hatayı yok sayar ve bir sonraki satırdan devam eder.
(B) `std::terminate` çağrılır ve program sonlanır.
(C) `main` fonksiyonu otomatik olarak `0` döndürür.
(D) Derleyici bu durumu derleme sırasında hata olarak bildirir.
(E) İstisna, programdaki ilk `catch (...)` bloğu tarafından yakalanır.

5 Aşağıdakilerden hangisi doğrudur?

(A) C++'ta fonksiyon imzasına fırlatabileceği istisnaları yazmak zorunludur.
(B) İstisnayı en güvenli biçimde değerle, yani `catch (exception e)` ile yakalarsın.
(C) `stoi("12abc")` çağrısı `invalid_argument` fırlatır.
(D) `noexcept` bir fonksiyonun dışına çıkan istisna, çağıranın `catch` bloğu tarafından yakalanır.
(E) Bir `catch` bloğunda `throw;` yazmak yakalanan istisnayı türünü koruyarak yeniden fırlatır.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır? (Yıkıcıların sırasına dikkat!)

```
#include <iostream>
#include <stdexcept>
#include <string>

using namespace std;

class Kutu {
public:
    Kutu(const string& adi) : ad{adi} { cout << ad << "+ "; }
    ~Kutu() { cout << ad << "- "; }

private:
    string ad;
};

void g() {
    Kutu k1{"A"};
    Kutu k2{"B"};
    throw runtime_error("hata");
}

int main() {
    try {
        Kutu t{"T"};
        g();
        cout << "X ";
    } catch (const exception&) {
        cout << "yakalandı\n";
    }
    cout << "son\n";
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```
#include <iostream>
#include <stdexcept>

using namespace std;

void f(int n) {
    try {
        switch (n) {
            case 1: throw invalid_argument("a");
            case 2: throw out_of_range("b");
            case 3: throw 7;
            default: cout << "ok ";
        }
    } catch (const out_of_range&) {
        cout << "oor ";
        throw;
    } catch (const logic_error&) {
        cout << "logic ";
    }
    cout << "f" << n << " ";
}

int main() {
    for (int n{0}; n <= 3; n++) {
        try {
            f(n);
        } catch (const exception&) {
            cout << "E ";
        } catch (...) {
            cout << "? ";
        }
        cout << '\n';
    }
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```
#include <iostream>
#include <stdexcept>
#include <string>

using namespace std;

int main() {
    for (string s : {"15", "7x", "x7", "3000000000"}) {
        try {
            cout << stoi(s) + 1 << " ";
        } catch (const out_of_range&) {
            cout << "R ";
        } catch (const invalid_argument&) {
            cout << "I ";
        }
    }
    cout << '\n';
    return 0;
}
```

C) Kod Yazma

9 Kullanıcıdan satır satır not okuyan `notOrtalamasi` programını yaz. Satır `bitti` olunca okuma durur. Her satır için:

- Metnin **tamamı** sayı değilse (`yetmiş`, `12abc`) `Geçersiz not (sayı değil): <satır>` yaz.
- Sayı 0-100 aralığında değilse `Geçersiz not (aralık dışı): <satır>` yaz.
- Geçerliyse nota ekle.

`stoi` ile çevir; notu doğrulayan ayrı bir fonksiyon yaz, hataları o fonksiyon **fırlatsın**, `main` yakalasın. Sonda geçerli not sayısını ve ortalamayı iki ondalıkla yazdır (geçerli not yoksa ortalama yazma). Beklenen çıktı, girdi `85, yetmiş, 120, 70, 12abc, bitti` için:

```
Geçersiz not (sayı değil): yetmiş
Geçersiz not (aralık dışı): 120
Geçersiz not (sayı değil): 12abc
Geçerli not sayısı: 2
Ortalama: 77.50
```

10 `Depo` sınıfını ve `StokYetersizHatasi` istisnasını yazarak `depoTakip` programını tamamla:

- `StokYetersizHatasi`, `runtime_error`'dan türesin; yapıcısı eksik miktarı alsın, mesajı `Stok yetersiz, X adet eksik` olsun.
- `Depo(string urun, int stok)`: stok negatifse `invalid_argument("Başlangıç stoğu negatif olamaz")` fırlatsın. Yıkıcısı `Depo kapandı: <ürün>` yazsın.
- `void cikar(int adet)`: `adet <= 0` ise `invalid_argument("Adet pozitif olmalı: X")`, `adet > stok` ise `StokYetersizHatasi` fırlatsın; aksi halde stoktan düşüp `X adet <ürün> çıkarıldı, kalan: Y` yazsın.
- `main`: önce stoğu `-5` olan bir depo oluşturmayı dene, sonra stoğu 10 olan `Kalem` deposundan sırasıyla 4, 0, 9, 6 adet çıkar; her denemeyi ayrı yakala. Beklenen çıktı:

```
Depo oluşturulamadı: Başlangıç stoğu negatif olamaz
4 adet Kalem çıkarıldı, kalan: 6
Geçersiz istek: Adet pozitif olmalı: 0
Satılamadı: Stok yetersiz, 3 adet eksik
6 adet Kalem çıkarıldı, kalan: 0
Depo kapandı: Kalem
```

Çözümler

Cevap anahtarı

1 C	2 A	3 D	4 B	5 E

1 Cevap: C

`overflow_error`, `runtime_error` ailesindendir. `invalid_argument`, `out_of_range` ve `length_error` `logic_error`'ın alt sınıflarıdır; `bad_alloc` doğrudan `exception`'dan türer.

2 Cevap: A

`out_of_range`, `logic_error`'ın alt sınıfıdır. Bloklar yukarıdan aşağıya denenir: `runtime_error` ile akraba değil (B yanlış), `invalid_argument` ile **kardeş** (C yanlış, biri diğerinin alt sınıfı değil), `logic_error`'ın alt sınıfı olduğu için üçüncü blok uyar ve çalışır. Dördüncü blok da uyardı ama **uyan ilk blok** çalıştığı için ona sıra gelmez (D yanlış). İstisna yakalandığı için program sonlanmaz (E yanlış).

3 Cevap: D

Yığın çözülürken **yerel nesnelerin** yıkıcıları çağrılır, bu yüzden `d` temizlenir. `p` ise yalnızca bir adrestir; `delete p;` satırı atlandığı için `*p` yıkılmaz ve bellek sızar (A, B, C yanlış). Yıkıcılar `catch` bloğundan **önce** çağrılır (E yanlış). Çözüm: ham işaretçi yerine `unique_ptr`.

4 Cevap: B

Yakalanmayan istisna için çalışma zamanı `std::terminate` çağırır ve program anında sonlanır (A ve C yanlış: hata yok sayılmaz, `main` değer döndürmez). Hangi istisnanın yakalanacağı çalışma zamanında belli olduğundan derleyici bunu hata saymaz (D yanlış). `catch` blokları, istisnanın yayıldığı çağrı zincirindeki `try` bloklarında aranır; programın bir yerinde yazılmış olması yetmez (E yanlış).

5 Cevap: E

`throw;` yakalanan nesneyi **olduğu gibi** yeniden fırlatır; türü korunur. A yanlış: C++'ta fırlatılabilecek istisnaları imzaya yazmak zorunlu değildir. B yanlış: değerle yakalamak kopya ve nesne dilimlemedir; doğru yazım `const` referanstır. C yanlış: `stoi("12abc")` hata vermez, `12` döndürür. D yanlış: `noexcept` bir fonksiyondan istisna çıkarsa `catch`'e gelinmez, doğrudan `terminate` çağrılır.

6

- `try` içinde `Kutu t{"T"}` oluşur → `T+`.
- `g()` çağrılır, `k1` ve `k2` sırayla oluşur → `A+ B+`.
- `throw` çalışır. `g` terk edilirken yerel nesneler **oluşma sırasının tersiyle** yok edilir → `B- A-`.
- Bloğun sonuna gelinmediği için `X` yazılmaz, ama `try` bloğundaki `t` yine yok edilir → `T-`. Hepsi `catch` çalışmadan **önce** olur.
- `catch` çalışır → `yakalandı`, sonra `son`.

Çıktı

```
T+ A+ B+ B- A- T- yakalandı
son
```

7

Her `n` için `main` döngüsü bir satır yazar:

- `n=0`: istisna yok. `try` içindeki `default` dalında `ok` yazılır; `catch`'ler atlanır, `f0` yazılır.
- `n=1`: `invalid_argument` fırlar. İlk `catch` (`out_of_range`) uymaz, ikinci (`logic_error`) uyar → `logic`, sonra `f1`.
- `n=2`: `out_of_range` fırlar. İlk `catch` uyar → `oor`, `throw;` istisnayı yukarı gönderir. `f2` hiç yazılmaz; `main`'de `catch (const exception&)` yakalar → `E`.
- `n=3`: `int` fırlar; `f` içindeki iki `catch` de uymaz. İstisna `main`'e gelir, `exception` bloğu uymaz, `catch (...)` yakalar → `?`.

Çıktı

```
ok f0
logic f1
oor E
?
```

8

- `"15"` → 15, `+ 1` = **16**.
- `"7x"` → `stoi` baştaki `7`'yi okur, `x`'i bırakır, hata **vermez** → **8**.
- `"x7"` → başta sayı yok → `invalid_argument` → `I`.
- `"3000000000"` → 3 milyar `int` sınırını (yaklaşık 2,1 milyar) aşar → `out_of_range` → `R`.

Çıktı

```
16 8 I R
```

9

`notOku` doğrulamayı yapıp hataları fırlatıyor; `main` yalnızca döngüyü ve `catch`'leri taşıyor. İstisnanın faydası bu ayrımdır. `stoi` çok büyük sayıda da `out_of_range` fırlattığı için `99999999999` gibi girdi "aralık dışı" sayılır. Toplam `double` tutulduğu için bölme ondalıklı yapılır (Bölüm 2'deki tamsayı bölmesi tuzağı).

notOrtalamasi.cpp

```cpp
#include <iomanip>
#include <iostream>
#include <stdexcept>
#include <string>

using namespace std;

int notOku(const string& satir) {
    size_t okunan{0};
    int puan{stoi(satir, &okunan)};
    if (okunan != satir.size()) {
        throw invalid_argument("sayı değil");
    }
    if (puan < 0 || puan > 100) {
        throw out_of_range("aralık dışı");
    }
    return puan;
}

int main() {
    double toplam{0};
```

```
    int adet{0};
    string satir;
    while (getline(cin, satir) && satir != "bitti") {
        try {
            toplam += notOku(satir);
            adet++;
        } catch (const invalid_argument&) {
            cout << "Geçersiz not (sayı değil): " << satir << '\n';
        } catch (const out_of_range&) {
            cout << "Geçersiz not (aralık dışı): " << satir << '\n';
        }
    }
    cout << "Geçerli not sayısı: " << adet << '\n';
    if (adet > 0) {
        cout << fixed << setprecision(2);
        cout << "Ortalama: " << toplam / adet << '\n';
    }
    return 0;
}
```

Çıktı

```
85
yetmiş
Geçersiz not (sayı değil): yetmiş
120
Geçersiz not (aralık dışı): 120
70
12abc
Geçersiz not (sayı değil): 12abc
bitti
Geçerli not sayısı: 2
Ortalama: 77.50
```

10

Yapıcı `throw` edince `Depo{"Kalem", -5}` nesnesi **hiç oluşmaz**; yıkıcısı da çağrılmaz; bu yüzden çıktıda bu nesneye ait bir `Depo kapandı` satırı yoktur. `cikar` önce kontrol edip sonra stoğu düşürür, böylece hata olursa stok bozulmaz. Sondaki `Depo kapandı: Kalem` satırını `main` biterken `depo` nesnesinin yıkıcısı yazar.

depoTakip.cpp

```
#include <iostream>
#include <stdexcept>
#include <string>

using namespace std;

class StokYetersizHatasi : public runtime_error {
public:
    StokYetersizHatasi(int eksik)
        : runtime_error("Stok yetersiz, " + to_string(eksik) + " adet eksik") {}
};

class Depo {
public:
    Depo(const string& urunAdi, int ilkStok) : urun{urunAdi}, stok{ilkStok} {
        if (ilkStok < 0) {
            throw invalid_argument("Başlangıç stoğu negatif olamaz");
        }
    }

    ~Depo() { cout << "Depo kapandı: " << urun << '\n'; }
```

```
    void cikar(int adet) {
        if (adet <= 0) {
            throw invalid_argument("Adet pozitif olmalı: " + to_string(adet));
        }
        if (adet > stok) {
            throw StokYetersizHatasi(adet - stok);
        }
        stok -= adet;
        cout << adet << " adet " << urun << " çıkarıldı, kalan: " << stok << '\n';
    }

private:
    string urun;
    int stok;
};

int main() {
    try {
        Depo hatali{"Kalem", -5};
    } catch (const invalid_argument& e) {
        cout << "Depo oluşturulamadı: " << e.what() << '\n';
    }
    Depo depo{"Kalem", 10};
    for (int adet : {4, 0, 9, 6}) {
        try {
            depo.cikar(adet);
        } catch (const StokYetersizHatasi& e) {
            cout << "Satılamadı: " << e.what() << '\n';
        } catch (const invalid_argument& e) {
            cout << "Geçersiz istek: " << e.what() << '\n';
        }
    }
    return 0;
}
```

Çıktı

```
Depo oluşturulamadı: Başlangıç stoğu negatif olamaz
4 adet Kalem çıkarıldı, kalan: 6
Geçersiz istek: Adet pozitif olmalı: 0
Satılamadı: Stok yetersiz, 3 adet eksik
6 adet Kalem çıkarıldı, kalan: 0
Depo kapandı: Kalem
```

BÖLÜM 15

STL: Kapsayıcılar ve Algoritmalar

Bu Bölümde Öğreneceklerin

- STL'nin üç parçasını: veriyi tutan yapıları, bu yapılarda gezmeyi ve hazır işlemleri
- Bir listeye ortadan eleman ekleyip silmeyi ve listeyi boşaltmayı
- Hazır işlemlerle sıralamayı, aramayı, saymayı, en büyük değeri bulmayı ve toplamayı
- Sıralama kuralını kendimiz yazmayı
- İki değeri tek pakette tutmayı
- Anahtar–değer çiftleriyle kelime saymayı ve tekrarsız veri tutmayı
- Yığın ve kuyruk gibi yalnız bir uçtan çalışan veri yapılarının mantığını
- Hangi iş için hangi veri yapısının uygun olduğunu ve işlemlerin ne kadar hızlı olduğunu ölçmenin yolunu

15.1 STL'nin üç parçası

Şimdiye kadar veriyi tek tek değişkenlerde, topluca ise dizilerde ve `vector`'de tuttun. Gerçek programlarda "sözlükte kelime ara", "tekrarlayanları ayıkla", "sıradaki işi al" gibi işler sürekli çıkar. Bunların her biri için çözüm yazmak yerine C++ sana hazır bir kütüphane verir: **STL** (Standard Template Library, Standart Şablon Kütüphanesi). Bölüm 13'te gördüğün **şablonlar** (template) üzerine kuruludur; `vector<int>` yazarken tip parametresini bu yüzden `<>` içine yazıyordun.

STL üç parçadan oluşur ve üçü birbirine kenetlenir:

Parça	Ne işe yarar?	Örnekler
Kapsayıcı (container)	Veriyi saklar	`vector`, `map`, `set`, `stack`, `queue`
İteratör (iterator)	Kapsayıcının elemanları arasında gezinen, işaretçiye benzeyen "gösterici"	`begin()`, `end()`
Algoritma (algorithm)	Kapsayıcıdaki veri üzerinde iş yapar	`sort`, `find`, `count`, `accumulate`

Fikir şudur: algoritma, kapsayıcının içini bilmez; yalnızca iteratör aracılığıyla elemanlara ulaşır. Bu sayede **tek bir `sort`** yazıldı ve `vector<int>`, `vector<string>`, dizi, hepsiyle çalışır. Kapsayıcılar kendi başlık dosyalarındadır (`<vector>`, `<map>`, `<set>`, `<unordered_map>`, `<stack>`, `<queue>`); algoritmalar `<algorithm>`'da, `accumulate` ise `<numeric>`'tedir. Hepsi `std` isim alanındadır; `using namespace std;` yazdığın için başlarına `std::` koymana gerek yok.

15.2 vector: hatırlatma ve yeni üyeler

`vector`'ü Bölüm 7'de gördün: `push_back`, `size`, `[]`, `at`, `front`, `back`, `pop_back`, `empty` ve `clear` oradaydı. Burada yalnızca eksik kalan üyeleri öğreniyoruz: elemanı **ortaya eklemek** ve **ortadan silmek**. Bunlar iteratör ister; yani konumu indeksle değil `begin() + indeks` şeklinde verirsin.

Üye fonksiyon	Ne yapar?
`v.insert(konum, deger)`	`konum`'dan **önce** `deger`'i ekler, sağdakiler kayar
`v.erase(konum)`	`konum`'daki elemanı siler, sağdakiler sola kayar
`v.erase(bas, son)`	`[bas, son)` aralığını siler (`son` dahil değil)

vektorEk.cpp

```cpp
#include <iostream>
#include <vector>

using namespace std;

void yazdir(const vector<int>& v) {
    cout << "[";
    for (size_t i = 0; i < v.size(); i++) {
        if (i > 0) {
            cout << ", ";
        }
        cout << v[i];
    }
    cout << "] boyut=" << v.size() << '\n';
}

int main() {
    vector<int> sayilar{10, 20, 30};
    yazdir(sayilar);
    cout << "ilk: " << sayilar.front() << ", son: " << sayilar.back() << '\n';

    sayilar.insert(sayilar.begin(), 5);          // en başa
    yazdir(sayilar);
    sayilar.insert(sayilar.begin() + 2, 15);     // 2. indekse
    yazdir(sayilar);

    sayilar.erase(sayilar.begin() + 1);          // 1. indeksteki elemanı sil
    yazdir(sayilar);
    sayilar.erase(sayilar.begin(), sayilar.begin() + 2);   // ilk iki eleman
    yazdir(sayilar);

    sayilar.pop_back();
    yazdir(sayilar);
    sayilar.clear();
    yazdir(sayilar);
    cout << "boş mu? " << (sayilar.empty() ? "evet" : "hayır") << '\n';
    return 0;
}
```

Çıktı

```
[10, 20, 30] boyut=3
ilk: 10, son: 30
[5, 10, 20, 30] boyut=4
[5, 10, 15, 20, 30] boyut=5
[5, 15, 20, 30] boyut=4
[20, 30] boyut=2
[20] boyut=1
[] boyut=0
boş mu? evet
```

`v.insert(v.begin() + 2, 15)` "2. indekse ekle" demektir; yeni eleman 2. indekste durur ve eski 2. indeksteki eleman bir sağa kayar. `erase(bas, son)` aralığında yarı açık aralık kuralı geçerlidir: `bas` dahil, `son` **dahil değil**. `v.erase(v.begin(), v.begin() + 2)` ilk iki elemanı siler.

> **İpucu: Başa ekleme ve ortadan silme pahalıdır**
>
> `insert` ve `erase`, eklenen/silinen konumun sağındaki **bütün** elemanları kaydırır. 10 elemanda fark etmezsin; bir milyon elemanda başa her ekleme bir milyon kaydırma demektir. Sona ekleme/silme (`push_back`, `pop_back`) ise hızlıdır. Neden böyle olduğunu bölümün sonundaki Big-O kısmında göreceksin.

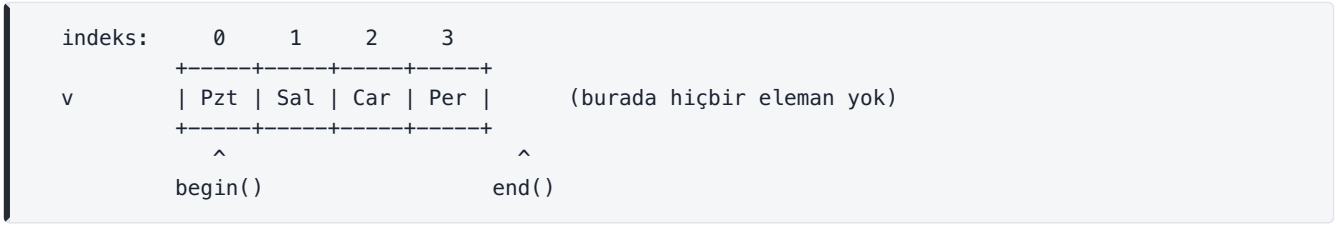
15.3 İteratörler

`sort` gibi bir algoritma "şu `vector`'ü sırala" diye çağrılsaydı yalnızca `vector` ile çalışırdı. Oysa aynı `sort` bir `string`'i, bir diziyi, bir `vector`'ün yalnızca yarısını da sıralayabilir. Bunu mümkün kılan, her kapsayıcının sunduğu ortak "gezinme" arayüzü, yani **iteratör**dür (iterator).

İteratörü, kapsayıcıdaki **bir elemanı gösteren** küçük bir nesne olarak düşün. Bölüm 9'daki işaretçiye çok benzer ve aynı işaretleri kullanır:

İşlem	Anlamı
`*it`	İteratörün gösterdiği elemana ulaş (dolaylama)
`it->uye`	Gösterilen elemanın üyesine ulaş; `(*it).uye` ile aynı
`++it`	Bir sonraki elemana geç
`it + n`	`n` eleman ileriyi göster (`vector` için)
`b - a`	İki iteratör arasındaki eleman sayısı
`it1 == it2`, `it1 != it2`	Aynı elemanı mı gösteriyorlar?

Her kapsayıcının iki özel iteratörü vardır: `begin()` **ilk elemanı**, `end()` ise **son elemanın bir ötesini** gösterir.



`end()`'in son elemanı değil, sonrasını göstermesinin iki sebebi var: Dolaşma döngüsü `it != end()` ile sade biçimde yazılır ve **boş** kapsayıcıda `begin() == end()` olur, yani döngü hiç dönmez.

iteratorGiris.cpp

```cpp
#include <iostream>
#include <string>
#include <vector>

using namespace std;

int main() {
    vector<string> gunler{"Pzt", "Sal", "Car", "Per"};

    vector<string>::iterator it = gunler.begin();
    cout << *it << '\n';
    ++it;
    cout << *it << " (" << it->size() << " harf)\n";
```

```
    it += 2;
    cout << *it << '\n';
    cout << "end - begin = " << gunler.end() - gunler.begin() << '\n';

    for (auto i = gunler.begin(); i != gunler.end(); ++i) {
        cout << *i << ' ';
    }
    cout << '\n';

    *gunler.begin() = "Pazartesi";           // iteratör üzerinden değiştir
    for (auto r = gunler.rbegin(); r != gunler.rend(); ++r) {
        cout << *r << ' ';                    // sondan başa
    }
    cout << '\n';
    return 0;
}
```

Çıktı

```
Pzt
Sal (3 harf)
Per
end - begin = 4
Pzt Sal Car Per
Per Car Sal Pazartesi
```

İki nokta:

1. `vector<string>::iterator` uzun bir tip adı; pratikte `auto` yazarsın (Bölüm 2).
2. `rbegin()` / `rend()` **ters iteratörlerdir** (reverse iterator): `++` yazdığında geriye gider. Sondan başa dolaşmanın en temiz yolu budur.

Sık Yapılan Hata: `end()` iteratörünü dolaylamak

`end()` hiçbir elemanı göstermez; ona `*` uygulamak **tanımsız davranıştır**. Aynı şey boş kapsayıcının `begin()`'i için de geçerli (o da `end()`'e eşit):

```
vector<int> v{1, 2, 3};
cout << *v.end();        // TANIMSIZ DAVRANIŞ: son elemanın ötesi
```

Çöp değer yazabilir ya da çökebilir. Her `*it` öncesinde `it != v.end()` olduğundan emin ol.

Dolaşırken eleman silmek

`erase` çağrıldığında elindeki iteratör **geçersiz** olur (silinen elemandan sonraki her şey kaymıştır); yani `for (auto it = ...; ++it) { ... v.erase(it); }` yazmak **tanımsız davranıştır**. Doğrusu, `erase`'in döndürdüğü yeni iteratörü almaktır: o, silinen elemanın **hemen sonrasındaki** elemanı gösterir. Silince `it`'i bu sonuca eşitle, **silmediysen** ilerlet:

silmeDongusu.cpp

```cpp
#include <iostream>
#include <vector>

using namespace std;

int main() {
    vector<int> v{3, 8, 12, 5, 6, 7};
    for (auto it = v.begin(); it != v.end();) {   // ++it burada YOK
        if (*it % 2 == 0) {
            it = v.erase(it);                     // sonraki elemanı döndürür
        } else {
            ++it;
        }
    }
    for (int x : v) {
        cout << x << ' ';
    }
    cout << '\n';
    return 0;
}
```

Çıktı

```
3 5 7
```

15.4 Algoritmalar: sort, find, count ve diğerleri

Algoritmalar `<algorithm>` (ve toplama için `<numeric>`) başlığındaki **fonksiyonlardır**. Hepsi aynı kalıbı izler: elemanlara bir **aralık** olarak bakarlar, yani `(baslangic, bitis)` iteratör çifti alırlar. Aralık yarı açıktır: `baslangic` dahil, `bitis` dahil değil. Tüm kapsayıcı için bu çift `v.begin(), v.end()` olur.

Algoritma	Ne yapar?	Döndürdüğü
`sort(b, e)`	Aralığı küçükten büyüğe sıralar	—
`reverse(b, e)`	Aralığı ters çevirir	—
`find(b, e, x)`	`x`'e eşit **ilk** elemanı arar	İteratör; bulunamazsa `e`
`count(b, e, x)`	`x`'e eşit eleman sayısı	Sayı
`count_if(b, e, kosul)`	`kosul`'u sağlayan eleman sayısı (koşul bir lambda; aşağıda)	Sayı
`max_element(b, e)`	En büyük elemanı bulur	**İteratör** (değer değil)
`min_element(b, e)`	En küçük elemanı bulur	**İteratör**
`accumulate(b, e, ilk)`	`ilk`'e aralıktaki her elemanı ekler (`<numeric>`)	Toplam

algoritmalar.cpp

```cpp
#include <algorithm>
#include <iomanip>
#include <iostream>
#include <numeric>
#include <vector>

using namespace std;

int main() {
    vector<int> notlar{75, 45, 88, 91, 60, 45, 100};
```

```
    sort(notlar.begin(), notlar.end());
    cout << "sıralı: ";
    for (int x : notlar) {
        cout << x << ' ';
    }
    reverse(notlar.begin(), notlar.end());
    cout << "\nters  : ";
    for (int x : notlar) {
        cout << x << ' ';
    }
    cout << '\n';

    auto it = find(notlar.begin(), notlar.end(), 88);
    if (it != notlar.end()) {
        cout << "88 bulundu, indeks " << it - notlar.begin() << '\n';
    }
    if (find(notlar.begin(), notlar.end(), 99) == notlar.end()) {
        cout << "99 yok\n";
    }
    cout << "45 kaç kez? " << count(notlar.begin(), notlar.end(), 45) << '\n';

    auto enBuyuk = max_element(notlar.begin(), notlar.end());
    auto enKucuk = min_element(notlar.begin(), notlar.end());
    cout << "en büyük " << *enBuyuk << " (indeks " << enBuyuk - notlar.begin()
         << "), en küçük " << *enKucuk << '\n';

    int toplam = accumulate(notlar.begin(), notlar.end(), 0);
    double ortalama = static_cast<double>(toplam) / notlar.size();
    cout << "toplam " << toplam << ", ortalama " << fixed << setprecision(2)
         << ortalama << '\n';
    return 0;
}
```

Çıktı

```
sıralı: 45 45 60 75 88 91 100
ters  : 100 91 88 75 60 45 45
88 bulundu, indeks 2
99 yok
45 kaç kez? 2
en büyük 100 (indeks 0), en küçük 45
toplam 504, ortalama 72.00
```

Dikkat edilecek yerler:

- **`find` ve `max_element` iteratör döndürür**, değer değil. Değere ulaşmak için `*` yazarsın, indekse ulaşmak için `it - v.begin()`. `find`'ın sonucunu **mutlaka `end()` ile karşılaştır**; bulunamadıysa `end()` döner.
- `reverse` ve `sort` yeni bir kapsayıcı üretmez, aralığı **yerinde** değiştirir. `45` iki kez geçtiği için `count` 2 verir; `find` ise tekrarlayan elemanlarda **ilkini** bulur.

Aralık, kapsayıcının yalnız bir kısmı da olabilir: `sort(d.begin() + 1, d.end() - 1)` yalnızca ilk ve son eleman hariç ortadakileri sıralar. Algoritmalar kapsayıcıyı değil iteratör aralığını görür. Sıradan bir C dizisiyle de çalışırlar: `sort(begin(dizi), end(dizi))`.

Sık Yapılan Hata: `accumulate`'in başlangıç değeri sonucun tipini belirler

`accumulate`'in üçüncü parametresi hem başlangıç değeridir hem de toplamın **tipini** belirler. `0` yazarsan toplam `int` olur ve her toplama adımında ondalık kısım atılır:

accumulateTuzak.cpp

```cpp
#include <iostream>
#include <numeric>
#include <vector>

using namespace std;

int main() {
    vector<double> fiyatlar{1.5, 2.5, 3.5};
    cout << accumulate(fiyatlar.begin(), fiyatlar.end(), 0) << '\n';
    cout << accumulate(fiyatlar.begin(), fiyatlar.end(), 0.0) << '\n';
    return 0;
}
```

Çıktı

```
6
7.5
```

Doğru toplam 7.5'tir. İlk satırda `0 + 1.5` işleminin sonucu `int`'e sığdırılırken 1 olur, sonra `1 + 2.5` → 3, sonra `3 + 3.5` → 6. Derleyici uyarı bile vermez. `double` vektörde başlangıç değerini `0.0` yaz.

Sınavda Çıkar: `find`'ın sonucunu kontrol etmek

`find`, `max_element` ve `min_element` bulunamayınca (ya da aralık boşsa) `end()` döndürür. `*find(v.begin(), v.end(), 99)` yazıp `99` yoksa program tanımsız davranışa girer. Sınavda "`find` bulamazsa `-1` mi döner?" diye sorarlar: **hayır**, `end()` iteratörü döner.

`sort` varsayılan olarak `<` operatörünü kullanır, yani küçükten büyüğe sıralar. Büyükten küçüğe ya da kendi ölçütünle sıralamak için sıralama kuralını üçüncü parametre olarak verirsin. Bunun en kısa yolu bir **lambda ifadesidir**.

15.5 Lambda ifadeleri

`sort`'a "şuna göre sırala" demek için bir **fonksiyon** vermen gerekir. Bunu sıradan bir fonksiyonla yapabilirsin; ama kural çoğu zaman tek satırdır ve yalnız o çağrıda kullanılır, ona ayrı fonksiyon yazmak yük olur. **Lambda ifadesi** (lambda expression), adı olmayan, çağrının tam ortasına yazılan küçük bir fonksiyondur:

```
[yakalama](parametreler) { gövde }
```

Yani `[]` içindeki **yakalama listesi** lambda'nın dışındaki hangi değişkenleri kullanabileceğini söyler; `(parametreler)` sıradan bir fonksiyonunki gibidir; `{ gövde }` ne yapılacağını anlatır. Dönüş tipini derleyici `return` ifadesinden çıkarır.

Sıralama ölçütü olan lambda iki eleman alır ve **birincinin ikinciden önce gelmesi gerekiyorsa** `true` döndürür:

lambdaSiralama.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

using namespace std;

int main() {
    vector<int> sayilar{42, 7, 19, 88, 3};
    sort(sayilar.begin(), sayilar.end(), [](int a, int b) { return a > b; });
    for (int x : sayilar) {
        cout << x << ' ';
    }
    cout << '\n';

    vector<string> kelimeler{"elma", "kiraz", "uzum", "muz", "armut"};
    sort(kelimeler.begin(), kelimeler.end(),
         [](const string& a, const string& b) {
             if (a.size() != b.size()) {
                 return a.size() < b.size();     // önce kısa olan
             }
             return a < b;                       // eşit uzunlukta alfabetik
         });
    for (const string& k : kelimeler) {
        cout << k << ' ';
    }
    cout << '\n';
    return 0;
}
```

Çıktı

```
88 42 19 7 3
muz elma uzum armut kiraz
```

İlk lambda `a > b` dediği için büyük sayı önce gelir (azalan sıralama). İkincisi önce uzunluğa bakar, uzunluk eşitse alfabetik sıraya başvurur. Parametreleri `const string&` aldık; Bölüm 6'dan bildiğin gibi büyük nesneyi kopyalamadan geçirmek gerekir.

Lambda'nın en kullanışlı yanı **yakalama listesidir**; dışarıdaki bir değişkeni içeri taşır:

Yakalama	Anlamı
`[]`	Dışarıdan hiçbir şey kullanılmaz
`[esik]`	`esik`'in **kopyasını** alır (değer ile)
`[&toplam]`	`toplam`'ı **referansla** alır; lambda içinde değiştirebilirsin
`[=]` / `[&]`	Kullanılan her şeyi kopyayla / referansla al

İki yeni algoritmayla birlikte görelim. **`count_if(b, e, kosul)`**, `count`'un koşullu hâlidir: aralıkta lambda'nın `true` döndürdüğü elemanları sayar. **`for_each(b, e, islem)`** ise her eleman için lambda'yı bir kez çağırır:

lambdaYakalama.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;

int main() {
    vector<int> notlar{75, 45, 88, 91, 60};
    int esik{70};
    int gecen = count_if(notlar.begin(), notlar.end(),
                         [esik](int n) { return n >= esik; });
    cout << esik << " ve üstü: " << gecen << " kişi\n";

    int toplam{0};
    for_each(notlar.begin(), notlar.end(), [&toplam](int n) { toplam += n; });
    cout << "toplam: " << toplam << '\n';
    return 0;
}
```

Çıktı

```
70 ve üstü: 3 kişi
toplam: 359
```

İlk lambda `esik`'in kopyasını aldığı için onu okuyabilir; `75`, `88` ve `91` koşulu sağlar. İkinci lambda `toplam`'ı referansla aldığı için `main`'deki `toplam`'ı değiştirir. `[toplam]` yazsaydın lambda yalnız kopyayı görürdü; kopyayla yakalanan değişken lambda içinde salt okunur olduğu için `toplam += n` derlenmezdi.

● Sık Yapılan Hata: Sıralama ölçütünde `<=` kullanmak

Ölçüt, **eşit elemanlar için `false`** döndürmelidir. `a <= b` yazarsan eşit iki eleman için `true` döner ve bu, algoritmanın dayandığı kuralı (sıkı zayıf sıralama) bozar:

```cpp
sort(v.begin(), v.end(), [](int a, int b) { return a <= b; });   // YANLIŞ
```

Sonuç **tanımsız davranıştır**: küçük veride doğru görünür, çok sayıda eşit elemanla program çökebilir. Doğrusu `<` (artan) ya da `>` (azalan). Sınavda "neden `<=` olmaz" diye sorulur.

15.6 pair: iki değeri tek pakette tutmak

Bir öğrencinin adını ve notunu birlikte taşımak istiyorsun. İki ayrı `vector` açmak (biri adlar, biri notlar) elemanların birbirinden kopmasına yol açar. **`pair`** (çift), farklı tipte **iki değeri** tek nesnede tutan şablondur; `<utility>` başlığındadır. Değerlere `first` ve `second` üyeleriyle ulaşılır. `map`'in her elemanı da bir `pair`'dir; bu yüzden şimdi görmen şart.

ciftOrnek.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <string>
#include <utility>
#include <vector>

using namespace std;

int main() {
    pair<string, int> p{"Ayşe", 85};
```

```
    cout << p.first << ": " << p.second << '\n';
    p.second += 5;
    cout << p.first << ": " << p.second << '\n';

    vector<pair<string, int>> sinif{
        {"Ece", 90}, {"Ali", 70}, {"Can", 90}, {"Bora", 60}};

    sort(sinif.begin(), sinif.end());            // önce first, eşitse second
    for (const auto& [ad, puan] : sinif) {       // C++17: çifti parçala
        cout << ad << "=" << puan << ' ';
    }
    cout << '\n';
    return 0;
}
```

Çıktı

```
Ayşe: 85
Ayşe: 90
Ali=70 Bora=60 Can=90 Ece=90
```

İki yeni şey var. `for (const auto& [ad, puan] : sinif)` C++17'nin **yapısal bağlama** (structured binding) özelliğidir: çifti `ad` ve `puan` adlı iki değişkene açar; `oge.first` / `oge.second` yazmaktan çok daha okunaklıdır. Kodda kullanmadık ama bilmen gerekir: `make_pair(3, 4.5)` tipleri yazmadan bir `pair<int, double>` üretir; eski kodlarda sık görürsün. `pair`'ler `<` ile karşılaştırılır: önce `first`'lere bakılır, eşitse `second`'lara; bu yüzden `sort`'a ölçüt vermen gerekmedi, ada göre sıralandı. Puana göre sıralamak için lambda gerekir (sorularda yapacaksın).

15.7 map: anahtar–değer çiftleri

Bir telefon rehberinde ada bakıp numarayı bulursun. Bunu `vector<pair<...>>` ile yapsan her aramada baştan sona tarardın. **`map`**, her **anahtarı** (key) bir **değere** (value) eşler ve anahtara göre arama yapmayı çok hızlı kılar. Başlığı `<map>`. Anahtarlar **benzersizdir** ve `map` elemanları **anahtara göre sıralı** tutar; değerler tekrar edebilir. Tanımı `map<AnahtarTipi, DegerTipi>` biçimindedir.

haritaTemel.cpp

```
#include <iostream>
#include <map>
#include <string>

using namespace std;

int main() {
    map<string, int> stok;
    stok["kalem"] = 40;                    // yoksa ekler
    stok["defter"] = 25;
    stok.insert({"silgi", 60});
    cout << "defter: " << stok["defter"] << '\n';

    stok["defter"] = 30;                   // varsa değerini değiştirir
    auto sonuc = stok.insert({"kalem", 99});   // varsa DOKUNMAZ
    cout << "eklendi mi? " << sonuc.second << ", kalem hâlâ "
         << sonuc.first->second << '\n';
    cout << "boyut: " << stok.size() << '\n';

    for (const auto& [ad, adet] : stok) {  // anahtara göre sıralı gelir
        cout << ad << " -> " << adet << '\n';
    }

    auto it = stok.find("silgi");
    if (it != stok.end()) {
```

```
        cout << "silgi bulundu: " << it->second << '\n';
    }
    cout << "cetvel kaç tane kayıtlı? " << stok.count("cetvel") << '\n';
    cout << "silinen: " << stok.erase("silgi") << '\n';
    return 0;
}
```

Çıktı

```
defter: 25
eklendi mi? 0, kalem hâlâ 40
boyut: 3
defter -> 30
kalem -> 40
silgi -> 60
silgi bulundu: 60
cetvel kaç tane kayıtlı? 0
silinen: 1
```

Üye fonksiyon	Ne yapar?
`m[k]`	`k`'nin değerine referans; `k` **yoksa** değeri sıfırlanmış olarak ekler
`m.at(k)`	`k`'nin değeri; yoksa `out_of_range` istisnası (Bölüm 14)
`m.insert({k, v})`	`k` yoksa ekler, **varsa dokunmaz**; `pair<iteratör, bool>` döndürür
`m.find(k)`	`k`'yi gösteren iteratör; yoksa `m.end()`
`m.count(k)`	`k` varsa 1, yoksa 0
`m.erase(k)`	`k`'yi siler; silinen eleman sayısını (0 ya da 1) döndürür
`m.size()`, `m.empty()`	Eleman sayısı, boş mu?

`map`'i dolaştığında her eleman bir `pair<const Anahtar, Deger>`'dir: `it->first` anahtar, `it->second` değerdir. Anahtar `const`tır, çünkü değiştirilirse sıralama bozulur. `insert`'in döndürdüğü `pair`'in `second`'ı eklemenin gerçekten yapılıp yapılmadığını, `first`'ü ise ilgili elemanı gösteren iteratörü verir.

Sık Yapılan Hata: `operator[]` okurken bile eleman ekler

`m[k]` anahtar yoksa onu **sıfır değerle ekler**. Bir anahtarın var olup olmadığına köşeli parantezle "bakmak" haritayı kirletir:

haritaTuzak.cpp

```cpp
#include <iostream>
#include <map>
#include <string>

using namespace std;

int main() {
    map<string, int> oy;
    oy["Ali"] = 3;
    cout << "boyut: " << oy.size() << '\n';
    if (oy["Veli"] > 0) {                // sadece bakmak istedik...
        cout << "Veli'nin oyu var\n";
    }
    cout << "boyut: " << oy.size() << '\n';
    cout << "Can kayıtlı mı? " << oy.count("Can") << '\n';   // count eklemez
    cout << "boyut: " << oy.size() << '\n';
    return 0;
}
```

Çıktı

```
boyut: 1
boyut: 2
Can kayıtlı mı? 0
boyut: 2
```

`oy["Veli"]` boyutu 1'den 2'ye çıkardı. Var mı diye bakmak için `count` ya da `find`, değeri okumak için bulunmasa istisna fırlatan `at` kullan. `[]`'i yalnız yazarken ya da sayarken kullan.

Kelime sayma

`map`'in klasik görevi **saymaktır**. Her kelime bir anahtar, kaç kez geçtiği değerdir. `sayac[kelime]++` tek satırda hem yeni kelimeyi sıfırdan başlatır (az önceki "ekleme" davranışı burada işe yarıyor) hem de bir artırır:

kelimeSay.cpp

```cpp
#include <iostream>
#include <map>
#include <string>

using namespace std;

int main() {
    map<string, int> sayac;
    string kelime;
    int toplam = 0;
    while (cin >> kelime) {              // girdi bitene kadar oku
        sayac[kelime]++;
        toplam++;
    }
    for (const auto& [k, n] : sayac) {
        cout << k << ": " << n << '\n';
    }
    cout << toplam << " kelime, " << sayac.size() << " farklı\n";
    return 0;
}
```

Çıktı

```
bir elma bir armut iki elma bir de kiraz
armut: 1
bir: 3
de: 1
elma: 2
iki: 1
kiraz: 1
9 kelime, 6 farklı
```

Kelimeler anahtar olarak alfabetik yazıldı. `cin >> kelime` boşlukla ayrılmış bir sözcük okur; girdi bitince (dosya sonu) `cin` başarısız olur ve döngü sona erer (Bölüm 3). Dikkat: sıralama karakterlerin sayısal kodlarına göredir; büyük harfler küçüklerden önce gelir ve Türkçe karakterler (`ç`, `ş` ...) UTF-8'de `z`'den sonra durur. Doğru Türkçe sıralama bu kitabın kapsamı dışında.

> **Sınavda Çıkar: Map soruları**
>
> - Aynı anahtarla ikinci `m[k] = ...` **yeni eleman eklemez**, değeri değiştirir; `size()` artmaz. `insert({k, v})` ise var olana **dokunmaz**.
> - `m[k]` olmayan anahtar için **eleman ekler** ve `0` döndürür (sayısal değerlerde). `m.at(k)` aynı durumda istisna fırlatır.
> - `map` dolaşırken **anahtara göre sıralı** verir; ekleme sırasını korumaz.

15.8 set: tekrarsız küme

Yoklama listesinde bir kişi birden fazla yazılmış olabilir; kaç **farklı** kişi geldiğini bulmak istiyorsun. `set` (küme), `<set>` başlığındadır ve elemanlarını **tekrarsız** ve **sıralı** tutar. Zaten olan bir elemanı eklemeye çalışmak hata değildir, sadece etkisizdir. İndeksle erişim yoktur; `set[2]` diye bir şey yazamazsın.

kumeOrnek.cpp

```cpp
#include <iostream>
#include <set>
#include <string>
#include <vector>

using namespace std;

int main() {
    set<int> notlar{70, 85, 70, 100, 55, 85};
    for (int n : notlar) {
        cout << n << ' ';
    }
    cout << "(boyut " << notlar.size() << ")\n";

    vector<string> yoklama{"Can", "Ece", "Ali", "Can", "Ece", "Can"};
    set<string> tekil(yoklama.begin(), yoklama.end());   // tekrarlar elenir
    cout << yoklama.size() << " kayıt, " << tekil.size() << " kişi:";
    for (const string& ad : tekil) {
        cout << ' ' << ad;
    }
    cout << '\n';
    return 0;
}
```

Çıktı
```
55 70 85 100 (boyut 4)
6 kayıt, 3 kişi: Ali Can Ece
```

`set`'i iteratör aralığıyla oluşturmak, bir `vector`'deki tekrarları elemenin en kısa yoludur. Elemanlar sıralı tutulduğu için en küçük eleman `*begin()`, en büyük eleman `*rbegin()`'dir. `insert(x).second` eklemenin yapılıp yapılmadığını söyler (`map::insert` ile aynı mantık). Sıra senin için önemli değilse, karma tablo kullanan `unordered_set` daha hızlı olabilir; `unordered_map` için aşağıya bak.

15.9 stack ve queue

Bazen elemanlara istediğin yerden değil, **yalnızca bir uçtan** ulaşmak istersin. İki klasik düzen vardır:

- **Yığın veri yapısı** (stack): **son giren ilk çıkar** (LIFO, last in first out). Üst üste dizilmiş tabaklar gibi: en son koyduğunu önce alırsın. Geri alma (undo) düğmesi ve parantez denetimi bu düzene dayanır.
- **Kuyruk** (queue): **ilk giren ilk çıkar** (FIFO, first in first out). Bankadaki sıra gibi: ilk gelen ilk hizmet alır.

> **Not: İki "yığın" var**
>
> Daha önce gördüğün **yığın** (stack), çağrı bilgilerinin tutulduğu bir **bellek bölgesiydi**. `stack` ise bir **veri yapısıdır**; ikisi aynı fikirden (son giren ilk çıkar) türemiş adlardır ama farklı şeylerdir. Kitapta `stack` yazarken veri yapısını, "yığın (bellek)" derken bellek bölgesini kastediyorum.

İkisi de **kapsayıcı adaptörüdür** (container adapter): arka planda başka bir kapsayıcıyı kullanır ve yalnızca kısıtlı bir arayüz sunar. İteratörleri yoktur, range-for ile dolaşamazsın; elemanı ancak çıkararak görebilirsin.

`stack` (`<stack>`)	`queue` (`<queue>`)	Ne yapar?
`push(x)`	`push(x)`	Ekler (`stack`'te tepeye, `queue`'da arkaya)
`top()`	`front()`, `back()`	Çıkarmadan bakar (`queue`'da ön ve arka)
`pop()`	`pop()`	**Çıkarır, ama değer döndürmez**
`empty()`, `size()`	`empty()`, `size()`	Boş mu? / Kaç eleman?

yiginKuyruk.cpp
```cpp
#include <iostream>
#include <queue>
#include <stack>
#include <string>

using namespace std;

int main() {
    stack<int> yigin;
    yigin.push(10);
    yigin.push(20);
    yigin.push(30);
    cout << "tepe: " << yigin.top() << ", boyut: " << yigin.size() << '\n';
    yigin.pop();                          // 30 gitti, değer geri gelmez
    cout << "pop sonrası tepe: " << yigin.top() << '\n';
    while (!yigin.empty()) {
        cout << yigin.top() << ' ';
        yigin.pop();
    }
    cout << '\n';
```

```
    queue<string> kuyruk;
    kuyruk.push("Ayşe");
    kuyruk.push("Burak");
    kuyruk.push("Cem");
    cout << "ön: " << kuyruk.front() << ", arka: " << kuyruk.back() << '\n';
    kuyruk.pop();
    cout << "sıradaki: " << kuyruk.front() << ", boyut: " << kuyruk.size()
         << '\n';
    return 0;
}
```

Çıktı

```
tepe: 30, boyut: 3
pop sonrası tepe: 20
20 10
ön: Ayşe, arka: Cem
sıradaki: Burak, boyut: 2
```

Yığına 10, 20, 30 sırasıyla girdi; en son giren 30 olduğu için `top()` 30 verdi ve boşaltırken sıra 20, 10 çıktı: giriş sırasının tersi. Kuyrukta ise ilk giren Ayşe önce çıktı, sıradaki Burak oldu.

Sık Yapılan Hata: `pop()` değer döndürmez, boş yığında `top()` tanımsızdır

`int x = yigin.pop();` **derlenmez**: `pop()` `void` döndürür. Önce `top()` ile oku, sonra `pop()` ile at. Boş bir `stack`'te `top()` (ya da boş `queue`'da `front()`) çağırmak ise **tanımsız davranıştır**:

```
stack<int> bos;
cout << bos.top();     // TANIMSIZ DAVRANIŞ: istisna fırlatmaz; çöp değer okur ya da çöker
```

Her `top()` / `front()` öncesi `!empty()` kontrolü yap. Döngüyü `while (!yigin.empty())` biçiminde kurmak bu yüzden kalıptır.

15.10 unordered_map (kısa)

`map` anahtarları sıralı tutmak için biraz emek harcar. Sıra önemli değilse **karma tablo** (hash table) üzerine kurulu `unordered_map` genellikle daha hızlıdır. Başlığı `<unordered_map>`; `[]`, `at`, `find`, `count`, `insert`, `erase` ve `size` `map` ile aynı işi yapar. Tek fark, elemanların dolaşma sırasının **belirsiz** olmasıdır. Bu yüzden bu bölümde dolaşma sırasını yazdırmıyoruz; elemanları sırayla yazdırman gerekiyorsa `map` kullan.

Not: g++ ile clang aynı sırayı vermez

Bu kitabın örnekleri GNU'nun standart kütüphanesiyle (libstdc++) üretildi. macOS'ta `g++` adıyla çağırdığın program aslında Apple clang'dir ve LLVM'in kendi kütüphanesini (libc++) kullanır. Aynı kodu ikisinde de çalıştırırsan dolaşma sırası farklı çıkabilir; ikisi de standarda uygundur. Bu yüzden sıraya bağlı kod yazma.

15.11 Hangi kapsayıcı ne zaman?

Kapsayıcı	Ne saklar?	Sıra	Eleman arama	Ne zaman?
`vector`	Dizi gibi ardışık elemanlar, indeksle erişim	Ekleme sırası	O(n) (sıralıysa ikili arama O(log n))	Varsayılan seçim; sona ekle, indeksle oku
`map`	Anahtar → değer	Anahtara göre sıralı	O(log n)	Sayaç, sözlük; sıralı gezmek gerekiyorsa
`set`	Tekrarsız elemanlar	Sıralı	O(log n)	Tekrarları ele, "var mı?" sor
`unordered_map`	Anahtar → değer	Belirsiz	Ortalama O(1)	Sıra gerekmeyen sözlük; en hızlı arama
`stack`	LIFO	Son giren ilk çıkar	Yalnız tepe	Geri alma, parantez denetimi
`queue`	FIFO	İlk giren ilk çıkar	Yalnız ön	Sıra bekleyen işler

Karar kuralı: **tek bir listen varsa `vector`**; "anahtarla bul" ihtiyacı doğarsa `map`; "tekrarsız ve sıralı" isteniyorsa `set`; yalnız bir uçtan çalışıyorsan `stack` / `queue`. Emin değilsen `vector` ile başla; ihtiyaç kendini gösterecektir.

Big-O'ya kısa giriş (İleri)

"`map`, `vector`'den daha hızlıdır" demek yeterince açık değil; **ne kadar** hızlı ve **neye göre**? Algoritmaların hızını saniyeyle değil, **veri büyüdükçe adım sayısının nasıl büyüdüğüyle** anlatırız. Buna **Big-O gösterimi** (Big-O notation) denir; n eleman sayısıdır:

Gösterim	Okunuşu	Anlamı	Örnek
O(1)	sabit	Eleman sayısından bağımsız	`v[i]`, `v.push_back` (ortalama), `unordered_map` araması
O(log n)	logaritmik	Veri ikiye katlanınca **bir adım** artar	`map` / `set` araması, sıralı dizide ikili arama
O(n)	doğrusal	Veri iki katına çıkınca adım da iki katı	`find`, `count`, `accumulate`, `v.insert` (ortaya)
O(n log n)	n log n (doğrusal-logaritmik)	Doğrusaldan biraz fazla	`sort`
$O(n^2)$	karesel	Veri iki katına çıkınca adım dört katı	İç içe iki döngü

`vector`'de `find` baştan sona tek tek bakar: en kötü durumda n adım atar. Sıralı bir `vector`'de ise **ikili arama** (binary search) ortadaki elemana bakıp aramayı her adımda yarıya indirir; `map` de elemanlarını dengeli bir ağaçta tutup aynı mantıkla O(log n) sürede arar.

Bir milyon elemanlı bir veride doğrusal arama en kötü durumda bir milyon adım atarken ikili arama yaklaşık yirmi adımda biter. Bu fark veri büyüdükçe daha da açılır; küçük verilerde kimse fark etmez. Bu yüzden Big-O'yu "büyük veride ne olur?" sorusunun cevabı olarak oku. Bu bölümde işine yarayacak sonuç şu: **sık sık anahtarla arama yapacaksan `vector` ve `find` yerine `map` / `set` / `unordered_map` kullan.**

Bölüm Özeti – Bilmen Gerekenler

- STL üç parçadır: **kapsayıcı** (veriyi saklar), **iteratör** (elemanlar arasında gezer), **algoritma** (veri üzerinde iş yapar). Algoritmalar iteratör aralığı `(b, e)` alır; aralık yarı açıktır.
- `vector`: `insert(konum, x)`, `erase(konum)`, `erase(b, e)`, `clear()`, `empty()`, `front()`, `back()`, `pop_back()`. Konum `v.begin() + indeks` ile verilir; ortaya ekleme/silme pahalıdır.
- `begin()` ilk elemanı, `end()` son elemanın **bir ötesini** gösterir; `*end()` tanımsız davranıştır. Döngüde silmek: `it = v.erase(it)` ve ilerletme `else` kolunda.
- `sort`, `reverse`, `find`, `count`, `max_element`, `min_element` `<algorithm>`'da; `accumulate` `<numeric>`'te. `find`/`max_element` **iteratör** döndürür; bulunamazsa `end()`. `accumulate`'in başlangıç değeri toplam tipini belirler: `double` için `0.0`.
- Lambda: `[yakalama](parametreler) { gövde }`. Sıralama ölçütü eşitte `false` döndürmeli (`<`, `>`; `<=` değil). `[x]` kopya, `[&x]` referans yakalar.
- `pair`: `first`, `second`, `make_pair`; `<` önce `first`'e bakar. C++17 ile `auto [a, b] = p;`.
- `map<K, V>`: anahtarlar benzersiz ve sıralı. `m[k]` yoksa **ekler**; `at` istisna fırlatır; `find`/`count` eklemez; `insert` var olana dokunmaz. Sayma kalıbı: `sayac[kelime]++`.
- `set<T>`: tekrarsız ve sıralı; `insert(x).second` eklendi mi; vektörden `set<T> s(v.begin(), v.end())` ile tekrarlar elenir.
- `stack` (LIFO): `push`, `top`, `pop`. `queue` (FIFO): `push`, `front`, `back`, `pop`. `pop()` değer döndürmez; boşken `top()`/`front()` tanımsız davranıştır.
- `unordered_map`: `map` ile aynı arayüz, sırası belirsiz, ortalamada O(1) arama.
- Big-O: `vector` araması O(n); `map`/`set` araması O(log n); `unordered_map` ortalama O(1); `sort` O(n log n).

Sorular

A) Çoktan Seçmeli

1 Aşağıdaki kod parçası ne yazdırır?

```
map<string, int> m;
m["a"] = 1;
m["b"];
m["a"] += 2;
cout << m["z"] << " " << m["a"];
cout << " " << m.size();
```

(A) `0 3 2`

(B) `0 1 3`

(C) `0 3 3`

(D) `out_of_range` istisnası fırlatır.

(E) Derlenmez, çünkü `m["b"];` tek başına ifade olamaz.

2 Aşağıdaki kod parçası ne yazdırır?

```
vector<double> v{0.5, 0.5, 0.5, 0.5};
cout << accumulate(v.begin(), v.end(), 0);
```

(A) `0`
(B) `2`
(C) `0.5`
(D) `1`
(E) Derleme hatası verir, çünkü `vector<double>` ile `int` başlangıç değeri uyuşmaz.

3 Aşağıdaki kod parçası ne yazdırır?

```
stack<int> s;
s.push(1);
s.push(2);
s.push(3);
s.pop();
s.push(4);
cout << s.top() << " " << s.size();
```

(A) `3 3` (B) `2 3` (C) `4 4` (D) `1 3` (E) `4 3`

4 Bir milyon elemanlı bir `map<int, string>` içinde bir anahtarı `find` ile aramanın Big-O karmaşıklığı aşağıdakilerden hangisidir?

(A) O(1) (B) O(log n) (C) O(n) (D) O(n log n) (E) $O(n^2)$

5 Aşağıdaki kod parçasının sonunda `v` hangi elemanları bu sırayla içerir?

```
vector<int> v{10, 20, 30, 40};
v.erase(v.begin() + 1);
v.insert(v.begin() + 2, 99);
```

(A) `10 99 30 40` (B) `10 20 99 40` (C) `10 30 40 99` (D) `10 30 99 40` (E) `99 10 30 40`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
#include <algorithm>
#include <iostream>
#include <string>
#include <utility>
#include <vector>

using namespace std;

int main() {
    vector<pair<string, int>> v{
        {"Ali", 70}, {"Ece", 90}, {"Can", 70}, {"Bora", 90}};
    sort(v.begin(), v.end(),
         [](const pair<string, int>& a, const pair<string, int>& b) {
             if (a.second != b.second) {
                 return a.second > b.second;
             }
             return a.first < b.first;
         });
    for (const auto& p : v) {
        cout << p.first << ' ';
    }
    cout << '\n';
    int yuksek = count_if(v.begin(), v.end(),
                          [](const pair<string, int>& p) {
                              return p.second >= 80;
                          });
    cout << yuksek << ' ' << v.front().first << ' ' << v.back().second << '\n';
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
#include <map>
#include <set>
#include <string>

using namespace std;

int main() {
    string s = "banana";
    map<char, int> m;
    for (char c : s) {
        m[c]++;
    }
    for (const auto& [harf, adet] : m) {
        cout << harf << adet << ' ';
    }
    cout << '\n';

    m.erase('b');
    m['z'];
    cout << m.size() << ' ' << m.count('b') << ' ' << m.count('z') << '\n';

    set<char> k(s.begin(), s.end());
    cout << k.size() << ' ' << *k.rbegin() << '\n';
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>
#include <queue>
#include <stack>

using namespace std;

int main() {
    queue<int> q;
    stack<int> s;
    for (int i = 1; i <= 5; i++) {
        q.push(i);
    }
    while (!q.empty()) {
        int x = q.front();
        q.pop();
        if (x == 3) {
            q.push(6);
        }
        if (x % 2 == 0) {
            s.push(x);
        } else {
            cout << x << ' ';
        }
    }
    cout << "| ";
    while (!s.empty()) {
        cout << s.top() << ' ';
        s.pop();
    }
    cout << '\n';
    return 0;
}
```

C) Kod Yazma

9 Girdiyi dosya sonuna kadar kelime kelime okuyan `siklik` programını yaz. Kelimeleri `map` ile say; sonra kelimeleri **sayısı çoktan aza**, sayıları eşit olanları **alfabetik** sırayla `kelime: sayı` biçiminde yazdır (sıralama için kelime–sayı çiftlerini bir `vector<pair<string, int>>`'a aktar ve lambda kullan). Örnek girdi ve çıktı:

```
kedi kopek kedi kus kopek kedi balik kus
```

```
kedi: 3
kopek: 2
kus: 2
balik: 1
```

10 Girdideki her satır, `()`, `[]`, `{}` ve başka karakterler içeren bir ifadedir. `parantez` programı her satır için parantezlerin **dengeli** olup olmadığını `stack<char>` ile denetlesin: Her açan parantez doğru türde bir kapatanla eşleşmeli ve sonunda açık parantez kalmamalıdır. Örnek:

```
(a+b)*[c-d]
((a+b]
{[()]}
())(
```

```
(a+b)*[c-d] -> dengeli
((a+b] -> dengesiz
{[()]} -> dengeli
())( -> dengesiz
```

Çözümler

Cevap anahtarı

1 C	2 A	3 E	4 B	5 D

1 Cevap: C

`m["a"] = 1;` `a`'yı ekler. `m["b"];` tek başına bir ifadedir ve geçerlidir: `b` yoksa **0 ile eklenir** (E yanlış). `m["a"] += 2` → `a` = 3. `m["z"]` de `z`'yi 0 ile ekler ve `0` yazdırır; sonunda `a`, `b`, `z` olmak üzere **3** anahtar vardır. `[]` istisna fırlatmaz (D yanlış, istisna fırlatan `at`'tır). A, `[]`'in okurken eklemediğini sananların; B ise `+=`'ı unutanların cevabıdır.

Çıktı
```
0 3 3
```

2 Cevap: A

`accumulate`'in başlangıç değeri `0` bir `int` olduğu için toplam `int` tutulur. Her adımda `int + 0.5` işleminin sonucu `int`'e sığdırılırken ondalık kısım atılır: `0 + 0.5` → 0, sonra yine 0 ... Sonuç `0`'dır. B (`2`) doğru matematiksel toplamdır; `0.0` başlangıcı bu değeri verirdi. C (`0.5`) ve D (`1`) bu kodun çıktısı değildir. Derleme hatası yoktur (E yanlış).

Çıktı
```
0
```

3 Cevap: E

İzleme: `push` ×3 → `1 2 3` (tepe 3). `pop()` → `1 2`. `push(4)` → `1 2 4`. `top()` 4, `size()` 3 → `4 3`. C (`4 4`) `pop()`'u unutanın cevabıdır; o zaman dört eleman kalır. A (`3 3`) hem `pop()`'u hem `push(4)`'ü unutanın cevabıdır. B (`2 3`), `stack`'i kuyruk gibi düşünüp en eski elemanı silenin; D (`1 3`) ise tepeyi en alttaki eleman sanmanın sonucudur.

Çıktı
```
4 3
```

4 Cevap: B

`map` elemanlarını dengeli bir arama ağacında tutar; her adımda arama alanı yaklaşık yarıya iner, bu yüzden arama O(log n)'dir (bir milyon elemanda yaklaşık 20 adım). O(1) `unordered_map` için ortalama durumdur; O(n) ise `vector`'de `find`'ın karmaşıklığıdır. D (`O(n log n)`) `sort`'un, E (`O(n²)`) iç içe iki döngünün karmaşıklığıdır.

5 Cevap: D

`erase(begin() + 1)` 1. indeksteki `20`'yi siler → `10 30 40`. `insert(begin() + 2, 99)` 2. indeksin **önüne** ekler: 2. indeksteki `40` bir sağa kayar → `10 30 99 40`. A (`10 99 30 40`), `insert`'i 1. indekse yapanın; B (`10 20 99 40`), `erase`'de 2. indeksteki `30`'u silenin cevabıdır.

Çıktı
```
10 30 99 40
```

6

Ölçüt: önce puan azalan, eşitse ad artan. Puanı 90 olanlar `Bora`, `Ece` (alfabetik), 70 olanlar `Ali`, `Can` gelir. `count_if` puanı 80 ve üstü olanları sayar → 2. `front()` `Bora`, `back().second` 70.

Çıktı
```
Bora Ece Ali Can
2 Bora 70
```

7

`banana` harfleri: `a` 3 kez, `b` 1 kez, `n` 2 kez. `map` anahtara göre sıralı gezer: `a3 b1 n2`. `erase('b')` → `a`, `n` kalır; `m['z']` `z`'yi 0 ile ekler → boyut 3. `count('b')` 0, `count('z')` 1. `set` yalnız farklı harfleri tutar: `a`, `b`, `n` → boyut 3; `rbegin()` en büyük elemanı (`n`) gösterir.

Çıktı
```
a3 b1 n2
3 0 1
3 n
```

8

Kuyruk `q`, yığın `s` (tepe sağda) ile izlersek: `1` tek, yazılır. `2` çift, `s`'e girer. `3` tek, yazılır; ama önce `6` kuyruğa eklenir (`q = 4 5 6`). `4` çift, `s = 2 4`. `5` tek, yazılır. `6` çift, `s = 2 4 6`.

Önce `1 3 5`, sonra `|`, sonra yığın tepeden boşaltılır: `6 4 2`.

Çıktı
```
1 3 5 | 6 4 2
```

9

`map` kelimeleri sayar; ama `map` anahtara göre sıralıdır, sayıya göre değil. Bu yüzden elemanları bir `vector<pair<string, int>>`'a kopyalayıp lambda ile sıralıyoruz: sayı azalan, eşitse kelime artan. `map`'in iteratör aralığı doğrudan `vector` yapıcısına verilebilir.

siklik.cpp
```cpp
#include <algorithm>
#include <iostream>
#include <map>
#include <string>
#include <utility>
#include <vector>

using namespace std;

int main() {
    map<string, int> sayac;
    string kelime;
    while (cin >> kelime) {
        sayac[kelime]++;
    }

    vector<pair<string, int>> liste(sayac.begin(), sayac.end());
    sort(liste.begin(), liste.end(),
         [](const pair<string, int>& a, const pair<string, int>& b) {
```

```
            if (a.second != b.second) {
                return a.second > b.second;
            }
            return a.first < b.first;
        });

    for (const auto& [k, n] : liste) {
        cout << k << ": " << n << '\n';
    }
    return 0;
}
```

Çıktı

```
kedi kopek kedi kus kopek kedi balik kus
kedi: 3
kopek: 2
kus: 2
balik: 1
```

10

Her açan parantez yığına atılır. Kapatan parantezle karşılaşınca yığın boşsa ya da tepedeki parantez türce eşleşmiyorsa ifade dengesizdir. Satırın sonunda yığında açan parantez kalmışsa yine dengesizdir. Satırları `getline` ile dosya sonuna kadar okuyoruz.

parantez.cpp

```cpp
#include <iostream>
#include <stack>
#include <string>

using namespace std;

int main() {
    string satir;
    while (getline(cin, satir)) {
        stack<char> yigin;
        bool dengeli = true;
        for (char c : satir) {
            if (c == '(' || c == '[' || c == '{') {
                yigin.push(c);
            } else if (c == ')' || c == ']' || c == '}') {
                if (yigin.empty()) {
                    dengeli = false;
                    break;
                }
                char acan = yigin.top();
                yigin.pop();
                bool eslesti = (acan == '(' && c == ')') ||
                               (acan == '[' && c == ']') ||
                               (acan == '{' && c == '}');
                if (!eslesti) {
                    dengeli = false;
                    break;
                }
            }
        }
        if (!yigin.empty()) {
            dengeli = false;
        }
```

```
        cout << satir << " -> " << (dengeli ? "dengeli" : "dengesiz") << '\n';
    }
    return 0;
}
```

Çıktı

```
(a+b)*[c-d]
(a+b)*[c-d] -> dengeli
((a+b]
((a+b] -> dengesiz
{[()]}
{[()]} -> dengeli
())(
())( -> dengesiz
```

BÖLÜM 16

Dosya İşlemleri

Bu Bölümde Öğreneceklerin

- Verinin neden dosyada saklandığını; metin ve ikili dosya farkını
- Dosyaya yazmak ve dosyadan okumak için hangi başlık dosyasının ve hangi sınıfların kullanıldığını
- Bir dosyanın açılıp açılmadığını nasıl denetleyeceğini
- Bir dosyayı satır satır ve kelime kelime okumayı; sayıları dosyadan okuyup ortalama almayı
- Var olan dosyanın sonuna ekleme yapmayı ve dosyayı yanlışlıkla silmemeyi
- Virgülle ayrılmış (CSV) satırları parçalara ayırıp bir listeye doldurmayı
- Okuma döngüsünde yapılan en yaygın hatayı ve doğru kalıbı
- Dosyanın "yanlış klasörde aranması" sorununu
- Dosyanın işi bitince kendiliğinden nasıl kapandığını
- Kayıtları ikili dosyada saklayıp istediğin kayda doğrudan gitmeyi

16.1 Dosya neden gerekir?

Şimdiye kadar bütün veriler değişkenlerdeydi, yani RAM'de; program bitince hepsi kaybolur. **Dosya** (file) diskte durur ve program bittikten sonra da yaşar: program veriyi yazar, bir sonraki çalışmada geri okur. Not listesi, ayar dosyası, günlük (log), oyunun kayıt noktası hep dosyadır. İki tür dosya vardır:

	Metin dosyası (text file)	İkili dosya (binary file)
İçerik	Okunabilir karakterler (`.txt`, `.csv`, `.cpp`)	Bellekteki baytların kopyası (`.bin`, `.dat`)
Editörle açınca	Okunur	Anlamsız semboller
Nasıl yazılır?	`<<` ve `>>` operatörleri, `getline`	`write` ve `read`
`int` değer 12345	5 karakter	Bellekteki 4 bayt

Bölüm 3'te klavye ve ekranın birer **akış** (stream) olduğunu gördün: `cin` içeri akan, `cout` dışarı akan veridir. C++'ta dosya da aynı fikirdir: dosyaya bağlanan bir akış açarsın ve ona `cin`/`cout` ile birebir aynı operatörleri uygularsın. Yeni bir şey öğrenmiyorsun; ekranın yerine dosyayı koyuyorsun.

16.2 fstream: ofstream, ifstream, fstream

Dosya akışları `<fstream>` başlık dosyasındadır ve üç sınıfla gelir:

Sınıf	Yön	Benzediği nesne	Ne için?
`ofstream` (output file stream)	Programdan dosyaya	`cout`	Dosyaya yazmak
`ifstream` (input file stream)	Dosyadan programa	`cin`	Dosyadan okumak
`fstream`	İkisi de		Aynı dosyada hem okuyup hem yazmak

Bir dosya akışı **nesnedir** (Bölüm 11'de gördüğün sınıflardan biri). Nesneyi tanımlarken dosya adını verirsin, dosya o anda açılır:

```
ofstream yaz("dosya.txt");     // yazmak için açar
ifstream oku("dosya.txt");     // okumak için açar
```

Her dosya işlemi üç adımdır: **aç** → **kullan** (oku/yaz) → **kapat**. İlk program bir dosya oluşturur, iki satır yazar, kapatır, sonra aynı dosyayı açıp satır satır okuyarak ekrana basar:

ilkDosya.cpp

```
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

int main() {
    ofstream yaz("selam.txt");
    if (!yaz.is_open()) {
        cout << "selam.txt açılamadı\n";
        return 1;
    }
    yaz << "Merhaba dosya!\n";
    yaz << "Sayı: " << 42 << '\n';
    yaz.close();

    ifstream oku("selam.txt");
    string satir;
    while (getline(oku, satir)) {
        cout << "Okunan: " << satir << '\n';
    }
    oku.close();
    return 0;
}
```

Çıktı

```
Okunan: Merhaba dosya!
Okunan: Sayı: 42
```

`yaz << ...` satırlarına bak: `cout << ...` ile aynıdır, yalnızca `cout` yerine `yaz` yazdın. Sayı `42` dosyaya `4` ve `2` karakterleri olarak gitti. `getline(oku, satir)` ise `cin`'den tanıdığın `getline(cin, satir)` kalıbıdır. Ayrıntısını birazdan göreceksin.

Açma kontrolü: is_open

Dosya açılamayabilir: adı yanlış yazdın, dosya yok, klasör yanlış, izin yok. `is_open()` dosyanın açılıp açılmadığını söyler. Akış nesnesi `if` içinde de doğrudan bir mantıksal değer gibi kullanılabilir: `if (!oku)` "akış hata durumunda mı?" diye sorar. Dosyayı açtıktan **hemen sonra** bu iki kontrol aynı sonucu verir, çünkü açılamayan akış hata durumuna da düşer. Sonrasında farklıdır: okuma bir kez başarısız olunca (dosya sonu ya da bozuk veri) `!oku` doğru olur, ama dosya hâlâ açık olduğu için `is_open()` yine `true` döndürür. Açma kontrolünde kitapta okunaklı olduğu için `is_open()` yazacağız. Olmayan bir dosyayı okumaya çalışalım:

olmayanDosya.cpp

```cpp
#include <iostream>
#include <fstream>

using namespace std;

int main() {
    ifstream oku("yok.txt");
    if (!oku.is_open()) {
        cerr << "yok.txt açılamadı\n";
        return 1;
    }
    cout << "Dosya açıldı\n";
    return 0;
}
```

Çıktı – program 1 çıkış koduyla biter

```
yok.txt açılamadı
```

`cerr`, `cout`'un hata mesajları için ayrılmış kardeşidir; ekranda ikisi iç içe görünür, farkı birazdan yönlendirmede göreceksin. `return 1;` programın sorunla bittiğini söyler (çıkış kodu, Bölüm 1); bu yüzden çıktı etiketi "1 çıkış koduyla biter" der. Yer kazanmak için bundan sonra kontrolü kısa yazacağız ama sen sınavda her zaman yaz.

> **Sık Yapılan Hata: Açılmamış akışı denetlememek**
>
> C'deki gibi çökmez, bu daha kötüdür: açılamamış bir `ifstream`'den okumaya çalışırsan akış sessizce **hata durumuna** geçer, hiçbir şey okunmaz ve program çalışmaya devam eder. Sonuç "sıfır öğrenci, ortalama 0" gibi sessiz yanlışlıklardır. Açılamamış bir `ofstream`'e yazarsan da hiçbir şey yazılmaz ve hata mesajı çıkmaz. Dosya adındaki tek bir yazım hatası bu yüzden saatlerce aranır. Her açıştan hemen sonra `is_open()` denetle.

Açma kipleri

Akış nesnesinin ikinci parametresi **kip**tir (mode) ve dosyayla ne yapacağını belirler. Kipler `ios::` öneki ile yazılır ve `|` ile birleştirilir:

Kip	Anlamı
`ios::in`	Okumak için aç (`ifstream`'in varsayılanı)
`ios::out`	Yazmak için aç; dosya yoksa oluşturur, varsa **içeriğini siler** (`ofstream`'in varsayılanı)
`ios::app`	Yazılanı hep dosyanın **sonuna** ekle (append); dosya yoksa oluşturur, içeriği korur
`ios::trunc`	Açılırken içeriği sil (`ios::out` zaten bunu yapar)
`ios::ate`	Açılınca konumu dosyanın sonuna al (at end)
`ios::binary`	İkili dosya olarak aç (bölümün sonunda)

`ofstream yaz("a.txt");` ile `ofstream yaz("a.txt", ios::out);` aynıdır. Okuyup yazan `fstream` için kipi kendin verirsin: `ios::in | ios::out` yoksa açılmaz, varsa içeriği korur; `ios::in | ios::out | ios::trunc` yoksa oluşturur, varsa siler.

> **Sınavda Çıkar**
>
> "`ofstream` ile açılan var olan dosyaya ne olur?" (içeriği silinir), "`ios::app` ile?" (sona eklenir), "`ifstream` ile olmayan dosya?" (açılmaz, `is_open()` yanlış döner) üçlüsü her sınavda gelir. `ios::in | ios::out` ile `ios::trunc` eklenmiş halinin farkı da sevilir: ikisi de okur-yazar, ama yalnız `trunc` olan içeriği siler.

Kapatma ve tampon

`close()` dosyayı kapatır. Yazdığın veri hemen diske gitmez: akış veriyi bellekte bir **tamponda** (buffer) biriktirir ve toplu yazar; kapatırken tampon boşaltılır. Bunu bölümün ilerisinde ayrıca göreceksin: dosya akışı nesnesi yok olurken kendiliğinden kapanır, yani çoğu zaman `close()` yazmak zorunda değilsin. Yine de bir dosyayı yazıp **hemen ardından aynı program içinde** okuyacaksan önce kapat (ya da `flush()` ile tamponu boşalt); yoksa okuyan akış henüz diske inmemiş veriyi göremez. Bölüm 3'te anlattığımız `endl`'in "satır sonu + `flush`" yaptığını hatırla; dosyaya yazarken her satırda tamponu boşaltmak gereksiz yavaşlık olduğu için `'\n'` yazarız.

16.3 Dosyadan okuma

Okumanın üç yolu vardır; ikisini `cin` ile zaten biliyorsun:

Yöntem	Ne okur?	Nerede durur?
`dosya >> degisken`	Bir değer (sayı, kelime)	Boşlukta (boşluk, sekme, satır sonu)
`getline(dosya, satir)`	Bir satırın tamamı	Satır sonunda; satır sonunu okur ama `satir` içine **koymaz**
`dosya.get(c)`	Tek karakter (boşluk dahil)	Hemen

Operatörle okuma: >>

`>>` boşlukları atlayıp bir değer okur ve **hangi tipte okuyorsan o tipe çevirir**. `notlar.txt` dosyasında dört satır var; program notları yazdırıp ortalamayı bulur.

> **İpucu: Veri dosyasını nereye koymalı?**
>
> Bu bölümdeki programlar okuyacakları dosyayı (burada `notlar.txt`) hazır bekler. Dosyayı VS Code'da **New File** ile oluştur, kutudaki satırları içine yaz, kaydet ve `.cpp` dosyanla **aynı klasöre** koy. Programı da o klasörde açılmış terminalden çalıştır; dosya, terminalin bulunduğu klasörde aranır. Nedenini ve dosyayı başka bir klasörde tutmanın yolunu "Dosya yolu ve çalışma klasörü" başlığında göreceksin.

notlar.txt

```
Ali 85
Ayse 92
Mehmet 70
Zeynep 100
```

notOrtalamasi.cpp

```cpp
#include <iostream>
#include <fstream>
#include <string>
#include <iomanip>

using namespace std;

int main() {
    ifstream oku("notlar.txt");
```

```
    if (!oku.is_open()) { cerr << "notlar.txt açılamadı\n"; return 1; }

    string ad;
    int puan{0};
    int toplam{0};
    int adet{0};
    while (oku >> ad >> puan) {
        cout << left << setw(8) << ad << " -> " << puan << '\n';
        toplam += puan;
        adet++;
    }
    if (adet > 0) {
        cout << adet << " öğrenci, ortalama " << fixed << setprecision(2)
             << static_cast<double>(toplam) / adet << '\n';
    }
    return 0;
}
```

Çıktı

```
Ali      -> 85
Ayse     -> 92
Mehmet   -> 70
Zeynep   -> 100
4 öğrenci, ortalama 86.75
```

Her şey `while (oku >> ad >> puan)` satırında. `>>` operatörü okuduktan sonra akışın kendisini geri verir; akış `while` içinde mantıksal değer olarak değerlendirilir: **son okuma başarılıysa doğru, başarısızsa yanlış**. Yani döngü "iki değeri de okuyabildiğin sürece" döner; dosya bitince ya da veri bozuksa okuma başarısız olur ve döngü biter. `adet > 0` denetimi boş dosyada sıfıra bölmeyi önler (Bölüm 2'deki tamsayı bölmesi dersi: bölmeden önce `static_cast<double>` ile ondalığa çevirdik).

Bu kalıp, dosyadan okumanın en önemli cümlesidir: **döngü koşulunu okuma işleminin kendisiyle kur.** Neden `eof()` kullanmadığımızı birazdan göreceksin.

Satır satır okuma: getline

`getline(dosya, satir)` bir satırı, satır sonuna kadar, boşluklar dahil `satir` string'ine koyar. Satır sonu karakterini okur ama string'e eklemez; klavyeden okuyan `getline(cin, ...)` ile aynı fonksiyondur, `cin` yerine dosya yazarsın. `metin.txt` dört satırdır, üçüncüsü boştur:

metin.txt

```
Dosyalar verileri kalıcı tutar.
Program kapanır, dosya kalır.

Son satır.
```

satirOku.cpp

```cpp
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

int main() {
    ifstream oku("metin.txt");
    if (!oku.is_open()) { cerr << "metin.txt açılamadı\n"; return 1; }

    string satir;
    int n{0};
    while (getline(oku, satir)) {
        n++;
        cout << n << ": [" << satir << "]\n";
    }
    return 0;
}
```

Çıktı

```
1: [Dosyalar verileri kalıcı tutar.]
2: [Program kapanır, dosya kalır.]
3: []
4: [Son satır.]
```

Boş satır, boş bir string olarak okunur (`[]`) ve atlanmaz. `getline` C'deki `fgets`'in aksine satırın uzunluğu için bir sınır istemez; `string` kendiliğinden büyür (Bölüm 8). Döngü koşulu yine okumanın kendisidir: `getline` dosya bittiğinde akışı hata durumuna sokar ve `while` sona erer.

Kelime kelime ve karakter karakter okuma

`string`'e `>>` ile okumak boşlukta durur; bu da dosyayı **kelime kelime** okumak demektir. Satır sonu da boşluk sayıldığı için satırların nerede bittiği önemsizdir. Kalıp, `getline` döngüsüyle aynıdır; okuma işlemi koşulun içindedir:

```cpp
string kelime;
int adet{0};
while (oku >> kelime) {         // her tur bir kelime
    adet++;
}
```

Noktalama kelimeye yapışıksa (`tutar.`) kelimenin parçası sayılır. `string::size()` karakter değil **bayt** sayar; `ı` gibi Türkçe harfler UTF-8'de iki bayttır (Bölüm 3 ve 8). Tek karakter okumak için `dosya.get(c)` vardır: boşluk ve satır sonu dahil bir `char` okur ve başarılıysa akışı "doğru" bırakır; `char c; while (oku.get(c)) { ... }` dosyayı karakter karakter işler (sayma, kopyalama, dönüştürme).

16.4 Döngü koşulunda eof tuzağı

Akışların `eof()` adında bir fonksiyonu vardır ("dosya sonu mu?") ve adı yüzünden herkesi yanıltır. Gerçek kural: `eof()`, dosya sonunu aşan bir **okuma girişimi başarısız olduktan sonra** doğru olur. Son satırı başarıyla okumak onu doğru yapmaz. Üç satırlık `meyve.txt` ile deneyelim; son satır dahil her satır `'\n'` ile biter:

meyve.txt

```
elma
kiraz
üzüm
```

eofTuzak.cpp

```cpp
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

int main() {
    ifstream oku("meyve.txt");
    if (!oku.is_open()) { cerr << "meyve.txt açılamadı\n"; return 1; }

    string satir;
    int n{0};
    while (!oku.eof()) {                    // YANLIŞ KALIP
        getline(oku, satir);
        n++;
        cout << n << ": [" << satir << "]\n";
    }
    return 0;
}
```

Çıktı

```
1: [elma]
2: [kiraz]
3: [üzüm]
4: []
```

Üç satırlık dosya **dört** satır yazdırdı. Üçüncü turda `üzüm` okundu ama dosya sonu henüz "görülmedi": `eof()` hâlâ yanlış. Döngü bir tur daha döndü, `getline` okuyacak bir şey bulamadı ve `satir`'ı boşalttı; boş satır yazıldı. (C'deki `feof` tuzağında son satır tekrar yazılırdı; burada `getline` string'i temizlediği için boş bir satır görürsün. Sayılar `>>` ile okunuyorsa değişken eski değerini korur ve son sayı **iki kez** işlenir, bir soruda göreceksin.)

Doğru kalıp, okuma işleminin kendisini koşul yapmaktır: `while (getline(oku, satir))`, `while (oku >> x)`, `while (oku.get(c))`. Okuma başarısızsa gövdeye girilmez, yani "okumadan işlem yapma" imkânı kalmaz.

> **Sık Yapılan Hata: while (!dosya.eof()) ile döngü kurmak**
>
> Eski ders kitaplarının ve internetteki örneklerin en yaygın dosya hatasıdır. Dosya `'\n'` ile bitiyorsa son kayıt iki kez işlenir ya da sonda boş bir kayıt belirir; sınavda "bu kod kaç satır yazdırır" diye tam bu hata sorulur. Her zaman okumayı koşula yaz: `while (getline(dosya, satir))`, `while (dosya >> x)`.

16.5 Akış durumu: fail ve clear (İleri)

Bir akış hata aldığında **durum bayrakları** (state flags) tutar: `eof()` okunacak bir şey kalmadan dosya sonuna gelindi, `fail()` okuma biçime uymadığı için (ya da dosya bittiği için) başarısız oldu, `good()` her şey yolunda, `bad()` ciddi G/Ç hatası (çok nadir). Okuma başarısız olunca akış "bozuk" hale gelir ve **sen `clear()` ile bayrakları temizleyene kadar sonraki bütün okumalar da başarısız olur**. Sayı beklerken `x` gelirse `fail()` doğru olur ve okunamayan `x` dosyada **kalır**. `veri.txt` içinde `3 9 x 4` yazıyor:

veri.txt

```
3 9 x 4
```

akisDurumu.cpp

```cpp
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

int main() {
    ifstream oku("veri.txt");
    if (!oku.is_open()) { cerr << "veri.txt açılamadı\n"; return 1; }

    int sayi{0};
    int toplam{0};
    while (oku >> sayi) {
        toplam += sayi;
    }
    cout << "Toplam: " << toplam << '\n';
    cout << "fail: " << oku.fail() << ", eof: " << oku.eof() << '\n';

    oku >> sayi;                        // akış bozuk: bu okuma çalışmaz
    oku.clear();                        // bayrakları temizle
    string kelime;
    oku >> kelime;                      // x hâlâ dosyada bekliyor
    cout << "Temizledikten sonra: " << kelime << '\n';
    oku >> sayi;
    cout << "Sonraki sayı: " << sayi << '\n';
    return 0;
}
```

Çıktı

```
Toplam: 12
fail: 1, eof: 0
Temizledikten sonra: x
Sonraki sayı: 4
```

Döngü `3` ve `9`'u okudu, `x` sayıya çevrilemediği için durdu: `fail()` doğru (1), `eof()` yanlış (0), çünkü dosyanın sonuna gelinmedi. Bozuk akışta yapılan okuma hiç denenmez. `clear()` bayrakları sildi; `x` hâlâ yerindedir, bu yüzden string olarak okunabildi, ardından `4` sayısı da okundu.

Dosyayı iki kez okuyacaksan (örneğin önce satırları say, sonra veriyi işle) ilk okuma `eof` ile bittiği için başa sarmadan önce bayrakları da temizle: `oku.clear(); oku.seekg(0);` (sıra önemli; `seekg`'yi birazdan göreceksin). Çoğu zaman en kolay yol, dosyayı bir kez okuyup `vector`'e koymak ve bellekte işlemektir.

Bölüm 3'te `cin` için gördüğün bir tuzak dosyada da aynen geçerlidir: `>>` değeri okur ama satır sonu karakterini **bırakır**. Hemen ardından `getline` çağırırsan o kalan satır sonunu okur ve sana **boş bir satır** verir. Çare Bölüm 3'tekiyle aynıdır: arada `dosya.ignore(numeric_limits<streamsize>::max(), '\n');` (`<limits>`) ile satırın kalanını at. Bu durumu bir soruda deneyeceksin.

16.6 Dosyaya ekleme: ios::app

`ios::app` kipi dosyanın **sonuna** ekler, içeriği silmez, dosya yoksa oluşturur. Günlük dosyaları böyle tutulur. Program önce `gunluk.txt` dosyasına tek satır yazar (varsayılan kip içeriği siler), sonra aynı dosyayı `ios::app` ile açıp iki satır ekler ve dosyanın tamamını gösterir. Veri dosyasını elle hazırlamana gerek yok; program kaç kez çalışırsa çalışsın aynı sonucu verir:

eklemeKipi.cpp

```
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

int main() {
    ofstream ilk("gunluk.txt");                 // varsayılan kip: sıfırdan yazar
    if (!ilk.is_open()) { cerr << "gunluk.txt açılamadı\n"; return 1; }
    ilk << "09:00 Ders başladı\n";
    ilk.close();

    ofstream ekle("gunluk.txt", ios::app);          // sona ekle
    if (!ekle.is_open()) { cerr << "gunluk.txt açılamadı\n"; return 1; }
    ekle << "10:00 Teneffüs\n";
    ekle << "10:15 Laboratuvar\n";
    ekle.close();

    ifstream oku("gunluk.txt");
    string satir;
    while (getline(oku, satir)) {
        cout << satir << '\n';
    }
    return 0;
}
```

Çıktı

```
09:00 Ders başladı
10:00 Teneffüs
10:15 Laboratuvar
```

`ios::app` ile açılmış bir dosyada `seekp` ile başa gitsen bile her yazma yine **sona** yapılır; standart böyle der. Düz `ofstream` yazmış olsaydın eski satır silinirdi.

16.7 CSV dosyaları

CSV (comma-separated values), her satırı bir kayıt, alanları virgülle ayrılmış metin dosyasıdır; Excel'in "CSV olarak kaydet" çıktısı budur. Yöntem iki adımdır: satırı `getline` ile al, sonra satırı alanlara ayır. Ayırmak için satırı bir **string akışına** (`stringstream`, Bölüm 8'de gördün) çevirip ondan `getline` yaparız. `getline(ss, alan, ',')` "bir sonraki virgüle kadar oku" demektir; üçüncü parametre satır sonu yerine kullanılacak **ayraç**tır. Alanlar yazı olarak gelir; sayıya çevirmek için `stoi` ve `stod` kullanılır (Bölüm 8). Gerçek veride bozuk satır çıkar; bu dosyada ilk satır başlık, beşinci satır da bilerek bozuktur:

ogrenciler.csv

```
ad,no,not
Ali Yilmaz,101,85.5
Ayse Demir,102,92
Mehmet Kaya,103,70.25
Can Er,abc,60
Zeynep Can,104,100
```

Örneğin `"Ali Yilmaz,101,85.5"` satırından `getline(ss, alan, ',')` üç kez çağrılınca sırayla `Ali Yilmaz`, `101` ve `85.5` gelir; dördüncü çağrı başarısız olur, çünkü satırda alan kalmamıştır. `stoi("101")` ve `stod("85.5")` sayıya çevirir. Yazı sayıya benzemiyorsa (`abc`) **`invalid_argument` istisnası** fırlatır (Bölüm 14); bozuk satırı yakalamak için tam bunu kullanırız. Şimdi hepsini birleştirelim: dosyayı oku, her satırı ayır, `Ogrenci` struct'ından oluşan bir `vector`'e doldur (`struct` Bölüm 11'de, `vector` Bölüm 7'de), bozuk satırları bildir, `rapor.txt` dosyasını yaz ve göster:

csvOku.cpp

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <iomanip>
#include <stdexcept>

using namespace std;

struct Ogrenci {
    string ad;
    int no;
    double puan;
};

int main() {
    ifstream oku("ogrenciler.csv");
    if (!oku.is_open()) { cerr << "ogrenciler.csv açılamadı\n"; return 1; }

    vector<Ogrenci> liste;
    string satir;
    getline(oku, satir);                          // başlık satırını atla
    while (getline(oku, satir)) {
        stringstream ss{satir};
        string ad, noYazi, puanYazi;
        getline(ss, ad, ',');
        getline(ss, noYazi, ',');
        getline(ss, puanYazi, ',');
        try {
            liste.push_back({ad, stoi(noYazi), stod(puanYazi)});
        } catch (const invalid_argument&) {
            cout << "Hatalı satır atlandı: " << satir << '\n';
        }
    }

    ofstream yaz("rapor.txt");
    if (!yaz.is_open()) { cerr << "rapor.txt açılamadı\n"; return 1; }
    double toplam{0};
    yaz << fixed << setprecision(2);
    for (const Ogrenci& o : liste) {
        yaz << o.no << ' ' << o.ad << ' ' << o.puan << '\n';
        toplam += o.puan;
    }
    if (!liste.empty()) {
```

```
        yaz << "Ortalama: " << toplam / liste.size() << '\n';
    }
    yaz.close();

    ifstream rapor("rapor.txt");                // raporu göster
    while (getline(rapor, satir)) {
        cout << satir << '\n';
    }
    return 0;
}
```

Çıktı

```
Hatalı satır atlandı: Can Er,abc,60
101 Ali Yilmaz 85.50
102 Ayse Demir 92.00
103 Mehmet Kaya 70.25
104 Zeynep Can 100.00
Ortalama: 86.94
```

Bozuk satır atlanıp bildirildi. Programda üç şeye dikkat et. Birincisi, `liste.push_back({ad, stoi(noYazi), stod(puanYazi)})` içindeki dönüşümler istisna fırlatırsa kayıt eklenmez; yarım kayıt oluşmaz. İkincisi, `stoi("12abc")` çağrısı **hata vermez**, `12` döndürür (sayı olan baştaki kısmı alır); bu kadar katı denetim gerekiyorsa dönüşümü `stringstream` ile `>>` kullanarak yapıp `ss.eof()`'a bakarsın, ama sınav düzeyinde `stoi`/`stod` + `try`/`catch` yeterlidir. Üçüncüsü, dosya Windows'ta `\r\n` ile kaydedilip Linux/macOS'ta okunursa her satırın sonunda `\r` kalır; son alan yazı olsaydı `\r` ona karışırdı. Önlem: `getline`'dan sonra satırın son karakteri `'\r'` ise at: `if (!satir.empty() && satir.back() == '\r') satir.pop_back();`.

> **İpucu: Türkçe Excel CSV'yi ; ile kaydeder**
>
> Türkçe ayarlı Excel ondalık ayırıcı virgül olduğu için alanları **noktalı virgülle** ayırır (`Ali;101;85,5`). Böyle bir dosyayı okurken `getline(ss, alan, ';')` yaz ve `stod` öncesinde ondalık virgülü noktaya çevir (`replace(alan.begin(), alan.end(), ',', '.')`, `<algorithm>`). Alanın içinde virgül varsa ve tırnakla korunmuşsa (`"Yilmaz, Ali",101`) bu basit yöntem şaşar; o durum için ayrı bir ayrıştırıcı gerekir.

16.8 cerr ve yönlendirme

`cerr` hata ve uyarı mesajları içindir; `cout`'tan **ayrı bir kanaldır** ve tamponsuzdur. Ekranda ikisi iç içe görünür; fark terminalde **yönlendirme** (redirection) yapınca ortaya çıkar: `>` normal çıktıyı (`cout`) dosyaya gönderir, `<` `cin`'i dosyadan okutur, `2>` yalnız `cerr`'i yönlendirir. Derlenmiş programın adı `program` olsun:

```
./program > cikti.txt       ← cout dosyaya gider; cerr ekranda kalır
./program < girdi.txt       ← cin klavye yerine dosyadan okur
./program 2> hatalar.txt    ← yalnız cerr dosyaya gider
```

Windows Komut İstemi'nde (cmd) komutlar `program > cikti.txt` biçiminde aynen çalışır. PowerShell'de `<` desteklenmez; yerine `Get-Content girdi.txt | .\program` yaz. Windows PowerShell 5.1'in `>` işlemi dosyayı UTF-16 yazar; Türkçe karakterli çıktıyı cmd ile yönlendirmek daha güvenlidir.

16.9 Dosya yolu ve çalışma klasörü

`ifstream oku("notlar.txt");` gibi yalnız ad yazarsan bu **göreli yoldur** (relative path): dosya, programın **çalıştığı klasörde** aranır. Bu klasör `.cpp` ya da `.exe` dosyanın yeri değil, **terminalin o anda bulunduğu** klasördür. VS Code'un terminali, açtığın proje klasöründe başlar:

```
CppDersleri> g++ -std=c++17 -Wall -Wextra ders/dosya.cpp -o dosya
CppDersleri> ./dosya        ← notlar.txt'yi CppDersleri klasöründe arar, ders/ içinde değil!
```

Bu, ilk dosya ödevinde herkesin yaşadığı olaydır: dosya `ders/` içinde duruyor, program `CppDersleri/` içinde çalışıyor. `ifstream` açılamadı der (ya da sen `is_open()` denetlemediysen hiçbir şey demez, bu yüzden denetle). Çözüm: terminalde `ls` (Windows cmd'de `dir`) ile dosyanın bulunduğu yeri gör ve ya terminali o klasöre taşı (`cd ders`) ya da dosyayı programın çalıştığı klasöre koy. Yazan programlarda da aynıdır: `ofstream` dosyayı çalışma klasöründe **oluşturur**; "dosyam nereye gitti" sorusunun cevabı çoğu zaman terminalin klasörüdür. (C++17'nin `<filesystem>` başlığındaki `filesystem::current_path()` çalışma klasörünü yazdırır.) Tam yol (absolute path) da yazabilirsin. Windows'ta ters bölü kaçış dizisinin başlangıcı olduğu için **çift** yazılır; eğik çizgi her sistemde kabul edilir. C++'a özgü üçüncü yol **ham string** (raw string) sabitidir: `R"(...)"` içinde ters bölüler olduğu gibi kalır. Üç yazılış da aynı yolu verir:

```
ifstream a("C:\\ders\\notlar.txt");     // her ters bölü çift
ifstream b("C:/ders/notlar.txt");         // eğik çizgi
ifstream c(R"(C:\ders\notlar.txt)");      // ham string
```

● Sık Yapılan Hata: Yolda tek ters bölü

`ifstream oku("C:\notlar.txt");` yazarsan `\n` yeni satır karakteri olur ve aranan ad bozulur; `"C:\test"` içindeki `\t` de sekmedir. Dosya bulunamaz. Kural Bölüm 1'deki gibidir: her ters bölü `\\` yazılır (ya da ham string kullan). Ayrıca Windows uzantıları gizliyorsa `notlar.txt` aslında `notlar.txt.txt` olabilir.

16.10 RAII: dosya kendiliğinden kapanır

Bölüm 11'de gördüğün gibi bir nesne kapsamından (`{ }` bloğundan) çıkarken **yıkıcısı** (destructor) otomatik çalışır. Dosya akışı sınıflarının yıkıcısı da tamponu boşaltır ve dosyayı kapatır. Bu yüzden çoğu zaman `close()` yazman gerekmez: nesneyi, dosyaya ihtiyaç duyulan en küçük kapsamda tanımla, kapsam bitince dosya kapanır:

```
{
    ofstream yaz("kayit.txt");
    yaz << "veri\n";
}                                   // kapsam bitti: yaz yok olur, dosya kapanır
```

Bu tasarım fikrinin adı **RAII**'dir (Resource Acquisition Is Initialization, "kaynağı edinmek, nesneyi oluşturmaktır"): kaynağı (dosya, bellek, kilit) bir nesnenin yapıcısında edinir, yıkıcısında bırakırsın; unutma ihtimali kalmaz. Bölüm 10'daki akıllı işaretçiler (`unique_ptr`) aynı fikrin bellek için olanıdır.

RAII'nin asıl değeri hata durumunda ortaya çıkar. Bir fonksiyon istisna fırlattığında **yığın çözülmesi** (stack unwinding) sırasında kapsamdaki nesnelerin yıkıcıları çalışır; dosya akışı da kapanır ve tamponundaki veri diske yazılır. `close()` satırına hiçbir zaman ulaşılamasa bile:

istisnaKapanma.cpp

```
#include <iostream>
#include <fstream>
#include <string>
#include <stdexcept>

using namespace std;

void yazVeFirlat() {
    ofstream yaz("istisna.txt");
    yaz << "Bu satır diske ulaşır\n";
    throw runtime_error("yazarken sorun çıktı");
```

```
}                                       // yığın çözülürken yaz kapanır

int main() {
    try {
        yazVeFirlat();
    } catch (const runtime_error& e) {
        cout << "Yakalandı: " << e.what() << '\n';
    }
    ifstream oku("istisna.txt");
    string satir;
    getline(oku, satir);
    cout << "Dosyada: " << satir << '\n';
    return 0;
}
```

Çıktı

```
Yakalandı: yazarken sorun çıktı
Dosyada: Bu satır diske ulaşır
```

C'de aynı durumda `fclose` satırı atlanır ve dosya açık kalırdı; C++'ta kapatmayı yıkıcı üstlenir.

Dosya akışları ayrıca **kopyalanamaz**: "aynı dosyaya iki ayrı nesne sahip" durumu yıkıcının iki kez kapatmasına yol açardı (`ofstream b = a;` derlenmez). Bu yüzden akışı fonksiyona hep **referansla** (Bölüm 9) verirsin: `void yazdir(ostream& cikis)`. `ostream`/`istream`, `cout`/`cin` ve dosya akışlarının ortak üst türleridir (Bölüm 12'de gördüğün kalıtım sayesinde); aynı fonksiyon hem dosyaya hem ekrana yazabilir.

> **Sınavda Çıkar**
>
> "`close()` yazılmazsa ne olur?" sorusunun doğru cevabı "nesne yok olurken dosya kapanır, veri diske yazılır" olur; "veri kaybolur" demek C'nin cevabıdır. Buna karşılık dosyayı yazıp **aynı kapsamda** hemen okuyacaksan önce `close()` (ya da `flush()`) gerekir; yoksa veri tamponda kalmış olabilir.

16.11 Rastgele erişim: seekg, tellg, seekp, tellp (İleri)

Dosya açılınca bir **konum göstergesi** vardır: sonraki okuma/yazma dosyanın hangi baytından başlayacak. Her okuma/yazma göstergeyi o kadar ilerletir. C++'ta okuma ve yazma konumlarının adları ayrıdır: `g` (get, okuma) ve `p` (put, yazma).

Okuma konumu için `seekg(kaydirma, nereden)` ve `tellg()` (şimdiki konum), yazma konumu için `seekp(kaydirma, nereden)` ve `tellp()` vardır; `seekg`/`seekp` göstergeyi `nereden`'den `kaydirma` bayt öteye taşır, `tellg` ise dosya başından itibaren bayt konumunu verir.

`nereden` üç değerden biridir: `ios::beg` dosyanın **başı**, `ios::cur` **şimdiki konum** (kaydırma negatif olabilir), `ios::end` dosyanın **sonu** (geri gitmek için negatif kaydırma). Tek parametreli `seekg(konum)` doğrudan baştan konuma gider. Konumlar dizi indeksi gibi 0'dan sayılır; `seekg(0, ios::end)` ardından `tellg()` **dosya boyutunu** verir. Programda 26 harflik bir dosya yazıp gezelim. Bayt konumuyla gezineceğimiz için dosyayı `ios::binary` ile açıyoruz; nedenini bölümün sonunda göreceksin:

harfler.cpp

```
#include <iostream>
#include <fstream>

using namespace std;

int main() {
    ofstream yaz("harfler.txt");
    yaz << "abcdefghijklmnopqrstuvwxyz";
```

```
    yaz.close();

    ifstream oku("harfler.txt", ios::binary);
    if (!oku.is_open()) { cerr << "harfler.txt açılamadı\n"; return 1; }
    oku.seekg(0, ios::end);
    cout << "Boyut: " << oku.tellg() << '\n';

    oku.seekg(5, ios::beg);              // baştan 5 ileri
    cout << static_cast<char>(oku.get()) << ' ';   // okuyunca konum 6
    oku.seekg(2, ios::cur);              // 6 + 2 = 8
    cout << static_cast<char>(oku.get()) << ' ';
    oku.seekg(-3, ios::end);             // 26 - 3 = 23
    cout << static_cast<char>(oku.get()) << ' ';
    cout << "konum: " << oku.tellg() << '\n';
    return 0;
}
```

Çıktı

```
Boyut: 26
f i x konum: 24
```

Parametresiz `oku.get()` sıradaki baytı okuyup `int` olarak döndürür (dosya bittiyse `EOF` değerini); harf olarak yazdırmak için `static_cast<char>` ile `char`'a çevirir. `get()` her okuyuşta konumu 1 ilerletir; konum izleme sorularının püf noktası budur. Yeni bir `seekg`'yi her zaman **o anki** konuma göre hesapla.

16.12 İkili dosyalar: write ve read (İleri)

Metin dosyasında sayıyı yazıya çevirir, okurken geri çevirirsin. **İkili** dosyada bellekteki baytlar olduğu gibi diske geçer: çevirme yok, kayıp yok. Dosyayı `ios::binary` kipiyle açarsın ve `<<`/`>>` yerine bayt yazan iki fonksiyon kullanırsın:

```
dosya.write(const char* veri, streamsize bayt);   // baytları dosyaya yazar
dosya.read(char* hedef, streamsize bayt);         // baytları dosyadan okur
```

İkisi de `char*` bekler. Bir struct'ın adresini `char*`'a çevirmek için `reinterpret_cast` kullanılır. Bu, Bölüm 2'de gördüğün `static_cast`'in daha kaba bir kardeşidir: anlamlı bir değer dönüşümü yapmaz, yalnızca aynı baytlara başka bir tipten bakar. Okuma başarılıysa `read` akışı "doğru" bırakır, yani `while (dosya.read(...))` döngüsü çalışır.

Dosya boyutu = kayıt sayısı × `sizeof(Kayit)`. Derleyici alanları hizalamak için struct'ın içine boş baytlar (**dolgu baytı**, padding) koyabilir: aşağıdaki `Kayit` için alan toplamı 15 + 4 + 8 = 27 bayt olsa da `sizeof` 32 çıkar. Bu, 64 bitlik derleyicilerdeki değerdir; 32 bitlik bir g++'da `double` 4 bayta hizalanabildiği için 28 çıkabilir. Dolgu baytları da dosyaya yazılır ve içerikleri belirsizdir. İkili dosyanın asıl kazancı boyut değil, **sabit boyutlu kayıt** (n'inci kayda doğrudan atlama), dönüşümsüz hız ve `double`'ın kayıpsız saklanmasıdır.

> **Not: İkili dosya taşınabilir değildir**
>
> İkili dosya baytları aynen yazdığı için bir bilgisayarda yazılıp başkasında okunabilmesi garanti değildir: `long` Windows'ta 4, Linux/macOS'ta 8 bayttır; struct hizalaması derleyiciye göre değişir; bazı işlemciler çok baytlı sayıları ters sırada (endianness) saklar. Yalnız aynı programla okuyacaksan ikili kullan; paylaşacaksan metin (CSV) güvenlidir. Ayrıca struct içinde `string` ya da `vector` olursa `write` metni değil nesnenin iç **işaretçisini** yazar ve bu adres sonra geçersizdir. Kaydedilecek metinler `char ad[15]` gibi **sabit boyutlu dizi** olmalıdır.

Kayıt numarasıyla okuma ve güncelleme

Kayıtlar sabit boyutlu olduğundan n'inci kaydın konumu **n × sizeof(Kayit)**'tir. Aşağıda dört kayıtlı dosyanın 2 numaralı (0'dan sayılır) kaydını okuyup puanını 10 artırır ve yerine geri yazarız. Dosyayı `fstream` ile `ios::in | ios::out | ios::binary` kipinde açıyoruz: hem okur hem yazar ama içeriği silmez (`ios::trunc` silerdi):

kayitGuncelle.cpp

```cpp
#include <iostream>
#include <fstream>
#include <iomanip>

using namespace std;

struct Kayit {
    char ad[15];
    int no;
    double puan;
};

int main() {
    Kayit liste[4] = {{"Ali", 101, 85.5}, {"Ayse", 102, 92.0},
                      {"Mehmet", 103, 70.25}, {"Zeynep", 104, 100.0}};
    cout << "Kayıt " << sizeof(Kayit) << " bayt\n";
    ofstream yaz("guncel.bin", ios::binary);
    yaz.write(reinterpret_cast<const char*>(liste), sizeof liste);
    yaz.close();

    fstream dosya("guncel.bin", ios::in | ios::out | ios::binary);
    if (!dosya.is_open()) { cerr << "guncel.bin açılamadı\n"; return 1; }
    Kayit k{};
    const streamoff konum = static_cast<streamoff>(2 * sizeof(Kayit));
    dosya.seekg(konum);                              // 2 numaralı kayıt
    if (dosya.read(reinterpret_cast<char*>(&k), sizeof k)) {
        cout << "Okunan: " << k.ad << ' ' << k.puan << '\n';
        k.puan += 10;
        dosya.seekp(konum);                          // aynı kaydın başına dön
        dosya.write(reinterpret_cast<const char*>(&k), sizeof k);
    }
    dosya.seekg(0);
    while (dosya.read(reinterpret_cast<char*>(&k), sizeof k)) {
        cout << k.no << ' ' << left << setw(7) << k.ad << k.puan << '\n';
    }
    return 0;
}
```

Çıktı

```
Kayıt 32 bayt
Okunan: Mehmet 70.25
101 Ali    85.5
102 Ayse   92
103 Mehmet 80.25
104 Zeynep 100
```

`read` konumu bir kayıt ilerlettiği için yazmadan önce `seekp(konum)` ile **geri** dönmek şarttır; yoksa güncel kayıt bir sonrakinin üstüne yazılırdı. Bir kural daha: aynı `fstream` ile okumadan yazmaya (ya da tersine) geçerken araya bir `seekg`/`seekp` (ya da `flush`) koymalısın; yukarıdaki iki `seek` bunu karşılar.

16.13 Metin ve ikili mod, Türkçe karakterli dosyalar

Windows satır sonunu iki baytla (`\r\n`), Linux ve macOS tek baytla (`\n`) yazar. **Metin kipinde** (`ios::binary` yok) C++ bu farkı gizler: Windows'ta `'\n'` yazarsan diske `\r\n` gider, okurken `\r\n` yine `'\n'` olarak gelir. **İkili kipte** (`ios::binary`) çeviri yapılmaz. Kural: **metin dosyasına metin kipi; ikili veri, bayt sayma ve konumla gezinme için ikili kip.** Resim, PDF gibi dosyaları `ios::binary` olmadan kopyalarsan Windows'ta bozarsın.

Bölüm 3'te gördüğün gibi `ş`, `ç`, `ğ`, `ü`, `ö`, `ı` UTF-8'de **iki bayt**tır ve dosyada da iki bayt tutarlar. `<<`, `>>`, `getline` baytları anlamadan taşıdığı için Türkçe metni sorunsuz kopyalar; sorun sayarken (`size()` bayt verir) ve `toupper` gibi ASCII'ye özgü fonksiyonlarda çıkar. Windows konsolunda bozuk görünürse çözüm `chcp 65001`'dir (Bölüm 3). Bir uyarı daha: Not Defteri bazen dosyayı "UTF-8 with BOM" kaydeder ve başa 3 görünmez bayt (`EF BB BF`) eklenir; okuduğun ilk kelime bu baytlarla başlar. Dosyayı VS Code'da **UTF-8** (BOM'suz) kaydet.

- **Bölüm Özeti – Bilmen Gerekenler**
  - Dosya kalıcıdır; **metin** dosyası okunabilir karakterlerden, **ikili** dosya bellek baytlarından oluşur. `<fstream>`: `ofstream` yazar, `ifstream` okur, `fstream` ikisini yapar. Dosya akışı `cin`/`cout` ile aynı operatörleri kullanır.
  - Her açıştan sonra `if (!dosya.is_open())` denetle; açılmayan akış sessizce başarısız olur. Kipler: varsayılan `ofstream` **var olanı siler**, `ios::app` sona ekler, `ios::in | ios::out` hem okur hem yazar, `ios::binary` ikili.
  - Okuma: `dosya >> x` (boşlukta durur), `getline(dosya, satir)` (satır sonu string'e girmez), `dosya.get(c)` (tek karakter). **Döngü koşulu okumanın kendisidir:** `while (getline(dosya, s))`, `while (dosya >> x)`.
  - **`while (!dosya.eof())` yanlıştır:** `eof()` okuma başarısız olduktan sonra doğru olur; son kayıt iki kez ya da sonda boş kayıt işlenir.
  - Akış hata alınca `fail()` doğru olur ve `clear()` yapılana kadar okumalar çalışmaz. İkinci kez okumadan önce `clear()` sonra `seekg(0)`.
  - `>>`'dan sonra `getline` boş satır okur; arada `ignore` kullan.
  - CSV: `getline` ile satırı al, `stringstream` + `getline(ss, alan, ',')` ile ayır, `stoi`/`stod` ile çevir, `invalid_argument` ile bozuk satırı yakala.
  - Akışlar kopyalanamaz, fonksiyona `istream&` / `ostream&` olarak verilir. Dosya nesnesi kapsamdan çıkınca (istisna olsa bile) **RAII** ile kapanır; yazıp hemen okuyacaksan önce `close()`.
  - Göreli yol terminalin bulunduğu klasörden aranır; Windows yolunda `\\` ya da ham string yaz.
  - `seekg`/`tellg` okuma, `seekp`/`tellp` yazma konumudur; `ios::beg`, `ios::cur`, `ios::end`. İkili: `write`/`read` + `reinterpret_cast`; n'inci kayıt = `n * sizeof(Kayit)`; ikili dosya taşınabilir değildir.

Sorular

A) Çoktan Seçmeli

1 Var olan `defter.txt` dosyası 5 satır içeriyor. Programda yalnızca `ofstream f("defter.txt");` satırı çalışıyor, hiçbir şey yazılmıyor ve program bitiyor. Dosyanın durumu nedir?

(A) 5 satırı da korunur.
(B) Dosya var olduğu için açılamaz, `is_open()` yanlış döner.
(C) Dosya boşalır (0 bayt olur).
(D) Dosya diskten silinir.
(E) Yalnız ilk satır korunur.

2 `sayilar.txt` dosyasında `1 2 3` yazıyor ve dosya satır sonuyla bitiyor. `int x{0};` ve `int toplam{0};` tanımlıyken aşağıdaki döngülerden hangisi `toplam`'ı doğru (6) bulur?

(A) `while (!dosya.eof()) { dosya >> x; toplam += x; }`
(B) `while (dosya.good()) { dosya >> x; toplam += x; }`
(C) `do { dosya >> x; toplam += x; } while (dosya);`
(D) `for (int i = 0; i < 100; i++) { dosya >> x; toplam += x; }`
(E) `while (dosya >> x) { toplam += x; }`

3 `gunluk.txt` içinde yalnızca `A` satırı var. Program `ofstream f("gunluk.txt", ios::app);` ile açıyor, sırayla `f.seekp(0);` ve `f << "B\n";` çalıştırıp dosyayı kapatıyor. Dosyanın içeriği (satır satır) ne olur?

(A) `B`, `A`
(B) `A`, `B`
(C) Yalnız `B`
(D) Dosya boş kalır.
(E) `seekp` yüzünden kod derlenmez.

4 Her kaydı 20 bayt olan ikili bir dosya `ifstream oku("k.bin", ios::binary);` ile açılmış. 0'dan başlayarak numaralanan **3 numaralı** kaydın başına gitmek için hangi çağrı yazılır?

(A) `oku.seekg(3, ios::beg);`
(B) `oku.seekg(3, ios::cur);`
(C) `oku.seekg(20, ios::beg);`
(D) `oku.seekg(3 * 20, ios::beg);`
(E) `oku.seekg(-3 * 20, ios::end);`

5 `kisi.txt` dosyasının ilk satırı `7`, ikinci satırı `Ali Veli`. Program `int n{0}; string ad;` tanımlayıp sırayla `dosya >> n;` ve `getline(dosya, ad);` çalıştırıyor. `ad` içinde ne vardır?

(A) Boş string (`""`)
(B) `Ali Veli`
(C) `7`
(D) `Ali`
(E) Okuma başarısız olur ve program çöker.

B) Çıktı Tahmini

6 `soru6.txt` dosyası aşağıdaki gibidir (tek satır; satır sonuyla biter):

soru6.txt

```
5 10 20
```

Aşağıdaki program ekrana ne yazdırır?

```
#include <iostream>
#include <fstream>

using namespace std;

int main() {
    ifstream f("soru6.txt");
    int x{0};
    int toplam{0};

    while (!f.eof()) {
        f >> x;
        toplam += x;
        cout << x << ' ';
    }
    cout << "\ntoplam=" << toplam << '\n';
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```
#include <iostream>
#include <fstream>

using namespace std;

int main() {
    fstream f("harf.txt", ios::in | ios::out | ios::trunc | ios::binary);
    f << "ABCDEFGHIJ";
    cout << f.tellp() << '\n';
    f.seekg(0);
    cout << static_cast<char>(f.get()) << '\n';
    f.seekg(3, ios::cur);
    cout << static_cast<char>(f.get()) << '\n';
    f.seekg(-2, ios::end);
    char c = static_cast<char>(f.get());
    cout << c << ' ' << f.tellg() << '\n';
    return 0;
}
```

8 `soru8.txt` dosyasının içeriği aşağıdadır:

soru8.txt

```
10 20 abc 30
```

Aşağıdaki program ekrana ne yazdırır?

```
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

int main() {
    ifstream f("soru8.txt");
    int x{0};
    int n{0};
    int toplam{0};

    while (f >> x) {
        toplam += x;
        n++;
    }
    cout << n << ' ' << toplam << ' ' << f.fail() << ' ' << f.eof() << '\n';
    f.clear();
    string s;
    f >> s;
    cout << s << '\n';
    f >> x;
    cout << x << '\n';
    return 0;
}
```

C) Kod Yazma

9 `soru9.txt` dosyasının her satırında bir öğrencinin adı ve notu vardır. Dosyayı **bir kez** okuyup `vector`'lere al, ortalamayı bul. **Ortalamanın üstünde** not alan öğrencileri `ustu.txt` dosyasına (`ad not` biçiminde, aynı sırayla) yazan ve aynı satırları ekrana da yazdıran `ortalamaUstu` programını yaz. Önce ortalama ekrana yazılsın.

soru9.txt

```
Ali 60
Ayse 90
Can 75
Demet 45
Efe 80
```

Beklenen ekran çıktısı:

```
Ortalama: 70.00
Ayse 90
Can 75
Efe 80
```

10 `satislar.csv` dosyasının ilk satırı başlıktır; diğer satırlar `urun,adet,fiyat` biçimindedir. Dosyayı oku; her ürünün tutarını (`adet` × `fiyat`) `Kalem: 250.00` biçiminde yazdır. Sayıya çevrilemeyen satırı `Hatali satir: ...` diye bildirip atla. Sonra `ozet.txt` dosyasına önce `Satis Ozeti` başlığını yaz ve kapat; ardından aynı dosyayı `ios::app` ile açıp `Genel toplam: ...` satırını ekle. En sonda `ozet.txt` dosyasının içeriğini ekrana yazdır. Programın adı `satisRapor` olsun.

satislar.csv

```
urun,adet,fiyat
Kalem,20,12.5
Defter,10,40
Silgi,abc,5
Kitap,3,85.75
```

Beklenen ekran çıktısı:

```
Kalem: 250.00
Defter: 400.00
Hatali satir: Silgi,abc,5
Kitap: 257.25
Satis Ozeti
Genel toplam: 907.25
```

Çözümler

Cevap anahtarı

1 C	2 E	3 B	4 D	5 A

1 Cevap: C

`ofstream` nesnesi varsayılan olarak `ios::out` kipiyle açılır ve bu kip dosyayı açar açmaz **siler** (0 bayta indirir); hiçbir şey yazılmasa da önceki içerik gider. A ve E içeriğin korunduğunu söylüyor, yanlış. B yanlış: yazma kipi dosya varsa da açar, `is_open()` yalnız açılamazsa yanlış döner. D yanlış, dosya diskte durur ama boştur.

2 Cevap: E

Okumayı koşula yazan `while (dosya >> x)` kalıbı, okuma başarısız olur olmaz döngüyü bitirir; son sayı bir kez işlenir. A, B ve C aynı hatayı yapar: son sayıdan sonra dosyada yalnız satır sonu kalır, `eof()` hâlâ yanlıştır, döngü bir tur daha döner, `>>` başarısız olur ve `x` son değeri olan `3`'ü korur; `3` iki kez toplanır (toplam 9). D ise 100 tur döner ve `3`'ü defalarca toplar.

3 Cevap: B

`ios::app` kipinde yazılan her veri, araya `seekp` girse bile **dosyanın sonuna** eklenir. A ve C `seekp`'in işe yaradığını varsayıyor. D yanlış, yazma yapılır. E yanlış, `seekp` `ofstream`'in geçerli bir üyesidir ve kod derlenir.

Çıktı

```
A
B
```

4 Cevap: D

3 numaralı kaydın konumu `3 * 20 = 60` bayttır ve baştan ölçüldüğü için `ios::beg` kullanılır. A kayıt boyutunu unutmuş, 3. bayta gider. B şimdiki konumdan 3 bayt ilerler. C 1 numaralı kaydın başına gider. E dosyanın **sonundan** 60 bayt geri gider; bu ancak dosyada tam 6 kayıt varsa 3 numaralı kaydın başına denk gelir, yani genel bir çözüm değildir.

5 Cevap: A

`dosya >> n` yalnızca `7`'yi okur; satır sonu karakteri dosyada **kalır**. Hemen ardından `getline` o satırın kalanını, yani boş bir parçayı okur ve `ad` boş string olur. B beklenen sonuç ama olmuyor: `Ali Veli` ikinci `getline` çağrısıyla gelir. C yanlış, `ad` string'ine `7` hiç girmez. D yanlış, `getline` kelimede durmaz. E yanlış, okuma başarılıdır.

Çıktı

```
[]
```

6

Dosya `5 10 20\n` içerir. İlk üç turda `5`, `10`, `20` okunur ve eklenir. Üçüncü turdan sonra dosyada yalnız satır sonu kalmıştır; `eof()` hâlâ yanlıştır. Dördüncü turda `>>` başarısız olur (satır sonunu atlayıp dosya sonuna çarpar), `x` son değeri olan `20`'yi korur ve `20` bir kez daha eklenir. Bu turdan sonra `eof()` doğru olur. Toplam 5 + 10 + 20 + 20 = 55.

```
Çıktı
5 10 20 20
toplam=55
```

7

Dosya `ABCDEFGHIJ`, 10 bayt (konumlar 0-9). Yazdıktan sonra konum 10 olduğundan ilk satır `10`. `seekg(0)` konumu 0 yapar; `get A` okur (konum 1). `seekg(3, ios::cur)` konumu 1 + 3 = 4 yapar; `get E`'yi okur (konum 5). `seekg(-2, ios::end)` konumu 10 - 2 = 8 yapar; `get I`'yı okur ve konum 9 olur: `I 9`. Sondan 2 geri giden bayt, 8 numaralı konumdaki `I`'dır.

```
Çıktı
10
A
E
I 9
```

8

Döngü `10` ve `20`'yi okur (`n = 2`, `toplam = 30`). Üçüncü okumada `abc` sayı değildir; okuma başarısız olur ve `fail()` doğru (1) olur. Dosyanın sonuna gelinmediği için `eof()` yanlıştır (0). `clear()` bayrakları siler; okunamayan `abc` dosyada kaldığı için `f >> s` onu okur. Son `f >> x` ise sıradaki `30`'u okur.

```
Çıktı
2 30 1 0
abc
30
```

9

Notlar ve adlar iki `vector`'e okunur, ortalama bulunur, ikinci geçişte (bellekte, dosyaya tekrar gitmeden) ortalamadan büyük notlar hem `ustu.txt`'ye hem ekrana yazılır. Boş dosya ihtimaline karşı `notlar.empty()` denetlenir.

ortalamaUstu.cpp

```cpp
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <iomanip>

using namespace std;

int main() {
    ifstream giris("soru9.txt");
    if (!giris.is_open()) { cerr << "soru9.txt açılamadı\n"; return 1; }

    vector<string> adlar;
    vector<int> notlar;
    string ad;
    int puan{0};
    int toplam{0};
    while (giris >> ad >> puan) {
        adlar.push_back(ad);
        notlar.push_back(puan);
        toplam += puan;
    }
```

```
    if (notlar.empty()) { return 0; }
    double ortalama = static_cast<double>(toplam) / notlar.size();
    cout << "Ortalama: " << fixed << setprecision(2) << ortalama << '\n';

    ofstream cikis("ustu.txt");
    if (!cikis.is_open()) { cerr << "ustu.txt açılamadı\n"; return 1; }
    for (size_t i = 0; i < notlar.size(); i++) {
        if (notlar[i] > ortalama) {
            cikis << adlar[i] << ' ' << notlar[i] << '\n';
            cout << adlar[i] << ' ' << notlar[i] << '\n';
        }
    }
    return 0;
}
```

Çıktı

```
Ortalama: 70.00
Ayse 90
Can 75
Efe 80
```

10

Her satır `stringstream` ile `getline(ss, alan, ',')` kullanılarak üç alana bölünür. `stoi` ve `stod` bozuk alanda `invalid_argument` fırlatır; `catch` satırı bildirip atlar. Özet dosyası önce `ofstream` ile sıfırdan yazılır, kapatılır, sonra `ios::app` ile yeniden açılıp sonuna eklenir. Dosyayı sonda okumadan önce eklemeyi yapan akışın kapanması (burada kapsamın bitmesi) şarttır.

satisRapor.cpp

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <iomanip>
#include <stdexcept>

using namespace std;

int main() {
    ifstream oku("satislar.csv");
    if (!oku.is_open()) { cerr << "satislar.csv açılamadı\n"; return 1; }

    cout << fixed << setprecision(2);
    double genel{0};
    string satir;
    getline(oku, satir);                          // başlık
    while (getline(oku, satir)) {
        stringstream ss{satir};
        string urun, adetYazi, fiyatYazi;
        getline(ss, urun, ',');
        getline(ss, adetYazi, ',');
        getline(ss, fiyatYazi, ',');
        try {
            double tutar = stoi(adetYazi) * stod(fiyatYazi);
            cout << urun << ": " << tutar << '\n';
            genel += tutar;
        } catch (const invalid_argument&) {
            cout << "Hatali satir: " << satir << '\n';
        }
    }
```

```
    { ofstream baslik("ozet.txt"); baslik << "Satis Ozeti\n"; }
    {
        ofstream ekle("ozet.txt", ios::app);
        ekle << fixed << setprecision(2) << "Genel toplam: " << genel << '\n';
    }

    ifstream ozet("ozet.txt");
    while (getline(ozet, satir)) {
        cout << satir << '\n';
    }
    return 0;
}
```

Çıktı

```
Kalem: 250.00
Defter: 400.00
Hatali satir: Silgi,abc,5
Kitap: 257.25
Satis Ozeti
Genel toplam: 907.25
```

BÖLÜM 17

Özyineleme (Recursion)

Bu Bölümde Öğreneceklerin

- Özyinelemenin (kendini çağıran fonksiyon) ne olduğunu ve hangi problemlerde işe yaradığını
- Temel durum ile özyinelemeli durumu; her çağrının temel duruma yaklaşması gerektiğini
- `faktoriyel(4)` çağrısında çağrı yığınının adım adım nasıl dolup boşaldığını
- Faktöriyel, üs alma, basamak toplamı ve EBOB'u özyinelemeyle yazmayı
- Fibonacci'de çağrı ağacını, tekrar eden hesapları ve bellekleme ile hızlandırmayı
- vector ve string üzerinde özyinelemeyi (toplam, en büyük, ters çevirme, palindrom)
- Hanoi kulelerini ve özyinelemeli ikili aramayı
- Yazdırmayı çağrıdan önce ya da sonra yapmanın çıktıyı nasıl tersine çevirdiğini
- Temel durum unutulunca olan yığın taşmasını; özyineleme ile döngü arasında seçim yapmayı

17.1 Özyineleme nedir

Rus **matruşka** bebeklerini bilirsin: bebeği açarsın, içinden daha küçük bir bebek çıkar; onu açarsın, içinden bir küçüğü daha çıkar. Her bebeğin içinde "kendisinin daha küçük hâli" vardır. En küçük bebek açılmaz; işlem orada biter. **Özyineleme** (recursion) tam olarak bu fikirdir: bir problemi, **aynı türden ama daha küçük** bir probleme indirgersin; en küçük problemi ise doğrudan çözersin.

Programlamada bunun anlamı basittir: **bir fonksiyon kendi kendini çağırır.** Bölüm 6'da bunun mümkün olduğunu söylemiştik; şimdi nasıl yazıldığına bakalım. 1'den `n`'e kadar sayıların toplamı şöyle düşünülebilir: `n` için toplam, `n` sayısına `n - 1` için toplamı eklemektir; 0 için toplam 0'dır.

ilkOzyineleme.cpp

```cpp
#include <iostream>

using namespace std;

int topla(int n) {
    if (n == 0) {
        return 0;
    }
    return n + topla(n - 1);
}

int main() {
    cout << "topla(1) = " << topla(1) << '\n';
    cout << "topla(5) = " << topla(5) << '\n';
    cout << "topla(100) = " << topla(100) << '\n';
    return 0;
}
```

```
Çıktı
topla(1) = 1
topla(5) = 15
topla(100) = 5050
```

`topla(3)` hesaplanırken fonksiyon `3 + topla(2)` ifadesine gelir ve `topla(2)`'nin cevabını **bekler**. O da `2 + topla(1)` der, o da `1 + topla(0)` der. `topla(0)` kendini çağırmadan `0` döndürür ve zincir geriye doğru çözülür: `1 + 0 = 1`, `2 + 1 = 3`, `3 + 3 = 6`.

Bu işi `for` ile de yazabilirsin (Bölüm 5). Özyinelemeyi öğrenmemizin nedeni, bazı problemlerin (Hanoi kuleleri, ikili arama, ağaçlar, bir klasörün içindeki klasörleri gezmek) özyinelemeyle döngüden çok daha kısa ve doğal yazılmasıdır.

17.2 Temel durum ve özyinelemeli durum

Her özyinelemeli fonksiyonun iki parçası vardır: önce kendini çağırmadan cevap veren bir `if` (temel durum), sonra problemi küçültüp kendini çağıran satır (özyinelemeli durum).

Parça	Görevi	`topla` örneğinde
Temel durum (base case)	Kendini çağırmadan doğrudan cevap verir; zinciri durdurur	`n == 0` ise `0`
Özyinelemeli durum (recursive case)	Problemi küçültüp kendini çağırır	`n + topla(n - 1)`
Yaklaşma	Her çağrı, temel duruma biraz daha yakın olmalıdır	`n` her çağrıda 1 azalır, sonunda 0 olur

Üçüncü satır en çok unutulandır: Temel durumun yazılması yetmez, çağrıların ona **ulaşması** gerekir. `topla(n + 1)` yazarsan `n` temel durumdan uzaklaşır ve zincir hiç bitmez.

Özyinelemeli fonksiyon yazarken en rahat yöntem **güven ilkesidir**: "`topla(n - 1)` doğru çalışıyor" diye kabul et ve yalnızca şunu sor: "`topla(n - 1)`'in cevabından `topla(n)`'in cevabını nasıl çıkarırım?" Çağrının içini kafanda açmaya çalışmak seni yorar; zinciri açıp izlemek ise sınavda iz sürerken gerekir.

> **Sınavda Çıkar**
>
> "Aşağıdaki özyinelemeli fonksiyonda hata nerede?" sorusunun cevabı hemen her zaman şunlardan biridir: **temel durum yok**, **temel duruma hiç ulaşılmıyor** (argüman yanlış yöne gidiyor) ya da **temel durum bazı girdileri kaçırıyor** (`n == 0` yazıp negatif sayı göndermek gibi). Önce temel durumu, sonra argümanın yönünü kontrol et.

17.3 Çağrı yığını ve faktoriyel

Bölüm 6'da her çağrının **yığın çerçevesi** (stack frame) açtığını, çerçevede parametrelerin ve yerel değişkenlerin durduğunu ve fonksiyon bitince çerçevenin silindiğini gördün. Çerçevelerin üst üste dizildiği yapıya **çağrı yığını** (call stack) diyoruz. Özyinelemede aynı fonksiyonun **birçok çerçevesi aynı anda** yığındadır ve her birinin kendi `n`'i, kendi yerel değişkenleri vardır. Faktöriyeli (`n! = n · (n−1)!`, `0! = 1! = 1`) izleyelim. Program, girinti ile derinliği göstererek her aşamayı yazdırıyor:

faktoriyellz.cpp

```cpp
#include <iostream>
#include <string>

using namespace std;

int derinlik{0};                    // çıktıdaki girintiyi ayarlamak için

void girinti() {
    cout << string(2 * derinlik, ' ');   // 2*derinlik tane boşluk
}

long long faktoriyel(int n) {
    long long sonuc{0};

    girinti();
    cout << "faktoriyel(" << n << ") başladı\n";
    if (n <= 1) {
        girinti();
        cout << "faktoriyel(" << n << "): temel durum, 1 döndürüyor\n";
        return 1;
    }
    derinlik++;
    sonuc = n * faktoriyel(n - 1);
    derinlik--;
    girinti();
    cout << "faktoriyel(" << n << "): sonuç = " << sonuc << '\n';
    return sonuc;
}

int main() {
    long long sonuc{faktoriyel(4)};
    cout << "Sonuç: " << sonuc << '\n';
    return 0;
}
```

Çıktı

```
faktoriyel(4) başladı
  faktoriyel(3) başladı
    faktoriyel(2) başladı
      faktoriyel(1) başladı
      faktoriyel(1): temel durum, 1 döndürüyor
    faktoriyel(2): sonuç = 2
  faktoriyel(3): sonuç = 6
faktoriyel(4): sonuç = 24
Sonuç: 24
```

Girinti her çağrının bir kat içeride başladığını gösterir. Aynı olayın yığındaki karşılığı (`f(4)`: `faktoriyel`'in `n = 4` olan çerçevesi; tablodaki ilk çerçeve en üsttedir, yani o an çalışandır):

Adım	Olay	Yığın (üstten alta)
1	`main`, `faktoriyel(4)`'ü çağırır	`f(4)` (n=4, sonuc=0) / `main`
2	`f(4)`, `faktoriyel(3)`'ü çağırır ve bekler	`f(3)` / `f(4)` / `main`
3	`f(3)`, `faktoriyel(2)`'yi çağırır	`f(2)` / `f(3)` / `f(4)` / `main`
4	`f(2)`, `faktoriyel(1)`'i çağırır	`f(1)` / `f(2)` / `f(3)` / `f(4)` / `main`
5	`f(1)` temel durum: `1` döndürür, çerçevesi silinir	`f(2)` / `f(3)` / `f(4)` / `main`
6	`f(2)`: `sonuc = 2 * 1 = 2` döndürür, silinir	`f(3)` / `f(4)` / `main`
7	`f(3)`: `sonuc = 3 * 2 = 6` döndürür, silinir	`f(4)` / `main`
8	`f(4)`: `sonuc = 4 * 6 = 24` döndürür, silinir	`main`

Üç şeye dikkat et:

- **Gidişte** (adım 1–4) hiçbir çarpma yapılmaz; çağrılar açılır ve cevap beklenir. Çarpmalar **dönüşte** (5–8) yapılır, çerçeveler kapanırken.
- Dört çerçevede dört ayrı `n` ve dört ayrı `sonuc` vardır; hiçbiri ötekini bozmaz. `f(2)`'nin `n`'ini değiştirmesi `f(4)`'ün `n`'ini değiştirmez; çünkü parametreler **değer ile geçirilir** (pass by value), her çağrı kendi kopyasıyla çalışır.
- Yığının en derin hâlinde `faktoriyel`'in 4 çerçevesi vardır (`main` ile 5). Yığın belleği çağrı derinliği kadar büyür.

Faktöriyel hızlı büyür: `long long` ile `faktoriyel(20)` sığar, `faktoriyel(21)` ise işaretli tamsayı taşmasıdır ve tanımsız davranıştır (Bölüm 2). `int` ile ise `faktoriyel(13)` zaten taşar (`12!` hâlâ sığar).

> **Sınavda Çıkar**
>
> "Yığında aynı anda kaç çerçeve bulunur?" sorusunun cevabı **özyineleme derinliğidir**. Temel durumu `n <= 1` olan `faktoriyel(5)` için derinlik 5'tir (`n = 5, 4, 3, 2, 1`); soru `main` dahil mi diyorsa 6. Temel durumu `n == 0` olsaydı derinlik 6 olurdu (`n = 0` da bir çerçeve açar). Sayarken temel durumun çerçevesini **unutma**.

17.4 Sayılarla özyineleme

Üs alma: us(a, n) ve hızlı üs

$a^n = a \cdot a^{n-1}$ ve $a^0 = 1$. Bu doğrudan `us(a, n)` fonksiyonudur. **Hızlı üs** ise daha akıllıdır: `y = a^(n / 2)` olsun, burada `n / 2` C++'taki gibi **tamsayı bölmesidir** (`n = 5` için `2`). `n` çiftse $a^n = y \cdot y$, tekse $a^n = a \cdot y \cdot y$ olur (örneğin $a^5 = a \cdot a^2 \cdot a^2$). Yani `n` her çağrıda **yarıya** iner. İkisinde de kaç çağrı yapıldığını global `cagri` sayacı ile sayalım:

usAlma.cpp

```
#include <iostream>

using namespace std;

int cagri{0};

long long us(long long a, int n) {
    cagri++;
    if (n == 0) {
        return 1;
    }
    return a * us(a, n - 1);
}
```

```cpp
long long hizliUs(long long a, int n) {
    long long yarim{0};

    cagri++;
    if (n == 0) {
        return 1;
    }
    yarim = hizliUs(a, n / 2);          // a^(n / 2) bir kez hesaplanır
    if (n % 2 == 0) {
        return yarim * yarim;
    }
    return yarim * yarim * a;
}

int main() {
    cout << "us(3, 20) = " << us(3, 20) << ", çağrı: " << cagri << '\n';
    cagri = 0;
    cout << "hizliUs(3, 20) = " << hizliUs(3, 20) << ", çağrı: " << cagri << '\n';
    return 0;
}
```

Çıktı

```
us(3, 20) = 3486784401, çağrı: 21
hizliUs(3, 20) = 3486784401, çağrı: 6
```

`hizliUs`'ta `n` şöyle iner: 20 → 10 → 5 → 2 → 1 → 0. `n = 5` ve `n = 1` tek olduğundan bu iki çağrı fazladan bir `a` ile çarpar. `us`'ta `n + 1` çağrı gerekirken `hizliUs`'ta yaklaşık $\log_2 n$ çağrı yeter (burada 6); `n = 1 000 000` için fark 1 000 001 çağrıya karşı yaklaşık 21 çağrıdır.

> **Sık Yapılan Hata: Aynı özyinelemeli çağrıyı iki kez yazmak**
>
> `return hizliUs(a, n / 2) * hizliUs(a, n / 2);` yazarsan sonuç doğru çıkar ama hız kazancı **yok olur**: Her çağrı artık iki çağrı açar ve toplam çağrı sayısı yine `n` ile orantılı olur. Yarım sonucu bir değişkene (`yarim`) koy ve **bir kez** hesapla.

Basamak toplamı ve EBOB

Bölüm 5'te döngüyle yazdığın iki algoritma, özyinelemeyle çok kısalır:

sayilarlaOzyineleme.cpp

```cpp
#include <iostream>

using namespace std;

int basamakToplami(int n) {
    if (n < 10) {
        return n;
    }
    return n % 10 + basamakToplami(n / 10);
}

int ebob(int a, int b) {
    if (b == 0) {
        return a;
    }
    return ebob(b, a % b);
}
```

```
int main() {
    cout << "basamakToplami(2749) = " << basamakToplami(2749) << '\n';
    cout << "ebob(48, 18) = " << ebob(48, 18) << '\n';
    return 0;
}
```

Çıktı

```
basamakToplami(2749) = 22
ebob(48, 18) = 6
```

- **`basamakToplami`**: Son basamak `n % 10`, geri kalanı `n / 10`'dur. `basamakToplami(2749)` = `9 + basamakToplami(274)` = `9 + 4 + basamakToplami(27)` = `9 + 4 + 7 + basamakToplami(2)`. `2` tek basamaklı olduğundan temel durumdur: `9 + 4 + 7 + 2 = 22`. Fonksiyon `n >= 0` için yazıldı; negatif sayıda `n % 10` negatif çıkar (C++'ta kalanın işareti bölünenin işaretidir).
- **`ebob`**: Öklid algoritması, Bölüm 5'teki döngüyle aynıdır: `(48, 18)` → `(18, 12)` → `(12, 6)` → `(6, 0)`. Döngüde `x` ve `y` değişkenlerine atama yapıyordun; burada yeni çağrının **argümanları** olarak geçer. `b` sıfır olunca cevap `a`'dır. (C++17'den beri `<numeric>` başlığında hazır `gcd` var; ama sınavda Öklid'in iki satırını yazman beklenir.)

17.5 Fibonacci

Fibonacci dizisi `0, 1, 1, 2, 3, 5, 8, ...`: `fib(0) = 0`, `fib(1) = 1`, diğerleri iki öncekinin toplamı. Tanım zaten özyinelemelidir ve çevirmesi çok kolaydır; ama bu kez fonksiyon **iki kez** kendini çağırır.

fibonacci.cpp

```
#include <iostream>
#include <vector>

using namespace std;

int cagri{0};

int fib(int n) {
    cagri++;
    if (n < 2) {
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}

int main() {
    vector<int> degerler{5, 10, 20};

    for (int n : degerler) {
        cagri = 0;
        int sonuc{fib(n)};
        cout << "fib(" << n << ") = " << sonuc
             << ", çağrı sayısı: " << cagri << '\n';
    }
    return 0;
}
```

Çıktı

```
fib(5) = 5, çağrı sayısı: 15
fib(10) = 55, çağrı sayısı: 177
fib(20) = 6765, çağrı sayısı: 21891
```

`fib(20)` için 21 891 çağrı yapıldı; oysa cevap yalnızca 20 adımla bulunabilirdi. Neden, `fib(5)`'in **çağrı ağacında** (call tree) görünür:

```
                        fib(5)
                 /                \
             fib(4)                fib(3)
            /      \               /     \
        fib(3)    fib(2)       fib(2)   fib(1)
        /    \     /   \        /   \
    fib(2) fib(1) fib(1) fib(0) fib(1) fib(0)
     /   \
 fib(1) fib(0)
```

`fib(3)` iki kez, `fib(2)` üç kez, `fib(1)` beş kez hesaplanıyor. Aynı işler defalarca tekrarlanıyor. Toplam 15 çağrı vardır (programın `fib(5)` satırında da 15 çıktı). Çağrı sayısı `C(n)` için `C(0) = C(1) = 1` ve `C(n) = 1 + C(n−1) + C(n−2)` kuralı geçerlidir; yani `C(2) = 3`, `C(3) = 5`, `C(4) = 9`, `C(5) = 15`. Formül olarak `C(n) = 2 · fib(n+1) − 1`.

`n` bir artınca çağrı sayısı yaklaşık 1,6 katına çıkar (**üstel** büyüme). `fib(30)` 2 692 537 çağrı, `fib(40)` 331 milyondan fazla çağrı ister; her 10 adımda süre yaklaşık 120 katına çıkar. `fib(50)` için 40 milyar çağrı gerekir, bu da bilgisayarına göre dakikalar ya da saatler demektir. Bu yüzden kitapta `fib(40)` ve üstünü çalıştırmıyoruz.

Sınavda "`fib(n)` toplam kaç kez çağrılır?" sorusu sık gelir. Formülü ezberleme, kuralla küçük `n` için tablo çıkar: **1, 1, 3, 5, 9, 15, 25** (`n = 0...6`). İlk çağrı da sayılır. "Kaç toplama yapılır?" diye sorarsa: toplama yalnız temel durum olmayan çağrılarda yapılır, `fib(4)` için `(9 − 1) / 2 = 4`.

Bellekleme (memoization)

Çözüm basit: bir kez hesapladığını bir **tabloda sakla**, tekrar gerekirse hesaplamak yerine tablodan oku. Buna **bellekleme** (memoization) denir. C++'ta tablo için `vector` doğal seçimdir (`vector`'ü Bölüm 7'de gördün). `-1` değeri "henüz hesaplanmadı" demektir; `vector<long long> hafiza(51, -1);` 51 elemanı `-1` ile başlatır.

Burada kritik bir ayrıntı var: tablo **referansla** geçirilmelidir (`vector<long long>& hafiza`; referans ile geçirme Bölüm 9'da). Değer ile geçirirsen her çağrı tablonun kendi kopyasını yazar ve yazdığı değerler çağıran fonksiyona hiç ulaşmaz. İkisini yan yana koyalım:

fibHafiza.cpp

```cpp
#include <iostream>
#include <map>
#include <vector>

using namespace std;

long long fibHafiza(int n, vector<long long>& hafiza, int& cagri) {
    cagri++;
    if (n < 2) {
        return n;
    }
    if (hafiza[n] != -1) {              // daha önce hesaplandı
        return hafiza[n];
    }
    hafiza[n] = fibHafiza(n - 1, hafiza, cagri)
              + fibHafiza(n - 2, hafiza, cagri);
    return hafiza[n];
}

long long fibKopya(int n, vector<long long> hafiza, int& cagri) {
    cagri++;                            // hafiza: BU ÇAĞRININ kopyası
    if (n < 2) {
```

```
        return n;
    }
    if (hafiza[n] != -1) {
        return hafiza[n];
    }
    hafiza[n] = fibKopya(n - 1, hafiza, cagri)
              + fibKopya(n - 2, hafiza, cagri);
    return hafiza[n];
}

long long fibMap(int n, map<int, long long>& hafiza) {
    if (n < 2) {
        return n;
    }
    auto it = hafiza.find(n);            // tabloda var mı?
    if (it != hafiza.end()) {
        return it->second;
    }
    long long sonuc{fibMap(n - 1, hafiza) + fibMap(n - 2, hafiza)};
    hafiza[n] = sonuc;
    return sonuc;
}

int main() {
    int cagri{0};
    vector<long long> tablo(51, -1);

    cout << "fib(50) = " << fibHafiza(50, tablo, cagri);
    cout << ", çağrı sayısı: " << cagri << '\n';

    cagri = 0;
    vector<long long> tablo20(21, -1);
    fibHafiza(20, tablo20, cagri);
    cout << "referansla  fib(20): " << cagri << " çağrı\n";

    cagri = 0;
    vector<long long> kopya20(21, -1);
    fibKopya(20, kopya20, cagri);
    cout << "değer ile   fib(20): " << cagri << " çağrı\n";

    map<int, long long> hafizaMap;
    cout << "map ile     fib(50) = " << fibMap(50, hafizaMap) << '\n';
    cout << "map tablosunda " << hafizaMap.size() << " değer var\n";
    return 0;
}
```

Çıktı

```
fib(50) = 12586269025, çağrı sayısı: 99
referansla  fib(20): 39 çağrı
değer ile   fib(20): 21891 çağrı
map ile     fib(50) = 12586269025
map tablosunda 49 değer var
```

`fib(50)` 40 milyar çağrı isterken burada 99 çağrı yeterli: `fibHafiza(n-1)` zinciri sonuna kadar iner, dönüşte tabloyu doldurur; `fibHafiza(n-2)` ise artık hazır cevabı okur. Genel olarak `fibHafiza(n)` için `2n - 1` çağrı yapılır. Son iki satır tuzağı gösteriyor: `fibKopya`, belleklemeyi yapıyormuş gibi görünür ama tablo her çağrıda kopyalandığı için 21 891 çağrı yapar, yani düz `fib` kadar yavaştır; üstüne her çağrıda tabloyu kopyalama maliyeti de ekler.

> **Sık Yapılan Hata: Bellekleme tablosunu değer ile geçirmek**
>
> Fonksiyonun parametresi `vector<long long> hafiza` (referanssız) ise her çağrı tablonun **kendi kopyasıyla** çalışır. Çağrının doldurduğu değerler çağıranın tablosuna geçmez, bellekleme hiç çalışmaz. Üstelik kod hatasız derlenir ve doğru sonucu verir; yalnızca yavaştır. Tabloyu `&` ile geçir (ya da global/üye değişken yap).

Anahtarlar `0...n` aralığında sıkışık değilse tablo için `vector` yerine `map` (Bölüm 15) kullanabilirsin: yalnız hesaplanan değerler saklanır. Programdaki `fibMap` bunu yapar; son iki satır sonucu ve tabloda kaç değer olduğunu yazar. Tabloda `n = 2...50` için 49 değer saklanır (`0` ve `1` temel durumdur, tabloya girmez). `find` ile bakıp `it->second` ile okuman, anahtar yoksa tabloya boş kayıt ekleyen `hafiza[n]` kullanımından daha temizdir.

Bölüm 5'teki **döngülü** Fibonacci de `n` adımda biter ve tablo gerektirmez; bellekleme ise özyinelemenin okunaklı yapısını korur. "Aynı alt problem tekrar tekrar çözülüyor" gördüğün her yerde bellekleme akla gelmelidir.

> **İpucu: Çağrı ağacını kâğıda çiz**
>
> Sınavda iki çağrılı bir fonksiyonun çıktısını ya da çağrı sayısını sorarlarsa en güvenli yol ağacı çizmektir. Her düğüme argümanı yaz; yapraklar temel durumlardır. Çıktı yazdırılıyorsa ağacı **soldan sağa, derinlemesine** gez.

17.6 vector ve string ile özyineleme

Bir `vector` üzerinde özyinelemede "daha küçük problem", dizinin **daha kısa hâlidir**. `vector`'ü kopyalamazsın (yavaş olurdu); onu `const` referansla geçirir, yalnızca "ilk `n` eleman" demek için `n`'i azaltırsın:

diziOzyineleme.cpp

```cpp
#include <iostream>
#include <vector>

using namespace std;

int toplam(const vector<int>& v, int n) {     // ilk n elemanın toplamı
    if (n == 0) {
        return 0;
    }
    return v[n - 1] + toplam(v, n - 1);
}

int enBuyuk(const vector<int>& v, int n) {    // n >= 1 olmalı
    if (n == 1) {
        return v[0];
    }
    int diger{enBuyuk(v, n - 1)};
    return (v[n - 1] > diger) ? v[n - 1] : diger;
}

int main() {
    vector<int> v{4, 9, 2, 7, 5};
    int n{static_cast<int>(v.size())};        // size() işaretsizdir; int'e çevir

    cout << "toplam = " << toplam(v, n) << '\n';
    cout << "enBuyuk = " << enBuyuk(v, n) << '\n';
    return 0;
}
```

Çıktı

```
toplam = 27
enBuyuk = 9
```

- `toplam`: Son elemanı ayır, geri kalanın toplamını özyinelemeyle bul. Temel durum **boş dizi** (`n == 0`) ve toplamı 0'dır.
- `enBuyuk`: Boş dizinin en büyüğü olmadığı için temel durum **tek elemandır** (`n == 1`). `n >= 1` olmayan çağrı temel duruma ulaşamaz.
- `v[n - 1]` köşeli parantezle erişimdir ve sınır denetimi yapmaz; `n`'in `v.size()`'ı aşmadığından sen emin olmalısın (Bölüm 7).

`string` için de aynı mantık geçerli. İki klasik iş: bir `string`'i ters çevirmek ve **palindrom** (tersi kendisine eşit sözcük, ör. "kayak") denetlemek:

stringOzyineleme.cpp

```cpp
#include <iostream>
#include <string>
#include <vector>

using namespace std;

string tersCevir(const string& s) {
    if (s.size() <= 1) {
        return s;                              // "" ve tek harf: kendisi
    }
    return tersCevir(s.substr(1)) + s[0];      // ilk harf sona gider
}

bool palindromMu(const string& s, int bas, int son) {
    if (bas >= son) {
        return true;
    }
    if (s[bas] != s[son]) {
        return false;
    }
    return palindromMu(s, bas + 1, son - 1);
}

int main() {
    string kelime{"merhaba"};

    cout << kelime << " tersi: " << tersCevir(kelime) << '\n';

    vector<string> kelimeler{"kayak", "araba", "radar"};
    for (const string& k : kelimeler) {
        int son{static_cast<int>(k.size()) - 1};

        cout << k << ": " << (palindromMu(k, 0, son) ? "palindrom" : "değil")
             << '\n';
    }
    return 0;
}
```

Çıktı

```
merhaba tersi: abahrem
kayak: palindrom
araba: değil
radar: palindrom
```

- `tersCevir("abc")` = `tersCevir("bc") + 'a'` = `(tersCevir("c") + 'b') + 'a'` = `"c" + 'b' + 'a'` = `"cba"`. Temel durum boş ya da tek karakterli `string`'dir. `s.substr(1)`, `s`'in 1. karakterinden sonuna kadar olan yeni bir `string` döndürür (Bölüm 8). Her çağrıda kopya üretildiği için kısa sözcüklerde sorun olmaz, uzun metinlerde yavaştır.
- `palindromMu`: **İki ucu** karşılaştırır, içeri doğru daralır. İki temel durum var: uçlar karşılaşırsa (`bas >= son`) hepsi eşleşmiştir, yani palindromdur; uçlardaki karakterler farklıysa palindrom değildir. Uzunluk tek de olsa çift de olsa `bas >= son` yakalar.
- `string` bayt dizisidir (Bölüm 8): `"ş"` 2 bayttır. Bu yüzden `tersCevir("şeker")` gibi Türkçe harfli sözcüklerde harfin iki baytı da yer değiştirir ve sonuç bozuk görünür. Bu örnekler ASCII harflerle çalışır.

17.7 Hanoi kuleleri

Üç çubuk (A, B, C) ve A'da üst üste dizili, küçükten büyüğe `n` disk var. Amaç, tüm diskleri **C'ye** taşımak. Kurallar: bir seferde yalnızca bir disk taşınır ve **büyük disk küçüğün üstüne konamaz**. Döngüyle yazması çok zordur; özyinelemeyle ise üç satırdır. Fikir: `n` diski A'dan C'ye taşımak için (B yedek)

1. üstteki `n - 1` diski A'dan B'ye taşı (C yedek),
2. en büyük diski A'dan C'ye taşı,
3. `n - 1` diski B'den C'ye taşı (A yedek).

Temel durum diskin kalmamasıdır (`n == 0`: yapılacak iş yok).

hanoi.cpp

```cpp
#include <iomanip>
#include <iostream>
#include <vector>

using namespace std;

void hanoi(int n, char kaynak, char hedef, char yedek) {
    if (n == 0) {
        return;
    }
    hanoi(n - 1, kaynak, yedek, hedef);
    cout << "Disk " << n << ": " << kaynak << " -> " << hedef << '\n';
    hanoi(n - 1, yedek, hedef, kaynak);
}

long long hamleSayisi(int n) {
    if (n == 0) {
        return 0;
    }
    return 2 * hamleSayisi(n - 1) + 1;
}

int main() {
    vector<int> olculer{1, 2, 3, 4, 10, 20, 30};

    hanoi(3, 'A', 'C', 'B');
    for (int n : olculer) {
```

```
        cout << "n = " << setw(2) << n << ": " << hamleSayisi(n) << " hamle\n";
    }
    return 0;
}
```

Çıktı

```
Disk 1: A -> C
Disk 2: A -> B
Disk 1: C -> B
Disk 3: A -> C
Disk 1: B -> A
Disk 2: B -> C
Disk 1: A -> C
n =  1: 1 hamle
n =  2: 3 hamle
n =  3: 7 hamle
n =  4: 15 hamle
n = 10: 1023 hamle
n = 20: 1048575 hamle
n = 30: 1073741823 hamle
```

Hamle sayısı `H(n)` için `H(0) = 0` ve `H(n) = 2 · H(n−1) + 1` yazılır: iki kez `n − 1` diski taşıyorsun, arada bir hamle var. Çözüm **`H(n) = 2ⁿ − 1`**: 1, 3, 7, 15, 31, … Programdaki tablo da bunu doğruluyor (`2¹⁰ − 1 = 1023`, `2²⁰ − 1 = 1 048 575`). Her disk hamle sayısını yaklaşık **iki katına** çıkarır; 64 diskin hamle sayısı `2⁶⁴ − 1`, yaklaşık 1,8 · 10¹⁹'dur. Saniyede bir hamle yapsan yaklaşık 585 milyar yıl sürer. Özyinelemenin kısa yazılması, işin hızlı bitmesi demek değildir.

17.8 Özyinelemeli ikili arama

Bölüm 15'te sıralı bir `vector`'de **ikili aramanın** (binary search) fikrini gördün: ortaya bak; bulduysan bitti, bulamadıysan **aynı aramayı yarım dizide** yinele. Bu fikir özyinelemeye çok uygundur. Aşağıdaki program her adımda aralığı yazdırır:

ikiliAraOzyineleme.cpp

```
#include <iostream>
#include <vector>

using namespace std;

int ikiliAra(const vector<int>& v, int alt, int ust, int aranan) {
    if (alt > ust) {
        return -1;
    }
    int orta{alt + (ust - alt) / 2};
    cout << "  alt=" << alt << " ust=" << ust << " orta=" << orta
         << " v[orta]=" << v[orta] << '\n';
    if (v[orta] == aranan) {
        return orta;
    }
    if (v[orta] < aranan) {
        return ikiliAra(v, orta + 1, ust, aranan);
    }
    return ikiliAra(v, alt, orta - 1, aranan);
}

int main() {
    vector<int> v{3, 7, 12, 19, 25, 31, 42, 56, 68, 77};

    cout << "31 aranıyor:\n";
```

```
    int sonuc{ikiliAra(v, 0, 9, 31)};
    cout << "  sonuç: " << sonuc << '\n';
    cout << "10 aranıyor:\n";
    sonuc = ikiliAra(v, 0, 9, 10);
    cout << "  sonuç: " << sonuc << '\n';
    return 0;
}
```

Çıktı

```
31 aranıyor:
  alt=0 ust=9 orta=4 v[orta]=25
  alt=5 ust=9 orta=7 v[orta]=56
  alt=5 ust=6 orta=5 v[orta]=31
  sonuç: 5
10 aranıyor:
  alt=0 ust=9 orta=4 v[orta]=25
  alt=0 ust=3 orta=1 v[orta]=7
  alt=2 ust=3 orta=2 v[orta]=12
  sonuç: -1
```

`31` için üç satır, aralığın her adımda yarıya indiğini gösterir. Orta noktayı `alt + (ust - alt) / 2` diye hesapladık: `(alt + ust) / 2` ile aynı sonucu verir, ama çok büyük indekslerde toplamın `int` sınırını aşma riskini ortadan kaldırır. `alt` ve `ust` sınırlarının güncellenmesi, burada yeni çağrının **argümanlarına** dönüşür (`orta + 1, ust` ya da `alt, orta - 1`). Temel durum `alt > ust` olduğu andır (aralık boşaldı, `-1` döner); bulunca `return orta;` de temel durumdur. Aynı aramayı döngüyle yazsan sabit bellek kullanırdın; özyineleme ise her adımda bir çerçeve açar: en çok $\lfloor \log_2 n \rfloor + 2$ çerçeve.

Özyinelemeli çağrı `return` ile **hemen sonuç olarak** döner, çağrıdan sonra yapılacak başka iş yoktur. Bu yapıya **kuyruk özyinelemesi** diyoruz (bölümün sonunda).

> **Sık Yapılan Hata: Özyinelemeli çağrının sonucunu return etmeyi unutmak**
>
> `ikiliAra(v, orta + 1, ust, aranan);` satırını `return` **olmadan** yazarsan çağrı yapılır ama sonuç atılır; akış fonksiyonun sonuna düşer. Değer döndürmeden biten bir `int` fonksiyon **tanımsız davranıştır** (sonucu kullanmasan bile program beklenmedik davranabilir); `g++ -Wall` ile `control reaches end of non-void function` uyarısını verir; bu uyarıyı **hata gibi** oku. Kural: Sonuç döndüren fonksiyonda özyinelemeli çağrı `return ikiliAra(...)` biçiminde yazılır ya da değeri başka bir ifadede kullanılır.

17.9 Yazdırmayı çağrıdan önce mi sonra mı yapmalı

Özyinelemeli fonksiyonda `cout`'u çağrıdan önce ya da sonra yazmak **çıktıyı tamamen değiştirir**. Çağrıdan önceki satırlar **gidişte** (yığın dolarken), sonraki satırlar **dönüşte** (yığın boşalırken, ters sırayla) çalışır:

onceSonra.cpp

```
#include <iostream>

using namespace std;

void once(int n) {                  // önce yaz, sonra çağır
    if (n == 0) {
        return;
    }
    cout << n << ' ';
    once(n - 1);
}

void sonra(int n) {                 // önce çağır, sonra yaz
```

```
    if (n == 0) {
        return;
    }
    sonra(n - 1);
    cout << n << ' ';
}

int main() {
    cout << "once(3):   ";
    once(3);
    cout << "\nsonra(3):  ";
    sonra(3);
    cout << '\n';
    return 0;
}
```

Çıktı

```
once(3):   3 2 1
sonra(3):  1 2 3
```

Yazdırma yeri	Ne zaman çalışır?	`n = 3` için çıktı
Çağrıdan **önce**	Giderken: `n` büyükten küçüğe	`3 2 1`
Çağrıdan **sonra**	Dönerken: en son açılan çerçeve ilk kapanır	`1 2 3`

`sonra(3)`'ü yığında izle: `sonra(3)` → `sonra(2)` → `sonra(1)` → `sonra(0)` temel durumdur ve döner. Şimdi çerçeveler kapanır: `sonra(1)` `1`'i yazar, `sonra(2)` `2`'yi, `sonra(3)` `3`'ü. Yığın "son giren ilk çıkar" çalıştığı için özyineleme bir şeyi **tersine çevirmenin** doğal yoludur; `tersCevir` de dönüşte bu yüzden harfleri ters sırada toplar.

> **Sınavda Çıkar**
>
> En klasik özyineleme sorusu budur: `cout` çağrıdan önce mi sonra mı? Yöntem: her çağrıyı kendi satırına aç. Çağrıdan **önceki** yazdırmaları aşağı inerken, **sonraki** yazdırmaları yukarı çıkarken yaz. `f(n-1)` iki kez yazılmışsa ikisini ayrı ayrı aç ve soldan sağa oku. Aşağı inerken yazdırdıklarını, çıkarken yazdıklarına karıştırma.

17.10 Temel durumu unutmak: yığın taşması

Temel durum yoksa zincir bitmez, ama bellek sonsuz değildir. Her çağrı bir çerçeve açar ve **çağrı yığını sınırlıdır** (genellikle birkaç MB: Windows'ta 1–2 MB, Linux ve macOS'ta 8 MB). Çerçeveler yığını doldurunca program **yığın taşması** (stack overflow) ile çöker. Aşağıdaki program kasıtlı olarak hatalıdır; çıktı karışmasın diye yalnızca ilk üç çağrıda yazdırıyor. Program çöktüğünde ara belleğe alınmış çıktı kaybolabilir; mesajların ekrana ulaşması için `endl` kullandık (farkını Bölüm 3'te gördün):

sonsuz.cpp

```cpp
#include <iostream>

using namespace std;

int sayac{0};

int sonsuzToplam(int n) {
    sayac++;
    if (sayac <= 3) {
        cout << "çağrı " << sayac << ": n = " << n << endl;
    }
    return n + sonsuzToplam(n - 1);        // temel durum yok!
}

int main() {
    cout << "Başlıyor" << endl;
    cout << sonsuzToplam(5) << endl;
    return 0;
}
```

Derleyici uyarısı – g++ -Wall -Wextra

```
sonsuz.cpp: In function 'int sonsuzToplam(int)':
sonsuz.cpp:7:5: warning: infinite recursion detected [-Winfinite-recursion]
    7 | int sonsuzToplam(int n) {
      |     ^~~~~~~~~~~~
sonsuz.cpp:12:28: note: recursive call
   12 |     return n + sonsuzToplam(n - 1);        // temel durum yok!
      |                ~~~~~~~~~~~~^~~~~~
```

Çıktı – program çöker (Segmentation fault)

```
Başlıyor
çağrı 1: n = 5
çağrı 2: n = 4
çağrı 3: n = 3
```

g++ bu hatayı `-Wall` ile (g++ 12'den beri) **derlerken** yakalayıp `infinite recursion` uyarısı verir; her zaman yakalayamaz, uyarıya güvenme. Program ilk üç çağrıdan sonra sessizce çerçeve açmaya devam eder, yığın dolar ve çöker. Linux ve macOS terminali `Segmentation fault` benzeri bir mesaj yazar (zsh'de `zsh: segmentation fault`); Windows'ta program çoğu zaman hiçbir şey yazmadan kapanır (arka planda "stack overflow" hatası). Bu, bilerek gösterebildiğimiz, her çalıştırmada aynı biçimde ortaya çıkan bir çökmedir.

Bölüm 14'te `try`/`catch` ile istisna yakalamayı gördün. **Yığın taşması bir C++ istisnası değildir:** `catch (...)` onu yakalayamaz, çünkü program yalnızca bir fonksiyon çağrısından değil, işletim sisteminin verdiği yığın bölgesinden taşmıştır. Bu hatadan kurtulmanın tek yolu, hatayı hiç yapmamaktır.

İkinci tür hata temel durum **var ama ulaşılamıyor**: yukarıdaki `topla` fonksiyonunu `topla(-1)` ile çağırmak gibi.

> **● Sık Yapılan Hata: Temel durumu == ile yazmak**
>
> `topla(-1)` çağrısı `n == 0` koşuluna hiç ulaşamaz: `-1`, `-2`, `-3` diye gider ve yığın taşar. Girdinin negatif olabileceği her yerde temel durumu `n <= 0` yaz (ya da girdiyi önceden denetle). Aynı hata, temel durumu `n == 1` olan bir faktöriyel fonksiyonunda `faktoriyel(0)` çağrıldığında da olur (`0`, `-1`, `-2` diye gider).

17.11 Özyineleme mi, döngü mü?

Her özyinelemeli çözüm döngüye (gerekirse kendi yığınınla birlikte) çevrilebilir. Seçim, hız, bellek ve okunabilirlik içindir:

Ölçüt	Döngü	Özyineleme
Bellek	Sabit (birkaç değişken)	Derinlik kadar çerçeve; çok derinse yığın taşar
Hız	Genellikle biraz daha hızlı	Her çağrının çerçeve açma/kapama maliyeti vardır; `fib` gibi yerlerde tekrar eden hesap üstel yavaşlık yaratır
Okunabilirlik	Doğrusal işlerde (toplam, faktöriyel, dizi gezme) daha doğal	Problem kendine benzeyen parçalara bölünüyorsa (Hanoi, ikili arama, ağaç ve klasör gezme) çok daha kısa ve net
Tipik hata	Sonsuz döngü	Temel durum eksikse yığın taşması

Pratik kural: İş **doğrusal** ise (1'den `n`'e topla, `vector` gez) döngü. Problem aynı türden **alt problemlere** bölünüyorsa (Hanoi, ağaç) ya da çözüm doğal olarak özyinelemeli tanımlanmışsa (Fibonacci, EBOB) özyineleme. Sınavda "özyinelemeyle yaz" dendiyse döngü kullanma.

Derinliğin sınırı pratikte önemlidir: 10 000 derinlikli bir özyineleme (`topla(10000)`) rahat çalışır; `topla(1000000)` ise, kitaptaki gibi `-O0` ile derlenen programda çoğu sistemde yığını taşırır (sınır sisteme ve çerçeve boyutuna bağlıdır). Aynı toplamı döngü sabit bellekle yapar. Bu yüzden milyonlarca adım süren işler döngüyle yazılır.

Kuyruk özyinelemesi (tail recursion)

Özyinelemeli çağrı fonksiyonun **son işi** ise (çağrıdan sonra çarpma, toplama gibi bir şey yapılmıyorsa) buna **kuyruk özyinelemesi** denir. `ebob` ve `ikiliAra` böyledir: `return ebob(b, a % b);`. `faktoriyel` böyle değildir: `n * faktoriyel(n - 1)` ifadesinde çağrıdan **sonra** çarpma yapılır. Sonucu fazladan bir parametrede biriktirirsen (`faktoriyelKuyruk(int n, long long biriken)`; temel durumda `biriken` döner, diğer durumda `return faktoriyelKuyruk(n - 1, n * biriken);` yazılır, ilk çağrı `faktoriyelKuyruk(5, 1)`) faktöriyel de kuyruk özyinelemeli olur.

Kuyruk çağrısında çağıran çerçevenin işi bittiği için derleyici `-O2` gibi optimizasyonlarla çerçeveyi yeniden kullanıp bunu döngüye çevirebilir; g++ bunu çoğu zaman yapar. Ama C++ standardı bunu **garanti etmez** ve kitaptaki `-O0` derlemede çerçeve yine açılır. Sınav için yeter bilgi: tanımı ve `ebob`'un kuyruk özyinelemeli olduğunu bil.

Bölüm Özeti – Bilmen Gerekenler

- **Özyineleme**: fonksiyonun kendini çağırması. Her özyinelemeli fonksiyonda **temel durum** (kendini çağırmadan cevap) ve **özyinelemeli durum** (daha küçük problemle kendini çağırma) vardır; her çağrı temel duruma **yaklaşmalıdır**.
- Her çağrı kendi **yığın çerçevesini** açar; parametreler ve yerel değişkenler çağrıya özeldir. Yığındaki çerçeve sayısı = özyineleme derinliği. Hesaplamalar genellikle **dönüşte** yapılır (`n * faktoriyel(n - 1)`).
- Kalıplar: `topla(n) = n + topla(n-1)`; `basamakToplami(n) = n % 10 + basamakToplami(n / 10)`; `ebob(a, b) = ebob(b, a % b)`, `b == 0` iken `a`; `us(a, n) = a * us(a, n-1)`; **hızlı üs** `n/2` ile `log n` çağrı (yarım sonucu **bir kez** hesapla).
- **Fibonacci** özyinelemeli: iki çağrı, çağrı ağacı, aynı değerler tekrar hesaplanır; çağrı sayısı üstel (`C(n) = 2·fib(n+1) - 1`: 1, 1, 3, 5, 9, 15, 25). **Bellekleme** (`vector` ya da `map` tablosu) bunu `2n - 1` çağrıya indirir; tablo **referansla** geçirilmelidir.
- `vector`/`string`: ilk `n` elemana ya da `bas`/`son` indekslerine indirge; büyük nesneleri `const` referansla geçir. Boş dizi (`n == 0`) temel durumdur; en büyük için `n == 1`. Palindromda iki uç karşılaştırılır.

- **Hanoi**: `n − 1` diski yedeğe, en büyüğü hedefe, `n − 1` diski hedefe. Hamle sayısı $2^n - 1$.
- **Özyinelemeli ikili arama**: temel durum `alt > ust` (`-1`) ve `v[orta] == aranan`; aralık argümanla daralır; çağrıyı `return` et.
- **Çağrıdan önce yazdırma** gidişte (sıra: büyükten küçüğe), **sonra yazdırma** dönüşte (ters sıra) çalışır. İki çağrılı fonksiyonlarda çağrıları sırayla aç.
- Temel durum yok ya da ulaşılamıyor → **yığın taşması** (Linux/macOS'ta `Segmentation fault`, Windows'ta sessiz kapanma); `catch` ile yakalanmaz. `n == 0` yerine `n <= 0` yaz. Döngü: sabit bellek, genelde hızlı; özyineleme: yığın kullanır ama kendine benzeyen problemlerde kısa. **Kuyruk özyinelemesi**: çağrı son iştir (`ebob`).

Sorular

A) Çoktan Seçmeli

1 `n >= 0` için `n!` değerini (`0! = 1`) doğru hesaplayan ve her `n >= 0` için sonlanan fonksiyon aşağıdakilerden hangisidir?

(A) `long long f(int n) { if (n == 1) return 1; return n * f(n - 1); }`
(B) `long long f(int n) { if (n <= 1) return 1; return n * f(n + 1); }`
(C) `long long f(int n) { if (n <= 1) return 1; return n * f(n - 1); }`
(D) `long long f(int n) { if (n <= 1) return n; return n * f(n - 1); }`
(E) `long long f(int n) { if (n == 0) return 0; return n * f(n - 1); }`

2 `int fib(int n) { if (n < 2) return n; return fib(n - 1) + fib(n - 2); }` fonksiyonu `fib(5)` ile çağrılırsa `fib` ilk çağrı dahil toplam kaç kez çağrılır?

(A) 15 (B) 5 (C) 8 (D) 11 (E) 25

3 `int basamakToplami(int n)` fonksiyonunun temel durumu `n < 10` ise `n` döndürmek, diğer durumda `n % 10 + basamakToplami(n / 10)` döndürmektir. `basamakToplami(2749)` çağrısında çağrı yığınında aynı anda **en çok** kaç `basamakToplami` çerçevesi bulunur? (`main`'in çerçevesi sayılmaz.)

(A) 3 (B) 5 (C) 10 (D) 22 (E) 4

4 Üç çubuklu Hanoi kulelerinde 5 diski bir çubuktan diğerine, kurallara uyarak taşımak için **en az** kaç hamle gerekir?

(A) 25 (B) 31 (C) 32 (D) 30 (E) 63

5 Özyineleme ile ilgili aşağıdakilerden hangisi **doğrudur**?

(A) Derleyici özyinelemeli her fonksiyonu mutlaka döngüye çevirir, bu yüzden yığın hiç dolmaz.

(B) Yığın taşması, `try { ... } catch (...) { ... }` ile yakalanıp program kaldığı yerden devam ettirilebilir.

(C) Bellekleme tablosunu `vector<long long> tablo` biçiminde değer ile geçirirsen tekrar eden hesaplar yine de ortadan kalkar.

(D) Bellekleme tablosu fonksiyona referansla geçirilmelidir; yoksa bir çağrının yazdığı değeri diğer çağrılar göremez.

(E) Temel durumu olmayan özyinelemeli bir fonksiyon derlenmez.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>

using namespace std;

void f(int n) {
    if (n < 1) {
        return;
    }
    cout << n << ' ';
    f(n / 2);
    cout << n << ' ';
}

int main() {
    f(13);
    cout << '\n';
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır? (Parametrenin değer ile mi, referansla mı geçtiğine dikkat et.)

```cpp
#include <iostream>

using namespace std;

int a(int n, int sayac) {
    sayac++;
    if (n == 0) {
        return sayac;
    }
    return a(n - 1, sayac);
}

int b(int n, int& sayac) {
    sayac++;
    if (n == 0) {
        return sayac;
    }
    return b(n - 1, sayac);
}

int main() {
    int x{0};
    int y{0};
    int s1{a(3, x)};
    int s2{b(3, y)};

    cout << s1 << ' ' << x << '\n';
    cout << s2 << ' ' << y << '\n';
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```cpp
#include <iostream>

using namespace std;

void g(int n) {
    if (n == 0) {
        cout << '*';
        return;
    }
    g(n - 1);
    cout << n;
    g(n - 1);
}

int main() {
    g(2);
    cout << '\n';
    return 0;
}
```

C) Kod Yazma

9 Klavyeden boşluksuz bir sözcük okuyan `sozcukAnaliz` programını yaz. İki **özyinelemeli** fonksiyon kullan (döngü yok): `string tersCevir(const string& s)` sözcüğün tersini döndürsün; `bool palindromMu(const string& s, int bas, int son)` sözcük palindromsa `true` döndürsün. Sözcük yalnız ASCII harflerden oluşur.

```
Sozcuk: araba
Tersi: abara
Palindrom degil
```

10 Bir merdiven `n` basamaklıdır ve her adımda 1 ya da 2 basamak çıkabilirsin. Tepeye kaç farklı yolla çıkılabileceğini hesaplayan `merdiven` programını yaz. Özyinelemeli `long long yolSayisi(int n, vector<long long>& hafiza)` fonksiyonu **bellekleme** kullansın (`yolSayisi(0) = yolSayisi(1) = 1`). `n` 1 ile 50 arasında olmalıdır; değilse hata mesajı yazıp `1` ile çık.

```
Basamak sayisi: 10
10 basamakli merdiven: 89 yol
```

Çözümler

Cevap anahtarı

1 C	2 A	3 E	4 B	5 D

1 Cevap: C

C'de temel durum `n <= 1` (hem `0!` hem `1!` için `1`) ve çağrı `n - 1` ile temel duruma yaklaşır. A'da temel durum `n == 1`: `n = 0` ile çağırınca `0, -1, -2, ...` diye temel duruma hiç ulaşılmaz, yığın taşar. B'de temel durum var ama argüman `n + 1` ile **temel durumdan uzaklaşır**. D'de temel durum `n` döndürür; `f(0)` yanlışlıkla `0` çıkar (`0! = 1` olmalıydı). E'de temel durum `0` döndürür, bu yüzden her çarpım `0` olur.

2 Cevap: A

Çağrı sayısı `C(n) = 1 + C(n-1) + C(n-2)` ve `C(0) = C(1) = 1`: `C(2) = 3`, `C(3) = 5`, `C(4) = 9`, `C(5) = 9 + 5 + 1 = 15`. B (5) `fib(5)`'in **değeridir**, çağrı sayısı değil. C (8) `fib(6)`'nın değeridir. D (11) bir tahmindir, hiçbir formülle çıkmaz. E (25) `fib(6)`'nın çağrı sayısıdır (bir sonraki terim).

3 Cevap: E

Yığın `n = 2749, 274, 27, 2` çerçeveleriyle en derin hâle ulaşır; `2 < 10` temel durumdur ve kendini çağırmadan döner. Yani 4 çerçeve (basamak sayısı kadar). A (3), temel durum çerçevesini saymayı unutmaktır. B (5) `main`'i de sayar; soru `main`'i dışarıda bırakıyor. C (10) temel durumdaki `10` sayısıyla karıştırmaktır. D (22) fonksiyonun **sonucudur** (`9 + 4 + 7 + 2`), çerçeve sayısı değil.

4 Cevap: B

`H(n) = 2ⁿ - 1` olduğundan `H(5) = 2⁵ - 1 = 31`. Adım adım: `H(1) = 1`, `H(2) = 3`, `H(3) = 7`, `H(4) = 15`, `H(5) = 2·15 + 1 = 31`. A (25) rastgele bir tahmin, C (32) `2⁵`, D (30) `2⁵ - 2`, E (63) ise `2⁶ - 1`'dir (6 disk için); hepsinde üs ya da "−1" hatalıdır.

5 Cevap: D

Doğrusu D: Tablo değer ile geçirilirse her çağrı kendi kopyasına yazar, yazılan değerler çağırana ulaşmaz; bellekleme çalışmaz. A yanlış: Derleyici `-O2` ile **bazı** kuyruk çağrılarını döngüye çevirebilir, ama standart bunu garanti etmez ve `-O0`'da hiç yapılmaz. B yanlış: Yığın taşması C++ istisnası değildir, `catch` onu yakalayamaz; program çöker. C yanlış: Kopyalanan tabloda değerler kaybolur, çağrı sayısı düz `fib` ile aynı kalır (bölümdeki `fibKopya` örneği). E yanlış: Temel durumsuz fonksiyon **derlenir** (g++ en fazla uyarı verir); hata çalışma zamanında yığın taşması olarak çıkar.

6

`f(n)`: önce `n`'i yazar, sonra `f(n / 2)` çağırır, sonra `n`'i yine yazar. `n / 2` tamsayı bölmesidir.

Argümanlar `13 → 6 → 3 → 1 → 0` diye iner. Gidişte `13 6 3 1` yazılır. `f(0)` temel durumdur ve hiçbir şey yazmaz. Dönüşte sırayla `f(1)` `1`, `f(3)` `3`, `f(6)` `6`, `f(13)` `13` yazar. Tuzak: `1`, çağrının hem önünde hem arkasında yazıldığı için iki kez, yan yana (`1 1`) çıkar.

Çıktı

```
13 6 3 1 1 3 6 13
```

7

`a` fonksiyonunda `sayac` **değer ile** geçer: her çağrı kendi kopyasını artırır ve yeni değeri bir sonraki çağrıya argüman olarak verir. `b` fonksiyonunda ise `sayac` **referanstır**: bütün çağrılar `main`'deki aynı değişkeni artırır.

`a(3, x)`: kopyalar sırayla `1, 2, 3, 4` olur (`n = 3, 2, 1, 0`) ve `4` döner; `main`'deki `x` hiç değişmez. Satır: `4 0`. `b(3, y)`: `y` dört kez artırılır, `4` döner ve `y` artık `4`'tür. Satır: `4 4`.

Tuzak: İki fonksiyonun **döndürdüğü** değer aynıdır; fark, çağırandaki değişkenin değişip değişmemesindedir.

Çıktı

```
4 0
4 4
```

8

`g(n)`: `n == 0` ise `*` yazar. Aksi hâlde `g(n-1)`, sonra `n`, sonra yine `g(n-1)`. Alt çağrıları sırayla aç:

- `g(0)` = `*`
- `g(1)` = `g(0)`, `1`, `g(0)` = `*1*`
- `g(2)` = `g(1)`, `2`, `g(1)` = `*1*` `2` `*1*` = `*1*2*1*`

Tuzak: Bu kez temel durum bir şey **yazdırır**; yıldızları saymayı unutma.

Çıktı

```
*1*2*1*
```

9

`tersCevir`'de temel durum boş ya da tek karakterli `string`'dir; diğer durumda ilk karakter atılır, kalanın tersinin sonuna eklenir. `palindromMu` iki ucu karşılaştırarak içeri daralır. Girdi okuması denetlenir.

sozcukAnaliz.cpp

```cpp
#include <iostream>
#include <string>

using namespace std;

string tersCevir(const string& s) {
    if (s.size() <= 1) {
        return s;
    }
    return tersCevir(s.substr(1)) + s[0];
}

bool palindromMu(const string& s, int bas, int son) {
    if (bas >= son) {
        return true;
    }
    if (s[bas] != s[son]) {
        return false;
    }
    return palindromMu(s, bas + 1, son - 1);
}

int main() {
    string sozcuk;

    cout << "Sozcuk: ";
```

```
    if (!(cin >> sozcuk)) {
        cout << "Girdi okunamadi.\n";
        return 1;
    }
    cout << "Tersi: " << tersCevir(sozcuk) << '\n';
    int son{static_cast<int>(sozcuk.size()) - 1};
    cout << (palindromMu(sozcuk, 0, son) ? "Palindrom" : "Palindrom degil");
    cout << '\n';
    return 0;
}
```

Çıktı

```
Sozcuk: araba
Tersi: abara
Palindrom degil
```

10

Son adım ya 1 ya 2 basamaktır: `yolSayisi(n) = yolSayisi(n−1) + yolSayisi(n−2)`. Bu Fibonacci'nin kendisidir ve aynı tekrar eden hesap sorunu vardır; tablo (`hafiza`) **referansla** geçirilir. `0` (zaten tepedesin) ve `1` basamak için tek yol vardır. `hafiza[n] == 0` "henüz hesaplanmadı" demektir, çünkü yol sayısı hiçbir zaman `0` olmaz.

merdiven.cpp

```
#include <iostream>
#include <vector>

using namespace std;

long long yolSayisi(int n, vector<long long>& hafiza) {
    if (n <= 1) {
        return 1;
    }
    if (hafiza[n] != 0) {
        return hafiza[n];
    }
    hafiza[n] = yolSayisi(n - 1, hafiza) + yolSayisi(n - 2, hafiza);
    return hafiza[n];
}

int main() {
    int n{0};

    cout << "Basamak sayisi: ";
    if (!(cin >> n) || n < 1 || n > 50) {
        cout << "Gecersiz basamak sayisi.\n";
        return 1;
    }
    vector<long long> hafiza(n + 1, 0);
    cout << n << " basamakli merdiven: " << yolSayisi(n, hafiza) << " yol\n";
    return 0;
}
```

Çıktı

```
Basamak sayisi: 10
10 basamakli merdiven: 89 yol
```

DENEME SINAVI

Deneme Final Sınavı

Bu deneme sınavı kitabın tamamını kapsar, ama ağırlık ikinci yarıdadır: işaretçiler ve referanslar, dinamik bellek ve `unique_ptr`, sınıflar (yapıcı / yıkıcı sırası), kalıtım ve `virtual`, operatör aşırı yükleme, şablonlar, kopya yapıcı, istisnalar, STL (`map`, `set`, `sort`, lambda), dosyalar ve özyineleme. İlk sekiz bölümden yalnız iki karışık temel soru var; çünkü gerçek final sınavında da temel bilgi, yeni konuların içine gömülü olarak sorulur. Tuzaklar **bilerek** kondu. Hocaların sevdiği şeylere dikkat et: yapıcı ve yıkıcıların çağrı sırası, `virtual` olmayan fonksiyonun taban sınıf türü üzerinden çağrılması, nesne dilimleme, `const` işaretçinin hangi yanı kilitlediği, `map` üzerinde `[]` ile yanlışlıkla eleman oluşturmak, `catch` sırası, yığın çözülmesinde yıkıcıların sırası, `eof` ile döngü kurmak ve özyinelemede yazdırmanın çağrıdan önce mi sonra mı olduğu.

Bölüm	Soru sayısı	Soru başına puan	Toplam
A) Çoktan seçmeli	12	3	36
B) Çıktı tahmini	5	4	20
C) Kod yazma	4	11	44
Toplam	**21**		**100**

Süre: 100 dakika. Bilgisayar, kitap ve not kullanmak yok; sadece kâğıt ve kalem. Bütün sorularda standart C++17 ve `g++` geçerlidir; `int` 4 bayttır. Aksi söylenmedikçe gerekli `#include` satırları ve `using namespace std;` var sayılır. Tanımsız davranış (undefined behavior) soran soru yoktur.

> **İpucu: Önce kâğıtta çöz**
>
> Sınavı gerçek sınav gibi çöz: 100 dakikalık bir sayaç kur ve süre bitene kadar VS Code'u açma, çözümlere bakma. Kabaca **25 dakikayı** testlere, **25 dakikayı** çıktı sorularına, **50 dakikayı** kod yazmaya ayır. Bir soruda takılırsan işaretle ve geç; sona dönersin. Süre bitince kodlarını bilgisayarda derleyip çalıştır, çözümlerle karşılaştır ve kendini puanla.

> **Not: Kendini nasıl puanlarsın?**
>
> - **Çıktı sorularında** çıktının her satırı eşit puan değerindedir. Eksik boşluk, fazladan satır veya `9` yerine `9.0` yazmak o satırı yanlış yapar; gerçek sınavda da böyledir.
> - **Kod sorularında** çözümdeki program tek doğru cevap değildir. Senin kodun aynı işi yapıyorsa tam puan al. Çözümlerin sonunda "hangi adım kaç puan" notu var. Derleme hatası olan her satır için 1 puan, yanlış sonuç veren her mantık hatası için 2–3 puan kır. Elle `new` ile ayırıp `delete` etmeyen ya da sanal yıkıcıyı unutan çözümden her seferinde 1 puan kır.
> - Her cevabın sonunda konunun anlatıldığı bölüm yazıyor. Yanlış yaptığın soruların bölümlerindeki **Sınavda Çıkar** ve **Sık Yapılan Hata** kutularını tekrar oku.

Not: Hangi soru hangi bölümden?

Bölüm	Sorular
1–8 (temel)	Test 1, 2
9 İşaretçiler ve referanslar	Test 3; Çıktı 15
10 Dinamik bellek	Test 4
11 Sınıflar	Test 5; Çıktı 13; Kod 18
12 Kalıtım ve çok biçimlilik	Test 6; Çıktı 13, 14; Kod 19
13 Operatör aşırı yükleme ve şablonlar	Test 7, 8; Kod 18
14 İstisnalar	Test 9; Çıktı 17; Kod 21
15 STL	Test 10; Kod 20
16 Dosyalar	Test 11; Kod 20
17 Özyineleme	Test 12; Çıktı 16; Kod 21

Sorular

A) Çoktan Seçmeli

1 Aşağıdaki kod parçası ekrana ne yazdırır?

```
int a{0}, b{5}, c{2};
if (a++ && b++) {
    c += 10;
}
if (b-- || ++c > 2) {
    c *= 2;
}
cout << a << ' ' << b << ' ' << c << '\n';
```

(A) `1 5 6` (B) `1 4 24` (C) `1 4 4` (D) `2 5 6` (E) `1 4 6`

2 Aşağıdaki kod parçası ekrana ne yazdırır?

```
vector<int> a(3, 7);
vector<int> b{3, 7};
string s{"merhaba"};
s[0] = 'M';
s.erase(s.size() - 2);
cout << a.size() << ' ' << b.size() << ' ' << a[1] + b[0] << ' '
     << s << '\n';
```

(A) `3 2 10 Merha` (B) `2 3 10 Merha` (C) `3 2 14 Merha` (D) `3 2 10 Merhaba` (E) `3 2 10 Merhab`

3 Aşağıdaki kod parçasında **derleme hatasına** yol açan tek satır hangisidir?

```
int a{1}, b{2};
const int* p = &a;
int* const q = &a;
const int* const r = &b;
p = &b;
*q = 5;
*p = 7;
cout << *p << *q << *r << '\n';
```

(A) `p = &b;`
(B) `*q = 5;`
(C) `cout << *p << *q << *r << '\n';`
(D) `*p = 7;`
(E) Hiçbiri; kod sorunsuz derlenir.

4 Aşağıdaki kod parçası ekrana ne yazdırır?

```
auto a = make_unique<int>(5);
auto b = std::move(a);
*b += 3;
cout << (a == nullptr) << ' ' << *b << '\n';
```

(A) `0 5`
(B) `1 8`
(C) `0 8`
(D) `1 5`
(E) Derleme hatası: `unique_ptr` taşınamaz, yalnız kopyalanabilir.

5 Aşağıdaki program ekrana ne yazdırır?

```
class Kutu {
public:
    static int canli;
    Kutu() { canli++; }
    ~Kutu() { canli--; }
};
int Kutu::canli{0};

int main() {
    Kutu a;
    {
        Kutu b;
        Kutu c = b;
        cout << Kutu::canli << ' ';
    }
    cout << Kutu::canli << '\n';
    return 0;
}
```

(A) `3 1` (B) `2 1` (C) `3 2` (D) `2 -1` (E) `2 0`

6 Aşağıdaki sınıflar ve `main` içinde, derleme hatasına yol açan satırlar hangileridir? (Satır numaraları yorumlardadır.)

```
class Hayvan {
public:
    virtual void sesCikar() const = 0;
    virtual ~Hayvan() = default;
};
class Kedi : public Hayvan {
public:
    void sesCikar() const override { cout << "Miyav\n"; }
};
class Kopek : public Hayvan {};

int main() {
    Kedi k;                 // 1
    Kopek d;                // 2
    Hayvan* p = &k;         // 3
    Hayvan& r = k;          // 4
    p->sesCikar();          // 5
    r.sesCikar();           // 6
    return 0;
}
```

(A) Hiçbiri; kod derlenir.
(B) Yalnız 2. satır.
(C) 2. ve 3. satırlar.
(D) 3. ve 4. satırlar.
(E) 5. ve 6. satırlar.

7 Aşağıdaki program ekrana ne yazdırır?

```
class Sayac {
    int d;
public:
    Sayac(int v) : d{v} {}
    Sayac& operator++() { ++d; return *this; }
    Sayac operator++(int) { Sayac eski{*this}; ++d; return eski; }
    int deger() const { return d; }
};

int main() {
    Sayac s{5};
    Sayac t = s++;
    Sayac u = ++s;
    cout << s.deger() << ' ' << t.deger() << ' ' << u.deger() << '\n';
    return 0;
}
```

(A) `7 5 7` (B) `7 6 7` (C) `7 5 6` (D) `6 5 7` (E) `7 7 7`

8 Aşağıdaki `main` içinde, derleme hatasına yol açan satırlar hangileridir? (Satır numaraları yorumlardadır.)

```
template <typename T>
T enBuyuk(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    cout << enBuyuk(3, 7) << '\n';                    // 1
    cout << enBuyuk(2.5, 1.5) << '\n';                // 2
    cout << enBuyuk<double>(3, 4.5) << '\n';          // 3
    cout << enBuyuk(3, 4.5) << '\n';                  // 4
    cout << enBuyuk(string{"a"}, string{"b"}) << '\n'; // 5
    return 0;
}
```

(A) Hiçbiri; kod derlenir.
(B) Yalnız 3. satır.
(C) 3. ve 4. satırlar.
(D) Yalnız 4. satır.
(E) 4. ve 5. satırlar.

9 Aşağıdaki kod parçası ekrana ne yazdırır?

```
try {
    cout << "1 ";
    vector<int> v{1, 2, 3};
    int n = stoi("abc");
    cout << "2 ";
    v.at(10) = n;
    cout << "3 ";
} catch (const out_of_range&) {
    cout << "O ";
} catch (const logic_error&) {
    cout << "L ";
} catch (const exception&) {
    cout << "E ";
}
cout << "bitti\n";
```

(A) `1 2 O bitti` (B) `1 O bitti` (C) `1 L bitti` (D) `1 E bitti` (E) `1 2 3 bitti`

10 Aşağıdaki kod parçası ekrana ne yazdırır?

```
map<string, int> m;
m["elma"] = 3;
m["armut"]++;
if (m["kiraz"] > 0) {
    m["elma"] = 0;
}
m.insert({"elma", 99});
cout << m.size() << ' ' << m["elma"] << ' ' << m.count("muz") << ' '
     << m["armut"] << '\n';
```

(A) `2 3 0 1` (B) `3 99 0 1` (C) `3 3 1 1` (D) `4 3 0 1` (E) `3 3 0 1`

11 `final-sayilar.txt` dosyasının içeriği `3 4 5` ve **satırın sonunda bir Enter (satır sonu)** vardır. Aşağıdaki kod parçası ekrana ne yazdırır?

```
ifstream dosya{"final-sayilar.txt"};
int sayi{0}, adet{0}, toplam{0};
while (!dosya.eof()) {
    dosya >> sayi;
    adet++;
    toplam += sayi;
}
cout << adet << ' ' << toplam << '\n';
```

(A) `3 12` (B) `4 17` (C) `4 12` (D) `3 17` (E) `5 17`

12 Aşağıdaki fonksiyon `fib(6)` ile çağrıldığında, `fib` fonksiyonu **toplam kaç kez** çalışır? (İlk çağrı da sayılır.)

```
int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}
```

(A) 8 (B) 15 (C) 21 (D) 25 (E) 41

B) Çıktı Tahmini

13 Aşağıdaki program ekrana ne yazdırır? Her sınıfın yapıcısı ve yıkıcısı bir iz bırakır; üye nesnelerin ve temel sınıfın sırasını dikkatle izle. Satır sonları kodda `\n` ile verilmiştir.

```cpp
#include <iostream>

using namespace std;

class Motor {
public:
    Motor() { cout << "Motor+ "; }
    ~Motor() { cout << "Motor- "; }
};

class Arac {
public:
    Arac() { cout << "Arac+ "; }
    virtual ~Arac() { cout << "Arac- "; }
};

class Araba : public Arac {
    Motor m;
public:
    Araba() { cout << "Araba+ "; }
    ~Araba() override { cout << "Araba- "; }
};

int main() {
    {
        Motor yedek;
        Araba a;
        cout << "ic\n";
    }
    Arac* p = new Araba;
    cout << '\n';
    delete p;
    cout << "\nson\n";
    return 0;
}
```

14 Aşağıdaki program ekrana ne yazdırır? `virtual` olan ve olmayan fonksiyonlara, referans, işaretçi ve **kopya** arasındaki farka dikkat et. `double` değerler `cout`'un varsayılan biçimiyle yazılır.

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <vector>

using namespace std;

class Sekil {
public:
    virtual string ad() const { return "Sekil"; }
    string tur() const { return "S"; }
    virtual double alan() const { return 0; }
    virtual ~Sekil() = default;
};

class Kare : public Sekil {
    double kenar;
```

```
public:
    Kare(double k) : kenar{k} {}
    string ad() const override { return "Kare"; }
    string tur() const { return "K"; }
    double alan() const override { return kenar * kenar; }
};

void degerle(Sekil s) {
    cout << "deger: " << s.ad() << ' ' << s.alan() << '\n';
}

void referansla(const Sekil& s) {
    cout << "ref: " << s.ad() << ' ' << s.alan() << '\n';
}

int main() {
    Kare k{3};
    Sekil& r = k;
    Sekil* p = &k;
    Sekil kopya = k;
    cout << r.ad() << ' ' << r.tur() << ' ' << k.tur() << '\n';
    cout << p->ad() << ' ' << p->alan() << '\n';
    cout << kopya.ad() << ' ' << kopya.alan() << '\n';
    degerle(k);
    referansla(k);
    vector<Sekil> v;
    v.push_back(k);
    vector<unique_ptr<Sekil>> w;
    w.push_back(make_unique<Kare>(2));
    cout << v[0].ad() << ' ' << w[0]->ad() << ' ' << w[0]->alan() << '\n';
    return 0;
}
```

15 Aşağıdaki program ekrana ne yazdırır? Hangi çağrının asıl değişkeni, hangisinin yalnız kopyasını değiştirdiğini ve `&` ile dönen fonksiyonun ne yaptığını izle.

```
#include <iostream>
#include <vector>

using namespace std;

void artir(int x) {
    x++;
    cout << "ic " << x << '\n';
}

void artirRef(int& x) { x++; }

void takas(int& a, int& b) {
    int t = a;
    a = b;
    b = t;
}

int& enBuyuk(int& a, int& b) { return (a > b) ? a : b; }

int main() {
    int a{4}, b{9};
    artir(a);
    artirRef(b);
    cout << a << ' ' << b << '\n';
    enBuyuk(a, b) = 0;
    cout << a << ' ' << b << '\n';
```

```
    takas(a, b);
    cout << a << ' ' << b << '\n';

    vector<int> v{1, 2, 3};
    int kopyaToplam{0};
    for (int x : v) {
        x *= 2;
        kopyaToplam += x;
    }
    for (auto& x : v) {
        x += 1;
    }
    int& r = v[1];
    r = 50;
    auto c = v[2];
    c = 0;
    int* p = &v[0];
    *p += 10;
    cout << v[0] << ' ' << v[1] << ' ' << v[2] << ' ' << c << ' '
         << kopyaToplam << '\n';
    return 0;
}
```

16 Aşağıdaki program ekrana ne yazdırır? `yaz` fonksiyonunda yazdırmanın özyinelemeli çağrıdan **önce** ve **sonra** yapıldığına dikkat et; `us` için çağrı sayısını da say.

```
#include <iostream>
#include <string>

using namespace std;

int sayac{0};

void yaz(int n) {
    if (n == 0) {
        cout << "dip ";
        return;
    }
    cout << n << ' ';
    yaz(n - 1);
    cout << n << ' ';
}

string ters(const string& s) {
    if (s.empty()) {
        return "";
    }
    return ters(s.substr(1)) + s[0];
}

int us(int a, int n) {
    sayac++;
    if (n == 0) {
        return 1;
    }
    if (n % 2 == 0) {
        int y = us(a, n / 2);
        return y * y;
    }
    return a * us(a, n - 1);
}

int main() {
```

```
    yaz(3);
    cout << '\n';
    cout << ters("abc") << '\n';
    cout << us(2, 10) << ' ' << sayac << '\n';
    return 0;
}
```

17 Aşağıdaki program ekrana ne yazdırır? İstisna fırlatıldığında **yıkıcıların hangi sırayla** çalıştığını ve hangi `catch` bloğunun yakaladığını izle. (`runtime_error`, `logic_error` değildir; ikisi de `exception`'dan türer.)

```
#include <iostream>
#include <stdexcept>
#include <string>

using namespace std;

class Koruma {
    string ad;
public:
    Koruma(const string& a) : ad{a} { cout << ad << "+ "; }
    ~Koruma() { cout << ad << "- "; }
};

void c(int n) {
    Koruma k{"c"};
    if (n == 2) {
        throw runtime_error{"hata"};
    }
    if (n == 3) {
        throw 7;
    }
    cout << "c-ok ";
}

void b(int n) {
    Koruma k{"b"};
    c(n);
    cout << "b-ok ";
}

int main() {
    for (int n = 1; n <= 3; n++) {
        try {
            Koruma k{"main"};
            b(n);
            cout << "bitti ";
        } catch (const logic_error&) {
            cout << "L ";
        } catch (const exception& e) {
            cout << "E:" << e.what() << ' ';
        } catch (...) {
            cout << "? ";
        }
        cout << '\n';
    }
    return 0;
}
```

C) Kod Yazma

18 İki boyutlu vektörü temsil eden `Vektor2` sınıfını yaz ve `vektorIslem` programında kullan.

- `x` ve `y` adlı iki `double` üye değişken **private** olsun.
- Yapıcı iki `double` alsın, ikisinin de varsayılan değeri `0` olsun ve üyeleri **üye ilk değer listesiyle** ilklesin.
- Üye operatörler: `operator+` (iki vektörün toplamı), `operator*` (vektör ile `double` skalerin çarpımı), `operator==` (iki bileşen de eşitse `true`). Bunlar nesneyi değiştirmediği için `const` olsun.
- `double uzunluk() const`: `sqrt(x * x + y * y)` döndürsün (`<cmath>`).
- `operator<<` **üye olmayan** bir fonksiyon olsun ve vektörü `(x, y)` biçiminde yazdırsın; `cout << a` ve zincirleme kullanım çalışmalı.

`main` iki vektörü ve bir skaleri klavyeden okusun, sonuçları aşağıdaki biçimde yazdırsın. Örnek çalışma:

```
a: 3 4
b: 1 2
k: 2
a + b = (4, 6)
a * 2 = (6, 8)
|a| = 5
a == b: hayır
```

19 Bir şirketin çalışanlarını **çok biçimli** biçimde tutan `calisanlar` programını yaz.

- `Calisan` **soyut** sınıf olsun: `ad` (private `string`), yapıcı, `adAl() const`, **saf sanal** `double maas() const` ve **sanal yıkıcı**.
- `Memur : public Calisan`: adı ve sabit maaşı alır, `maas()` sabit maaşı döndürür.
- `Satisci : public Calisan`: adı, taban maaşı ve satış adedini alır; `maas()` değeri `taban + satisAdedi * 150` olsun.
- `main` içinde `vector<unique_ptr<Calisan>>` oluştur; sırasıyla `Memur("Ayse", 30000)`, `Satisci("Can", 20000, 40)` ve `Memur("Deniz", 35000)` nesnelerini `make_unique` ile ekle.
- Her çalışanın adını ve maaşını (**iki ondalık basamakla**), sonra toplam maaşı ve **en yüksek maaşlı** çalışanın adını yazdır.

Elle `new` / `delete` kullanma. Beklenen çıktı:

```
Ayse: 30000.00
Can: 26000.00
Deniz: 35000.00
Toplam: 91000.00
En yüksek: Deniz
```

20 Çalışma klasöründeki `notlar.txt` dosyasının her satırında bir öğrencinin adı ve bir sınav puanı vardır; aynı öğrenci birden çok satırda geçebilir:

```
Ali 80
Ayse 90
Ali 70
Can 55
Ayse 100
Can 65
Deniz 85
```

`notOrtalamasi` programını yaz. Dosyayı `ifstream` ile aç; açılamazsa `cerr` ile `Dosya açılamadı!` yazıp `1` döndür. Puanları `map<string, vector<int>>` içinde öğrenci adına göre topla. Her öğrencinin ortalamasını hesapla, `vector<pair<string, double>>` içine koy ve **lambda ifadesi** kullanan `sort` ile ortalamaya göre **büyükten küçüğe** sırala (ortalamalar eşitse ada göre alfabetik). Her satıra ad, ortalama (**bir ondalık basamakla**) ve not sayısını yaz. (Değişken adı olarak `not` kullanma; C++'ta `!` işlecinin alternatif yazımıdır.) Beklenen çıktı:

```
Ayse: 95.0 (2 not)
Deniz: 85.0 (1 not)
Ali: 75.0 (2 not)
Can: 60.0 (2 not)
```

21 Üç özyinelemeli fonksiyon yazıp `ozyinelemeAraclari` programında kullan:

- `int basamakToplami(int n)`: `n`'in basamaklarının toplamını döndürür (`1221` için `6`). `n` negatifse `invalid_argument` istisnası fırlatır (`"negatif sayi"` iletisiyle).
- `string ikilik(int n)`: `n >= 0` sayısının ikilik (taban 2) gösterimini `string` olarak döndürür (`0` için `"0"`, `6` için `"110"`). **Döngü kullanma.** İpucu: Bölüm 5'teki basamak ayırmanın aynısıdır, `10` yerine `2` ile: `n % 2` son ikilik basamağı verir, `n / 2` onu atar.
- `bool palindromMu(const string& s, int bas, int son)`: `s[bas..son]` aralığı tersiyle aynıysa `true` döndürür.

`main` bir tamsayı okusun (okunamazsa `Geçersiz girdi!` yazıp bitsin) ve sırasıyla basamak toplamını, ikilik gösterimi ve sayının palindrom olup olmadığını (`to_string(n)` üzerinden) yazdırsın. İstisnayı `main` içinde yakala ve `Hata: <ileti>` yaz. Örnek çalışmalar:

```
Sayı: 1221
Basamak toplamı: 6
İkilik: 10011000101
Palindrom: evet
```

```
Sayı: -5
Hata: negatif sayi
```

Çözümler

Cevap anahtarı

1 C	2 A	3 D	4 B	5 E	6 B	7 A	8 D
9 C	10 E	11 B	12 D				

1 Cevap: C

Başlangıç: `a = 0`, `b = 5`, `c = 2`.

- İlk `if`: `a++` ifadesi `a`'nın **eski** değerini (0) verir ve `a` 1 olur. `0` yanlıştır; `&&` operatörünün sol tarafı yanlış olduğu için sağ taraf (`b++`) **hiç değerlendirilmez** (kısa devre değerlendirme). `b` 5 kalır, `if` gövdesi çalışmaz, `c` 2 kalır.
- İkinci `if`: `b--` eski değeri (5) verir ve `b` 4 olur. `5` doğrudur; `||` operatörünün sol tarafı doğru olduğu için sağ taraf (`++c > 2`) **değerlendirilmez**. Gövde çalışır: `c = 2 * 2 = 4`.

Sonuç: `a = 1`, `b = 4`, `c = 4`.

Çıktı
```
1 4 4
```

- A) Kısa devre olmadığını varsaymak: `b++` ile `b` geçici olarak 6 olur, sonra `b--` ile 5'e döner; `++c` de çalışır, `c` 3'e çıkıp 6 olur.
- B) `c += 10` satırı çalışsaydı `c` 12 olurdu. İlk koşulun sol tarafı `0` olduğu için gövde çalışmaz.
- D) `a++` yalnız bir kez çalışır; `a` 2 olamaz. `b` de 5 değil, `b--` yüzünden 4'tür.
- E) `||` sağ tarafı (`++c > 2`) değerlendirilseydi `c` 3 olur ve 6'ya çıkardı. Oysa sol taraf doğru olduğu için sağ taraf hiç çalışmaz; `c` 4 kalır.

Konu: Bölüm 2 ve 4.

2 Cevap: A

- `vector<int> a(3, 7)` **parantezle** çağrıldığı için "3 tane 7" demektir: üç eleman (`7 7 7`).
- `vector<int> b{3, 7}` **süslü parantezle** çağrıldığı için eleman listesidir: iki eleman (`3`, `7`).
- `a[1] + b[0]` → `7 + 3 = 10`.
- `s[0] = 'M'` ile `"Merhaba"` olur. `erase(konum)` tek parametreyle çağrılırsa `konum`'dan **sona kadar** siler; `size() - 2` yani 5. indeksten itibaren siler: `"Merha"`.

Çıktı
```
3 2 10 Merha
```

- B) Boyutlar ters yazılmış: parantez 3 eleman, süslü parantez 2 eleman verir.
- C) `b[0]` 3'tür, 7 değil; toplam 10 olur.
- D) `erase` sondaki iki karakteri siler; `"Merhaba"` kalmaz.
- E) Sondan iki karakter silinir (`ba`), tek karakter (`a`) değil; `"Merhab"` yanlıştır.

Konu: Bölüm 7 ve 8.

3 Cevap: D

- `const int* p`: **gösterilen** değer sabittir (`*p` değiştirilemez), ama `p`'nin kendisi başka adrese çevrilebilir. `p = &b;` geçerlidir.
- `int* const q`: işaretçinin **kendisi** sabittir (`q` başka yeri gösteremez), ama gösterdiği `int` değiştirilebilir. `*q = 5;` geçerlidir.
- `const int* const r`: ikisi de sabit; yalnız okunabilir. `*r` okumak serbesttir.
- `*p = 7;` ise `const int`'e yazmaktır: "assignment of read-only location".

Derleyici çıktısı – program derlenmez

```
test3.cpp: In function 'int main()':
test3.cpp:10:8: error: assignment of read-only location '* p'
   10 |     *p = 7;
      |     ~~~^~~
```

- A) `p`'nin kendisi sabit değildir; `const` yıldızdan **önce** olduğu için kilitlenen gösterilen değerdir.
- B) `const` yıldızdan **sonra** gelmiş (`int* const`); kilitlenen işaretçidir, `*q` yazılabilir.
- C) Okuma işlemi hiçbir `const` kuralını bozmaz.
- E) Kodda tam bir hata var.

Okuma ipucu: `const`'u **soldaki şeye** bağla; en soldaysa hemen sağındaki türe bağlanır. Konu: Bölüm 9.

4 Cevap: B

`make_unique<int>(5)` ile tek sahipli işaretçi `a` oluşur. `std::move(a)` sahipliği `b`'ye **taşır**; `a` boş (`nullptr`) kalır. `*b += 3` ile gösterilen değer 8 olur. `a == nullptr` doğru olduğu için `1` yazılır.

Çıktı

```
1 8
```

- A) `a` hâlâ 5'i gösteriyor sanmak: taşıma sahipliği tek bir yerde tutar, kaynağı boşaltır.
- C) Taşıma sonrası `a == nullptr` doğrudur, yani `0` değil `1` yazılır.
- D) `*b += 3` çalışır; değer 5 kalmaz.
- E) `unique_ptr` kopyalanamaz ama **taşınabilir**; `move` ile yazılan bu kod derlenir.

Konu: Bölüm 10.

5 Cevap: E

`Kutu c = b;` satırı **kopya yapıcıyı** çağırır. Kopya yapıcıyı biz yazmadığımız için derleyicinin yazdığı kullanılır; o da üyeleri kopyalar ama `canli++` yapmaz. Yani:

Olay	`canli`
`Kutu a;`	1
`Kutu b;`	2
`Kutu c = b;` (kopya yapıcı, sayaç artmaz)	2
`cout → 2`	
blok biter: `c` ve `b` için **iki** yıkıcı	0
`cout → 0`	

`a`, `main` bitince yok olur ve `canli` bu sırada -1 olur; ama o noktada artık bir şey yazdırılmaz.

```
Çıktı
2 0
```

- A) Kopya yapıcının da sayacı artırdığını varsaymak: `3 1` olurdu.
- B) Blok bitince iki nesne birden yok olur; bir tanesi değil.
- C) Sayaç kopya yapıcıda artmadığı için ilk değer 3 olamaz; ayrıca blok sonunda iki yıkıcı çalışır, sayaç 2'de kalmaz.
- D) `-1`, `main` bittikten sonra oluşur; ikinci `cout` o anda `0` görür.

Bu soru kopya yapıcıyı yazmadığında sayaçların neden şaştığını gösterir. Konu: Bölüm 11 ve 13.

6 Cevap: B

`Hayvan`, saf sanal `sesCikar` fonksiyonuyla soyut sınıftır. `Kedi` bu fonksiyonu ezdiği için somuttur. `Kopek` ise onu **ezmediği için** saf sanal fonksiyonu devralmış olur ve **o da soyuttur**. Soyut sınıftan nesne üretilemez: `Kopek d;` satırı (2) hata verir.

Soyut sınıf türünden **işaretçi** ve **referans** tanımlamak serbesttir (3 ve 4); sanal çağrılar da düzgün çalışır (5 ve 6). Çok biçimlilik zaten bunun üzerine kuruludur.

```
Derleyici çıktısı – program derlenmez
test6.cpp: In function 'int main()':
test6.cpp:16:11: error: cannot declare variable 'd' to be of abstract type 'Kopek'
   16 |     Kopek d;
      |           ^
test6.cpp:12:7: note: because the following virtual functions are pure within 'Kopek':
   12 | class Kopek : public Hayvan {};
      |       ^~~~~
  * 'virtual void Hayvan::sesCikar() const'
    test6.cpp:5:18:
        5 |     virtual void sesCikar() const = 0;
          |                  ^~~~~~~~
```

- A) `Kopek d;` derlenmez.
- C) İşaretçi tanımı (3) hata değildir; soyut sınıfın işaretçisi çok biçimli kodun temelidir.
- D) Referans (4) de aynı şekilde serbesttir.
- E) `p->sesCikar()` ve `r.sesCikar()` sanal çağrılardır ve `Kedi`'ninkini çalıştırır.

Konu: Bölüm 12.

7 Cevap: A

Önek ve sonek artırma aynı sınıfta iki ayrı fonksiyondur; sonek olanı sahte `int` parametresiyle ayrılır.

Satır	`s`	Sonuç
`Sayac s{5};`	5	
`Sayac t = s++;`	6	sonek **eski** kopyayı (5) döndürür, `t = 5`
`Sayac u = ++s;`	7	önek yeni değeri döndürür, `u = 7`

```
Çıktı
7 5 7
```

- B) `t`'nin 6 olması için `s++` ifadesinin yeni değer döndürmesi gerekirdi; o önek davranışıdır.
- C) `u` 7'dir, çünkü `++s` önce artırır sonra döndürür.
- D) `s` iki kez artırıldı (5 → 6 → 7); 6'da kalmaz.
- E) `t`, `s++`'ın döndürdüğü eski kopyadır (5); `s`'e bağlı değildir. `t` 7 olamaz.

Konu: Bölüm 13.

8 Cevap: D

Şablonda `T` **tek** bir tip olmalıdır. `enBuyuk(3, 4.5)` çağrısında birinci argüman `int`, ikincisi `double` olduğu için derleyici `T` için iki farklı sonuç çıkarır ve çağrıyı eşleştiremez ("no matching function"). Dönüşüm yapmaz.

- Satır 1: `T = int`; satır 2: `T = double`; satır 5: `T = string`. Üçü de geçerlidir.
- Satır 3: `T` açıkça `double` verilmiştir, çıkarım yapılmaz; `3` örtük olarak `double`'a dönüşür. Geçerlidir.

```
Derleyici çıktısı – program derlenmez
test8.cpp: In function 'int main()':
test8.cpp:13:20: error: no matching function for call to 'enBuyuk(int, double)'
   13 |     cout << enBuyuk(3, 4.5) << '\n';
      |             ~~~~~~~^~~~~~~~
  * there is 1 candidate
    * candidate 1: 'template<class T> T enBuyuk(T, T)'
      test8.cpp:5:3:
          5 | T enBuyuk(T a, T b) {
            |   ^~~~~~
      * template argument deduction/substitution failed:
        *   deduced conflicting types for parameter 'T' ('int' and 'double')
          test8.cpp:13:20:
             13 |     cout << enBuyuk(3, 4.5) << '\n';
                |             ~~~~~~~^~~~~~~~
```

- A) Satır 4 çağrısı derlenmez; "hata yok" diyemezsin.
- B) Satır 3'te `T` açıkça `double` verildiği için çıkarım yapılmaz ve `3` örtük olarak `double`'a dönüşür; hata yoktur.
- C) Satır 3 geçerlidir (yukarıya bak).
- E) Satır 5'te iki argüman da `string`'dir, `T = string` olur; hata yoktur.

Satır 4 çıkarıldığında kalan program sorunsuz derlenir ve çalışır:

```
Çıktı
7
2.5
4.5
b
```

Çözüm: ya iki argümanı aynı tipe getirirsin (`enBuyuk(3.0, 4.5)`), ya da `enBuyuk<double>(3, 4.5)` yazarsın.
Konu: Bölüm 13.

9 Cevap: C

`try` bloğunda `"1 "` yazılır. `stoi("abc")` sayıya çevrilemez ve `invalid_argument` fırlatır; kalan satırlar (`"2 "`, `at(10)`, `"3 "`) hiç çalışmaz.

`catch` blokları **yukarıdan aşağıya** denenir ve ilk uyan seçilir:

1. `out_of_range`: `invalid_argument` bu türden değildir, uymaz.
2. `logic_error`: `invalid_argument`, `logic_error`'dan türer (is-a), **uyar**.

`"L "` yazılır, `exception` bloğu çalışmaz.

```
Çıktı
1 L bitti
```

- A) `at(10)` satırına hiç gelinmez; `stoi` daha önce fırlatır.
- B) `invalid_argument` ile `out_of_range` kardeş sınıflardır; biri diğerini yakalamaz.
- D) `exception` bloğu da uyardı, ama listede daha önce uyan bir blok (`logic_error`) var.
- E) İstisna fırlatıldığı için `try` içinde devam edilmez.

Konu: Bölüm 14.

10 Cevap: E

`map` üzerinde `m[anahtar]` anahtar yoksa **eleman oluşturur** (değeri `0`); bu yüzden `if (m["kiraz"] > 0)` bile haritaya `kiraz` ekler.

Satır	Harita
`m["elma"] = 3;`	elma 3
`m["armut"]++;`	+ armut 1
`m["kiraz"] > 0`	+ kiraz 0 (koşul yanlış)
`m.insert({"elma", 99})`	`elma` zaten var, **eklenmez**

`m.size()` → 3, `m["elma"]` → 3, `m.count("muz")` → 0 (`count` eleman oluşturmaz), `m["armut"]` → 1.

```
Çıktı
3 3 0 1
```

- A) `m["kiraz"]` okurken de eleman ekler; 2 yanlıştır.
- B) `insert` mevcut anahtarın değerini değiştirmez; `99` yazılmaz.
- C) `count`, `muz` yoksa 0 döndürür.
- D) `count` hiçbir şey eklemez; 4 eleman olmaz.

Elemanı eklemeden aramak için `find` ya da `count` kullan. Konu: Bölüm 15.

11 Cevap: B

`eof()` yalnız dosya sonunu **aştıktan sonra** `true` olur; son sayıdan sonra kalan satır sonu karakteri yüzünden `5`'i okuduktan sonra bayrak henüz yükselmemiştir.

Tur	Okuma	`sayi`	`adet`	`toplam`
1	`3`	3	1	3
2	`4`	4	2	7
3	`5`	5	3	12
4	boşlukları atlarken dosya sonu: **okuma başarısız**, `sayi` değişmez	5	4	17

Dördüncü turda okuma başarısız olur ve `eof` bayrağı ancak şimdi yükselir; ama gövdenin kalanı (`adet++`, `toplam += sayi`) yine çalışır. `sayi` hâlâ 5 olduğu için son sayı **iki kez** toplanır.

```
Çıktı
4 17
```

- A) `eof()` ile kurulan döngünün tam doğru çalıştığını sanmak: 3 tur olurdu.
- C) Başarısız okuma `sayi`'ye 0 yazmaz (boşluk atlama aşamasında dosya bittiği için okuma denenmez bile); toplam 12 olmaz.
- D) Tur sayısı 3 değil 4'tür.
- E) Beşinci tur olmaz: dördüncü turda `eof` bayrağı yükselir ve döngü biter.

Doğrusu `while (dosya >> sayi)` kalıbıdır: okuma başarısızsa döngü gövdeye girmez. Konu: Bölüm 16.

---

12 Cevap: D

Çağrı sayısı `C(n)` için `C(0) = C(1) = 1` ve `C(n) = 1 + C(n−1) + C(n−2)` bağıntısı geçerlidir (bir çağrı + iki alt çağrı):

n	0	1	2	3	4	5	6
`C(n)`	1	1	3	5	9	15	25

`fib(6)` değeri 8'dir ama bu sonuca **25 çağrıyla** ulaşılır (`C(6) = 1 + 15 + 9`); aynı alt problemler tekrar tekrar çözülür (üstel büyüme; bellekleme bunu çözer). Aşağıdaki çıktıda ilk sayı `fib(6)` değeri, ikinci sayı çağrı sayısıdır.

```
Çıktı
8 25
```

- A) 8, `fib(6)` değerinin kendisidir; çağrı sayısı değildir.
- B) 15, `fib(5)` için çağrı sayısıdır (Bölüm 17'deki çağrı ağacı); bir basamak eksik hesaplanmış.
- C) 21 bir çağrı sayısı değil, `fib(8)` değeridir; Fibonacci değerleriyle çağrı sayılarını karıştırma.
- E) 41, `fib(7)` için çağrı sayısıdır (`1 + 25 + 15`).

Konu: Bölüm 17.

13

Kuralı hatırla: nesne kurulurken **önce temel sınıf**, sonra **üye nesneler**, en son kendi yapıcı gövdesi çalışır. Yıkılırken tam **ters** sıra izlenir: önce kendi yıkıcı gövdesi, sonra üye nesneler, en son temel sınıf. Aynı bloktaki yerel nesneler de bildirim sırasının tersiyle yok edilir.

1. İç blokta `yedek` kurulur: `Motor+`. Sonra `Araba a`: `Arac+` (temel sınıf), `Motor+` (üye `m`), `Araba+`. Ardından `ic` yazılır.
2. Blok biterken ters sıra: önce `a` yıkılır: `Araba− Motor− Arac−`; sonra `yedek`: `Motor−`.
3. `new Araba` aynı kurulum dizisini yazar: `Arac+ Motor+ Araba+`. Satır sonu gelir.
4. `delete p`: `p` bir `Arac*`, ama yıkıcı `virtual` olduğu için gerçek tür olan `Araba`'nın yıkıcısı çalışır, zincir tam yürür: `Araba− Motor− Arac−`. Yıkıcı `virtual` olmasaydı bu kod tanımsız davranış olurdu.

Satırların sonunda bir boşluk kalır; sınavda bunu yazmaman sorun olmaz, ama sıra tam olmalı.

Çıktı
```
Motor+ Arac+ Motor+ Araba+ ic
Araba- Motor- Arac- Motor- Arac+ Motor+ Araba+
Araba- Motor- Arac-
son
```

Konu: Bölüm 11 ve 12.

14

`virtual` olan fonksiyon **nesnenin gerçek (dinamik) türüne** göre, `virtual` olmayan fonksiyon **ifadenin (statik) türüne** göre seçilir. Değer ile geçirme ve kopyalama ise türetilmiş kısmı kesip atar (nesne dilimleme).

Satır	Neden	Yazılan
`r.ad()`	`virtual`, nesne `Kare`	`Kare`
`r.tur()`	`virtual` değil, `r` türü `Sekil&`	`S`
`k.tur()`	`k` türü `Kare`	`K`
`p->ad()`, `p->alan()`	`virtual`, nesne `Kare`; alan 3 × 3	`Kare 9`
`kopya.ad()`, `kopya.alan()`	`Sekil kopya = k;` nesneyi **dilimledi**; artık gerçek bir `Sekil`	`Sekil 0`
`degerle(k)`	parametre `Sekil s` değerle geçer, dilimlenir	`deger: Sekil 0`
`referansla(k)`	referans nesnenin kendisine bağlanır	`ref: Kare 9`
`v[0].ad()`	`vector<Sekil>` nesnenin **kopyasını** (dilimli) saklar	`Sekil`
`w[0]->ad()`, `->alan()`	işaretçi `Kare`'yi gösterir; 2 × 2	`Kare 4`

`double` değerler `cout` ile `9` ve `0` biçiminde yazılır (`9.0` değil).

Çıktı
```
Kare S K
Kare 9
Sekil 0
deger: Sekil 0
ref: Kare 9
Sekil Kare 4
```

Konu: Bölüm 9 ve 12.

15

`artir(a)` parametreyi **kopyaladığı** için `main`'deki `a` değişmez; fonksiyonun içinde `x` 5 olur ve `ic 5` yazılır. `artirRef(b)` asıl `b`'yi değiştirir (9 → 10).

Adım	a	b	Yazılan
başlangıç	4	9	
`artir(a)`	4	9	`ic 5`
`artirRef(b)`	4	10	`4 10`
`enBuyuk(a, b) = 0` (`b` büyük; referans `b`'nin kendisi)	4	0	`4 0`
`takas(a, b)`	0	4	`0 4`

`vector` kısmı: `for (int x : v)` her elemanın kopyasını verir; `x *= 2` kopyayı değiştirir, `v` değişmez. Kopyalar toplamı `2 + 4 + 6 = 12`, `kopyaToplam` olur. `for (auto& x : v)` ise elemanın kendisi: `v = {2, 3, 4}`. `r` referansı `v[1]`'e bağlıdır: `v[1] = 50`. `auto c = v[2]` bir **kopya** alır (`auto` referansı atar), `c = 0` yalnızca kopyayı değiştirir. `p = &v[0]` ile `*p += 10` → `v[0] = 12`.

Çıktı
```
ic 5
4 10
4 0
0 4
12 50 4 0 12
```

Konu: Bölüm 6, 7 ve 9.

16

`yaz(3)`. Önce çağrıdan önceki `cout`, sonra özyineleme, sonra çağrıdan sonraki `cout` çalışır. Giderken `3 2 1` yazılır; temel durumda `dip`; dönerken yığın ters sırada açılır ve `1 2 3` yazılır.

`ters("abc")`. `ters("abc") = ters("bc") + 'a' = (ters("c") + 'b') + 'a' = ((ters("") + 'c') + 'b') + 'a' = "cba"`. Temel durum boş `string`'dir.

`us(2, 10)`. Çağrı zinciri: `us(2,10)` → `us(2,5)` → `us(2,4)` → `us(2,2)` → `us(2,1)` → `us(2,0)`: **6 çağrı**. Dönüşler: `us(2,0) = 1`, `us(2,1) = 2·1 = 2`, `us(2,2) = 2·2 = 4`, `us(2,4) = 4·4 = 16`, `us(2,5) = 2·16 = 32`, `us(2,10) = 32·32 = 1024`.

Çıktı
```
3 2 1 dip 1 2 3
cba
1024 6
```

Konu: Bölüm 8 ve 17.

17

`throw` ile `catch` arasındaki yerel nesneler **yığın çözülmesi** (stack unwinding) sırasında, oluşturulmalarının tersine yıkılır. `catch` bloğu ancak bu yıkımdan **sonra** çalışır.

- **n = 1:** İstisna yok. `main+ b+ c+ c-ok` sonra `c` biter: `c-`; `b-ok b-` ve `bitti`; `try` bloğu biter, `main-`.
- **n = 2:** `main+ b+ c+` yazılır, `throw runtime_error` fırlar. Çözülme: `c- b- main-` (içten dışa). Sonra `catch`: `runtime_error`, `logic_error` değildir (ilk blok uymaz); `exception&` uyar: `E:hata`. `c-ok`, `b-ok`, `bitti` hiç yazılmaz.
- **n = 3:** `throw 7` bir `int` fırlatır; ne `logic_error` ne `exception` olduğu için yalnız `catch (...)` yakalar. Yıkıcı sırası aynıdır: `c- b- main-` ve ardından `?`.

Çıktı
```
main+ b+ c+ c-ok c- b-ok b- bitti main-
main+ b+ c+ c- b- main- E:hata
main+ b+ c+ c- b- main- ?
```

Konu: Bölüm 11 ve 14.

18

Yaklaşım: Üyeler `private` kalır, yapıcı `x{x0}, y{y0}` ile **üye ilk değer listesini** kullanır ve iki varsayılan argüman verir. `+`, `*` ve `==` nesneyi değiştirmediği için `const` üye fonksiyondur; `+` ve `*` yeni bir `Vektor2` **değer olarak** döndürür. `operator<<` sınıfın üyesi olamaz (sol operand `ostream`'dir); `private` üyelere erişmesi için `friend` yaptık (herkese açık `getX()` / `getY()` yazmak da kabul edilir). `ostream&` döndürmek `cout << a << b` zincirini mümkün kılar.

vektorIslem.cpp

```
#include <cmath>
#include <iostream>

using namespace std;

class Vektor2 {
    double x;
    double y;
public:
    Vektor2(double x0 = 0, double y0 = 0) : x{x0}, y{y0} {}

    Vektor2 operator+(const Vektor2& d) const {
        return Vektor2{x + d.x, y + d.y};
    }
    Vektor2 operator*(double k) const { return Vektor2{x * k, y * k}; }
    bool operator==(const Vektor2& d) const {
        return x == d.x && y == d.y;
    }
    double uzunluk() const { return sqrt(x * x + y * y); }

    friend ostream& operator<<(ostream& os, const Vektor2& v);
};

ostream& operator<<(ostream& os, const Vektor2& v) {
    return os << '(' << v.x << ", " << v.y << ')';
}

int main() {
    double ax, ay, bx, by, k;
    cout << "a: ";
    cin >> ax >> ay;
    cout << "b: ";
    cin >> bx >> by;
    cout << "k: ";
    cin >> k;
    Vektor2 a{ax, ay};
    Vektor2 b{bx, by};
    cout << "a + b = " << a + b << '\n';
    cout << "a * " << k << " = " << a * k << '\n';
    cout << "|a| = " << a.uzunluk() << '\n';
    cout << "a == b: " << (a == b ? "evet" : "hayır") << '\n';
    return 0;
}
```

Çıktı

```
a: 3 4
b: 1 2
k: 2
a + b = (4, 6)
a * 2 = (6, 8)
|a| = 5
a == b: hayır
```

Tuzaklar: `operator<<`'ı sınıfın **içine üye** olarak yazmak (`cout << a` yerine `a << cout` gerekir); `operator<<` içinde `os` yerine `cout` kullanmak (başka akışlara yazdırılamaz); `==` ile `double` karşılaştırmasının bu soruda tam sayılarla çalıştığını, genel olarak ondalık sayılarda toleransla karşılaştırmak gerektiğini unutmak.

Puanlama (11): private üyeler ve üye ilk değer listesi + varsayılan argümanlar 2; `operator+` ve `operator*` doğru ve `const` 3; `operator==` ve `uzunluk` 2; üye olmayan `operator<<` ve `ostream&` dönüşü 3; `main` okuma ve çıktı biçimi 1.

Konu: Bölüm 11 ve 13.

19

Yaklaşım: `Calisan` saf sanal `maas()` ve `virtual ~Calisan() = default;` ile soyut kök sınıftır. `ad` `private` olduğu için türetilmiş sınıflar onu taban sınıf yapıcısına `Calisan{a}` ile geçirir. Nesneler `make_unique` ile üretilip `vector<unique_ptr<Calisan>>` içine **taşınarak** girer; `unique_ptr` yıkımı sanal yıkıcı sayesinde doğru türü siler, elle `delete` gerekmez. Dolaşırken `const auto&` kullandık: `unique_ptr` **kopyalanamaz**, `for (auto p : liste)` derlenmez. `liste[0].get()` ham işaretçi verir; sahiplik `vector`'de kalır.

calisanlar.cpp

```cpp
#include <iomanip>
#include <iostream>
#include <memory>
#include <string>
#include <vector>

using namespace std;

class Calisan {
    string ad;
public:
    Calisan(const string& a) : ad{a} {}
    virtual ~Calisan() = default;
    virtual double maas() const = 0;
    const string& adAl() const { return ad; }
};

class Memur : public Calisan {
    double sabit;
public:
    Memur(const string& a, double s) : Calisan{a}, sabit{s} {}
    double maas() const override { return sabit; }
};

class Satisci : public Calisan {
    double taban;
    int satisAdedi;
public:
    Satisci(const string& a, double t, int adet)
        : Calisan{a}, taban{t}, satisAdedi{adet} {}
    double maas() const override { return taban + satisAdedi * 150; }
};
```

```cpp
int main() {
    vector<unique_ptr<Calisan>> liste;
    liste.push_back(make_unique<Memur>("Ayse", 30000));
    liste.push_back(make_unique<Satisci>("Can", 20000, 40));
    liste.push_back(make_unique<Memur>("Deniz", 35000));

    cout << fixed << setprecision(2);
    double toplam{0};
    const Calisan* enYuksek = liste[0].get();
    for (const auto& c : liste) {
        cout << c->adAl() << ": " << c->maas() << '\n';
        toplam += c->maas();
        if (c->maas() > enYuksek->maas()) {
            enYuksek = c.get();
        }
    }
    cout << "Toplam: " << toplam << '\n';
    cout << "En yüksek: " << enYuksek->adAl() << '\n';
    return 0;
}
```

Çıktı

```
Ayse: 30000.00
Can: 26000.00
Deniz: 35000.00
Toplam: 91000.00
En yüksek: Deniz
```

Tuzaklar: yıkıcıyı `virtual` yapmamak (taban işaretçisiyle silinen türetilmiş nesne eksik yıkılır); `override` yazarken `const` unutup fonksiyonu ezmemek (`override` bunu derleme hatası yapar); `for (auto c : liste)` ile `unique_ptr`'yi kopyalamaya çalışmak.

Puanlama (11): soyut sınıf, saf sanal fonksiyon ve sanal yıkıcı 3; `Memur` ve `Satisci` (taban yapıcıya geçirme, `override`, maaş formülü) 3; `vector<unique_ptr<Calisan>>` ve `make_unique` 2; toplam ve en yüksek hesabı 2; biçimli çıktı 1.

Konu: Bölüm 10 ve 12.

20

Yaklaşım: Dosya `ifstream` ile açılır ve `is_open()` ile kontrol edilir. `while (dosya >> ad >> puan)` kalıbı okuma başarısız olunca kendiliğinden biter; `eof()` kullanma. `notlar[ad].push_back(puan)` ifadesinde `[]` yeni öğrenci için boş `vector` oluşturur; burada eleman oluşturması **istenen** davranıştır. Ortalamayı hesaplarken `static_cast<double>` kullanmak tamsayı bölmesini önler. Sıralamada lambda iki `pair` alır; ortalamalar eşitse ada göre alfabetik karşılaştırır ki sonuç belirli olsun. `for (const auto& [isim, liste] : notlar)` yapısal bağlama ile anahtar–değer çiftini açar.

notOrtalamasi.cpp

```cpp
#include <algorithm>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <string>
#include <utility>
#include <vector>

using namespace std;
```

```cpp
int main() {
    ifstream dosya{"notlar.txt"};
    if (!dosya.is_open()) {
        cerr << "Dosya açılamadı!\n";
        return 1;
    }
    map<string, vector<int>> notlar;
    string ad;
    int puan;
    while (dosya >> ad >> puan) {
        notlar[ad].push_back(puan);
    }

    vector<pair<string, double>> ortalamalar;
    for (const auto& [isim, liste] : notlar) {
        int toplam{0};
        for (int p : liste) {
            toplam += p;
        }
        ortalamalar.push_back(
            {isim, static_cast<double>(toplam) / liste.size()});
    }

    sort(ortalamalar.begin(), ortalamalar.end(),
         [](const pair<string, double>& x, const pair<string, double>& y) {
             if (x.second != y.second) {
                 return x.second > y.second;
             }
             return x.first < y.first;
         });

    cout << fixed << setprecision(1);
    for (const auto& o : ortalamalar) {
        cout << o.first << ": " << o.second << " ("
             << notlar.at(o.first).size() << " not)\n";
    }
    return 0;
}
```

Çıktı

```
Ayse: 95.0 (2 not)
Deniz: 85.0 (1 not)
Ali: 75.0 (2 not)
Can: 60.0 (2 not)
```

Tuzaklar: `while (!dosya.eof())` döngüsü (son satır iki kez işlenir); `int / int` ile ortalama almak; `sort` karşılaştırıcısında `>=` kullanmak (sıkı zayıf sıralama kuralı bozulur, `sort` hatalı çalışabilir); değişken adına `not` demek.

Puanlama (11): dosyayı açma ve `is_open` kontrolü 2; `>>` ile okuma döngüsü ve `map`'e ekleme 3; ortalama hesabı (`double`) 2; lambda ile doğru yönde `sort` 3; çıktı biçimi 1.

Konu: Bölüm 15 ve 16.

21

Yaklaşım: Her fonksiyon önce **temel durumu**, sonra **küçülen** özyinelemeli çağrıyı yazar. `basamakToplami`: temel durum `n < 10` (tek basamak kendisidir); aksi hâlde son basamak (`n % 10`) + kalan sayı (`n / 10`). `ikilik`: temel durum `n < 2` (`"0"` ya da `"1"`); aksi hâlde `ikilik(n / 2)` önce, **sonra** `n % 2` eklenir; bu sıra basamakları doğru yöne koyar (önce eklemek tersini verirdi). `palindromMu`: `bas >= son` ise aralık tek ya da boş karakter içerir,

palindromdur; uçlar farklıysa `false`; değilse iki ucu içeri çekerek devam eder. İndeksleri `int` tuttuk; boş `string`'de `size() - 1` işaretsiz çıkarma olurdu (Bölüm 8). İstisna `basamakToplami` içinde fırlatılır, `main` içinde `catch` edilir; hata durumunda sonraki satırlar hiç yazılmaz.

ozyinelemeAraclari.cpp

```cpp
#include <iostream>
#include <stdexcept>
#include <string>

using namespace std;

int basamakToplami(int n) {
    if (n < 0) {
        throw invalid_argument{"negatif sayi"};
    }
    if (n < 10) {
        return n;
    }
    return n % 10 + basamakToplami(n / 10);
}

string ikilik(int n) {
    if (n < 2) {
        return to_string(n);
    }
    return ikilik(n / 2) + to_string(n % 2);
}

bool palindromMu(const string& s, int bas, int son) {
    if (bas >= son) {
        return true;
    }
    if (s[bas] != s[son]) {
        return false;
    }
    return palindromMu(s, bas + 1, son - 1);
}

int main() {
    int n;
    cout << "Sayı: ";
    if (!(cin >> n)) {
        cout << "Geçersiz girdi!\n";
        return 0;
    }
    try {
        int bt = basamakToplami(n);
        cout << "Basamak toplamı: " << bt << '\n';
        cout << "İkilik: " << ikilik(n) << '\n';
        string s = to_string(n);
        bool pal = palindromMu(s, 0, static_cast<int>(s.size()) - 1);
        cout << "Palindrom: " << (pal ? "evet" : "hayır") << '\n';
    } catch (const invalid_argument& e) {
        cout << "Hata: " << e.what() << '\n';
    }
    return 0;
}
```

```
Çıktı
Sayı: 1221
Basamak toplamı: 6
İkilik: 10011000101
Palindrom: evet
```

İkinci örnekte `-5` girilirse `basamakToplami` `invalid_argument` fırlatır; `ikilik` ve `palindromMu` çağrılmaz ve ekrana `Hata: negatif sayi` yazılır. Programı `-5` ile çalıştırdığında ekranda yalnız `Hata:` satırını görürsün. Aşağıdaki çıktı, `basamakToplami`'nı `-5` ile çağıran kısa bir deneme programından alındı:

```
Çıktı
Hata: negatif sayi
```

Tuzaklar: temel durumu unutmak (yığın taşması); `ikilik` içinde `n % 2` ifadesini özyinelemeli çağrıdan **önce** eklemek (sonuç ters çıkar); `palindromMu` içinde her seferinde `substr` ile yeni `string` üretmek (çalışır ama gereksiz kopya); istisnayı `main` yerine fonksiyonun içinde yutmak.

Puanlama (11): `basamakToplami` (temel durum, özyineleme, istisna) 3; `ikilik` (temel durum ve doğru sıra) 3; `palindromMu` 3; `main` içinde `try/catch` ve çıktı 2.

Konu: Bölüm 14 ve 17.

EK A

C++ Cheat Sheet

Sınavdan önceki son tekrar için kitabın sıkıştırılmış hali. Anlamadığın satırda, parantez içindeki bölüm numarasına dön.

```cpp
#include <iostream>            // ön işlemci satırı: sonunda ; YOK
using namespace std;           // cout, string gibi adları std:: olmadan yazmak için

int main() {                   // program buradan başlar
    cout << "Merhaba\n";       // her deyim ; ile biter
    return 0;                  // 0 = başarılı
}
```

Temeller

Derleme (Bölüm 1)

```
g++ -std=c++17 -Wall -Wextra \
    -Wpedantic a.cpp -o a
./a                 (Windows: a.exe)
```

- Derleme sırası: ön işlemci, derleyici, bağlayıcı. Bağlayıcı hatası `undefined reference` ile çıkar.
- `-Wall -Wextra -Wpedantic` uyarıları açar. Uyarıyı görmezden gelme.
- Satır sonu için `'\n'` yaz. `endl` de satırı bitirir ama tamponu her seferinde boşaltır (Bölüm 3).

Tipler ve sizeof (Bölüm 2)

```
bool         1 B    true / false
char         1 B    'A' == 65
short        2 B
int          4 B    yaklaşık ±2,1 milyar
long long    8 B
float        4 B    ~7 anlamlı basamak
double       8 B    ~15 anlamlı basamak
long         4/8 B  Windows'ta 4, diğer 8
```

```cpp
// değiştirilemez sabit
const int MAX_NOT{100};
// tip derleyici tarafından: int
auto yas{19};
double boy{1.75};
cout << sizeof(yas) << '\n'; // 4
```

- `unsigned int` 0 … 4294967295. `int` taşması tanımsız davranıştır.
- `long` taşınabilir değildir: Windows'ta 4 B, macOS/Linux'ta 8 B.
- `string` bir sınıftır; `<string>` ile gelir. `bool` yerleşik tiptir.
- Sabit için `const`. Tipi derleyiciye bırakmak için `auto`.

Operatörler, yüksekten düşüğe (Bölüm 2, 4)

```
::  ()  []  .  ->  x++  x--
! ~ ++x --x -x *p &x (tip)
sizeof new delete
*  /  %
+  -
<<  >>
<  <=  >  >=
==  !=
&
^
|
&&
||
?:  =  +=  -=  *=  /=  %=
,
```

- Aynı satırdakiler aynı önceliktedir. `sizeof new delete` satırı da ikinci satırdaki tekli operatörlerle aynı düzeydedir; okunsun diye ayrı yazıldı.
- `7 / 2` sonucu `3`; `7 / 2.0` sonucu `3.5`; `-7 % 2` sonucu `-1`.
- `x++` eski değeri, `++x` yeni değeri verir. Aynı ifadede bir değişkeni iki kez değiştirmek (`int a = b++ + ++b;`) tanımsız davranıştır; yazma.
- `&&` ve `||` kısa devre yapar: sağ taraf gerekmiyorsa çalışmaz.
- `cout << x & 1` derlenmez, çünkü `<<` önce gelir: `(cout << x) & 1` olur. `cout << (x & 1)` yaz.
- `x & 1 == 0` ise `x & (1 == 0)` olur. Parantez koy.
- Bit işlemleri: `&` VE, `|` VEYA, `^` XOR, `~` değil, `<<` `>>` kaydırma. `^` üs değildir.

Girdi ve çıktı

cout biçimlendirme (`<iomanip>`, Bölüm 3)

Bayrak	Anlamı
`fixed`	ondalıklı sayı, sabit nokta: 3.142
`setprecision(n)`	fixed ile n ondalık; yoksa n anlamlı basamak
`setw(n)`	SONRAKİ tek çıktının genişliği
`setfill('c')`	boşluk yerine `c` (kalıcı)
`left` / `right`	`setw` içindeki hizalama
`boolalpha`	`bool`: `true` / `false` yazar
`showpos`	pozitif sayılara `+` koyar
`hex` / `dec` / `oct`	onaltılı, onlu, sekizli taban

```
#include <iomanip>
cout << fixed << setprecision(2);
cout << 3.14159 << '\n';        // 3.14
// 000042
cout << setw(6) << setfill('0')
     << 42 << '\n';
cout << setfill(' ') << left
     << setw(8) << "Ali" << "|\n";
// geri al
cout << setfill(' ') << right;
```

- `fixed`, `setprecision`, `setfill`, `left`, `hex` kalıcıdır; sen değiştirene kadar sonraki çıktıları da etkiler.
- `setw` yalnız bir sonraki çıktıya uygulanır.
- Parametre alan `setw`, `setprecision`, `setfill` için `<iomanip>` gerekir.

cin ve getline (Bölüm 3, 5)

```
int n;
// boşluğa kadar okur (sayı ya da kelime)
cin >> n;
string satir;
// <limits>: kalan satırı at
cin.ignore(
    numeric_limits<streamsize>::max(),
    '\n');
// satırın tamamı, boşluklar dahil
getline(cin, satir);
```

```
// okuma başarısız oldu (ör. "abc")
if (!(cin >> n)) {
    // hata bayrağını temizle
    cin.clear();
    cin.ignore(
      numeric_limits<streamsize>::max(),
        '\n');
}
```

- `cin.fail()` okuma hatasını, `cin.eof()` girdinin bittiğini söyler.
- Hata bayrağı `cin.clear()` çağrılana kadar kalır; sonraki her okuma sessizce başarısız olur.
- `cin >> n` sonrası `getline` boş satır okur. Aradaki `cin.ignore(...)` çözer.
- `cin.ignore()` parametresiz yazılırsa yalnız 1 karakter atar. Uzun biçimi kullan.

Kontrol ve döngüler

if, switch, ?: (Bölüm 4)

```
if (x > 0) {
    cout << "pozitif\n";
} else if (x == 0) {
    cout << "sıfır\n";
} else {
    cout << "negatif\n";
}
// C++17: if içinde tanım
if (int n{a % 3}; n == 0) { }
```

```
switch (gun) {
    case 1:
    case 7:
        cout << "Tatil\n";
        // yoksa alttaki case'ler
        // de çalışır
        break;
    default:
        cout << "İş günü\n";
}
// koşullu ifade (?:)
int m = a > b ? a : b;
```

- Yanlış = `0`: `if (x)` ile `if (x != 0)` aynıdır.
- `case` etiketi sabit tamsayı olmalı. `string` ile `switch` olmaz.
- Bir `case` içinde değişken tanımlıyorsan süslü parantez aç: `case 1: { ... }`.

Döngüler (Bölüm 5)

```
// sayaçlı döngü
for (int i{0}; i < n; ++i) { }
// aralık tabanlı: v'deki her eleman için
for (int x : v) { }
// koşul doğruyken
while (n > 0) { n /= 10; }
// en az bir kez; sondaki ; ZORUNLU
do { n--; } while (n > 0);
```

- `break` döngüden ya da `switch`'ten çıkar; `continue` sonraki tura geçer.
- `for (int x : v)` her elemanı **kopyalar**. Değiştirmek için `for (int& x : v)` yaz.
- Sonsuz döngü: `while (true)`; içinde `break` olmalı.

Fonksiyonlar (Bölüm 6)

```
// bildirim (prototip)
int kare(int x);
// tanım
int kare(int x) { return x * x; }

// referans: çağıranın değişkeni değişir
void artir(int& sayi) { ++sayi; }
// const &: kopya yok, değiştirilmez
void yaz(const string& metin);

// varsayılan: yalnız bildirimde
double faiz(double anapara,
            double oran = 0.05);
double faiz(double anapara, double oran)
{
    return anapara * oran;
}

// aşırı yükleme: aynı ad,
// farklı parametre listesi
int topla(int a, int b);
double topla(double a, double b);
```

- `int x` gibi değerle geçirilen parametre **kopyalanır**. Fonksiyon içinde değiştirmek çağırana yansımaz.
- Varsayılan parametreler sondan başlar: `f(int a = 1, int b)` derlenmez.
- Yalnız dönüş tipi farklı iki fonksiyon aşırı yüklenemez.
- `static int sayac{0};` yerel değişkeni çağrılar arasında korunur.
- `void` fonksiyonda `return;` erken çıkış yapar.

Diziler, vector ve string

Dizi (C tarzı, Bölüm 7)

```
int a[5]{};   // hepsi 0
int b[]{3, 1, 4};   // boyut 3
// 3: eleman sayısı
auto n{sizeof b / sizeof b[0]};
```

vector (`<vector>`, Bölüm 7)

```
// başlangıç listesi
vector<int> v{3, 1, 4};
// 5 eleman, hepsi 0
vector<int> w(5, 0);
v.push_back(9);           // sona ekle
// sınır dışı: out_of_range istisnası
v.at(7);
// sınır dışı: tanımsız davranış
v[7];
```

Üye	Ne yapar
`v.size()`	eleman sayısı (işaretsiz: `size_t`)
`v.empty()`	boş mu?
`v.push_back(x)`	sona ekler
`v.pop_back()`	sondakini siler (değer döndürmez)
`v.front()`, `v.back()`	ilk ve son elemanı verir
`v.insert(it, x)`	`it` konumunun önüne ekler (Bölüm 15)
`v.erase(it)`	`it` konumundaki elemanı siler
`v.resize(n)`	boyutu `n` yapar
`v.clear()`	hepsini siler

- `vector`'ün `sort()` üyesi yoktur. Sıralama için `sort(v.begin(), v.end())` kullan.
- `vector<vector<int>>` iki boyutlu tablodur (Bölüm 7).

string (`<string>`, Bölüm 8)

```
string s{"Merhaba"};
s += " dünya";            // birleştir
string metin{"42"};
// string -> int
// (metin sayı değilse istisna)
int sayi{stoi(metin)};
// int -> string
string yazi{to_string(85)};
```

Üye	Ne yapar
`s.size()`, `s.length()`	uzunluk, bayt cinsinden (Türkçe harf 2 bayt)
`s[i]`, `s.at(i)`	karakter; `at` sınır denetimli
`s.substr(i, n)`	`i`'den `n` karakter (yeni string)
`s.find("ab")`	ilk konum; yoksa `string::npos`
`s.insert(i, "x")`	`i`'nin önüne ekler
`s.erase(i, n)`	`i`'den `n` karakter siler
`s.replace(i, n, "x")`	`i`'den `n` karakteri `"x"` ile değiştirir
`s.push_back(c)`, `s.pop_back()`	sona ekle, sondan sil
`s.append(x)`	sona ekler (`+=` ile aynı)
`s.clear()`	metni boşaltır

map ve set (`<map>`, `<set>`, Bölüm 15)

Üye	Ne yapar
`m[k]`	`k`'nin değeri; yoksa 0 olarak ekler (dikkat)
`m.at(k)`	`k` yoksa `out_of_range` istisnası
`m.insert({k, v})`	`k` yoksa ekler; varsa dokunmaz
`m.find(k)`	`k`'yi gösteren iteratör; yoksa `m.end()`
`m.count(k)`	`k` varsa 1, yoksa 0
`m.erase(k)`	`k`'yi siler
`m.size()`, `m.empty()`	boyut ve boş mu?

```
map<string, int> puan;
// ekler ya da günceller
puan["Ali"] = 90;
// yoksa 0'dan başlar: 1
puan["Veli"]++;
// C++17: çifti parçala
for (const auto& [ad, deger] : puan)
    cout << ad << " " << deger << '\n';

// {1, 3}: tekrar yok, sıralı
set<int> kume{3, 1, 3};
kume.insert(5);
cout << kume.count(3) << '\n';   // 1
```

- `m[k]` yalnız okumak için bile anahtar ekler. Sadece var mı diye bakmak için `count` ya da `find` kullan.
- C++20 ile `m.contains(k)` gelir. C++17'de `count(k) == 1` yaz.

İşaretçi, referans ve bellek (Bölüm 9, 10)

```
int x{7};
// p, x'in adresini tutar
int* p{&x};
// dolaylama: x artık 5
*p = 5;
// boş işaretçi (C'deki NULL yerine)
int* q{nullptr};
// r, x'in takma adı; ilk değeri şarttır
int& r{x};
r = 9;                    // x de 9 oldu
```

```
// öbekte bir int
int* a{new int{42}};
// işi bitince sil
delete a;
// artık gösterme
a = nullptr;
// 10 int, hepsi 0
int* d{new int[10]{}};
// dizi için delete[] şart
delete[] d;
```

```
#include <memory>
// unique_ptr<int>: tek sahip
auto u{make_unique<int>(42)};
cout << *u << '\n';   // 42
// u kapsamdan çıkınca bellek
// kendiliğinden silinir (RAII)
```

- `unique_ptr` kopyalanamaz. Sahipliği `std::move(u)` ile taşırsın; sonra `u` boş olur.
- Her `new` için bir `delete`, her `new[]` için bir `delete[]`. `make_unique` bunu senin yerine yapar.

Sınıf iskeleti (Bölüm 11, 12)

```
class BankaHesabi {
public:
    // yapıcı (constructor)
    BankaHesabi(string sahip,
                double bakiye)
        // üye ilk değer listesi
        : sahip_{sahip},
          bakiye_{bakiye} {}
    // sanal yıkıcı (destructor)
    virtual ~BankaHesabi() = default;
    // const üye: değiştirmez
    string sahip() const {
        return sahip_;
    }
    void yatir(double miktar) {
        bakiye_ += miktar;
    }
    // alt sınıf ezebilir
    virtual void bilgi() const {
        cout << sahip_ << ": "
             << bakiye_ << '\n';
    }
private:
    // üye değişkenler
    string sahip_;
    double bakiye_;
};

// kalıtım (inheritance)
class Vadeli : public BankaHesabi {
public:
    Vadeli(string sahip, double bakiye)
        // temel sınıf yapıcısı
        : BankaHesabi{sahip, bakiye} {}
    // imza tutmazsa derlenmez
    void bilgi() const override {
```

```
        cout << "Vadeli ";
        BankaHesabi::bilgi();
    }
};
```

- `class` varsayılan olarak `private`, `struct` varsayılan olarak `public` erişir.
- Yıkıcının adı `~` ile sınıf adıdır; parametresi yoktur.
- Sanal fonksiyonu olan bir temel sınıfın yıkıcısı `virtual` olmalıdır.
- `virtual double faiz() const = 0;` saf sanal fonksiyondur; sınıfı soyut yapar, nesnesi üretilemez.
- `const` nesne yalnız `const` üye fonksiyonları çağırabilir.
- Kendi başına bir işaretçi tutan sınıfta kopya yapıcı ve atama operatörünü de yaz (Bölüm 13, üçler kuralı):

```
// kopya yapıcı
Sinif(const Sinif& d);
// atama operatörü
Sinif& operator=(const Sinif& d);
```

Operatör aşırı yükleme ve şablon (Bölüm 13)

```
class Sure {
public:
    Sure(int dakika) : dakika_{dakika} {}
    Sure operator+(const Sure& d) const {
        return Sure{dakika_ + d.dakika_};
    }
    bool operator==(
        const Sure& d) const {
        return dakika_ == d.dakika_;
    }
    // üye olmayan, ostream& döndürür
    friend ostream& operator<<(
        ostream& os, const Sure& s) {
        return os << s.dakika_ << " dk";
    }
private:
    int dakika_;
};
```

- `cout << s` için `operator<<` **üye olmayan** ve `ostream&` döndüren bir fonksiyon olmalıdır.
- Karşılaştırma ve `+` gibi işleçler işlenenlerini değiştirmez; `const` ile yaz.

```
// T'yi derleyici çıkarır
template <typename T>
T enBuyuk(T a, T b) {
    return a > b ? a : b;
}

template <typename T>
class Kutu {
public:
    Kutu(T deger) : deger_{deger} {}
    T al() const { return deger_; }
private:
    T deger_;
};
// tipi kendin yaz
Kutu<string> k{"ilk"};
```

- `enBuyuk(3, 2.5)` derlenmez: iki argüman aynı `T`'ye çıkarılamaz. `enBuyuk<double>(3, 2.5)` yaz.

İstisnalar (Bölüm 14)

```
#include <stdexcept>
try {
    // "abc" gelirse invalid_argument
    int n{stoi(metin)};
    // sınır dışıysa out_of_range
    cout << v.at(10);
} catch (const invalid_argument& e) {
    cerr << "Geçersiz: "
         << e.what() << '\n';
    // genel yakalayıcı en sona
} catch (const exception& e) {
    cerr << "Hata: " << e.what() << '\n';
}
// istisna fırlatır
throw runtime_error("dosya yok");
```

- `try` hatasız biterse `catch` blokları atlanır.
- `catch` her zaman `const &` ile yazılır; değerle yakalamak nesne dilimlemeye yol açar.
- İstisna fırladığında yerel nesnelerin yıkıcıları çalışır (yığın çözülmesi).

`<algorithm>` ve lambda (Bölüm 15)

```
#include <algorithm>
// küçükten büyüğe
sort(v.begin(), v.end());
// büyükten küçüğe
sort(v.begin(), v.end(),
     [](int a, int b) {
         return a > b;
     });
// bulunamazsa v.end()
auto it{find(v.begin(), v.end(), 9)};
// 1'lerin sayısı
auto adet{count(v.begin(), v.end(), 1)};
// ters çevir
reverse(v.begin(), v.end());
```

```
int esik{50};
// esik kopyalanır
auto gecer{[esik](int x) {
    return x > esik;
}};
int toplam{0};
// referans ile: toplam değişir
for_each(v.begin(), v.end(),
         [&toplam](int x) {
             toplam += x;
         });
```

- `[]` dış değişken almaz; `[x]` kopyalar; `[&x]` referansla alır; `[&]` hepsini referansla alır.
- `find` ve `max_element` iteratör döndürür; değeri `*it` ile alırsın.

Dosyalar (`<fstream>`, Bölüm 16)

```
#include <fstream>
// yazmak için açar: ESKİ İÇERİK SİLİNİR
ofstream yaz{"notlar.txt"};
// sonuna ekler
ofstream ekle{"notlar.txt", ios::app};
// okumak için açar
ifstream oku{"notlar.txt"};
// açılamadı mı?
if (!oku) {
    cerr << "dosya yok\n";
    return 1;
}
string satir;
// satır satır oku
while (getline(oku, satir))
    cout << satir << '\n';
// blok bitince dosyalar kendiliğinden
// kapanır (RAII)
```

- Dosya yolu göreli ise çalışma klasörüne göre çözülür.
- `>>` boşlukta durur. Döngü kalıbı olarak `while (oku >> x)` kullan; `while (!oku.eof())` tuzaktır.

Özyineleme (Bölüm 17)

```
long long fakt(int n) {
    // temel durum: burada durur
    if (n <= 1) return 1;
    // küçülür, kendini çağırır
    return n * fakt(n - 1);
}
```

- Temel durum yoksa çağrılar bitmez ve yığın taşar (stack overflow).
- Her çağrının yerel değişkenlerinin kendi kopyası vardır (yığın çerçevesi).
- `fakt(20)` `long long` sınırına sığar; `fakt(21)` taşar ve tanımsız davranıştır.

C'den farklar (kısa)

C	C++
`printf` / `scanf`	`cout` / `cin` (`<<`, `>>`)
`char s[]` ve `strcmp`	`string`; `==` içeriği karşılaştırır
`NULL`	`nullptr`
`malloc` / `free`	`new` / `delete` (ya da `make_unique`)
`//` ve `/* */` yorumları	aynı; ayrıca `bool`, `true`, `false` yerleşik
`struct` (yalnız veri)	`class`: üye fonksiyon, yapıcı, yıkıcı, erişim

Sık yapılan hatalar

1. `if (x = 5)` **atar**; karşılaştırma `==` ile yapılır.
2. `if (x > 0);` sonuna konan `;` gövdeyi boş bırakır.
3. `switch`'te `break` yoksa alttaki `case`'ler de çalışır.
4. `7 / 2` sonucu `3`: ondalık için en az bir taraf `double` olmalı.
5. `for (int i{0}; i <= v.size(); ++i)` son elemanı aşar. Son indeks `v.size() - 1`.
6. `for (int i{0}; i < v.size(); ++i)` ile `int` karşılaştırması `-Wsign-compare` uyarısı verir. İndeksi `size_t` yap.
7. `cin >> n;` sonrası `getline` boş okur. Araya `cin.ignore(...)` koy.
8. `string` için `==` içeriği karşılaştırır; C'deki gibi `strcmp` gerekmez. Ama `char*` iki işaretçiyi adres olarak karşılaştırır.
9. `v[i]` sınır dışına çıkarsa tanımsız davranış; `v.at(i)` istisna fırlatır.
10. `delete` yazmayı unutmak bellek sızıntısıdır; iki kez `delete` ya da `delete` sonrası kullanım tanımsız davranıştır.
11. Sınıf tanımının sonundaki `};` unutulursa derlenmez.
12. `using namespace std;` başlık dosyasında yazılmaz; büyük projelerde ad çakışmalarına yol açar.
13. `void main` standart değildir; `int main()` yaz.
14. `virtual` yazılmazsa `Temel*` işaretçisi üzerinden çağrıda temel sınıfın fonksiyonu çalışır (statik bağlama). Ezmek istediğin fonksiyona `virtual` ve `override` yaz.

EK B

Terimler Sözlüğü

Bu sözlükte kitapta geçen C++ terimleri Türk alfabesi sırasıyla dizilmiştir. Kitapta ya da `g++`'nın İngilizce mesajında bir terimle karşılaşırsan, İngilizce sütununa bakarak Türkçe karşılığını bulabilirsin. Kitap boyunca her kavram için tek bir Türkçe ad kullanılır.

A	İngilizce	Açıklama
açık dönüşüm	*explicit conversion (cast)*	Tip dönüşümünün programcı tarafından `static_cast` gibi yapılarla açıkça yapılması.
ad alanı	*namespace*	Aynı adların karışmaması için adları gruplayan bölge; standart kütüphane `std` içindedir.
adres	*address*	Bellekteki bir baytın sıra numarası; bir değişkenin adresi, kapladığı baytların ilkinin numarasıdır.
adres operatörü	*address-of operator*	Değişkenin adresini veren `&` işleci (ifadede kullanıldığında).
akıllı işaretçi	*smart pointer*	Yok edilirken gösterdiği bloğu kendiliğinden geri veren işaretçi nesnesi.
akış	*stream*	Veriyi bir yönde taşıyan yapı; `cout` ekrana, `cin` klavyeden veri akıtır.
akış durumu	*stream state*	Bir akışın okuma durumunu tutan bayraklar; `good`, `fail`, `eof` ve `bad` ile sorulur.
akış manipülatörü	*stream manipulator*	`cout` gibi bir akışa yazılarak biçimini değiştiren yapı (`boolalpha`, `fixed`, `setprecision`).
alan genişliği	*field width*	Bir değerin çıktıda ayrılan en az karakter sayısı; `setw` ile belirlenir.
algoritma	*algorithm*	Kapsayıcıdaki veri üzerinde sıralama, arama, sayma gibi iş yapan hazır fonksiyon.
anahtar	*key*	`map` içinde değere ulaşmak için kullanılan benzersiz değer.
anahtar sözcük	*keyword*	Dil için ayrılmış olduğundan değişken adı olarak kullanılamayan sözcük.
anlamlı basamak	*significant digit*	Sayının soldan ilk sıfırdan farklı rakamından başlayarak sayılan rakamı.
aralık tabanlı for	*range-based for*	Bir dizinin ya da vector'ün her elemanını indeks kullanmadan dolaşan for biçimi.
arayüz	*interface*	Yalnızca saf sanal fonksiyonlardan oluşan ve başka sınıfların uymak zorunda olduğu sözleşme sınıfıdır.
argüman	*argument*	Fonksiyon çağrılırken parantez içinde verilen ve parametreye eşlenen değer.
arkadaş fonksiyon	*friend function*	Sınıfın `friend` ile izin verdiği, özel üyelerine erişebilen üye olmayan fonksiyon.
asal sayı	*prime number*	1 ve kendisinden başka böleni olmayan, 1'den büyük doğal sayı.
ASCII	*ASCII*	İngilizce harfleri, rakamları ve noktalama işaretlerini 0-127 arası kodlarla gösteren karakter tablosu.
assembly	*assembly*	İşlemci komutlarının insanın okuyabileceği metin hali.

A	İngilizce	Açıklama
aşırı yükleme	*overloading*	Aynı adlı, parametre listeleri farklı birden çok fonksiyon tanımlama.
atama	*assignment*	Var olan bir değişkenin içeriğini `=` ile değiştirme işlemi.
ayraç	*delimiter*	Alanları birbirinden ayıran karakter; CSV dosyasında virgül, `getline`'da üçüncü parametre.

B	İngilizce	Açıklama
bağlama hatası	*linker error*	Bağlayıcının gereken bir parçayı bulamaması.
bağlayıcı	*linker*	Derlenmiş parçaları birleştirip çalıştırılabilir programı üreten araç.
başlık dosyası	*header file*	Kütüphane bildirimlerini içeren ve `#include` ile programa eklenen dosya.
başlık dosyası bölme	*header/source split*	Sınıf bildirimini `.h`, üye fonksiyon tanımlarını `.cpp` dosyasına yazma düzeni.
bayrak	*flag*	Bir durumun açık mı kapalı mı olduğunu tutan işaret; akışın durum bitleri (ör. `failbit`) ya da döngüde kullanılan `bool` değişken.
bayt	*byte*	8 bitten oluşan, belleğin adreslenebilen en küçük birimi.
bayt sırası	*endianness*	Çok baytlı bir sayının baytlarının bellekte hangi sırayla dizildiğini belirleyen düzen.
belirsiz çağrı	*ambiguous call*	Aşırı yüklenmiş fonksiyonlardan birden çoğunun eşit uygun olduğu, derlenmeyen çağrı.
bellek sızıntısı	*memory leak*	Öbekten alınan bloğun geri verilmeden adresinin kaybedilmesi.
bellekleme	*memoization*	Bir kez hesaplanan sonuçları bir tabloda saklayıp tekrar hesaplamak yerine oradan okuma tekniği.
Big-O gösterimi	*Big-O notation*	Veri büyüdükçe bir işlemin adım sayısının nasıl arttığını anlatan gösterim.
bildirim	*declaration*	Derleyiciye bir adın var olduğunu ve tipini söyleyen, kod içermeyen ifade (fonksiyonda prototip).
bileşik atama	*compound assignment*	Atama ve aritmetik işlemin tek adımda yapılması (`+=`, `*=` gibi).
bilimsel gösterim	*scientific notation*	Sayıyı bir katsayı ile 10'un kuvvetinin çarpımı olarak yazma biçimi (ör. `1.23457e+06`).
blok	*block*	Süslü parantezler arasındaki deyimler topluluğu.
bool	*bool*	Yalnızca `true` ya da `false` değeri taşıyan, 1 bayt yer kaplayan C++ türü.
boş deyim	*empty statement*	Tek başına `;` ile yazılan, hiçbir şey yapmayan deyim.
boş gövde	*empty body*	Döngü başlığının hemen ardından gelen noktalı virgülün oluşturduğu, hiçbir şey yapmayan gövde.
boş işaretçi	*null pointer*	Hiçbir yeri göstermeyen, `nullptr` değerine sahip işaretçi.
boşaltma	*flush*	Tampondaki bekleyen verinin hemen hedefine gönderilmesi.
boşluk karakteri	*whitespace*	Boşluk, sekme ve satır sonu gibi ekranda görünmeyen karakterler.
büyük/küçük harf duyarlılığı	*case sensitivity*	`cout` ile `Cout`'un farklı sayılması.

C	İngilizce	Açıklama
C tarzı dönüştürme	*C-style cast*	C dilinden gelen `(tip)ifade` biçimli, denetimsiz tip dönüşümü.
C tarzı karakter dizisi	*C-style string*	Sonunda `'\0'` bulunan `char` dizisi olarak tutulan eski tür metin.
camelCase	*camelCase*	İlk sözcüğün küçük, sonrakilerin baş harfinin büyük yazıldığı adlandırma biçimi (`ogrenciSayisi`).
case etiketi	*case label*	`switch` içinde, ifade bu sabite eşitse çalışmanın başlayacağı yeri gösteren etiket.
const doğruluğu	*const correctness*	Değiştirmeyen her şeyi `const` yapma alışkanlığı; `const` nesneler yalnız `const` üye fonksiyon çağırabilir.
const üye fonksiyon	*const member function*	Nesnenin üyelerini değiştirmeyeceğini taahhüt eden, parantezden sonra `const` yazılan üye fonksiyon.
CSV	*comma-separated values*	Her satırda bir kayıt bulunan, alanları virgülle ayrılmış metin dosyası biçimi.

Ç	İngilizce	Açıklama
çağrı ağacı	*call tree*	Bir özyinelemeli çağrının açtığı bütün alt çağrıları gösteren ağaç çizimi.
çağrı ifadesi	*function call*	Bir fonksiyonu adı ve parantezleriyle çağıran ifade; değeri fonksiyonun döndürdüğü değerdir.
çağrı yığını	*call stack*	Yığın çerçevelerinin üst üste dizildiği ve fonksiyon bitince silindiği bellek bölgesi.
çalışma zamanı hatası	*runtime error*	Program çalışırken ortaya çıkan ve derleyicinin önceden yakalayamadığı hata.
çalıştırılabilir dosya	*executable*	Derleme ve bağlama sonunda oluşan, doğrudan çalıştırılan dosya.
çarpım	*product*	Değerleri çarparak biriktiren, birden başlayan değişken.
çıkarma operatörü	*extraction operator*	`>>` operatörü; akıştan okunan değeri bir değişkene koyar.
çıkış kodu	*exit status*	`main`'in işletim sistemine döndürdüğü sayı; 0 başarı demektir.
çıkış parametresi	*output parameter*	Fonksiyonun sonucunu çağıranın değişkenine yazması için referansla geçirilen parametre.
çift	*pair*	Farklı tipte iki değeri `first` ve `second` üyeleriyle tutan şablon.
çift silme	*double delete*	Aynı öbek bloğunu iki kez geri vermeye çalışmak; tanımsız davranıştır.
çok biçimlilik	*polymorphism*	Aynı çağrının nesnenin gerçek tipine göre farklı davranmasıdır.
çoklu kalıtım	*multiple inheritance*	Bir sınıfın birden fazla temel sınıftan türetilmesidir.
çöp değer	*garbage value*	İlk değer verilmemiş değişkenin bulunduğu bellek bölgesindeki rastgele değer.

D	İngilizce	Açıklama
daralma	*narrowing*	Bir değerin hedef tipe sığmayacak biçimde küçük bir tipe çevrilmesi (ör. ondalıklı sayıyı tamsayıya çevirmek); veri kaybına yol açar.

D	İngilizce	Açıklama
De Morgan kuralları	*De Morgan's laws*	`!(a && b)` ile `!a || !b` ve `!(a || b)` ile `!a && !b` ifadelerinin eşdeğer olduğunu söyleyen kurallar.
değer	*value*	Bir değişkenin tuttuğu veri; `map`'te ise bir anahtara eşlenen veri (anahtar–değer çiftinin ikinci parçası).
değer ile geçirme	*pass by value*	Fonksiyonun argümanın kopyasıyla çalıştığı, çağıranın değişkeninin değişmediği geçirme biçimi.
değişken	*variable*	Bellekte adı olan ve içinde bir değer tutan kutu.
derin kopya	*deep copy*	İşaretçinin gösterdiği veriyi de yeni belleğe kopyalayan kopyalama.
derleme hatası	*compile-time error*	Kod dilin kurallarına uymadığında derleyicinin programı üretmeden verdiği hata.
derleyici	*compiler*	Kaynak kodu makine diline çeviren program; C++ için `g++` budur.
devreden yapıcı	*delegating constructor*	İşini aynı sınıfın başka bir yapıcısına devreden yapıcı.
deyim	*statement*	Programın noktalı virgülle biten tek bir komutu.
dinamik bağlama	*dynamic binding*	Sanal fonksiyon çağrısının çalışma zamanında nesnenin gerçek tipine göre bağlanmasıdır.
dinamik bellek ayırma	*dynamic memory allocation*	Bellek miktarını program çalışırken belirleyip öbekten istemek.
dinamik tip	*dynamic type*	Nesnenin gerçek tipidir; sanal fonksiyonların hangi sürümünün çalışacağını belirler.
dizi	*array*	Aynı tipten birçok değeri tek bir adla ve art arda bellekte tutan veri yapısı.
dizi bozunması	*array-to-pointer decay*	Dizi adının ifadelerde ilk elemanın adresine dönüşmesi.
doğrusal arama	*linear search*	Elemanları baştan sona tek tek gezerek bir değeri bulma yöntemi.
dolaylama	*dereference*	İşaretçinin gösterdiği yerdeki değere `*p` ile erişme işlemi.
dolgu baytı	*padding*	Derleyicinin hizalama için struct içine koyduğu, değeri belirsiz boş bayt.
dolgu karakteri	*fill character*	Alan genişliğinde boş kalan yeri dolduran karakter (varsayılan boşluk).
dosya	*file*	Diskte kalıcı olarak saklanan, adı olan veri topluluğu; program bittikten sonra da var olmaya devam eder.
dosya akışı	*file stream*	Bir dosyaya bağlı akış; `ofstream`, `ifstream` ve `fstream` ile açılır.
döngü	*loop*	Bir kod bloğunu koşul doğru olduğu sürece tekrar tekrar çalıştıran yapı.
döngü gövdesi	*loop body*	Döngünün her turda tekrarlanan kod bloğu.
dönüş tipi	*return type*	Fonksiyonun çağırana geri verdiği değerin tipi; değer vermiyorsa `void`.
durum bayrağı	*state flag*	Akışın durumunu gösteren işaret (`failbit`, `eofbit`, `badbit`); okuma başarısız olunca `failbit` açılır, `clear()` temizler.
düşme	*fall-through*	`break` olmadığı için çalışmanın bir alttaki `case`'e devam etmesi.

E	İngilizce	Açıklama
EBOB	*GCD*	İki sayıyı da tam bölen en büyük sayı (en büyük ortak bölen).
editör	*editor*	Kaynak kodu yazmaya yarayan program; kitapta VS Code kullanılır.
ekleme operatörü	*insertion operator*	`<<` işareti; sağındaki veriyi soldaki akışa gönderir.
EKOK	*LCM*	İki sayının da tam böldüğü en küçük pozitif sayı (en küçük ortak kat).
eleman	*element*	Dizinin ya da vector'ün tuttuğu değerlerden her biri.
elmas problemi	*diamond problem*	Çoklu kalıtımda ortak üst sınıfın iki kopyasının oluşup çağrıların belirsizleşmesi sorunudur.
else if merdiveni	*else-if ladder*	İkiden fazla yolu yukarıdan aşağıya sırayla denenen koşullarla ayıran `if`/`else if`/`else` dizisi.
en can sıkıcı ayrıştırma	*most vexing parse*	`Hesap h();` satırının nesne değil, fonksiyon bildirimi sayılması.
enum class	*scoped enumeration*	Adlandırılmış sabitlerden oluşan, `Tip::Ad` ile kullanılan ve `int`'e kendiliğinden dönüşmeyen tip.
EOF	*end of file*	Girdinin tükendiğini bildiren durum; klavyede Ctrl+D (macOS, Linux) ya da Ctrl+Z (Windows) ile verilir.
erişim belirteci	*access specifier*	Üyelerin sınıf dışından görünürlüğünü belirleyen `public`, `private`, `protected` anahtar sözcükleri.
ezme	*override*	Sanal fonksiyonu türetilmiş sınıfta aynı imzayla yeniden yazmaktır.

F	İngilizce	Açıklama
final belirteci	*final specifier*	Bir sınıfın türetilmesini ya da bir fonksiyonun ezilmesini yasaklayan anahtar sözcüktür.
fonksiyon	*function*	Adı olan, çağrıldığında çalışan kod parçası; program `main` fonksiyonundan başlar.
fonksiyon şablonu	*function template*	Farklı tiplerle çalışan fonksiyon tarifi; `template <typename T>` ile yazılır.

G	İngilizce	Açıklama
geçici nesne	*temporary object*	Bir ifade içinde yaşayıp kullanıldıktan sonra yok olan, adı olmayan nesne (`c + b` sonucu gibi).
geçiş	*pass*	Sıralama algoritmasında dizinin baştan sona bir kez taranması.
getter	*getter*	Bir `private` üyenin değerini dışarıya okutan üye fonksiyon.
gezen işaretçi	*walking pointer*	Dizinin elemanları arasında dolaşan, her adımda bir sonraki elemanı gösteren işaretçi.
girdi doğrulama	*input validation*	Kullanıcının girdiği değerin uygun olup olmadığını denetleyip gerekirse yeniden sorma.
gizleme	*hiding*	Yeniden tanımlamanın temel sınıf fonksiyonunu görünmez kılmasıdır.
global değişken	*global variable*	Hiçbir fonksiyonun içinde olmayan, tanımından sonra her yerden görünen değişken.
gölgeleme	*shadowing*	İç bloktaki bir adın, dış bloktaki aynı adı o blok boyunca görünmez kılması.
göreli yol	*relative path*	Programın çalıştığı klasöre göre çözülen dosya yolu (ör. `"notlar.txt"`).
gövde	*body*	Fonksiyonun süslü parantez içindeki deyimleri.
güçlü istisna güvencesi	*strong exception guarantee*	İşlem bir hatayla yarıda kalırsa nesnenin önceki tutarlı durumda kalmasını sağlayan güvence.
günlük	*log*	Programın çalışırken ne olduğunu sonradan okumak için yazılan kayıt.
güven ilkesi	*leap of faith*	Özyinelemeli çağrının doğru çalıştığını varsayıp yalnızca sonucu nasıl birleştireceğini düşünme yöntemi.

H	İngilizce	Açıklama
ham string	*raw string*	`R"(...)"` biçiminde yazılan string sabiti; içindeki ters bölüler kaçış dizisi sayılmaz.
Hanoi kuleleri	*Towers of Hanoi*	Diskleri bir çubuktan diğerine, büyük diski küçüğün üstüne koymadan taşıma bulmacası.
harita	*map*	Anahtar–değer çiftlerini anahtara göre sıralı tutan kapsayıcı.
has-a ilişkisi	*has-a*	"X bir Y'ye sahiptir" ilişkisidir; üye değişkenle kurulur.
hata akışı	*error stream*	Hata mesajları için ayrılmış standart akış (`cerr`).
hata durumu	*fail state*	Okuma başarısız olunca akışın girdiği, temizlenene kadar hiçbir okumanın çalışmadığı durum (failbit).
hata kodu	*error code*	Hatayı fonksiyonun dönüş değeriyle bildiren sayı (örneğin `-1`); geçerli bir sonuçla karışabilir.
hızlı üs	*fast exponentiation*	Üssü her çağrıda yarıya indirerek yaklaşık log n çağrıda üs hesaplayan özyinelemeli yöntem.
histogram	*histogram*	Değerlerin gruplara göre dağılımını sütunlar ya da yıldızlarla gösteren çıktı.

İ	İngilizce	Açıklama
iç içe döngü	*nested loop*	Gövdesinde başka bir döngü bulunan döngü; dış her turda iç döngü baştan çalışır.
iç içe if	*nested if*	Bir `if` bloğunun gövdesine başka bir `if` yazılmasıyla oluşan yapı.
ifade	*expression*	Değer üreten kod parçası (`a + b`, `x > 3`, `koşul ? a : b`).

İ	İngilizce	Açıklama
ikili arama	*binary search*	Sıralı veride ortadaki elemana bakıp aramayı her adımda yarıya indiren algoritma.
ikili dosya	*binary file*	Bellekteki baytların olduğu gibi kopyalandığı dosya; bir metin editöründe anlamsız semboller olarak görünür.
ikili kip	*binary mode*	`ios::binary` ile açma; baytlar olduğu gibi okunur ve yazılır, satır sonu çevrilmez.
ilişkisel operatör	*relational operator*	İki değeri karşılaştırıp `bool` sonuç veren operatör (`<`, `>`, `<=`, `>=`, `==`, `!=`).
ilk değer listesi	*initializer list*	Tanım satırında küme parantezi içinde verilen başlangıç değerleri.
ilk değer verme	*initialization*	Değişkeni açarken içine ilk değerini koyma işlemi.
ilk giren ilk çıkar	*FIFO (first in first out)*	İlk eklenen elemanın ilk çıktığı düzen; kuyruk veri yapısının çalışma biçimi.
imza	*signature*	Fonksiyonun adı ile parametre listesinin (sayı, tip, sıra) toplamı; dönüş tipi dahil değildir.
indeks	*index*	Bir elemanın dizideki sıra numarası; 0'dan başlar.
is-a ilişkisi	*is-a*	"X bir Y'dir" ilişkisidir; kalıtımın kullanılacağını gösterir.
istem	*prompt*	Kullanıcıdan değer istemek için okumadan önce yazdırılan soru metni.
istisna	*exception*	Bir hata olduğunda normal akışı durdurup hatayı yakalayan yere taşıyan mekanizma.
istisna güvenli kod	*exception-safe code*	İstisna fırlatılsa bile kaynak sızdırmayan ve tutarsız duruma düşmeyen kod.
istisna nesnesi	*exception object*	`throw` ile fırlatılan ve `catch` ile yakalanan, hatanın bilgisini taşıyan nesne.
istisna sınıfı	*exception class*	Programa özgü bir hatayı tanımlayan, çoğunlukla `runtime_error`'dan türeyen sınıf.
işaretçi	*pointer*	Değeri başka bir değişkenin adresi olan değişken.
işaretçi aritmetiği	*pointer arithmetic*	İşaretçiye tamsayı ekleyip çıkararak eleman eleman ilerleme ve geri gitme.
işaretçinin işaretçisi	*pointer to pointer*	Başka bir işaretçinin adresini tutan işaretçi (`int** pp`).
işaretli	*signed*	Negatif ve pozitif değer tutabilen tamsayı tipi (varsayılan).
işaretsiz	*unsigned*	Yalnız sıfır ve pozitif değer tutan tamsayı tipi; aynı bit sayısında pozitif değer aralığı yaklaşık iki katına çıkar.
işlenen	*operand*	Operatörlerin üzerinde işlem yaptığı değerler.
iteratör	*iterator*	Kapsayıcının elemanları arasında gezinmeyi sağlayan, işaretçiye benzeyen nesne.
izleme tablosu	*trace table*	Programın her adımında değişkenlerin değerlerini sütun sütun yazarak çıktıyı bulma yöntemi.

K	İngilizce	Açıklama
kabarcık sıralaması	*bubble sort*	Komşu elemanları karşılaştırıp yer değiştirerek en büyüğü sona taşıyan sıralama.
kaçış dizisi	*escape sequence*	Ters bölü ile başlayan, özel karakter ifade eden yazım (`\n`, `\t` gibi).

K	İngilizce	Açıklama
kalıtım	*inheritance*	Bir sınıfın başka bir sınıfın üyelerini devralmasıdır.
kalıtım ağacı	*inheritance tree*	Sınıfların hangi sınıftan türediğini gösteren ağaç biçimindeki şema (ör. `std::exception` ağacı).
kapasite	*capacity*	Vector'ün yeniden bellek ayırmadan tutabileceği eleman sayısı.
kapsam	*scope*	Bir değişkenin kullanılabildiği, tanımlandığı süslü parantez bloğu ile sınırlı bölge.
kapsam çözümleme operatörü	*scope resolution operator*	Bir adın hangi ad alanına, sınıfa ya da kapsama ait olduğunu belirten `::` operatörü (`std::cout`, `Sinif::uye`, gölgelenmiş global için `::x`).
kapsayıcı	*container*	Birden çok değeri tutan hazır veri yapısı; vector en çok kullanılanıdır.
kapsayıcı adaptörü	*container adapter*	Başka bir kapsayıcıyı kullanıp kısıtlı bir arayüz sunan yapı (`stack`, `queue`).
kapsülleme	*encapsulation*	Veriyi dışarıdan gizleyip yalnızca sınıfın sunduğu işlemlerle erişilmesini sağlama.
karakter kodlaması	*character encoding*	Karakterlerin bayt olarak nasıl saklanacağını belirleyen kural (ör. UTF-8).
karma tablo	*hash table*	Anahtardan hesaplanan bir değerle elemanı doğrudan bulan, sırasız veri yapısı.
kavram	*concept*	C++20'de şablon parametresine konan ve tipin hangi işlemleri desteklemesi gerektiğini belirten koşul.
kayıt	*record*	Dosyada birlikte tutulan, aynı yapıdaki veri parçası; CSV'de bir satır, ikili dosyada bir struct nesnesi.
kaynak dosyası	*source file*	Üye fonksiyonların gövdelerinin yazıldığı, `.cpp` uzantılı dosya.
kaynak kod	*source code*	İnsanın yazdığı, derleyicinin çevireceği metin dosyası (`.cpp`).
kendine atama	*self-assignment*	Bir nesnenin kendisine atanması (`a = a;`); atama operatörü bunu doğru ele almalıdır.
kısa devre değerlendirme	*short-circuit evaluation*	`&&` ve `||`'nin sonucu belli olur olmaz sağ tarafı atlayan değerlendirme yöntemi.
kip	*mode*	Dosya açılırken verilen, dosyayla ne yapılacağını belirleyen seçenek (`ios::app`, `ios::binary` gibi).
kod sayfası	*code page*	Konsolun baytları karaktere çevirirken kullandığı karakter tablosu.
konum göstergesi	*file position*	Dosyada bir sonraki okuma ya da yazmanın hangi bayttan başlayacağını tutan sayaç.
kopya atama operatörü	*copy assignment operator*	Var olan bir nesneye başka bir nesnenin değerini atayan `operator=` fonksiyonu.
kopya yapıcı	*copy constructor*	Aynı sınıftan bir nesneyle başlatılan yeni nesne için çağrılan, parametresi `const Sınıf&` olan yapıcı.
koruma satırları	*include guard*	Bir başlığın aynı dosyaya birden çok kez girmesini önleyen `#ifndef` / `#define` / `#endif` satırları.
korumalı üye	*protected member*	Sınıfın dışından erişilemeyen, ama türetilmiş sınıfların erişebildiği üyedir.
koşul	*condition*	Doğru ya da yanlış sonucu veren, programın hangi yoldan gideceğini belirleyen ifade.
köşegen	*diagonal*	Kare matriste sol üstten sağ alta (ana) ya da sağ üstten sol alta (ikincil) inen elemanlar.

K	İngilizce	Açıklama
kuyruk	*queue*	İlk giren ilk çıkar (FIFO) düzeniyle çalışan veri yapısı.
kuyruk özyinelemesi	*tail recursion*	Özyinelemeli çağrının fonksiyonun yaptığı son iş olduğu özyineleme biçimi.
küme	*set*	Tekrarsız ve sıralı elemanlar tutan kapsayıcı.
kütüphane	*library*	Hazır yazılmış, programlara eklenebilen kod topluluğu.
kütüphane fonksiyonu	*library function*	Standart kitaplıkta yazılmış, ilgili başlık dosyasını ekleyerek hazır kullanılan fonksiyon.

L	İngilizce	Açıklama
lambda ifadesi	*lambda expression*	Adı olmayan, çağrının yerinde yazılan küçük fonksiyon.
leksikografik karşılaştırma	*lexicographical comparison*	Metinleri soldan sağa karakter kodlarına göre sözlük sırasıyla karşılaştırma.

M	İngilizce	Açıklama
main fonksiyonu	*main function*	Programın çalışmaya başladığı, her C++ programında bulunan özel fonksiyon.
makine dili	*machine language*	İşlemcinin doğrudan anladığı, 0 ve 1'lerden oluşan komut dili.
manipülatör	*manipulator*	Akışın ayarını değiştiren, kendisi bir şey yazmayan özel değer (ör. `setw`, `fixed`).
mantık hatası	*logic error*	Programın çalışıp yanlış sonuç üretmesi.
mantıksal operatör	*logical operator*	Koşulları birleştiren ya da tersine çeviren operatör (`&&`, `||`, `!`).
matris	*matrix*	Satır ve sütunlardan oluşan, iki boyutlu dizi ile tutulan tablo.
menü döngüsü	*menu loop*	Seçenekleri gösterip kullanıcı çıkış seçene kadar tekrar eden döngü.
metin dosyası	*text file*	İçeriği okunabilir karakterlerden oluşan dosya; `.txt`, `.csv` ve `.cpp` dosyaları böyledir.
metin kipi	*text mode*	Dosyanın `ios::binary` olmadan açılması; Windows'ta satır sonu çevirisi yapılır.

N	İngilizce	Açıklama
nesne	*object*	Bir sınıftan üretilmiş somut örnek; kendi üye değişken değerlerini taşır.
nesne dilimleme	*object slicing*	Türetilmiş nesnenin temel sınıf nesnesine kopyalanırken türetilmiş kısmının kaybolmasıdır.
nesne dosyası	*object file*	Derleyicinin ürettiği, henüz kütüphanelerle birleşmemiş makine kodu dosyası.
nokta operatörü	*dot operator*	Nesnenin üyesine ulaşmayı sağlayan `.` operatörü (işaretçiyle `->`).
null karakter	*null character*	Kodu 0 olan `'\0'` karakteri; C tarzı metnin sonunu işaretler.

O	İngilizce	Açıklama
off-by-one	*off-by-one*	Döngünün bir tur eksik ya da fazla dönmesine yol açan sınır hatası.
ok operatörü	*arrow operator*	İşaretçinin gösterdiği nesnenin üyesine ulaşan `->` operatörü; `p->uye`, `(*p).uye` ile aynıdır.
operatör	*operator*	Değerler üzerinde işlem yapan sembol (örneğin `+` veya `%`).
operatör aşırı yükleme	*operator overloading*	Bir operatörü (`+`, `==`, `<<` gibi) sınıf için özel bir `operator` fonksiyonuyla yeniden tanımlama.

Ö	İngilizce	Açıklama
öbek	*heap*	Programın çalışma anında `new` ile bellek istediği, büyük ve elle yönetilen bellek bölgesi.
Öklid algoritması	*Euclidean algorithm*	İki sayının EBOB'unu `(a, b)` yerine `(b, a % b)` ikilisiyle tekrar tekrar hesaplayarak bulan yöntem.
ön ek	*prefix*	Operatörün işlenenden önce yazıldığı biçim; `++x` önce artırır, yeni değeri verir.
ön işlemci	*preprocessor*	Derlemeden önce `#include` gibi satırları işleyen ilk aşama.
öncelik	*precedence*	Birden fazla operatörün bulunduğu ifadelerde hangi işlemin önce yapılacağını belirleyen kurallar dizisi.
öncelikli okuma	*priming read*	Döngü koşulundan önce yapılan ilk okuma; döngü içinde okuma tur sonunda tekrarlanır.
örnekleme	*instantiation*	Şablonun kullanılan her tip için gerçek bir fonksiyona ya da sınıfa dönüştürülmesi.
örtük dönüşüm	*implicit conversion*	Derleyicinin veri tiplerini kendiliğinden uyumlu hale getirmesi.
özyineleme	*recursion*	Bir fonksiyonun kendi kendini, daha küçük bir problemle çağırarak çalışması.
özyineleme derinliği	*recursion depth*	Aynı anda çağrı yığınında bekleyen özyinelemeli çağrı (çerçeve) sayısı.
özyinelemeli durum	*recursive case*	Fonksiyonun problemi küçültüp kendini çağırdığı dal.

P	İngilizce	Açıklama
palindrom	*palindrome*	Tersten okunuşu kendisine eşit olan sözcük ya da cümle.
paralel diziler	*parallel arrays*	Aynı indeksi aynı kayda ait olan, birlikte güncellenen birden çok dizi ya da `vector`; bir eleman eklenip silinirken hepsini güncellemek gerektiği için kırılgandır.
parametre	*parameter*	Bir fonksiyonun dışarıdan aldığı değer.
PATH	*PATH*	İşletim sisteminin komutları aradığı klasörlerin listesi.
paylaşımlı işaretçi	*shared_ptr*	Sahiplerini sayan, son sahip yok olunca bloğu geri veren akıllı işaretçi.
program	*program*	Bilgisayara ne yapacağını adım adım anlatan komutlar dizisi.
prototip	*function prototype*	Fonksiyonun gövdesiz, `;` ile biten bildirimi; adını, dönüş tipini ve parametre tiplerini tanıtır.

R	İngilizce	Açıklama
RAII	*Resource Acquisition Is Initialization*	Kaynağı bir nesneye bağlayıp nesne yok olunca kaynağı otomatik bırakma fikri.
rastgele erişim	*random access*	Dosyayı baştan sona okumadan doğrudan istenen konuma gitme.
referans	*reference*	Var olan bir değişkene verilen ikinci ad (`int &r = x;`).
referans ile geçirme	*pass by reference*	Parametrenin çağıranın değişkeninin takma adı olduğu, fonksiyonun asıl değişkeni değiştirebildiği geçirme biçimi.
referans sayacı	*reference count*	Bir bloğu kaç `shared_ptr` nesnesinin tuttuğunu gösteren sayaç.

S	İngilizce	Açıklama
sabit	*constant*	Program boyunca değeri değişmeyen yapı (`const`).
sabit değer	*literal*	Koda doğrudan yazılan değer (örneğin `5` veya `'A'`).
sabit ifade	*constant expression*	Değeri derleme zamanında bilinen ifade (`3`, `2 + 3`, `const` sabit); `case` etiketi olabilir.
sabit noktalı gösterim	*fixed-point representation*	Ondalıklı bir değeri (para gibi) en küçük biriminin tamsayı katı olarak saklama yöntemi; örneğin tutarı kuruş cinsinden long long tutmak.
sabit referans	*const reference*	Kopyalamadan ama değiştirmeden geçirmeyi sağlayan `const &` parametresi.
saf sanal fonksiyon	*pure virtual function*	Gövdesi olmayan, imzanın sonuna `= 0` yazılarak tanımlanan sanal fonksiyondur.
sağ değer	*rvalue*	Atamanın yalnız sağ tarafında durabilen, kalıcı bir yeri olmayan geçici değer (`5`, `a + b`); sol değerin karşıtı.
sahip olmayan işaretçi	*non-owning pointer*	Bir nesneyi yalnızca göstermek için tutulan, silme görevi olmayan işaretçi; nesnenin sahibi başka bir akıllı işaretçidir.
sahiplik	*ownership*	Bir bloğu geri verme sorumluluğunun kimde olduğu.
salt okunur	*read-only*	Değeri ilk verildikten sonra değiştirilemeyen (const) değişken.
sanal fonksiyon	*virtual function*	Hangi sürümün çalışacağı çalışma zamanında, nesnenin gerçek tipine göre seçilen fonksiyondur.
sanal fonksiyon tablosu	*vtable*	Sanal fonksiyonu olan sınıflar için derleyicinin hazırladığı, doğru fonksiyonun adresini tutan tablodur.
sanal yıkıcı	*virtual destructor*	Temel sınıf işaretçisiyle silinen nesnenin bütün yıkıcılarının doğru sırayla çağrılmasını sağlayan yıkıcıdır.
sarkan else	*dangling else*	Bir `else`'in girintisine değil, kendinden önceki en yakın `if`'e bağlandığı durum.
sarkan işaretçi	*dangling pointer*	Artık var olmayan bir nesnenin adresini tutan işaretçi.
sarkan referans	*dangling reference*	Artık var olmayan bir değişkene (örneğin bitmiş bir fonksiyonun yerel değişkenine) bağlı kalmış referans; kullanmak tanımsız davranıştır.
satır içi fonksiyon	*inline function*	Derleyiciye çağrı yerine gövdeyi yerleştirebileceğini söyleyen `inline` fonksiyon.
sayaç	*counter*	Bir şeyin kaç kez olduğunu sayan, sıfırdan başlayan değişken.
seçmeli sıralama	*selection sort*	Her turda sıralanmamış kısmın en küçüğünü bulup başa yerleştiren sıralama.

S	İngilizce	Açıklama
sentinel	*sentinel value*	Girdinin bittiğini bildiren, veri olamayacak özel değer.
setter	*setter*	Bir `private` üyeyi, doğrulama yaparak değiştiren üye fonksiyon.
sıfırlar kuralı	*rule of zero*	Kaynağı `vector` ve `string` gibi hazır sınıflara bırakıp kopya ve yıkıcı fonksiyonları hiç yazmama yaklaşımı.
sığ kopya	*shallow copy*	Üyeleri adresleriyle birlikte olduğu gibi kopyalama; işaretçi üyeler aynı belleği paylaşır.
sınıf	*class*	Veriyi ve o veriyle çalışan fonksiyonları bir arada tutan kullanıcı tanımlı tip.
sınıf içi üye ilk değeri	*default member initializer*	Üyenin sınıf tanımında verilen ilk değeri (`double ortalama{0};`); yapıcı listesi değer vermezse kullanılır.
sınıf şablonu	*class template*	Farklı tiplerle çalışan sınıf tarifi; kullanırken tip yazılır (`Kutu<int>`).
sınır dışı erişim	*out-of-bounds access*	Geçerli indeks aralığının dışındaki bir elemana ulaşmaya çalışmak; tanımsız davranıştır.
sınırın bir ötesi	*one-past-the-end*	Dizinin son elemanından bir sonraki adres; sınır olarak kullanılır, okunmaz.
sıralama	*sorting*	Elemanları küçükten büyüğe (ya da tersine) dizme problemi.
sihirli sayı	*magic number*	Kodun içinde ne anlama geldiği belli olmadan kullanılan, sabit olarak tanımlanmamış sayılar.
silinmiş fonksiyon	*deleted function*	Bildiriminin sonuna `= delete` yazılarak yasaklanmış, çağrılması derleme hatası veren fonksiyon (ör. kopya yapıcı).
size_t	*size_t*	Boyut ve indeks tutmak için kullanılan işaretsiz tamsayı tipi.
sol değer	*lvalue*	Atamanın sol tarafında durabilen, bellekte bir yeri gösteren ifade (genellikle bir değişken).
son ek	*postfix*	Operatörün işlenenden sonra yazıldığı biçim; `x++` eski değeri verir, sonra artırır.
son giren ilk çıkar	*LIFO (last in first out)*	Son eklenen elemanın ilk çıktığı düzen; yığın veri yapısının çalışma biçimi.
sona ekleme	*append*	Yazılanı dosyanın sonuna ekleyen ve eski içeriği koruyan açma kipi (`ios::app`).
sonsuz döngü	*infinite loop*	Koşulu hiç yanlış olmayan ve `break` ile çıkılmadıkça bitmeyen döngü.
soyut sınıf	*abstract class*	En az bir saf sanal fonksiyonu olan ve nesnesi yaratılamayan sınıftır.
sözde rastgele sayı	*pseudo-random number*	Bir formülle üretilen ve rastgele görünen sayı; aynı tohum aynı diziyi verir.
sözdizimi	*syntax*	Dilin yazım kuralları.
standart	*standard*	Dilin kurallarını belirleyen, yıla göre adlandırılan resmî belge (C++17 gibi).
standart kütüphane	*standard library*	C++ ile birlikte gelen hazır kod topluluğu (`cout`, `string` vb.).
static üye	*static member*	Nesneye değil sınıfa ait olan, tüm nesneler arasında paylaşılan üye.
static yerel değişken	*static local variable*	Çağrılar arasında değerini koruyan, ilk değeri yalnızca bir kez verilen yerel değişken.

S	İngilizce	Açıklama
statik bağlama	*static binding*	Çağrının derleme sırasında değişkenin ya da işaretçinin tipine göre bağlanmasıdır.
statik tip	*static type*	Değişkenin tanımlandığı tiptir; derleme zamanında hangi üyelerin çağrılabileceğini belirler.
STL	*STL (Standard Template Library)*	Standart C++ kütüphanesinin kapsayıcı, iteratör ve algoritmalardan oluşan şablon tabanlı bölümü.
string	*string*	Metni kendisi yöneten, `<string>` başlığındaki hazır metin tipi.
string sabiti	*string literal*	Çift tırnak içinde doğrudan koda yazılan metin.
string::npos	*string::npos*	`find` aranan şeyi bulamayınca döndürdüğü, `size_t`'nin en büyük değeri.
stringstream	*stringstream*	Kaynağı klavye değil bir string olan akış; `>>` ve `getline` ile string'i parçalamaya yarar.
süslü parantezle ilk değer verme	*uniform initialization*	Değişkeni {} ile başlatma; daralma (narrowing) hatalarını derleme zamanında yakalar.
switch deyimi	*switch statement*	Bir tamsayı ifadesini sabit `case` etiketleriyle karşılaştırıp uygun yerden çalışmaya başlayan çok yollu seçim deyimi.

Ş	İngilizce	Açıklama
şablon	*template*	Tipi yer tutucu bir parametreyle yazılan, derleyicinin gerçek tiple kod üretmesini sağlayan tarif.
şablon parametresi	*template parameter*	Şablonun `template <typename T>` ile tanıtılan tip yer tutucusu.

T	İngilizce	Açıklama
takas	*swap*	İki değişkenin değerlerinin karşılıklı değiştirilmesi.
takma ad	*alias*	Var olan bir değişkene verilen ikinci ad; referansın temel anlamı.
tam eşleşme	*exact match*	Argümanın tipinin parametre tipiyle birebir aynı olduğu durum; aşırı yüklemede en öncelikli seçimdir.
tam yol	*absolute path*	Kök klasörden başlayarak yazılan, nerede çalışılırsa çalışılsın aynı dosyayı gösteren yol.
tampon	*buffer*	Veriyi hedefine gönderilmeden önce geçici olarak biriktiren bellek bölgesi.
tamsayı	*integer*	Ondalık kısmı olmayan sayı (ör. 5, -3).
tamsayı bölmesi	*integer division*	İki tamsayının bölünmesi sonucu kesirli kısmın atılarak yine bir tamsayı elde edilmesi.
tanım	*definition*	Fonksiyonun gövdesiyle birlikte yazılmış tam hâli; gövdesiz hâli bildirimdir. Değişkenler için tanımlama denir.
tanımlama	*definition*	Derleyiciye belirli tipte ve adda bir değişken açmasını (bellekte yer ayırmasını) söyleyen ifade: `int yas;`. Fonksiyonlar için tanım denir.
tanımsız davranış	*undefined behavior*	Standardın sonucunu belirlemediği durum; programın ne yapacağı garanti değildir.
taşıma	*move*	Sahipliği bir nesneden diğerine devretmek; `unique_ptr`'de eski nesne boş kalır.

T	İngilizce	Açıklama
taşma	*overflow*	Hesaplanan bir değerin, değişkenin tipinin sınırlarını aşması sonucu oluşan hata.
tek sahipli işaretçi	*unique_ptr*	Bloğun tek sahibi olan, kopyalanamayan ama taşınabilen akıllı işaretçi.
tekli operatör	*unary operator*	Tek işleneni olan operatör (`!`, tekli `-`, `++x`).
temel durum	*base case*	Özyinelemenin durduğu, kendini çağırmadan sonuç veren durum.
temel sınıf	*base class*	Ortak üyelerin yazıldığı, başka sınıfların devraldığı sınıftır.
terfi	*promotion*	Küçük bir tipin (`char`, `bool`, `float`) kayıpsızca daha büyük tipe (`int`, `double`) çevrilmesi.
ters çevirme	*reversal*	İki indeksle dizinin iki ucundan ortaya doğru ilerleyip elemanları yer değiştirerek diziyi tersine çevirme işlemi.
ters iteratör	*reverse iterator*	`rbegin()` ve `rend()` ile elde edilen, `++` ile geriye giden iteratör.
this	*this*	Üye fonksiyonun içinde, fonksiyonun çağrıldığı nesneyi gösteren gizli işaretçi.
tip çıkarımı	*type deduction*	Derleyicinin tipi kendisinin bulması: `auto` ile tanımlanan değişkende ilk değerden, şablonda (Bölüm 13) fonksiyon argümanlarından.
tohum	*seed*	Sözde rastgele sayı üretecinin başlangıç değeri (`srand`).
tolerans	*tolerance*	İki ondalıklı sayının "eşit" sayılması için aralarındaki farkın altında kalması gereken küçük sınır.
toplam	*sum*	Değerleri biriktiren, sıfırdan başlayan değişken.
transpoz	*transpose*	Bir matrisin satırlarını sütun yapan dönüşüm: t[j][i] = m[i][j].
türetilmiş sınıf	*derived class*	Bir temel sınıftan kalıtım yoluyla üretilen ve temel sınıfın üyelerine sahip olan sınıftır.

U	İngilizce	Açıklama
uyarı	*warning*	Derlemeyi durdurmayan, olası bir hataya işaret eden derleyici mesajı.
uzay gemisi operatörü	*spaceship operator*	C++20'de `<=>` ile üç yönlü karşılaştırma yapan operatör.

Ü	İngilizce	Açıklama
üçler kuralı	*rule of three*	Yıkıcı, kopya yapıcı ve kopya atama operatöründen birini yazıyorsan üçünü de yazman gerektiği kural.
üçlü operatör	*ternary operator*	Koşula göre iki değerden birini seçen, üç işlenenli `?:` operatörü.
üstel büyüme	*exponential growth*	Girdi bir arttığında işin kabaca sabit bir katsayıyla çarpılarak büyümesi.
üye değişken	*member variable*	Sınıfın içinde tanımlanan, her nesnenin kendi kopyasını taşıdığı değişken.
üye fonksiyon	*member function*	Sınıfın içinde tanımlanan ve çağrıldığı nesnenin üyelerini kullanan fonksiyon.
üye ilk değer listesi	*member initializer list*	Yapıcının `:` sonrasında üyeleri doğdukları anda başlatan listesi; `const` ve referans üyeler için zorunludur.
üye olmayan fonksiyon	*non-member function*	Bir sınıfın üyesi olmayan, sıradan fonksiyon; sol işleneni başka tip olan operatörler için kullanılır.

V	İngilizce	Açıklama
varsayılan parametre	*default argument*	Çağrıda atlandığında kullanılan, parametre listesinde verilmiş ön tanımlı değer.
varsayılan yapıcı	*default constructor*	Argüman almadan çağrılabilen yapıcı; kendi yapıcını yazınca derleyici üretmez.
vector	*vector*	Elemanları art arda tutan, eleman sayısı çalışma anında büyüyüp küçülebilen standart kapsayıcı.

Y	İngilizce	Açıklama
yakalama listesi	*capture list*	Lambda ifadesinin dışındaki hangi değişkenleri kullanabileceğini belirleyen köşeli parantez içi.
yakalanmayan istisna	*uncaught exception*	Hiçbir `catch` bloğunun yakalamadığı istisna; program `std::terminate` ile sonlanır.
yaklaşma	*convergence*	Her özyinelemeli çağrının temel duruma biraz daha yaklaşması gereken koşul.
yan etki	*side effect*	İfadenin değer üretmenin yanında bir değişkeni değiştirmesi (`++b` gibi).
yapı	*struct*	Birbirine ait verileri tek tipte toplayan, üyeleri varsayılan olarak `public` olan sınıf türü (`struct`).
yapıcı	*constructor*	Nesne oluşturulurken otomatik çalışan, adı sınıfla aynı ve dönüş tipi olmayan üye fonksiyon.
yapısal bağlama	*structured binding*	Bir çifti ya da yapıyı tek satırda birden çok değişkene açan C++17 özelliği (`auto [a, b] = p;`).
yarı açık aralık	*half-open range*	Başlangıcın dahil, bitişin dahil olmadığı `[b, e)` biçimindeki aralık.
yeniden fırlatma	*rethrow*	`catch` bloğunda `throw;` yazarak yakalanan istisnayı olduğu gibi yukarı göndermek.
yeniden tanımlama	*redefinition*	Türetilmiş sınıfta temel sınıftakiyle aynı adlı ve parametreli fonksiyon yazmaktır.
yerel değişken	*local variable*	Bir fonksiyon ya da blok içinde tanımlanan, yalnızca orada görünen ve blok bitince yok olan değişken.
yığın	*stack*	Yerel değişkenlerin yaşadığı, küçük ve otomatik yönetilen bellek bölgesi.
yığın çerçevesi	*stack frame*	Her fonksiyon çağrısı için açılan; parametreleri, yerel değişkenleri ve dönüş bilgisini tutan bellek bloğu.
yığın çözülmesi	*stack unwinding*	İstisna fırlatılınca fonksiyonların ters sırayla terk edilip yerel nesnelerin yıkıcılarının çağrılması.
yığın taşması	*stack overflow*	Yığın bölgesinin kapasitesinin aşılması ve programın çökmesi.
yığın veri yapısı	*stack*	Son giren ilk çıkar (LIFO) düzeniyle çalışan veri yapısı; bellek bölgesi olan yığından farklıdır.
yıkıcı	*destructor*	Nesne yok edilirken otomatik çalışan, `~` ile başlayan ve kaynakları geri veren üye fonksiyon.
Yoda koşulu	*Yoda condition*	Karşılaştırmada sabiti sola yazma alışkanlığı (`5 == x`); `=` ile `==` karışıklığını derleme hatasına çevirir.
yorum	*comment*	Derleyicinin yok saydığı, okuyucuya yönelik not.
yorumlayıcı	*interpreter*	Kodu doğrudan okuyup çalıştıran program; C++ bu yolla çalışmaz.

Y	İngilizce	Açıklama
yönlendirme	*redirection*	Kabukta `>`, `<` ve `2>` ile çıktının ya da girdinin ekran ve klavye yerine dosyaya bağlanması.
yukarı dönüşüm	*upcasting*	Türetilmiş sınıf nesnesine temel sınıf işaretçisi ya da referansı üzerinden bakmaktır; cast gerektirmez.
yüksek seviyeli programlama dili	*high-level language*	İnsanın okuyabileceği, makine dilinden bağımsız programlama dili.

Z	İngilizce	Açıklama
zincirleme	*chaining*	Bir işlemin sonucunun bir sonrakine girdi olduğu yazım biçimi; `cout << a << b` buna örnektir.
zincirleme çağrı	*method chaining*	Üye fonksiyonun `*this` döndürmesiyle çağrıların arka arkaya yazılması.

EK C

Sık Karşılaşılan Hata Mesajları

Bu ekte, C++ öğrenirken g++'tan en sık alacağın mesajları topladık. Derleme hataları ve uyarılar, onları **gerçekten** üreten küçük programlarla gösteriliyor; altındaki çıktı g++ 16 ile alınmış gerçek mesajdır. Bağlama hataları (`undefined reference`) macOS'ta farklı yazıldığı için elle verilmiştir ve bunu ilgili yerde belirttik. Her maddede önce mesaj, sonra "ne demek?", en sonda "çözüm" var. Mesajın hangi konuyla ilgili olduğunu bulmak için maddedeki bölüm atıflarına bak.

Bir g++ mesajı şu kalıptadır:

```
dosya.cpp:SATIR:SÜTUN: error: MESAJ
    6 |     cout << "Merhaba" << '\n'
      |                           ^   ← g++'nın takıldığı yer
```

Başındaki sözcük mesajın ağırlığını söyler:

Sözcük	Anlamı	Sonuç
`error`	Kod C++ kurallarına uymuyor	Program **derlenmez**, çalıştırılabilir dosya oluşmaz
`warning`	Kod derlenir ama büyük olasılıkla bir yanlışlık var	Program çalışır, ama sonucu güvenilmez olabilir
`note`	Bir önceki mesaja ek bilgi (ör. "tanım burada")	Tek başına bir sorun değildir

`In function 'int main()':` gibi bir satır, mesajın hangi fonksiyonun içinde olduğunu söyler; `In instantiation of ...` ve `required from here` satırlarını şablon bölümünde açıklayacağız. Uyarıların sonundaki `[-Wunused-variable]` gibi köşeli parantezli etiket, uyarıyı hangi `-W` seçeneğinin ürettiğini gösterir; arama motoruna yapıştırınca işe yarar.

> **İpucu: Hata mesajı okuma alışkanlığı**
>
> - Derleme hatalarında **her zaman ilk hatadan başla**. Tek bir eksik `;`, `}` ya da `"`, arkasından sahte hatalar zinciri doğurabilir; ilkini düzeltip yeniden derle.
> - Satır numarası, hatanın **fark edildiği** yeri gösterir; asıl neden bir üst satırda olabilir. Bulamazsan yukarıya doğru bak.
> - Uyarıyı hata gibi ciddiye al: `-Wall -Wextra` ile derle (Bölüm 1) ve uyarısız derlenene kadar bırakma.

> **Not: Senin derleyicin farklı yazabilir**
>
> - Çıktılardaki `/opt/homebrew/...` gibi kütüphane yolları bu kitabın üretildiği bilgisayara aittir. Windows'ta MinGW kullanıyorsan aynı satırlar `C:/msys64/ucrt64/include/c++/...` gibi başlar; anlamları aynıdır.
> - g++ sürümü eskiyse mesajların biçimi biraz değişir (ör. "there are N candidates" yerine uzun bir `candidate:` listesi). Cümlelerin kendisi hemen hemen aynı kalır.
> - **macOS'ta `g++` yazınca çalışan program aslında Apple clang'dir** (Bölüm 1). Aynı hatalar clang'de başka cümlelerle yazılır: `expected ';' after expression`, `use of undeclared identifier 'Toplam'; did you mean 'toplam'?`, `use of undeclared identifier 'cout'; did you mean 'std::cout'?`. Anlam aynıdır; bu ekte g++ cümlelerini veriyoruz çünkü sınavlarda ve Windows'ta karşına çıkacak olan odur.
> - Mesajlar `-std=c++17 -Wall -Wextra -Wpedantic` ile alınmıştır.

Derleme hataları

`expected ';' before 'return'`

Bölüm 1'de gördüğün en klasik hatadır.

```cpp
#include <iostream>

using namespace std;

int main() {
    cout << "Merhaba" << '\n'
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
noktaliVirgul.cpp: In function 'int main()':
noktaliVirgul.cpp:6:30: error: expected ';' before 'return'
    6 |     cout << "Merhaba" << '\n'
      |                               ^
      |                               ;
    7 |     return 0;
      |     ~~~~~~
```

Ne demek? Komutun sonundaki noktalı virgül eksik. g++ bunu, sonraki komuta (`return`) gelince fark eder ve eksik `;`'nin gireceği yeri `^` ile ve altındaki `;` ile gösterir. Gerçek konum, mesajda yazan satırın bir üstü olabilir; bu örnekte g++ 16 doğru satırı gösteriyor.

Çözüm:

- `^` işaretinin gösterdiği yere `;` koy. Sınıf tanımının kapanış `}` karakterinden sonra da `;` gerekir (Bölüm 11); unutulursa mesaj `expected ';' after class definition` olur.
- Benzer mesajlar: `expected ')' before ...` (kapanmayan parantez), `expected primary-expression before ...` (operatörden sonra operand eksik).

`expected '}' at end of input`

```cpp
#include <iostream>

using namespace std;

int main() {
    int sayi{5};
    if (sayi > 3) {
        cout << "büyük" << '\n';
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
kapanmayanSus.cpp: In function 'int main()':
kapanmayanSus.cpp:10:2: error: expected '}' at end of input
   10 | }
      |  ^
kapanmayanSus.cpp:5:12: note: to match this '{'
    5 | int main() {
      |            ^
```

Ne demek? Dosyanın sonuna gelindi ama açık bir `{` hâlâ kapanmamış. Yukarıda `if`'in `}` karakteri eksik; son `}` ona gitti, `main` açık kaldı. Altındaki `note: to match this '{'`, kapanmayan süslü parantezin yerini gösterir; hata ve çözüm bu iki satırın arasında yatar.

Çözüm:

- Girintilere bak: her `{`'nin altına, aynı girintide bir `}` düşmeli. Hata satırı hep dosyanın sonunu gösterir; asıl eksiklik daha yukarıdadır.
- VS Code'da kodu otomatik biçimlendir (`Shift+Alt+F`); girinti kayan yer, eksik `}` ya da fazla `{`'nin yeridir. Eşleşen parantezler renkli çizilir.

`missing terminating " character`

```cpp
#include <iostream>

using namespace std;

int main() {
    cout << "Merhaba\n;
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
tirnakEksik.cpp:6:13: warning: missing terminating " character
    6 |     cout << "Merhaba\n;
      |             ^
tirnakEksik.cpp:6:13: error: missing terminating " character
    6 |     cout << "Merhaba\n;
      |             ^~~~~~~~~~
tirnakEksik.cpp: In function 'int main()':
tirnakEksik.cpp:7:5: error: expected primary-expression before 'return'
    7 |     return 0;
      |     ^~~~~~
```

Ne demek? Bir metin (string sabiti, Bölüm 1) çift tırnakla açılmış ama aynı satırda kapanmamış. Metin satır sonuna kadar sürer; C++'ta bir string sabiti satır sonunu geçemez. Arkasından gelen `expected primary-expression before 'return'` sahte bir hatadır: derleyici bozuk satırdan sonra `return`'ü beklemiyordu. İlk hatayı düzeltince o da kaybolur.

Çözüm: Kapanış tırnağını ekle: `cout << "Merhaba\n";`. Metnin içinde tırnak karakteri gerekiyorsa `\"` yaz (Bölüm 3'teki kaçış dizileri).

`'Toplam' was not declared in this scope`

```cpp
#include <iostream>

using namespace std;

int main() {
    int toplam{10};
    cout << toplam << ' ' << Toplam << '\n';
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
tanimsizAd.cpp: In function 'int main()':
tanimsizAd.cpp:7:30: error: 'Toplam' was not declared in this scope; did you mean 'toplam'?
    7 |     cout << toplam << ' ' << Toplam << '\n';
      |                              ^~~~~~
      |                              toplam
```

Ne demek? "Bu adı tanımıyorum." g++ çoğu zaman `did you mean 'toplam'?` diye en yakın adı önerir ve altına önerdiği yazımı çizer. Bu, C++'ın en sık hatasıdır ve üç olası nedeni vardır:

- Ad yanlış yazılmış ya da büyük/küçük harf farklı (`Toplam` ≠ `toplam`, Bölüm 2).
- Değişken **kapsam dışında**: bir `for` ya da `{ }` bloğunun içinde tanımlanıp dışında kullanılmış (Bölüm 5, 6).
- Değişken ya da fonksiyon, kullanıldığı satırdan **sonra** tanımlanmış. Kendi fonksiyonunu `main`'den sonra yazdıysan `main`'den önce **prototipini** yaz: `int kare(int x);` (Bölüm 6).

Çözüm: Adı düzelt, ya da tanımı kullanımdan önceye ve doğru kapsama taşı.

`redeclaration of 'int x'`

```cpp
int main() {
    int x{5};
    int x{7};
    return x;
}
```

Derleyici çıktısı – program derlenmez

```
yenidenTanim.cpp: In function 'int main()':
yenidenTanim.cpp:3:9: error: redeclaration of 'int x'
    3 |     int x{7};
      |         ^
yenidenTanim.cpp:2:9: note: 'int x' previously declared here
    2 |     int x{5};
      |         ^
```

Ne demek? Aynı kapsamda (Bölüm 6) aynı adla iki değişken tanımlanamaz. İkinci `int x` yeni bir değişken açmaya çalışıyor; `note: 'int x' previously declared here` ilk tanımın yerini gösterir. Global değişkenlerde aynı hata `redefinition of 'int x'` diye yazılır.

Çözüm: Değeri değiştirmek istiyorsan tipi yazmadan ata: `x = 7;`. Gerçekten ikinci bir değişken gerekiyorsa başka bir ad ver.

`'cout' was not declared in this scope` (`std::` ve `#include`)

Aynı mesaj, `cout` ve `string` gibi standart kütüphane adlarında iki farklı nedenle çıkar. Birincisi `using namespace std;` satırını (ya da `std::` önekini) unutmaktır (Bölüm 1):

```
#include <iostream>

int main() {
    cout << "Merhaba" << '\n';
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
stdYok.cpp: In function 'int main()':
stdYok.cpp:4:5: error: 'cout' was not declared in this scope; did you mean 'std::cout'?
    4 |     cout << "Merhaba" << '\n';
      |     ^~~~
      |     std::cout
In file included from stdYok.cpp:1:
/opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/iostream:65:18: note: 'std::cout' declared here
   65 |   extern ostream cout;          ///< Linked to standard output
      |                  ^~~~
```

İkincisi ilgili başlık dosyasını eklememektir:

```
using namespace std;

int main() {
    string ad{"Ali"};
    cout << ad << '\n';
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
includeYok.cpp: In function 'int main()':
includeYok.cpp:4:5: error: 'string' was not declared in this scope
    4 |     string ad{"Ali"};
      |     ^~~~~~
includeYok.cpp:1:1: note: 'std::string' is defined in header '<string>'; this is probably fixable by
adding '#include <string>'
  +++ |+#include <string>
    1 | using namespace std;
includeYok.cpp:5:5: error: 'cout' was not declared in this scope
    5 |     cout << ad << '\n';
      |     ^~~~
includeYok.cpp:1:1: note: 'std::cout' is defined in header '<iostream>'; this is probably fixable by
adding '#include <iostream>'
  +++ |+#include <iostream>
    1 | using namespace std;
includeYok.cpp:5:13: error: 'ad' was not declared in this scope
    5 |     cout << ad << '\n';
      |             ^~
```

Ne demek? Birinci örnekte `<iostream>` eklenmiş, ama `cout` aslında `std` ad alanının içindedir; g++ bunu `did you mean 'std::cout'?` diyerek ve `std::cout declared here` notuyla söyler. İkinci örnekte `using namespace std;` var ama başlık dosyası yok; g++ `note` satırında hangi başlık dosyasının gerektiğini ve eklenecek satırı (`+++ |+#include <string>`) hazır yazar. İkinci örnekte üçüncü bir hata daha vardır (`'ad' was not declared`); o, ilk hatanın **zincirleme sonucudur** (`string ad` tanımlanamayınca `ad` da yoktur) ve ilk ikisini düzeltince kendiliğinden kaybolur.

Çözüm:

- `std::cout` yaz ya da dosyanın başına `using namespace std;` ekle.
- Başlık dosyası eşlemesi: `cout`, `cin`, `endl` için `<iostream>`; `string`, `getline` için `<string>`; `vector` için `<vector>`; `setw`, `setprecision` için `<iomanip>`; `sqrt`, `pow` için `<cmath>`; `ifstream`, `ofstream` için `<fstream>`; `unique_ptr`, `make_unique` için `<memory>`.
- Bir başlık, başka bir başlığı içerdiği için `#include` eksikken kod bazen yine de derlenir (`<iostream>` çoğu g++ sürümünde `<string>`'i de getirir). Böyle koda güvenme: kullandığın her şeyin başlığını kendin yaz; yoksa kod başka bilgisayarda derlenmez.

`'cout' does not name a type`

```cpp
#include <iostream>

using namespace std;

cout << "Merhaba" << '\n';

int main() {
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
globalKomut.cpp:5:1: error: 'cout' does not name a type
    5 | cout << "Merhaba" << '\n';
      | ^~~~
```

Ne demek? `{ }` dışında, fonksiyonların arasında yalnızca **tanımlar** durabilir: fonksiyon, sınıf, global değişken. `cout << ...` bir komuttur ve bir fonksiyonun içinde olmalıdır. g++ dışarıda bir tip beklediği için "`cout` bir tip adı değil" der.

Çözüm: Komutu `main` ya da başka bir fonksiyonun içine al. Fazladan bir `}` bir fonksiyonu erken kapatmışsa da aynı mesajı alırsın; süslü parantezleri say.

`fatal error: iostrem: No such file or directory`

```cpp
#include <iostrem>

int main() {
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
basliYok.cpp:1:10: fatal error: iostrem: No such file or directory
    1 | #include <iostrem>
      |          ^~~~~~~~
compilation terminated.
```

Ne demek? `#include` satırındaki başlık dosyası bulunamadı. Burada `iostream` yanlış yazılmış. `fatal error` derlemenin **hemen durduğunu** söyler (`compilation terminated.`); başka hata aramaz. Kendi yazdığın başlıkta (`#include "hesap.h"`, Bölüm 11) aynı mesaj, dosyanın adı ya da klasörü yanlışsa çıkar.

Çözüm: Başlığın adını düzelt (`<iostream>`, `<string>`, `<vector>`...). Kendi başlığın için dosyanın `.cpp` ile aynı klasörde olduğunu ve adının birebir aynı yazıldığını kontrol et.

`no match for 'operator<<'`

Bölüm 13'te `cout << nesne` yazabilmek için `<<` operatörünü kendin yüklemen gerektiğini gördün. Yüklemezsen:

```cpp
#include <iostream>

using namespace std;

class Nokta {
public:
    int x{1};
    int y{2};
};

int main() {
    Nokta p;
    cout << p << '\n';
    return 0;
}
```

g++ 16 bu hatayı 300'den fazla satırla yazar, çünkü `cout << ...` için yüklenmiş her `<<` operatörünü tek tek "aday" (candidate) diye sıralar. Sayfayı doldurmasın diye yalnız ilk satırları gösteriyoruz; önemli olan da onlardır:

Derleyici çıktısı – program derlenmez

```
operatorYok.cpp: In function 'int main()':
operatorYok.cpp:13:10: error: no match for 'operator<<' (operand types are 'std::ostream' {aka
'std::basic_ostream<char>'} and 'Nokta')
   13 |     cout << p << '\n';
      |     ~~~~ ^~ ~
      |     |       |
      |     |       Nokta
      |     std::ostream {aka std::basic_ostream<char>}
  * there are 33 candidates
... (g++ 300 satır daha yazar; burada kısaltıldı)
```

Ne demek? "`<<` operatörünün sol tarafı `std::ostream` (yani `cout`), sağ tarafı `Nokta` olan bir sürümü bulamadım." Parantez içindeki `operand types are` (işlenen tipleri) ifadesi, eksik olan işlemi tam olarak tarif eder. Sonraki yüzlerce `candidate` satırı "bunlara baktım, hiçbiri uymadı" demektir; **okuma, geç**. Aynı kalıp başka operatörlerde de çıkar: `cout >> x` yazarsan `no match for 'operator>>' (operand types are 'std::ostream' ... and 'int')` olur (okuma yönü `cin >> x`, yazma yönü `cout << x`).

Çözüm:

- Sınıfın için `ostream &operator<<(ostream &cikis, const Nokta &p)` fonksiyonunu yaz (Bölüm 13), ya da alanları tek tek yazdır: `cout << p.x << ' ' << p.y`.
- `Nokta + Nokta`, `a < b` gibi işlemlerde de aynı mesaj çıkar: `no match for 'operator+' (operand types are 'Nokta' and 'Nokta')`. Çözüm yine ilgili operatörü yüklemektir.

`invalid operands of types 'const char [10]' and 'const char [7]' to binary 'operator+'`

```cpp
#include <iostream>

using namespace std;

int main() {
    cout << "Merhaba, " + "dünya" << '\n';
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
metinToplama.cpp: In function 'int main()':
metinToplama.cpp:6:25: error: invalid operands of types 'const char [10]' and 'const char [7]' to
binary 'operator+'
    6 |     cout << "Merhaba, " + "dünya" << '\n';
      |             ~~~~~~~~~~~ ^ ~~~~~~~
      |             |           |
      |             |           const char [7]
      |             const char [10]
```

Ne demek? İki **metin sabitini** (`"..."`) `+` ile toplayamazsın. `"Merhaba, "` bir `string` değil, C tarzı karakter dizisidir; `[10]` ve `[7]` bayt sayılarıdır (`ü` iki bayttır, sonlandırıcı `'\0'` dahil). `+` ile birleştirme, **en az bir tarafı `string` olan** ifadelerde çalışır (Bölüm 8).

Çözüm: Taraflardan birini `string` yap: `string("Merhaba, ") + "dünya"`, ya da hiç toplama; `cout << "Merhaba, " << "dünya"` yaz.

`invalid conversion from 'const char*' to 'int'`

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    int sayi = "5";
    string metin = 5;
    int yas = metin;
    return sayi + yas;
}
```

Derleyici çıktısı – program derlenmez

```
donusumYok.cpp: In function 'int main()':
donusumYok.cpp:7:16: error: invalid conversion from 'const char*' to 'int' [-fpermissive]
    7 |     int sayi = "5";
      |                ^~~
      |                |
      |                const char*
donusumYok.cpp:8:20: error: conversion from 'int' to non-scalar type 'std::string' {aka
'std::__cxx11::basic_string<char>'} requested
    8 |     string metin = 5;
      |                    ^
donusumYok.cpp:9:15: error: cannot convert 'std::string' {aka 'std::__cxx11::basic_string<char>'} to
'int' in initialization
    9 |     int yas = metin;
      |               ^~~~~
      |               |
      |               std::string {aka std::__cxx11::basic_string<char>}
```

Ne demek? Değişkenin tipi ile atanan değerin tipi uyuşmuyor ve derleyici bir dönüşüm yolu bulamıyor. Üç satır, üç farklı cümle üretir:

- `invalid conversion from 'const char*' to 'int'`: metin sabiti `"5"`, tamsayıya atanamaz. Tırnak içindeki `5` bir **sayı değil, metindir** (Bölüm 2, 8). Sonundaki `[-fpermissive]`, bu hatanın g++ bayrağıyla uyarıya çevrilebileceğini söyler; yapma.
- `conversion from 'int' to non-scalar type 'std::string' requested`: `string metin = 5;` geçerli değildir. Tamsayıyı metne çevirmek için `to_string(5)` kullanılır.
- `cannot convert 'std::string' to 'int' in initialization`: `string` değerini `int`'e vermek de olmaz; metni sayıya `stoi` çevirir (Bölüm 8).

Çözüm: Tipleri eşleştir (`int sayi = 5;`) ya da açık dönüşüm fonksiyonunu kullan: `stoi(metin)`, `to_string(sayi)`. Girdiyi doğrudan doğru tipe `cin >> sayi;` ile okumak (Bölüm 3) çoğu zaman en kolay yoldur.

`narrowing conversion of '3.7' from 'double' to 'int'`

```cpp
int main() {
    int yas{3.7};
    return yas;
}
```

Derleyici çıktısı – program derlenmez

```
daralma.cpp: In function 'int main()':
daralma.cpp:2:13: error: narrowing conversion of '3.7000000000000002e+0' from 'double' to 'int' [-
Wnarrowing]
    2 |     int yas{3.7};
      |             ^~~
```

Ne demek? Süslü parantezle ilk değer verirken (Bölüm 2), bilgi kaybettirecek **daralan dönüşüme** (narrowing conversion) izin verilmez: `3.7` değeri `int`'e sığarken `.7` kaybolurdu. Eşittir ile yazsaydın (`int yas = 3.7;`) g++ sessizce `3` yapardı; süslü parantez bu sessizliği engeller. Mesajdaki `3.7000000000000002e+0`, `3.7`'nin `double` olarak bellekte tutulan halidir.

Çözüm: Değer gerçekten tamsayı olmalıysa `int yas{3};` yaz. Ondalık kısmı bilerek atıyorsan açık dönüştür: `int yas{static_cast<int>(3.7)};`. Çoğu zaman doğrusu değişkeni `double` yapmaktır.

`too few arguments to function` ve `too many arguments to function`

```cpp
int topla(int a, int b) {
    return a + b;
}

int main() {
    int x = topla(1);
    int y = topla(1, 2, 3);
    return x + y;
}
```

Derleyici çıktısı – program derlenmez

```
argumanSayisi.cpp: In function 'int main()':
argumanSayisi.cpp:6:18: error: too few arguments to function 'int topla(int, int)'
    6 |     int x = topla(1);
      |             ~~~~~^~~
argumanSayisi.cpp:1:5: note: declared here
    1 | int topla(int a, int b) {
      |     ^~~~~
argumanSayisi.cpp:7:18: error: too many arguments to function 'int topla(int, int)'
    7 |     int y = topla(1, 2, 3);
      |             ~~~~~^~~~~~~~
argumanSayisi.cpp:1:5: note: declared here
    1 | int topla(int a, int b) {
      |     ^~~~~
```

Ne demek? Fonksiyon çağrısında verdiğin argüman sayısı, tanımdaki parametre sayısıyla uyuşmuyor (Bölüm 6). Mesajdaki `int topla(int, int)`, fonksiyonun beklediği imzayı gösterir; `note: declared here` fonksiyonun tanımlandığı satıra götürür.

Çözüm: Parametre sayısıyla argüman sayısını eşitle. Sıra ve tipler de aynı olmalı. Parametrenin **varsayılan değeri** varsa (`int topla(int a, int b = 0)`, Bölüm 6) argüman eksik verilebilir.

`call of overloaded 'yaz(double)' is ambiguous`

```cpp
#include <iostream>

using namespace std;

void yaz(int x) { cout << "int " << x << '\n'; }
void yaz(long x) { cout << "long " << x << '\n'; }

int main() {
    yaz(2.5);
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
belirsizCagri.cpp: In function 'int main()':
belirsizCagri.cpp:9:8: error: call of overloaded 'yaz(double)' is ambiguous
    9 |     yaz(2.5);
      |     ~~~^~~~~
  * there are 2 candidates
    * candidate 1: 'void yaz(int)'
      belirsizCagri.cpp:5:6:
          5 | void yaz(int x) { cout << "int " << x << '\n'; }
            |      ^~~
    * candidate 2: 'void yaz(long int)'
      belirsizCagri.cpp:6:6:
          6 | void yaz(long x) { cout << "long " << x << '\n'; }
            |      ^~~
```

Ne demek? Aşırı yüklenmiş fonksiyonlardan (Bölüm 6) hiçbiri argümanla **tam** eşleşmiyor ve dönüşümle eşleşen birden fazla aday var: `2.5` hem `int`'e hem `long`'a aynı derecede çevrilebilir. Derleyici birini seçemediği için çağrıyı belirsiz (ambiguous) sayar; `candidate` satırları iki adayı gösterir.

Çözüm: Argümanı istediğin tipe açıkça çevir (`yaz(static_cast<int>(2.5))`) ya da argümanla tam eşleşen bir sürüm ekle (`void yaz(double x)`).

`no matching function for call to 'Nokta::Nokta()'`

```cpp
class Nokta {
public:
    Nokta(int x, int y) : x_{x}, y_{y} {}
private:
    int x_;
    int y_;
};

int main() {
    Nokta p;
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
varsayilanYok.cpp: In function 'int main()':
varsayilanYok.cpp:10:11: error: no matching function for call to 'Nokta::Nokta()'
   10 |     Nokta p;
      |           ^
  * there are 3 candidates
    * candidate 1: 'Nokta::Nokta(int, int)'
      varsayilanYok.cpp:3:5:
          3 |     Nokta(int x, int y) : x_{x}, y_{y} {}
            |     ^~~~~
      * candidate expects 2 arguments, 0 provided
    * candidate 2: 'constexpr Nokta::Nokta(const Nokta&)'
      varsayilanYok.cpp:1:7:
          1 | class Nokta {
            |       ^~~~~
      * candidate expects 1 argument, 0 provided
    * candidate 3: 'constexpr Nokta::Nokta(Nokta&&)'
      * candidate expects 1 argument, 0 provided
varsayilanYok.cpp:10:11: warning: unused variable 'p' [-Wunused-variable]
   10 |     Nokta p;
      |           ^
```

Ne demek? `Nokta p;` parametresiz yapıcıyı (varsayılan yapıcı) çağırır; ama sınıfa kendi yapıcını yazınca derleyici otomatik varsayılan yapıcıyı **kaldırır** (Bölüm 11). Elinde yalnızca iki `int` alan yapıcı vardır, `0 provided` (hiç argüman verilmedi) uymaz. Hatayı kapsayan `candidate 1`, `candidate 2` satırları "denediğim yapıcılar şunlar, hepsi uymadı" listesidir. Kopya yapıcı ve taşıma yapıcısı (`Nokta(const Nokta&)`, `Nokta(Nokta&&)`) derleyicinin kendiliğinden eklediği adaylardır; onları okumana gerek yok.

Çözüm: `Nokta p{3, 4};` yaz, ya da parametresiz bir yapıcı ekle: `Nokta() : x_{0}, y_{0} {}`. Bir yapıcıya varsayılan değerler (`Nokta(int x = 0, int y = 0)`) vermek de hem iki hem sıfır argümanlı çağrıyı kapsar.

`assignment of read-only variable 'MAX_NOT'`

```
int main() {
    const int MAX_NOT{100};
    MAX_NOT = 90;
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
sabitAtama.cpp: In function 'int main()':
sabitAtama.cpp:3:13: error: assignment of read-only variable 'MAX_NOT'
    3 |     MAX_NOT = 90;
      |     ~~~~~~~~^~~~
```

Ne demek? `const` ile tanımlanan değişkenin değeri değiştirilemez (Bölüm 2). "Salt okunur değişkene atama yapıldı." Aynı hata `const` referans (`const string &`, Bölüm 6) ve `const` işaretçi (Bölüm 9) üzerinden değiştirmeye çalışırken de çıkar; ifade değişse de kalıp aynıdır (`assignment of read-only location`).

Çözüm: Değerin sonradan değişmesi gerekiyorsa `const`'ı kaldır. Değişmemesi gerekiyorsa atama satırı yanlıştır; sil.

`lvalue required as left operand of assignment`

```
int main() {
    int x{3};
    if (5 = x) {
        return 1;
    }
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
solDeger.cpp: In function 'int main()':
solDeger.cpp:3:9: error: lvalue required as left operand of assignment
    3 |     if (5 = x) {
      |         ^
```

Ne demek? Atamanın (`=`) sol tarafında bir **sol değer** (lvalue), yani değer konabilecek bir değişken olmalı (Bölüm 4). `5` bir sabittir; içine değer konamaz. Bu hata çoğu zaman karşılaştırma için `==` yazmak isterken `=` yazınca çıkar. `a + b = 10;` gibi bir satır da aynı mesajı verir.

Çözüm: Karşılaştırma istiyorsan `==` yaz: `if (5 == x)`. Atama istiyorsan değişkeni sola al: `x = 5;`.

`passing 'const Sayac' as 'this' argument discards qualifiers`

```cpp
#include <iostream>

using namespace std;

class Sayac {
public:
    int al() { return deger; }
private:
    int deger{0};
};

void yaz(const Sayac &s) {
    cout << s.al() << '\n';
}

int main() {
    Sayac s;
    yaz(s);
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
constUye.cpp: In function 'void yaz(const Sayac&)':
constUye.cpp:13:17: error: passing 'const Sayac' as 'this' argument discards qualifiers [-
fpermissive]
   13 |     cout << s.al() << '\n';
      |             ~~~~^~
constUye.cpp:7:9: note:   in call to 'int Sayac::al()'
    7 |     int al() { return deger; }
      |         ^~
```

Ne demek? İlk bakışta anlaşılmaz bir cümledir. Çözümü: `yaz` fonksiyonu nesneyi `const Sayac &` olarak aldı, yani "nesneyi değiştirmeyeceğim" sözü verdi. Ama `al()` üye fonksiyonu `const` olarak işaretlenmediği için derleyici bunun nesneyi **değiştirebileceğini** varsayar ve çağrıya izin vermez. Mesajdaki `'this' argument`, çağrının üstünde yapıldığı nesnedir; `discards qualifiers` ise `const` niteleyicisinin düşürüldüğü anlamına gelir (Bölüm 11).

Çözüm: Nesneyi değiştirmeyen üye fonksiyonun sonuna `const` yaz: `int al() const { return deger; }`. Kural: **okuma amaçlı bütün üye fonksiyonlar `const` olmalıdır.**

`'double Hesap::bakiye' is private within this context`

```cpp
#include <iostream>

using namespace std;

class Hesap {
private:
    double bakiye{0.0};
};

int main() {
    Hesap h;
    cout << h.bakiye << '\n';
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
privateErisim.cpp: In function 'int main()':
privateErisim.cpp:12:15: error: 'double Hesap::bakiye' is private within this context
   12 |     cout << h.bakiye << '\n';
      |               ^~~~~~
privateErisim.cpp:7:12: note: declared private here
    7 |     double bakiye{0.0};
      |            ^~~~~~
```

Ne demek? `private` üyeye sınıfın dışından erişemezsin (Bölüm 11, kapsülleme). Mesaj hangi üyenin (`Hesap::bakiye`) ve tipinin (`double`) erişime kapalı olduğunu söyler; `note: declared private here` tanımına götürür. Türetilmiş sınıf da temel sınıfın `private` üyesini göremez; orada `protected` gerekir (Bölüm 12).

Çözüm: Değere `public` bir üye fonksiyonla eriş (`double bakiyeAl() const`), erişimin gerçekten gerekli olduğu durumlarda üyeyi `public` yap, ya da türetilmiş sınıfın göreceği değerleri `protected` yap.

`cannot bind non-const lvalue reference of type 'int&' to an rvalue of type 'int'`

```
void artir(int &sayi) {
    sayi++;
}

int main() {
    artir(5);
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
referansBagla.cpp: In function 'int main()':
referansBagla.cpp:6:11: error: cannot bind non-const lvalue reference of type 'int&' to an rvalue of
type 'int'
    6 |     artir(5);
      |           ^
referansBagla.cpp:1:17: note: initializing argument 1 of 'void artir(int&)'
    1 | void artir(int &sayi) {
      |            ~~~~~^~~~
```

Ne demek? `int &` parametresi, değeri **değiştirmek** için bir değişkenin kendisine bağlanır (Bölüm 9). `5` ise geçici bir değerdir; bellekte adı, adresi, değiştirilecek kutusu yoktur. Tırnak içindeki `lvalue`, adı olan değişken; `rvalue`, geçici değerdir.

Çözüm:

- Fonksiyona bir değişken ver: `int n{5}; artir(n);`.
- Fonksiyon değeri **değiştirmiyorsa** parametreyi `const int &` yap (Bölüm 6); `const` referans geçici değere bağlanabilir.

`use of deleted function`

```cpp
#include <memory>

using namespace std;

int main() {
    unique_ptr<int> a = make_unique<int>(5);
    unique_ptr<int> b = a;
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
silinmisFonk.cpp: In function 'int main()':
silinmisFonk.cpp:7:25: error: use of deleted function 'std::unique_ptr<_Tp, _Dp>::unique_ptr(const
std::unique_ptr<_Tp, _Dp>&) [with _Tp = int; _Dp = std::default_delete<int>]'
    7 |     unique_ptr<int> b = a;
      |                         ^
In file included from /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/memory:80,
                 from silinmisFonk.cpp:1:
/opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/bits/unique_ptr.h:543:7: note: declared here
  543 |       unique_ptr(const unique_ptr&) = delete;
      |       ^~~~~~~~~~
silinmisFonk.cpp:7:25: note: use '-fdiagnostics-all-candidates' to display considered candidates
    7 |     unique_ptr<int> b = a;
      |                         ^
```

Ne demek? “Çağırdığın fonksiyonu `= delete` ile yasakladılar.” `unique_ptr`’nin kopya yapıcısı kasten silinmiştir; çünkü bir nesnenin tek sahibi olmalıdır (Bölüm 10). `note: declared here` satırı, kütüphane başlığında `= delete` yazan yeri gösterir. Mesajdaki `[with _Tp = int; ...]` kısmı, şablonun hangi tiple çalıştığıdır (aşağıdaki şablon bölümüne bak).

Çözüm: Sahipliği devret: `unique_ptr<int> b = std::move(a);` (Bölüm 10). Gerçekten paylaşılacaksa `shared_ptr` kullan. Aynı mesaj, `= delete` yapılmış kendi sınıfın için de çıkar.

`marked 'override', but does not override`

```cpp
class Hayvan {
public:
    virtual void sesCikar() const {}
};

class Kedi : public Hayvan {
public:
    void sesCikar() override {}
};

int main() {
    Kedi k;
    k.sesCikar();
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
overrideYok.cpp:8:10: error: 'void Kedi::sesCikar()' marked 'override', but does not override
    8 |     void sesCikar() override {}
      |          ^~~~~~~~
overrideYok.cpp:3:18: warning: 'virtual void Hayvan::sesCikar() const' was hidden [-Woverloaded-
virtual=]
    3 |     virtual void sesCikar() const {}
      |                  ^~~~~~~~
overrideYok.cpp:8:10: note:   by 'void Kedi::sesCikar()'
    8 |     void sesCikar() override {}
      |          ^~~~~~~~
```

Ne demek? `override` anahtar sözcüğü, "bu fonksiyon temel sınıftaki bir sanal fonksiyonu geçersiz kılıyor" iddiasıdır (Bölüm 12). Derleyici imzaları karşılaştırır ve eşleşen sanal fonksiyon bulamaz. Burada fark `const`'tur: temel sınıftaki `sesCikar() const`, türetilmiş sınıftaki `sesCikar()` ile aynı fonksiyon değildir. Altındaki `was hidden` uyarısı aynı sorunu başka yönden anlatır: türetilmiş sınıftaki fonksiyon, temel sınıftakini **gizledi**, geçersiz kılmadı.

Çözüm: İki imzayı birebir aynı yap (`const`, parametre tipleri, referans işaretleri dahil). `override`'ı **her zaman yaz**; yazmasaydın derleyici sorunu fark edemez, program sessizce yanlış çalışırdı.

Şablon ve kapsayıcı hataları: uzun mesajı okumak

Şablon (template) kodu derlenirken hata çıkarsa mesaj kütüphanenin derinliklerinden gelir ve sayfalarca uzayabilir. İlk bakışta korkutucu görünür ama bir okuma yöntemi vardır. Önce kısa olanla başlayalım: Bölüm 13'ün `enBuyuk` şablonunu `<` operatörü olmayan bir sınıfla çağırıyoruz.

```cpp
template <typename T>
T enBuyuk(T a, T b) {
    return (a < b) ? b : a;
}

class Nokta {
public:
    int x;
    int y;
};

int main() {
    Nokta p{1, 2};
    Nokta q{3, 4};
    enBuyuk(p, q);
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
sablonHata.cpp: In instantiation of 'T enBuyuk(T, T) [with T = Nokta]':
sablonHata.cpp:15:12:   required from here
   15 |     enBuyuk(p, q);
      |     ~~~~~~~^~~~~
sablonHata.cpp:3:15: error: no match for 'operator<' (operand types are 'Nokta' and 'Nokta')
    3 |     return (a < b) ? b : a;
      |            ~~~^~~~
```

Üç satırı sırayla oku. Dikkat: şablon hatalarında sıra alışılmışın tersidir; senin satırını gösteren `required from here`, `error:` satırının **üstünde** gelir:

1. `In instantiation of 'T enBuyuk(T, T) [with T = Nokta]'`: "`enBuyuk` şablonunu `T = Nokta` ile **örneklendirirken** (instantiation) bir şey oldu." Köşeli parantezdeki `[with T = Nokta]` en değerli bilgidir; şablondaki `T`'nin hangi tip olduğunu söyler.
2. `required from here` ve altındaki `enBuyuk(p, q);`: "bu örneklendirmeyi **senin kodundaki** şu satır istedi." Gerçek hatanın nedeni budur; satırı ve sütunu not et.
3. `error: no match for 'operator<' (operand types are 'Nokta' and 'Nokta')`: şablonun içinde `a < b` yazılmış, ama `Nokta` için `<` tanımlı değil. Hatanın **kendisi** bu satırdır; şablonun içindeki satırı (`return (a < b) ? ...`) gösterir.

Çıkarım: hata şablonda değildir, **şablonu uygun olmayan bir tiple çağırmandadır**. Şablon `<` bekliyor; `Nokta` bunu sağlamıyor. Çözüm: `Nokta` için `operator<` yaz (Bölüm 13), ya da `<` destekleyen bir tiple çağır.

Şimdi gerçek bir uzun mesaj. Bölüm 15'te `map`'in anahtarlarını sıralı tuttuğunu gördün; sıralamak için anahtarın `<` ile karşılaştırılabilmesi gerekir. `Nokta` bunu sağlamayınca:

```cpp
#include <iostream>
#include <map>
#include <string>

using namespace std;

class Nokta {
public:
    int x;
};

int main() {
    map<Nokta, string> adlar;
    adlar[Nokta{1}] = "bir";
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
In file included from /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/string:52,
                 from /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/bits/locale_classes.h:43,
                 from /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/bits/ios_base.h:43,
                 from /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/ios:46,
                 from /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/bits/ostream.h:43,
                 from /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/ostream:42,
                 from /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/iostream:43,
                 from mapHata.cpp:1:
/opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/bits/stl_function.h: In instantiation of 'constexpr
bool std::less<_Tp>::operator()(const _Tp&, const _Tp&) const [with _Tp = Nokta]':
/opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/bits/stl_map.h:558:32:   required from
'std::map<_Key, _Tp, _Compare, _Alloc>::mapped_type& std::map<_Key, _Tp, _Compare,
_Alloc>::operator[](key_type&&) [with _Key = Nokta; _Tp = std::__cxx11::basic_string<char>; _Compare
= std::less<Nokta>; _Alloc = std::allocator<std::pair<const Nokta, std::__cxx11::basic_string<char>
> >; mapped_type = std::__cxx11::basic_string<char>; key_type = Nokta]'
  558 |         if (__i == end() || key_comp()(__k, (*__i).first))
      |                             ~~~~~~~~~~^~~~~~~~~~~~~~~~~~~
mapHata.cpp:14:19:   required from here
   14 |     adlar[Nokta{1}] = "bir";
      |                   ^
/opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/bits/stl_function.h:408:20: error: no match for
'operator<' (operand types are 'const Nokta' and 'const Nokta')
  408 |       { return __x < __y; }
      |                ~~~~^~~~~
```

```
  * there are 4 candidates
In file included from /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/string:51:
    * candidate 1: 'template<class _Iterator> constexpr bool std::operator<(const
reverse_iterator<_Iterator>&, const reverse_iterator<_Iterator>&)'
      /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/bits/stl_iterator.h:450:5:
        450 |     operator<(const reverse_iterator<_Iterator>& __x,
            |     ^~~~~~~~
      * template argument deduction/substitution failed:
        *   'const Nokta' is not derived from 'const std::reverse_iterator<_Iterator>'
          /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/bits/stl_function.h:408:20:
            408 |       { return __x < __y; }
                |                ~~~~^~~~
    * candidate 2: 'template<class _IteratorL, class _IteratorR> constexpr bool std::operator<(const
reverse_iterator<_Iterator>&, const reverse_iterator<_IteratorR>&)'
      /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/bits/stl_iterator.h:495:5:
        495 |     operator<(const reverse_iterator<_IteratorL>& __x,
            |     ^~~~~~~~
      * template argument deduction/substitution failed:
        *   'const Nokta' is not derived from 'const std::reverse_iterator<_Iterator>'
          /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/bits/stl_function.h:408:20:
            408 |       { return __x < __y; }
                |                ~~~~^~~~
    * candidate 3: 'template<class _IteratorL, class _IteratorR> constexpr bool std::operator<(const
move_iterator<_IteratorL>&, const move_iterator<_IteratorR>&)'
      /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/bits/stl_iterator.h:1702:5:
       1702 |     operator<(const move_iterator<_IteratorL>& __x,
            |     ^~~~~~~~
      * template argument deduction/substitution failed:
        *   'const Nokta' is not derived from 'const std::move_iterator<_IteratorL>'
          /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/bits/stl_function.h:408:20:
            408 |       { return __x < __y; }
                |                ~~~~^~~~
    * candidate 4: 'template<class _Iterator> constexpr bool std::operator<(const
move_iterator<_IteratorL>&, const move_iterator<_IteratorL>&)'
      /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/bits/stl_iterator.h:1768:5:
       1768 |     operator<(const move_iterator<_Iterator>& __x,
            |     ^~~~~~~~
      * template argument deduction/substitution failed:
        *   'const Nokta' is not derived from 'const std::move_iterator<_IteratorL>'
          /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/bits/stl_function.h:408:20:
            408 |       { return __x < __y; }
                |                ~~~~^~~~
```

Mesaj elli satırı aşan bir duvardır. Şu sırayla oku:

- **En üstteki `In file included from ...` satırlarını atla.** Hangi başlığın hangisini içerdiğini listeler; hatayla ilgisi yoktur.
- **`error:` sözcüğünü bul.** İlk (genellikle tek) `error:` satırı sorunun kendisidir: `no match for 'operator<' (operand types are 'const Nokta' and 'const Nokta')`. Yani `Nokta` iki nesneyi `<` ile karşılaştıramıyor. Dosya adı kütüphane başlığına (`stl_function.h`) aittir; bu yalnızca "hata orada **fark edildi**" demektir, hata orada **yapılmadı**.
- **Kendi dosya adını ara.** `required from here` satırına bak. Burada `mapHata.cpp:14:19: required from here` ve altında `adlar[Nokta{1}] = "bir";` satırı var. Hatayı **sen buraya** yazdın.
- **`[with ...]` ve `std::map<...>` parçalarındaki tip adlarını oku.** `[with _Key = Nokta ...]` ve `[with _Tp = Nokta]` sana sorunlu tipin `Nokta` olduğunu söyler. Uzun, `std::__cxx11::basic_string<char>` gibi adlar `std::string` demektir; `{aka ...}` ("also known as") ekleri genellikle yok sayılabilir.
- **`candidate` listesini okuma.** "Şunları denedim, hiçbiri uymadı" listesidir.

Kural özet olarak: **ilk `error:` satırını, ardından kendi dosya adının geçtiği ilk `required from here` satırını oku. Gerisini bırak.** Bu iki satır, çoğu zaman hatanın yerini ve nedenini verir.

Son örnek, en sık yapılan kapsayıcı hatasıdır: `vector<int>`'e metin eklemek.

```cpp
#include <vector>

using namespace std;

int main() {
    vector<int> sayilar;
    sayilar.push_back("elma");
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
vectorHata.cpp: In function 'int main()':
vectorHata.cpp:7:22: error: no matching function for call to 'push_back(const char [5])'
    7 |     sayilar.push_back("elma");
      |     ~~~~~~~~~~~~~~~~~^~~~~~~
  * there are 2 candidates
In file included from /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/vector:68,
                 from vectorHata.cpp:1:
    * candidate 1: 'void std::vector<_Tp, _Alloc>::push_back(const value_type&) [with _Tp = int;
_Alloc = std::allocator<int>; value_type = int]' (near match)
      /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/bits/stl_vector.h:1406:7:
       1406 |       push_back(const value_type& __x)
            |       ^~~~~~~~~
      * conversion of argument 1 would be ill-formed:
      * error: invalid conversion from 'const char*' to 'std::vector<int>::value_type' {aka 'int'}
[-fpermissive]
        vectorHata.cpp:7:23:
            7 |     sayilar.push_back("elma");
              |                       ^~~~~~
              |                       |
              |                       const char*
    * candidate 2: 'void std::vector<_Tp, _Alloc>::push_back(value_type&&) [with _Tp = int; _Alloc =
std::allocator<int>; value_type = int]' (near match)
      /opt/homebrew/Cellar/gcc/16.2.0/include/c++/16/bits/stl_vector.h:1423:7:
       1423 |       push_back(value_type&& __x)
            |       ^~~~~~~~~
      * conversion of argument 1 would be ill-formed:
      * error: invalid conversion from 'const char*' to 'std::vector<int>::value_type' {aka 'int'}
[-fpermissive]
        vectorHata.cpp:7:23:
            7 |     sayilar.push_back("elma");
              |                       ^~~~~~
              |                       |
              |                       const char*
```

Ne demek? İlk satır yeterlidir: `no matching function for call to 'push_back(const char [5])'`. "`push_back` fonksiyonuna metin sabiti (`const char [5]`) verecek bir sürüm yok." Altındaki iki aday da `int` bekler ve dönüşüm başarısız olur: `invalid conversion from 'const char*' to 'int'`. Kısacası **kapsayıcının eleman tipi ile eklediğin değerin tipi uyuşmuyor** (Bölüm 7, 15).

Çözüm: Eleman tipine uygun değer ver (`push_back(4)`) ya da kapsayıcının tipini değiştir (`vector<string>`).

Önemli uyarılar

Uyarılar hata değildir; program derlenir. Ama `-Wall -Wextra` ile açılan uyarıların neredeyse tamamı **gerçek bir hatanın** habercisidir. Bu kitapta uyarılı kod teslim etmiyoruz; aşağıdakilerin hepsini görünce kodu düzelt. Her örnekte uyarıyı gösterdikten sonra düzeltmeyi yazıyoruz.

`unused variable 'boy'`

```cpp
#include <iostream>

using namespace std;

int main() {
    int yas{19};
    int boy{175};
    cout << yas << '\n';
    return 0;
}
```

Derleyici uyarısı – g++ -Wall -Wextra

```
kullanilmayan.cpp: In function 'int main()':
kullanilmayan.cpp:7:9: warning: unused variable 'boy' [-Wunused-variable]
    7 |     int boy{175};
      |         ^~~
```

Ne demek? `boy` tanımlanmış ama hiç kullanılmamış. Gereksiz kod olabilir; ya da asıl kullanmak istediğin değişkenin yerine yanlış adı yazmış olabilirsin (Bölüm 1, 2).

Çözüm: Değişkeni sil ya da kullan. Fonksiyonda kullanılmayan parametre için de `unused parameter 'sayi'` uyarısı çıkar; parametrenin adını silmek (`void selamla(int)`) uyarıyı kapatır.

`'toplam' is used uninitialized`

```cpp
#include <iostream>

using namespace std;

int main() {
    int toplam;
    cout << toplam << '\n';
    return 0;
}
```

Derleyici uyarısı – g++ -Wall -Wextra

```
kullanilmamis.cpp: In function 'int main()':
kullanilmamis.cpp:7:23: warning: 'toplam' is used uninitialized [-Wuninitialized]
    7 |     cout << toplam << '\n';
      |                       ^~~~
kullanilmamis.cpp:6:9: note: 'toplam' was declared here
    6 |     int toplam;
      |         ^~~~~~
```

Ne demek? `toplam` değişkenine hiç değer verilmeden okundu. İçindeki **çöp değerdir** ve bu okuma tanımsız davranıştır (Bölüm 2). `note: 'toplam' was declared here` satırı değişkenin tanımına götürür.

Çözüm: Her değişkene tanımlarken ilk değer ver: `int toplam{0};`. g++ bu uyarıyı her yerde yakalamaz; özellikle `if` içinde atanıp dışında okunan değişkenlerde sessiz kalabilir. Uyarının **olmaması** değişkenin başlatılmış olduğunu göstermez.

`comparison of integer expressions of different signedness`

```cpp
#include <iostream>
#include <vector>

using namespace std;

int main() {
    vector<int> sayilar{4, 8, 15};
    for (int i = 0; i < sayilar.size(); i++) {
        cout << sayilar[i] << '\n';
    }
    return 0;
}
```

Derleyici uyarısı – g++ -Wall -Wextra

```
isaretFarki.cpp: In function 'int main()':
isaretFarki.cpp:8:23: warning: comparison of integer expressions of different signedness: 'int' and
'std::vector<int>::size_type' {aka 'long unsigned int'} [-Wsign-compare]
    8 |     for (int i = 0; i < sayilar.size(); i++) {
      |                     ~~^~~~~~~~~~~~~~~~
```

Ne demek? `i` işaretli (`int`), `sayilar.size()` ise işaretsiz (`size_t`, `unsigned long`) bir tamsayıdır. İkisini karşılaştırınca negatif sayılar beklenmedik biçimde çok büyük pozitif sayılara dönüşür (Bölüm 4, 7).

Çözüm: Sayacı `size_t` yap (`for (size_t i = 0; ...)`), ya da aralık tabanlı `for` kullan (`for (int s : sayilar)`, Bölüm 7). Geriye doğru sayarken (`i >= 0`) `size_t` kullanma; döngü hiç bitmez (Bölüm 5).

`control reaches end of non-void function`

```cpp
#include <iostream>

using namespace std;

int buyuk(int a, int b) {
    if (a > b) {
        return a;
    } else if (b > a) {
        return b;
    }
}

int main() {
    cout << buyuk(3, 7) << '\n';
    return 0;
}
```

Derleyici uyarısı – g++ -Wall -Wextra

```
donusYok.cpp: In function 'int buyuk(int, int)':
donusYok.cpp:11:1: warning: control reaches end of non-void function [-Wreturn-type]
   11 | }
      | ^
```

Ne demek? `int` döndüren fonksiyonun bazı yolları `return` olmadan bitiyor. Burada `a == b` olursa hiçbir `return` çalışmaz ve fonksiyondan çöp değer döner (Bölüm 6). g++ kodun **sonuna** işaret eder.

Çözüm: Fonksiyonun bütün yollarının `return` ile bittiğinden emin ol. Çoğu zaman son `else if`'i `else` yapmak (ya da en sona varsayılan bir `return` yazmak) yeterlidir. `-Wall` bunu uyarı olarak verir, ama değeri kullanırsan program tanımsız davranışa girer.

`suggest parentheses around assignment used as truth value`

```cpp
#include <iostream>

using namespace std;

int main() {
    int not1{50};
    if (not1 = 100) {
        cout << "tam not\n";
    }
    return 0;
}
```

Derleyici uyarısı – g++ -Wall -Wextra

```
atamaIf.cpp: In function 'int main()':
atamaIf.cpp:7:14: warning: suggest parentheses around assignment used as truth value [-Wparentheses]
    7 |     if (not1 = 100) {
      |         ~~~~^~~~~
```

Ne demek? `if (not1 = 100)` içinde karşılaştırma (`==`) değil **atama** (`=`) yazılmış (Bölüm 4). Atama `100` değerini üretir, `100` sıfırdan farklı olduğu için koşul her zaman doğru çıkar. Hem `not1`'i bozar hem de `if`'i anlamsız yapar. g++ "koşul olarak atama kullanmak istiyorsan parantezle" diye uyarır.

Çözüm: `==` yaz: `if (not1 == 100)`. Sabiti sola yazmak (`100 == not1`) da yanlışlıkla `=` yazmayı derleme hatasına çevirir.

`suggest braces around empty body in an 'if' statement`

```cpp
#include <iostream>

using namespace std;

int main() {
    int x{-3};
    if (x > 0);
    {
        cout << "pozitif\n";
    }
    return 0;
}
```

Derleyici uyarısı – g++ -Wall -Wextra

```
bosGovde.cpp: In function 'int main()':
bosGovde.cpp:7:15: warning: suggest braces around empty body in an 'if' statement [-Wempty-body]
    7 |     if (x > 0);
      |               ^
bosGovde.cpp:7:5: warning: this 'if' clause does not guard... [-Wmisleading-indentation]
    7 |     if (x > 0);
      |     ^~
bosGovde.cpp:8:5: note: ...this statement, but the latter is misleadingly indented as if it were
guarded by the 'if'
    8 |     {
      |     ^
```

Ne demek? `if (x > 0);` satırının sonundaki `;` **boş bir deyimdir** ve `if`'in gövdesi odur (Bölüm 4). Alttaki süslü parantezli blok `if`'e bağlı değildir, her zaman çalışır: `x` negatif olsa da `pozitif` yazılır. g++ iki uyarı verir: gövde boş (`-Wempty-body`) ve girinti yanıltıcı (`-Wmisleading-indentation`).

Çözüm: `if (...)` satırının sonuna `;` koyma: `if (x > 0) {`. Aynı tuzak `for (...);` ve `while (...);` için de geçerlidir (Bölüm 5).

`this statement may fall through`

```cpp
#include <iostream>

using namespace std;

int main() {
    int gun{1};
    switch (gun) {
        case 1:
            cout << "Pazartesi" << '\n';
        case 2:
            cout << "Salı" << '\n';
            break;
        default:
            break;
    }
    return 0;
}
```

Derleyici uyarısı – g++ -Wall -Wextra

```
breakEksik.cpp: In function 'int main()':
breakEksik.cpp:9:36: warning: this statement may fall through [-Wimplicit-fallthrough=]
    9 |             cout << "Pazartesi" << '\n';
      |                                    ^~~~
breakEksik.cpp:10:9: note: here
   10 |         case 2:
      |         ^~~~
```

Ne demek? `case 1`'in sonunda `break` yok; program `case 2`'nin içine **düşer** (fall-through) ve iki satır da yazdırılır (Bölüm 4). g++ düşmenin başladığı satıra ve altındaki `note: here` ile düştüğü etikete işaret eder.

Çözüm: Her `case`'in sonuna `break;` (ya da `return`) koy. Düşmeyi bilerek yapıyorsan C++17'nin `[[fallthrough]];` ifadesini yaz; uyarı susar ve okuyan kişi niyetini anlar.

`suggest explicit braces to avoid ambiguous 'else'`

```cpp
#include <iostream>

using namespace std;

int main() {
    int a{5};
    int b{2};
    if (a > 3)
        if (b > 3)
            cout << "ikisi de büyük" << '\n';
    else
        cout << "a küçük" << '\n';
    return 0;
}
```

Derleyici uyarısı – g++ -Wall -Wextra

```
sarkanElse.cpp: In function 'int main()':
sarkanElse.cpp:8:8: warning: suggest explicit braces to avoid ambiguous 'else' [-Wdangling-else]
    8 |     if (a > 3)
      |        ^
```

Ne demek? Girinti `else`'in dıştaki `if`'e ait olduğunu düşündürür; ama derleyici girintiye bakmaz, `else`'i **en yakın** `if`'e bağlar (Bölüm 4, sarkan else). Burada `a = 5` ve `b = 2` iken program `a küçük` yazdırır; oysa `a` küçük değildir, `else` içteki `if`'e bağlanmıştır.

Çözüm: Her `if` ve `else` gövdesini süslü parantez içine al. Tek satır olsa bile.

`will be initialized after` (`-Wreorder`)

```cpp
class Ogrenci {
public:
    Ogrenci(int no, int yas) : yas_{yas}, no_{no} {}
private:
    int no_;
    int yas_;
};

int main() {
    Ogrenci o{1, 20};
    return 0;
}
```

Derleyici uyarısı – g++ -Wall -Wextra

```
siraUyari.cpp: In constructor 'Ogrenci::Ogrenci(int, int)':
siraUyari.cpp:6:9: warning: 'Ogrenci::yas_' will be initialized after [-Wreorder]
    6 |     int yas_;
      |         ^~~~
siraUyari.cpp:5:9: warning:   'int Ogrenci::no_' [-Wreorder]
    5 |     int no_;
      |         ^~~
siraUyari.cpp:3:5: warning:   when initialized here [-Wreorder]
    3 |     Ogrenci(int no, int yas) : yas_{yas}, no_{no} {}
      |     ^~~~~~~
```

Ne demek? Üye ilk değer listesindeki (Bölüm 11) sıra ile üyelerin sınıf içinde **tanımlandığı** sıra farklı. Üyeler her zaman sınıfta tanımlandıkları sırayla başlatılır; listede başka sırada yazman bu sırayı değiştirmez, yalnızca seni yanıltır. Üyelerden biri diğerine bağımlıysa (ör. `yas_{no_ * 2}`) çöp değer okursun.

Çözüm: Listeyi, üyelerin sınıfta tanımlandığı sırayla yaz: `: no_{no}, yas_{yas}`.

`reference to local variable 'sonuc' returned`

```cpp
int& kareAl(int sayi) {
    int sonuc{sayi * sayi};
    return sonuc;
}

int main() {
    return 0;
}
```

Derleyici uyarısı – g++ -Wall -Wextra

```
yerelReferans.cpp: In function 'int& kareAl(int)':
yerelReferans.cpp:3:12: warning: reference to local variable 'sonuc' returned [-Wreturn-local-addr]
    3 |     return sonuc;
      |            ^~~~~
yerelReferans.cpp:2:9: note: declared here
    2 |     int sonuc{sayi * sayi};
      |         ^~~~~
```

Ne demek? Fonksiyon bittiğinde yerel değişken `sonuc` yok olur (çağrı yığınındaki yığın çerçevesi kaldırılır); fonksiyon ise bu yok olmuş değişkene bir **referans** döndürüyor. Çağıran taraf, geçersiz bellekte yaşayan bir **sarkan referans** (dangling reference; sarkan işaretçinin referanslı hali) elde eder (Bölüm 9). Değerini okumak tanımsız davranıştır. İşaretçi döndürürsen mesaj `address of local variable 'sonuc' returned` olur.

Çözüm: Değeri kopyala döndür: dönüş tipini `int` yap. Referansla yalnızca fonksiyondan **sonra da yaşayan** nesneleri döndür (sınıfın üye değişkeni, parametre olarak gelen referans).

> **Sınavda Çıkar**
>
> Sınavda bir kod parçası verilip "çıktısı nedir?" sorulurken uyarının nedeni çoğu zaman sorunun cevabıdır: `if (x = 5)`, eksik `break`, sarkan `else`. Kod görünce önce bu tuzakları ara.

Bağlama (linker) hataları

Bağlama hatası, derleme **bittikten sonra** gelir: kodda söz dizimi sorunu yoktur, ama bağlayıcı (linker, `ld`) çağrılan bir fonksiyonun ya da nesnenin kodunu hiçbir yerde bulamaz (Bölüm 1, derleme süreci). Mesajda satır numarası yerine `.o` dosyası ve `.text+0x...` gibi bellek konumları görünür. Son satırdaki `collect2: error: ld returned 1 exit status`, "bağlayıcı hata verdi" demektir ve her bağlama hatasının sonunda aynen çıkar; asıl bilgi **üstündeki** `undefined reference` satırındadır.

Bu mesajlar macOS'ta farklı yazılır (`Undefined symbols for architecture arm64`), o yüzden aşağıdaki örnekleri **Windows (MinGW) ve Linux'taki g++ mesajları** olarak elle verdik; uzun yolları `...` ile kısalttık. Eski `ld` sürümleri tırnağı `'main'` yerine `` `main' `` biçiminde yazar.

`undefined reference to 'WinMain'`

Bölüm 1'de gördüğün durum: programın başlangıç noktası `main` bulunamıyor. Windows'ta (MinGW):

```
.../ld.exe: .../crt2.o:crtexe.c:(.text+0x...): undefined reference to 'WinMain'
collect2.exe: error: ld returned 1 exit status
```

Linux'ta aynı hata şöyle görünür:

```
/usr/bin/ld: .../Scrt1.o: in function `_start':
(.text+0x1b): undefined reference to 'main'
collect2: error: ld returned 1 exit status
```

Ne demek? Bağlayıcı `main`'i bulamadı. Windows'ta `main` yoksa grafik arayüzlü programların başlangıç fonksiyonu `WinMain` aranır ve mesaj onu söyler; sorun değişmez.

Çözüm:

- `main`'in yazımını kontrol et: `Main`, `mian`, `int maın` (Türkçe `ı`!) gibi yazım hataları g++ için sıradan fonksiyon adıdır; ancak bu aşamada yakalanır.
- Doğru dosyayı mı derliyorsun? VS Code'da başka bir dosya açıkken `F5`/derle düğmesi o dosyayı derler. `main`'in bulunduğu dosyayı seçip yeniden derle.

`undefined reference to 'topla(int, int)'`

Bölüm 6'da prototip (bildirim) ile tanımı ayırmıştık. Prototip yazıp tanımı yazmazsan ya da bir yazım hatasıyla başka bir ad/parametre listesiyle yazarsan:

```cpp
#include <iostream>

using namespace std;

int topla(int a, int b);      // sadece bildirim, gövde hiçbir yerde yok

int main() {
    cout << topla(2, 3) << '\n';
    return 0;
}
```

```
/usr/bin/ld: /tmp/ccXXXXXX.o: in function `main':
hata.cpp:(.text+0x...): undefined reference to 'topla(int, int)'
collect2: error: ld returned 1 exit status
```

Ne demek? Derleyici `topla`'nın var olduğuna güvenip dosyayı geçti; bağlayıcı kodu hiçbir yerde bulamadı. C++ mesajında fonksiyon adının yanında **parametre tipleri** (`topla(int, int)`) yazılır; çünkü C++'ta aynı adlı birden çok fonksiyon olabilir (aşırı yükleme, Bölüm 6), bağlayıcı da hangisinin eksik olduğunu böyle ayırt eder.

Çözüm:

- Fonksiyonun tanımını yaz. Parametre listesi, `const` ve referans işaretleri bildirimle **birebir** aynı olmalı: `topla(int, int)` ile `topla(double, double)` bağlayıcı için iki ayrı fonksiyondur.
- Sınıfın üye fonksiyonu eksikse mesaj `undefined reference to 'Hesap::bakiyeAl() const'` biçiminde olur; `.cpp` dosyasında `Hesap::` önekini ve `const`'u unutmuş olabilirsin (Bölüm 11).
- Program birden çok `.cpp` dosyasındansa hepsini aynı komuta ver: `g++ main.cpp hesap.cpp -o program`. Eksik dosya, bağlama hatasının en sık nedenidir. VS Code'un varsayılan derleme görevi yalnız açık dosyayı derler; çok dosyalı projede komutu terminalden yaz.

`undefined reference to 'vtable for Hayvan'`

```
.../ld.exe: ...o: in function `Hayvan::Hayvan()':
hayvan.cpp:(.text+0x...): undefined reference to 'vtable for Hayvan'
collect2.exe: error: ld returned 1 exit status
```

Ne demek? `vtable`, sanal fonksiyonların adreslerini tutan gizli tablodur (Bölüm 12). Bir sınıfta `virtual` bir fonksiyonu bildirip **gövdesini yazmazsan** (`virtual void sesCikar();` ve devamında tanım yok) bağlayıcı tabloyu oluşturamaz. Hata mesajı fonksiyonu değil, sınıfı gösterir; bu yüzden ilk bakışta anlaşılmaz.

Çözüm: Sanal fonksiyonların hepsine gövde yaz. Gövde yoksa fonksiyonu saf sanal yap: `virtual void sesCikar() = 0;` (soyut sınıf). `virtual ~Hayvan();` yazıp gövdesini unutmak da aynı hatayı üretir; `virtual ~Hayvan() = default;` yaz.

`multiple definition of 'topla(int, int)'`

```
.../ld.exe: ...o: in function `topla(int, int)':
b.cpp:(.text+0x0): multiple definition of 'topla(int, int)'; a.o:a.cpp:(.text+0x0): first defined
here
collect2.exe: error: ld returned 1 exit status
```

Ne demek? Aynı fonksiyonun gövdesi birden fazla `.cpp` dosyasında bulunuyor. En sık neden: fonksiyon gövdesini bir `.h` dosyasına yazıp o başlığı iki `.cpp`'ye `#include` etmek (Bölüm 11, `.h` ve `.cpp` ayrımı).

Çözüm: Başlık dosyasına yalnızca bildirimi (prototip, sınıf tanımı) yaz; gövdeyi tek bir `.cpp` dosyasına koy. Sınıf içinde tanımlanan üye fonksiyonlar ve `inline` fonksiyonlar bu kuralın dışındadır.

`cannot open output file ... Permission denied`

```
.../ld.exe: cannot open output file merhaba.exe: Permission denied
collect2.exe: error: ld returned 1 exit status
```

Ne demek? Bağlayıcı çıktı dosyasını (`merhaba.exe`) yazamıyor, çünkü program **hâlâ çalışıyor**. Windows, çalışan `.exe` dosyasının üzerine yazılmasına izin vermez. Genellikle önceki çalıştırmanın penceresi açık kalmıştır ya da program bir `cin` bekleyerek asılıdır.

Çözüm: Programı kapat (terminalde `Ctrl+C`, açık konsol penceresini kapat), sonra yeniden derle. Görev Yöneticisi'nde (`Ctrl+Shift+Esc`) takılı kalmış `merhaba.exe`'yi sonlandırmak da işe yarar.

Çalışma zamanı hataları

Program derlendi ve bağlandı; ama çalışırken bir şey ters gitti. Bu hatalarda derleyici susar; mesajı çalışan program yazar ya da hiçbir şey yazmaz.

`terminate called after throwing an instance of 'std::out_of_range'`

Yakalanmayan istisna, Bölüm 14'ün konusudur. `vector::at()` sınır dışı bir indeksle çağrılınca `out_of_range` istisnası fırlatır; yakalayan `catch` yoksa program çöker:

```cpp
#include <iostream>
#include <vector>

using namespace std;

int main() {
    vector<int> sayilar{10, 20, 30};
    cout << "başlıyor" << endl;
    cout << sayilar.at(5) << '\n';
    return 0;
}
```

Çıktı – program çöker (Aborted)

```
başlıyor
terminate called after throwing an instance of 'std::out_of_range'
  what():  vector::_M_range_check: __n (which is 5) >= this->size() (which is 3)
```

Ne demek? İstisna fırlatıldı ama hiçbir `try-catch` onu yakalamadı; C++ çalışma zamanı `std::terminate` fonksiyonunu çağırıp programı `abort` ile sonlandırdı. Mesajda iki parça önemlidir: tırnak içindeki **istisna tipi** (`std::out_of_range`) ve `what():` ile başlayan **açıklama** (burada `vector::_M_range_check: __n (which is 5) >= this->size() (which is 3)`: "5. eleman istedin, `size()` 3"). `Aborted` ya da `Abort trap`, programın `SIGABRT` ile bittiği anlamına gelir.

Bu mesaj g++'ya özgüdür. **Visual Studio'da** program farklı bir pencereyle çöker ("Debug Error! abort() has been called"). **Windows'ta MinGW** ile aynı `terminate called ...` satırlarını görür, ardından `This application has requested the Runtime to terminate it in an unusual way` yazısıyla bir pencere ya da satır alırsın. Terminalde sonuç aynı: program yarıda biter, kodunda `return 0` çalışmaz.

Çözüm: İndeksin geçerli aralıkta (`0 ... size() - 1`) olduğundan emin ol; ya da `try { ... } catch (const out_of_range &e) { ... }` ile yakala (Bölüm 14). `[]` ile erişseydin istisna değil **tanımsız davranış** olurdu: hata sessiz kalırdı (Bölüm 7).

`terminate called after throwing an instance of 'std::invalid_argument'`

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    string girdi{"abc"};
    int sayi = stoi(girdi);
    cout << sayi << '\n';
    return 0;
}
```

Çıktı – program çöker (Aborted)

```
terminate called after throwing an instance of 'std::invalid_argument'
  what():  stoi
```

Ne demek? `stoi`, metni sayıya çeviremediğinde `invalid_argument` istisnası fırlatır; `what(): stoi` hangi fonksiyonun fırlattığını söyler (Bölüm 8, 14). Çok büyük bir sayı verirsen (`"99999999999"`) tip `std::out_of_range` olur.

Çözüm: Kullanıcıdan gelen metni çevirirken `try-catch` kullan (Bölüm 14). Sayıyı doğrudan `cin >> sayi;` ile okumak ve `cin.fail()` ile denetlemek de bir yoldur (Bölüm 3).

`Segmentation fault`

```cpp
#include <iostream>

using namespace std;

int main() {
    int *p = nullptr;
    cout << "başlıyor" << endl;
    cout << *p << '\n';
    return 0;
}
```

Çıktı – program çöker (Segmentation fault)

```
başlıyor
```

Ne demek? Program, erişim hakkı olmayan bir bellek adresine dokundu; işletim sistemi onu sonlandırdı. En sık nedenler: boş işaretçiyi (`nullptr`) dolaylama ile okumak (Bölüm 9), silinmiş (`delete`) belleğe erişmek (Bölüm 10) ve yığın taşması (aşağıya bak). Linux'ta terminal `Segmentation fault (core dumped)` yazar; macOS'ta `Segmentation fault: 11` ya da `Bus error`; Windows'ta çoğu zaman **hiçbir mesaj yazılmadan** program kapanır, çıkış kodu `-1073741819` (`0xC0000005`, *access violation*) olur. Mesajın yokluğu hatanın olmadığı anlamına gelmez.

Tanımsız davranışın çıktısını önceden bilemeyiz. Aşağıdakilerin hepsi çökebilir de çökmeyebilir de; çökme yalnızca şanslı durumdur:

```cpp
vector<int> v{1, 2, 3};
cout << v[10];                  // sınır dışı: çöp değer, çökme ya da sessiz hata
int *p;                         // ilk değer verilmemiş işaretçi
*p = 5;                         // rastgele adrese yazmak
delete p;                       // ikinci delete: çift silme
```

Çözüm:

- İşaretçiyi kullanmadan önce `nullptr` olup olmadığına bak; tanımlarken **her zaman ilk değer ver** (`int *p{nullptr};`). Mümkünse işaretçi yerine referans, `vector`, `string` ve akıllı işaretçi kullan (Bölüm 10).
- `[]` yerine geçici olarak `at()` kullan: sınır dışı erişim sessiz bellek hatası yerine yakalanabilir bir istisna olur.
- Linux ve macOS'ta programı `-fsanitize=address -g` ile derle: AddressSanitizer bu hataları **olduğu satırda**, adıyla raporlar (MinGW'de çalışmaz). `-g` bayrağı, raporun satır numarası içermesini sağlar.

`free(): double free detected in tcache 2`

```cpp
int *p = new int{5};
delete p;
delete p;                       // aynı belleği ikinci kez silmek
```

Ne demek? Aynı belleği iki kez serbest bıraktın (Bölüm 10). Linux'ta (glibc) program `free(): double free detected in tcache 2` yazarak `Aborted` ile biter; macOS'ta `pointer being freed was not allocated`, Windows'ta çoğu zaman sessiz çökme ya da hiçbir şey görürsün. Yani mesaj **sistemden sisteme değişir**; bu kod tanımsız davranıştır.

Çözüm: `delete`'ten sonra işaretçiyi `nullptr` yap (`nullptr`'ı `delete` etmek zararsızdır) ve `new`/`delete` sayısını eşleştir. Asıl çözüm, işi `unique_ptr`'ye bırakmaktır (RAII, Bölüm 10); orada çift silme mümkün değildir. `new[]` ile ayrılan belleği `delete[]` ile (köşeli parantezle) sil; `delete` yazmak da tanımsız davranıştır.

Programın yanıt vermemesi: sonsuz döngü ve `cin`

Hata mesajı olmadan programın **donması** da bir hata türüdür:

- **Sonsuz döngü.** `i++` unutulmuştur, koşul hiç yanlış olmaz ya da `while (x < 10);` sonundaki `;` döngüyü boşa çevirmiştir (Bölüm 5). Terminalde `Ctrl+C` ile durdur.
- **Program girdi bekliyor.** Ekranda bir şey yoksa `cin` sana soru sormadan bekliyor olabilir (Bölüm 3). Bir şey yaz ve `Enter`'a bas, ya da `Ctrl+C` ile çık.
- **`cin` kilitlendi.** Sayı beklenen yere harf yazılırsa akış hata durumuna girer ve sonraki `cin >> ...` komutları hiçbir şey okumaz. Döngü içindeyse program sonsuz döner (Bölüm 3, akışın kilitlenmesi).
- **Yığın taşması (stack overflow).** Temel durumu olmayan bir özyineleme çağrı yığınını doldurur ve program `Segmentation fault` ile (Windows'ta `0xC00000FD` koduyla) çöker (Bölüm 17).

Türkçe karakterler bozuk çıkıyor

```
Merhaba D╠╝nya!        ← Windows'ta  "Merhaba Dünya!"  yerine
```

Ne demek? Bu bir hata mesajı değil, **gösterim sorunudur**. Programın ürettiği baytlar doğrudur; Windows konsolu onları UTF-8 olarak yorumlamıyor ve her Türkçe harfin iki baytını iki ayrı garip karakter olarak çiziyor (`ü` yerine `╠╝` ya da `Ã¼` gibi).

Çözüm: Programı çalıştırmadan önce aynı terminalde `chcp 65001` yaz ve kaynak dosyanın UTF-8 olarak kaydedildiğinden emin ol. Ayrıntılı anlatım, kod sayfası, VS Code'da kodlama ayarı ve alternatif çözüm Bölüm 3'tedir.

Hata ayıklama ipuçları

- **İlk hatadan başla.** Sonraki mesajlar çoğunlukla zincirleme sonuçtur. Düzelt, yeniden derle, yeni ilk hataya bak.
- **Bir üst satıra da bak.** Eksik `;`, kapanmayan `(`, `{` ve `"` hatayı sonraki satırda ya da dosya sonunda ortaya çıkarır.
- **Her zaman `-Wall -Wextra` ile derle:** `g++ -std=c++17 -Wall -Wextra -g program.cpp -o program`. Uyarısız derlenmeyen programı teslim etme. `-g`, hata ayıklayıcının satır numarasını göstermesini sağlar.
- **Uzun şablon mesajında `error:` ve `required from here` satırlarını oku.** Hatanın yerini ve nedenini bu iki satır verir; `candidate` listesini geç.
- **`cout` ile izle.** Şüpheli yerlere değişkenin değerini yazdır: `cout << "i=" << i << " toplam=" << toplam << endl;`. Program çökerse `'\n'` ile yazılan satırlar tamponda kalıp ekrana ulaşmayabilir; `endl` (ya da `cerr`, Bölüm 3) satırı hemen yazar. Bunun için yukarıdaki çökme örneklerinde `endl` kullandık.
- **VS Code hata ayıklayıcısı.** Satırın soluna tıklayıp kırmızı nokta (*breakpoint*) koy, `F5` ile başlat. `F10` bir sonraki satıra geçer, `F11` çağrılan fonksiyonun içine girer; sol paneldeki *Variables* bölümünde değişkenlerin değerlerini, *Call Stack* bölümünde programın hangi fonksiyonlardan geçerek bu noktaya geldiğini görürsün (Bölüm 6, çağrı yığını).
- **Mesajı ara.** Hata mesajını, kendi değişken adlarını çıkararak arama motoruna yapıştır (ör. `cannot bind non-const lvalue reference of type`). Aynı hatayı yüzlerce kişi yaşamıştır.

EK D

ASCII Tablosu

C++'ta `char` aslında küçük bir tamsayıdır (Bölüm 2): `'A'` yazdığında bellekte 65 sayısı durur. Hangi sayının hangi karaktere karşılık geldiğini **ASCII** (*American Standard Code for Information Interchange*) tablosu belirler. Tablo 0–127 arasındaki 128 kodu kapsar; 0–31 ve 127 ekranda görünmeyen **kontrol karakterleridir**. Türkçe harfler (`ç`, `ğ`, `ı`, `ö`, `ş`, `ü`) ASCII'de yoktur; UTF-8'de 2 baytla yazılırlar (Bölüm 3 ve 8).

Akılda tutulacak kodlar

Karakter	Kod	Neden önemli?
`'\0'`	0	String sonlandırıcı (Bölüm 8)
`'\n'`	10	Yeni satır; `cin >>` sonrası tamponda kalıp `getline`'ı etkileyen Enter (Bölüm 3)
`' '`	32	Boşluk; büyük-küçük harf farkı da 32'dir
`'0'` – `'9'`	48 – 57	Rakam karakteri → sayı: `c - '0'`
`'A'` – `'Z'`	65 – 90	Büyük harfler
`'a'` – `'z'`	97 – 122	Küçük harfler: `'a' - 'A' == 32`

`char` olarak yazdırınca karakter, `int`'e çevirince kodu görünür:

asciiOrnek.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string harfler = "Aa0";

    for (char c : harfler) {
        cout << "'" << c << "' = " << static_cast<int>(c) << '\n';
    }
    cout << "'A' + 2 = '" << static_cast<char>('A' + 2) << "'\n";
    cout << "'7' - '0' = " << '7' - '0' << '\n';
    cout << "'b' - 32 = '" << static_cast<char>('b' - 32) << "'\n";
    return 0;
}
```

Çıktı

```
'A' = 65
'a' = 97
'0' = 48
'A' + 2 = 'C'
'7' - '0' = 7
'b' - 32 = 'B'
```

Kodları ezberlemen gerekmez: `'0'`, `'A'` ve `'a'`'nın 48, 65 ve 97 olduğunu bil, gerisini aradaki farktan bulursun.

Tablo (0–127)

Dec	Hex	Kr.	Dec	Hex	Kr.	Dec	Hex	Kr.	Dec	Hex	Kr.
0	00	NUL `'\0'`	32	20	boşluk	64	40	`@`	96	60	`` ` ``
1	01	SOH	33	21	`!`	65	41	`A`	97	61	`a`
2	02	STX	34	22	`"`	66	42	`B`	98	62	`b`
3	03	ETX	35	23	`#`	67	43	`C`	99	63	`c`
4	04	EOT	36	24	`$`	68	44	`D`	100	64	`d`
5	05	ENQ	37	25	`%`	69	45	`E`	101	65	`e`
6	06	ACK	38	26	`&`	70	46	`F`	102	66	`f`
7	07	BEL `'\a'`	39	27	`'`	71	47	`G`	103	67	`g`
8	08	BS `'\b'`	40	28	`(`	72	48	`H`	104	68	`h`
9	09	TAB `'\t'`	41	29	`)`	73	49	`I`	105	69	`i`
10	0A	LF `'\n'`	42	2A	`*`	74	4A	`J`	106	6A	`j`
11	0B	VT	43	2B	`+`	75	4B	`K`	107	6B	`k`
12	0C	FF	44	2C	`,`	76	4C	`L`	108	6C	`l`
13	0D	CR `'\r'`	45	2D	`-`	77	4D	`M`	109	6D	`m`
14	0E	SO	46	2E	`.`	78	4E	`N`	110	6E	`n`
15	0F	SI	47	2F	`/`	79	4F	`O`	111	6F	`o`
16	10	DLE	48	30	`0`	80	50	`P`	112	70	`p`
17	11	DC1	49	31	`1`	81	51	`Q`	113	71	`q`
18	12	DC2	50	32	`2`	82	52	`R`	114	72	`r`
19	13	DC3	51	33	`3`	83	53	`S`	115	73	`s`
20	14	DC4	52	34	`4`	84	54	`T`	116	74	`t`
21	15	NAK	53	35	`5`	85	55	`U`	117	75	`u`
22	16	SYN	54	36	`6`	86	56	`V`	118	76	`v`
23	17	ETB	55	37	`7`	87	57	`W`	119	77	`w`
24	18	CAN	56	38	`8`	88	58	`X`	120	78	`x`
25	19	EM	57	39	`9`	89	59	`Y`	121	79	`y`
26	1A	SUB	58	3A	`:`	90	5A	`Z`	122	7A	`z`
27	1B	ESC	59	3B	`;`	91	5B	`[`	123	7B	`{`
28	1C	FS	60	3C	`<`	92	5C	`\`	124	7C	`|`
29	1D	GS	61	3D	`=`	93	5D	`]`	125	7D	`}`
30	1E	RS	62	3E	`>`	94	5E	`^`	126	7E	`~`
31	1F	US	63	3F	`?`	95	5F	`_`	127	7F	DEL